КОНСПЕКТ ПО КУРСУ

# Тестирование ПО с нуля. Теория + практика. Продвинутый курс с ИИ

Дата обновления конспекта: 18 июля 2026 г.

Артём Русов · Сайт

# Об авторе

Артём Русов — тестировщик, блогер, преподаватель

Сайт

Авторский канал

Бесплатные ресурсы

© 2026 Артём Русов. Конспект собран автоматически из материалов курса на Stepik и предназначен для личного некоммерческого использования. Права на тексты, изображения и схемы принадлежат автору. Копирование, публикация и коммерческое использование без письменного разрешения автора не допускаются.

# Оглавление

ТЕОРИЯ ТЕСТИРОВАНИЯ (СИНХРОНИЗИРОВАНА С ISTQB)

1. Вступительное слово перед модулем "Теория тестирования"
2. Что такое тестирование, QC и QA?
3. Верификация и валидация
4. Принципы тестирования
5. SDLC и STLC. Модели разработки ПО
6. Уровни тестирования
7. Типы тестирования. Часть 1
8. Типы тестирования. Часть 2
9. Scrum
10. ДОП: Kanban и XP
11. Работа с требованиями

ТЕСТ-ДИЗАЙН

12. Вступительное слово перед модулем "Тест-дизайн"
13. Эквивалентное разбиение и анализ граничных значений
14. Попарное тестирование
15. Диаграмма переходов и состояний, таблица принятия решений

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

16. Вступительное слово перед модулем "Тестовая документация"
17. Тест-план и тест-стратегия
18. Отчет по результатам тестирования
19. Чек-листы
20. Тест-кейсы
21. Работа в QASE (версия 2026)
22. Работа в Test IT (версия 2026)
23. Отчет о дефекте. Виды улучшений. Жизненный цикл дефекта
24. Работа в Jira (версия 2026)

25. Работа в Youtrack (версия 2026)
26. Матрица трассировки и метрики в тестировании
27. Оценка трудозатрат в тестировании. Виды эстимации

ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЙ

28. Вступительное слово перед модулем "Тестирование веб-приложений"
29. Как работает Internet? IP, MAC, URL, DNS
30. Клиент-серверная архитектура. Балансировщики
31. HTTP-протокол. HTTP-методы. Запросы и ответы. Коды ответов
32. Кэш, куки, хранилища. Веб-сайт, приложения, сервис
33. Основы HTML, CSS и JS (версия 2026)
34. Chrome DevTools (версия 2026)
35. ПРАКТИКА 10: Работа в DevTools
36. Виды UI и верстки (версия 2026)
37. Тестирование веб-элементов и веб-форм (версия 2026)
38. ДОП: Обзор клиентской части приложения на курсе

ТЕСТИРОВАНИЕ API

39. Вступительное слово перед модулем "Тестирование API"
40. Виды API. SOAP и REST
41. Установка и обзор функций Postman (версия 2026)
42. API-документация. Swagger и типы данных (версия 2026)
43. cURL в Postman и Swagger (версия 2026)
44. Идентификация, аутентификация и авторизация (версия 2026)
45. HTTP-методы в Postman. Цепочка запросов (версия 2026)
46. ДОП: Обзор серверной части приложения на курсе
47. Простые автотесты. Тест-кейсы и чек-листы (версия 2026)
48. Работа с тестовыми данными в Postman (версия 2026)
49. Перехват трафика в Postman (версия 2026)
50. Тестирование SOAP-сервисов в Postman (версия 2026)

РАБОТА С ДАННЫМИ

51. Вступительное слово перед модулем "Работа с данными"
52. Базы данных. Типы БД и СУБД. Нормальные формы
53. Работа с DBeaver и MySQL server. Создание таблиц (версия 2026)
54. SELECT запросы в SQL. Агрегатные функции
55. JOIN запросы в SQL
56. NoSQL база данных MongoDB (версия 2026)

ТЕСТИРОВАНИЕ МОБИЛЬНЫХ ПРИЛОЖЕНИЙ

57. Вступительное слово перед модулем "Тестирование мобильных Apps"
58. Обзор Android и iOS
59. Тестовые фермы
60. Типы мобильных приложений
61. Особенности тестирования мобильных приложений
62. Типы тестирования мобильных приложений
63. Android Studio. Функции и логи
64. ДОП: Xcode. Функции и логи
65. Charles Proxy. Установка и использование
66. ДОП: Fiddler Everywhere. Установка и использование
67. ДОП: Proxyman. Установка и использование

РАБОТА С GIT И BASH

68. Работа с bash
69. Работа с git

ДОП: CI/CD, CHATGPT И ДРУГИЕ ИНСТРУМЕНТЫ

70. Тестовые окружения
71. Что такое CI/CD?
72. ChatGPT в тестировании
73. Основы работы с Jenkins
74. Основы работы с GitHub Actions

ДОП: СОЗДАНИЕ РЕЗЮМЕ

**75.** Создание резюме

**76.** Создание сопроводительного письма

**77.** Создание профиля в LinkedIn

# 1. Вступительное слово перед модулем "Теория тестирования"

**Вступительное слово перед стартом модуля "Теория тестирования"**

Бытует ошибочное мнение, что тестировщик – это простая точка входа в IT. На протяжении 60+ следующих уроков я буду опровергать этот миф.

Тестирование – это полноценная профессия, которая требует от будущего специалиста как глубоких технических навыков, так и набора личностных характеристик и предрасположенностей, так называемых софт-скиллов. В первой лекции мы поговорим про это подробнее.

Если вы чувствуете, что вам не хватает базовой компьютерной грамотности, то дополнительно перед курсом можно пройти обучение здесь и здесь (на английском). Оба курса позволяют освоить всю необходимую предварительную базу за короткий срок.

Данный модуль включает в себя необходимые фундаментальные знания, которые позволят вам в будущем лучше разбираться в особенностях профессии не только с точки зрения использования целевых инструментов, но и со стороны классических подходов в тестировании.

Именно теоретические вопросы занимают большую часть на собеседованиях на начальные позиции в любой сфере, тестирование не исключение. Теория будет сопровождать вас во всех модулях курса, без теории невозможна практика. Теория это не вода, а скорее фундамент для вашего обучения.

Хорошего тестировщика отличает также умение работать с большим количеством текстовой информации и технической документации, готовьте себя и к этому.

Напоследок, рекомендую параллельно подключать к обучению и книги. Главное помните, что чтение книг не заменяет ваши знания и не должно занимать больше 20% от общего времени на обучение.

Здесь и далее на курсе в текстовой части вы будете видеть отдельные статьи, материалы или их части из следующих пособий и книг:

https://svyatoslav.biz/software_testing_book/

https://www.rstqb.org/ru/istqb-downloads.html (Программа подготовки ISTQB Базового уровня)

https://vladislaveremeev.gitbook.io/qa_bible/

**В конце модуля вы будете:**

- Без проблем ориентироваться в терминологии тестирования
- Понимать роль тестировщика на проекте
- Отличать методы, типы и уровни тестирования
- Понимать основные этапы разработки и тестирования
- Работать по Scrum
- Анализировать требования и находить недочеты для экономии денег бизнеса

ТЕОРИЯ ТЕСТИРОВАНИЯ (СИНХРОНИЗИРОВАНА С ISTQB)

# 2. Что такое тестирование, QC и QA?

**Тестирование** - процесс в рамках жизненного цикла разработки программного обеспечения, который оценивает качество компонента или системы, а также связанных с ними рабочих продуктов. [ISTQB Glossary]

**Тестирование ПО** — проверка соответствия между реальным и ожидаемым поведением программы [Что-то из интернета]

**Контроль качества (QC)** - набор действий, предназначенных для оценивания качества компонента или системы. [ISTQB Glossary]

**Обеспечение качества (QA)** - активности, направленные на обеспечение уверенности в том, что требования к качеству будут выполнены [ISTQB Glossary]

**Управление качеством (quality management)** - скоординированные действия по руководству и контролю организации в отношении качества, которые включают в себя установление политики качества и целей качества, планирование качества, контроль качества, обеспечение качества и повышение качества. [ISTQB Glossary]

**Отладка (debugging)** – процесс поиска, анализа, и устранения причин отказов в компоненте или системе. [ISTQB Glossary]

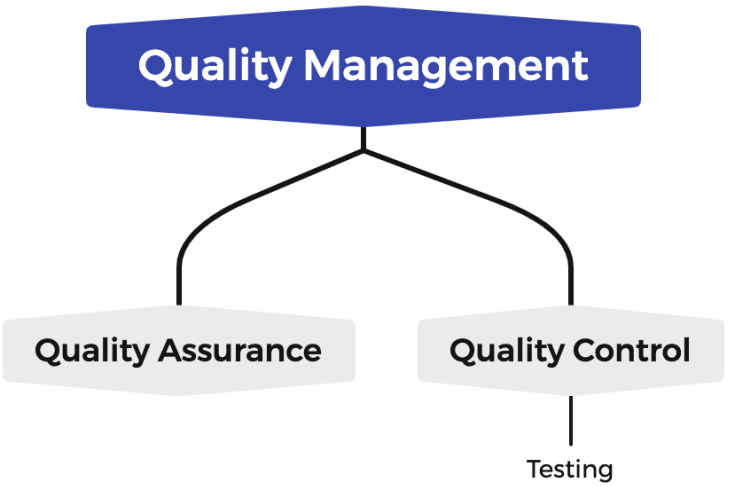
Давайте разберемся на примере создания мобильного приложения, потому что определения не всегда отражают суть:

1. В рамках **тестирования** мы выполним проверки и задокументируем дефекты, убедимся, что продукт соответствует требованиям.
2. В рамках **контроля качества** мы проанализируем полученные данные и убедимся, что соблюдены все требования, предъявляемые к качеству как продукта, так и самого процесса. Мы должны убедиться, что уровень качества нашего продукта высокий и он готов к релизу.
3. В рамках **обеспечения качества** мы формируем процесс QA для соответствия стандартам качества на всех этапах SDLC, еще до этапа создания нашего продукта, который будет минимизировать количество дефектов и предупреждать их.

**Основные отличия процессов:**

1. Контроль качества и тестирование (как его часть) направлены на продукт, а обеспечение качества на процесс.
2. Тестирование и контроль качества являются контролирующими мерами, а обеспечение качества - превентивными, или предупреждающими.

Связь процессов между собой согласно ISTQB CTFL Syllabus 2018*




**В Syllabus 2023 данная концепция не рассматривается, но это не значит, что она перестала работать.*

**Цели тестирования [ISTQB CTFL Syllabus 2023]:**

1. Оценка рабочих продуктов, таких как требования, пользовательские истории, проекты и код.
2. Провоцирование отказов и обнаружение дефектов.
3. Обеспечение необходимого покрытия объекта тестирования.
4. Снижение уровня риска ненадлежащего качества программного обеспечения.
5. Проверка выполнения зафиксированных требований.
6. Проверка того, что объект тестирования соответствует контрактным, юридическим и нормативным требованиям.
7. Предоставление информации заинтересованным сторонам для принятия обоснованных решений.
8. Создание уверенности в качестве объекта тестирования.

9. Проверка того, завершен ли объект тестирования и работает ли он так, как ожидают заинтересованные стороны.

**Зачем это знать тестировщику?**

Начинающие специалисты часто думают, что единственная цель тестирования – это "найти как можно больше багов". Если вы посмотрите на список выше, поиск дефектов – это лишь один пункт из девяти!

Как мы помним из прошлого шага, **Обеспечение качества (QA)** направлено на предотвращение ошибок, и цели тестирования отлично помогают реализовать этот превентивный подход.

**Разберем на примере несколько целей:** Представьте, что бизнес решил добавить новую систему оплаты в корзину в интернет-магазине.

- **Оценка рабочих продуктов (до написания кода):** Ваша работа начинается еще до того, как программист напишет первую строчку. Вы читаете требования к новой оплате и замечаете, что аналитик забыл описать, что будет, если на карте клиента не хватит денег. Вы нашли ошибку в документации и предотвратили баг до его появления в коде!
- **Предоставление информации для решений:** Вы протестировали готовую функцию оплаты и нашли пару неочевидных багов. Тестировщик не принимает решение "отменять релиз или нет". Ваша цель: дать менеджеру прозрачную картину: *"Я нашел баг в оплате. Риск потери денег минимален, но в 1% случаев чек не приходит на почту"*. Теперь менеджер может принять обоснованное решение: чинить сейчас или выпустить как есть и починить завтра.
- **Снижение рисков и создание уверенности:** Когда вы проверили все основные сценарии покупки (обеспечили покрытие), вы снижаете риск того, что магазин потеряет прибыль из-за неработающей кассы. Вы даете команде **уверенность**, что продукт можно смело показывать реальным пользователям.
- **Проверка юридических требований:** Так как мы работаем с банковскими картами, вы проверяете, что приложение не сохраняет CVV-коды пользователей в открытом виде, тем самым защищая компанию от огромных штрафов.

**Итог:** Знание этих целей поможет вам правильно расставлять приоритеты в работе. Вы будете понимать, что идеального ПО без багов не существует, но ваша главная задача: защитить бизнес от рисков, проверить соответствие продукта ожиданиям пользователей и дать команде честную информацию о состоянии системы.

## Реальная жизнь и советы

1. Зачастую нанимающая компания не понимает разницу между тестированием, QC и QA, поэтому работа тестировщика может включать в себя все эти активности. Важно понимать, что делают в каждом из описанных процессов, так как незнание общей специфики компанией может привести к навешиванию дополнительных функций PM, HR и аналитика, от которых вам нужно отказываться
2. В интернете вы можете найти следующую трактовку: тестирование является частью QC, а QC частью QA. На самом деле это не совсем так, однако будьте готовы к тому, что на интервью могут возникнуть споры. Будьте готовы апеллировать ссылкой на ISTQB 2018.
3. Чаще всего начинающие специалисты выполняют только тестирование и контроль качества. Обеспечение качества, как одна из зон ответственности, появляется на более поздних этапах вашей карьеры.

## Полезные материалы

1. Презентация "QA, QC, Testing" https://drive.google.com/file/d/1j9GpoIbwMAusjtbqWnW_VQGacNWp2nMW/view?usp=share_link
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/qa-qc
3. Библия QA. Статья и дополнительные материалы - https://vladislaveremeev.gitbook.io/qa_bible/obshee/qa-qc-testing

# 3. Верификация и валидация

**Верификация** - доказанное объективными результатами исследования подтверждение того, что определенные требования были выполнены. [ISTQB Glossary]

**Валидация** - подтверждение путем экспертизы того, что рабочий продукт соответствует потребностям заинтересованной стороны. [ISTQB Glossary]

**Верификация** показывает то, что продукт разработан согласно требованиям к нему и соответствует им (нуждам заказчика). Отвечает на вопрос: правильно ли мы разрабатываем продукт?

Чаще всего под верификацией подразумевают статические методы проверки (без запуска кода), которые включают в себя анализ требований, дизайна, кода.

**Валидация** показывает то, что продукт разработан согласно ожиданиям конечного пользователя (нуждам пользователя). Отвечает на вопрос: правильный ли продукт мы разработали?

Валидация включает в себя динамические проверки (с запуском кода) и тестирование уже рабочего продукта.

**Как это работает на практике?**

**Разберем на примере нашего интернет-магазина:** Заказчик хочет добавить функцию "Купить в один клик".

1. **Верификация (Делаем ли мы продукт правильно?):** Вы берете техническое задание (ТЗ) и проверяете: описано ли там, куда придет уведомление о заказе? Указаны ли требования к безопасности платежа? Соответствует ли макет дизайна фирменному стилю? *Если код написан строго по ТЗ, то верификация пройдена.*
2. **Валидация (Делаем ли мы ПРАВИЛЬНЫЙ продукт?):** Вы запускаете приложение и пытаетесь совершить покупку в один клик в реальных условиях (динамическое тестирование). И тут выясняется: кнопка

настолько огромная, что перекрывает описание товара, а форма подтверждения заказа требует ввести 10 полей (что противоречит самой идее "одного клика"). *ТЗ выполнено, но пользователь недоволен, то валидация провалена.*

**Почему это критично в работе:**

- **Экономия времени:** Верификация (проверка требований и кода) позволяет находить баги "на бумаге". Исправить строчку в ТЗ – это 5 минут. Исправить уже написанную и протестированную функцию – это дни работы команды.
- **Снятие ответственности:** Если вы провели только верификацию ("сделали по ТЗ"), но продукт никому не нужен, то вы формально правы, но проект провален. Валидация позволяет вовремя сказать: «Ребята, мы делаем то, что написано в документах, но это не будет работать для пользователя».

**Итог:** Хороший тестировщик сначала верифицирует (проверяет планы и чертежи), а затем валидирует (проверяет, как готовый продукт решает проблему клиента).

**Реальная жизнь и советы**

1. На проектах вы редко будете сталкиваться с разделением этих понятий, однако к интервью нужно понимать четкую разницу между этими процессами и приводить примеры.
2. Часто можно встретить следующую трактовку: верификация это всегда статическая проверка, а валидация - динамическая. На самом деле оба процесса могут выполняться как с запуском кода, так и без него.
3. Справедливости ради, существует большая группа статических методов анализа, которые относятся конкретно к верификации. Подробнее о них можно почитать в ISTQB Syllabus (но не стоит забивать ими голову, на практике они редко встречаются).

**Полезные материалы**

1. Презентация "Верификация и валидация" https://drive.google.com/file/d/18yB08fp8XS_aQUgtnqVTxdBkohermbmV/view
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/qa-qc

3. Библия QA. Статья и дополнительные материалы - https://vladislaveremeev.gitbook.io/qa_bible/obshee/verifikaciya-i-validaciya-verification-and-validation

# 4. Принципы тестирования

### 1. Тестирование демонстрирует наличие дефектов, а не их отсутствие

Тестирование снижает вероятность того, что дефекты в объекте тестирования останутся необнаруженными, но даже если дефекты не были обнаружены, тестирование не доказывает корректности объекта тестирования.[ISTQB CTFL 2023]

**Примечание автора курса**

Например, вы тестируете магазин приложений. В рамках тестирования вы нашли 10 дефектов и уверены, что в приложении их больше нет. Приложение успешно отправляется в релиз к конечному пользователю. Конечный пользователь получает сообщение об ошибке и не может зайти в приложение, потому что версия его браузера не поддерживается. [ISTQB CTFL 2023]

### 2. Исчерпывающее тестирование недостижимо

Полное тестирование с использованием всех комбинаций вводов и предусловий физически невыполнимо, за исключением тривиальных случаев. [ISTQB CTFL 2023]

**Примечание автора курса**

В вашем приложении используется 15 различных фильтров с возможностью выбора 20 различных параметров в каждом фильтре.

В итоге вам нужно провести 15*20 = 300 тестов только для фильтрации, а есть еще и другие функции приложений, которые нужно проверить.

В таком случае тестировщик оптимизирует и приоритизирует свою работу, сокращая объем проверок, например, используя техники тест-дизайна, о которых мы поговорим дальше.

Провести исчерпывающее тестирование невозможно.

### 3. Раннее тестирование сохраняет время и деньги

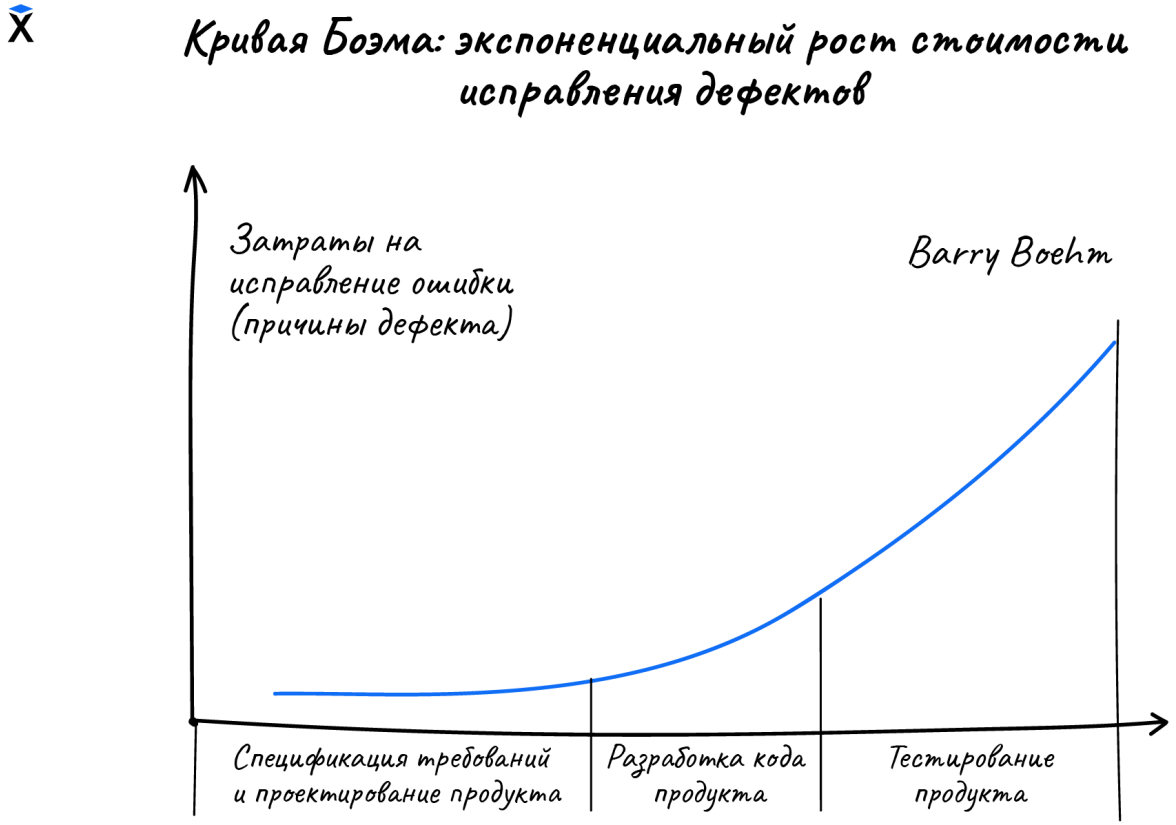
Дефекты, устраненные на ранней стадии процесса, не вызовут последующих дефектов в производных рабочих продуктах. Стоимость качества будет

снижена, так как позже в жизненном цикле ПО будет происходить меньше отказов. [ISTQB CTFL 2023]

**Примечание автора курса**

Существует так называемая кривая Боэма, в которой отражена стоимость исправления дефекта по отношению к фазе разработки конечного продукта. Чем раньше начинается тестирование, тем дешевле исправить дефект. Это и есть графическое отражение принципа раннего тестирования.

Такой подход называется Shift Left Testing, где тестирование смещается влево относительно всех этапов SDLC и начинается как можно раньше. В идеальном случае, это этапы идеи и анализа требований.




Источник картинки

### 4. Кластеризация дефектов

Обычно небольшое количество модулей содержит большинство дефектов, обнаруженных во время тестирования перед выпуском, или отвечает за большинство эксплуатационных отказов. Это явление является иллюстрацией принципа Парето. [ISTQB CTFL 2023]

**Примечание автора курса**

В таком случае обычно руководствуются частным случаем закона Парето: в 20% модулей приложения сокрыто 80% всех дефектов.

Например, при тестировании модулей регистрации, авторизации и создания профиля пользователя большая часть дефектов сконцентрирована в модуле авторизации.

При планировании дальнейшего тестирования мы будем обращать внимание на этот модуль, например, включать его в регрессию (тестирование влияния изменений на другие функции приложения) и учитывать в анализе рисков.

Другое название этого принципа: скопление дефектов.

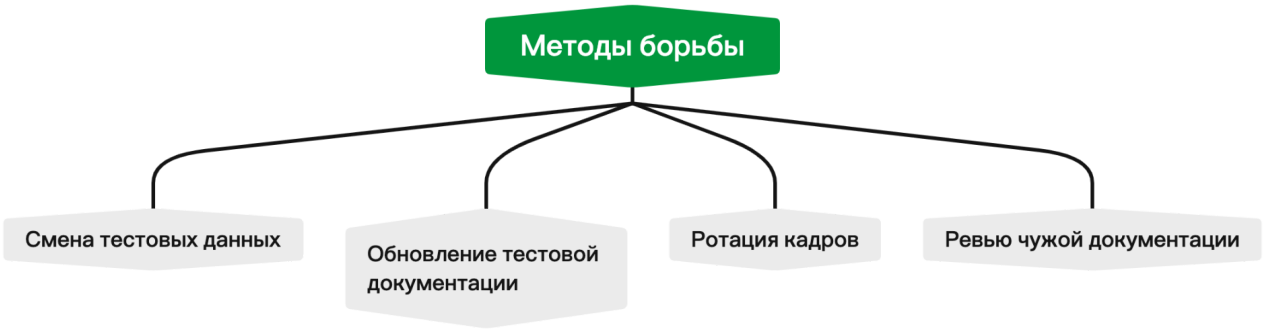
## 5. Тесты устаревают

Если одни и те же тесты повторяются много раз, они становятся все более неэффективными в обнаружении новых дефектов. [ISTQB CTFL 2023]

**Примечание автора курса**

Другое название этого принципа: парадокс пестицида.

Для борьбы с эффектом пестицида можно использовать следующие методы:




## 6. Тестирование зависит от контекста

Не существует единого универсального подхода к тестированию. Тестирование выполняется по-разному в зависимости от контекста. [ISTQB CTFL 2023]

**Примечание автора курса**

У тестировщиков есть арсенал из различных инструментов, техник и знаний в области различных типов тестирования, которые они используют в зависимости от природы самого приложения.

### 7. Заблуждение об отсутствии ошибок

Тщательное тестирование всех указанных требований и исправление всех обнаруженных дефектов может привести к созданию системы, которая не будет соответствовать потребностям и ожиданиям пользователей, не будет помогать в достижении бизнес-целей заказчика и будет уступать другим конкурирующим системам. [ISTQB CTFL 2023]

**Примечание автора курса**

Перефразирую на человеческий язык. Компания разработала интернет-магазин для страны, в которой часть населения не может использовать онлайн-оплату. В итоге, конечные пользователи не смогли приобрести желаемый товар, перестали пользоваться приложением и занизили его рейтинг.

В данном случае при сборе требований компания не учла все кейсы, разработчики создали программу, которая соответствует предоставленным им требованиям, однако не соответствует нуждам пользователя.

### Реальная жизнь и советы

1. Обычно принципы тестирования в работе применяются на подсознательном уровне, но к интервью их стоит выучить. Особое внимание уделите парадоксу пестицида и методам борьбы с ним
2. Всегда помните о контексте тестирования и создавайте тестовый план, согласно ему
3. В приложениях всегда есть дефекты, поэтому не принимайте слишком близко на свой счет тот факт, что они будут появляться в проде (а потом еще и не исправляться :)
4. Всегда учитывайте раннее тестирование и начинайте все процессы с анализа требований
5. Помните, что протестировать все невозможно (даже если очень сильно просят). Всегда используйте приоритизацию
6. Учитывайте скопление дефектов при формировании регрессионного набора

### Полезные материалы

1. Презентация https://drive.google.com/file/d/1JETdtmKOb4X745yHq51kARdnSFpP-Uu6/view
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/principles
3. Старая версия https://youtu.be/vVr2FqiTC_g
4. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1y8t19Y8kL_LshkRidR5cZlEHcBzvwsqT
5. Тестирование программного обеспечения. Базовый курс, стр.29 - https://svyatoslav.biz/software_testing_book/
6. Библия QA. Статья и дополнительные материалы - https://vladislaveremeev.gitbook.io/qa_bible/obshee/principy-testirovaniya

# 5. SDLC и STLC. Модели разработки ПО

## Жизненный цикл разработки ПО (SDLC - Software Development Lifecycle)

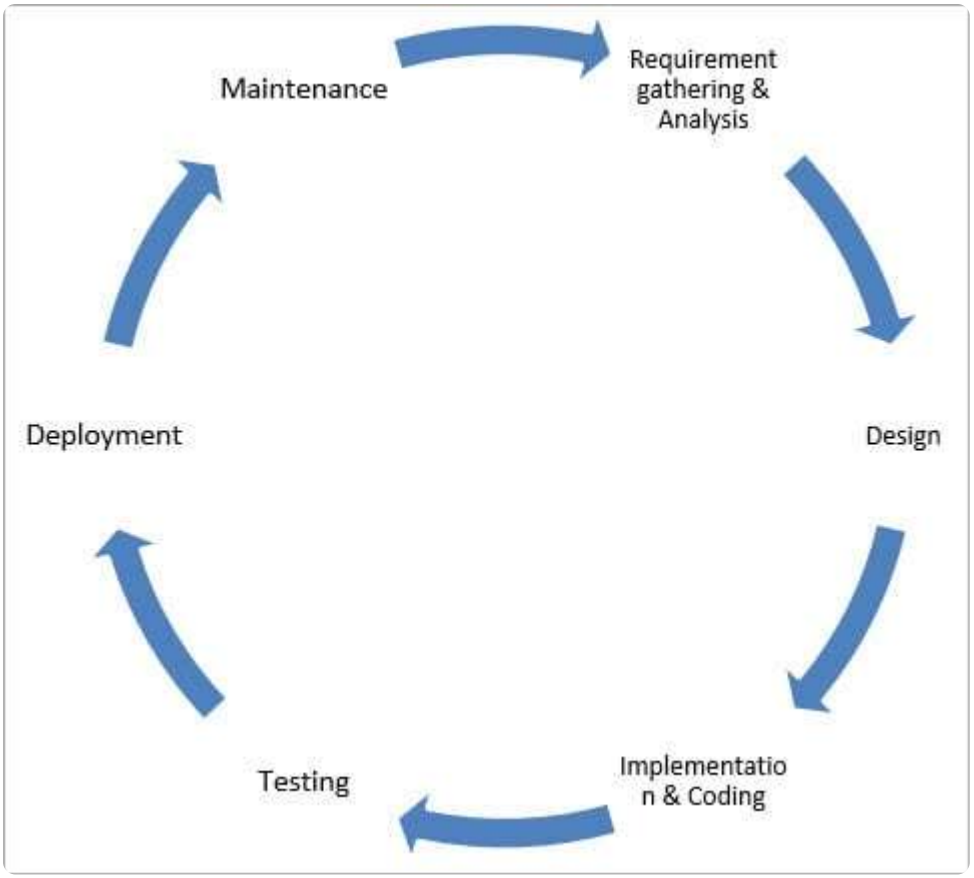
*Жизненный цикл программного обеспечения (software lifecycle): Период времени, начинающийся с момента появления концепции программного обеспечения и заканчивающийся тогда, когда дальнейшее использование программного обеспечения невозможно. Жизненный цикл программного обеспечения обычно включает в себя следующие этапы: концепт, описание требований, дизайн, реализация, тестирование, инсталляция и наладка, эксплуатация и поддержка и, иногда, этап вывода из эксплуатации. Данные фазы могут накладываться друг на друга или проводиться итерационно. (ISTQB)*

*Период времени от концепции до первоначальной версии известен как жизненный цикл разработки, который является частью жизненного цикла программного обеспечения. С момента первого запуска система переходит в эксплуатацию (функционирование). Система остается в эксплуатации до момента прекращения использования. (ГОСТ 56920)*

**SDLC** - это систематизированный процесс, этапы которого охватывают полный жизненный цикл программного обеспечения (Software Lifecycle) и который определяет различные этапы разработки программного обеспечения для создания высококачественного программного обеспечения, отвечающего ожиданиям клиентов и для улучшения эффективности разработки. Разработка системы должна быть завершена в заранее определенные сроки и стоимость. Каждая фаза жизненного цикла SDLC имеет свой собственный процесс и результаты, которые используются в следующей фазе.

Обычно он делится на шесть-восемь шагов, но менеджеры проектов могут объединять, декомпозировать или пропускать шаги, в зависимости от скоупа проекта.

В разных источниках фазы немного отличаются, но глобально суть везде одинакова.




https://www.softwaretestinghelp.com/wp-content/qa/uploads/2018/04/SDLC-Cycle.jpg

**Фазы SDLC**:

- **Сбор и анализ требований** (Requirement Gathering and Analysis): На этом этапе от клиента собирается вся необходимая информация для разработки продукта в соответствии с их ожиданиями. Любые неясности должны быть разрешены сразу на этом этапе. Бизнес-аналитик и менеджер проекта назначили встречу с заказчиком, чтобы собрать всю информацию, например, что заказчик хочет построить, кто будет конечным пользователем, какова цель продукта. Перед созданием продукта очень важно понимание или знание продукта. Например, клиент хочет иметь приложение, которое включает денежные транзакции. В этом случае требование должно быть четким, например, какие транзакции будут выполняться, как они будут проводиться, в какой валюте они будут проводиться и т. д. После того, как сбор требований завершен, проводится анализ для проверки возможности разработки продукта. После четкого понимания требования создается документ SRS (Спецификация требований к программному обеспечению). Этот документ должен быть

полностью понятен разработчикам, а также должен быть рассмотрен заказчиком для использования в будущем;

- **Дизайн** (Design): На этом этапе требования, собранные в документе SRS, используются в качестве входных данных, и создается архитектура программного обеспечения, которая используется для реализации разработки системы.
- **Разработка** (Implementation or Coding): Реализация / кодирование начинается, как только разработчик получает Design document. Дизайн программного обеспечения переведен в исходный код. На этом этапе реализуются все компоненты программного обеспечения;
- **Тестирование** (Testing): Тестирование начинается после завершения кодирования и выпуска модулей для тестирования. На этом этапе разработанное программное обеспечение тщательно тестируется, и все обнаруженные дефекты передаются разработчикам для их исправления. Повторное тестирование, регрессионное тестирование проводится до тех пор, пока программное обеспечение не будет соответствовать ожиданиям клиента. Тестировщики обращаются к документу SRS, чтобы убедиться, что программное обеспечение соответствует стандарту заказчика;
- **Развертывание** (Deployment): После тестирования продукта он развертывается в производственной среде или выполняется первое UAT (пользовательское приемочное тестирование), в зависимости от ожиданий клиента. В случае UAT создается копия производственной среды, и заказчик вместе с разработчиками выполняет тестирование. Если клиент остается доволен, то предоставляет согласие на релиз;
- **Поддержка** (Maintenance): Основное внимание на этом этапе SDLC уделяется обеспечению того, чтобы потребности продолжали удовлетворяться и чтобы система продолжала работать в соответствии со спецификацией, упомянутой в первом этапе. После того как система развернута и клиенты начинают использовать разработанную систему следует 3 вида активностей:
- Исправление ошибок;
- Обновление;
- Улучшение.

**Источник:** https://vladislaveremeev.gitbook.io/qa_bible/sdlc-i-stlc/zhiznennyi-cikl-razrabotki-po-sdlc-software-development-lifecycle

**Заметка от автора курса**

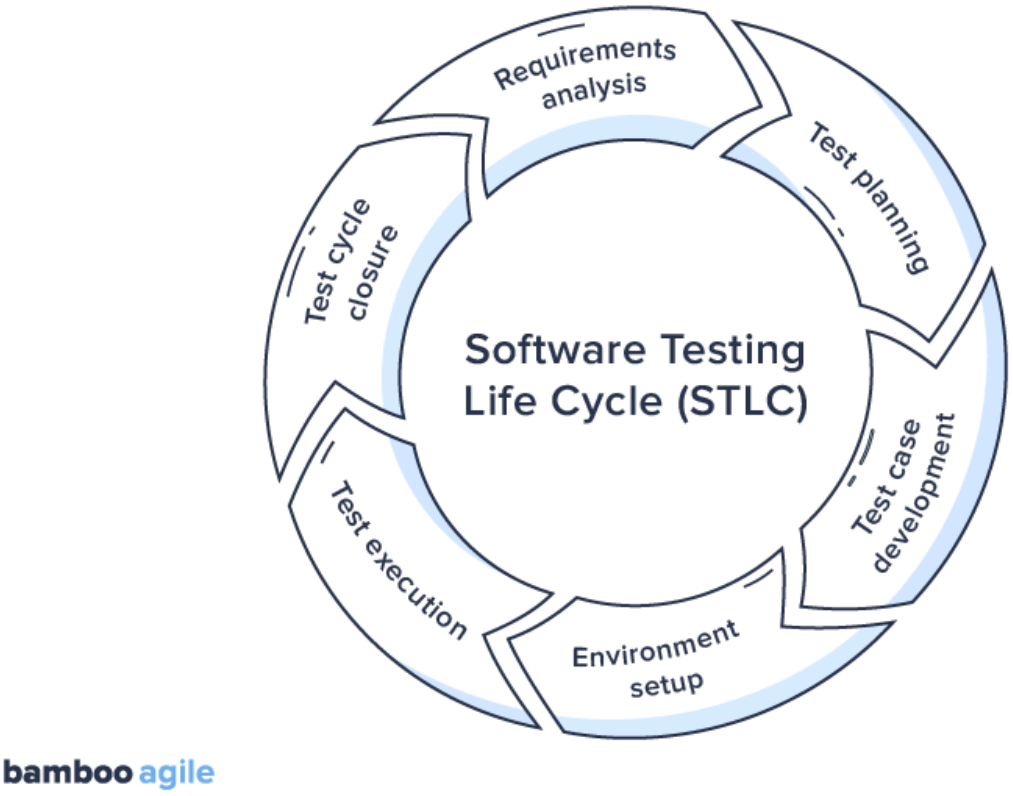
В некоторых источниках дополнительно выделяют следующие этапы:

- **Идея, или планирование системы,** который предшествует этапу "Анализ требований". Представляет собой определение проблемы, которую поможет решить разрабатываемый продукт.
- **Ввод в эксплуатацию** - синоним этапа "Развертывание".
- **Вывод из эксплуатации,** который завершает SDLC. Например, при устаревании продукта и прекращения его поддержки, необходимо сформировать процесс вывода продукта из эксплуатации, который включает в себя: сохранение информации, удаление данных, удаление аппаратного и программного обеспечения

### Жизненный цикл тестирования ПО (STLC - Software Testing Lifecycle)

**STLC** - это процесс тестирования, который включает в себя определенную последовательность шагов, чтобы гарантировать достижение целей в области качества. В процессе STLC каждое действие выполняется планомерно и систематически. Каждый этап имеет разные цели и результаты. У разных организаций разные этапы STLC, однако основа остается прежней.

### Фазы STLC




*https://bambooagile.eu/insights/wp-content/uploads/2021/01/In-article-STLC-copy.png*

STLC имеет несколько взаимосвязанных фаз и в целом очень похож на SDLC. Эти фазы являются последовательными и называются:

- **Анализ требований** (Requirement Analysis): один из важнейших этапов, потому что именно на нем можно почти бесплатно исправить недостатки проекта. Этап анализа требований также определяет потенциальную потребность в автоматизированном тестировании и позволяет производить экономические расчеты затрат на рабочую силу на основе оценки проекта. На этом же этапе обсуждаются и документируются критерии начала и окончания тестирования
- **Планирование тестирования** (Test Planning): на этом этапе формируется план тестирования, т.е. мы определяем действия и ресурсы, которые помогут достичь целей тестирования (участники и их роли, инструменты, окружение). Во время планирования мы также пытаемся определить метрики, метод сбора и отслеживания этих метрик. План составляют исходя из требований, тестовой стратегии и анализа рисков

- **Разработка тест-кейсов** (Test Case Development): подразумевает использование ручного и автоматизированного тестирования для достижения полного охвата функциональности программного обеспечения, при этом процесс основан на заранее установленных требованиях. Чаще всего тест-кейсы для автоматического тестирования пишутся отдельно, так как кейсы для ручного тестирования описаны в виде шпаргалок (cheat sheets)
- **Настройка тестовой среды** (Test Environment Setup): в плане тестирования четко указано, какую тестовую среду следует использовать. На этом этапе STLC настраиваются операционные системы и виртуальные машины, развертываются инструменты тестирования, такие как Selenium, Katalon Studio, а также тестовая среда и базы данных проекта. Мы также обращаемся с запросами к DevOps и администраторам, если требуется поддержка
- **Выполнение тестов** (Test Execution): тесты выполняются на основе готовой тестовой документации и правильно настроенной тестовой среды. Все результаты тестирования регистрируются в Системе управления тестированием. Отрицательно пройденные тесты, в которых фактический результат отличается от ожидаемого, регистрируются как ошибки и передаются команде разработчиков на доработку с последующей перепроверкой после исправления
- **Завершение цикла испытаний** (Test Cycle Closure): окончательная генерация отчетов о тестировании для клиента. Они должны включать затраченное время, процент обнаруженных ошибок и положительных результатов тестирования, общее количество обнаруженных и исправленных ошибок. Что касается отдела тестирования, то это момент для анализа его работы, подведения итогов, анализа его продуктивности и возможности внести предложения по улучшению качества тестирования

**Источник:** https://vladislaveremeev.gitbook.io/qa_bible/sdlc-i-stlc/zhiznennyi-cikl-testirovaniya-po-stlc-software-testing-lifecycle

### Разница STLC и SDLC

STLC и SDLC тесно связаны друг с другом, но они одновременно преследуют разные задачи с одной и той же целью, а именно:

- сбор требований в желаемой форме и разработка заявленной функциональности (SDLC);

- анализ требований, помощь клиенту и команде разработчиков и подтверждение качества реализованной функциональности (STLC).

Общая цель - удовлетворение клиента и получение максимально возможного балла на этапах верификации и валидации.

**Источник:** https://vladislaveremeev.gitbook.io/qa_bible/sdlc-i-stlc/zhiznennyi-cikl-testirovaniya-po-stlc-software-testing-lifecycle

**Зачем это знать тестировщику?**

Глядя на глобальный цикл разработки (SDLC), новички часто думают: *"Я включаюсь в работу, когда мне дадут готовый код"*. Это самое опасное заблуждение!

Чтобы не сидеть без дела, пока программисты пишут код, и не пытаться протестировать всё в последнюю минуту, существует **STLC – Жизненный цикл тестирования**. Главное правило, которое вы должны запомнить: **STLC работает параллельно с SDLC**. Пока вся команда двигается по своим этапам разработки, команда QA двигается по своим этапам тестирования.

**Разберем, как эти два цикла живут вместе (на примере внедрения промокодов в наш интернет-магазин):**

- **SDLC (Сбор требований) ↔ STLC (Анализ требований):** Заказчик рассказывает, как должны работать промокоды. Вы проводите **верификацию** требований. Если в ТЗ не описано, что будет при вводе просроченного промокода, вы задаете вопрос. На этапе сбора требований, также может рисоваться **дизайн макета –** пока дизайнер рисует корзину, вы оцениваете: сколько времени займет проверка и правильно ли он нарисован?
- **После или во время сбора требований ↔ STLC (Планирование тестирования):** Понадобится ли нам автоматизация, или справимся вручную? Вы формируете план тестирования.
- **SDLC (Разработка/Кодирование) ↔ STLC (Разработка тест-кейсов и Настройка среды):** Пока программисты пишут код промокодов, вы создаете документацию (тест-кейсы, чек-листы) для будущих проверок. Параллельно вы просите администраторов (или DevOps) настроить вам

тестовый сервер с копией базы данных магазина, чтобы не проверять новые фичи на реальных покупателях.

- **SDLC (Тестирование) ↔ STLC (Выполнение тестов):** Код готов и передан вам. Вот здесь начинается **Валидация**. Вы идете по своим тест-кейсам, вводите промокоды, провоцируете отказы системы и заводите баг-репорты.
- **SDLC (Развертывание и Поддержка) ↔ STLC (Завершение цикла):** Промокоды уходят реальным пользователям ("на прод"). Ваша задача перед этим этапом: *предоставить информацию для принятия решений*. Вы формируете отчет: сколько багов нашли, сколько починили, какие остались, оцениваете качество приложения. А затем команда QA анализирует свою работу: что мы можем улучшить в нашем процессе (QA), чтобы в следующий релиз багов было меньше?

**Итог:** Жизненный цикл разработки (SDLC) показывает, на каком этапе находится весь проект. А жизненный цикл тестирования (STLC) дает вам четкую инструкцию, **что конкретно вам нужно делать прямо сейчас**.

**Модель разработки ПО (Software Development Model, SDM)** — структура, систематизирующая различные виды проектной деятельности, их взаимодействие и последовательность в процессе разработки ПО. [Куликов]

В курсе мы рассмотрим следующие модели разработки:

1. Водопадная модель (waterfall model)
2. V-образная модель (V-model)
3. Итерационная инкрементальная модель (iterative model, incremental model) - также можно использовать их по отдельности
4. Спиральная модель (spiral model)
5. Гибкая модель (agile model), в частности Scrum

Это далеко не весь перечень доступных моделей, остальные вы можете изучить самостоятельно.

В современных командах чаще всего используются гибкие модели разработки, в основе которых лежит итерационная инкрементальная модель и спиральная модель.

**Водопадная модель** (waterfall model) сейчас представляет скорее исторический интерес, т.к. в современных проектах практически неприменима, исключая авиастроение, военную или космическую отрасли, медицину и финансовый сектор. Она предполагает однократное выполнение каждой из фаз проекта, которые, в свою очередь, строго следуют друг за другом. Очень упрощенно можно сказать, что в рамках этой модели в любой момент времени команде «видна» лишь предыдущая и следующая фаза. В реальной же разработке ПО приходится «видеть весь проект целиком» и возвращаться к предыдущим фазам, чтобы исправить недоработки или что-то уточнить.

Т.е. вернуться на предыдущую фазу в случае ошибки нельзя, все нужно начинать сначала (уточнение автора курса).

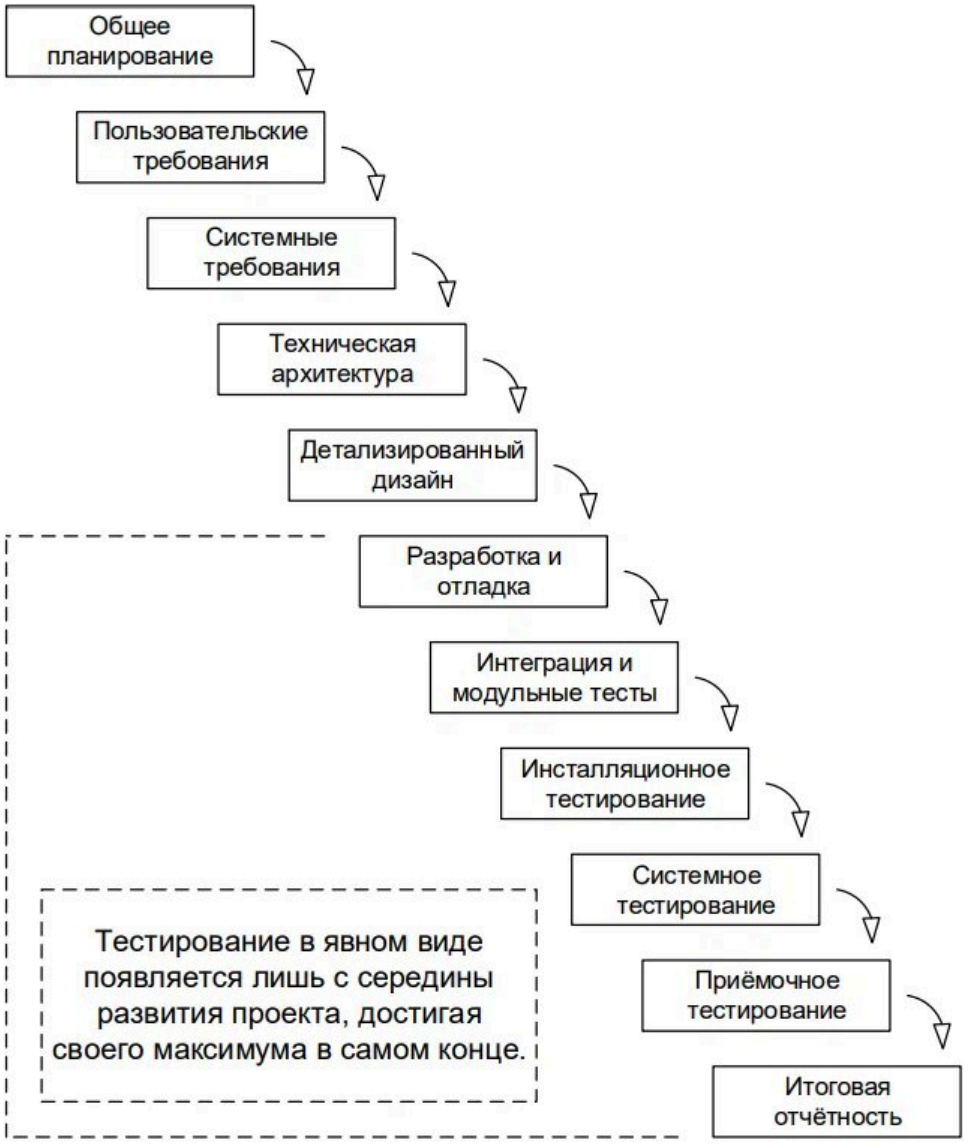



Рисунок 2.1.а — Водопадная модель разработки ПО

К недостаткам водопадной модели принято относить тот факт, что участие пользователей ПО в ней либо не предусмотрено вообще, либо предусмотрено лишь косвенно на стадии однократного сбора требований. С точки зрения же тестирования эта модель плоха тем, что тестирование в явном виде появляется здесь лишь с середины развития проекта, достигая своего максимума в самом конце.

***Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”***

V-образная модель часто вызывает путаницу у новичков, потому что выглядит сложнее, чем классическая «водопадная». Но на самом деле она просто показывает **одну важную мысль**: каждую стадию разработки нужно заранее **сопоставлять с будущим этапом тестирования**.

Например, если вы сейчас пишете требования, подумайте сразу – **как вы это будете тестировать**? Если проектируетесь — **какие проверки подтвердят, что архитектура работает как задумано**? Это и есть логика «спуск-подъём» в V-модели.

Такой подход особенно полезен в проектах с высокой ценой ошибки — например, в медицине, финансах или авиации. Вы не начинаете тестировать только когда продукт готов, а **встраиваете тестирование в каждую фазу проекта**, начиная с самой первой.

V-модель — не просто метод разработки, а **способ мышления**. Это важная концепция для начинающего тестировщика: **всё, что создаётся, должно быть проверено** – и вы должны знать, **на какой стадии и чем именно** это можно протестировать.

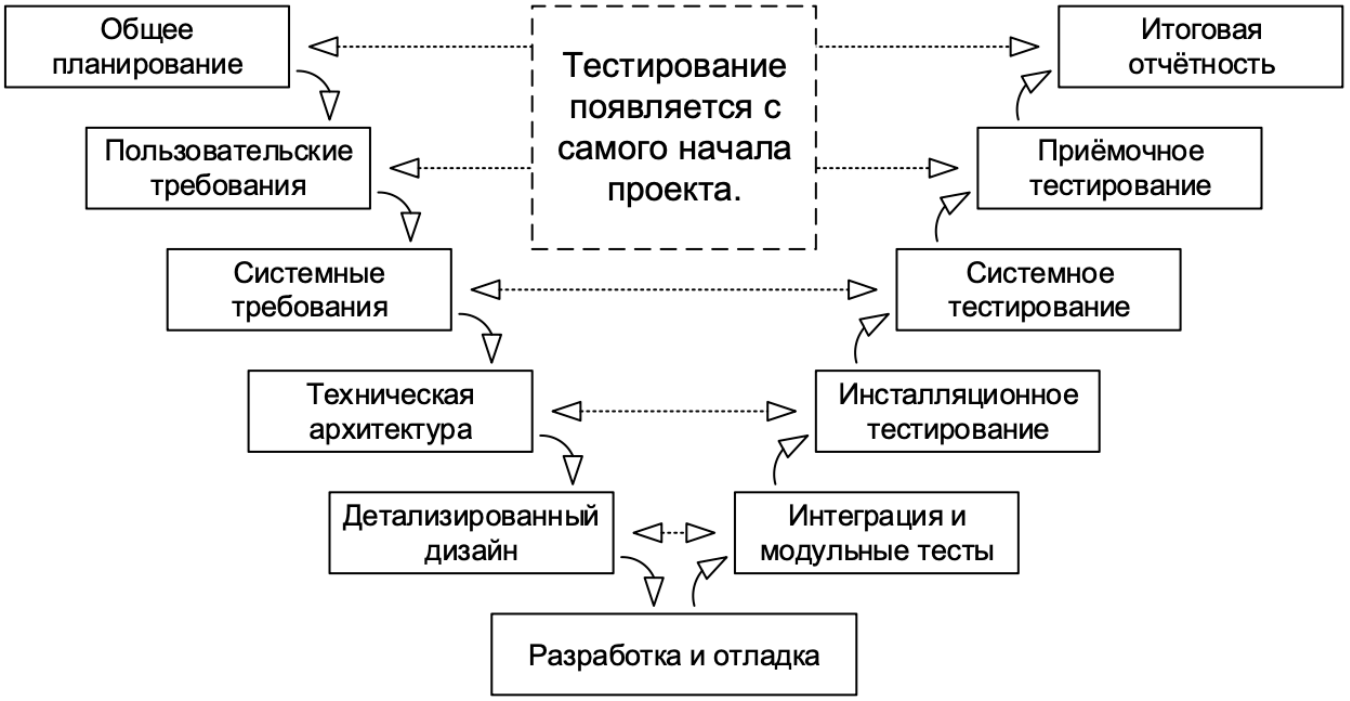



Рисунок 2.1.b — V-образная модель разработки ПО

***Источник картинки: Святослав Куликов “Тестирование ПО. Базовый курс.”***

### Итерационная инкрементальная модель

Итерационная инкрементальная модель (iterative model, incremental model) является фундаментальной основой современного подхода к разработке ПО. Как следует из названия модели, ей свойственна определённая двойственность (а ISTQB-глоссарий даже не приводит единого определения, разбивая его на отдельные части):

- с точки зрения жизненного цикла модель является итерационной, т.к. подразумевает многократное повторение одних и тех же стадий;
- с точки зрения развития продукта (приращения его полезных функций) модель является инкрементальной.

Ключевой особенностью данной модели является разбиение проекта на относительно небольшие промежутки (итерации), каждый из которых в общем случае может включать в себя все классические стадии, присущие водопадной и v-образной моделям (рисунок 2.1.c). Итогом итерации является приращение (инкремент) функциональности продукта, выраженное в промежуточном билде (build).

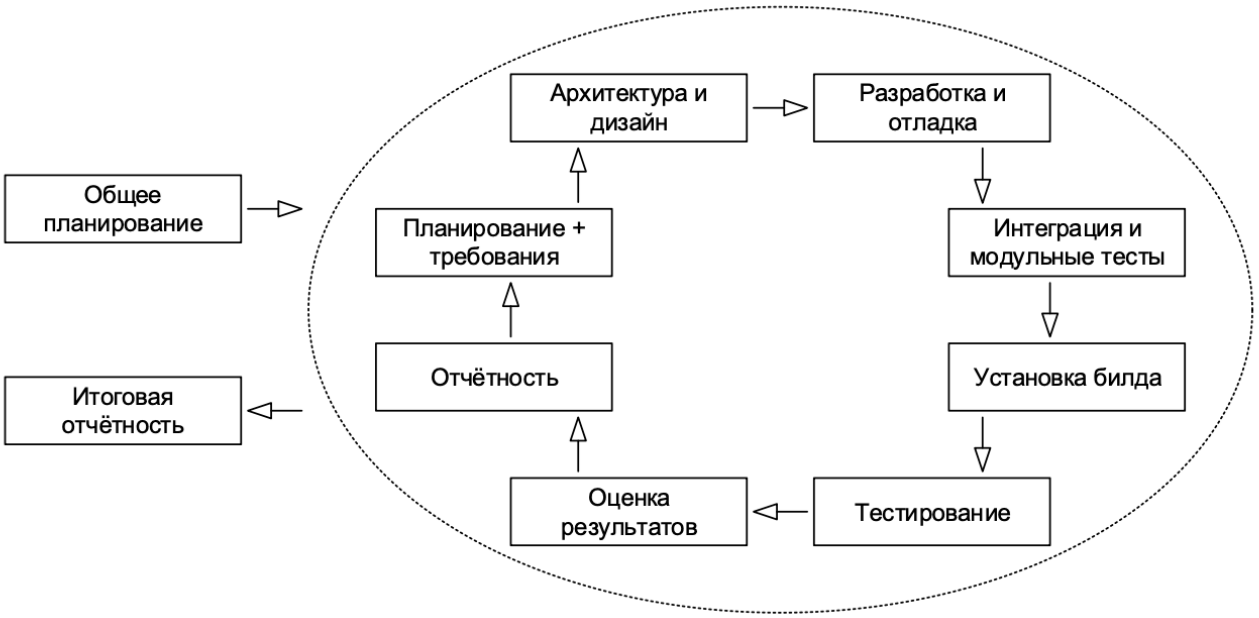



Рисунок 2.1.c — Итерационная инкрементальная модель разработки ПО

Длина итераций может меняться в зависимости от множества факторов, однако сам принцип многократного повторения позволяет гарантировать, что и тестирование, и демонстрация продукта конечному заказчику (с получением обратной связи) будет активно применяться с самого начала и на протяжении всего времени разработки проекта.

Итерационная инкрементальная модель очень хорошо зарекомендовала себя на объемных и сложных проектах, выполняемых большими командами на протяжении длительных сроков. Однако к основным недостаткам этой модели часто относят высокие накладные расходы, вызванные высокой «бюрократизированностью» и общей громоздкостью модели.

***Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”***

### Примечание автора курса

Инкремент это не просто итоговая функциональность полученная в конце итерации. Это рабочий продукт из прошлой итерации, к которому была добавлена новая функциональность. Каждую новую итерацию инкремент прирастает новым кодом.

## Спиральная модель

Спиральная модель *(знать для общего развития)* — это про **разработку с оглядкой на риски**. Вы не просто делаете продукт шаг за шагом, а каждый раз **останавливаетесь и оцениваете**, какие есть угрозы: технические, организационные, бизнесовые.

Чаще всего её применяют в **сложных и нестабильных проектах**, где всё может меняться — например, если заказчик сам ещё не до конца понимает, чего хочет. Или если технология новая и риск не уложиться в сроки или бюджет — высокий.

Для тестировщика это значит, что **качество и риски обсуждаются с самого начала**, а не в последнюю неделю перед релизом. Тестирование может включать **прототипы**, **оценку уязвимых зон** и даже **влияние изменений требований** на уже готовую часть продукта.

Если вы только начинаете путь в тестировании, просто запомните: **спиральная модель делает акцент не только на разработке, но и на постоянной переоценке рисков**. А значит, вы как тестировщик — не просто ловите баги, а участвуете в снижении рисков проекта.

## Как это выглядит на проекте?

Перед вами – диаграмма, иллюстрирующая **спиральную модель разработки ПО**. Это наглядное объяснение того, как команда движется шаг за шагом, итерация за итерацией, **постоянно анализируя риски** и уточняя продукт.

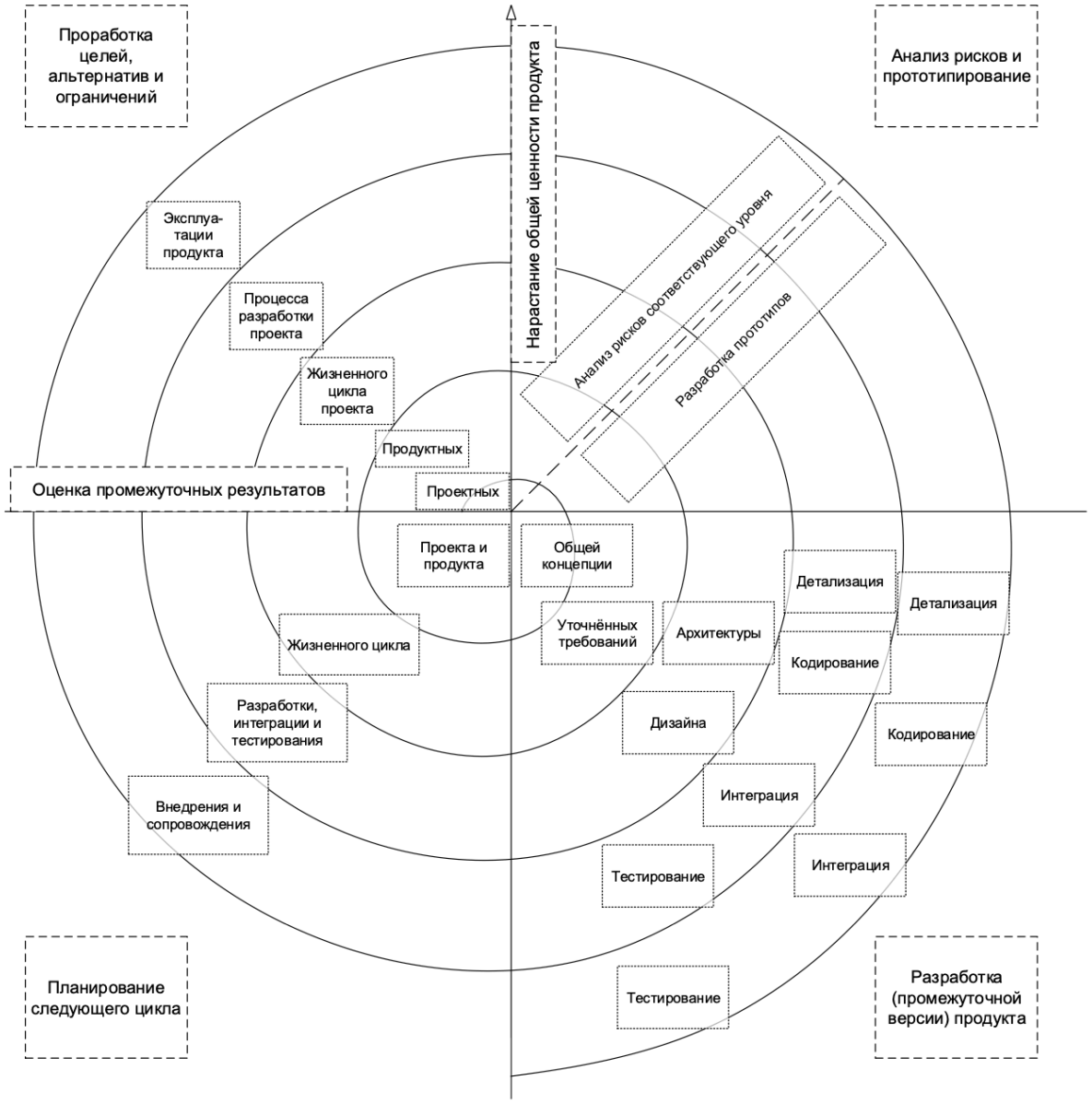



Рисунок 2.1.d — Спиральная модель разработки ПО

**Форма спирали** здесь символизирует итерационный подход — вы как бы «проходите круг», но каждый раз поднимаетесь на новый уровень и двигаетесь ближе к конечному решению. С каждым витком ценность продукта растёт, а команда лучше понимает, что именно нужно реализовать.

**Каждая итерация включает четыре ключевых фазы**:

1. **Проработка целей, альтернатив и ограничений** — команда определяет, что именно нужно сделать, какие есть варианты реализации и что может этому мешать.

2. **Анализ рисков и прототипирование** — вы оцениваете, какие есть потенциальные проблемы, и делаете прототип (черновой вариант), чтобы проверить, работает ли решение.
3. **Разработка и тестирование** — создаётся промежуточная версия продукта и сразу проверяется её работоспособность.
4. **Планирование следующего витка** — на основе полученного опыта команда планирует, что делать дальше.

Важно: **тестирование присутствует на каждом витке**, а не только в самом конце. Это делает процесс гибким и помогает находить ошибки на ранних этапах, пока их проще и дешевле исправить.

Для начинающего тестировщика: если в проекте используется спиральная модель, это значит, что вы будете **постоянно участвовать в обсуждении требований, оценке рисков и проверке промежуточных версий**. Не бойтесь вникать в продукт — здесь ваша обратная связь особенно ценна.

***Источник диаграммы: Святослав Куликов “Тестирование ПО. Базовый курс.”***

**Гибкая модель**

Гибкая модель представляет собой совокупность различных подходов к разработке ПО и базируется на т.н. «agile-манифесте».

- Люди и взаимодействие важнее процессов и инструментов.
- Работающий продукт важнее исчерпывающей документации.
- Сотрудничество с заказчиком важнее согласования условий контракта.
- Готовность к изменениям важнее следования первоначальному плану.

Стоит понимать, что левая часть утверждения не отменяет наличие правой. Речь лишь про то, что является более приоритетным для данной модели. *(примечание автора курса)*

В отличие от классических подходов, они позволяют быстро адаптироваться к изменениям, фокусируются на ценности для пользователя и вовлекают команду в процесс принятия решений.

В центре таких подходов – **итерационная разработка**, когда работа делится на короткие циклы (итерации), обычно по 1–2 недели. После каждой итерации команда выпускает улучшенную версию продукта, оценивает результаты, получает обратную связь и планирует следующий шаг. Всё строится на **прозрачности**, **сотрудничестве** и **непрерывных улучшениях**.

Работа команды включает несколько уровней:

- **Стратегия** – определение целей, бюджета и границ проекта.
- **Планирование релиза** – формирование бэклога и видения ближайших версий.
- **Итерации** – циклическая работа: план → выполнение → тестирование → ретроспектива → снова план.
- **Ежедневные процессы** – короткие собрания, где команда обсуждает, что сделано, что предстоит и какие есть препятствия.
- **Непрерывные практики** – автоматическое тестирование, рефакторинг, интеграция, постоянное взаимодействие между участниками.

Такая модель разработки требует от тестировщика **вовлеченности с самого начала**. Ты не просто «проверяешь продукт», а активно участвуешь в его создании, формулировке требований, выявлении рисков и улучшении качества на каждом этапе.

Поэтому в Agile-командах особенно ценятся не только технические навыки, но и **гибкость, умение общаться, инициативность и способность видеть продукт целиком**.

*Далее в курсе мы подробнее поговорим про фреймворк Scrum, который является одним из самых популярных представителей гибкой модели, так что не пугайтесь, если что-то в тексте выше вам осталось непонятным.*

**Полезные материалы**

1. Презентация https://drive.google.com/file/d/1zObEXUavcC5XA8wrNSHcvOfhiwChOPsm/view?usp=share_link
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/sdlc-stlc
3. Старая версия урока https://youtu.be/5uYZvf9o4MM

4. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/185uHoyLxNyojTpRFcI-w6uGYxyCnCNo9
5. Тестирование программного обеспечения. Базовый курс, стр.18 - https://svyatoslav.biz/software_testing_book/
6. Библия QA. Модели разработки ПО https://vladislaveremeev.gitbook.io/qa_bible/sdlc-i-stlc/modeli-razrabotki-po

# 6. Уровни тестирования

**Уровни тестирования (test levels)** – это группы активностей тестирования, которые организованы и управляются как единое целое. [ISTQB CTFL Syllabus 2023]

**Уровень тестирования (test level)** – конкретная реализация процесса тестирования. [ISTQB Glossary]

**Уровни тестирования:**

- Unit/component/program/module testing - тестируется минимально-атомарный модуль программы, чаще всего это одна функция или метод. Таких тестов должно быть больше всего;
- Integration testing - несколько модулей программы тестируются вместе;
- System testing - вся программа тестируется полностью;
- Acceptance testing - программа принимается заказчиком на соответствие заявленным требованиям либо тестировщики проходят end-to-end сценарии с точки зрения пользователя;

**Источник:** https://vladislaveremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/piramida-urovni-testirovaniya-test-pyramid-testing-levels

**Модульное (юнит, компонентное) тестирование**

**Компонентное тестирование (component testing)** - уровень тестирования отдельных аппаратных или программных компонентов системы. [ISTQB Glossary]

**Модульное тестирование (оно же юнит-тестирование)** используется для тестирования какого-либо одного логически выделенного и изолированного элемента системы (отдельные методы класса или простая функция, subprograms, subroutines, классы или процедуры) в коде. [Библия QA]

**Компонентное тестирование** - тип тестирования ПО, при котором тестирование выполняется для каждого отдельного компонента отдельно, без интеграции с другими компонентами. [Библия QA]

Юнит-тест (unit test) — это автоматизированная проверка **маленькой отдельной части кода**, чаще всего **одной функции**. Такие тесты пишут сами

разработчики, чтобы убедиться, что логика внутри функции работает корректно при разных входных данных.

Ручной тестировщик напрямую **не пишет юнит-тесты**, но важно понимать, как они работают, чтобы:

– отличать их от других видов тестирования (например, интеграционного или системного);

– понимать, что именно уже протестировано на уровне кода и не дублировать это вручную;

– уметь общаться с разработчиками на одном языке, особенно в команде Agile.

**Простой пример**:

Допустим, есть функция, которая складывает два числа:

```
def add(a, b):
    return a + b
```

Юнит-тест для этой функции будет проверять:

1. Что add(2, 3) возвращает 5
2. Что add(-1, -2) возвращает -3
3. Что add(0, 5) возвращает 5

```
import unittest

class TestAddFunction(unittest.TestCase):
    def test_add_positive_numbers(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_negative_numbers(self):
        self.assertEqual(add(-1, -2), -3)

    def test_add_zero(self):
        self.assertEqual(add(0, 5), 5)

if __name__ == '__main__':
    unittest.main()
```

### Примечание автора курса

Различные источники по-разному трактуют подходы к компонентному тестированию. Где-то утверждается, что компонент, модуль и юнит одно и то же, где-то им приписываются разные свойства.

В контексте разработки вы будете чаще всего встречаться именно с unit-тестированием.

### Зачем это знать тестировщику?

Развею главный страх: ручному тестировщику **не нужно** уметь программировать и писать юнит-тесты. Это задача разработчиков. Но понимать, что это такое, критически важно для вашей эффективности.

Вспомните наш прошлый урок про **SDLC и STLC**. На этапе "Разработка/ Кодирование" программист пишет код, и сразу же покрывает его юнит-тестами. Это самая первая линия обороны от багов. И то, что команда обязывает программистов писать эти тесты — это чистый процесс **QA**, который предотвращает попадание глупых ошибок к вам на проверку.

К сожалению, на проектах не всегда есть юнит-тестирование, особенно, если он уже долго существует без этих тестов.

**Разберем на нашем примере с промокодами:** Разработчик написал небольшую функцию (модуль) `calculate_discount`, которая отнимает процент промокода от суммы в корзине.

- **Что делает юнит-тест (Пишет разработчик):** Проверяет математику в изоляции от реальных бизнес-процессов. Программист пишет автотест, который за долю секунды проверяет: если передать в функцию сумму 1000 и скидку 10%, она вернет 900. Если передать 0, то вернет 0.
- **Что делаете вы (Ручной тестировщик):** Зная, что математика уже надежно проверена юнит-тестами, вы **не тратите время** на ввод 50 разных сумм в корзину. Вы фокусируетесь на валидации пользовательского сценария: что будет, если применить промокод, а потом удалить товар? Не сломается ли верстка (дизайн), если скидка будет слишком длинной? Придет ли правильная сумма в чек на почту?

**Итог:** Знание того, как работают юнит-тесты, спасает вас от лишней работы (вы не дублируете проверки разработчиков). Кроме того, когда вы найдете сложный баг, вы сможете сказать программисту: *"Слушай, на интерфейсе скидка не применяется, хотя в базе всё ок. Кажется, где-то ошибка. Можешь проверить юнит-тесты для функции расчета?"*. Это покажет вас как профессионала, который понимает архитектуру приложения и говорит с командой на одном языке.

## Интеграционное тестирование

**Интеграционное тестирование (integration testing)** - уровень тестирования, который фокусируется на взаимодействии между компонентами или системами

**Уровни интеграционного тестирования** :

## Примечание автора курса

Существует и другой подход к определению системного интеграционного уровня, при котором тестируется взаимодействие двух и более систем. Например, использование прикладного программного интерфейса (API), с помощью которого можно передавать информацию из одного веб-приложения в другое.

## Зачем это знать тестировщику?

> *"Два юнит-теста прошли успешно, но интеграционный провален".*

Компоненты программы могут работать безупречно по отдельности, но при попытке обменяться данными система падает. Знание принципов интеграционного тестирования помогает тестировщику искать баги там, где они прячутся чаще всего. Вы перестаете смотреть на приложение как на черный ящик.

**Разберем на примере нашего интернет-магазина:** У нас есть модуль "Корзина" и модуль "Оплата".

- **Компонентная интеграция:** В прошлом шаге разработчик проверил математику внутри модулей. А теперь мы проверяем их связь. Что будет, когда "Корзина" попытается передать итоговую сумму в "Оплату"? Вдруг "Корзина" передает число дробью (1000.50), а "Оплата" умеет принимать только целые числа? По отдельности код работает, но вместе они спровоцируют отказ.
- **Системная интеграция:** Вы нажимаете "Оплатить", и ваш магазин обращается к совершенно чужой системе: API банка. Вы тестируете этот "мост" между вашим сайтом и банком. Понял ли банк наш запрос? Что будет, если банк ответит ошибкой "Недостаточно средств", не упадет ли наш сайт?

Да, отладка — это задача программиста. Но если вы понимаете интеграцию, ваш баг-репорт будет звучать не как бесполезное *«Оплата не работает»*, а как *«Ошибка при передаче суммы из Корзины в API банка»*. Разработчику не придется перерывать весь код, он сразу пойдет чинить конкретный "мост" между модулями.

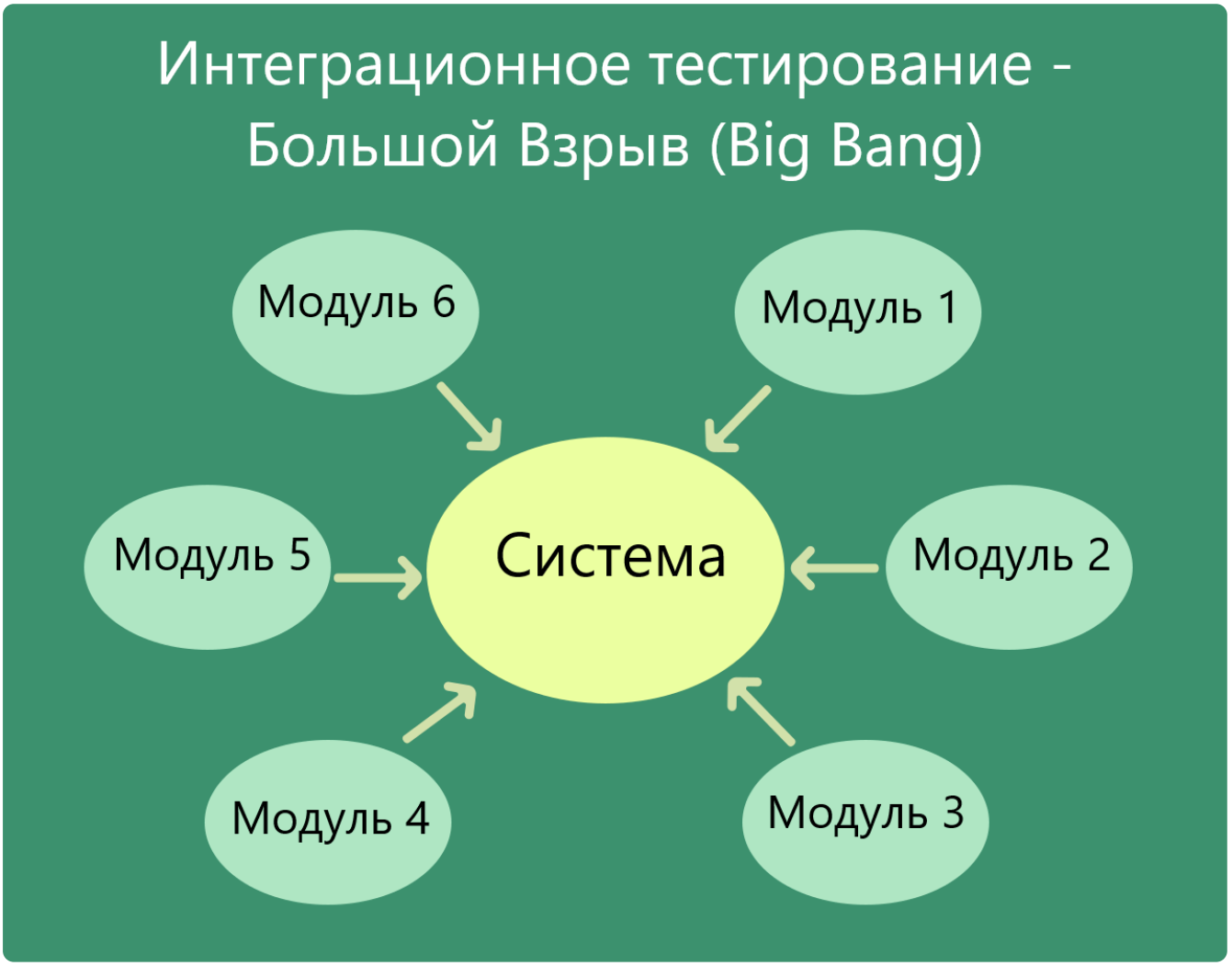
**Итог:** Интеграционное тестирование учит вас проверять надежность связей. Вы фокусируетесь на том, как передаются данные между экранами, функциями и сторонними сервисами, гарантируя, что все детали пазла идеально подходят друг другу.

Хотя **подходы к интеграционному тестированию** не являются обязательными для запоминания новичками, понимание их в общем виде поможет лучше ориентироваться в том, **как в реальности происходит сборка и проверка сложных систем**. Особенно это полезно, если вы участвуете в проектах с модульной архитектурой или вендорских интеграциях.

### 🟡 Big Bang (подход большого взрыва)

Все модули собираются сразу, и проверка проводится на полной системе. Быстро, но может быть сложно понять, откуда именно пошел сбой. Такой подход используется редко и скорее говорит о спешке, чем о хорошей инженерной культуре.

**Пример:** продукт состоит из 10 модулей (UI, API, платежи, база данных, авторизация, отчёты и т.д.). Разработчики закончили всё примерно одновременно и сразу отдали на интеграционное тестирование.




### 🟢 Инкрементальный подход

Более гибкий и управляемый. Модули интегрируются постепенно, что позволяет **выявлять дефекты локально, а не в конце**.

**— Top-down (сверху вниз)**

Начинаем с основных (высокоуровневых) компонентов. Те модули, которые пока не готовы, заменяются заглушками (stubs). Такой подход удобен, когда у нас уже есть интерфейсы, и важно убедиться, что “верх работает”.

**Пример:** вы тестируете банковское мобильное приложение. Готов дизайн, навигация, верхнеуровневый интерфейс. Но модули для операций по счету ещё не реализованы.

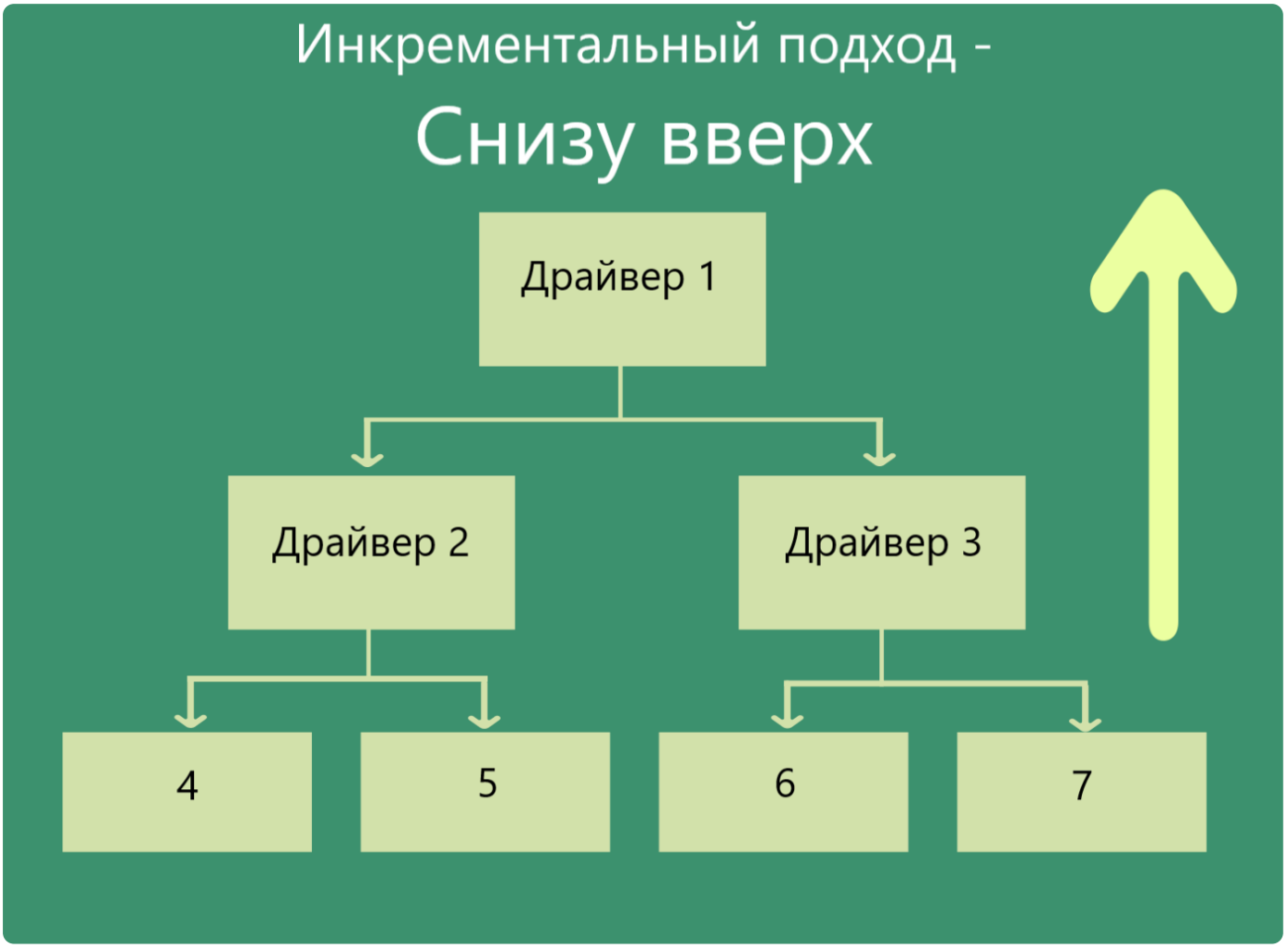
**Как работает:** вы используете заглушки для ответа API, чтобы не ждать бэкенд. Проверяете логику и переходы между экранами, проверку форм, отображение ошибок.

— **Bottom-up (снизу вверх)**

Сначала тестируются низкоуровневые модули, а высокоуровневые пока заменяются драйверами. Подходит, если сначала реализуется “внутренность” системы, а UI или API верхнего уровня еще в разработке.

**Пример:** в HR-системе готова вся логика: работа с БД, генерация PDF, рассылка писем. Но интерфейс ещё не реализован.

**Как работает:** вы пишете или запускаете SQL-запросы, тестируете REST API в Postman, подключаете логгеры, чтобы проверить, как формируются отчёты.




— **Sandwich (сэндвич или гибрид)**

Смешанный подход — тестируются одновременно модули сверху и снизу, фокус в середине. Используется при комплексной разработке с параллельными командами.

**Пример:** вы тестируете маркетплейс. Команда работает параллельно: одна делает фронт (UI), другая — API, третья — middle layer (логика ценообразования).

**Как работает:** UI подключён к заглушкам бекенда, бекенд работает с драйверами, имитирующими фронт. При этом middle-layer уже можно интегрировать полностью.

*Такую информацию вряд ли спросят прямо на интервью, но если вы работаете на проекте, где нужно понимать связи между системами, — базовое представление об этих подходах точно не повредит.*

**Информация взята и адаптирована из:** https://vladislaveremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/integracionnoe-testirovanie-integration-testing

## Системное тестирование

**Системное тестирование** означает тестирование всей системы в целом, оно выполняется после интеграционного тестирования, чтобы проверить, работает ли вся система целиком должным образом. В основном это тестирование типа "черный ящик", которое оценивает работу системы с точки зрения пользователя с помощью документа спецификации и оно не требует каких-либо внутренних знаний о системе, таких как дизайн или структура кода.

**Зачем нужно системное тестирование**?

- Очень важно завершить полный цикл тестирования, и ST - это этап, на котором это делается;
- ST выполняется в среде, аналогичной production environment, и, следовательно, заинтересованные стороны могут получить хорошее представление о реакции пользователя;
- Это помогает свести к минимуму устранение неполадок после развертывания и количество обращений в службу поддержки;
- На этом этапе STLC тестируются архитектура приложения и бизнес-требования. Это тестирование очень важно, и оно играет важную роль в предоставлении клиенту качественного продукта.

**Источник:** https://vladislaveremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/sistemnoe-testirovanie-system-testing

## Примечание от автора курса

**Системное тестирование** – это и есть ваша основная работа. Юнит-тесты пишут разработчики, интеграцию часто проверяют скриптами, а здесь начинаются будни ручного QA. Вы берете собранное приложение и просто им пользуетесь.

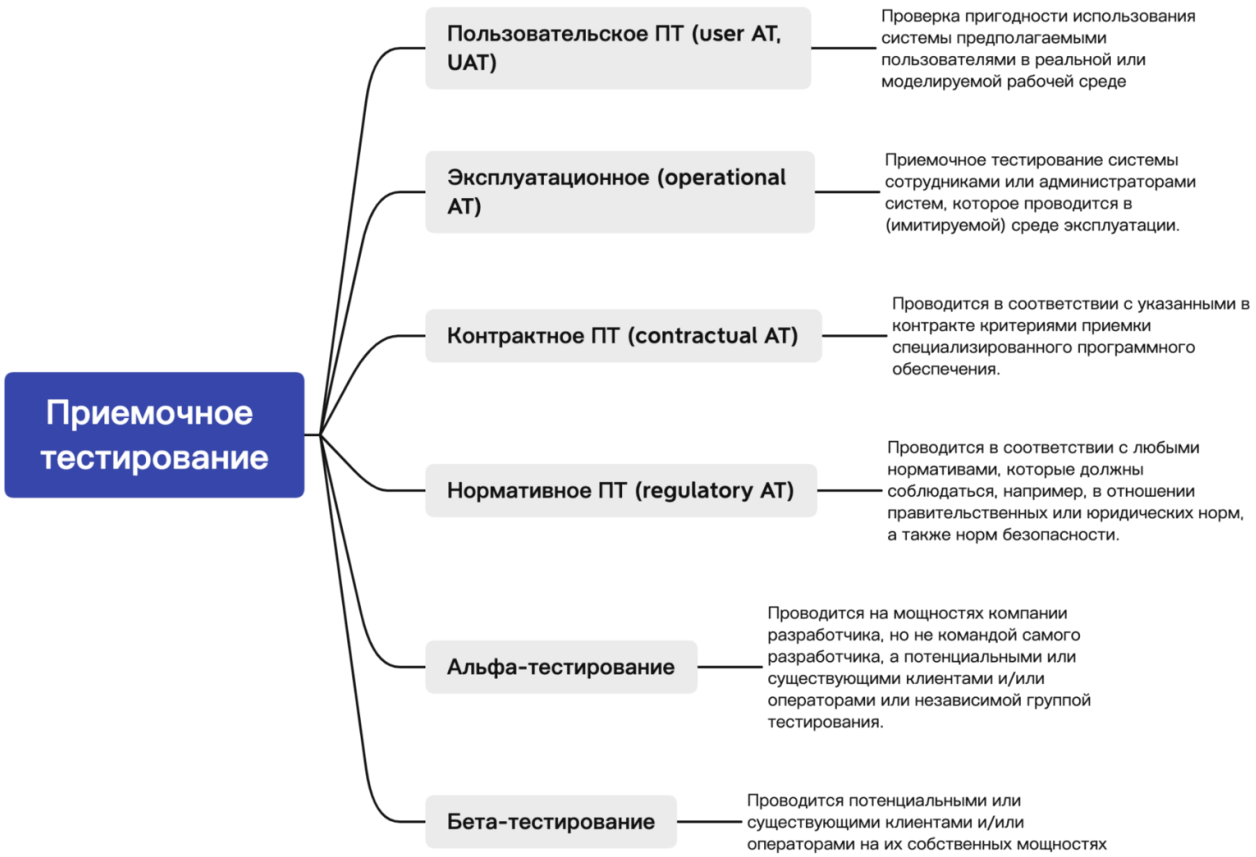
Ваша задача — проверить конкретный бизнес-сценарий. Например, можно ли положить телефон в корзину, применить промокод, выбрать доставку курьером и оплатить картой. Работает? Значит, система в порядке.

Тут всплывает критически важный нюанс: среда тестирования. На практике это значит, что вы никогда не проверяете задачи на компьютере программиста, локально. Там всегда "всё работает". Вы требуете от админов настроить специальный тестовый сервер (стенд), который будет точной копией реального проекта (production).

Если тестовый стенд настроен криво или там лежит устаревшая база данных, то это нужно менять. Вы дадите зеленый свет, релиз уйдет к реальным клиентам, корзина сломается, и техподдержка сойдет с ума от жалоб. На системном уровне вы работаете как последний фильтр.

## Приемочное тестирование

**Приемочное тестирование (acceptance testing)** - уровень тестирования, целью которого является определение того, принимать ли систему



## Зачем это знать тестировщику?

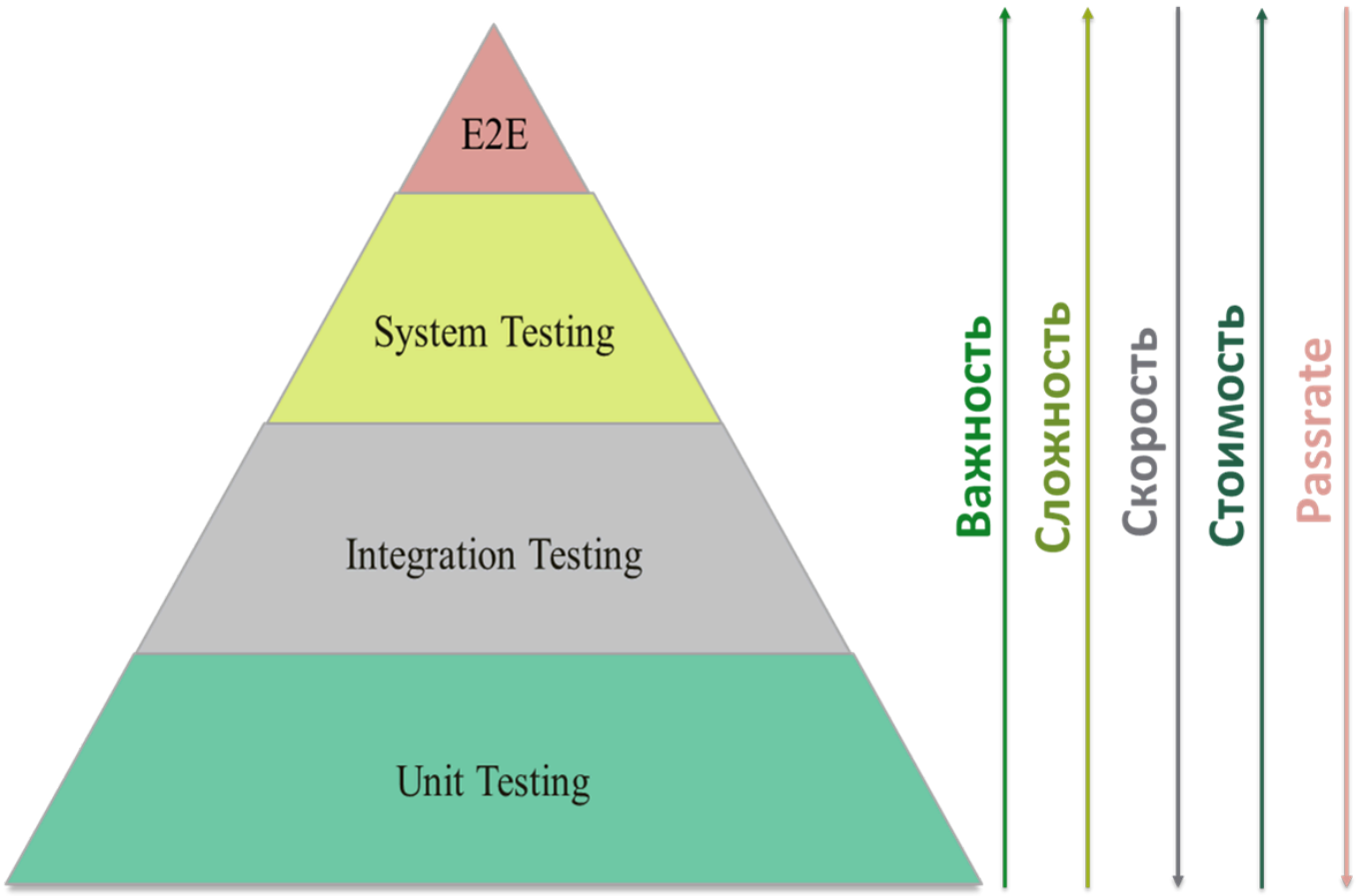
Казалось бы, приемочное тестирование проводят заказчики или сами пользователи. Зачем в это вникать QA-инженеру?

На практике именно вы будете этот процесс организовывать. Заказчик редко знает, как методично проверить продукт. Ваша задача: подготовить для него понятные сценарии, завести нужные тестовые аккаунты, настроить среду и собрать обратную связь.

Это финальная стадия валидации. На системном уровне вы убедились, что приложение не падает и работает по документации. На приемочном – бизнес смотрит, решает ли программа его реальные проблемы.

Главное правило: приемочное тестирование проводится не для поиска дефектов. Этот этап нужен исключительно для того, чтобы получить финальное одобрение перед выходом продукта на реальный рынок.

**«Пирамида тестов»** - метафора, которая означает группировку динамических тестов программного обеспечения по разным уровням. Она также дает представление, какое количество тестов должно быть в каждой из этих групп. Основной принцип разделения уровней - тест должен быть на том же уровне, что и тестируемый объект. В тесте более высокого уровня вы не тестируете всю условную логику и пограничные случаи, которые уже покрыты тестами более низкого уровня.




**Примечание автора курса**

На некоторых вариантах пирамид вы можете увидеть пирамиду тестирования, где уровень UAT заменен на E2E тесты. В контексте данной пирамиды это одно и то же.

Если копнуть глубже, то **end-to-end тестирование, или сквозное тестирование** - это вид тестирования, используемый для проверки программного обеспечения от начала до конца, а также его интеграцию с внешними интерфейсами.

На примере интернет магазина, это одна объемная проверка, которая включает в себя все шаги пользователя, начиная от регистрации в системе и заканчивая покупкой товара в магазине.

Подробнее можно прочитать здесь.

Также вы можете обратить внимание на увеличение/уменьшение важности, сложности, скорости, стоимости, уровня прохождения на этих уровнях.

**Значение для тестирования**

Пирамида тестов решает самую больную проблему на реальных проектах: нехватку времени и денег. Вы физически не сможете проверить каждую мелочь на системном уровне руками или через автоматизацию интерфейса. Это слишком долго.

Допустим, у вас есть форма регистрации. Проверять все ошибки валидации почты, спецсимволы и пустые поля через интерфейс браузера – плохая идея. Зная принцип пирамиды, вы договариваетесь с разработчиками: пусть они напишут десяток быстрых юнит-тестов (база пирамиды), которые проверят логику валидации за пару миллисекунд. А на самом верху, на уровне E2E, вы оставите только один длинный сценарий, имитирующий реального пользователя: успешная регистрация, добавление товара в корзину и покупка.

Если QA-инженер не понимает этого распределения, начинается жесткое дублирование проверок. Команда пытается автоматизировать всё через пользовательский интерфейс. В индустрии это называется антипаттерном "рожок мороженого" (ice-cream cone): огромная тяжелая шапка медленных, дорогих и хрупких E2E-тестов наверху, и почти полное отсутствие дешевых проверок кода внизу.

В результате прогон таких тестов перед релизом занимает часы. Они постоянно падают из-за любой смены цвета кнопки или задержки сети, а команда тратит дни на их починку вместо поиска реальных багов. Пирамида учит вас главному правилу оптимизации: тестировать приложение нужно на максимально низком из возможных уровней. Сдвигайте проверки "вниз", оставляя наверху только самые критичные сквозные бизнес-пути.

**Полезные материалы**

1. Презентация https://drive.google.com/file/d/1kh8lIU3z26c2CC2QF_vZ_LFVLSHcATPl/view?usp=share_link
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/pyramid
3. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1bTpTj05t_Ul9OHV3WRoHfwSDdUsKylIG
4. Видео про различные интерфейсы https://youtu.be/zs13PZhW9lM. Также вы можете пройти тест по нему.
5. Тестирование программного обеспечения. Базовый курс, стр.77 - https://svyatoslav.biz/software_testing_book/

6. Библия QA. Пирамида тестирования - https://vladislaveremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/piramida-urovni-testirovaniya-test-pyramid-testing-levels
7. Модульное/юнит/компонентное тестирование (Module/Unit/Component testing) https://vladislaveremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/modulnoe-yunit-komponentnoe-testirovanie-module-unit-component-testing
8. Интеграционное тестирование (Integration testing) https://vladislaveremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/integracionnoe-testirovanie-integration-testing
9. Системное тестирование (System Testing) https://vladislaveremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/sistemnoe-testirovanie-system-testing
10. Приемочное тестирование (AT - Acceptance testing) https://vladislaveremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/priemochnoe-testirovanie-at-acceptance-testing
11. Обзорная статья с реальными примерами для каждого уровня https://habr.com/ru/company/otus/blog/720664/
12. Схема из урока про "Виды интерфейсов" https://drive.google.com/drive/u/3/folders/1A7zlUcAcOUaBMEHkl_EnhrNgn53wi3pk

# 7. Типы тестирования. Часть 1

### Примечание от автора курса

Существует большое количество классификаций тестирования от разных авторов и стандартов. В данном курсе мы сконцентрируемся на типах и методах тестирования, определенных в ISTQB CTFL и пособии Святослава Куликова.

Мы не сможем рассмотреть абсолютно все типы, поэтому обозрим только самые основные.

Советую подойти к изучению этой темы последовательно и не стараться разобраться во всех типах тестирования за один раз.

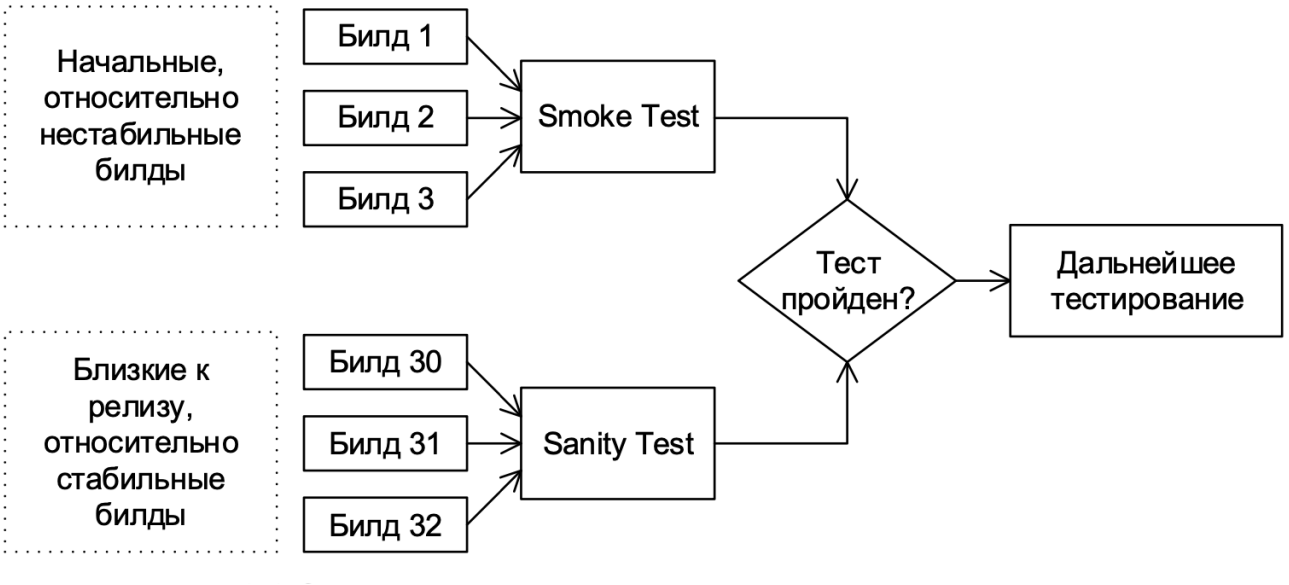
Старайтесь находить примеры для каждого типа тестирования в контексте общих процессов и практики. Например, вы можете использовать ChatGPT или Deepseek с таким запросом:

> Представь, что ты тестировщик ПО. Тебе надо разобраться с типами тестирования на реальных примерах из практики. Приведи 5 примеров тестирования локализации, которые будут понятны новичку.

Используйте эту технику и для других определений из теории. Нам важно не просто заучить термин и его определение, а понимать контекст!

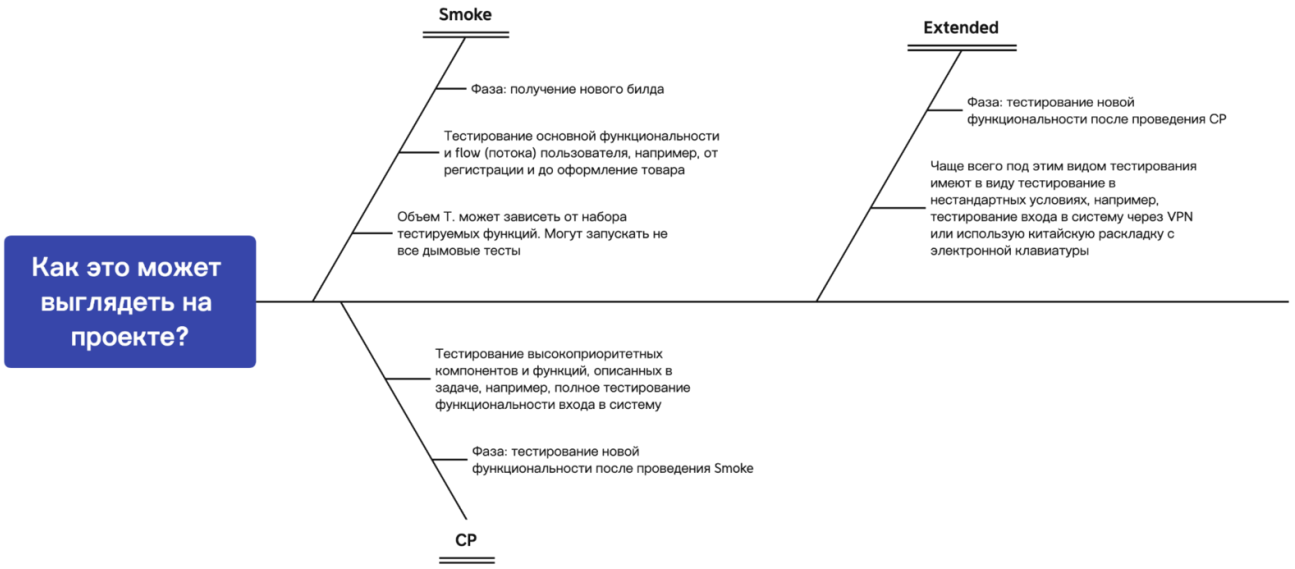
### Классификация по (убыванию) степени важности тестируемых функций

- **Дымовое тестирование (smoke test)** направлено на проверку самой главной, самой важной, самой ключевой функциональности, неработоспособность которой делает бессмысленной саму идею использования приложения (или иного объекта, подвергаемого дымовому тестированию).
- Очень часто можно услышать вопрос о том, чем «smoke test» отличается от **«sanity test»**. В глоссарии ISTQB сказано просто: «sanity test: See smoke test». Но некоторые авторы утверждают, что разница есть и может быть выражена следующей схемой




- **Тестирование критического пути (critical pathtest)** направлено на исследование функциональности, используемой типичными пользователями в типичной повседневной деятельности.
- **Расширенное тестирование (extended test)** направлено на исследование всей заявленной в требованиях функциональности — даже той, которая низко проранжирована по степени важности. Ещё одним направлением исследования в рамках данного тестирования являются нетипичные, маловероятные, экзотические случаи и сценарии использования функций и свойств приложения, затронутых на предыдущих уровнях.

*Источник: [Святослав Куликов “Тестирование ПО. Базовый курс.”]*



### Пояснения автора курса

- **Smoke:** пользователь авторизовался, смог найти и добавить товар в корзину, совершил оплату и оформил доставку - в качестве примера для онлайн-магазина.

Проверка основной функциональности обычно не меняется и проводится на каждой новой промежуточной версии продукта, или билде. Иногда это процесс называют **сертификацией билда**.

Если Smoke не проходит, то остальное тестирование останавливается и ждет доработки кода.

- **Critical path:** тестируются новые или измененные функции, которые попали в промежуточную версию продукта с точки зрения использования конечного пользователя.
- **Extended:** все остальное.

Что касается **Sanity** тестирования, то с одной точки зрения это синоним Smoke.

Есть и другая точка зрения, при которой Smoke является тестированием основной функциональности на нестабильных билдах, а Sanity проверяет то же самое, но уже на стабильных билдах, близких к релизу.

*Шпаргалка (адаптирована под десктоп) https://rusau.net/phase*

### Ретест и регрессионное тестирование

**Подтверждающее (re-testing, ретест, повторное тестирование)** - тип тестирования, связанного с изменениями, которое выполняется после исправления дефекта для подтверждения того, что отказ, вызванный этим дефектом, не воспроизводится. [ISTQB Glossary]

**Регрессионное тестирование (regression testing)** - тип тестирования, связанного с изменениями, чтобы найти привнесенные или ранее не обнаруженные дефекты в не менявшихся областях программного обеспечения. [ISTQB Glossary]

### Примечание автора курса

**Ретест** подтверждает факт того, что дефект был исправлен. Он не требует оценку влияния изменений, хотя хороший тестировщик всегда потратит

несколько минут на общую оценку работы приложения после такого фикса.

**Регрессионное тестирование** проводится после любого изменения (новая функциональность, исправление дефекта, удаление функциональности), чтобы убедиться в том, что приложение работает стабильно. Оно не включает в себя ретест, а проводится уже после него.

### Особенности регрессионного тестирования из практики

Регрессионное тестирование является одной из самых главных активностей тестировщика и проводится перед каждым релизом (выпуском продукта для конечного пользователя).

Если релиз запланирован на каждую неделю, то регрессия проводится с такой же периодичностью, поэтому тесты для регрессии, как и дымные тесты очень часто автоматизируют для экономии времени.

Обычно для проведения регрессии выделяют несколько дней, код проекта "замораживают"(code freeze) и в него нельзя вносить изменения, кроме исправлений критичных дефектов.

### Важно понимать: какие тесты в конечном итоге попадут в регрессионный набор?

Регрессионные тесты выбираются из уже существующих тестовых наборов на основании следующих принципов:

- *Тесты, проверяющие часть приложения, в которые вносились изменения*

Например, если дефект или новая функция локализованы в модули регистрации, необходимо убедиться, что остальные компоненты этого модуля работоспособны

- *Тесты с высоким приоритетом*

Мы не можем рисковать работоспособностью самых важных функций приложения, поэтому необходимо дополнительно проверять и их

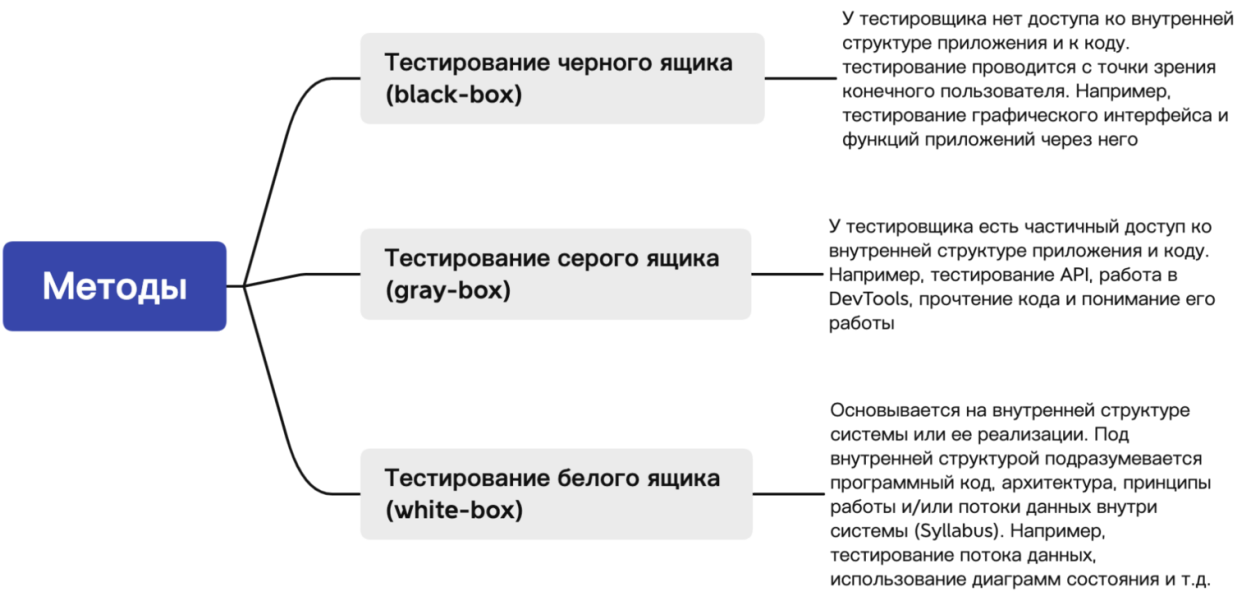
- *Тесты, которые проверяют модули с наибольшей концентрацией дефектов*

Согласно принципу скопления дефектов необходимо убедиться, что в уязвимой части приложения нет дополнительных проблем.

## Методы тестирования

Ручные тестировщики чаще всего работают с методами черного и серого ящика.

Метод тестирования серого ящика не упоминается в ISTQB CTFL.




Шпаргалка (адаптирована под десктоп) https://rusau.net/methods

## Функциональное и нефункциональное тестирование

**Функциональное тестирование системы (functional testing)** включает тесты по оценке функций, которые должна выполнять система. Данный вид тестирования можно проводить на всех уровнях.

**Нефункциональное тестирование системы (non-functional testing)** выполняется для оценки таких характеристик системы и программного обеспечения, как удобство использования, производительность или безопасность. За классификацией характеристик качества программного обеспечения следует обратиться к стандарту ИСО (ISO/IEC 25010). Данный вид тестирования можно проводить на всех уровнях.

*Источник: ISTQB CTFL Syllabus 2018*

## Примечание автора курса

*Функциональное тестирование* отвечает на вопрос *"Что делает система?".*

*Нефункциональное тестирование* отвечает на вопрос *"Как система это делает?".*

*Подробнее, что именно относится к нефункциональному и функциональному тестированию поговорим далее.*

## Нефункциональное тестирование

В практике тестирования принято разделение тестирования на тестирование функционального показателя качества, называемое «функциональным тестированием», и тестирование других показателей качества, называемое «нефункциональным тестированием».

Тип тестирования, используемого для определения показателя качества, отличного от функциональной пригодности, обычно называют нефункциональным типом тестирования и к нему можно отнести такие типы тестирования, как нагрузочное тестирование, стрессовое тестирование, тестирование на возможность проникновения, тестирование удобства использования и т. д. [ГОСТ Р 56920-2016/ISO/IEC/IEEE 29119-1:2013]

**ГОСТ Р ИСО/МЭК 25010—2015**

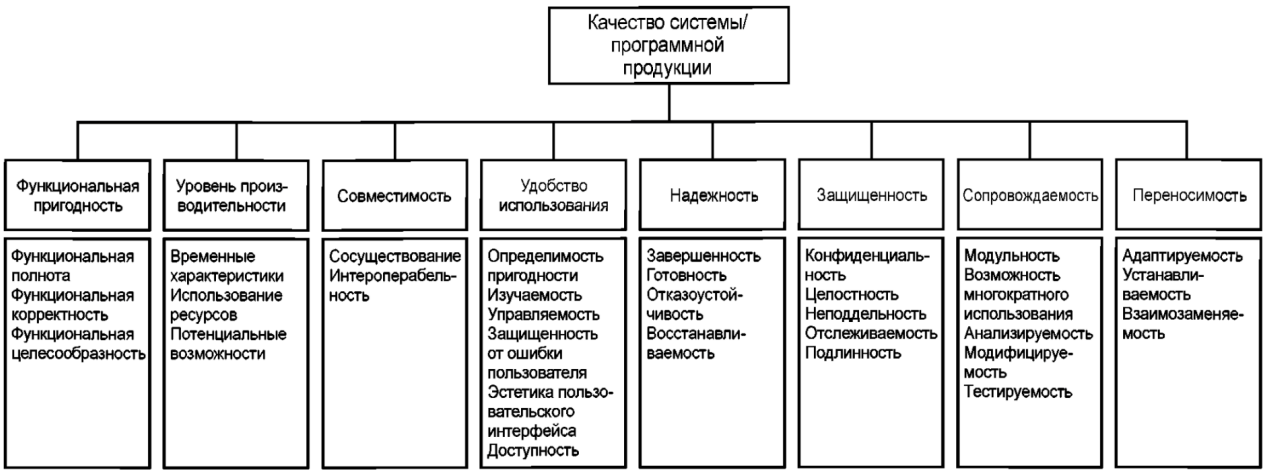



**Рисунок 4 — Модель качества продукта**

*Они будут отдельно помечены как нефункциональные. Не пугайтесь! Просто держите в голове, что относится к нефункциональному тестированию.*

### Что относят к функциональному тестированию?

Здесь все сложнее, так как нет единого стандарта, в котором четко определяются функциональные характеристики по аналогии с нефункциональными. Однако я поделюсь с вами некоторыми примерами, которые можно приводить на собеседовании:

1. Smoke, Sanity и Regression относят к функциональному тестированию, так как данные типы направлены на тестирование отдельных функций на разных стадиях разработки продукта.
2. Тестирование графического интерфейса, когда дело касается не просто внешнего вида элементов, но и их практическое применение в рамках функции приложения.
3. Тестирование безопасности, как часть функциональности самой системы, а не общая защищенность от проникновения.
4. Функциональное тестирование проводится на всех уровнях согласно ISTQB.

*Обо всех типах мы поговорим далее. Не пугайтесь! Просто держите в голове, что относится к функциональному тестированию.*

### Классификация по степени автоматизации

**Ручное тестирование (manual testing)** - тестирование, в котором тест-кейсы выполняются человеком вручную без использования средств автоматизации. [Куликов]

*Примеры: практически все, что мы освоим с вами в курсе :)*

**Автоматизированное тестирование (automated testing, test automation)** - набор техник, подходов и инструментальных средств, позволяющий исключить человека из выполнения некоторых задач в процессе тестирования. [Куликов]

*Пример: в данном случае автоматизируют проверки, используя определенный язык программирования, которые до этого написал человек*

Важно помнить, что автоматизировать все нельзя, поэтому и существует ручное тестирование. В том числе есть специалисты, которые совмещают в себе две функции (General QA, Fullstack QA, Hybrid QA).

## Классификация по принципам работы с приложением

**Позитивное тестирование (positive testing) -** исследование приложения в ситуации, когда все действия выполняются строго по инструкции без каких бы то ни было ошибок, отклонений, ввода неверных данных и т.д.

*Пример: ввод валидных значений в поле имя и пароль для успешной регистрации пользователя.*

**Негативное тестирование (negative testing, invalid testing)** — направлено на исследование работы приложения в ситуациях, когда с ним выполняются (некорректные) операции и/или используются данные, потенциально приводящие к ошибкам (классика жанра — деление на ноль).

*Пример: ввод невалидных значений в поле имя и пароль, появление сообщения об ошибке, неуспешная регистрация.*

**Деструктивное тестирование (destructive testing)** - одна из форм негативного тестирования с целью нарушить работоспособность приложения и обнаружить точку отказа.

*Пример: нагрузка приложения выше его предела, чтобы оно перестало работать.*

**В реальном тестировании следует использовать несколько правил:**

1. Любое тестирование необходимо начинать с позитивных проверок. Наша первоочередная задача убедиться, что приложение работает в стандартных условиях и готово для использования.
2. Нельзя объединять позитивные и негативные тесты, так как это затрудняет локализацию дефектов.

*Источник определений: Святослав Куликов “Тестирование ПО. Базовый курс.”*

## Классификация по природе приложения

- Тестирование веб-приложений (web-applications testing)
- Тестирование мобильных приложений (mobile-applications testing)
- Тестирование настольных приложений (desktop-applications testing)
- Тестирование игр (games-testing)

- Embedded-testing (встроенное тестирование?) - тестируется не только программное обеспечение, но и его работа на определенном аппаратном обеспечении
- Тестирование по бизнес-доменам. Например, банки, медицина, образование.

**Классификация по запуску кода на исполнение**

**Статическое тестирование (static testing)** - тестирование, которое не предполагает выполнение тестируемого компонента или системы

*Примеры: тестирование документации, прототипов, кода (в рамках code review), тестовых данных. Есть специальные техники для статического анализа.*

**Динамическое тестирование (dynamic testing)** - тестирование, проводимое во время выполнения тестируемого элемента.

*Пример: проверка реального поведения приложения при запуске кода на разных уровнях тестирования*

*Источники: ISTQB CTFL Syllabus 2018 + Святослав Куликов “Тестирование ПО. Базовый курс.”*

**Классификация по степени формализации**

**Тестирование на основе тест-кейсов (scripted testing, test case based testing)** — формализованный подход, в котором тестирование производится на основе заранее подготовленных тест-кейсов, наборов тест-кейсов и иной документации.

**Исследовательское тестирование (exploratory testing)** — частично формализованный подход, в рамках которого тестировщик выполняет работу с приложением по выбранному сценарию, который, в свою очередь, дорабатывается в процессе выполнения с целью более полного исследования приложения.

**Свободное (интуитивное) тестирование (ad hoc testing)** — полностью неформализованный подход, в котором не предполагается использования ни тест-кейсов, ни чек-листов, ни сценариев — тестировщик полностью опирается на свой профессионализм и интуицию (experience-based testing) для

спонтанного выполнения с приложением действий, которые, как он считает, могут обнаружить ошибку.

*Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”*

**Примеры от автора курса**

1. **Т. на основе тест-кейсов:** у вас есть четкие требования, вы написали тестовую документацию с конкретными шагами и проверками и приступаете к тестированию.

2. **Исследовательское Т.:** у вас есть требования или общее понимание работы продукта, вы набросали первичные проверки, которые будете дорабатывать по ходу тестирования.

3. **Свободное Т.:** у вас могут быть требования или полностью отсутствовать, вы не создаете набросок тестирования или официальную документацию, а тестируете приложение, основываясь только на своем опыте.

Частные случаи: monkey testing, gorilla testing, о которых вы можете почитать дополнительно.

# 8. Типы тестирования. Часть 2

### Классификация по целям и задачам

**Инсталляционное тестирование (installation testing, installability testing)** — тестирование, направленное на выявление дефектов, влияющих на протекание стадии инсталляции (установки) приложения.

Включает в себя следующие процессы:

1. Установка ПО
2. Удаление ПО
3. Обновление ПО
4. Откат на предыдущую версию
5. Повторный запуск установки после возникновения ошибки или исправления уже возникших проблем
6. Автоматическая установка
7. Установка отдельного компонента из общего пакета программ

*Также можно отнести к нефункциональному тестированию.*

*Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”*

**Тестирование удобства использования (usability testing)** — тестирование, направленное на исследование того, насколько конечному пользователю понятно, как работать с продуктом (understandability, learnability, operability), а также на то, насколько ему нравится использовать продукт (attractiveness).

Что нужно тестировать:

1. Общая доступность
2. Скорость, производительность
3. Удобство навигации и интерфейс
4. Плавность

Также этот тип тестирования относится к нефункциональному тестированию.

*Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”*

**Тестирование доступности (accessibility testing, A11Y)** — тестирование, направленное на исследование пригодности продукта к использованию

людьми с ограниченными возможностями (слабым зрением и т.д.).

Что можно тестировать:

1. Использование вспомогательных технологий в ПО (распознавание речи, экранная клавиатура и лупа, скринридеры)
2. Возможность использовать приложение одной рукой
3. Настройки специальной цветопередачи
4. Наличие понятных инструкций и руководства пользователя

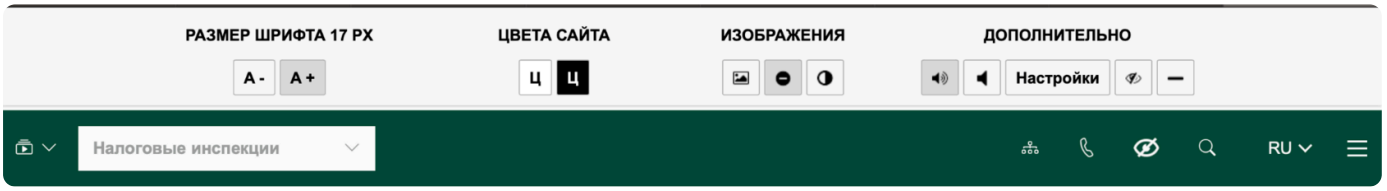
Доступность сайта можно тестировать специальными инструментами, например, WAVE, TAV, Accessibility Valet, Accessibility Developer Tools.

*Также этот тип тестирования относится к нефункциональному тестированию.*

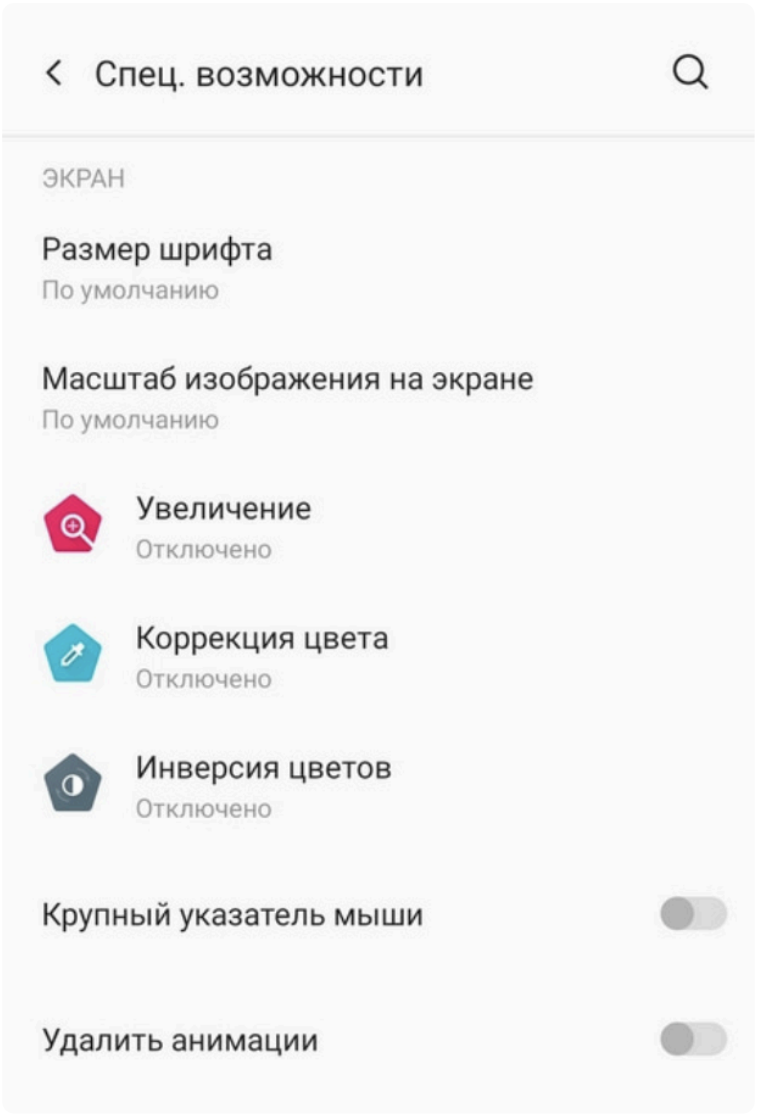
*Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”*

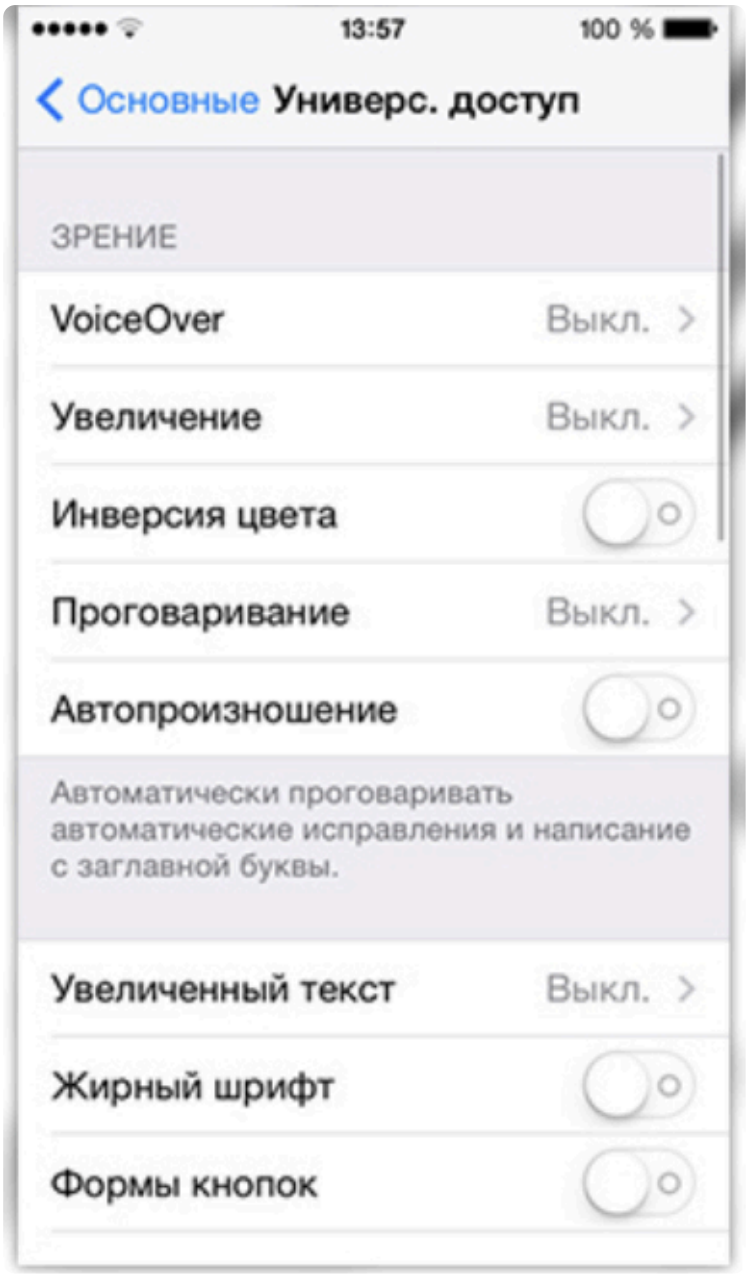
**Различные примеры того, что можно протестировать:**

- Панель для изменения размера шрифта, цветов сайта, формата изображений, звуковой навигации в веб-приложении




- Универсальный доступ и специальные возможности на различных мобильных платформах


Спец. возможности
ЭКРАН
Размер шрифта
По умолчанию
Масштаб изображения на экране
По умолчанию
Увеличение
Отключено
Коррекция цвета
Отключено
Инверсия цветов
Отключено
Крупный указатель мыши
Удалить анимации




**Тестирование безопасности (security testing)**

— тестирование, направленное на проверку способности приложения противостоять злонамеренным попыткам получения доступа к данным или функциям, права на доступ к которым у злоумышленника нет.

От чего нужно защищать ПО:

1. SQL-инъекции
2. XSS-инъекции
3. Перехват трафика
4. Брутфорсинг (полный перебор данных для получения доступа)

Т*акже этот тип тестирования относится к нефункциональному тестированию.*

*Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”*

**Дополнение от автора курса**

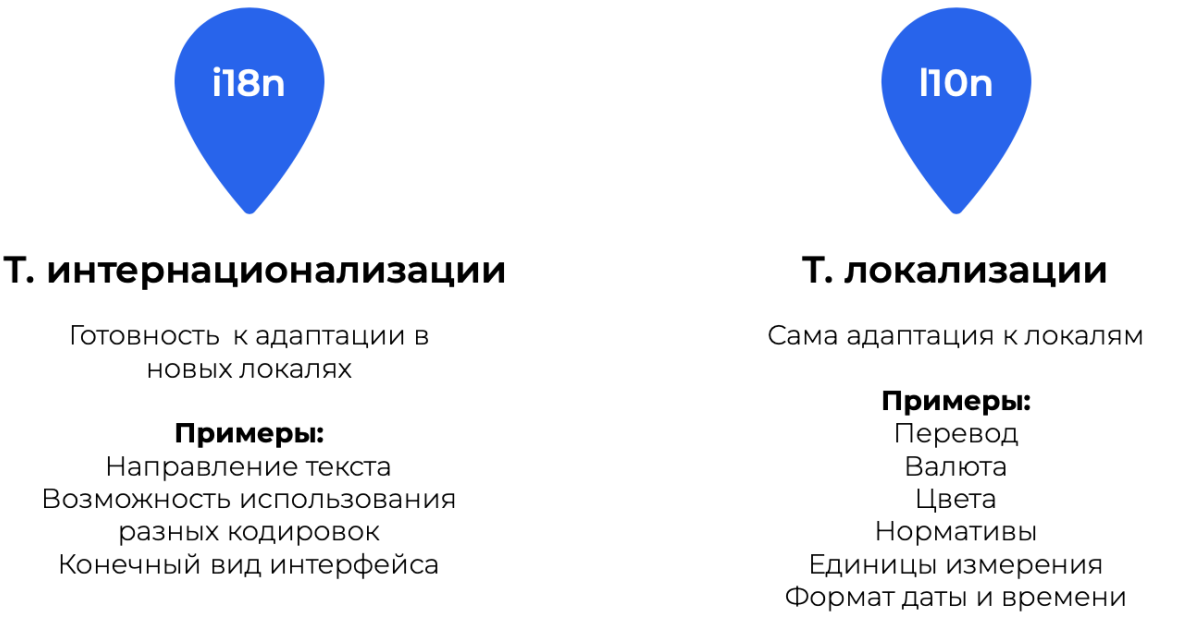
Существует OWASP TOP 10, в котором собраны самые популярные и опасные уязвимости, а также рекомендации по борьбе с ними и тестированию. Подробнее о нем можно узнать в этом видео

**Тестирование интернационализации (internationalization testing, i18n testing, globalization testing, localizability testing) —** тестирование, направленное на проверку готовности продукта к работе с использованием различных языков и с учётом различных национальных и культурных особенностей.

**Тестирование локализации (localization testing, l10n)** — тестирование, направленное на проверку корректности и качества адаптации продукта к использованию на том или ином языке с учётом национальных и культурных особенностей.

Оба типа тестирования относятся к нефункциональному тестированию.

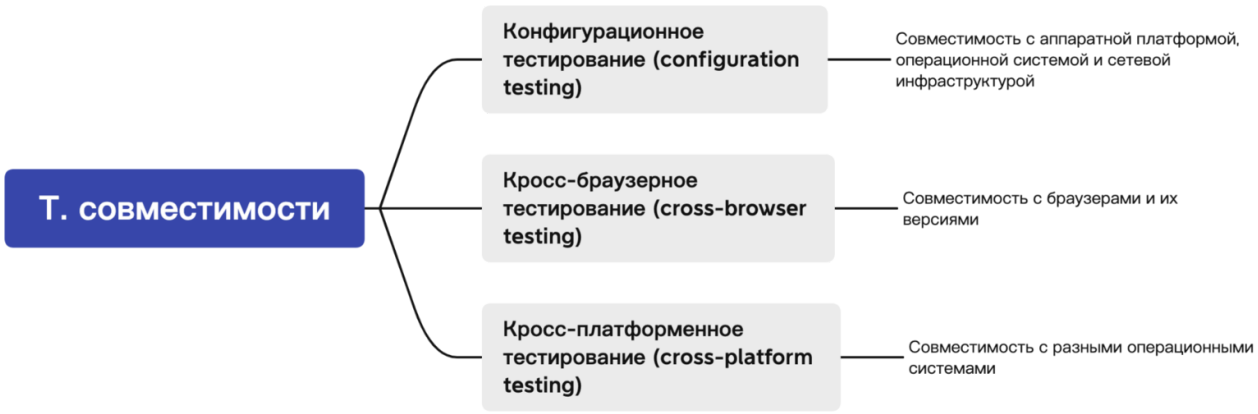
*Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”*



**Тестирование совместимости (compatibility testing, interoperability testing)** — тестирование, направленное на проверку способности приложения работать в указанном окружении.

Также относится к нефункциональному тестированию.

*Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”*




**Тестирование надёжности (reliability testing)** — тестирование способности приложения выполнять свои функции в заданных условиях на протяжении заданного времени или заданного количества операций.

**Тестирование восстанавливаемости (recoverability testing)** — тестирование способности приложения восстанавливать свои функции и заданный уровень производительности, а также восстанавливать данные в случае возникновения критической ситуации, приводящей к временной (частичной) утрате работоспособности приложения.

**Тестирование отказоустойчивости (failover testing)** — тестирование, заключающееся в эмуляции или реальном создании критических ситуаций с целью проверки способности приложения задействовать соответствующие механизмы, предотвращающие нарушение работоспособности, производительности и повреждения данных.

Также относятся к нефункциональному тестированию.

*Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”*

**Тестирование производительности (performance testing)** — исследование показателей скорости реакции приложения на внешние воздействия при

различной по характеру и интенсивности нагрузке.

Подвиды:

1. Нагрузочное тестирование (load testing, capacity testing)
2. Тестирование масштабируемости (scalability testing)
3. Объёмное тестирование (volume testing)
4. Стрессовое тестирование (stress testing)
5. Конкурентное тестирование (concurrency testing)

**Нагрузочное тестирование** — исследование способности приложения сохранять заданные показатели качества при нагрузке в допустимых пределах и некотором превышении этих пределов (определение «запаса прочности»).

**Тестирование масштабируемости** — исследование способности приложения увеличивать показатели производительности в соответствии с увеличением количества доступных приложению ресурсов.

**Объёмное тестирование** — исследование производительности приложения при обработке различных (как правило, больших) объёмов данных.

**Стрессовое тестирование** — исследование поведения приложения при нештатных изменениях нагрузки, значительно превышающих расчётный уровень, или в ситуациях недоступности значительной части необходимых приложению ресурсов.

**Конкурентное тестирование** — исследование поведения приложения в ситуации, когда ему приходится обрабатывать большое количество одновременно поступающих запросов, что вызывает конкуренцию между запросами за ресурсы (базу данных, память, канал передачи данных, дисковую подсистему и т.д.).

*Все описанные виды тестирования также относятся к нефункциональным.*

*Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”*

**Примеры для различных типов тестирования производительности от автора курса**

### Двоякая природа тестирования безопасности и производительности

Существует трактовка, по которой к функциональному тестированию можно отнести тестирование безопасности и тестирование производительности, но только в тех случаях, когда они представляют основную функциональность приложения, а не просто являются дополнительными характеристиками.

Например, защита денежных транзакций, функции антивируса, возможность одновременной работы над задачей несколькими пользователями.

**Использовать с осторожностью и только, если спросят про такие кейсы. В стандартах это не упоминается.**

### Реальная жизнь и советы

1. Не нужно знать абсолютно все типы тестирования. Чаще всего на интервью вас спросят: какие виды тестирования вы знаете? Советую начинать с нефункционального и функционального тестирования, а дальше переходить к более мелким видам, постепенно раскрывая матрешку
2. При выходе на проект начинайте всегда с исследовательского тестирования для того, чтобы определить будущую стратегию тестирования

3. На практике используется ограниченное количество видов тестирования. Особый упор при запоминании сделайте на тестирование, связанное с изменениями, Smoke, тестирование по важности функций, методы тестирования
4. Большая часть вашей работы будет крутиться вокруг регрессионного и функционального тестирования

**Полезные материалы**

1. Презентация
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/types
3. Старые версии уроков
4. https://youtu.be/S1pCvIPag_c
5. https://youtu.be/H158MGHtWKg
6. Схемы из старых версий уроков https://drive.google.com/drive/u/3/folders/1bTpTj05t_Ul9OHV3WRoHfwSDdUsKylIG
7. Библия QA. Функциональное тестирование (Functional/Behavioral testing) https://vladislaveremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/funkcionalnoe-testirovanie-functional-behavioral-testing
8. Библия QA. Нефункциональное тестирование (Non-Functional testing) https://vladislaveremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/nefunkcionalnoe-testirovanie-non-functional-testing
9. Ответ на часто задаваемый вопрос на интервью: можно ли отнести тестирование безопасности или нагрузочное тестирование к функциональным видам тестирования? https://vladislaveremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/mozhno-li-otnesti-testirovanie-bezopasnosti-ili-nagruzochnoe-testirovanie-k-funkcionalnym-vidam-test

# 9. Scrum

**Scrum** – это **фреймворк (набор правил)**, который помогает создавать качественный продукт в условиях неопределённости. Он не даёт готовых инструкций, а задаёт рамки, внутри которых команда сама выбирает, как работать.

Чтобы Scrum работал, важно не только следовать его событиям и ролям, но и **жить по его ценностям**. Ценности напрямую влияют на атмосферу в команде и эффективность работы.

Вот пять ключевых ценностей Scrum:

- **Приверженность** – каждый берёт ответственность за общий результат. Команда нацелена не просто на выполнение задач, а на достижение целей.
- **Сфокусированность** – команда сосредоточена на том, что важно в текущем спринте. Никаких распылений на лишнее.
- **Открытость** – проблемы не замалчиваются, прогресс обсуждается честно. Это делает Scrum прозрачным.
- **Уважение** – каждый участник команды ценен. Нет «важных» и «второстепенных» ролей.
- **Смелость** – команда не боится изменений, сложных решений, честной обратной связи и ответственности.

Если в команде соблюдаются эти ценности, Scrum начинает работать по-настоящему. Без них это просто набор встреч и терминов.

*В конце данного урока я собрал самое важное на примере реального рабочего процесса в рамках одной итерации, поэтому не пугайтесь, если шаги до этого вам будут не совсем понятны.*

*В рамках данного текстового модуля я упростил некоторые вещи, так как официальный Scrum Guide может быть сложен для восприятия новичкам.*

**Роли Scrum**

Scrum часто звучит на собеседованиях и в рабочей практике, но многие новички путаются в ролях и структуре. Давайте разложим это простыми словами.

**Scrum Team** – это одна компактная команда (до 10 человек), в которой нет начальников, отделов и подчинённых. Это группа профессионалов, работающая как единое целое. Внутри команды три роли: **Scrum Master**, **Product Owner** и **Developers** (разработчики и все, кто создаёт продукт).

Команда должна быть **самостоятельной** (сама решает, как и что делать) и **кросс-функциональной** (у всех вместе есть нужные навыки, чтобы создать готовый продукт в течение одного спринта).

Scrum Team делает всё: анализирует задачи, общается со стейкхолдерами, пишет код, тестирует, выпускает и дорабатывает продукт. И всё это – ради одной общей цели: достичь **Product Goal**.

**Developers** в Scrum – это не только программисты. Это все специалисты в команде, которые создают продукт: аналитики, дизайнеры, тестировщики, DevOps и т.д. Их основная задача – выдать **готовый функционал** к концу каждого спринта.

Важно, что Developers:

- сами планируют, что сделают в спринте (это называется **Sprint Backlog**),
- следят за качеством работы и тем, чтобы фича соответствовала **Definition of Done** (определению готовности),
- каждый день адаптируют свои действия под цели спринта,
- несут ответственность друг перед другом, а не перед “начальником”.

**Product Owner (PO, владелец продукта)** – это человек, который отвечает за то, **что именно** мы делаем и **почему**. Он не управляет командой, а формирует и объясняет **цели продукта** и **приоритеты задач**.

PO – связующее звено между бизнесом и командой. Он ведёт **Product Backlog** (список задач и требований), делает его понятным, логичным и доступным. В идеале, PO должен каждый день быть на связи с командой и бизнесом.

**Scrum Master (SM)** — это не менеджер, не начальник и не руководитель. Его задача – помогать команде правильно применять Scrum и становиться сильнее.

Scrum Master:

- помогает команде быть **самостоятельной и универсальной**,
- устраняет препятствия, которые мешают команде двигаться,
- обучает Scrum – и внутри команды, и в компании в целом,
- помогает наладить **прозрачное взаимодействие** со всеми, кто заинтересован в продукте.

Хороший Scrum Master – это не тот, кто “контролирует”, а тот, кто **делает команду сильнее.**

## События в Scrum

Scrum предлагает фиксированный набор событий, который помогает команде двигаться к цели, сохраняя прозрачность и возможность адаптации. Эти события не просто «митинги ради митингов» — каждое из них имеет практический смысл и влияет на результат работы.

**Sprint** – это основная единица времени в Scrum, в рамках которой команда создает ценность. Всё, что делает команда — планирование, работа, проверки, обсуждение улучшений – происходит внутри Sprints. Как правило, продолжительность одного спринта не превышает месяца, а часто – 1–2 недели. Это позволяет быстро получать обратную связь и корректировать курс.

**Sprint Planning** – важное событие, где команда определяет, зачем она работает этот спринт (Sprint Goal), что берёт в работу и как будет реализовывать задачи. Здесь важно участие всех – Product Owner приносит цели и приоритеты, Developers – оценивают и планируют реализацию. Результатом становится Sprint Backlog – план на ближайшие дни.

**Daily Scrum** — короткая встреча каждый день. Её цель — синхронизация. Это не отчёт перед Scrum Master, а разговор внутри команды: где мы, что мешает, что нужно скорректировать. Даже если вы работаете удалённо, такие встречи помогают держать фокус.

Еще это мероприятие могут называть stand-up, так как раньше на нем все стояли. Обычно в рамках дейлика обсуждается три вопроса: что мы сделали вчера, что мы сделаем сегодня, какие есть блокирующие факторы для выполнения задачи.

**Sprint Review** — в конце спринта команда показывает то, что получилось. Это не формальная презентация, а рабочий разговор с заказчиком или заинтересованными сторонами. Обсуждаются итоги, собранные инсайты, планы на будущее. На основе этого Product Backlog может быть изменён.

**Sprint Retrospective** – закрытие спринта и момент для роста. Это не “разбор полётов”, а конструктивное обсуждение: что пошло хорошо, где мы буксовали, что можно улучшить. Это помогает не повторять одни и те же ошибки и постоянно повышать эффективность команды.

Важно: все события ограничены по времени. Это не значит, что вы обязаны тратить весь лимит – но он служит защитой от бесконечных обсуждений.

Используйте эти события осознанно – это опора для команды, а не бюрократия.

## Артефакты в Scrum

Scrum предлагает три ключевых артефакта – это не просто термины, а реальные инструменты, которые помогают команде понимать, **что делать**, **зачем делать** и **что уже сделано**. Разберём каждый на практике:

**Product Backlog** это главный список всех задач, которые нужно выполнить, чтобы улучшить продукт. Его ведёт Product Owner, и только он определяет, что туда попадает. Этот список никогда не бывает “закончен” – он постоянно уточняется и пересматривается. Например: сначала в Backlog может быть написано “добавить регистрацию”, а потом команда уточняет – нужна форма, отправка email, проверка пароля и т.д. Эти уточнения называются **refinement**.

**Sprint Backlog** – это отфильтрованный список из Product Backlog, который команда берёт в работу на текущий Sprint. Плюс к нему добавляется цель (Sprint Goal) и план, как эта работа будет выполняться. Это как “план на неделю” для разработчиков. Он может меняться в процессе работы, если появляются новые детали. Главное – команда сама решает, как работать, и отслеживает свой прогресс.

**Increment** – это то, что получилось в конце Sprint и что можно показать пользователю. Например: если команда реализовала новый фильтр по дате в приложении – это и есть Increment. Он должен быть полностью готов, протестирован и потенциально пригоден для релиза. Даже если релиз не запланирован – Increment всегда должен быть в рабочем состоянии. Это поддерживает доверие и прозрачность внутри команды и с заказчиками.

**Оценка (estimation)** – это способ прикинуть, **насколько трудоёмкой будет задача**, прежде чем браться за неё. Это не точное предсказание, а скорее ориентир: “эта задача в 3 раза сложнее, чем вот та”.

Чаще всего Scrum-команды используют **стори поинты** (story points) – абстрактные единицы, не привязанные напрямую ко времени. Один и тот же объём работы может занять у разных команд разное количество часов, но в поинтах он будет оцениваться одинаково.

Простой пример:

- Регистрация с валидацией и письмом – 5 поинтов
- Добавление новой кнопки на уже готовый экран – 1 поинт
- Разработка фильтра с сохранением состояния – 8 поинтов

💡 Самое важное – **не путать стори поинты с часами**. Они нужны, чтобы понять **относительную сложность задач**, а не сказать, сколько часов уйдёт.

Для оценки часто используется техника **покер-планирования**:

Каждый участник команды выбирает число по Фибоначчи (1, 2, 3, 5, 8, 13 и т.д.), которое по его мнению отражает сложность задачи. После чего все одновременно “открывают карты” и обсуждают различия в оценках. Это помогает избежать давления, быстро прийти к консенсусу и услышать разные мнения.

Также стоит помнить:

- Оценка проводится **до начала работы** – чаще всего на груминге (refinement).
- **Сама команда** делает оценку – никто не навязывает цифры извне.

- Оценка помогает Владельцу Продукта (Product Owner) **понимать, что реально можно успеть в спринте**, а команде – отслеживать свою **производительность**.

👉 Не зацикливайтесь на цифрах — задача оценки в том, чтобы создать общее понимание внутри команды, а не выдать точный прогноз в часах.

**Velocity (Скорость)** – это показатель того, **сколько работы команда реально выполняет за один спринт**. Обычно измеряется в **стори-поинтах**, и считается просто:

Если за спринт команда завершила 5 задач с оценками 3, 5, 2, 2 и 8 – её velocity будет **20**.

Этот показатель помогает:

– Владельцу продукта понять, сколько примерно задач можно запланировать на будущие спринты.

– Команде отслеживать свой стабильный темп.

– Не перегружать себя в следующем спринте.

Важно: **Velocity не сравнивают между командами**. У каждой может быть своя шкала сложности. У кого-то “5” — это простая задача, у кого-то — почти невыполнимая.

**Capacity (Вместимость)** – это то, **сколько команда планирует взять в спринте**. Это тоже выражается в стори-поинтах, но в отличие от velocity, это **оценка вперёд**, а не результат.

Capacity зависит от:

– количества доступных рабочих дней;

– отпусков, больничных и других отвлекающих факторов;

– состава команды в конкретный спринт.

Например, если обычно команда делает 20 поинтов, но в этот раз один человек в отпуске — можно запланировать 16–18, и это будет нормальный capacity.

📌 Velocity — **факт**, измеряется **в конце**.

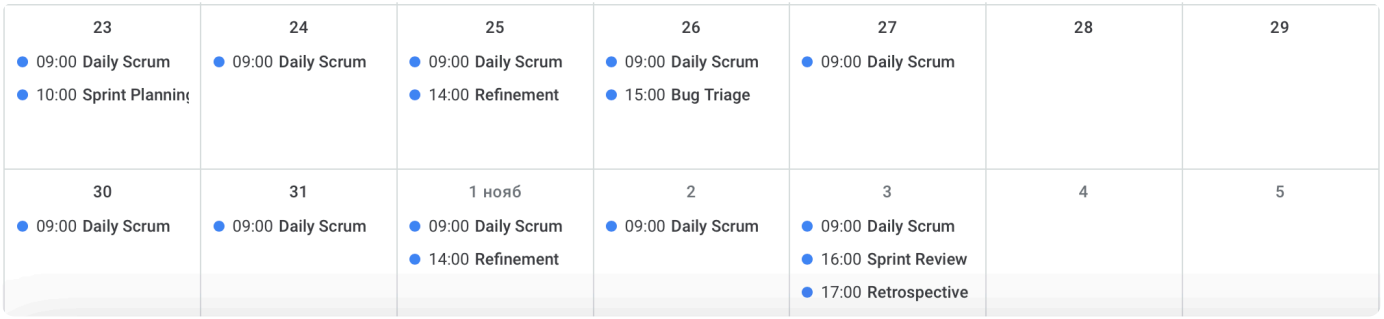
📌 Capacity — **прогноз**, оценивается **в начале**.

Они вместе помогают планировать спринт реалистично и стабильно.

Помните, что для оценки этих двух показателей недостаточно поработать с командной одну итерацию, важно, чтобы эти оценки строились на исторических данных, собранных в течение нескольких спринтов.

**Как это устроено на самом деле?**

Примерно так выглядит календарь любого участника команды разработки в течение одного спринта, если команда работает по Scrum. В нашем случае спринт будет равен две недели.




Давайте поговорим про периодичность событий и что на них происходит.

Я не буду сосредотачиваться на глубокой характеристики каждого мероприятия, так как это было обсуждено в шагах ранее, лишь упрощу формулировки и механики проведения для новичков.

1. **Daily Scrum (дейлик, стендап)** - каждый день команда собирается до 15 минут, чтобы обсудить текущий статус по задачам, преграды для их выполнения и запланировать работу на ближайший день. Можно провести аналогию с планерками.

2. **Sprint Planning (планирование спринта)** обычно происходит в самом начале итерации, некоторые команды проводят его в последний день прошедшей итерации. На этом мероприятии команда приоритезирует задачи в бэклоге спринта (то, что нужно сделать в ближайшие две недели), определяет очередность их выполнения, может назначить ответственных. Например, более сложные задачи, которые требуют больше времени стараются брать в начале спринта.

3. **Backlog Refinement (уточнение бэклога, груминг)** может проходить как каждую неделю, так и раз в две недели. На данном мероприятии Product Owner презентует новые пользовательские истории из бэклога продукта (все задачи, которые нужно реализовать в продукте), а команда задает вопросы и уточняет требование. Часто на этом мероприятии проходит оценка трудозатрат и пользовательским историям присваивают story points. Для тестировщика важно задавать как можно больше вопросов, так как это является одним из этапов анализа требований и отличной возможности прояснить требования для всей команды.

После уточнения задачи из бэклога продукта попадают в бэклог будущих спринтов. В хорошей команде уточненных задач должно быть на несколько спринтов вперед, иначе есть большая вероятность простоев.

4. **Bug Triage (сортировка багов)** не является событием Scrum, однако может использоваться командой для приоритизации дефектов, которые долго находятся в бэклоге и не были исправлены или так или иначе зависли. Нерегулярное мероприятие.

5. **Sprint Review** проводится в конце итерации и служит для предоставления отчета по пройденной итерации всем заинтересованным лицам: команде, менеджменту, заказчику. Для этого собираются определенные метрики и чаще всего презентуются скрам-мастером.

6. **Sprint Retrospective** проводится в конце итерации и служит для сбора обратной связи от скрам-команды.

Стандартный формат проведения: доска с тремя столбцами (что было хорошо в итерации, что требует улучшения, план действий).

Каждый член команды добавляет стикеры в первые два столбца, а после в ходе обсуждения из них формируется план действий и самые важные задачи попадают в бэклог следующего спринта. Не игнорируйте это мероприятие и активно участвуйте в обсуждениях. Это один из способов улучшить существующий процесс.

7. Дополнительно следует отметить такое мероприятие как **демонстрация**

**рабочего продукта**, где любой член команды разработки презентует разработанную в рамках спринта функциональность заинтересованным лицам. Обычно демо-сессии проводятся в конце итерации.

Это далеко не исчерпывающий список всех митингов, но его будет достаточно для вашего понимания и комфортного вливания в первые рабочие дни на проекте.

### Дополнительные термины, которые вы сможете встретить на работе

**Минимально Жизнеспособный Продукт (Minimum Viable Product, MVP)** - версия продукта, позволяющая команде с минимальными затратами собрать максимум информации о клиентах и обратной связи от них.

**Заинтересованное лицо [Стейкхолдер] (Stakeholder)** - лицо, дающее обратную связь Владельцу Продукта и Скрам-команде в целом по видению, Бэклогу Продукта и Инкрементам. Нередко участвует в Обзоре спринта. Зачастую является частью организации, которая разрабатывает продукт.

**Доска Спринта [Скрам-доска] (Sprint Board)** - инструмент, помогающий визуализировать Бэклог Спринта на протяжении Спринта. Доска Спринта (часто называемая Скрам-доской) может быть организована различными способами, например, с помощью онлайн-сервисов или как физическая доска с тремя колонками. Колонки могут называться «Сделать», «В работе», «Готово» или иметь аналогичные названия.

Источник

### Полезные материалы

1. Презентация https://drive.google.com/file/d/1gRTegKyfY8U6sSJp6lGvNj_9kfHSOT80/view?usp=share_link
2. *Шпаргалка (адаптирована под десктоп)* https://rusau.net/scrum
3. Старая версия https://youtu.be/bv4L_32J-5A
4. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1oEuvV0_EGUx_VaqWTPXNbuhHFwkl8wuw
5. Инструмент для покер-планирования https://www.planitpoker.com/
6. Agile-манифест https://agilemanifesto.org/
7. Scrum Guide https://www.scrumguides.org/

8. Kanban Guide https://www.scrum.org/resources/kanban-guide-scrum-teams
9. Библия QA. Agile https://vladislaveremeev.gitbook.io/qa_bible/sdlc-i-stlc/agile
10. Библия QA. Scrum https://vladislaveremeev.gitbook.io/qa_bible/sdlc-i-stlc/scrum

# 10. ДОП: Kanban и XP

*Примечание автора: изучение данной темы является дополнительной. Не советую делать ее высокоприоритетной в обучении, а запомнить только основные моменты.*

**Канбан** — это способ организации работы, который помогает **не перегружать команду задачами**, а **поддерживать стабильный поток выполнения**.

Главная идея Канбана – **визуализировать процесс** (например, на доске), **ограничить количество задач в работе (WiP)** и **улучшать поток**.

Например, у вас есть доска с колонками:

To Do | In Progress (WiP=3) | Done

Правило WiP (Work in Progress) говорит: в колонке “In Progress” одновременно может находиться **не больше 3 задач**. Это помогает:

– не хвататься за всё сразу;

– завершать начатое;

– находить узкие места (например, если всё застряло в тестировании).

Когда задача из “In Progress” переходит в “Done”, появляется свободное место – **можно “вытянуть” следующую задачу** из списка “To Do”. Это и есть **вытягивающая система** – задачи не “пихаются” в процесс, а поступают, когда на них реально есть ресурсы.

📌 Канбан помогает **навести порядок, сфокусироваться и стабильно двигаться вперёд** — особенно в задачах, которые сложно распланировать заранее.

**Поточные системы**

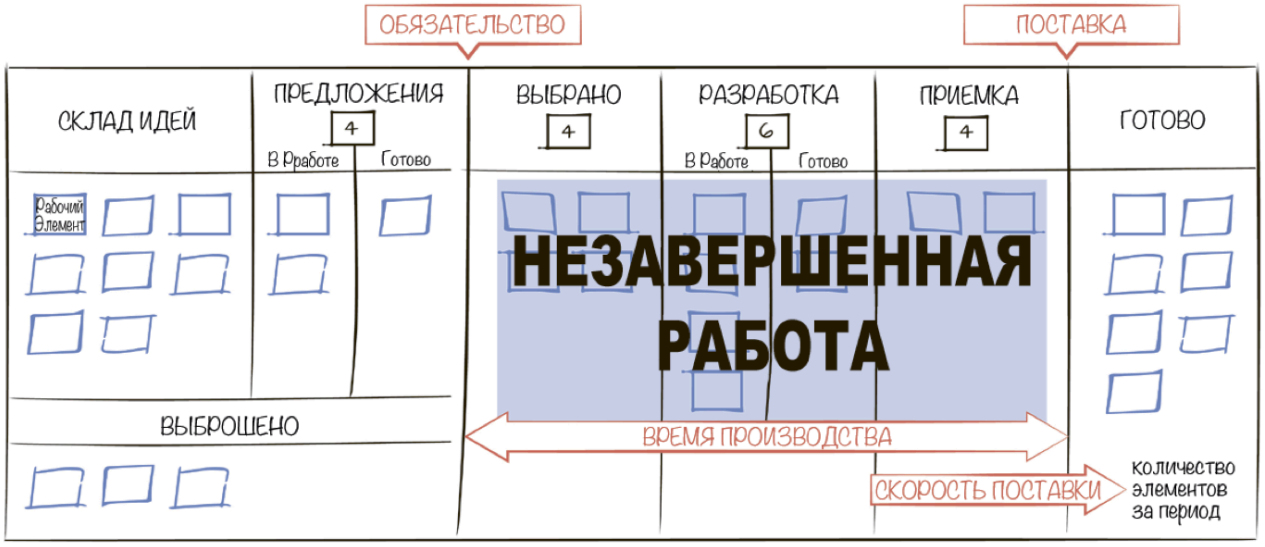



*Рис. 4. Пример канбан-доски*

В Канбане есть несколько ключевых метрик, которые помогают **оценивать и улучшать работу команды**.

**1. Время производства (Lead Time)** – это сколько времени задача находится в процессе с момента, когда **команда взяла на себя обязательство**, и до момента **реальной поставки** результата. Например, как долго баг находился в работе до его исправления и деплоя.

**2. Время ожидания заказчика (Customer Lead Time)** – это сколько времени **ждёт клиент**, начиная с момента, когда он отправил запрос. Это время может быть больше, чем Lead Time, если задача долго ждала в очереди.

**3. Отложенные обязательства** – это когда запрос **ещё не взят в работу**. Он есть, но команда пока **не пообещала его выполнять**. Разделение этих этапов помогает управлять ожиданиями и приоритетами.

**4. WIP (Work in Progress)** – это количество задач, которые **одновременно находятся в процессе**. Например, баги, над которыми кто-то работает прямо сейчас. Контроль WIP помогает **не перегружать команду**.

**5. Скорость поставки (Delivery Rate)** – это насколько быстро команда **реально завершает задачи**. Например, 10 задач в неделю. Это помогает прогнозировать сроки и понимать эффективность команды.

Канбан помогает **не только отслеживать задачи**, но и видеть всю систему работы как **сервис**. У каждой команды есть свой “заказчик” — например,

разработка работает для бизнеса, поддержка — для клиентов. Цель Канбана — **делать работу стабильной, прозрачной и предсказуемой**.

## Ценности Канбана

## (для общего развития, запоминать не нужно)

**1. Прозрачность** – все участники видят, что происходит. Работа и процессы отображаются на доске, чтобы каждый понимал текущее состояние задач.

**2. Баланс** – важно находить баланс между срочностью, качеством, скоростью и возможностями команды. Нельзя перегружать участников.

**3. Сотрудничество** – успешный Канбан невозможен без постоянного взаимодействия между членами команды, заказчиками и другими участниками процесса.

**4. Клиентоориентированность** – главная цель команды — **решать задачи клиента** и приносить ему ценность. Канбан помогает фокусироваться на нуждах заказчика.

**5. Поток** – стабильный и непрерывный поток задач от начала до завершения. Важна предсказуемость: сколько задач мы можем завершить за день, неделю, спринт.

**6. Лидерство** – каждый может проявлять инициативу и вносить улучшения. Лидерство в Канбане распределено, а не централизовано.

**7. Понимание** – команда должна понимать, **как устроены процессы**, почему они такие, и как можно их улучшить.

**8. Согласие** – изменения внедряются, **когда команда согласна** с ними. Это снижает сопротивление и повышает вовлечённость.

**9. Уважение** – каждый член команды ценен. В Канбане уважают опыт, мнение и ритм работы каждого.

## Каденции

Вот краткая таблица по **7 каденциям Канбана** – регулярно проводимым встречам для получения обратной связи и улучшения процессов:

| Каденция | Цель встречи |
| --- | --- |
| **Ревью стратегии** | Определение приоритетов, выбор сервисов, анализ внешней среды |
| **Операционное ревью** | Балансировка всех сервисов, выявление узких мест, анализ потоков |
| **Ревью рисков** | Обнаружение и анализ рисков (например, блокеров), влияющих на поставку |
| **Ревью сервиса поставки** | Улучшение конкретного сервиса, анализ эффективности и метрик |
| **Собрание по пополнению** | Принятие решений о включении новых задач в поток работы (вход в систему) |
| **Канбан-митинг** | Ежедневная быстрая встреча для синхронизации, устранения препятствий |
| **Собрание планирования поставки** | Планирование и контроль над своевременной поставкой задач клиенту |

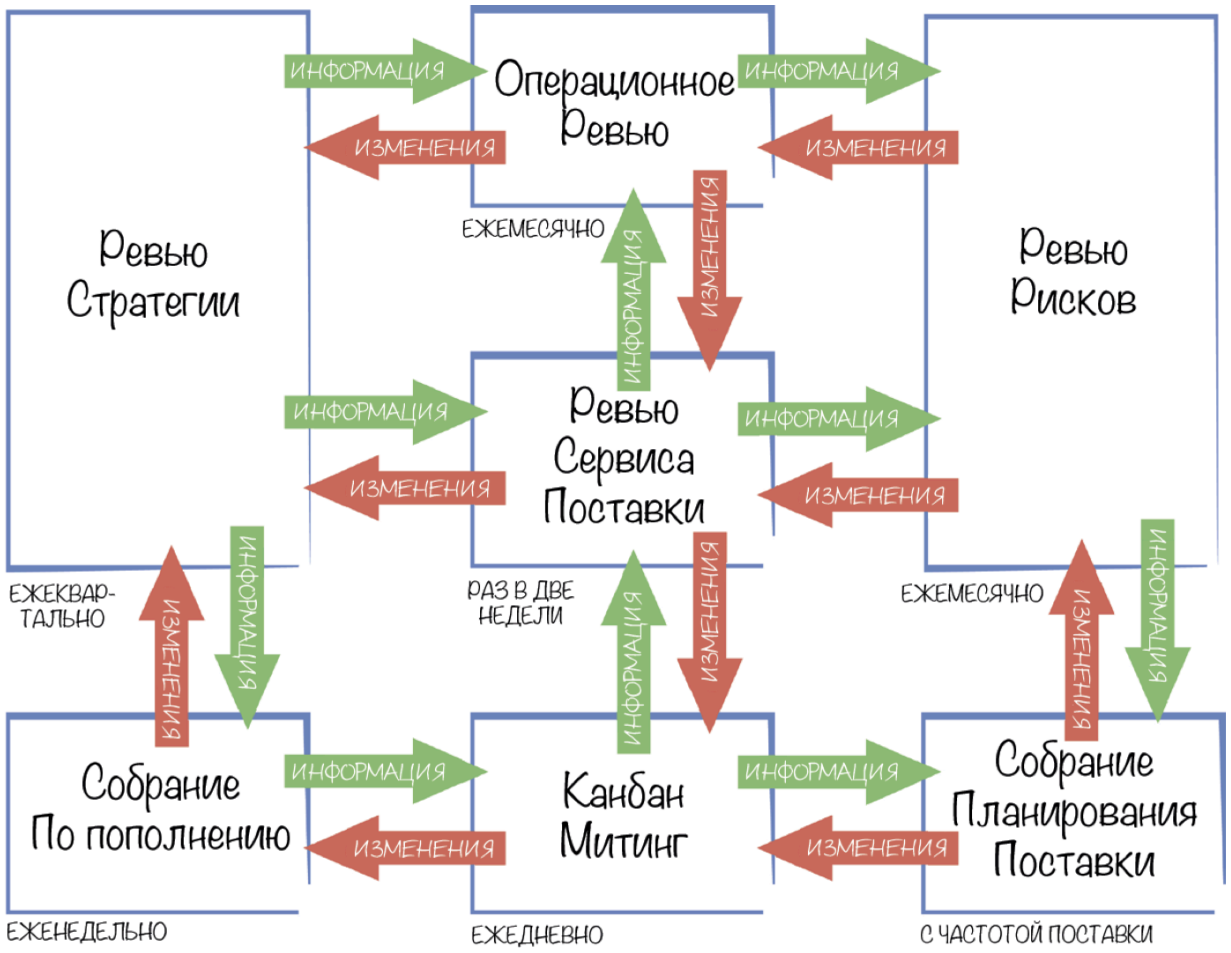



*Рис. 10. Каденции, демонстрирующие циклы обратной связи*

Все эти встречи помогают **визуализировать проблемы, обсудить прогресс, устранить блокеры и наметить улучшения**.

Удобно использовать их как регулярный ритм — раз в день, неделю или месяц, в зависимости от типа каденции.

### Роли в Канбан

В Канбане нет строгих ролей, как в Scrum, но со временем сформировались два типичных типа ответственности: за работу с заказчиками и за организацию процесса поставки.

Если вы только начинаете знакомство с Канбаном, то вам будет проще представить это так:

– Первый человек отвечает за то, **что мы делаем и зачем это нужно**. Он общается с заказчиком, уточняет ожидания, помогает выбирать и расставлять

приоритеты. Это и есть **Менеджер Запросов (SRM)**. В других подходах его могут называть Product Owner или менеджер продукта.

– Второй человек отвечает за то, **как именно мы будем выполнять работу** и **как сделать процесс стабильным и понятным для команды**. Он следит за потоком задач, помогает команде договориться, какие задачи брать в работу, и следит, чтобы ничто не мешало выполнению. Это **Менеджер Поставки (SDM)**. Эту роль можно сравнить со Scrum Master или тимлидом в других командах.

Эти роли могут совмещаться в одном человеке или распределяться гибко – это нормально для Канбана. Важно понимать суть: кто-то помогает правильно выбрать задачи, а кто-то следит, чтобы они доходили до результата.

## Экстремальное программирование (XP)

Если Scrum и Kanban можно применять даже за пределами ИТ – например, в управлении задачами или логистике, — то экстремальное программирование (XP) работает **только в разработке программного обеспечения**.

Эта методология появилась как ответ на постоянные изменения требований, когда нужно быстро адаптироваться, но при этом не потерять в качестве.

В XP всё крутится вокруг **практик для разработчиков** — как они пишут код, как тестируют, как обсуждают архитектуру и работают вместе. Поэтому, если вы тестировщик, работающий в команде с XP, вы, скорее всего, будете вовлечены в автоматизацию, тесное взаимодействие с разработчиками и короткие циклы обратной связи.

Совет: обратите внимание на ценности XP – **простота, обратная связь, смелость, уважение и коммуникация**. Они пригодятся и за пределами XP, особенно в кросс-функциональных командах.

Вот 13 ключевых практик экстремального программирования (XP). Они могут звучать «разработчески», но знание принципов поможет тестировщику правильно встроиться в процесс и понимать, что происходит в команде.

### 1. Вся команда

Все участники проекта (разработчики, тестировщики, заказчики) вовлечены и работают как единое целое. Решения принимаются совместно. Это значит, что и вы как тестировщик не просто “выполняете”, а участвуете в обсуждении требований и решений.

**2. Игра в планирование**

Совместное обсуждение задач, оценок и сроков. Часто – в формате “планирования по карточкам”. Это не Waterfall, где вам спускают план сверху. Вы вместе определяете приоритеты и берёте задачи в работу.

**3. Маленькие релизы**

Вместо одного большого релиза в конце – частые и небольшие. Это упрощает поиск багов и делает обратную связь быстрой. Вам, как тестировщику, придётся часто проверять новые версии – это нормально.

**4. Метафора**

Общее видение архитектуры или устройства системы, выраженное простыми словами. Например: «наш продукт как конструктор LEGO». Это помогает всей команде говорить на одном языке.

**5. Простой дизайн**

Делается только то, что нужно прямо сейчас – без избыточных решений «на будущее». Это снижает количество багов и делает систему более понятной и удобной для тестирования.

**6. Разработка через тестирование (TDD)**

Код сначала пишется с учётом тестов. Сначала пишется тест, который не проходит, потом код – пока тест не пройдёт. Это снижает количество багов и делает автоматизацию обязательной частью процесса.

**7. Рефакторинг**

Постоянное улучшение кода без изменения его поведения. Ваша задача – убедиться, что после рефакторинга всё осталось стабильным, а тесты (в том числе регрессионные) – зелёные.

**8. Парное программирование**

Два разработчика работают за одним компьютером. Один пишет, другой подсказывает, проверяет, думает. Тестировщик может быть вовлечён на ранних этапах, чтобы задать вопросы и предотвратить баги ещё до кода.

**9. Коллективная собственность кода**

Любой разработчик может изменить любой участок кода. Это увеличивает ответственность команды в целом и означает, что любые изменения нужно будет быстро протестировать.

**10. Стандарт кодирования**

Все пишут код по единым правилам (написание имён, структура файлов и т.п.). Это помогает легче находить ошибки и понимать логику приложения.

**11. Устойчивый темп (Sustainable pace)**

Работа без переработок. Продуктивность важнее часов. Если в команде применяется XP, значит, дедлайны не должны давить, а качество – не страдать.

**12. Пользовательское тестирование**

Регулярная обратная связь от конечных пользователей. Это не “один раз перед релизом”, а постоянный процесс. Задача тестировщика – помогать получать и анализировать такую обратную связь.

**13. Непрерывная интеграция**

Каждый день (а лучше – несколько раз в день) код коммитится и собирается в одну общую ветку. Это позволяет быстрее выявлять конфликты и баги. От тестировщика часто требуется быстрая проверка, автоматизация или smoke.

Эти практики не требуют, чтобы вы их применяли все сразу. Но если вы понимаете, **зачем они нужны**, и можете говорить о них на интервью — вы уже выделяетесь.

**Полезные материалы**

1. Шпаргалка (адаптирована под десктоп) https://rusau.net/kanban

ТЕОРИЯ ТЕСТИРОВАНИЯ (СИНХРОНИЗИРОВАНА С ISTQB)

# 11. Работа с требованиями

### Источники и пути выявления требований

**Требование (requirement)** - условие, которое включает обязательные для выполнения критерии *[ISTQB Glossary]*

Перед тем как к команде разработки попадают финальные требования их необходимо собрать у заказчика. Для этого используют ряд техник на рисунке ниже.

Чаще всего этим занимаются бизнес-аналитики или владельцы продукта, поэтому в курсе мы не будем акцентировать на этом пристальное внимание.

Рисунок 2.2.d — Основные техники сбора и выявления требований

Подробнее о них можно почитать здесь

### Уровни и типы требований

Требования — это основа, на которой строится всё тестирование. И чем лучше вы понимаете, **что именно нужно системе и зачем**, тем проще будет планировать тесты, писать кейсы и находить баги.

Чтобы не запутаться, начнём с самых нужных уровней требований, которые часто встречаются на практике и могут быть полезны на собеседовании:

- **Бизнес-требования** — это про **зачем**. Здесь не описывают кнопки и экраны. Это уровень заказчика: какую проблему решает продукт и какую пользу он должен приносить. Примеры: «повысить количество заявок», «показывать актуальный курс валют».
- **Пользовательские требования** — это про **что может делать пользователь**. Обычно они оформляются в виде user stories *(о них позже в уроке)*, сценариев или просто списков действий. Примеры: «пользователь должен видеть лицензионное соглашение при первом входе», «админ может просматривать список активных пользователей».
- **Функциональные требования** — это про **что должна делать система**. Здесь уже описываются действия, реакции и поведение на уровне интерфейса и логики. Примеры: «система проверяет почту на валидность», «производит резервное копирование каждый день».
- **Нефункциональные требования** — это про **качество системы**. Скорость, безопасность, масштабируемость. Примеры: «не более 2 ГБ памяти», «работа под нагрузкой 1000 пользователей», «время между сбоями — минимум 100 часов».

**Спецификация требований** (software requirements specification, SRS) объединяет в себе описание всех требований уровня продукта и может представлять собой весьма объёмный документ (сотни и тысячи страниц). Сейчас на проектах используется редко.

**Важно:** на практике вам не всегда дадут готовый документ с требованиями. Иногда их придётся уточнять вручную — у продакта, аналитика или даже разработчика. И именно понимание **разных уровней требований** поможет не пропустить важные детали и грамотно формулировать вопросы.

Постарайтесь запомнить, что бизнес говорит «что нужно», пользователь — «как я это использую», а система — «как я это реализую».

*С остальными уровнями и типами требований вы можете ознакомиться здесь, однако приоритет при обучении и подготовке к интервью советую сделать на уровни выше.*

### Неявные требования

Не всегда требования будут описаны на проекте в виде спецификации или

пользовательской истории. В таком случае необходимо изучать неявные требования из других источников:

1. Законы, регламенты, инструкции
2. Список задач, существующие тесты и баг-репорты
3. Руководство пользователя
4. Реклама продукта
5. Интервью с командой и заказчиками
6. Чаты и email-переписка
7. Прототип, дизайн-макет
8. Конкурентный анализ, личный опыт

Это далеко не весь перечень, подробнее здесь

**Свойства качественных требований**

Когда вы работаете с требованиями, важно понимать, что они могут быть написаны по-разному – и далеко не всегда удачно.

Хорошо написанное требование – это не просто текст, а основа для всей работы команды: разработки, тестирования, аналитики. Поэтому существует набор критериев, по которым можно оценивать качество требований. Привожу краткий обзор с примерами, чтобы вы могли распознавать «плохие» и «хорошие» требования на практике.

**Завершенность (completeness)**

Требование должно содержать всю необходимую информацию.

*Нельзя писать “Экспорт осуществляется в PDF, PNG и т.д.” — формулировка неполная.*

*Фраза “См. выше” — неприемлема: никто не должен гадать, о каком разделе идет речь.*

*“Пароли хранятся в зашифрованном виде” — без указания алгоритма или контекста звучит как догадка, а не как требование.*

**Атомарность, единичность (atomicity)**

Требование должно описывать только одну вещь.

*Если вы пишете: “Кнопка Restart не отображается при остановленном сервисе, окно Log вмещает 20 записей”, — это два разных требования.*

*Когда вы перечисляете несколько действий в одной фразе — вход, отображение имени, выход — это тоже надо разделить.*

*Даже если логика кажется связанной, её лучше вынести в отдельные пункты: так проще поддерживать и тестировать.*

**Непротиворечивость (consistency)**

Внутренние и внешние противоречия должны быть исключены.

*Пример из жизни: “Кнопка Close всегда синяя” и “Кнопка Close всегда красная” — это две разные реальности.*

*Требование “Пользователь без прав может войти в систему” — вызывает вопросы.*

*Также избегайте технической путаницы: “разрешение окна” — некорректный термин, правильнее — “размер окна”.*

**Недвусмысленность (clearness)**

Требование должно быть написано так, чтобы его понял любой участник команды.

*Избегайте жаргона и сокращений: если пишете “ФС” — расшифровывайте.*

*Фразы вроде “прозрачное шифрование” или “история всех изменений” — нуждаются в конкретике.*

*Нужно исключить любые трактовки: одно требование — одно понимание.*

**Выполнимость (feasibility)**

Фантазии — не требования.

*Если вы пишете: “ИИ должен выносить однозначный вердикт о выгоде договора”, — это не реализуемо.*

*Фраза “Система должна предусматривать все возможные варианты запросов и кэшировать их” — невозможно реализовать в рамках проекта.*

*Также остерегайтесь «эффекта озолочения» — дорогих и бессмысленных требований вроде анимаций на каждой кнопке.*

**Обязательность и актуальность (obligatoriness, up-to-date)**

Каждое требование должно быть обосновано.

*Если его добавили “на всякий случай”, — лучше убрать.*

*Устаревшие требования нужно удалять или пересматривать.*

*Приоритеты должны выставляться осознанно, иначе команда будет тратить ресурсы впустую.*

**Прослеживаемость (traceability)**

Хорошие требования можно связать с другими артефактами проекта: кейсами, задачами, багами.

*Если требования не пронумерованы, не структурированы, без ссылок — с ними сложно работать.*

*Также важно использовать инструменты управления требованиями, а не хранить всё в хаосе Word-доков.*

**Модифицируемость (modifiability)**

Требование должно быть написано так, чтобы его можно было легко изменить.

*Если вы меняете одно, и рушится другое — проблема в архитектуре требований.*

*Если вся спецификация — это огромный текстовый файл без структуры, править его будет больно.*

**Проранжированность (ranked for importance, stability, priority)**

Каждому требованию нужно задать три параметра: насколько оно важно, насколько стабильно и насколько срочно.

*Например, авторизация важна и срочна, дизайн профиля — не так срочно. Для этого используется приоритет.*

*Это помогает планировать спринты и правильно расставлять акценты.*

**Корректность и проверяемость (correctness, verifiability)**

Вы должны уметь составить тест-кейс по требованию.

*Если тест невозможен — требование плохое.*

*Хорошее требование: “Имя пользователя отображается в правом верхнем углу после входа”.*

*Плохое: “Профиль должен быть красивым и удобным” — непонятно, как это проверить.*

## Документирование требований

### Пользовательская история (user story)

**Пользовательская история (user story)** - общее описание функций программы, написанное как бы от имени пользователя. На современных проектах чаще используют именно ее для документирования требований. Чаще всего вы будете работать именно с ними.

User story создается по шаблону:

*“As a [persona], I [want to], [so that].”*

*«Как [тип клиента], [хочу то-то], [чтобы делать что-то]».*

*Упрощенные примеры:*

- Как Макс, я хочу пригласить друзей, чтобы мы вместе могли пользоваться этим замечательным сервисом.
- Как Саша, я хочу организовать свою работу, чтобы лучше контролировать ситуацию.
- Как менеджер, я хочу видеть, как продвигается работа у моих коллег, чтобы можно было составлять более точные отчеты о наших успехах и неудачах.

Помимо описания самой user story в ней обязательно содержатся критерии готовности (acceptace criteria), в которых и будут отражены конкретные требования к разрабатываемой функции + ссылка на макет.

Подробнее можно прочитать здесь.

**Пример полной истории:**

**User Story: Вход в систему**

**ID:** US-001

**Название:** Аутентификация зарегистрированного пользователя

**Как:** зарегистрированный пользователь

**Я хочу:** ввести свои email и пароль на странице входа

**Чтобы:** получить доступ к своему личному кабинету

📌 **Acceptance Criteria (Критерии приёмки):**

1. На странице входа доступны поля **Email** и **Пароль**, а также кнопка **“Войти”**.
2. Если email и пароль введены корректно – пользователь попадает на главную страницу.
3. Если email/пароль неверны – появляется сообщение: *“Неверный email или пароль”*.
4. Поле email должно валидироваться на корректность формата (например, user@domain.com).
5. Кнопка “Войти” недоступна, если одно из полей пустое.
6. После успешного входа создаётся авторизационный токен в localStorage.
7. Если пользователь ранее был авторизован – он автоматически попадает в ЛК без ввода логина.

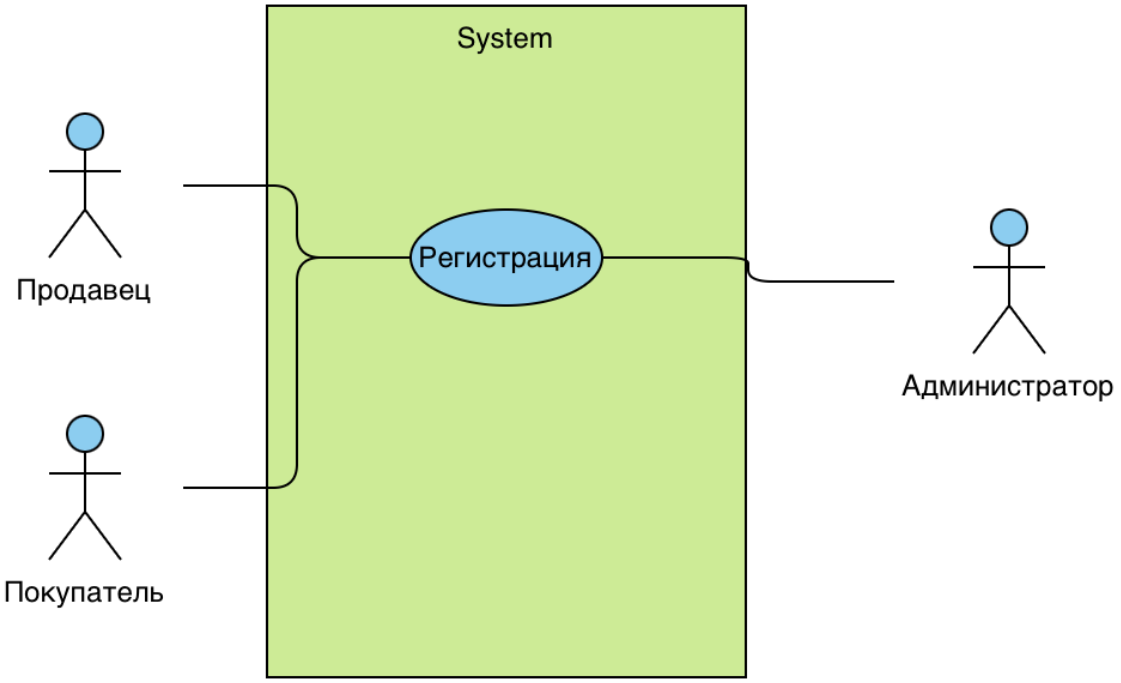
**Варианты использования (use cases)**

**Варианты использования (use cases)** представляют собой сценарии взаимодействия пользователя (или пользователей) с программным продуктом для достижения конкретной цели.

Человечками на схеме обозначаются акторы (не опечатка), которые

используют систему. Прямоугольником обозначается система, а сам вариант использования (функция) овалом. Это так называемая use case диаграмма.

Подробнее можно прочитать *здесь*




**Mockup (макет)**

**Mockup (макет)** - реалистичная модель нашего приложения, которая помогает показать, как дизайн будет выглядеть на frontend.

В тестировании макеты используются для проверки frontend и именно согласно им мы проверяем наличие определенных элементов, их размер, стиль, поведение.

Пример mockup

**Алгоритм работы с требованиями со стороны тестировщика**

**Идеальный процесс**

1. Требования создают бизнес-аналитики или владельцы продукта. Обычно они содержат в себе дизайнерские макеты.

2. Требования попадают к тестировщику, который проверяет их на соответствие свойствам качественных требований. Дополнительно в Scrum есть Backlog Refinement, где требования обсуждаются и уточняются командой на отдельном собрании.
3. В случае обнаружения неточностей, требования отправляются бизнес-аналитику на доработку, например, оставляется комментарий в системе.
4. Пункты 2-3 повторяются до тех пор, пока не будут устранены несоответствия. Параллельно требования могут изучать другие участники команды: разработчики, дизайнеры
5. После того как требования согласованы, они берутся в разработку, а тестировщики создают тестовую документацию для дальнейших проверок. Важно: тест-кейсы и чек-листы создаются до того, как функциональность из требования будет разработана. Вспоминаем Shift Left Testing и раннее тестирование.
6. Вносить изменения в требования после начала спринта не рекомендуется и по-хорошему должно быть запрещено.

**Неидеальный процесс**

1. Анализ требований не всегда осуществляется до этапа разработки и уточнения могут вноситься уже во время нее.
2. Тестовая документация может создаваться уже после разработки функциональности.
3. Требования могут вообще не документироваться и быть на проекте в неявном виде.

## Полезные материалы

1. Презентация
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/reqs
3. Старая версия урока https://youtu.be/Ct8mK-lFw1Q
4. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1G2OeGqcf7MFELnOJdJJC0uo2Z8KIkQXv
5. Тестирование программного обеспечения. Базовый курс. стр. 32 https://svyatoslav.biz/software_testing_book/
6. Библия QA. Требования (Requirements) https://vladislaveremeev.gitbook.io/qa_bible/testovaya-dokumentaciya-i-artefakty-test-deliverablestest-artifacts/trebovaniya-requirements

7. Чек-лист тестирования требований https://habr.com/ru/post/543340/
8. Визуализация ТЗ — диаграммы, схемы, картинки https://habr.com/ru/post/550498/
9. Figma + mockup https://www.figma.com/file/fqZlcMyvIY1CJCJiPUjPnb/Web-Site-(Community)?node-id=186-78&t=lHk8XGCpwKYRSwET-0

ТЕСТ-ДИЗАЙН

# 12. Вступительное слово перед модулем "Тест-дизайн"

**Вступительное слово перед стартом модуля "Техники тест-дизайна"**

Хорошего тестировщика от плохого отличает умение проектировать оптимальные тесты, которые за минимальное количество шагов обеспечат максимальное тестовое покрытие. Именно для этого используется тест-анализ и тест-дизайн.

- Зачем тестировать все значения из одного диапазона, если они выдают один и тот же результат?
- Зачем тестировать все параметры, если достаточно проверить только взаимодействие пары из них?
- Зачем писать много текста для проверки двух правил, если можно создать одну таблицу и поместить в нее результаты?

**Все это мы научимся делать в новом модуле** и на практике используем **технику эквивалентного разбиения, попарного тестирования и таблицу принятия решений.**

Не забывайте применять ваши знания на практике и не ограничивайтесь только заданиями из курса. Возьмите любой сайт или мобильное приложение и подумайте: какие техники тест-дизайна вы можете к нему применить, а после создайте таблицу на гугл-диске с идеями и реализацией. После вы сможете использовать это в своем портфолио.

Помните, что вы выбрали для себя формат самообучения, поэтому его успех зависит только от вас!

ТЕСТ-ДИЗАЙН

# 13. Эквивалентное разбиение и анализ граничных значений

*Перед тем как мы начнем изучение каждой техники тест-дизайна по отдельности, давайте поговорим, что это такое и чем дизайн отличается от анализа.*

**Анализ тестирования (test analysis)** - деятельность, которая направлена на определение тестовых условий путем анализа базиса тестирования. [ISTQB Glossary]

**Проектирование теста (test design)** - активность по получению и определению тест-кейсов в соответствии с условиями тестирования. [ISTQB Glossary]

**Перевожу на человеческий язык**

*Когда вы впервые слышите термины анализ тестирования и проектирование теста, может показаться, что это одно и то же — ведь и там, и там речь идет о подготовке тестов. Но на самом деле это **разные этапы**, и их важно не путать.*

**Анализ тестирования (test analysis)** — это когда вы внимательно изучаете всё, что у вас есть: требования, макеты, спецификации, баг-репорты и т.п.

Ваша цель — **понять, что вообще нужно проверять**.

На этом этапе появляются **тестовые условия** (test conditions) — абстрактные идеи, которые позже превратятся в конкретные кейсы.

**Пример:**

У вас есть требование: *“Пользователь должен иметь возможность восстановить пароль по email”*.

Тестовое условие: *“Проверить возможность восстановления пароля через email”*.

**Проектирование теста (test design)** — это следующий шаг. Вы берёте тестовые условия и **превращаете их в конкретные тест-кейсы** с шагами, входными данными и ожидаемыми результатами.

Здесь вы решаете: **что именно, как и с какими данными будете проверять**.

**Пример:**

Вы превращаете тестовое условие в тест-кейс:

1. Перейти на страницу логина
2. Нажать “Забыли пароль?”
3. Ввести валидный email
4. Проверить, что приходит письмо со ссылкой
5. Пройти по ссылке и сменить пароль

**Важно:**

Анализ — это про **что тестировать**

Проектирование — это про **как тестировать**

Обычно эти этапы идут друг за другом, но иногда пересекаются или повторяются, особенно в гибкой разработке.

## Эквивалентное разбиение

Предположим, что для расчета скидки в онлайн-магазине, пользователь должен ввести в поле свой возраст. Допускается ввод только чисел. Расчет скидки производится следующим образом:

Возраст покупателя от 0 до 17 лет - **скидка 50%**

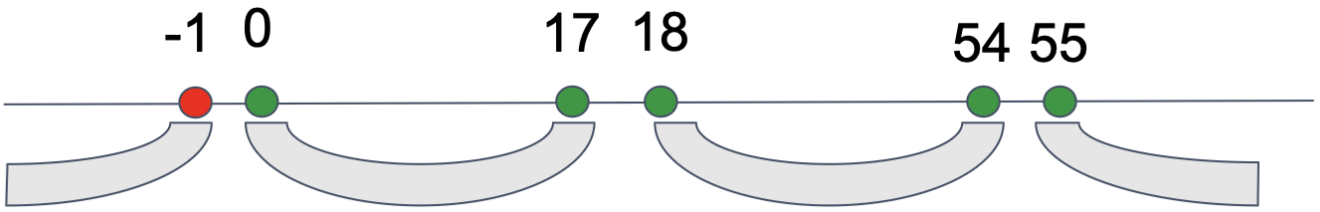
Возраст покупателя от 18 до 54 лет - **скидка 25%**

Возраст покупателя от 55 и выше - **скидка 75%**

### Шаг 1. Определяем классы эквивалентности и граничные значения

Класс эквивалентности (equivalence class) — набор данных, обрабатываемых одинаковым образом и приводящих к одинаковому результату

Граничное условие (border condition, boundary condition) — значение, находящееся на границе классов эквивалентности.




**Негативный класс (работает по аналогии с негативным тестированием):**

Возраст покупателя до 0 лет - сообщение об ошибке

**Позитивные классы:**

Возраст покупателя от 0 до 17 лет - скидка 50%

Возраст покупателя от 18 до 54 лет - скидка 25%

Возраст покупателя от 55 и выше - скидка 75%

**Шаг 2. Тестируем хотя бы одно значение из каждого класса. Так как код пишется таким образом, что все правила к одному из представителей класса применяются ко всем другим.**

Помимо очевидного негативного класса можно определить дополнительные. Например, если поле может принимать только числа, то следует проверить как поведет себя система при вводе букв, спецсимволов и других значений.

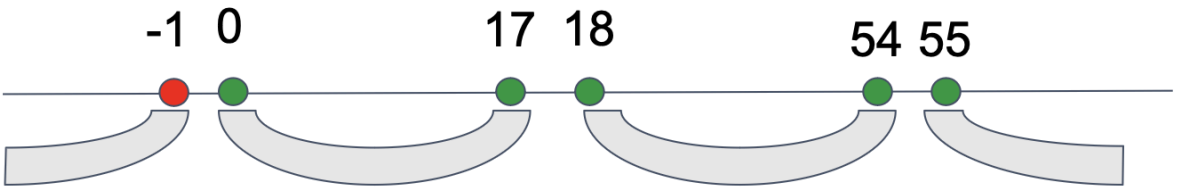
**Анализ граничных значений**

На практике одного эквивалентного разбиения недостаточно.

Метод анализа граничных значений является продолжением метода эквивалентного разбиения, но может быть применим, только если классы состоят из **упорядоченных числовых значений***. Максимальное и минимальное значение класса являются его границами [Beizer 1990]. Некорректное поведение более вероятно на границах класса, чем внутри класса. [ISTQB CTFL Syllabus 2018]

*Данная формулировка отсутствует в версии 2023.

Напоминаю про диапазоны из предыдущего шага, где определены их границы:



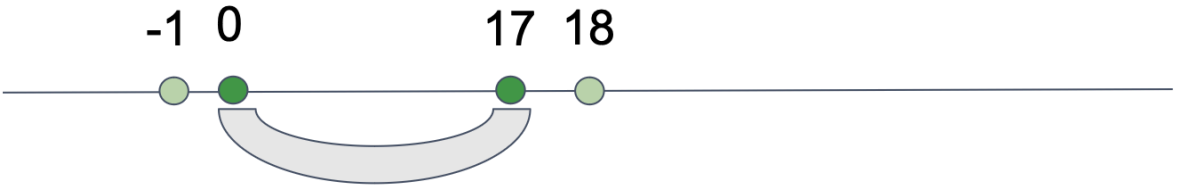

**Шаг 1. Определяем остальные граничные значения**

Определяем остальные граничные значения. Существует два метода.

Рассмотрим **первый метод [Craig, 2002; Myers, 2011]** :

а) Тестирование нижней границы и значения до нее

б) Тестирование верхней границы и значения после нее



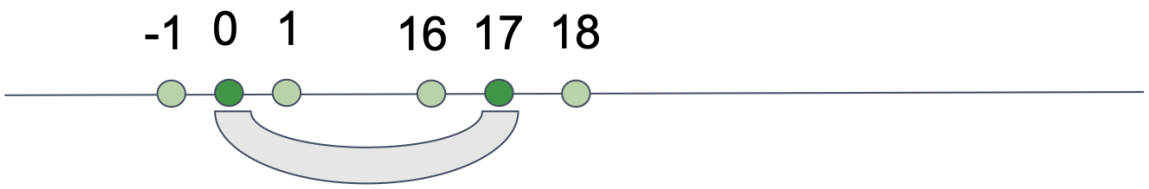

В итоге выходит следующая таблица для тестирования:

| **Позитивные классы** | **Граничные значения** | **Ожидаемый результат** |
|---|---|---|
| 0 - 17 | -1, 0 и 17, 18 | скидка 50% |
| 18 - 54 | 17, 18 и 54, 55 | скидка 25% |
| 55 - +беск | 54, 55, для +беск нет | скидка 75% |
| **Негативные классы** | | |
| от -беск до -1 | для -беск нет и -1, 0 | сообщение об ошибке |
| Буквы | нельзя определить | сообщение об ошибке |
| спецсимволы | нельзя определить | сообщение об ошибке |

**Второй метод** [Koomen, 2006; O’Regan, 2019]:

а) Тестирование нижней границы и значения как до нее, так и после нее

б) Тестирование верхней границы и значения как до нее, так и после нее




В итоге выходит следующая таблица для тестирования:

| **Позитивные классы** | **Граничные значения** | **Ожидаемый результат** |
|---|---|---|
| 0 - 17 | -1, 0, 1 и 16, 17, 18 | скидка 50% |
| 18 - 54 | 17, 18, 19 и 53, 54, 55 | скидка 25% |
| 55 - +беск | 54, 55, 56 для +беск нет | скидка 75% |
| **Негативные классы** | | |
| от -беск до -1 | для -беск нет и -2, -1, 0 | сообщение об ошибке |
| Буквы | нельзя определить | сообщение об ошибке |
| спецсимволы | нельзя определить | сообщение об ошибке |

*В сообществе до сих пор идут споры: какой из методов считать правильным. При выполнении тестовых заданий советую делать уточнение, какой из них вы используете.*

*Полученные таблицы используют для формирования чек-листов или тест-кейсов.*

**Дополнительная информация:**

1. В случае тестирования десятичных дробей, граничные значения будут зависеть не только от целой, но и от дробной части.

2. Например, допустимые значения от 0,01 до 10.00. Граничные значения для этого класса: 0.00, 0.01, 0.02 и 9.99, 10.00, 10.01
3. Значения, которые повторяются, второй раз тестирование не нужно.
4. При выполнении тестовых заданий рекомендую оставлять комментарий о том, что они были удалены
5. Лучше указывать не конкретные примеры представителей, а их привязку к количеству символов и типу данных. Например, “абвг” заменить на 4 буквы.
6. Существуют разные типы границ: технологические, логические и произвольные. О них можно прочитать здесь.
7. Не всегда на проектах точно описаны все границы, но надо уметь их определять и тестировать. Как это сделать, можно узнать в этой статье.
8. Логическая ловушка: умение различать представителей класса и представителей граничных значений из смежных классов. Например, для позитивного класса от 0 до 17 граничными значениями для 17 будут 16,17,18, где 18 будет представителем негативного смежного класса. Это не делает его представителем позитивного класса, но в таблицах и кейсах оно будет фигурировать как граничное значение позитивного класса.
9. Классы эквивалентности можно использовать и для других аспектов, например, для формирования выборки устройств для тестирования (разрешение экрана, операционные системы)
10. Тестирование классов эквивалентности в равной степени применимо на модульном, интеграционном, системном и приемочном уровнях тестирования. Все это требует входных или выходных значений, которые могут быть разделены на основе системных требований. [Lee Copeland]

**Полезные материалы**

1. Общая презентация по модулю
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/bva
3. Старые версии урока
4. https://youtu.be/HJPF5qJx7jg
5. https://youtu.be/NQBi716WCOU
6. Схемы из старых версий урока - https://drive.google.com/drive/u/3/folders/17CXTVPerHNk751hnVFWciEhHJ3zpP_WV
7. https://drive.google.com/file/d/1geWjwH_-7evCRzeQdgGO7cug5ha9TQr5/view?usp=sharing

8. Тестирование программного обеспечения. Базовый курс, стр.237 - https://svyatoslav.biz/software_testing_book/
9. Библия QA. Тест-дизайн и техники тест-дизайна (Test Design and Software Testing Techniques) https://vladislaveremeev.gitbook.io/qa_bible/test-dizain/test-dizain-i-tekhniki-test-dizaina-test-design-and-software-testing-techniques
10. Dynamic - Black box https://vladislaveremeev.gitbook.io/qa_bible/test-dizain/dynamic-black-box
11. Lee Copeland на русском языке - https://drive.google.com/file/d/1big8X9HH0JUsyf_iVuMWqJZ7cRxRwBxR/view?usp=sharing
12. Анализ тестов - как выкидывать лишнее? https://habr.com/ru/post/670428/
13. Ссылка на силлабус ISTQB (п.4.2. Методы черного ящика) https://www.rstqb.org/ru/istqb-downloads.html

ТЕСТ-ДИЗАЙН

# 14. Попарное тестирование

**Попарное тестирование (pairwise testing)** - техника тестирования методом черного ящика, при которой тестовые сценарии разрабатываются таким образом, чтобы выполнить тестирование для всех комбинаций параметр-значение.[ISTQB Glossary]

Для формирования пар используется два подхода: ортогональные массивы *(знать для общего развития, как они работают не спросят на интервью)* и алгоритм All Pairs. В практике тестирования вы будете использовать последний, поэтому давайте сконцентрируемся на нем.

**Алгоритм All Pairs** позволяет генерировать все пары автоматически. Самым популярным инструментом для этого является PICT, но существует еще несколько десятков других решений. Подробнее об использовании PICT в видео-уроке.

Выходные данные полученные этим способом могут отличаться от данных, полученных в результате применения ортогональных массивов. Про ортогональные массивы можно прочитать в пособии *Lee Copeland, «A practitioner’s guide to software test design».*

**Применение:** на всех уровнях тестирования. Всё, что требуется – это комбинации входных точек (параметров), каждый из которых принимает различные значения до такой степени, что результат может привести к комбинационному взрыву, так как для тестирования существует слишком много комбинаций.

### Пример использования

Давайте рассмотрим пример применения этого алгоритма с тестированием фильтрации на примере интернет-магазина.

Пользователь может осуществить фильтрацию по следующем параметрам:

- *Цвет:* красный, белый, черный, серый
- *Размер:* S, M, L, XL

- *Пол:* мужской, женский, унисекс
- *Доставка:* есть, нет

Общее количество всех проверок без применения попарного тестирования: 4*4*3*2 = 96.

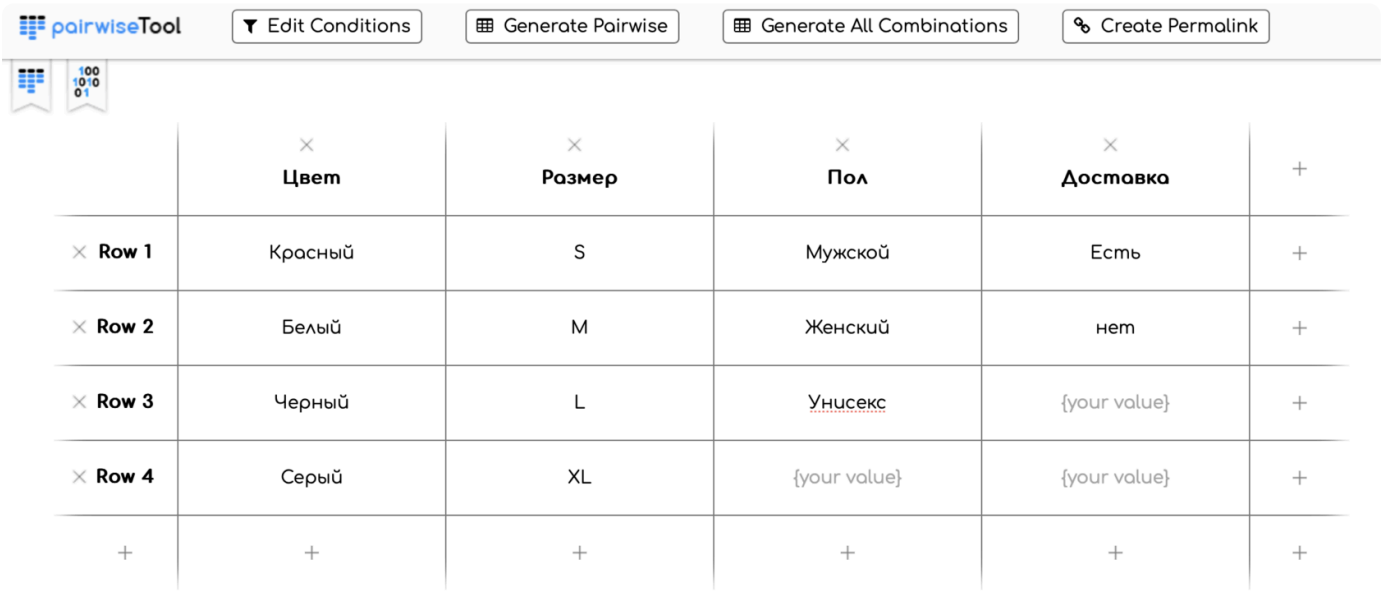
Согласитесь, что проводить 96 тестов всех комбинаций времязатратно. Для оптимизации нам пригодится Pairwise.

Согласно этой технике, необходимо протестировать взаимодействие всех пар параметров и их значений, что будет соответствовать практически полному тестовому покрытию, если бы мы тестировали все комбинации.

**Какие шаги нужно сделать?**

Для выполнения задания будем использовать ресурс https://pairwise.teremokgames.com, который не требует предварительной установки.

1. Создадим таблицу с параметрами и значениями. Всегда начинаем с параметра, у которого больше всего значений.




| | Цвет | Размер | Пол | Доставка |
|---|---|---|---|---|
| Row 1 | Красный | S | Мужской | Есть |
| Row 2 | Белый | M | Женский | нет |
| Row 3 | Черный | L | Унисекс | {your value} |
| Row 4 | Серый | XL | {your value} | {your value} |

2. Для генерации пар необходимо нажать кнопку

3. После нажатия сформируется таблица с необходимыми тестовыми данными. Количество проверок сократится с 96 до 16. Таблицу необходимо использовать для создания чек-листа или тест-кейсов.

| | Цвет | Размер | Пол | Доставка |
|---|---|---|---|---|
| 1 | Красный | S | Мужской | Есть |
| 2 | Красный | M | Женский | нет |
| 3 | Красный | L | Унисекс | Есть |
| 4 | Красный | XL | Мужской | нет |
| 5 | Белый | M | Унисекс | нет |
| 6 | Белый | L | Мужской | Есть |
| 7 | Белый | XL | Мужской | нет |
| 8 | Белый | S | Женский | Есть |
| 9 | Черный | L | Мужской | Есть |
| 10 | Черный | XL | Женский | нет |
| 11 | Черный | S | Унисекс | Есть |
| 12 | Черный | M | Мужской | нет |
| 13 | Серый | XL | Унисекс | нет |
| 14 | Серый | S | Мужской | Есть |
| 15 | Серый | M | Мужской | нет |
| 16 | Серый | L | Женский | Есть |

**Полезные материалы**

1. Общая презентация по модулю
2. Старая версия урока https://youtu.be/-Lu27061BiY
3. В 9 строке необходимо заменить Курсив-Черный на Курсив-Красный. В самих ресурсах верная таблица с исправленным значением
4. Тестирование программного обеспечения. Базовый курс, стр.245 - https://svyatoslav.biz/software_testing_book/
5. Статья о попарном тестировании https://cutt.ly/EjYCcDF
6. Статья о попарном тестировании на украинском языке https://training.qatestlab.com/blog/technical-articles/pairwise-testing/
7. Ссылка на pict https://drive.google.com/drive/u/3/folders/1yEaw5ZDQiUIY3W4jlRznrMs5jZg

WotUa

8. Веб-версия pict https://pairwise.yuuniworks.com/
9. Ссылка на инструменты для попарного тестирования https://www.pairwise.org/tools.html
10. Онлайн инструмент для Pairwise Testing (+ генератор файлов) https://pairwise.teremokgames.com
11. Таблица из видео https://cutt.ly/QjHx8Nb

ТЕСТ-ДИЗАЙН

# 15. Диаграмма переходов и состояний, таблица принятия решений

### Таблица принятия решений

**Таблицы альтернатив (*прим. синоним*)** – хороший способ записи сложных бизнес-правил, которые должны быть реализованы в системе. В процессе создания таблицы, тестировщик определяет условия (входы) и результирующие действия системы (выходы). Пары условий и действий образуют строки таблицы, при этом условия указываются сверху, а действия – снизу. *[ISTQB CTFL Syllabus]*

| | |
|---|---|
| *Условие 1* | |
| *Условие 2* | |
| **Действие 1** | |
| **Действие 2** | |

**Пример**

Оклад сотрудника рассчитывается следующим образом:

- без опыта и высшего образования - 500
- с высшим образованием, но без опыта - 600
- без высшего образования, с опытом - 600
- с опытом и высшим образованием - 700

Премия для сотрудника рассчитывается следующим образом:

- без опыта и высшего образования - 5%
- с высшим образованием, но без опыта - 10%
- без высшего образования, с опытом - 15%
- с опытом и высшим образованием - 20%

Ваша задача протестировать приложение, которое должно рассчитывать оклад и премию сотрудника.

Так как в данном случае можно четко определить условия, действия и правила, то это замечательный кандидат для создания таблицы принятия решений.

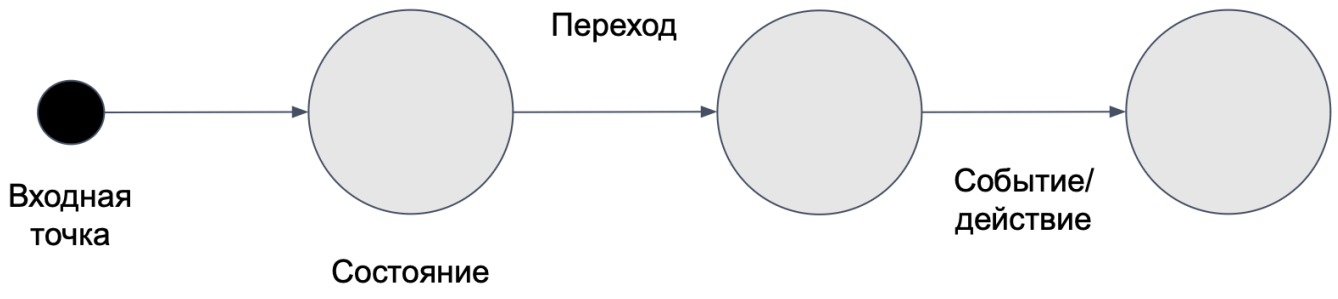
| | Правило 1 | Правило 2 | Правило 3 | Правило 4 |
|---|---|---|---|---|
| *Наличие высшего образования* | Нет | Есть | Нет | Есть |
| *Опыт работы* | Нет | Нет | Есть | Есть |
| **Оклад** | 500 | 600 | 600 | 700 |
| **Премия** | 5% | 10% | 15% | 20% |

Итоговую таблицу необходимо использовать для создания чек-листа или тест-кейсов.

## Тестирование состояний и переходов

**Тестирование таблицы переходов (state transition testing)** - разработка тестов методом черного ящика, при котором сценарии тестирования строятся на основе модели переходов состояний. [ISTQB Glossary]

Диаграмма состояний и переходов включает в себя состояние, переход, событие, действие и входную точку. Всегда включает в себя только один объект.



**Состояние** - это состояние, в котором система ожидает возникновения одного или нескольких событий.

**Переход** - это изменение состояния из одного в другое, произошедшее благодаря какому-то событию.

**Событие -** что-то, что вызывает изменение состояния системы.

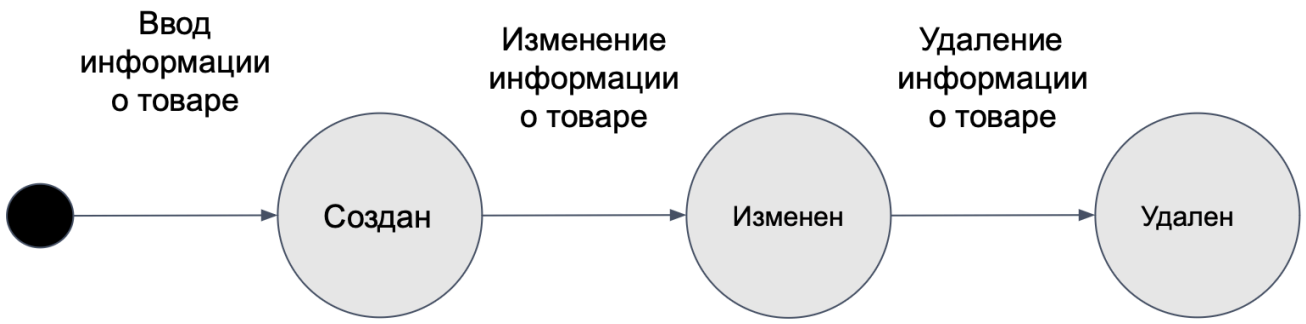
**Действие** - это операция, которая вызвана изменением состояния.

*Источник: Lee Copeland, "A practitioner’s guide to software test design".*

**Пример**

Предположим, что нам надо протестировать процедуру добавления информации в карточку товара в интернет-магазине.

Товар можно создать, изменить и удалить. Диаграмма может выглядеть таким образом.




Подобное схематическое изображение упрощает понимание требований, а также помогает в создании полных тестовых наборов.

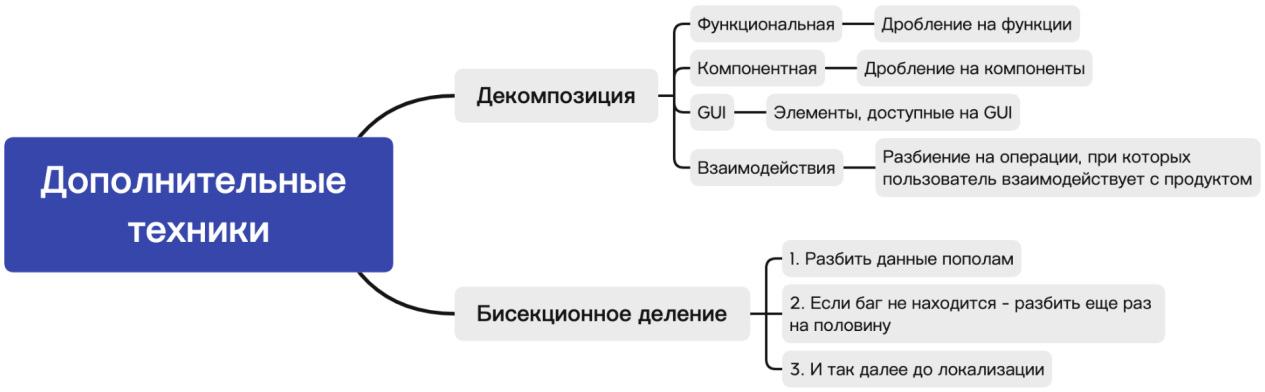
## Дополнительные техники тест-дизайна

**Предположение об ошибках** (error guessing) - техника тестирования, в которой тесты получены на основе знаний тестировщика о ранее обнаруженных сбоях или общих знаниях о типах отказов

**Отображение причинно-следственных связей** (cause-effect graphing) - метод тестирования черного ящика, при котором тестовые сценарии

разрабатываются на основе диаграмм причинно-следственных связей.

**Диаграмма причинно-следственных связей** - графическое представление логических отношений между входными данными (причинами) и связанными с ними выходными данными (следствиями).




**Источники:** https://qaschool.ru/blog/dekompozirui-eto/
https://habr.com/ru/post/468087/

### Полезные материалы

1. Общая презентация по модулю
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/state
3. Старая версия урока https://youtu.be/e84cyz2HC24
4. Таблица из старой версии урока https://drive.google.com/drive/u/3/folders/1zYJu-BQE-OHacojOlCpAYJtnyE1GtFZ_
5. Библия QA. Dynamic - Black box https://vladislaveremeev.gitbook.io/qa_bible/test-dizain/dynamic-black-box
6. Decision Table — что это и как применять https://habr.com/ru/post/546432/
7. State & Transition Diagram — что это и как применять https://habr.com/ru/post/548192/

# 16. Вступительное слово перед модулем "Тестовая документация"

**Вступительное слово перед стартом модуля "Тестовая документация"**

Несмотря на кажущуюся простоту, писать качественную тестовую документацию умеют немногие, даже специалисты с опытом этим грешат.

**В данном модуле мы научимся** создавать чек-лист, тест-кейсы, отчеты о дефекте, а также поверхностно затронем тест-план, тестовую стратегию, отчеты по результатам тестирования (новички редко участвуют в их создании, но бывают разные кейсы). Научимся работать с системами по управлению документацией: *Jira, TestRail, QASE, TestIT.*

**Все это на примере реального веб-приложения.**

*На что стоит обратить внимание в первую очередь?*

1. **Структура и обязательные атрибуты.** Большинство ошибок возникает по причине невнимательности, хотя каждый атрибут очень важен.
2. **Заголовки.** У каждого вида документации есть свои требования к заголовкам. Короткое название проверки из чек-листа не подходит для тест-кейса. Заголовок кейса не подходит для баг-репорта, так как несет совсем другую функцию.
3. **Особенности использования систем для хранения документации.** Вы должны научиться не только создавать документацию в специальных инструментах, но без проблем находить ее и понимать, на каких этапах SDLC она используется.

Помните, что ваша документация создается для команды, а не только для вас. Поэтому она должна быть понятная и не порождать новые вопросы.

Я всегда советую давать первые версии ваших кейсов, чек-листов и баг-репортов друзьям и близким на проверку. Если они не смогут воспроизвести все действия из нее, значит вы делаете что-то не так :)

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

# 17. Тест-план и тест-стратегия

### Тест-план

**План тестирования (test plan)** - документация, описывающая цели тестирования, которые должны быть достигнуты, средства и график их достижения, организованная для координации тестовой деятельности. *[ISTQB Glossary]*

**Виды тест-плана**

**Структура тест-плана**

1. Цель (purpose)

2. Области, подвергаемые тестированию (features to be tested)
3. Области, не подвергаемые тестированию (features not to be tested)
4. Тестовая стратегия (test strategy) и подходы (test approach)
5. Критерии (criteria). Чаще всего это критерии начала и завершения тестирования
6. Ресурсы (resources): программные, аппаратные, человеческие, финансовые
7. Расписание (test schedule)
8. Роли и ответственность (roles and responsibility)
9. Оценка рисков (risk evaluation)
10. Документация (documentation)
11. Метрики (metrics) - числовые характеристики показателей качества

*Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”*

Примеры тест-планов

**Тестовая стратегия**

**Тестовая стратегия (как отдельный документ)** – это общий подход к обеспечению качества продукта. Она определяет принципы и методы тестирования, которые будут использоваться на уровне всей организации или продукта. В стратегии описываются ключевые виды тестирования, критерии качества, подходы к управлению рисками, принципы выбора инструментов и автоматизации, а также общие правила приоритизации и взаимодействия в команде. Это документ высокого уровня, который редко меняется и служит фундаментом для всех последующих тестовых активностей.

**Тестовая стратегия в тест-плане** – адаптация или конкретизация общей стратегии **под текущий проект**. В этом разделе описываются:

- **Какие типы тестирования будут использоваться** (например, функциональное, регрессионное, интеграционное);
- **Какие уровни тестирования применимы** (например, модульное, системное);
- **Какие методы тест-дизайна** будут использоваться (например, классы эквивалентности, граничные значения);

- **Какие инструменты и подходы** к автоматизации, ручному тестированию, управлению дефектами применяются в данном контексте;
- **Как будет обеспечено покрытие требований тестами**;
- **Как будут использоваться данные тестирования**, окружения и т.д.

### Примеры критериев начала и завершения тестирования

*Entry criteria:*

1. Выход билда для тестирования
2. 100% требований и мокапов утверждены и проверены

*Exit criteria:*

1. Выполнение более 80 % запланированных на итерацию тест-кейсов
2. Исправлено 100% критических багов
3. Автоматизированно 80% регрессионных тест-кейсов

Старайтесь использовать в критериях количественные метрики, которые можно посчитать.

### Полезные материалы

1. Шпаргалка (адаптирована под десктоп) https://rusau.net/test-plan
2. Общая презентация
3. Шаблоны тест-плана
4. Старая версия урока https://youtu.be/j06wnjHIeYU
5. Схема из старой версии урока - https://drive.google.com/drive/u/3/folders/1EWN7vC_YSRL5HNnOZOG13bJPsTIUsN_J
6. Об этапах подготовки тест-стратегии https://software-testing.ru/library/testing/test-management/2047-2015-03-18-07-17-46
7. Шаблон тест-стратегии https://tmguru.ru/baza-znanij/protsess-testirovaniya/planirovanie/strategiya-testirovaniya/
8. Самый полезный русскоязычный мануал по написанию тест-плана (не верьте названию, это не тест-стратегия) https://habr.com/ru/company/true_engineering/blog/428053/
9. Библия QA. План тестирования (Test plan) https://vladislaveremeev.gitbook.io/qa_bible/testovaya-dokumentaciya-i-artefakty-test-deliverablestest-artifacts/plan-testirovaniya-test-plan

10. Стратегия тестирования (Test strategy) https://vladislaveremeev.gitbook.io/qa_bible/testovaya-dokumentaciya-i-artefakty-test-deliverablestest-artifacts/strategiya-testirovaniya-test-strategy

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

# 18. Отчет по результатам тестирования

**Отчет о ходе тестирования* (test progress report)** - тип периодического отчета о тестировании, который включает информацию о ходе активностей тестирования в сравнении с исходными планами, риски и альтернативы, требующие принятия решения. [ISTQB Glossary]

*прим. автора: чаще всего его называют отчет по результатам тестирования.

**Основная цель отчета о тестировании:** дать общую оценку качества продукта и рекомендации по возможности или невозможности его релиза.

### Атрибуты отчета о тестировании

1. Краткое описание (summary) отражает основные достижения, проблемы, выводы и рекомендации.
2. Команда тестировщиков (test team)
3. Описание процесса тестирования (testing process description) - перечень работ за конкретный период
4. Расписание (timetable) - детализированное расписание работы команды тестировщиков и/или личные расписания участников команды.
5. Статистика по новым дефектам (new defects statistics) в виде таблицы
6. Список новых дефектов (new defects list)
7. Статистика по всем дефектам (overall defects statistics) - таблица, содержащая данные обо всех дефектах за время проекта
8. Рекомендации (recommendations)
9. Приложения (appendixes) - фактические данные (графики и диаграммы)

### Полезные материалы

1. Общая презентация
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/report
3. Советы по написанию отчета http://web.archive.org/web/20220516150525/https://www.testuastartups.com/post/testsummaryreport
4. Советы и шаблон для отчета https://www.softwaretestinghelp.com/test-summary-report-template-download-sample/

5. Еще один шаблон отчета https://www.guru99.com/how-test-reports-predict-the-success-of-your-testing-project.html
6. Шаблон отчета по тестированию (pdf) https://www.performance-lab.ru/wp-content/themes/pureengineering/images/sitetesting/test_report_example.pdf
7. Шаблон отчета по тестированию (xls) https://vk.com/@usetalkrostov-otchet-o-testirovanii-reliza
8. Виды отчетов (Reports) https://vladislaveremeev.gitbook.io/qa_bible/testovaya-dokumentaciya-i-artefakty-test-deliverablestest-artifacts/vidy-otchetov-reports

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

# 19. Чек-листы

**Тестирование на основе чек-листов (checklist-based testing)** - техника тестирования на основе опыта, при которой тестовые сценарии разрабатываются для выполнения пунктов чек-листа.

**Чек-лист (checklist)** - набор идей: идей по тестированию, идей по разработке, идей по планированию и управлению — любых идей.

*Самый простой пример чек-листа в отрыве от тестирования: список покупок, где покупка является проверкой, а чек-бокс - ее статус.*

### Обязательные части чек-листа

- **Шапка:** содержит информацию о названии приложения, его версии, окружении, на котором проводится тестирование (версия ОС, браузера, эмулятора), ответственного за тестирование, дату тестирования
- **Тестируемые модули, субмодули:** например, регистрация, аутентификация авторизация
- **Список проверок:** они должны отражать основную суть, без лишней детализации
- **Статус:** информация о статусе прохождения проверки - пройдено/не пройдено (passed/failed)

### Дополнительные части чек-листа

- **Ожидаемый результат:** то, что мы ожидаем увидеть после запуска проверки согласно требованиям
- **Типы тестирования:** к какому типу относится проверка?
- **Отчеты о дефекте:** ссылки на отчеты о дефектах для прослеживаемости
- **Заметки:** если нужно добавить комментарии
- Также у проверок может быть **уникальный идентификатор** для прослеживаемости.

### Фрагмент реального чек-листа

| Project | | | | Alexpress | | | | |
|---|---|---|---|---|---|---|---|---|
| Date | | | | 06.08.2022 | | | | |
| Build | | | | 1.0 | | | | |
| Tester | | | | Artsiom Rusau | | | | |
| Environment | | | | Windows 11 Version 22H2<br>Google Chrome103.0.5060.134 | Safari | Firefox | Opera | Edge |
| Module | Submodule | Element/ function | Summary | Status | Status | Status | Status | Status |
| | Choose a location | Dropdown | All countries are loaded | passed | | | | |
| | | | User can choose a country | failed | | | | |
| | | | The country search is working | | | | | |
| | | | Cancel button | | | | | |
| | | | Input numbers > error message | | | | | |
| | | | Input special characters > error message | | | | | |

## Рекомендации

1. Если чек-лист используется для разных итераций, то можно создавать отдельную вкладку в таблице для каждой итерации.
2. Если необходимо проводить тестирования на разных платформах, то есть смысл добавить дополнительные столбцы.
3. Всегда начинайте проводить тестирование с позитивных кейсов и не объединяйте негативные проверки между собой.

Примеры чек-листов

## Чит-листы

Существуют так называемые чит-листы. Обычно в них объединяют часто повторяющиеся проверки, например:

1. Общие проверки для веб и мобильных приложений
2. Список проверок для тестирования веб-форм
3. Список проверок для определенного типа тестирования, например, usability или локализация

Также в такие чит-листы иногда относят:

1. Список самых популярных команд в Git
2. Список самых популярных команд в bash
3. Список самых популярных операторов в SQL

## Полезные материалы

1. Общая презентация
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/checklist
3. Старые версии уроков
4. https://youtu.be/IdjlZhVdiO4

5. https://youtu.be/D2shLisnEBM
6. Тестирование программного обеспечения. Базовый курс, стр.115 - https://svyatoslav.biz/software_testing_book/
7. Шаблоны чек-листов + схема из урока - https://is.gd/2Mslke
8. Чек-лист - как тестировать поиск? https://habr.com/ru/post/577448/
9. Ссылка на чек-лист и тест-кейс для регистрации https://docs.google.com/spreadsheets/d/1iFrmTyODtKgHnMHlqFWVVsKWqPisH_pf34TgxMvvPWA/edit?usp=sharing
10. Информация о тестовых сценариях https://www.guru99.com/test-scenario.html

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

# 20. Тест-кейсы

**Тестовый случай (test case)** - набор предусловий, входных данных, действий (где применимо), ожидаемых результатов и постусловий, разработанных на основе тестовых условий.

**Тестовый сценарий (test scenario)** - последовательность действий над продуктом, которые связаны единым ограниченным бизнес-процессом использования, и сообразных им проверок корректности поведения продукта в ходе этих действий

**Набор тестов** (test suite) - набор тестовых сценариев или тестовых процедур, выполняемых в определенном тестовом прогоне.

В отличие от чек-листа, тест-кейс содержит в себе больше деталей: идентификатор, приоритет, заголовок, шаги и ожидаемые результаты.

Тест-кейсы объединяют в **тестовые наборы (test suites)** по одинаковым признакам: проверки для одного модуля, проверки для определенного типа тестирования, регрессионные наборы.

**Высокоуровневый тест-кейс** (high level test case) — тест-кейс без конкретных входных данных и ожидаемых результатов.

**Низкоуровневый тест-кейс** (low level test case) — тест-кейс с конкретными входными данными и ожидаемыми результатами.

**Атрибуты тест-кейса**

- **Идентификатор (ID):** уникальный номер, необходимый для прослеживаемости. В системах по управлению кейсами (TMS - test management system) проставляется автоматически
- **Приоритет (Priority):** срочность и важность выполнения задачи
- **Требование (Requirement):** ссылка на требование, для проверки которого служит кейс
- **Модуль (Module):** название структурной части, в которой находится предмет тестирования
- **Заголовок (Title):** Отражает суть проверки

- **Тестовые данные и предусловия (input data, test data, preconditions):** информация о данных, которые необходимы для тестирования (данные для ввода, файлы с определенным расширением и размером и т.д.) + специальное состояние системы до начала тестирования (пользователь зарегистрирован, созданы объекты в базе данных и т.д.)
- **Шаги (Steps):** последовательность действий для получения ожидаемого результата
- **Ожидаемые результаты (Expected results):** ссылка на требование, для проверки которого служит кейс. Результат должен быть для каждого шага.
- **Постусловия (Postconditions):** возвращение систему в исходное состояние (удаление данных, пользователей, отключение виртуальной машины и т.д.)

**Обязательными атрибутами для тест-кейса являются:** идентификатор, приоритет, заголовок, шаги и ожидаемые результаты.

*Все остальное используется в зависимости от конкретной задачи.*

## Статусы

В системах по управлению тестовой документацией есть атрибут статус (Status).
Стоит различать статус кейса и статус проверки при их запуске во время тестирования (тестовый прогон, test-run).

**Статус кейса:** черновик (draft), активный (active), устарел (outdated) и др.

**Статус проверки:** пройдена (passed), провалилась (failed), пропущена (skipped) и др. Ориентируйтесь на жизненный цикл тест-кейса на вашем проекте.

*Шпаргалка (адаптирована под десктоп) [https://rusau.net/case-cycle](https://rusau.net/case-cycle)*

## Пример тест-кейса

| ID | Summary | Pre-conditions | Test data | Steps | Expected results |
|---|---|---|---|---|---|
| 1 | The registration via email by non-existed user | 1. The email was created<br>2. The user is not registered | Email: test@gmail.com | 1. Go to https://best.aliexpress.com/<br>2. Go to Account > Register button<br>3. Choose a location<br>4. Input email from test data<br>5. Input a password (6-20 symbols)<br>6. Click 'Create accout' button<br>7. Check a mailbox<br>8. Copy a verification code and fill in a verification window<br>9. Click the 'Verify email' button<br>10. Log out<br>11. Try to log in witn user credentials | 1. The main page is opened<br>2. The registration window is opened<br>3. The dropdown with location is opened. User can choose a location<br>4. The email is displayed withoout error message<br>5. The password is displayed withoout error message. All validations are OK<br>6. The verification window is dispalyed. The 'Verify Email' button is disabled<br>7. The verification code was received<br>8. The 'Verify' button is active<br>9. The user was registered<br>10. The user is logged out<br>11. The user is logged in without error messages |

[Примеры тест-кейсов]

### Отличия чек-листа и тест-кейса

*ВИД ДОКУМЕНТАЦИИ МОЖЕТ ЗАВИСЕТЬ ОТ ЗАКАЗЧИКА И УСЛОВИЙ КОНТРАКТА*

На проекте могут одновременно использоваться и чек-лист, и тест-кейс. Например, чек-лист для проверки GUI, а тест-кейсы для проверок сложной бизнес-логики.

### Полезные материалы

1. Общая презентация

2. Шпаргалка (адаптирована под десктоп) https://rusau.net/case
3. Старые версии уроков
4. https://youtu.be/IdjlZhVdiO4
5. https://youtu.be/D2shLisnEBM
6. Тестирование программного обеспечения. Базовый курс, стр.115 - https://svyatoslav.biz/software_testing_book/
7. Шаблоны чек-листов + схема из урока - https://drive.google.com/drive/u/3/folders/1Mvd8rVbfq4cQlQG7VflmPqzZdug6uAP-
8. Чек-лист - как тестировать поиск? https://habr.com/ru/post/577448/
9. Ссылка на чек-лист и тест-кейс для регистрации https://docs.google.com/spreadsheets/d/1iFrmTyODtKgHnMHlqFWVVsKWqPisH_pf34TgxMvvPWA/edit?usp=sharing
10. Информация о тестовых сценариях https://www.guru99.com/test-scenario.html

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

# 21. Работа в QASE (версия 2026)

### Прочитайте перед просмотром

> Видео-часть в курсе может содержать интерфейс более ранних версий приложения. В текстовой части наиболее актуальная информация.

**Вы можете выбрать любую систему из предложенных для выполнения заданий на курсе.** Настоятельно рекомендую посмотреть видео про QASE, так как оно самое свежее и содержит примеры кейсов, рекомендации по их созданию. Test IT работает без VPN в России и Беларуси, так что вам может быть удобнее тренироваться в нем.

**Если вы научитесь работать в одной системе, то без проблем сможете переключиться на другую, так как принципы взаимодействия и функции там одинаковые. Отличается только интерфейс.**

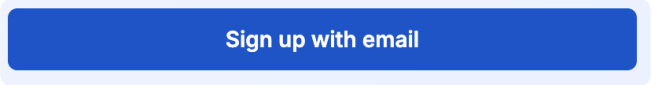
**QASE** – это современная TMS, ориентированная на скорость работы и интеграцию с процессами разработки (CI/CD). Она особенно популярна в продуктовых IT-компаниях, работающих по Agile.

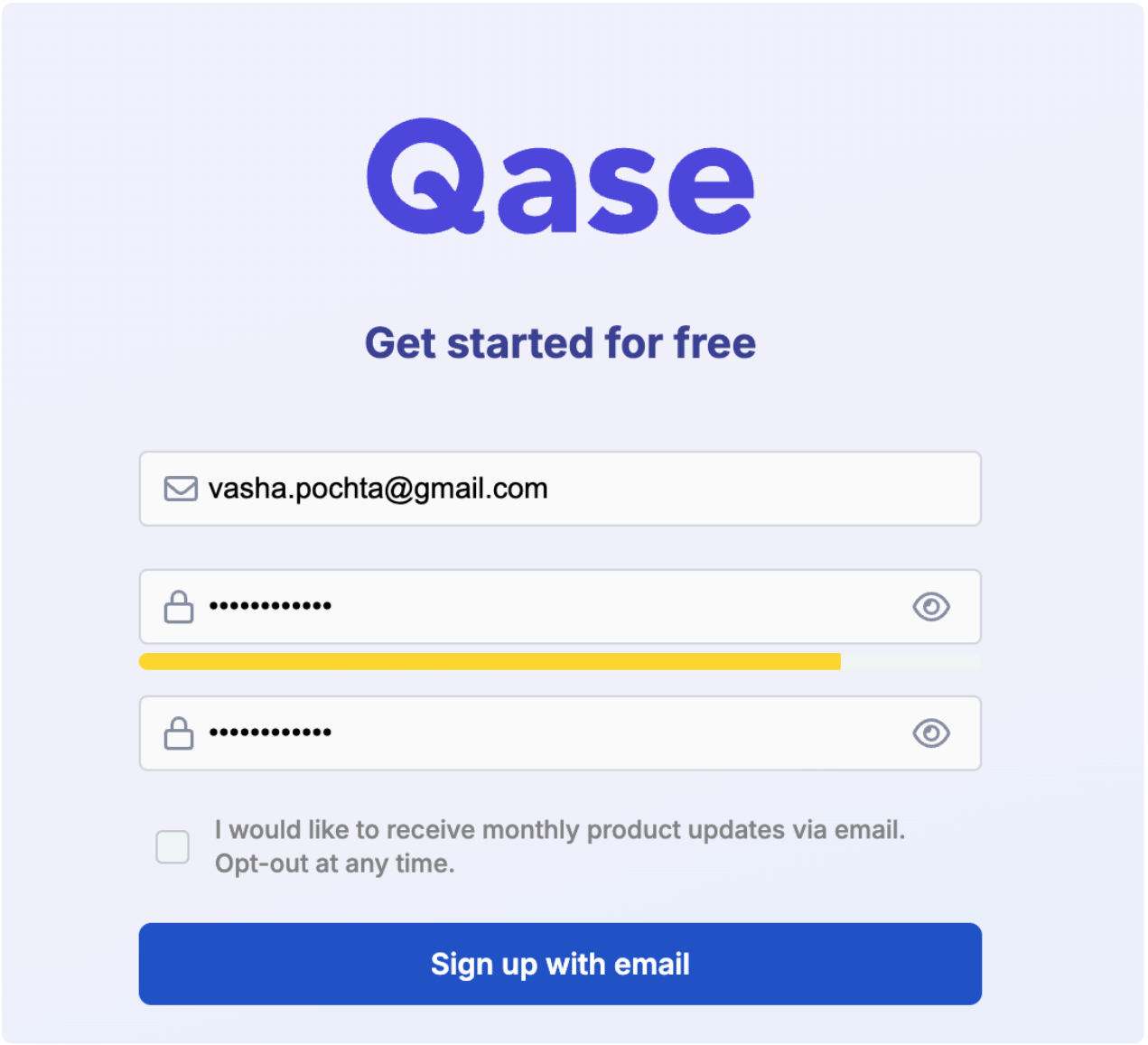
### Инструкция по регистрации и созданию окружения

1. Перейдите на https://app.qase.io/signup

2. Введите свой личный email. Придумайте и подтвердите пароль.

3. Отключите рекламные рассылки и нажмите на кнопку






4. Откройте письмо на почте и подтвердите свой email

Qase

# Complete registration

Thanks for signing up! Here’s your verification link:

Confirm email

If you don’t see a button, copy the following link and paste it in your browser:
https://app.qase.io/signup/confirm?token=2ac5fdf4617e983d28129f4fe9d01f858955b332002cca79b69a3a2e6c32

5. Заполните имя, фамилию и должность. Нажмите

Save personal information & Continue

1 About you
2 About your team
3 Invite Your Team

STEP 1 / 3

## Tell us about you

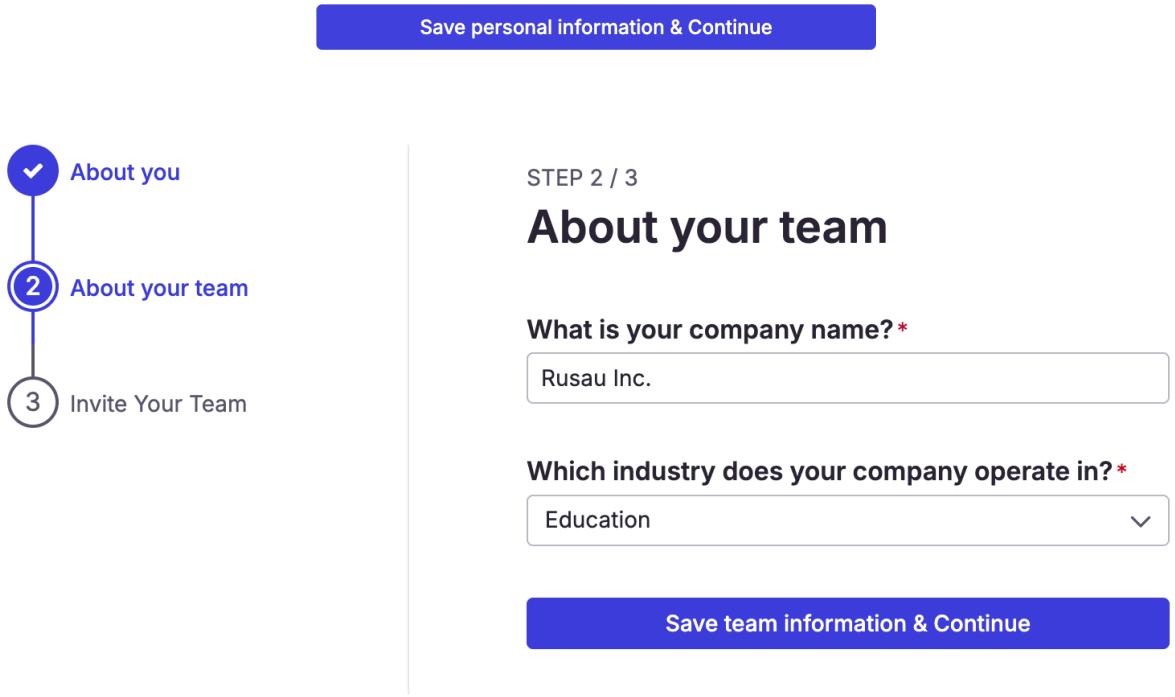
**First Name** *
Artsiom

**Last Name** *
Rusau

**Job title** *
QA Engineer

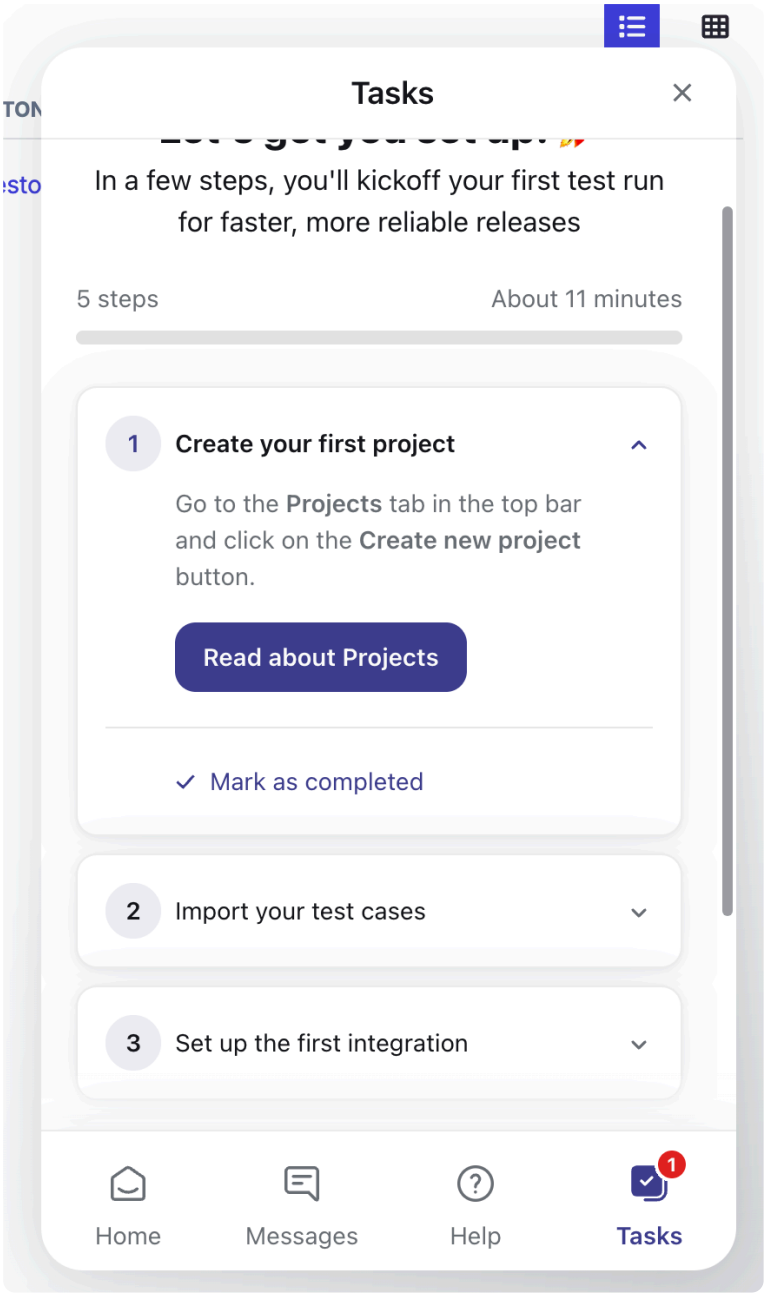
Save personal information & Continue

6. Придумайте название для компании и выберите сферу.  Нажмите

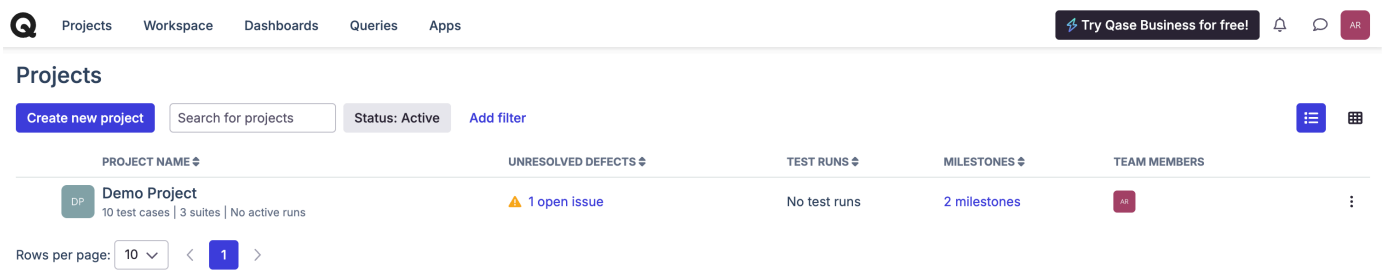



7. Можно добавить дополнительных участников команды. Они нам не нужны, так что нажмите кнопку "Later".

8. Далее вы можете пройтись по предложенным задачам для знакомства с QASE, или действовать согласно плану в этом уроке, закрыв окно ниже.




9. Поздравляю! Вы настроили свое пространство и вам доступен демо-проект!



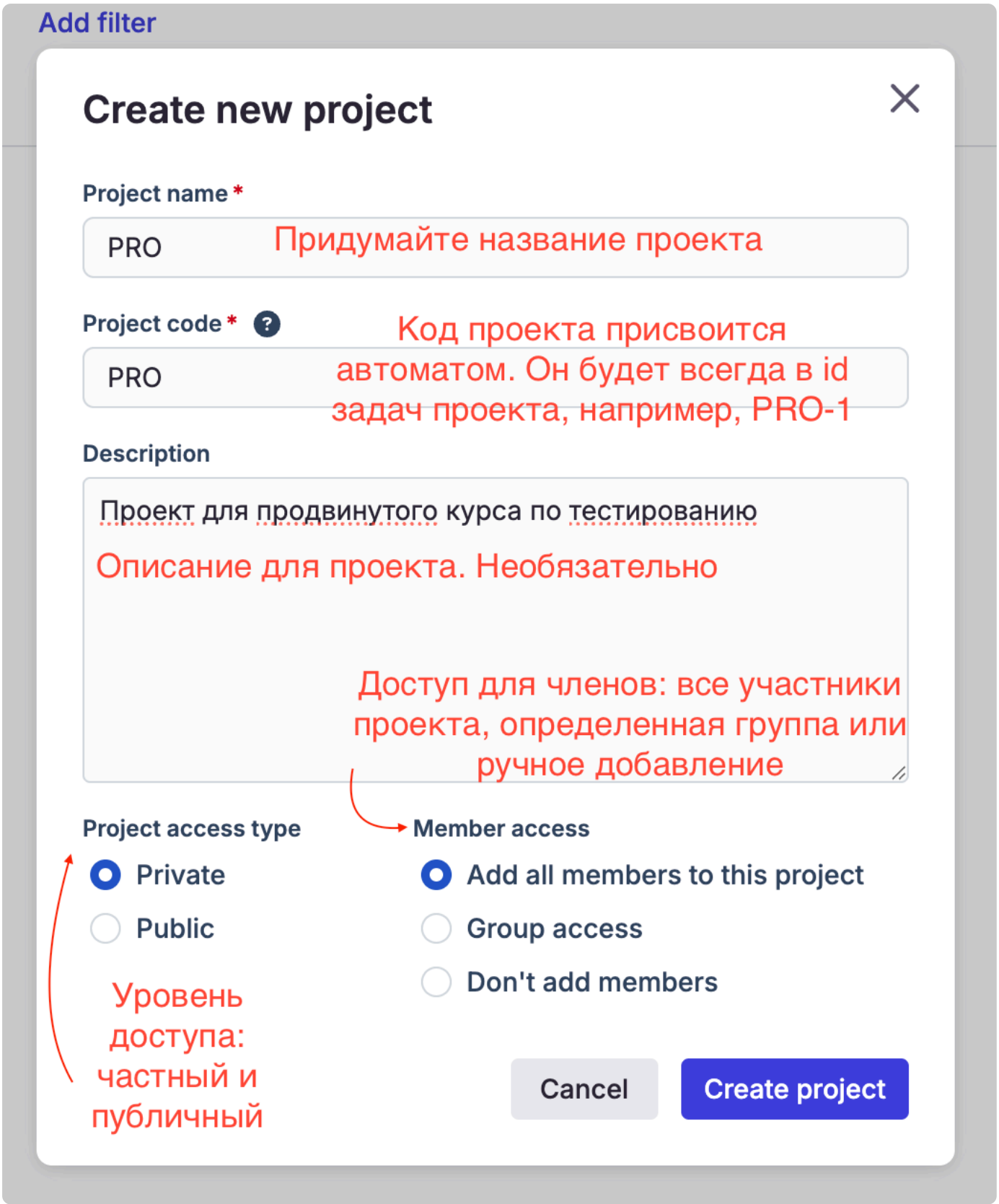

## Создание проекта

Проект объединяет репозиторий (хранилище) тестов, планы и отчеты. Это центр вашей работы. Вы можете изучить демо-проект

, который был создан автоматически, или создать свой. Я выберу второй вариант:

1. Нажмите

2. Далее заполните все обязательные поля в окне создания проекта.


Add filter
Create new project
Project name *
PRO
Придумайте название проекта
Project code *
PRO
Код проекта присвоится автоматом. Он будет всегда в id задач проекта, например, PRO-1
Description
Проект для продвинутого курса по тестированию
Описание для проекта. Необязательно
Доступ для членов: все участники проекта, определенная группа или ручное добавление
Project access type
Private
Public
Уровень доступа: частный и публичный
Member access
Add all members to this project
Group access
Don't add members
Cancel
Create project

3. Нажмите

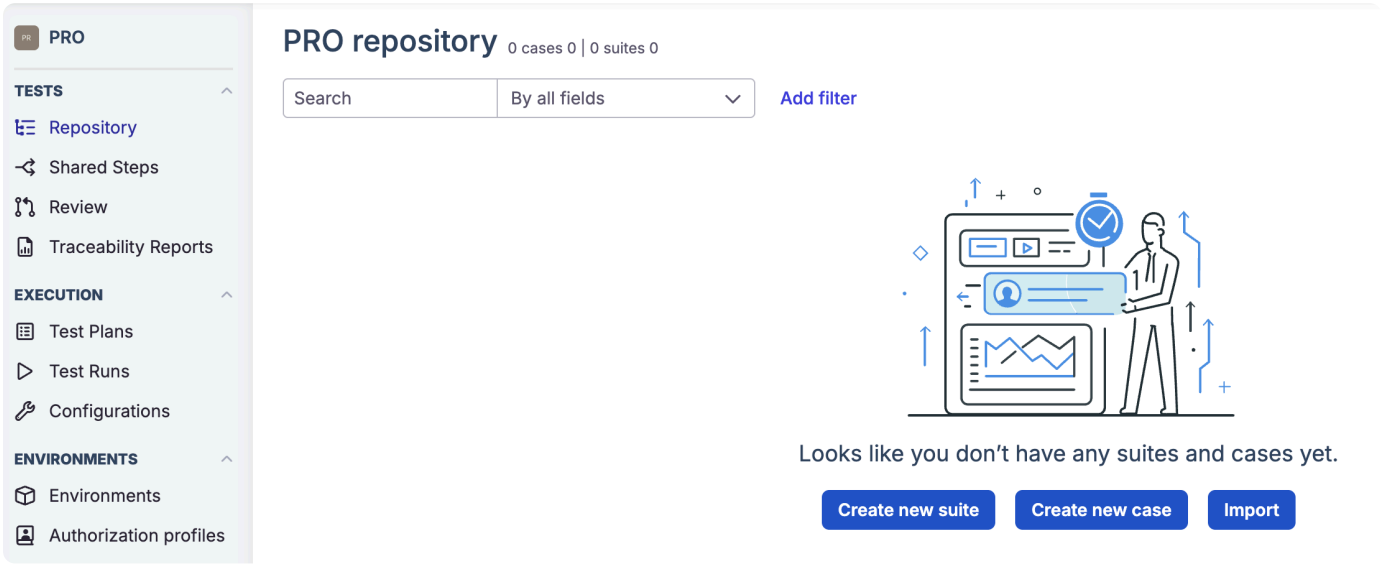
Create project

4. Поздравляю! Вы создали свой проект!

## Создание тестового набора

1. Во вкладке

мы можем создать тестовые наборы и тест-кейсы.




2. Для создания набора надо нажать

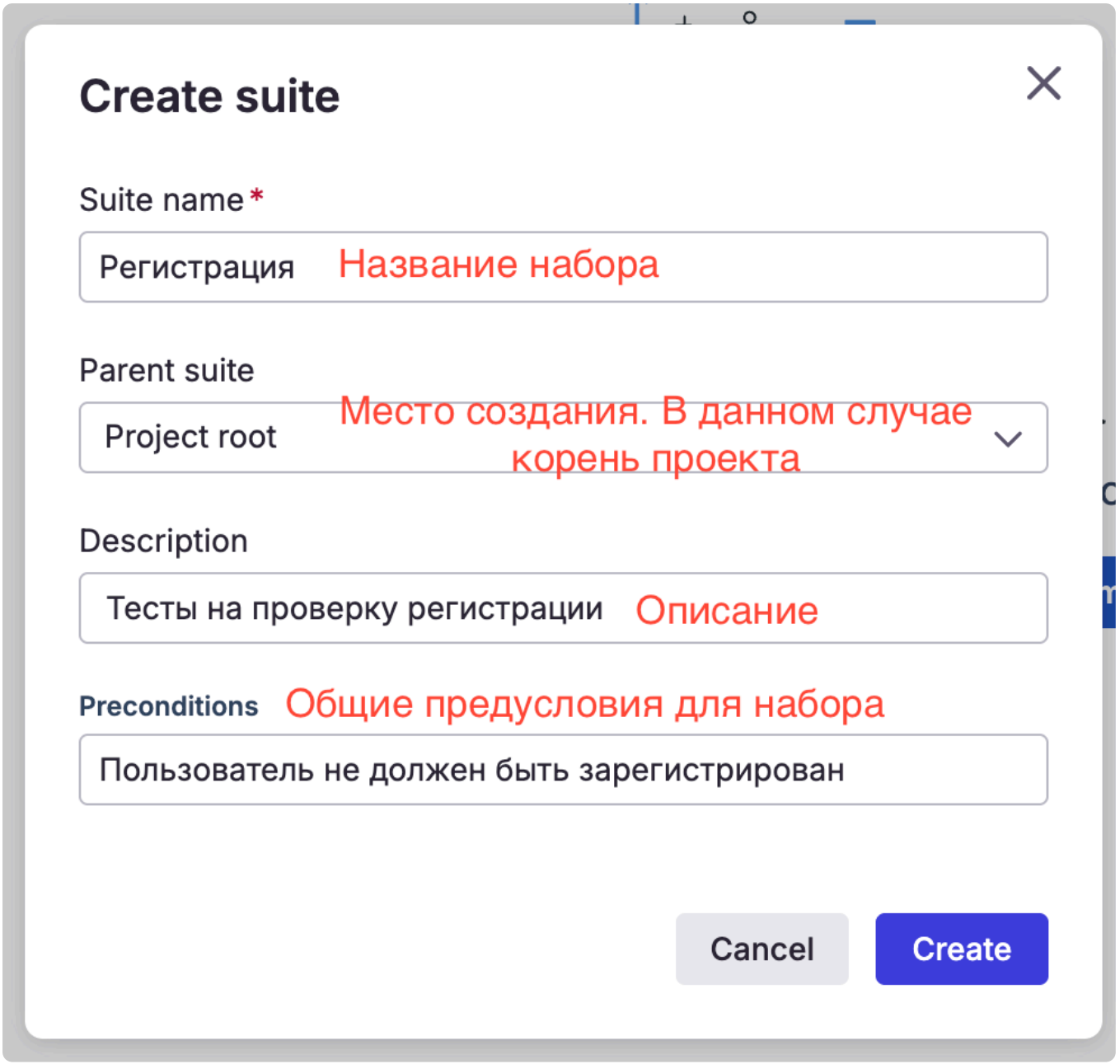
для создания набора. Для созданий кейса вне набора можно нажать

. Для импорта готовой документации

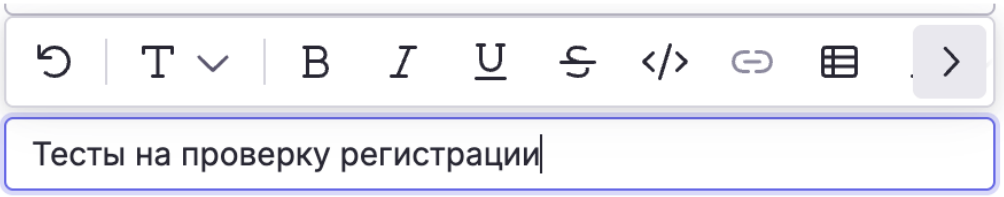
.

3. Давайте создадим набор

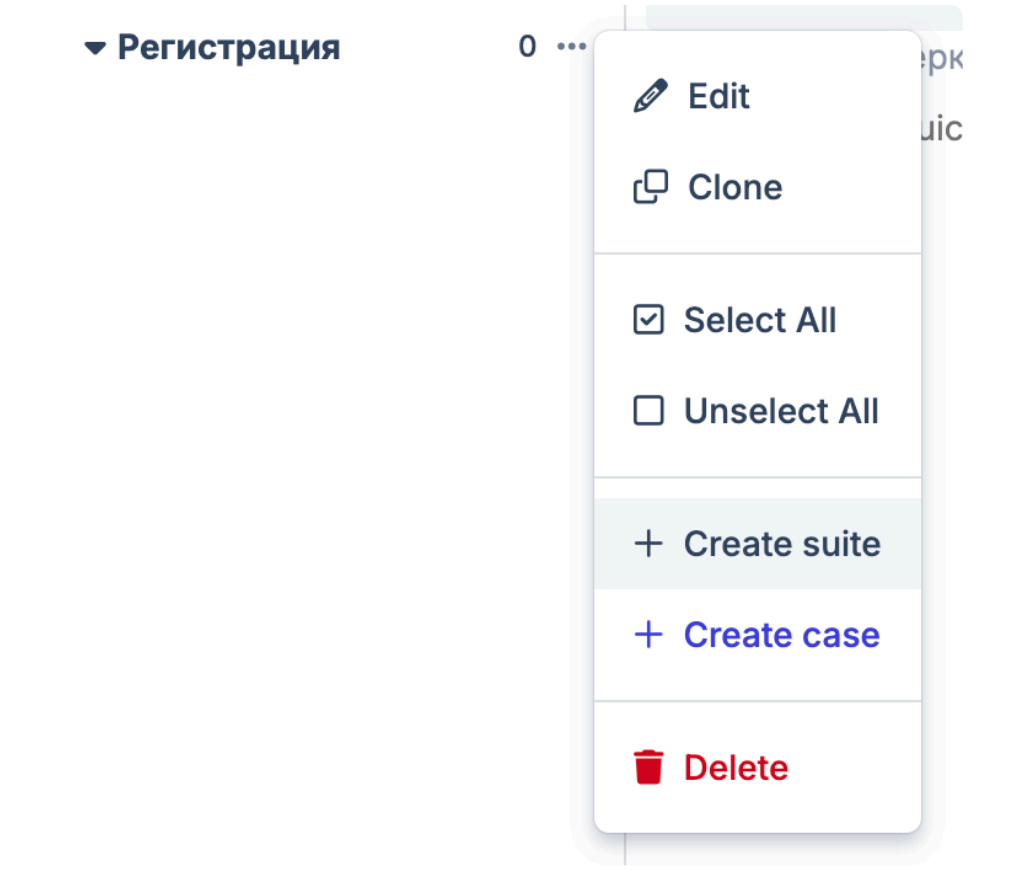
.

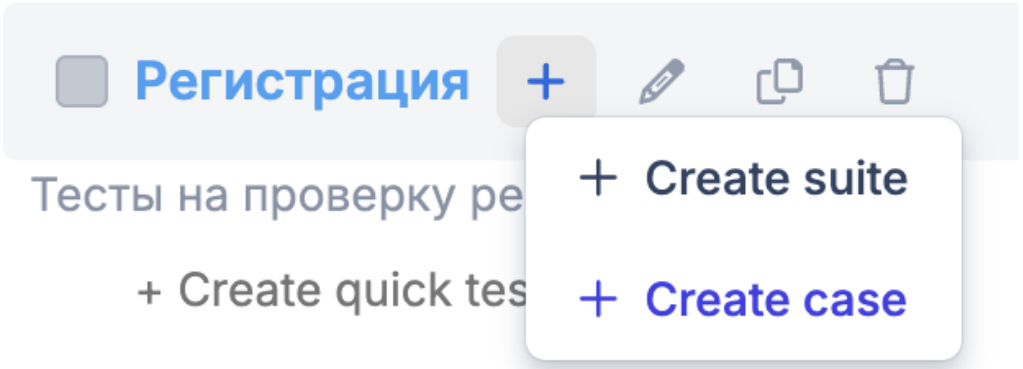



Обратите внимание, что в полях описание и предусловий можно применять различное форматирование.

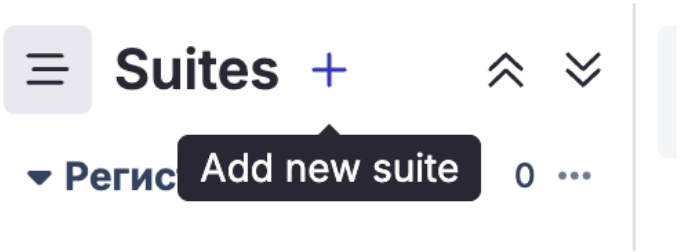



4. Далее наборы можно создавать как внутри уже созданных наборов,



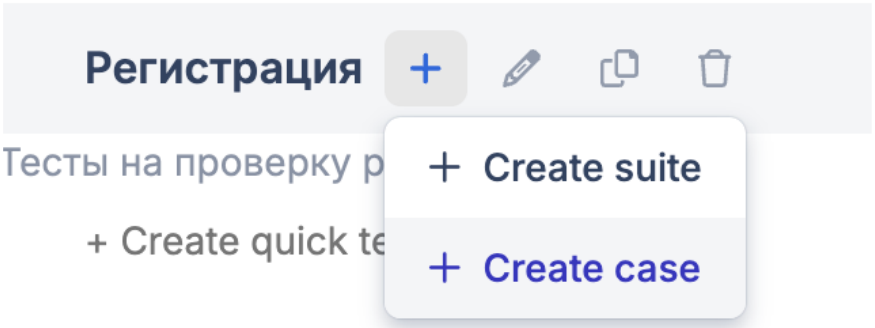




так и внутри корня проекта.

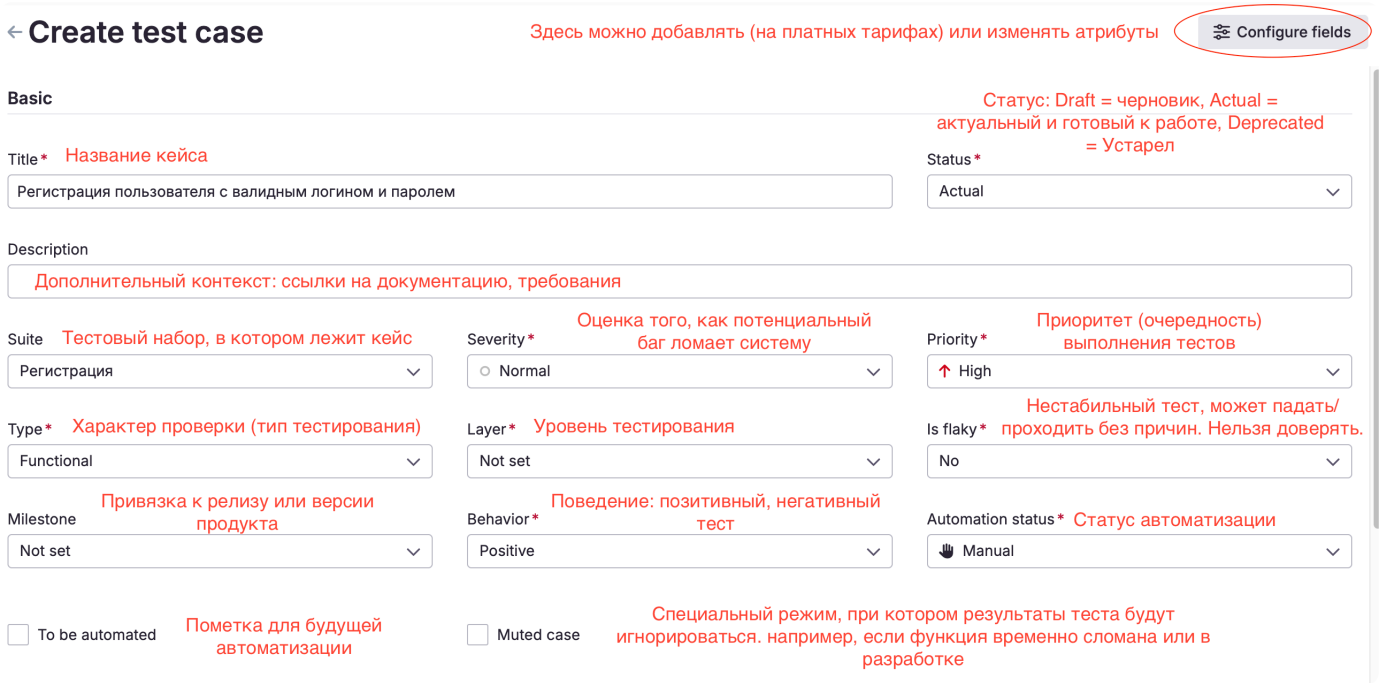


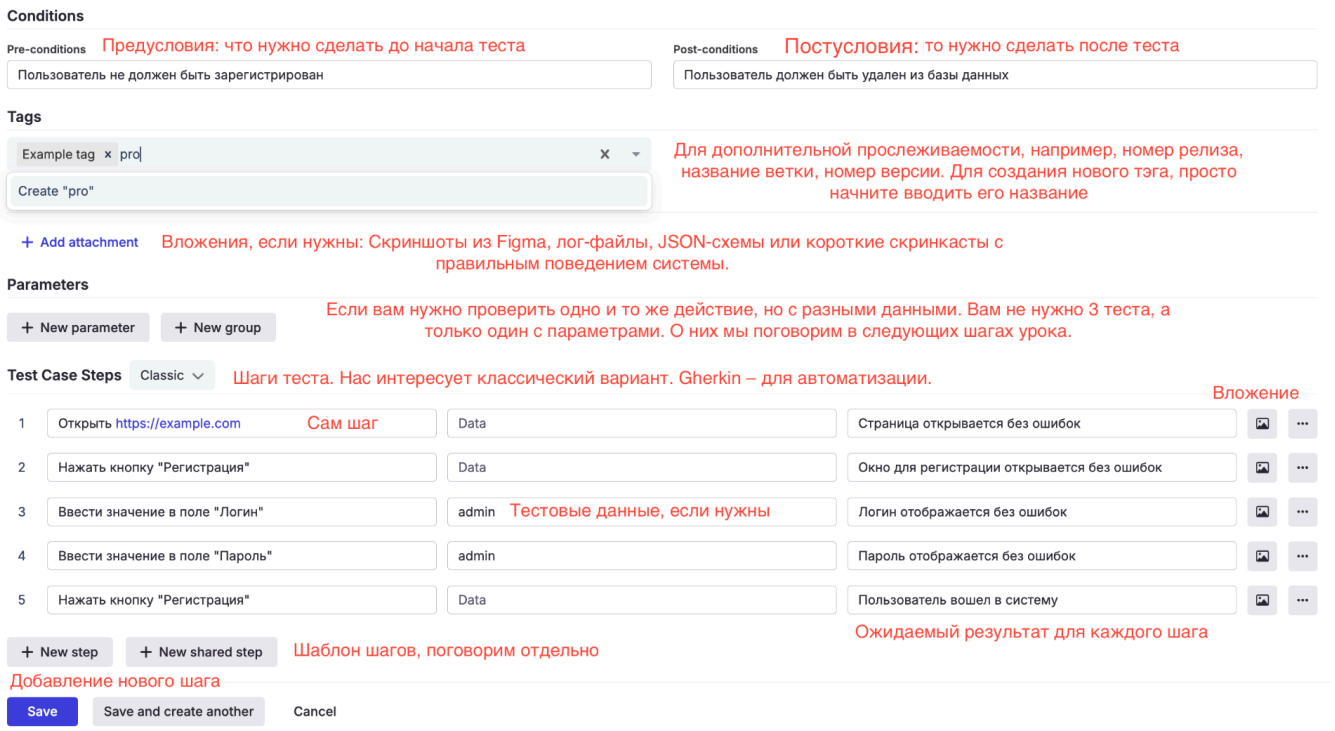

**Создание тест-кейса**

1. Внутри созданного набора выберите опцию "Create case".




2. Давайте разберемся с атрибутами кейса в QASE:






3. После заполения всех атрибутов нажмите

4. Поздравляю, вы создали свой первый тест-кейс!

Как работать с созданным кейсом?

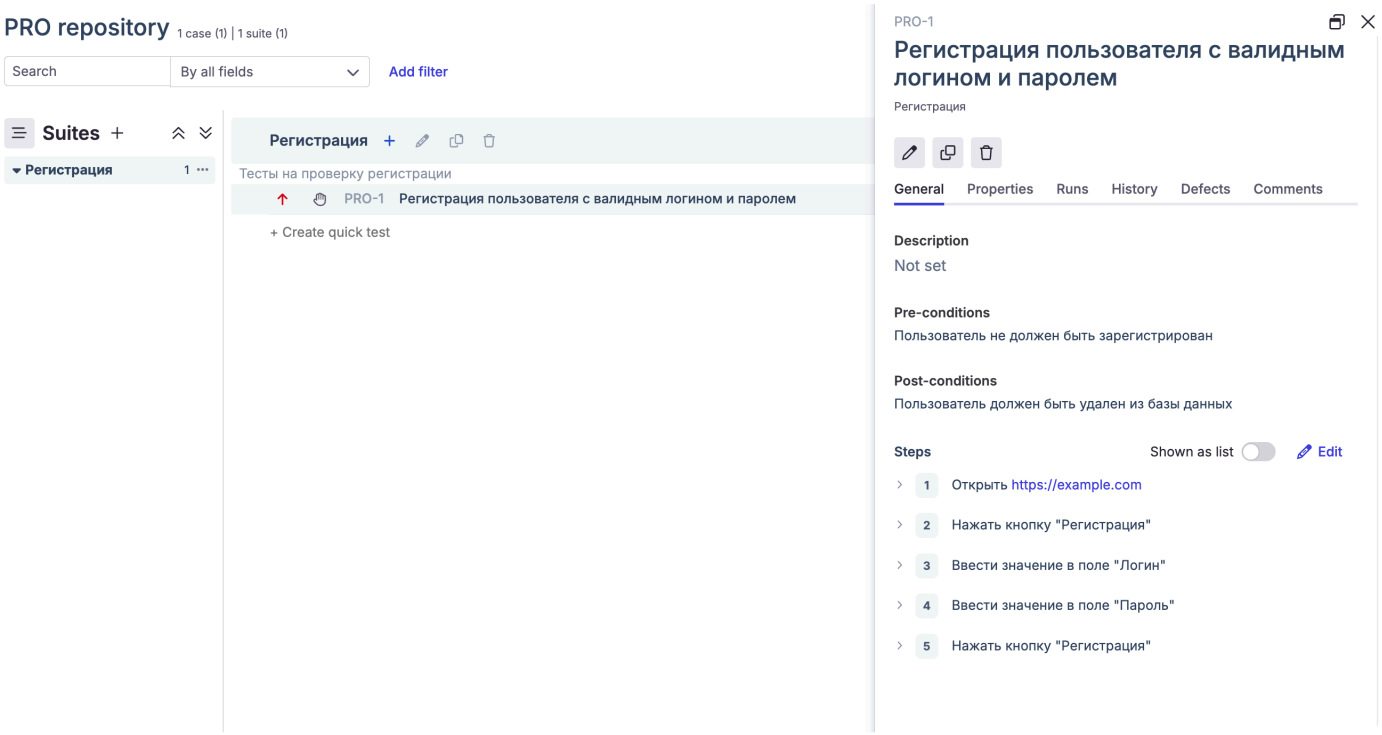
После создания кейса он будет доступно в новом окне. Давайте разберемся с его вкладками.

1. Во **вкладке General** вы будете видеть общую информацию по кейсу, с возможностью изменить

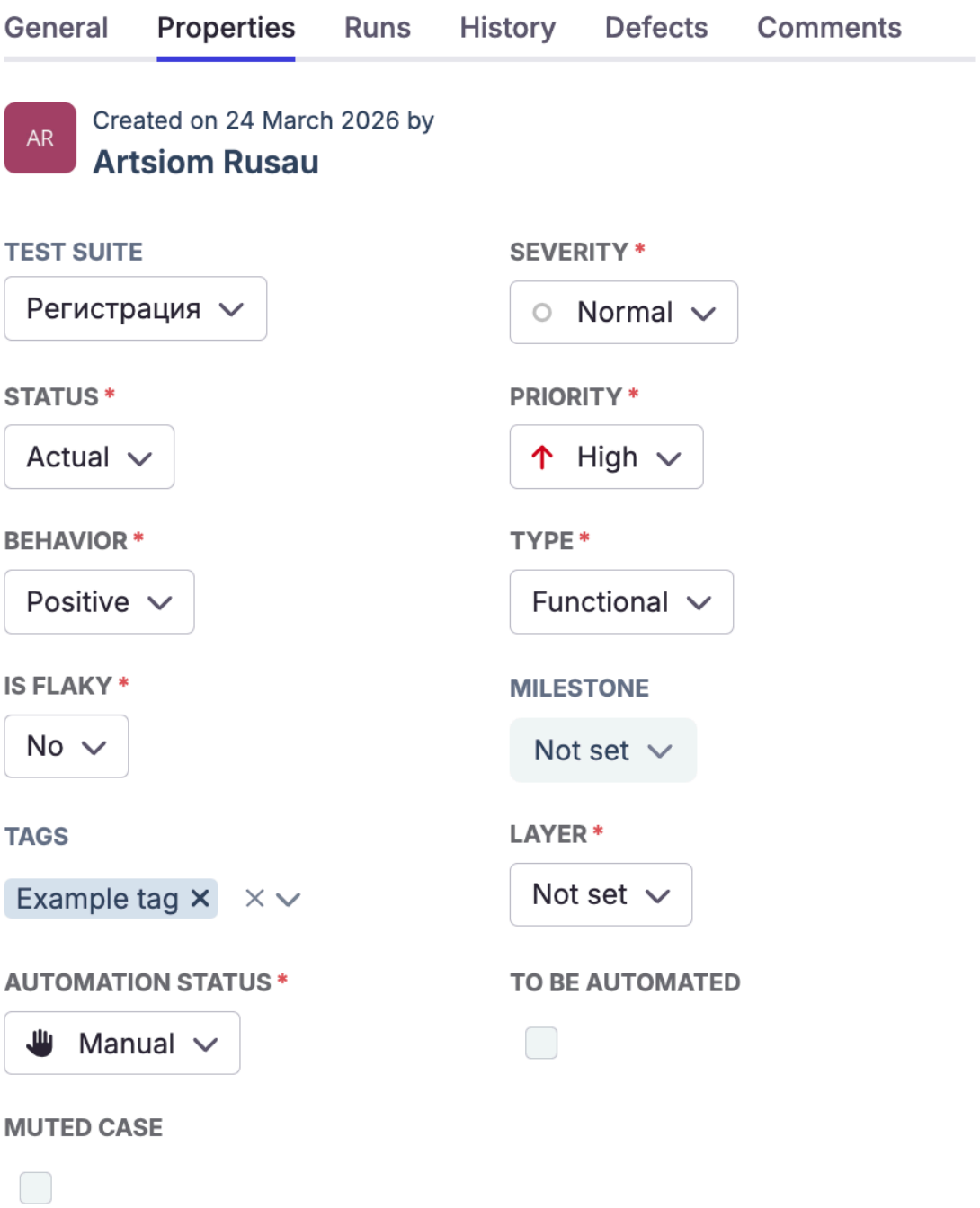
, клонировать

и удалить кейс

.

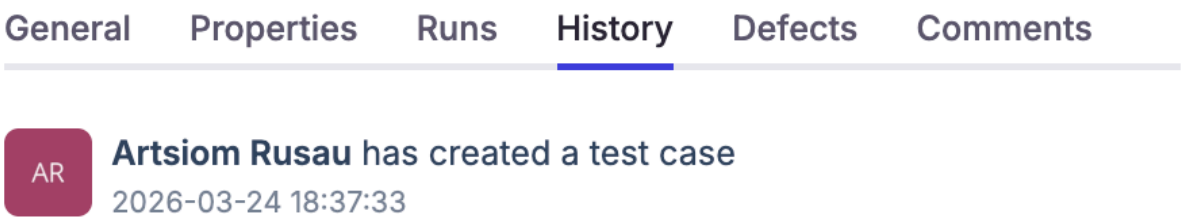



2. Во вкладке Properties можно изменить атрибуты кейса.



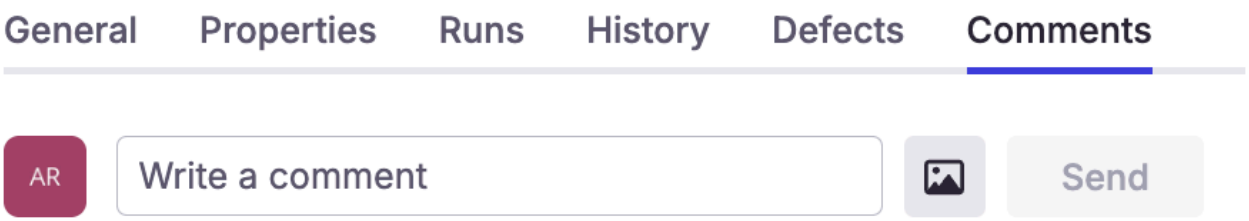

3. Во вкладке Runs можно будет увидеть историю тестовых прогонов, в которых участвовал кейс. О прогонах мы поговорим отдельно.

4. Во вкладке History можно увидеть историю изменений кейса. Всегда помните, что в любой документации видно, кто и какие изменения вносил.




5. Во вкладке Defects можно будет увидеть дефекты, которые привязаны к этому кейсу.

6. Во вкладке Comments можно оставлять комментарии к кейсу.

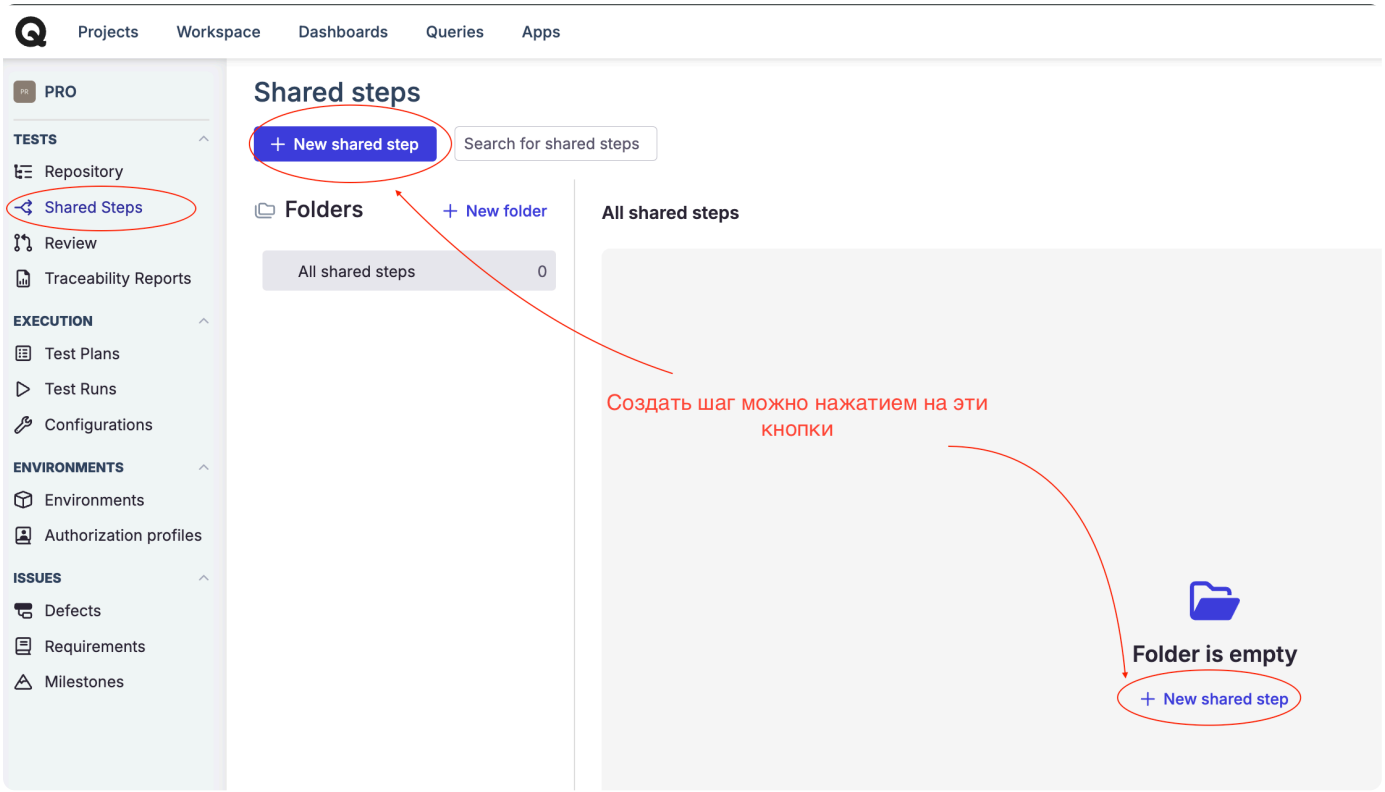



## Shared Steps (Общие шаги)

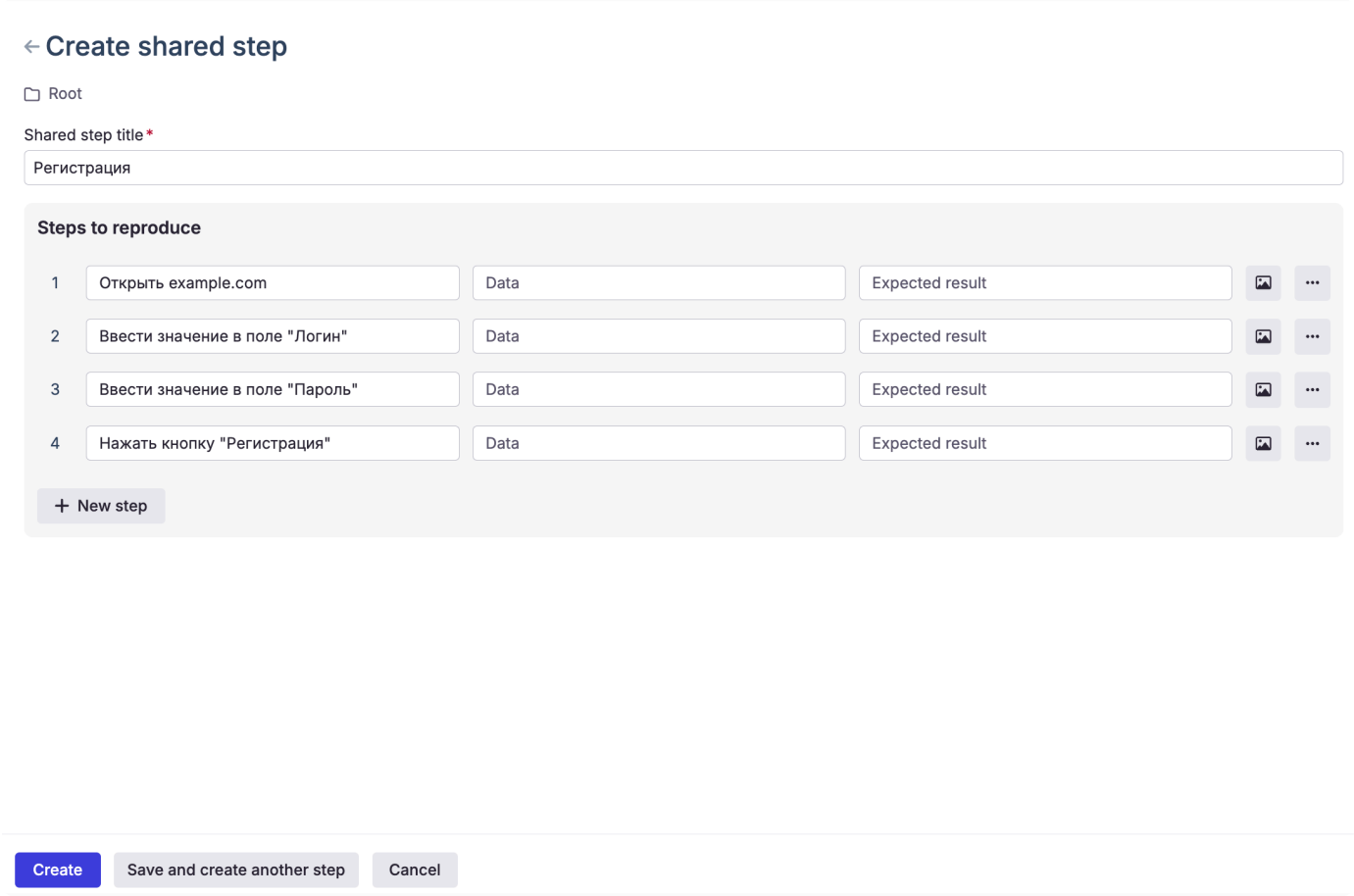
**Shared Steps (Общие шаги)** – заранее созданная группа повторяющихся шагов, которая может использоваться во многих кейсах. Это позволяет не делать однотипную работу и экономит время тестировщика.

### Создание через вкладку Shared Steps

1. Откройте окно для создания шагов, используя выделенные кнопки.

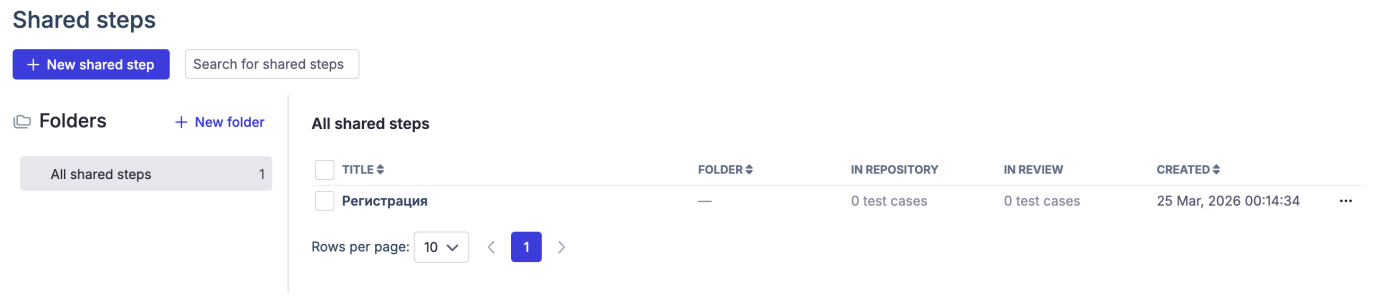



2. Создайте шаги по аналогии с реальным кейсом. Нажмите на кнопку



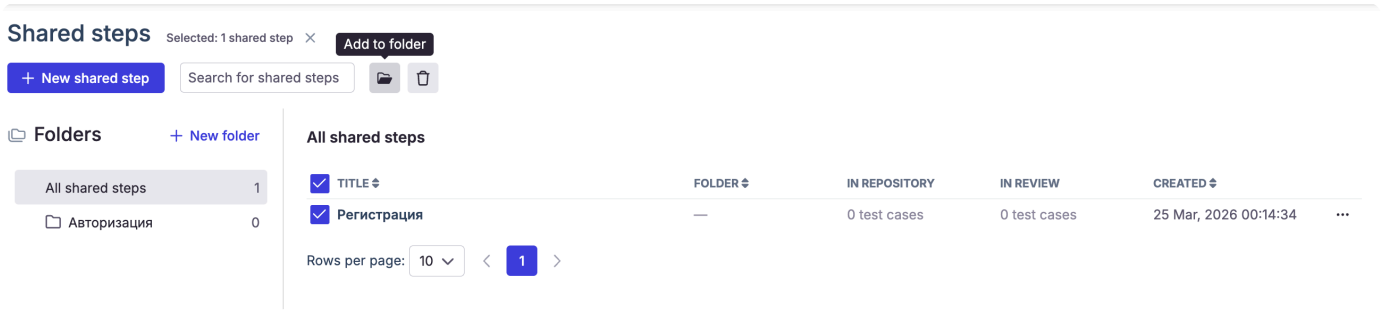

*Для примера я не заполняю Data и Expected Result, но это тоже надо сделать.*

3. Созданная группа шагов должна отобразиться в списке.




4. Также вы можете создавать папки кнопкой

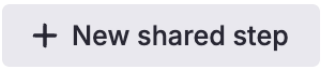
. Для перемещения в нужную папку, выберите шаги и нажмите на кнопку




## Использование общих шагов

1. Вернитесь во вкладку Repository и создайте новый кейс.

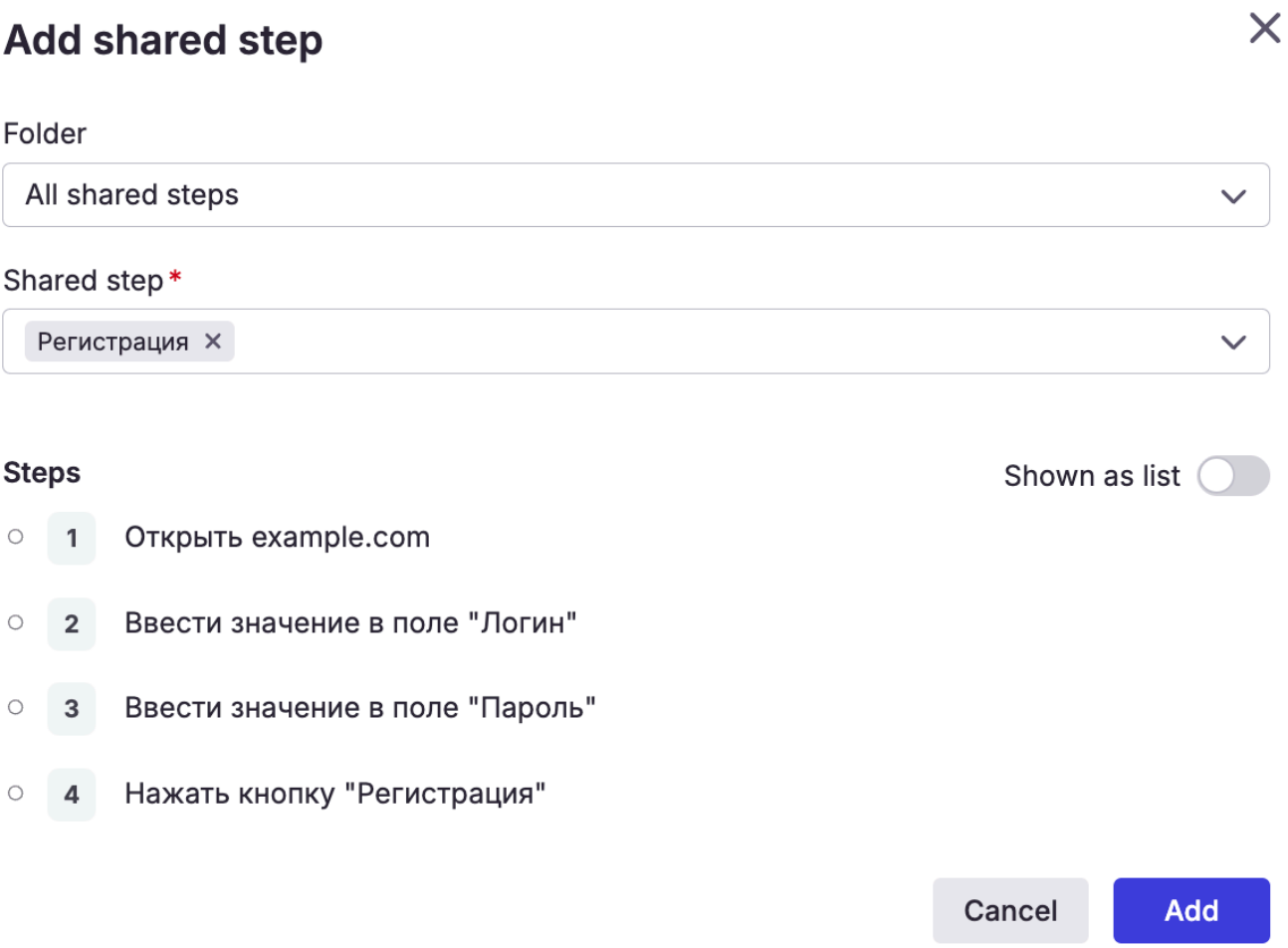
2. Перейдите в область Test Case Steps > нажмите




.

3. Выберите нужные шаги из списка и нажмите

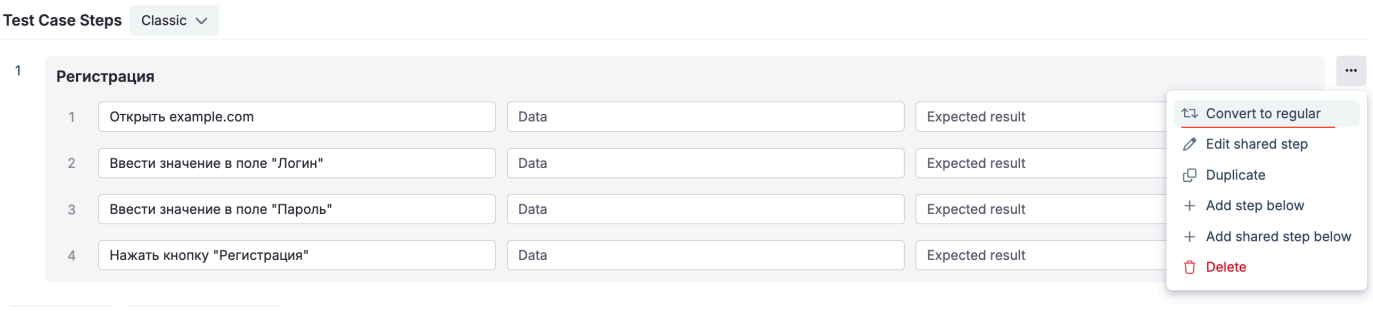
.




4. Шаги появятся в кейсе без возможности редактирования. Если вам нужно их отредактировать, то перейдите в контекстное меню



и конвертируйте их в обычные шаги (regular steps).



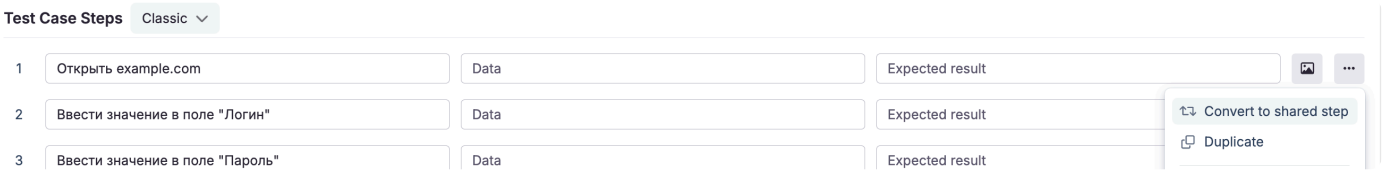

Теперь вы можете вносить любые изменения, однако они не применятся к изначальному шаблону, который мы создавали ранее.

5. Если вы хотите изменить изначальный шаблон, то выберите

Edit shared step

. Обратите внимание, что это закроет текущий кейс без сохранения и перенаправит вас на страницу Shared steps.

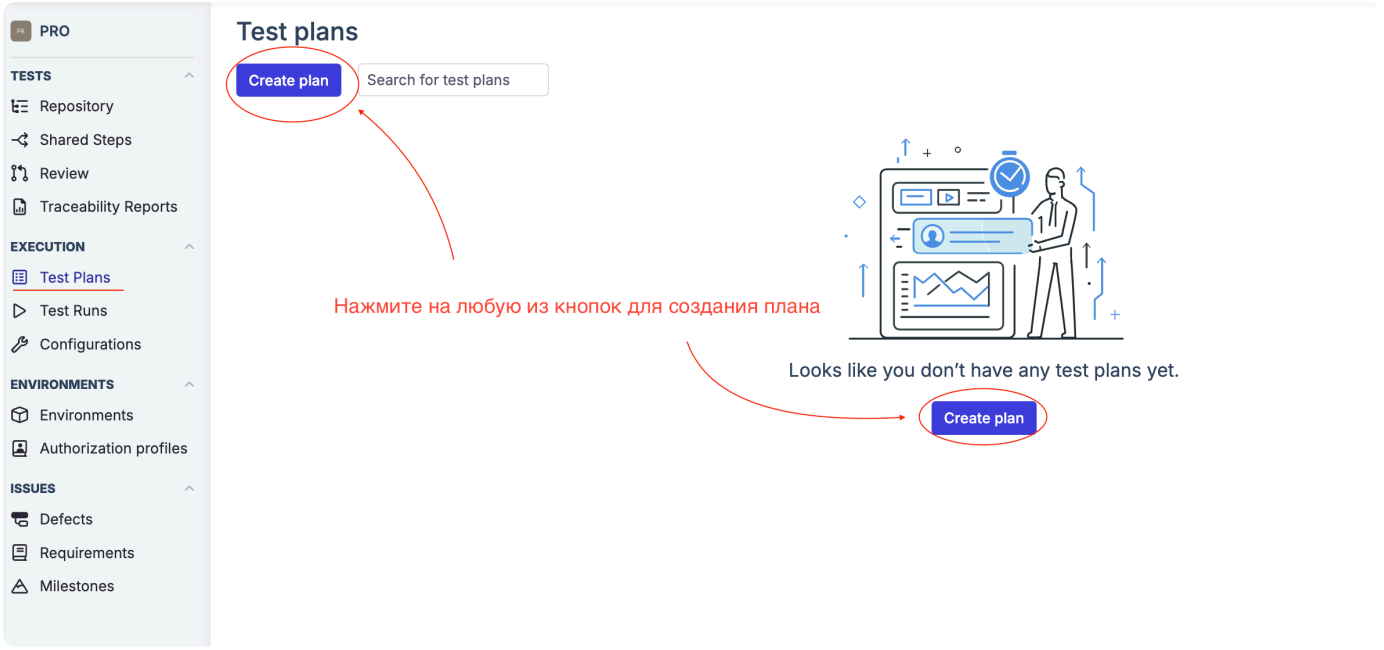
6. Также вы можете создавать Shared steps прямо из кейса. Для этого выберите следующую опцию в меню, назовите группу шагов и сохраните их в нужную папку.




**Test Plan (Тест-план)  в QASE** – это статический набор тест-кейсов из вашего общего репозитория. Он служит шаблоном для будущих проверок. Например, вы можете заранее создать подборку кейсов для Smoke, регрессии и пост-релиза, чтобы потом запускать его много раз в тестовом прогоне (Test Run).

## Создание тест-плана

1. Перейдите во вкладку Test Plan и откройте окно для его создания.




2. Введите название плана и его описание. Далее нажмите на

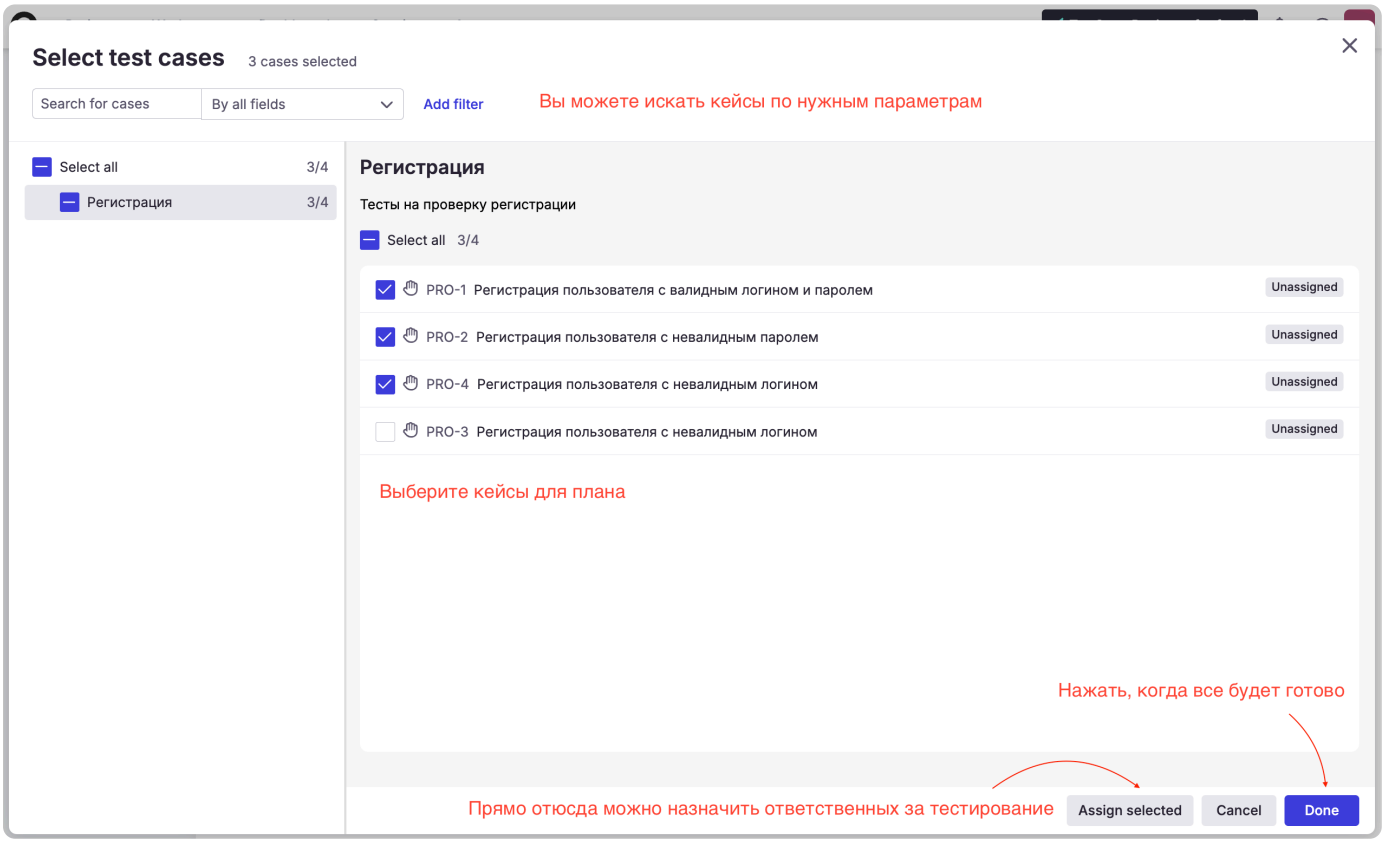
Add cases

.

3. Выберите кейсы, которые должны попасть в план и нажмите




.




4. Нажмите

.

5. Ваш план готов и может использоваться в будущих прогонах! Вы можете создать несколько планов для разных целей.

**Параметры в QASE** – это продвинутая функция, которая реализует принцип **Data-Driven Testing (тестирование на основе данных)**. Она позволяет избежать дублирования тестов, когда один и тот же сценарий нужно проверить с разными вводными данными.

Вместо того чтобы создавать 10 одинаковых кейсов для разных браузеров или типов учетных записей, вы создаете **один кейс** и привязываете к нему **параметры**.

**Создание параметров  на уровне тест-кейса**

Представьте, что вам нужно создать кейс "Регистрация с невалидным паролем". Вариантов такого пароля может быть много: граничные значения, разные типы данных. При классическом подходе вам нужно будет создать 5-10 кейсов на каждый случай. Параметры позволяют это сделать в одном кейсе, а при тестовом прогоне создать отдельный тест для каждой параметра.

Существует два типа параметров: **single и group.**

**1. Single Parameters (Одиночные параметры)**

Это независимые переменные. Когда вы добавляете несколько одиночных параметров в один тест-кейс, Qase применяет правило **декартова произведения** (перемножает все значения между собой).

- **Как это работает:** Если у вас есть параметр Login (admin) и параметр Password (пусто, кириллица, 1 символ), система создаст 3 **итерации в тестовом прогоне**:

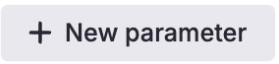
    1. admin + пусто
    2. admin + кириллица
    3. admin + 1 символ

- **Когда использовать:** Когда каждое значение одного параметра должно быть проверено с каждым значением другого. Это классическое тестирование.

**Как это сделать в кейсе?**

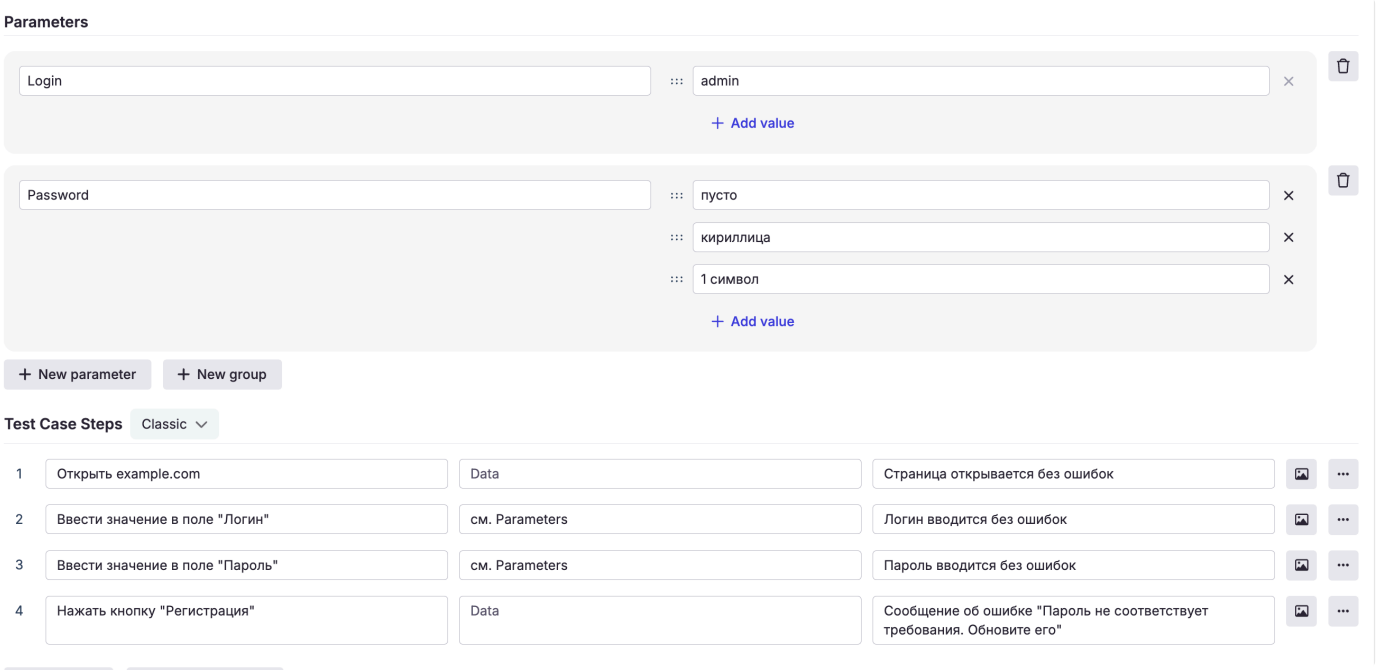
1. Создайте новый кейс.

2. Создайте одиночные параметры Login и Password, используя кнопку




.

3. В поле Data для целевых шагов укажите, что значение нужно смотреть в параметрах.




4. Сохраните кейс. Результат такого подхода мы изучим в следующих шагах.

## 2. Parameter Groups (Группы параметров)

Группы позволяют объединить несколько параметров в логические связки (строки таблицы). В этом случае система создает итерации только для тех комбинаций, которые вы явно указали.

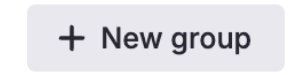
- **Как это работает:** Вы создаете группу "Аккаунты" и внутри нее определяете конкретные пары параметров `Login` и `Password`.

    - Строка 1: `admin` / `admin_pass`
    - Строка 2: `user` / `user_pass`
    - Система создаст всего **2 итерации в тестовом прогоне**. Она **не будет** пытаться подставить пароль админа к логину пользователя.

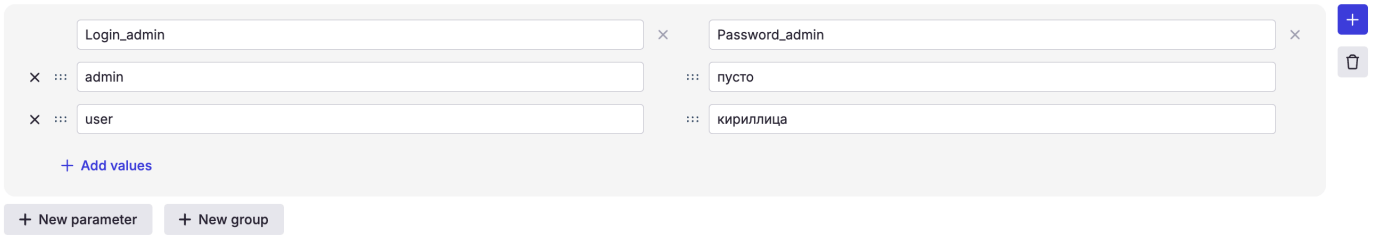
- **Когда использовать:** Для тестирования форм (логин, регистрация, заполнение анкет), где значения жестко связаны друг с другом и перемножать их бессмысленно.

**Как это сделать в кейсе?**

1. В том же кейсе создайте групповые параметры Login_Admin и Password_Admin, используя кнопку




.




2. Сохраните кейс. Результат такого подхода мы изучим в следующих шагах.

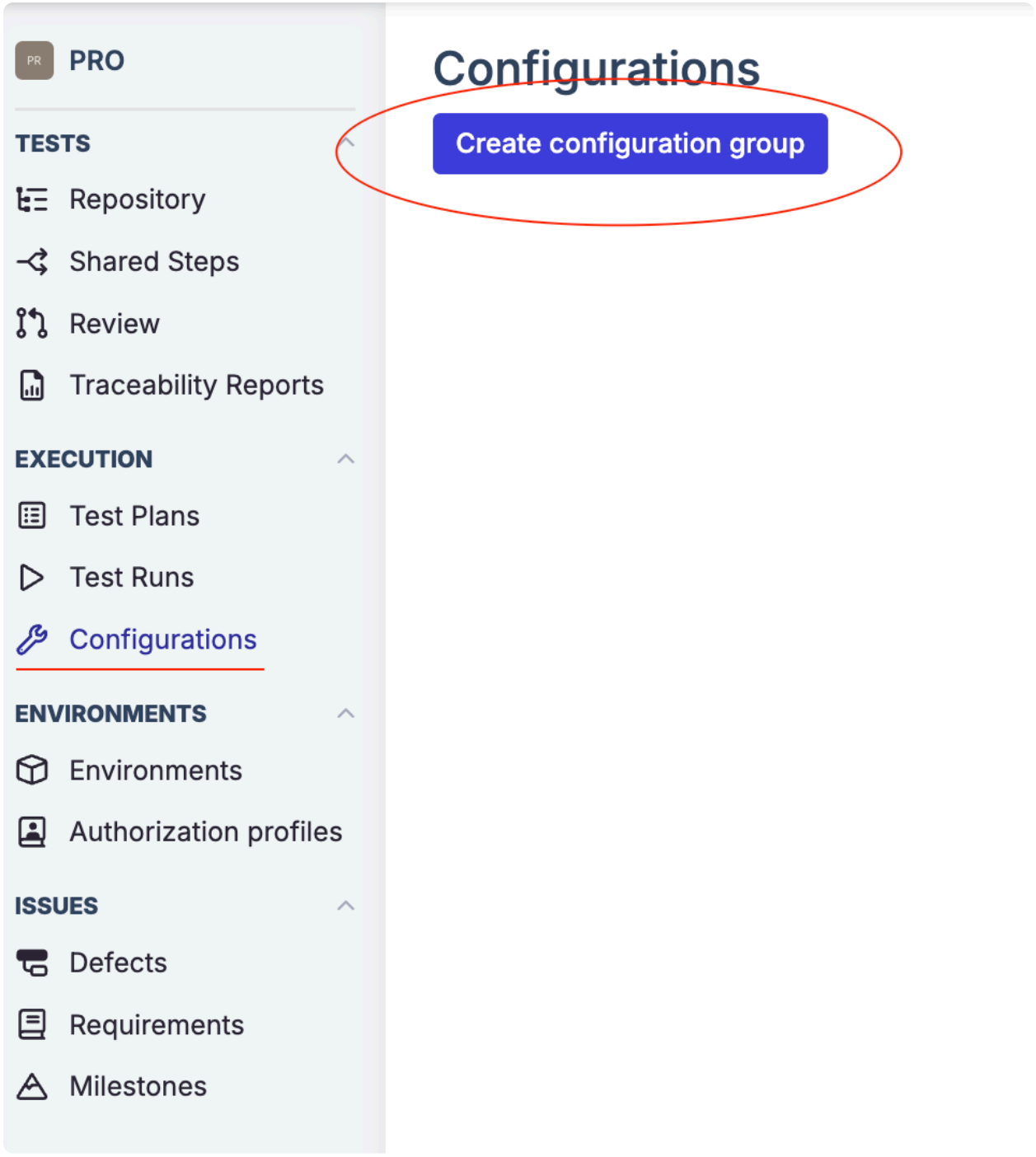
**Configurations (конфигурации)** — это набор технических характеристик на стороне пользователя или оборудования, в которых должен быть выполнен тест.

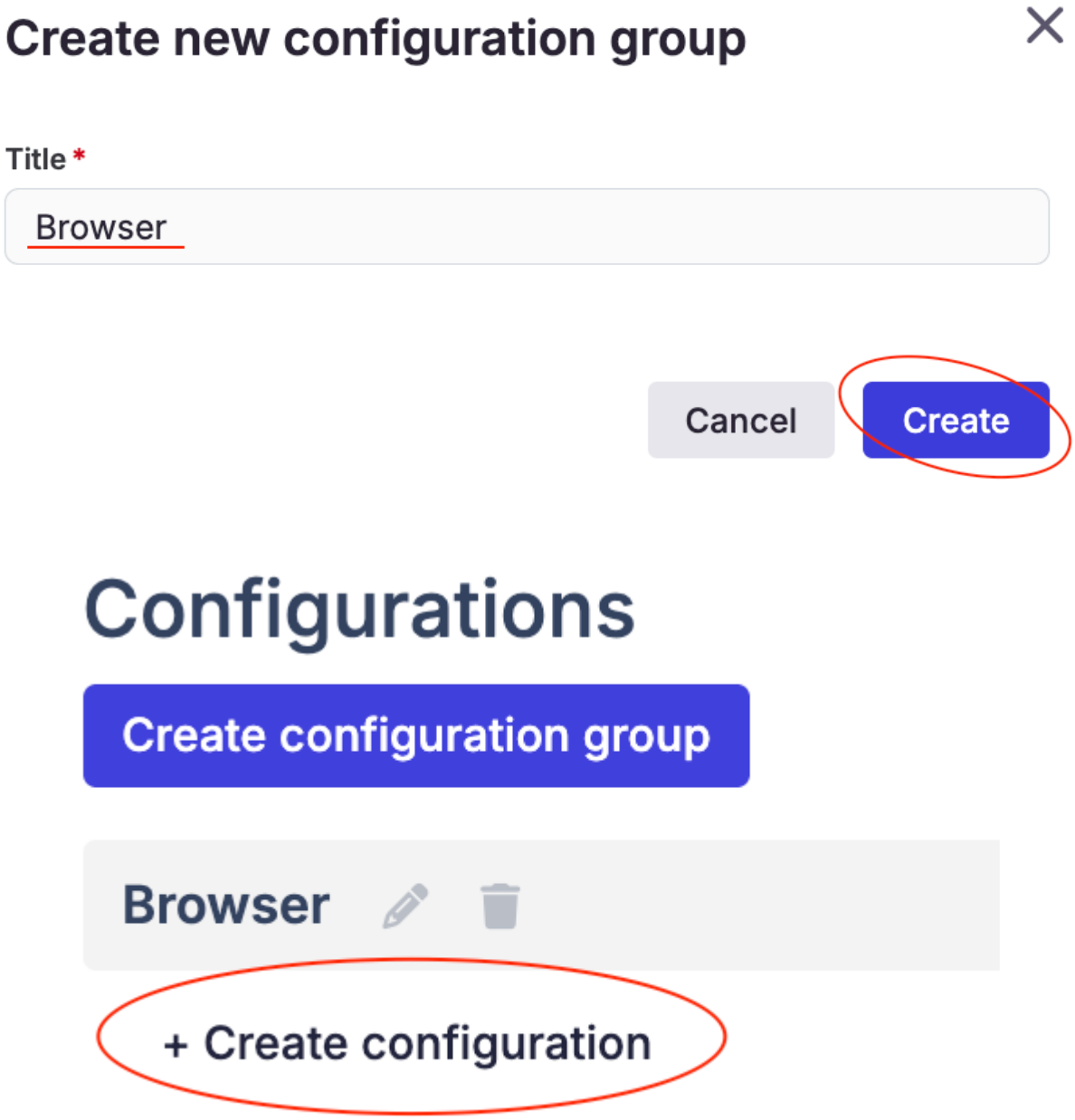
**Как это работает?**

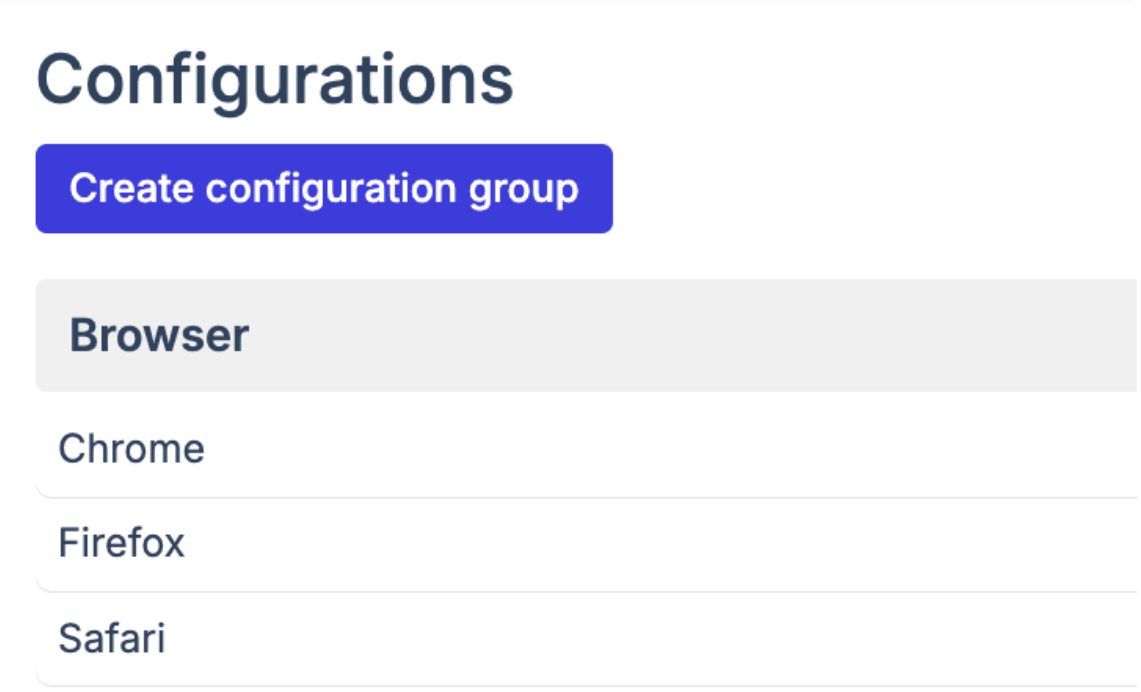
Конфигурации в Qase работают через **Configuration Groups** (Группы).

- **Группа:** Browser. **Конфигурации:** Chrome, Firefox, Safari.
- **Группа:** OS. **Конфигурации:** Windows 11, macOS Sequoia.

1. Перейдите во вкладку Configarations и создайте первую конфигурацию.


PR
PRO
TESTS
Repository
Shared Steps
Review
Traceability Reports
EXECUTION
Test Plans
Test Runs
Configurations
ENVIRONMENTS
Environments
Authorization profiles
ISSUES
Defects
Requirements
Milestones
Configurations
Create configuration group


Create new configuration group
Title *
Browser
Cancel
Create
Configurations
Create configuration group
Browser
+ Create configuration




2. Самостоятельно создайте еще одну группу **OS** со значениями: *Windows 11, macOS Sequoia.*

3. Далее эти конфигурации будут использоваться в тестовом прогоне.

**Environments (Окружения)** – это конкретные стенды (серверы), на которых развернуто ваше приложение.

**Как это работает в QASE?**

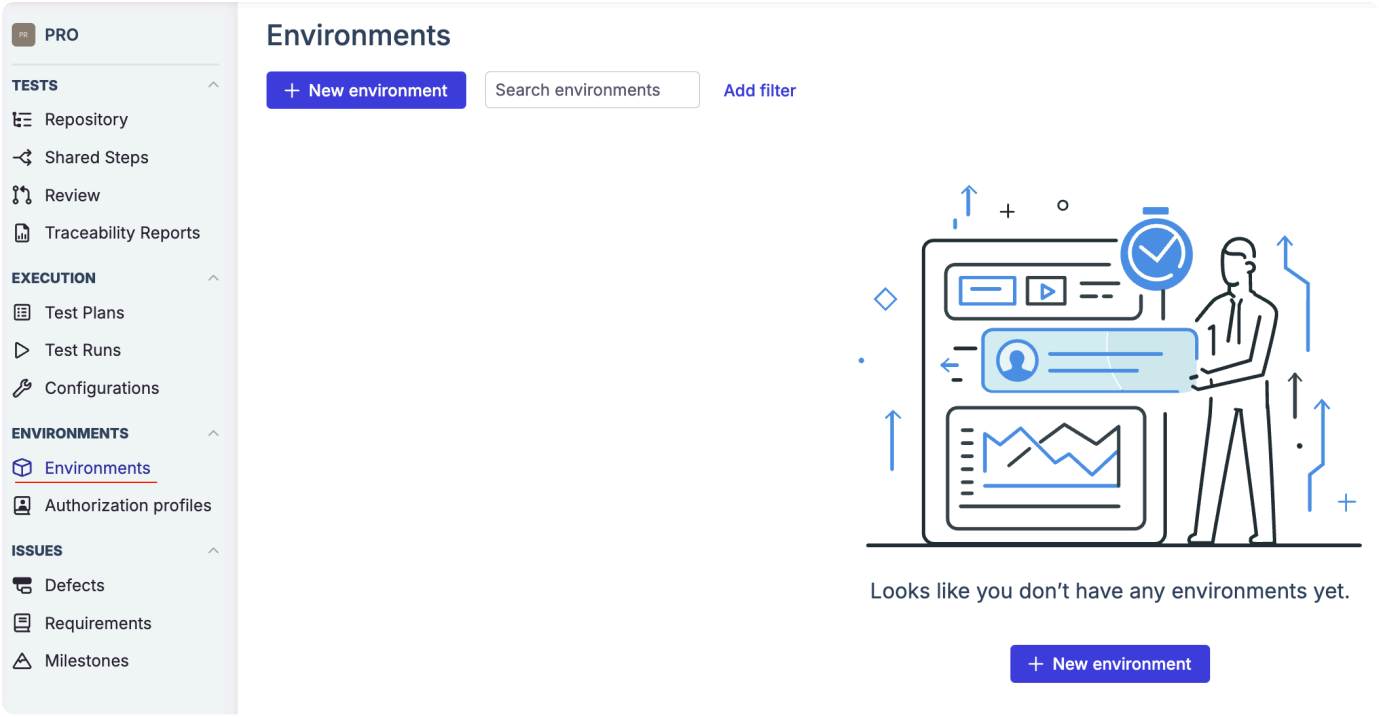
Окружение указывает, где именно тестировщик проводит проверку.

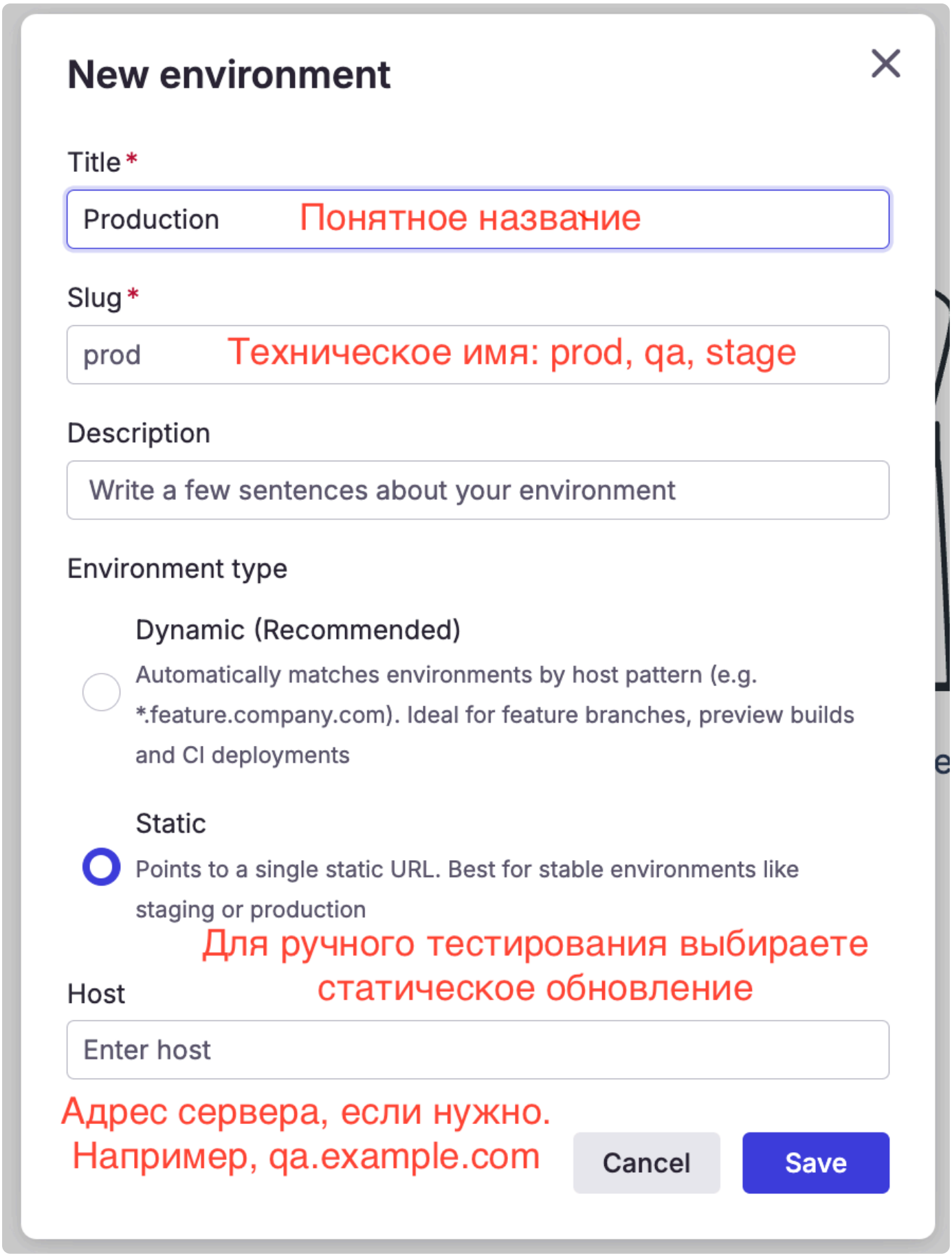
**Типичные примеры окружений:**

- **Dev:** Стенд для разработчиков (самый нестабильный).
- **QA:** Стенд для основного тестирования.
- **Staging / Pre-prod:** Копия реальной системы для финальных тестов и регрессии.
- **Production:** Реальная система, где работают пользователи.

1. Перейдите во вкладку Environments и создайте несколько окружений, нажав

.


PRO
TESTS
Repository
Shared Steps
Review
Traceability Reports
EXECUTION
Test Plans
Test Runs
Configurations
ENVIRONMENTS
Environments
Authorization profiles
ISSUES
Defects
Requirements
Milestones
Environments
New environment
Search environments
Add filter
Looks like you don't have any environments yet.
New environment




2. Самостоятельно создайте окружения QA и Stage. Host можно не указывать.

3. Далее эти окружения будут использоваться в тестовом прогоне.

**Milestone (веха, ключевая точка)** – это временной контейнер, который привязывает вашу работу к календарю и бизнес-целям.

**Зачем их использовать:**

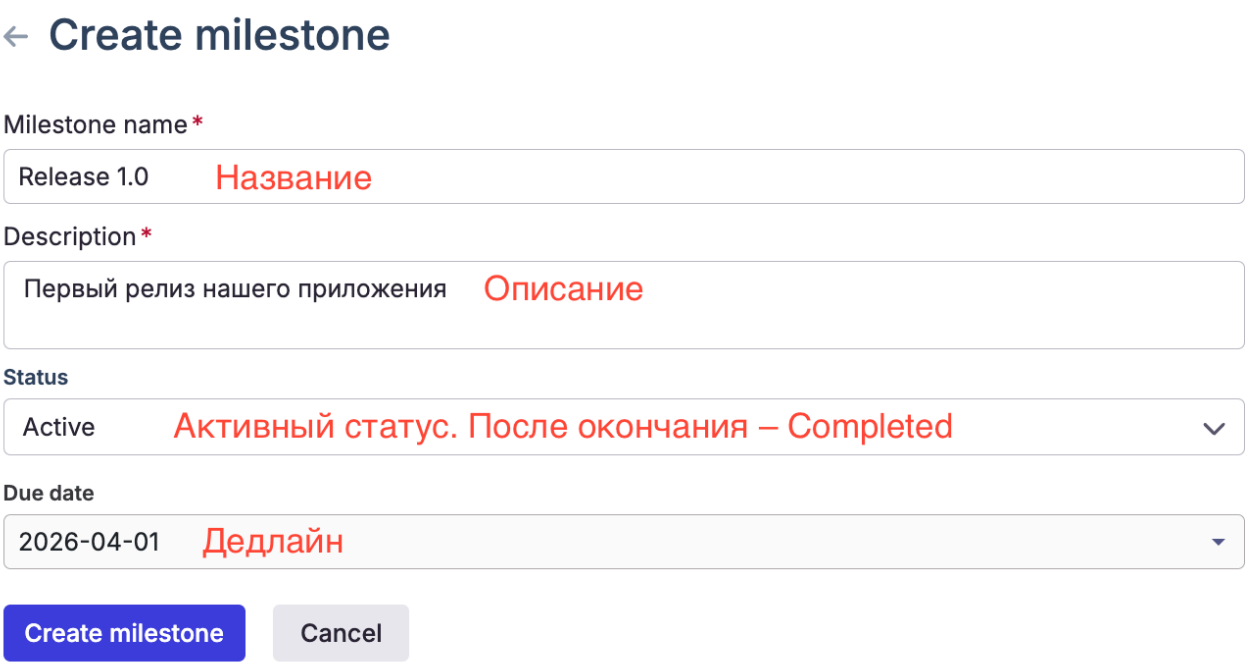
- **Отслеживание прогресса релиза:** Вы видите общую картину: сколько тестов пройдено во всех ранах, относящихся к этой версии.
- **Соблюдение дедлайнов:** Система подсвечивает, сколько времени осталось до завершения этапа.
- **Группировка данных:** Позволяет отделить тесты "Спринта №15" от "Спринта №16".

**Как их использовать в QASE?**

1. Перейдите во вкладку Milestones и создайте первую веху, нажав кнопку

.

2. Заполните нужные данные и нажмите Create.



3. Теперь вы можете выбирать эту веху при запуске тестового прогона.

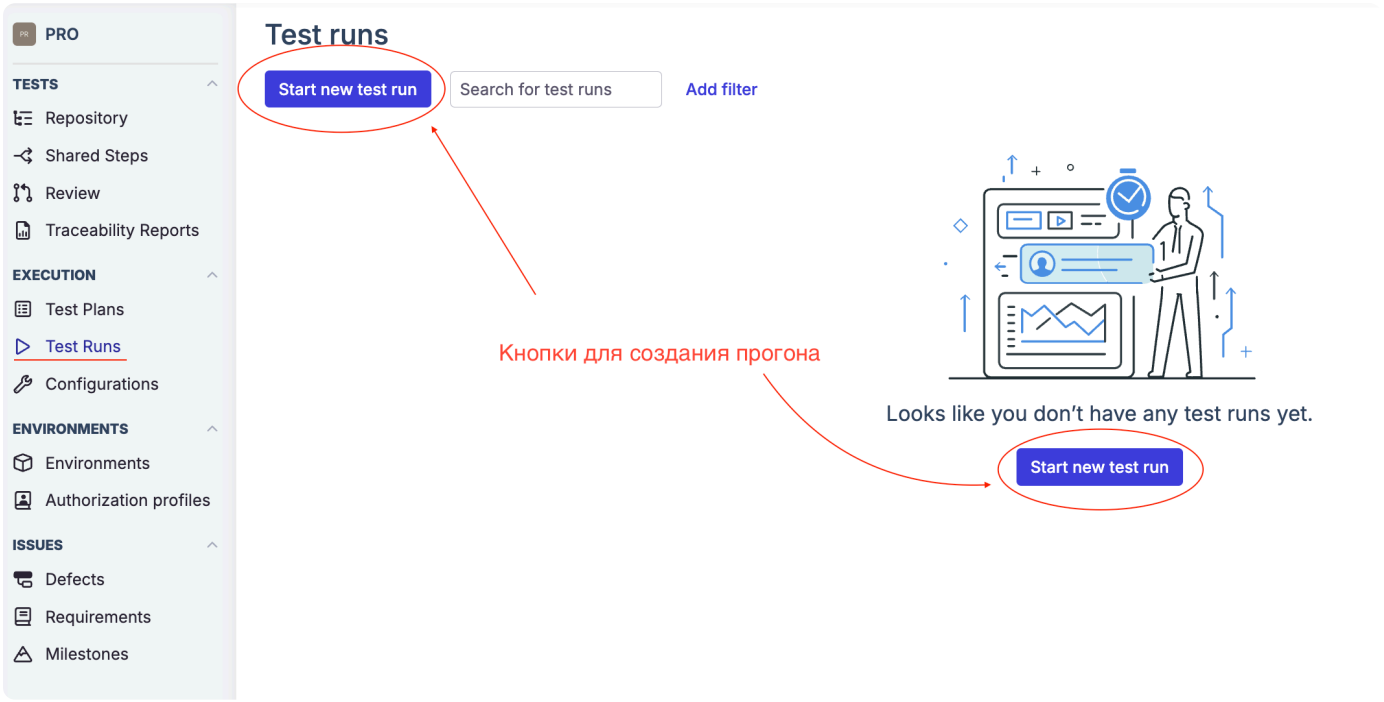
**Test Run (Тестовый прогон)  в QASE** – это **активная стадия** тестирования. Когда вы запускаете Test Run, вы создаете "копию" тестов, в которой будете проставлять результаты (прошел/упал).

Давайте используем все, что мы делали в рамках всего урока для запуска первого прогона.

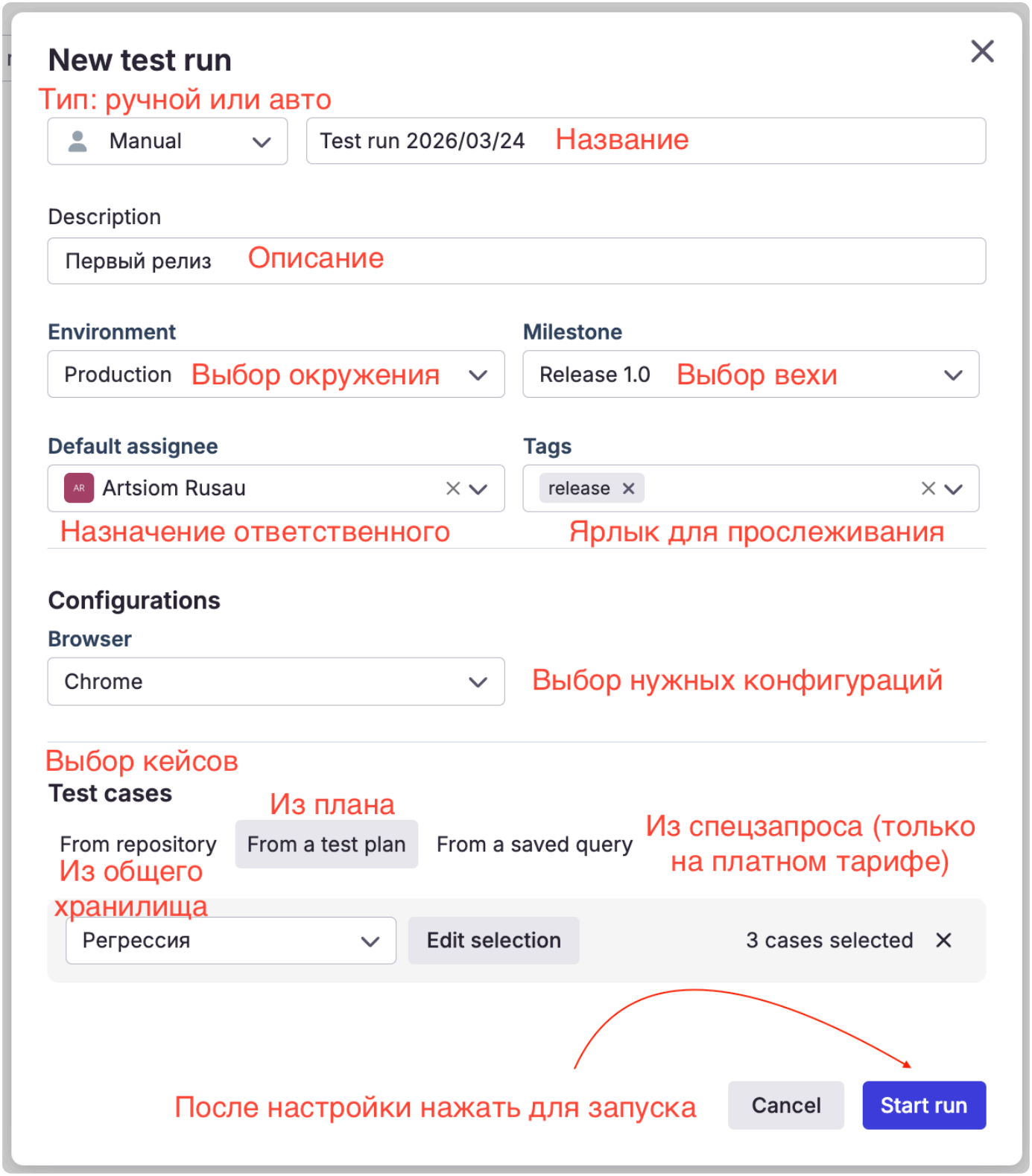
## Создание прогона

1. Перейдите во вкладку Test Runs и нажмите на кнопку

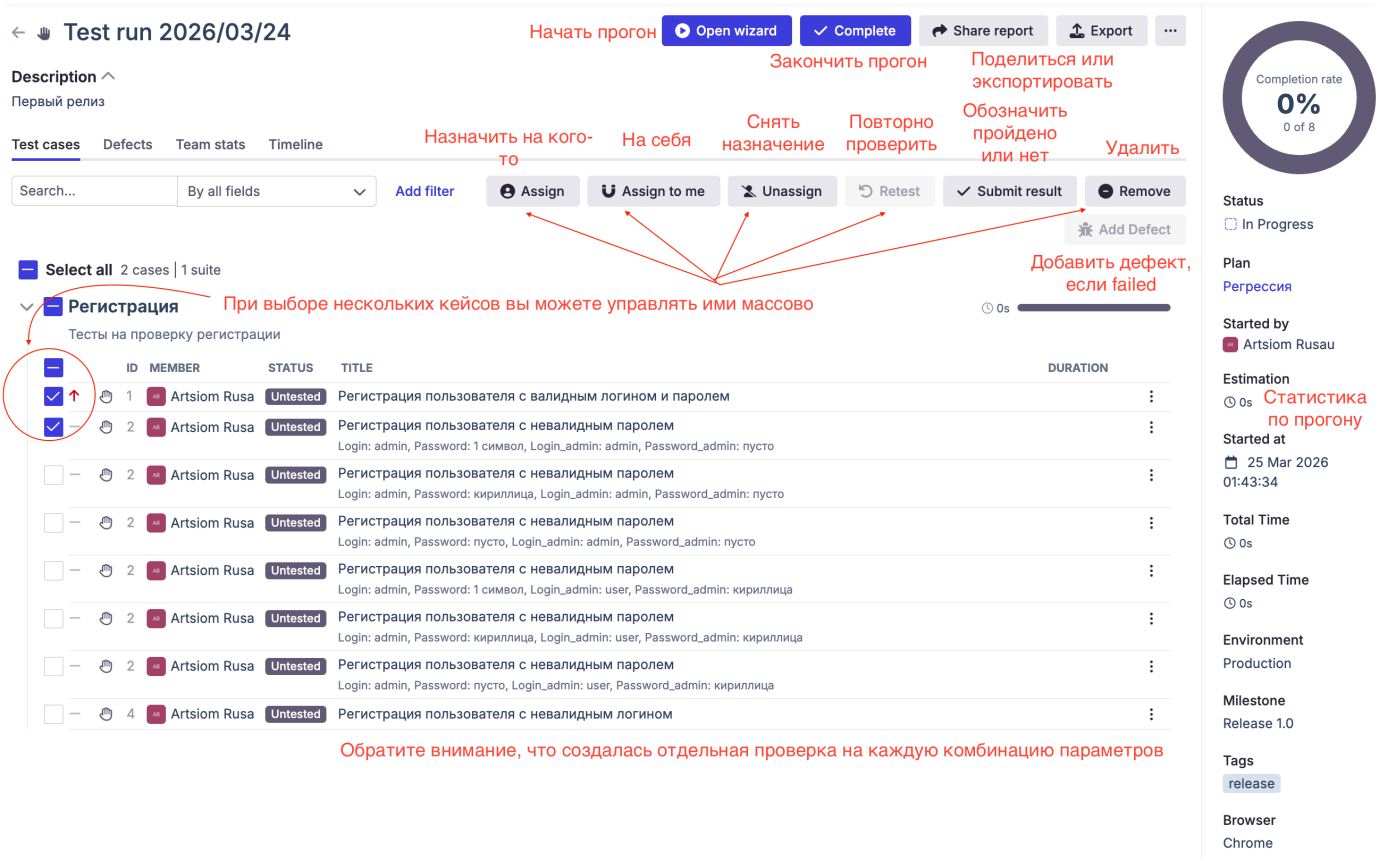
.



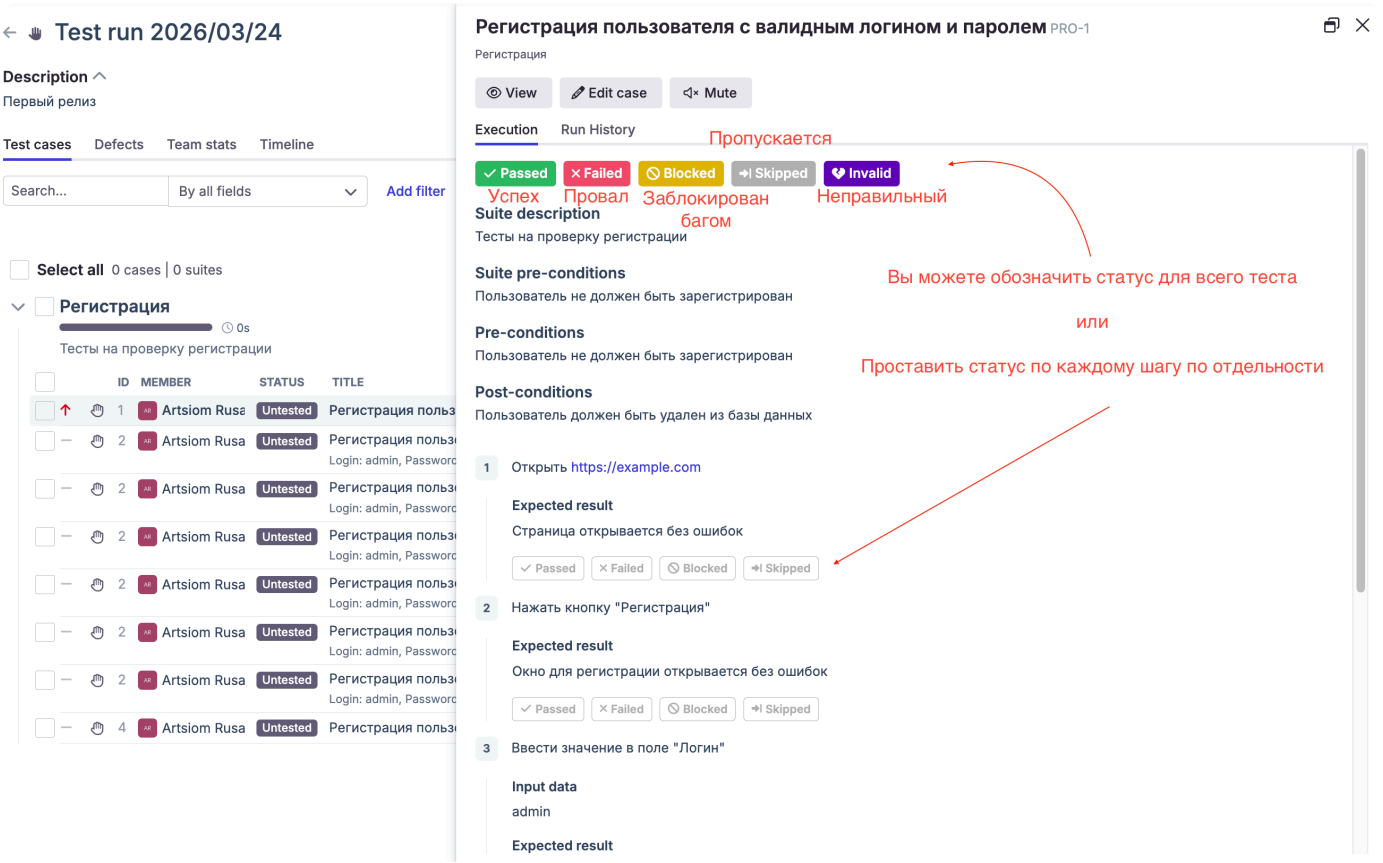

2. Далее выполните настройку прогона. Если вы пропустили действия из прошлых шагов урока, то чего-то может не хватать.




3. Так выглядит окно прогона до запуска. Изучите его перед тем, как перейти к шагу 4.

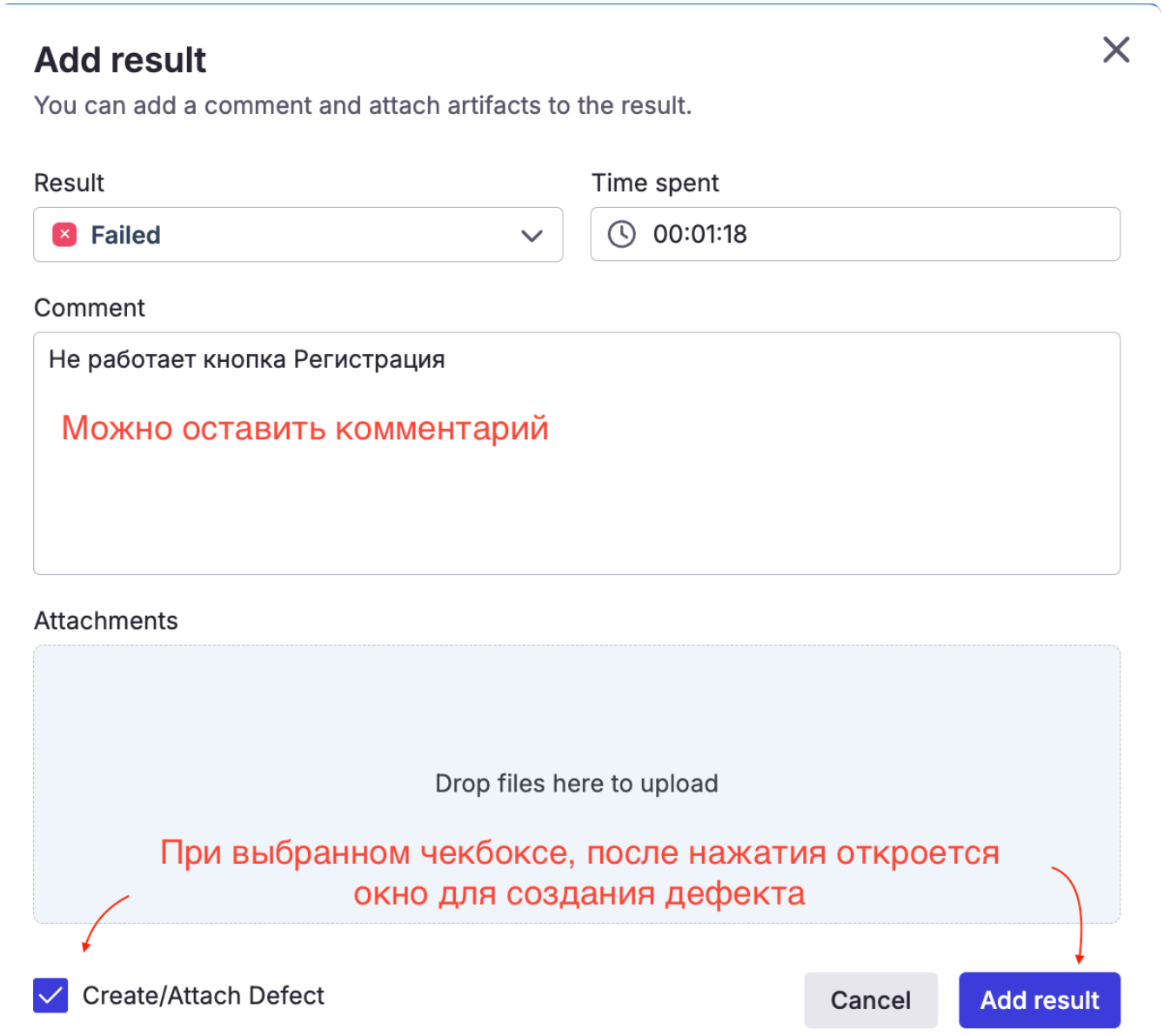


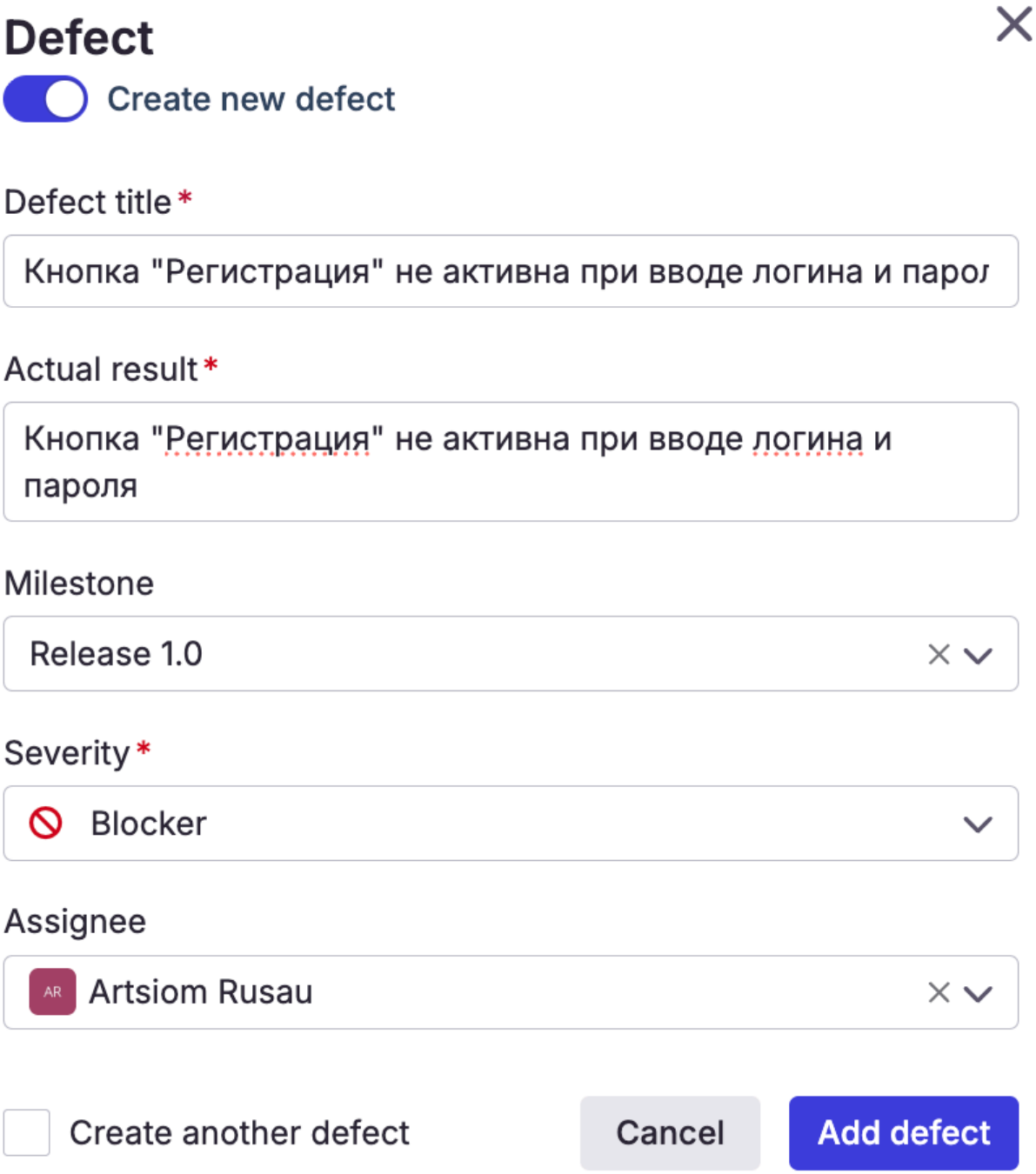

## 4. Давайте разберем, как выглядит конкретная проверка.



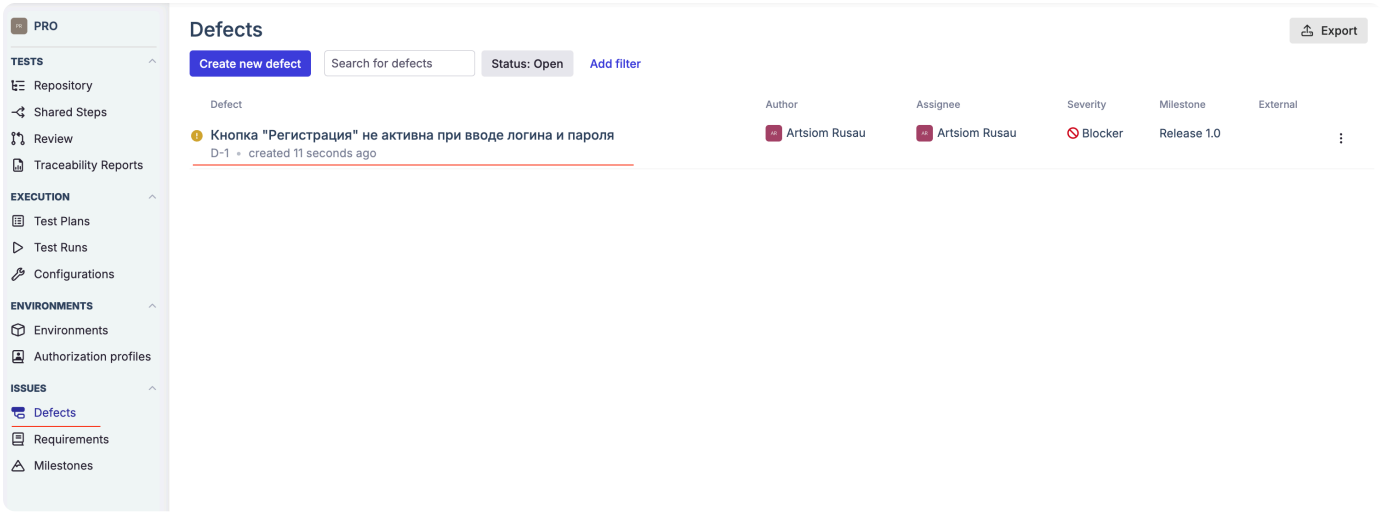
5. Если тесту присвоен статус Passed или Skipped, то тестировщик автоматически переходит к следующему тесту.

В остальных случаях открывается окно для создания комментария и возможного дефекта.

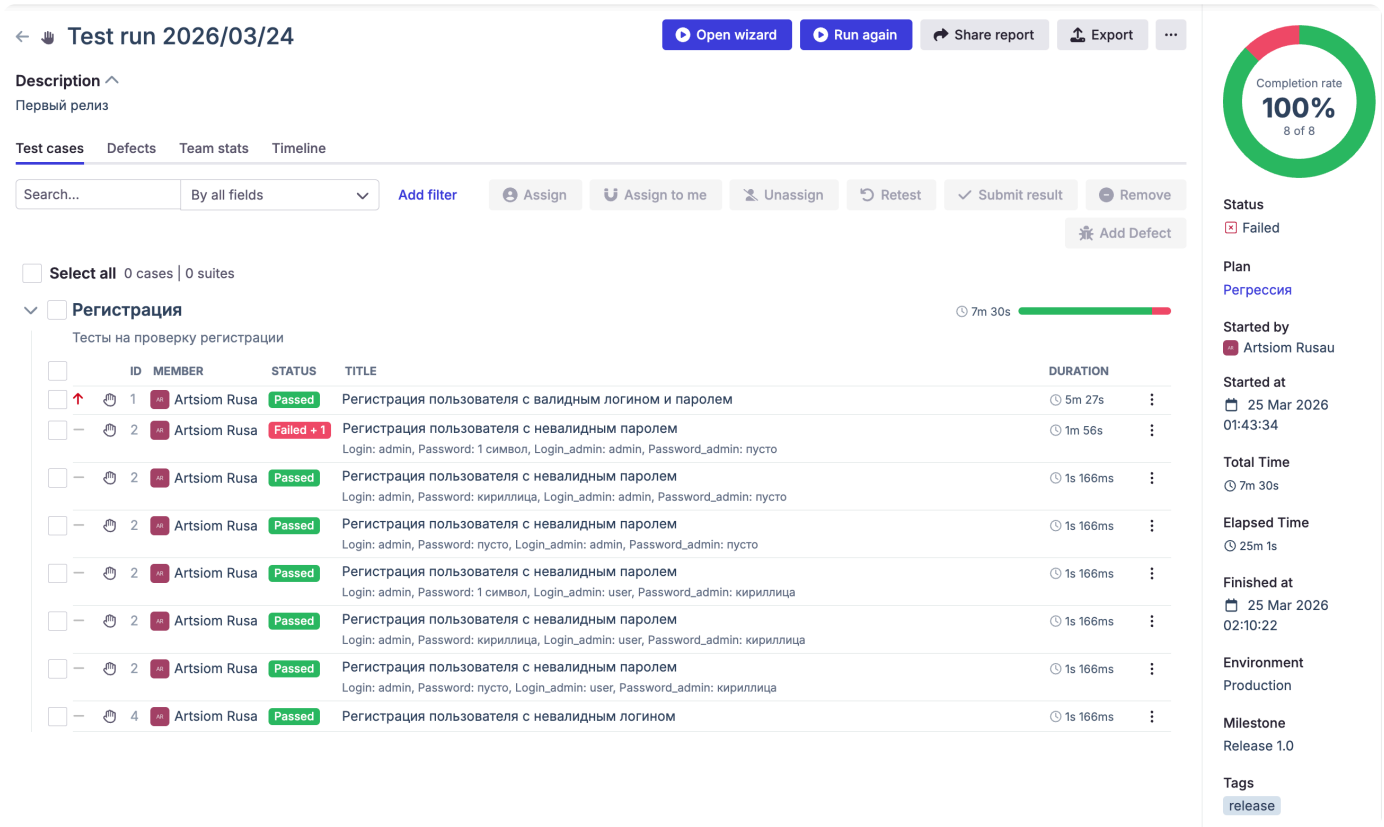




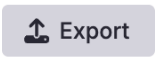

Созданные дефекты можно найти в соответствующей вкладке.




6. Вернемся к прогону и проставим для оставшихся тестов статус Passed. Обратите внимание, что время прогона посчиталось автоматически, а прогон стал завершенным.




7. Вы можете экспортировать его результаты в pdf




и использовать для своих отчетов.

**8. Поздравляю! Вы освоили все базовые функции QASE и готовы к его использованию в реальной практике!**

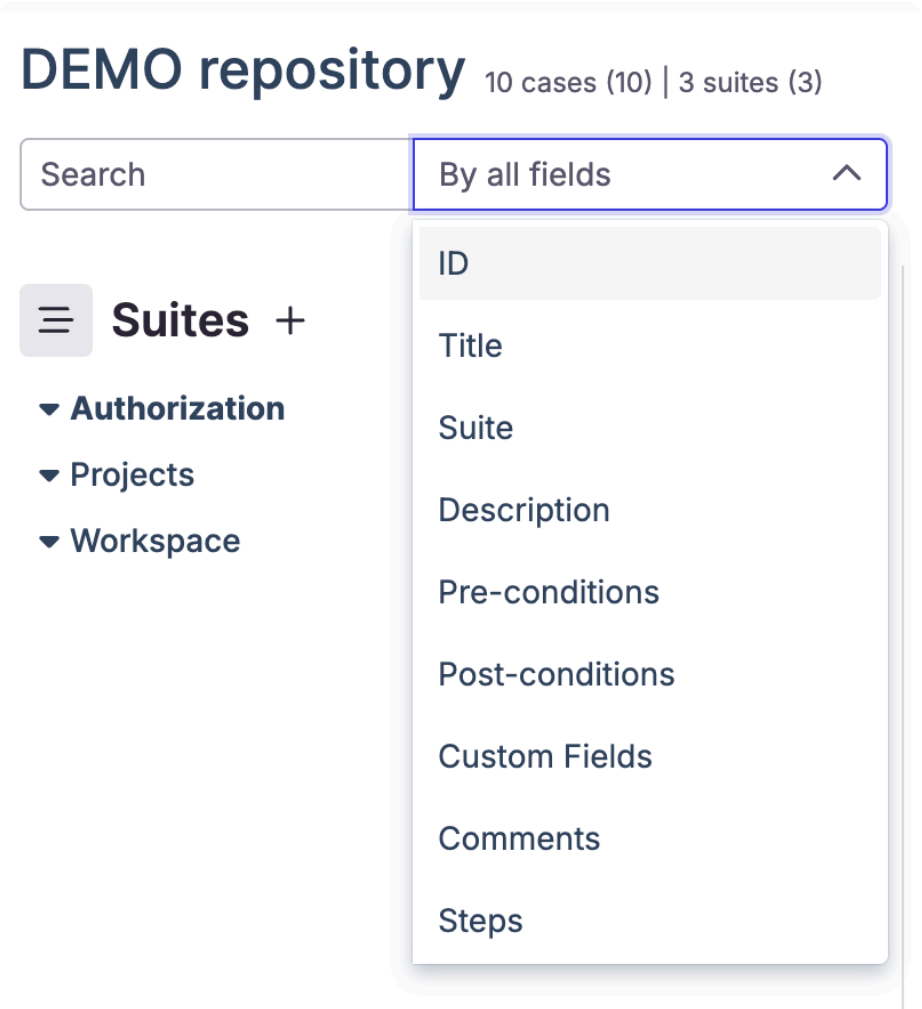
## Дополнительные возможности

### Поиск и фильтрация

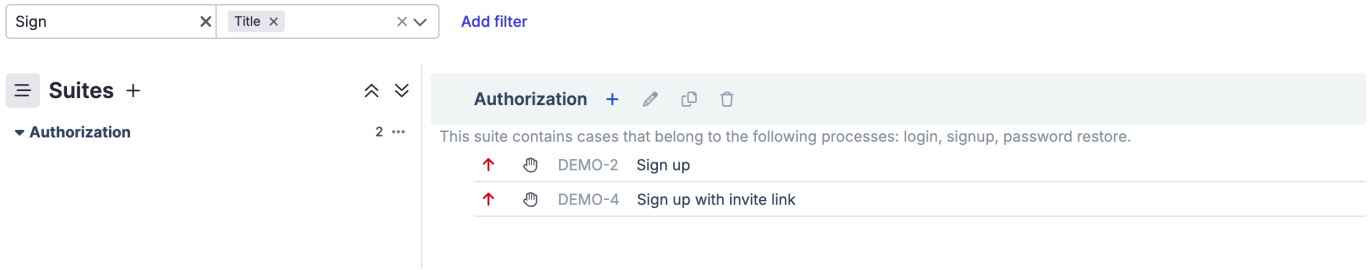
Одной из важных функций любой системы по управлению документации является поиск и фильтрация для формирования выборок: для регрессии, Smoke, для отслеживания прогресса, тестирования нужных модулей, сортировки и так далее.

### Поиск

1. Перейдите в репозиторий демо-проекта. Выберите по какому атрибуту будет происходить поиск.




2. Введите целевой запрос и изучите результат.



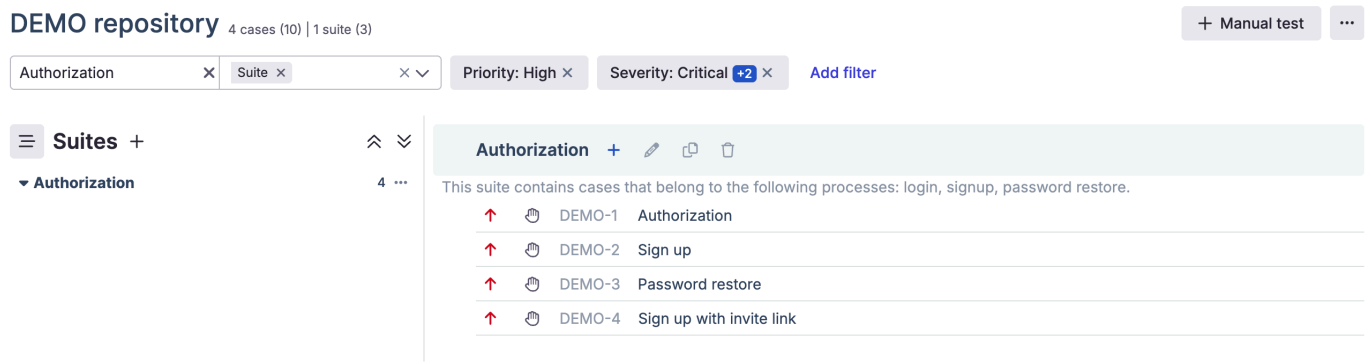

Фильтрация

1. Представим, что вам надо сделать для поиска тестов в регрессионный набор. Отметим только тест-кейсы с высоким приоритетом, которые уже готовы и относятся к авторизации. Нажмите

Add filter

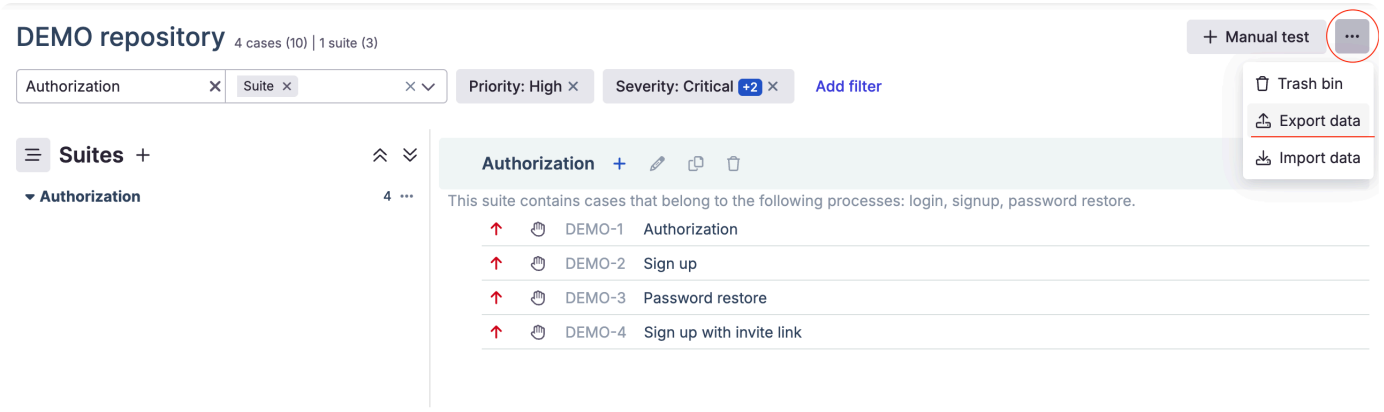
.

2. Отметить последовательно фильтры, которые вам нужны. После отметки можно выбрать целевые значения. Получится примерно такое результат.

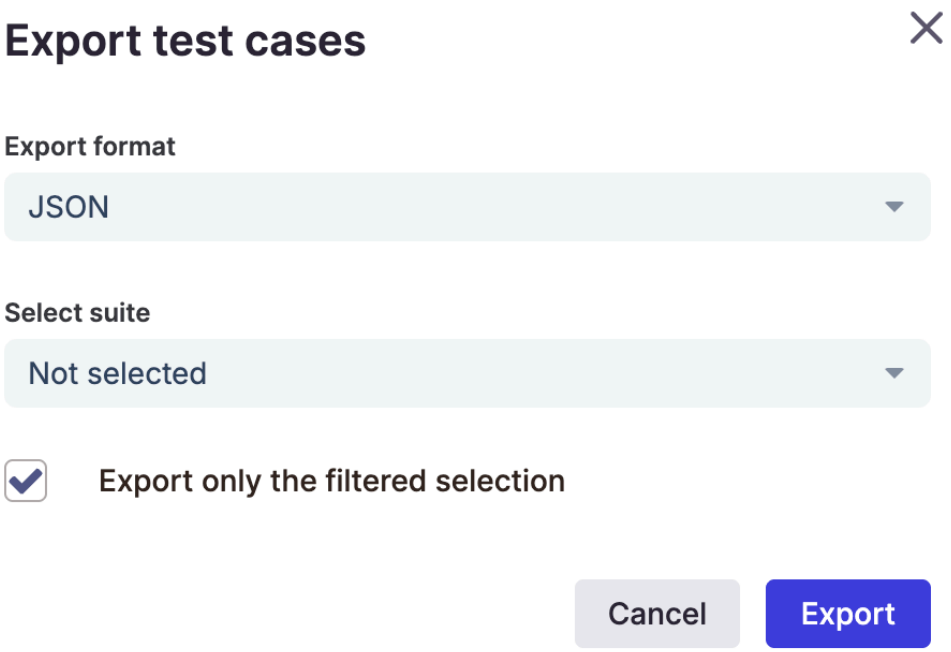



## Экспорт и импорт

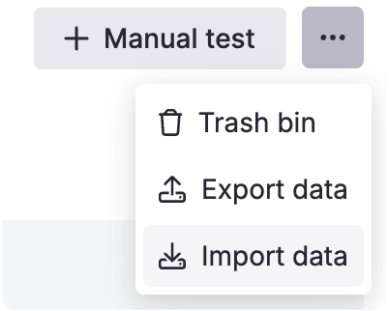
1. Отфильтрованные результаты можно экспортировать в нужном формате. Напоминаю, что это может быть полезно для отчетов или миграции в другие сервисы.



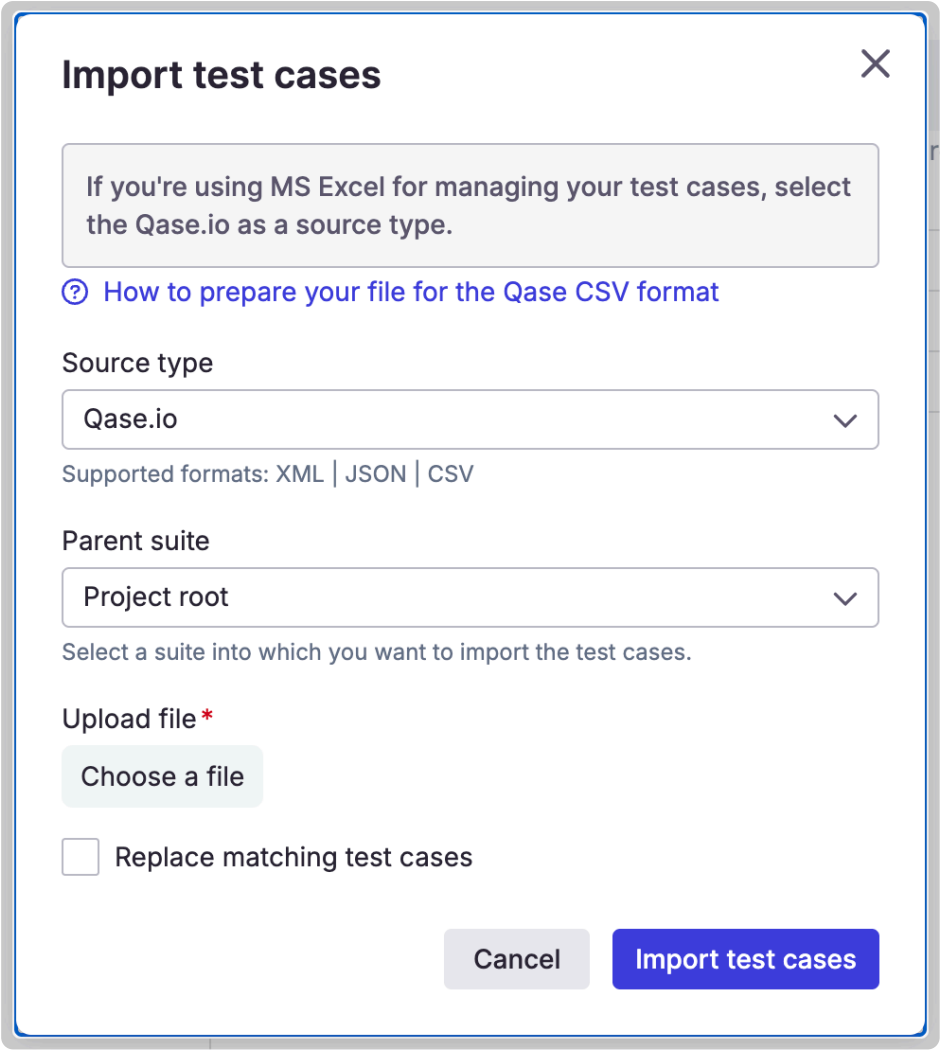

2. Выберите нужный формат для экспорта, набор (если нужно). В нашем случае я поставлю чекбокс для экспорта только отфильтрованных кейсов.




3. Также можно сделать импорт из готового файла. Причем из разных систем.




4. Выберите нужную систему, целевой набор, в который сделать импорт, загрузите файл. Если вы хотите заменить совпадающие кейсы, то поставьте чекбокс в последней строке.




5. После импорта я получил идентичный набор.

**Authorization**

This suite contains cases that belong to the following processes: login, signup, password restore.

| | |
|---|---|
| DEMO-1 | Authorization |
| DEMO-2 | Sign up |
| DEMO-3 | Password restore |
| DEMO-4 | Sign up with invite link |

**Authorization**

This suite contains cases that belong to the following processes: login, signup, password restore.

| | |
|---|---|
| DEMO-11 | Authorization |
| DEMO-12 | Sign up |
| DEMO-13 | Password restore |
| DEMO-14 | Sign up with invite link |

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

# 22. Работа в Test IT (версия 2026)

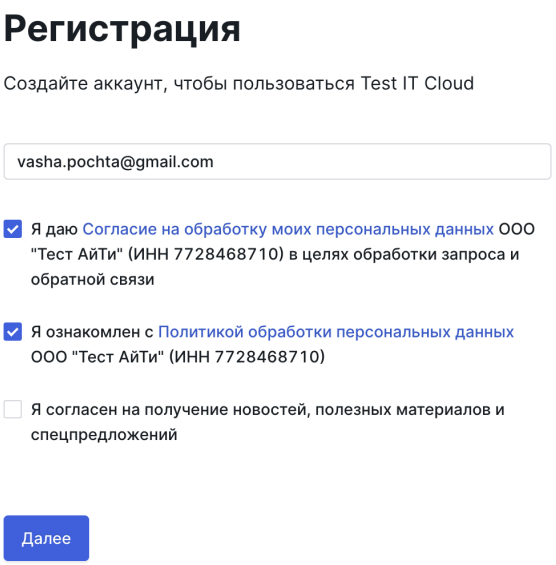
**Test IT** – это система управления тестированием (TMS), предназначенная для организации жизненного цикла тестирования: от написания проверок до формирования отчетности о качестве продукта.

Для работы с системой существует два основных пути:

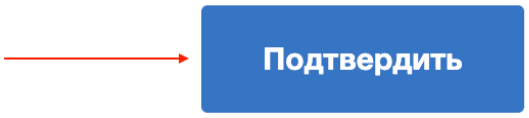
1. **Облачная версия,** которая не требует настройки серверов. В ней мы будем работать на курсе.
2. **Серверная версия.** Установка на собственные мощности компании. С ней вы чаще всего будете работать в реальных компаниях.

**Начнем с регистрации в облачной версии:**

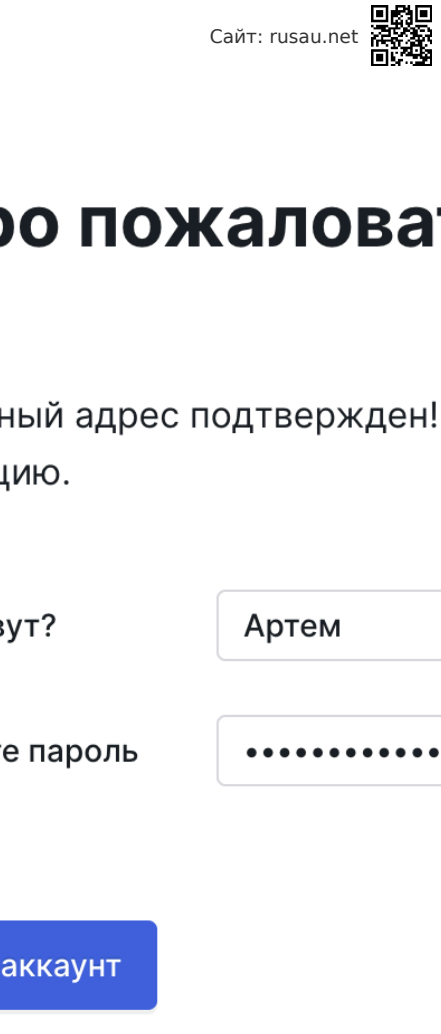
1. Перейдите на страницу регистрации https://id.testit.software/registration
2. Введите вашу почту и активируйте обязательные пункты.
3. Нажмите кнопку "Далее"



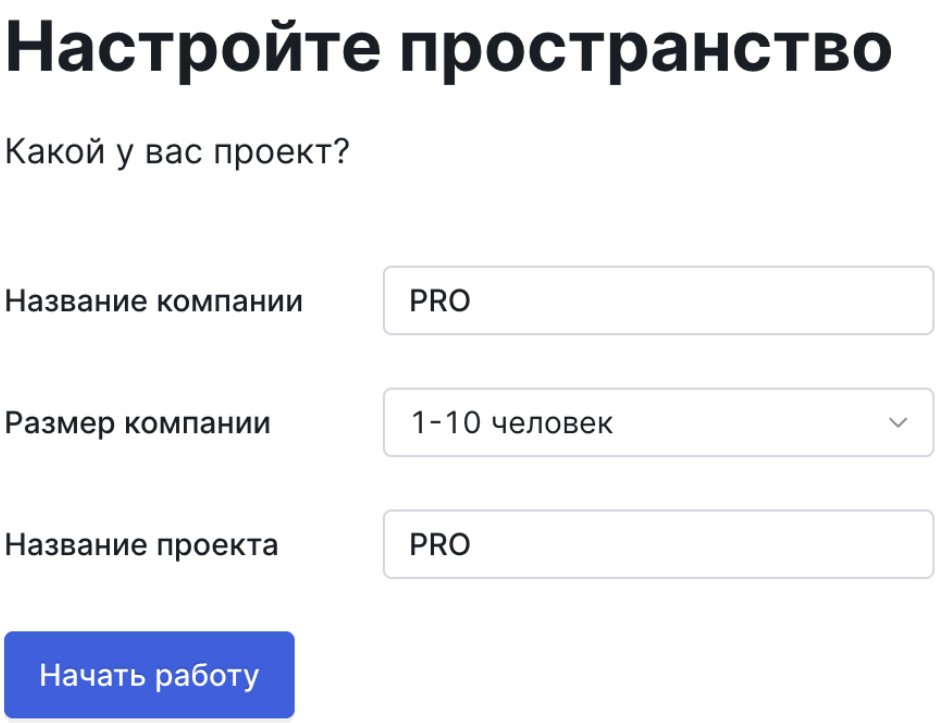

4. Подтвердите вашу почту по ссылке из письма.




5. Напишите ваше имя и придумайте пароль. Далее нажмите кнопку "Создать аккаунт".

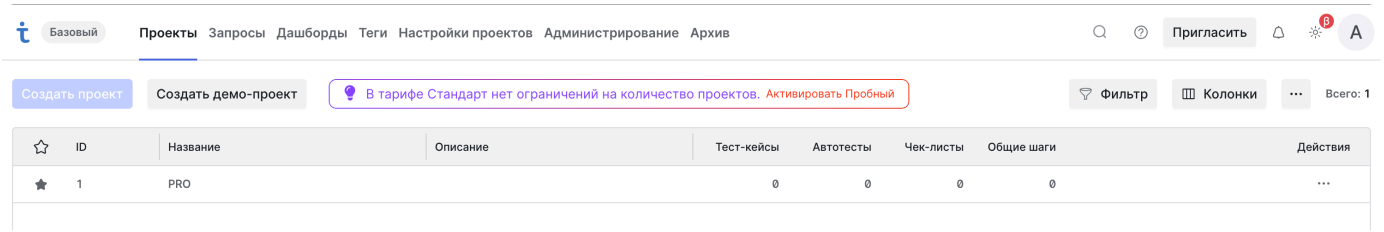



6. Придумайте любое название компании и проекта. Нажмите кнопку "Начать работу".




7. Пройдите небольшое обучение, нажимая на каждом этапе кнопку "Далее". В конце вы можете подключить пробный период для доступа ко всем функциям, но в курсе мы будем работать с базовой версией.

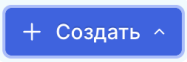
8. Поздравляю! Ваш первый проект готов к работе!



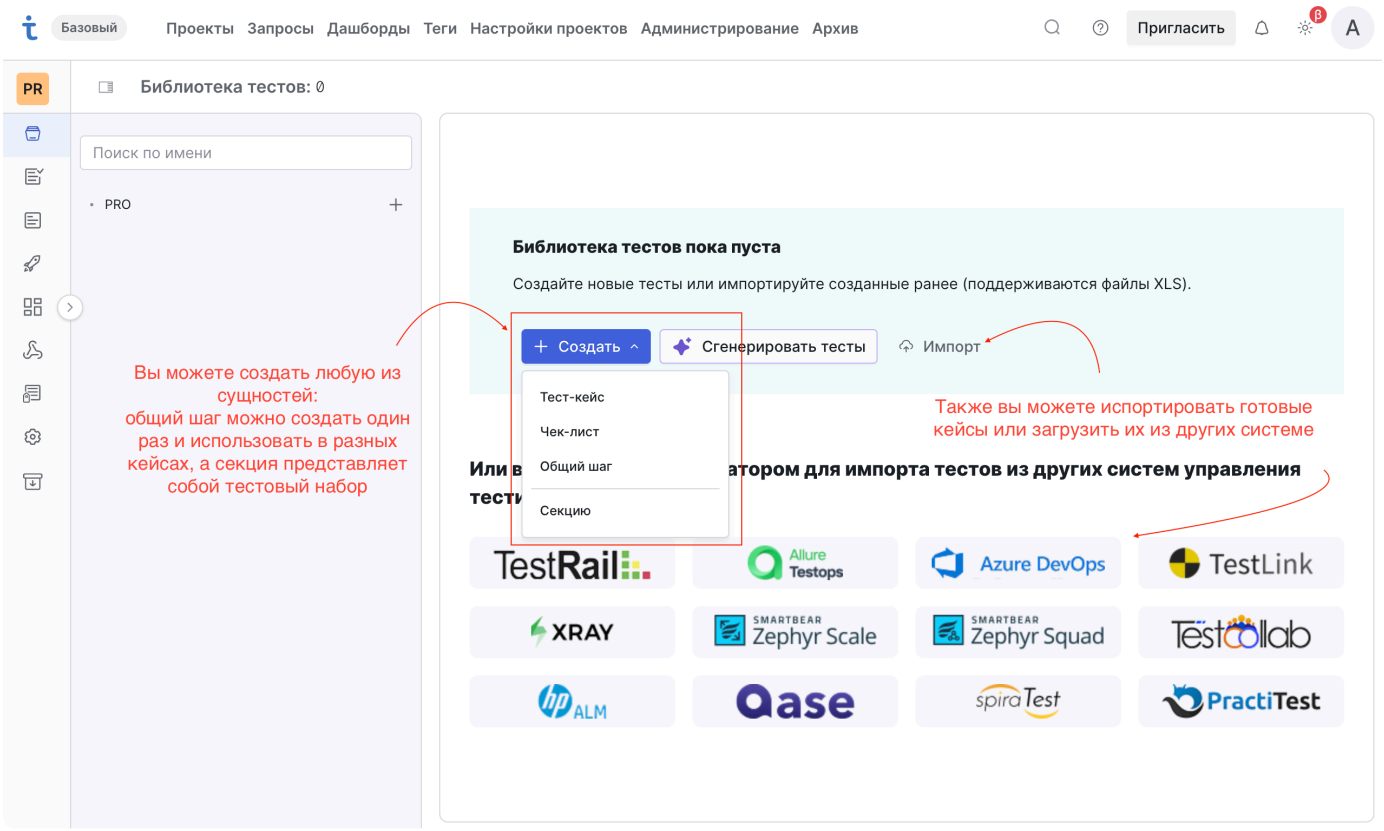

## Создание тест-кейса

1. Откройте проект, созданный в прошлом шаге.

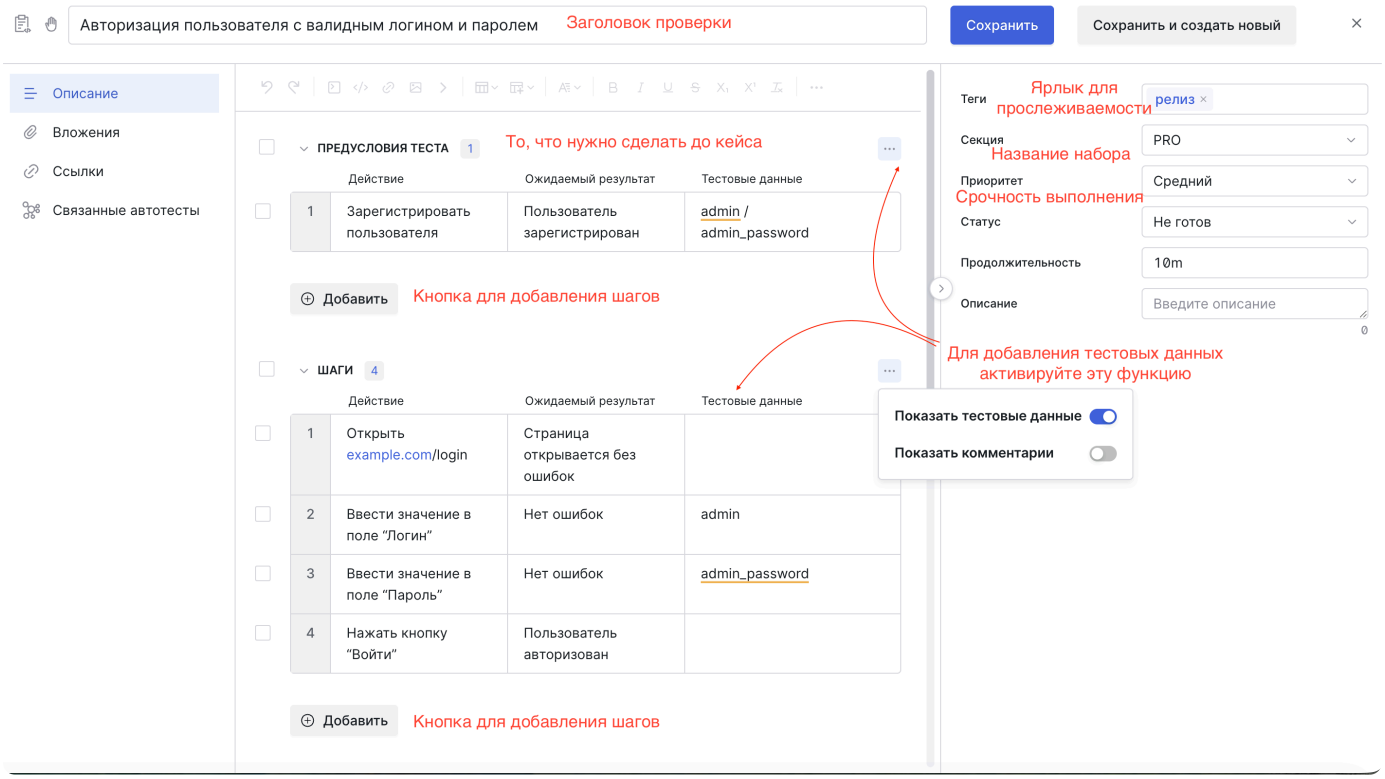
2. Нажмите




и выберите "Тест-кейс".

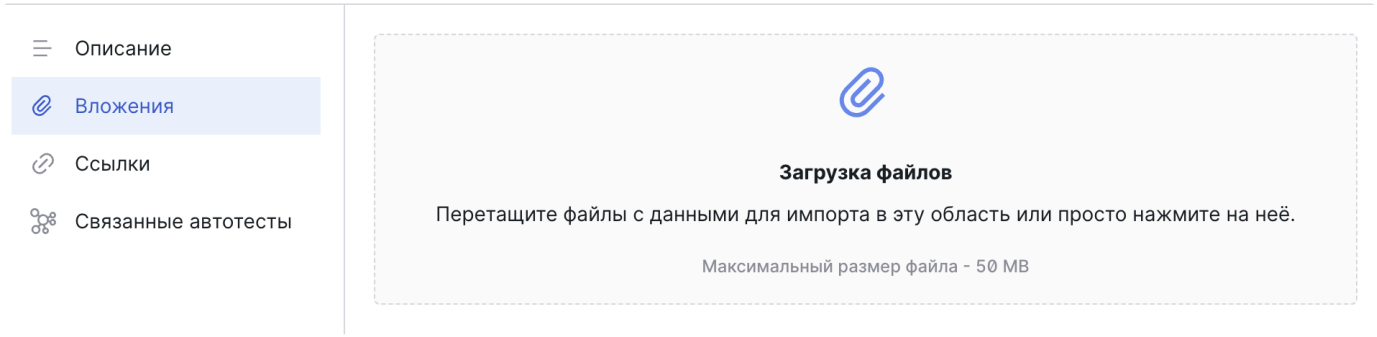



## 3. Заполните обязательные атрибуты.

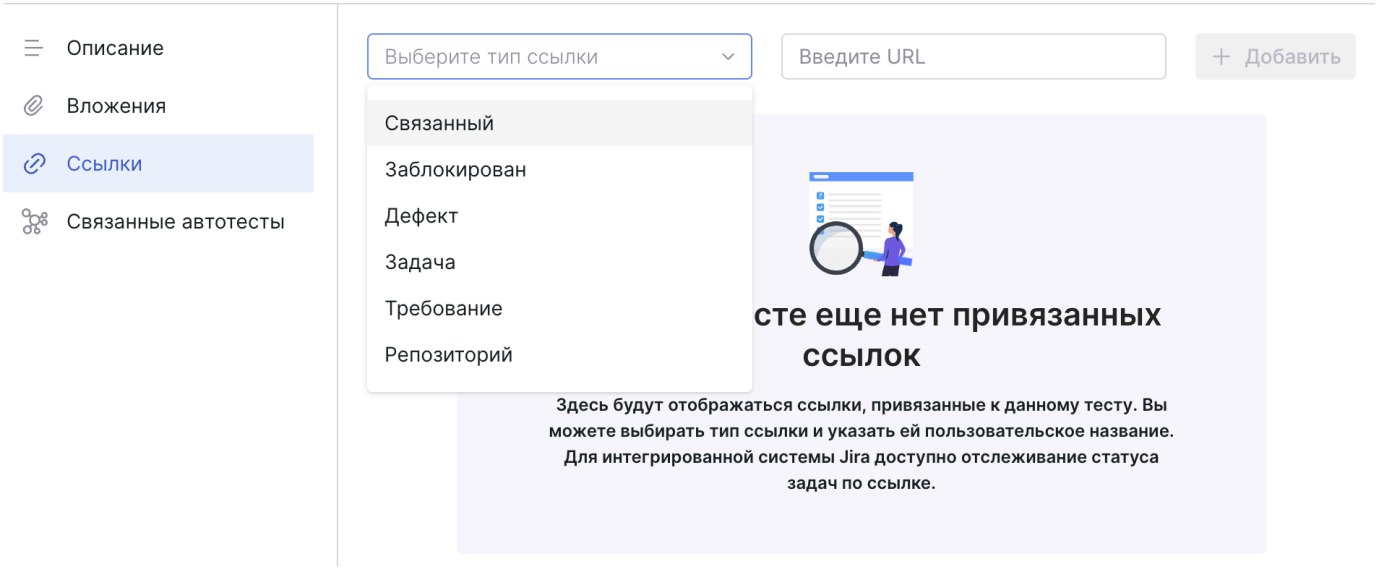



Работу с параметрами мы рассмотрим в следующих шагах урока.

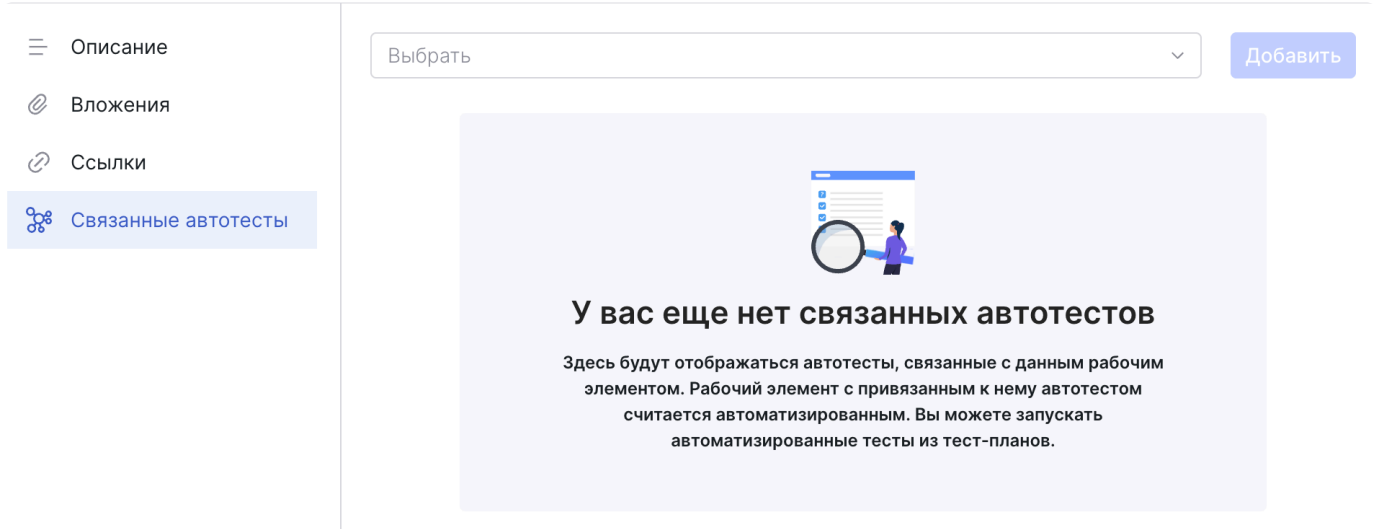
4. Во вкладке "Вложения" можно добавить файлы, связанные с кейсом: скрины, видео, тестовые данные.




5. Во вкладке "Ссылки" можно добавить ссылки, на связанную с кейсом документацию.




6. Во вкладке "Связанные автотесты" можно добавить ссылки на автотесты.



7. Сохраните тест-кейс, нажав

Сохранить

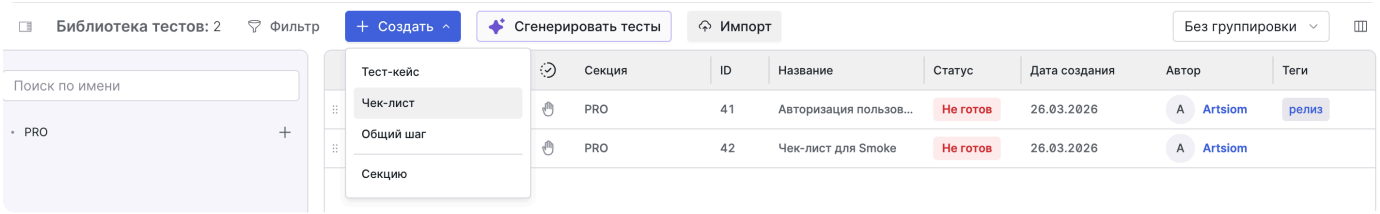
8. После создание кейса, появится три новые вкладки:

- **История результатов**, которая будет содержать результаты прохождения этого теста.
- **Изменения**, в которой находится история всех обновлений кейса.
- **Комментарии**, в которых будут содержаться комментарии к кейсу.

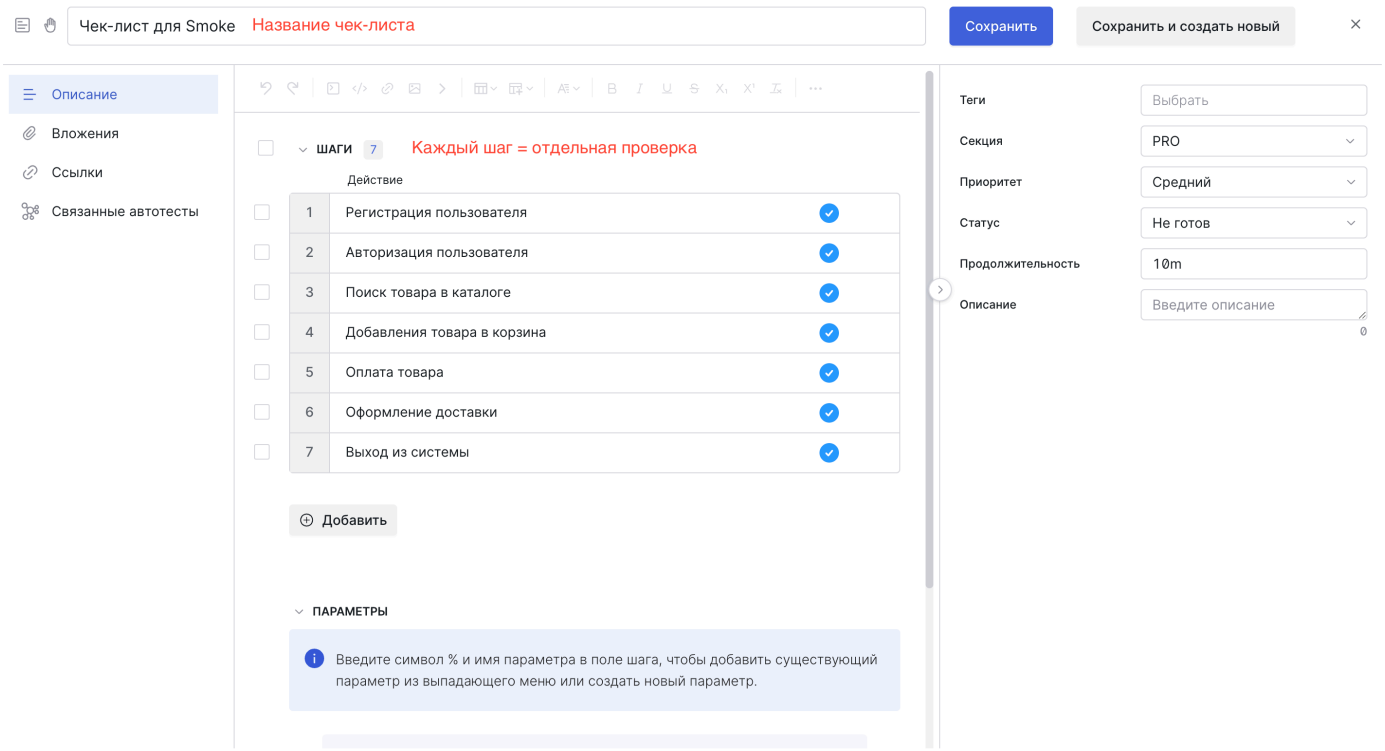
9. Далее в уроке мы научимся создавать тестовый прогон, чтобы запустить тест-кейс и отметить статусы его проверки.

## Создание чек-листа

1. Выберите опцию чек-лист при создании элемента в проекте.




2. Заполните обязательные атрибуты. Набор вкладок и его назначение будет точно таким же, как и в тест-кейсе.



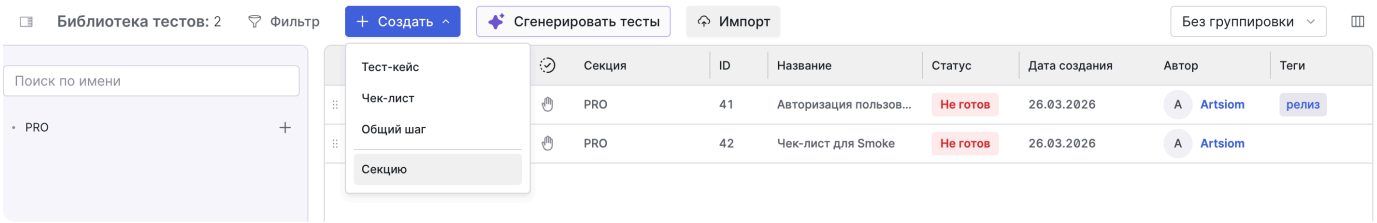

3. Нажмите "Сохранить". В чек-лист появятся новые вкладки, аналогичные тест-кейсу.

4. Далее в уроке мы научимся создавать тестовый прогон, чтобы запустить чек-лист и отметить статусы его проверок.

## Создание секции

**Секция в Test IT** – тестовый набор, в который объединяются тест-кейсы и чек-листы по одному признаку.

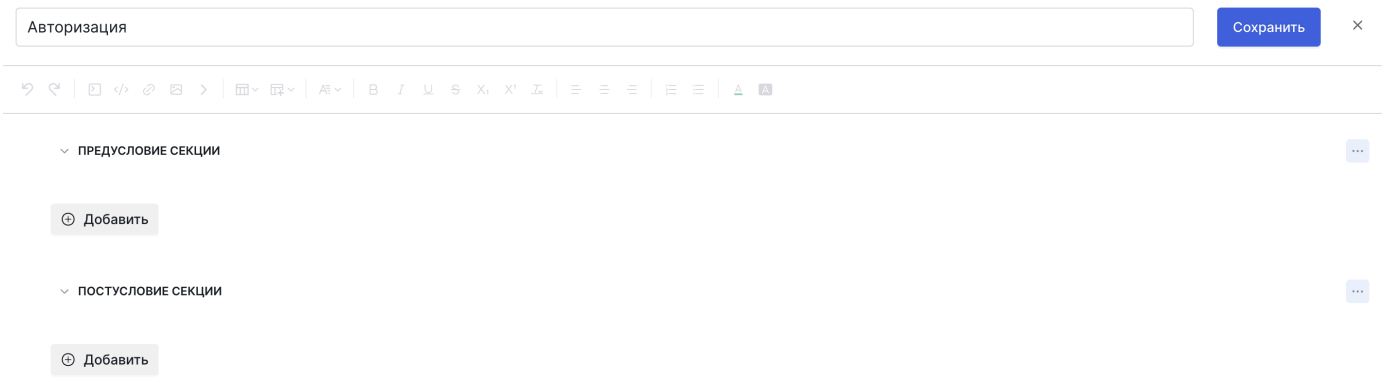
1. Выбери секцию в меню создания элемента вашего проекта.



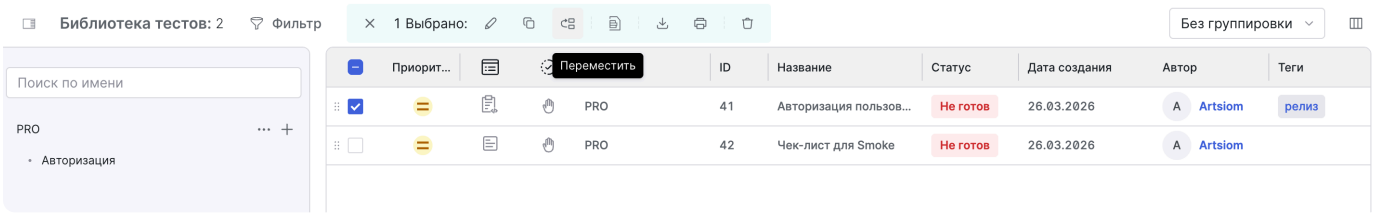

2. Добавьте название секции, а также общие предусловия и постусловия для всего набора (если нужны). Нажмите

Сохранить

.

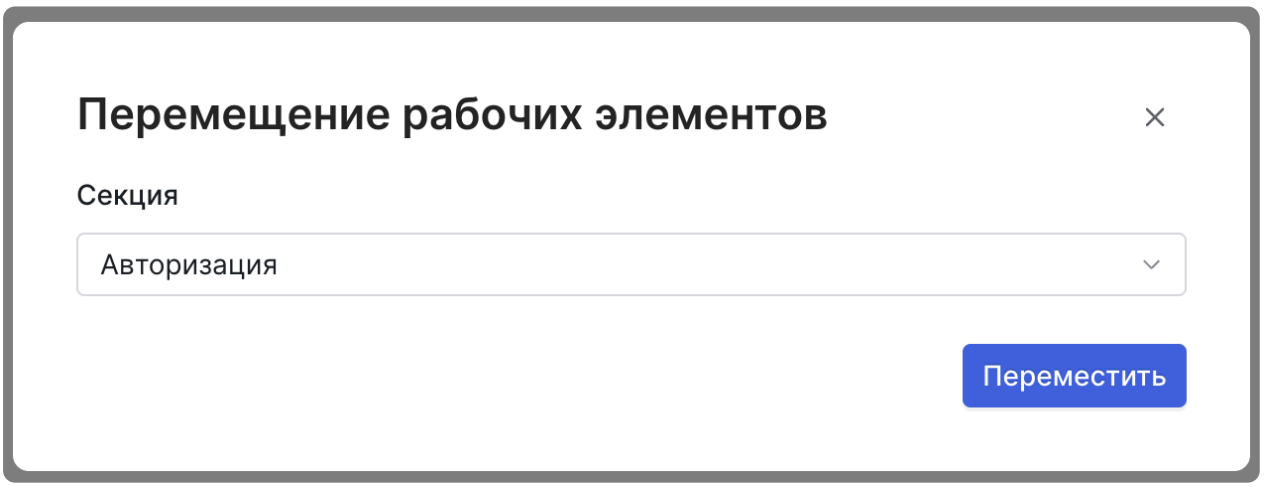



3. Теперь вы можете переместить тест-кейс "Авторизация" в эту секцию. Отметьте нужный кейс и нажмите "Переместить".




4. В контекстном меню выберите нужную секцию и нажмите

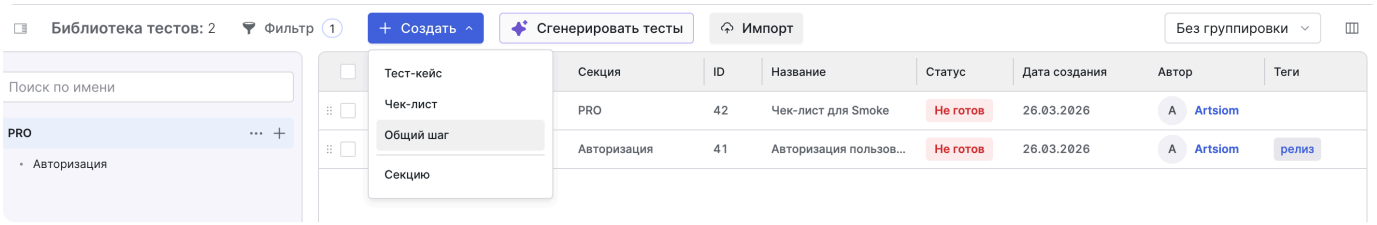
.



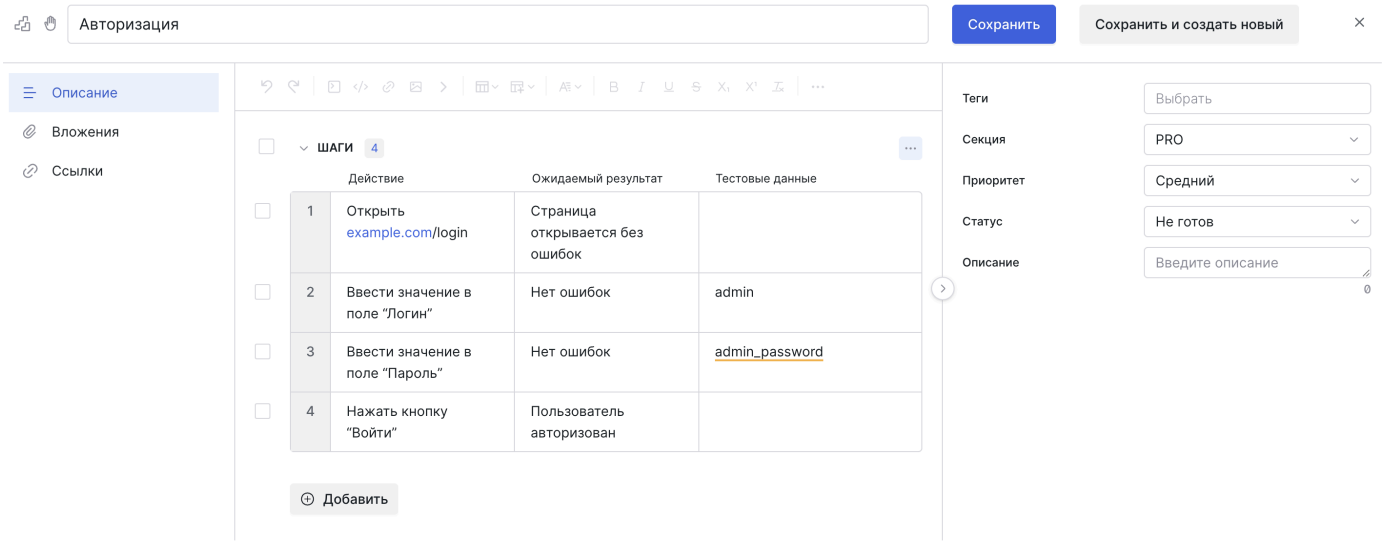

## Создание общих шагов

**Общие шаги в Test IT** – это шаблон из шагов, которые можно создать один раз и использовать в разных кейсах. Например, вы можете создать группу шагов для авторизации, чтобы не повторять одну и ту же работу много раз.

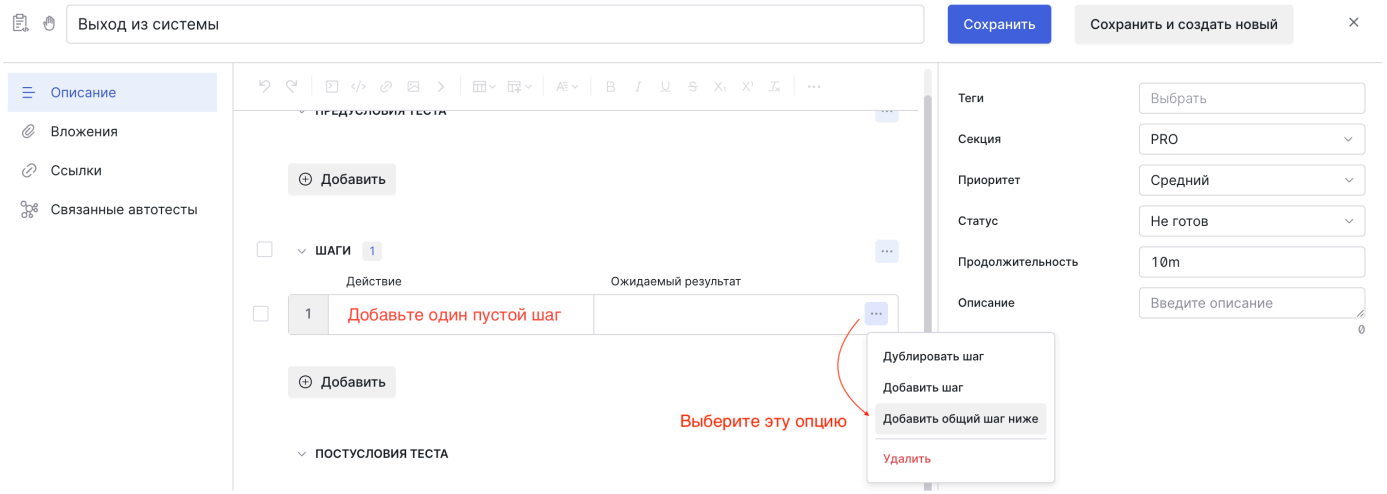
1. Выберите "Общий шаг" в меню для создания новых элементов проекта.

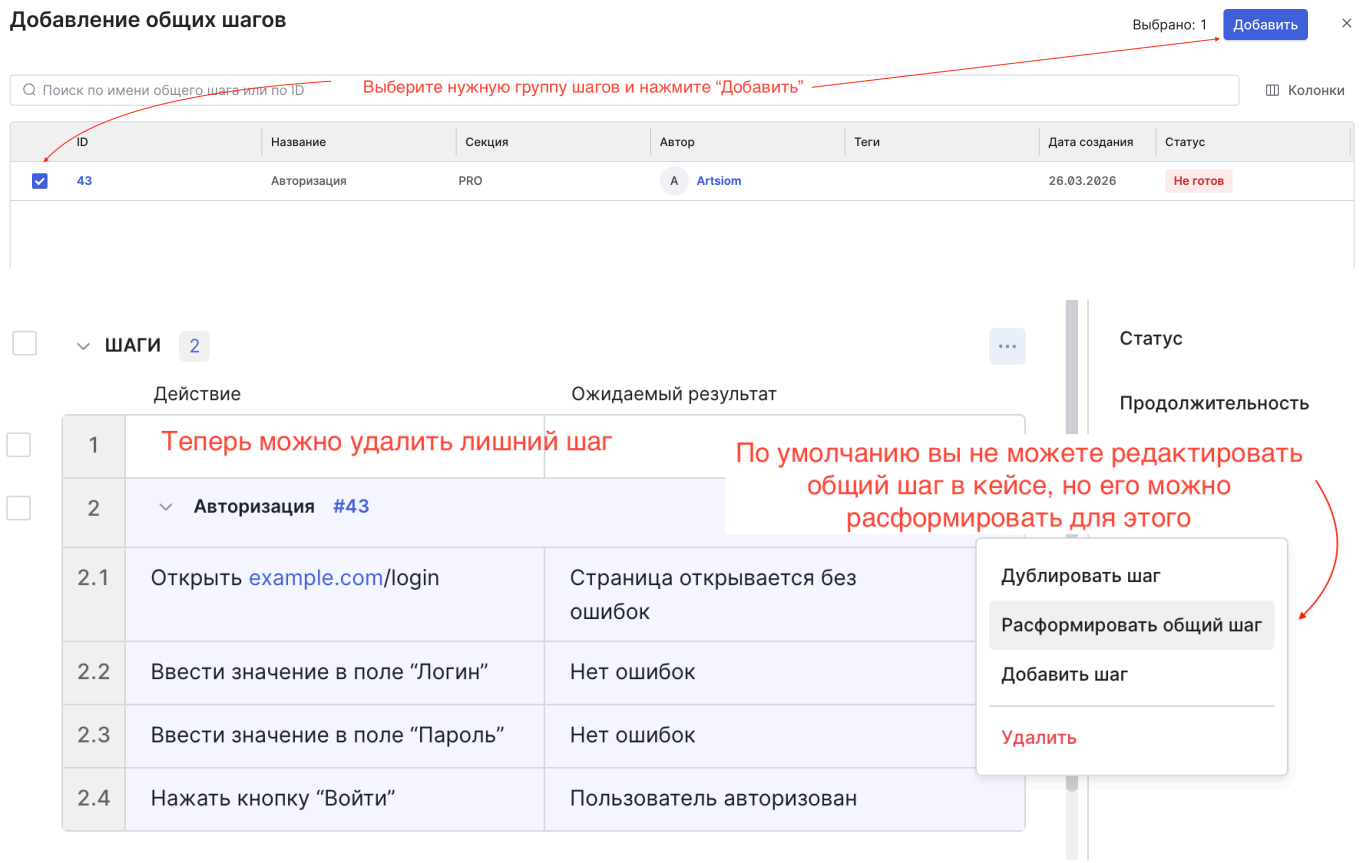



2. Процесс создания аналогичен тест-кейсу. Не забудьте сохранить результат.

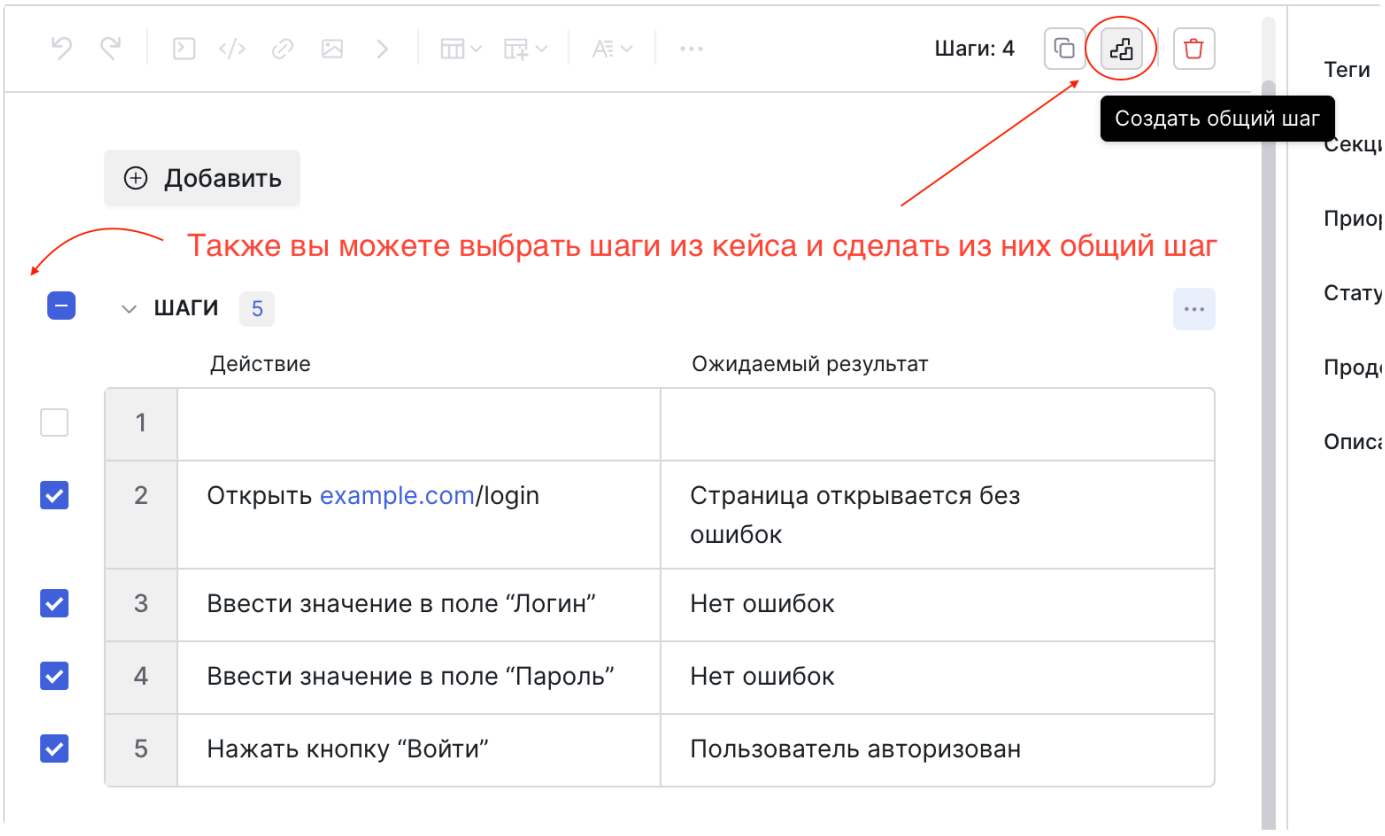



3. Давайте создадим новый тест-кейс с заголовком "Выход из системы" и добавим в него общий шаг.






4. Также вы можете создать общие шаги прямо из кейса.



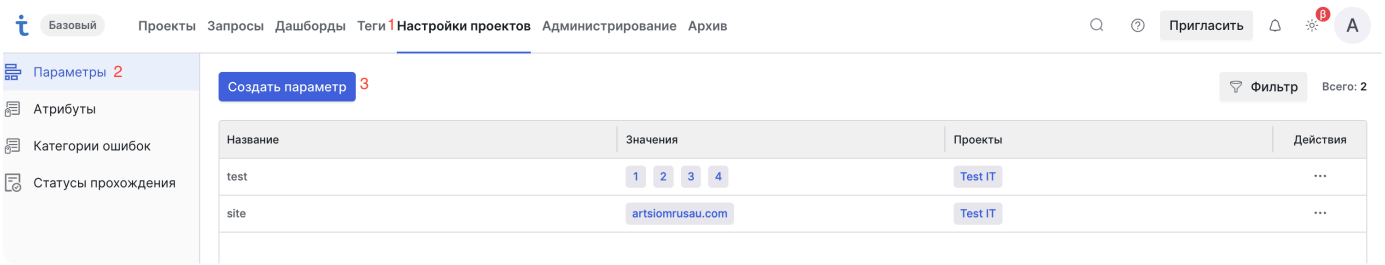
5. Теперь вы умеете создавать "Общий шаг" и использовать его для упрощения вашей работы!

**Параметры в Test IT** – это значения, которые можно использовать для избежания дублирования кейсов и минимизации их количества.

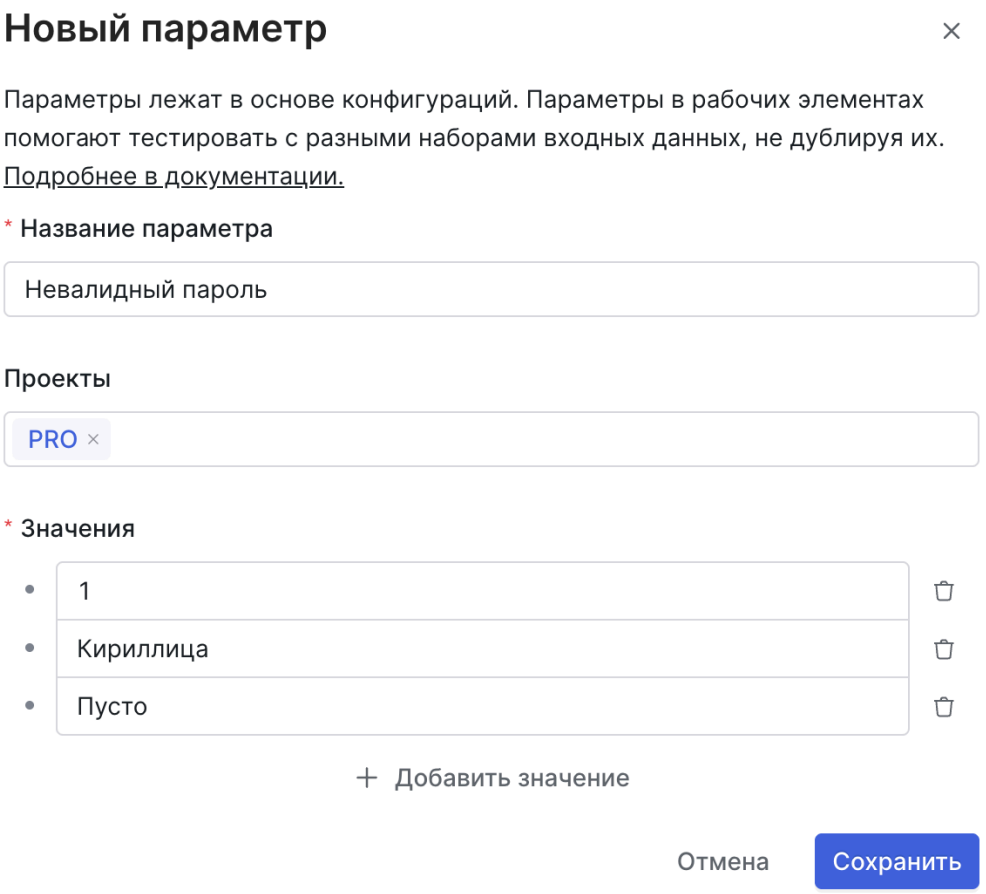
Параметры можно использовать для создания тестовых данных. Например, при регистрации пользователя вы можете создать один параметр логин с несколькими значениями и пароль – с несколькими значениями. После запуска тестового прогона для каждого значения параметра создастся отдельная проверка. Это особенно полезно для негативных проверок.

## Создание параметров

1. Перейдите в Настройки проектов и следуйте за порядком на скрине.




2. Создайте параметр "Невалидный пароль".

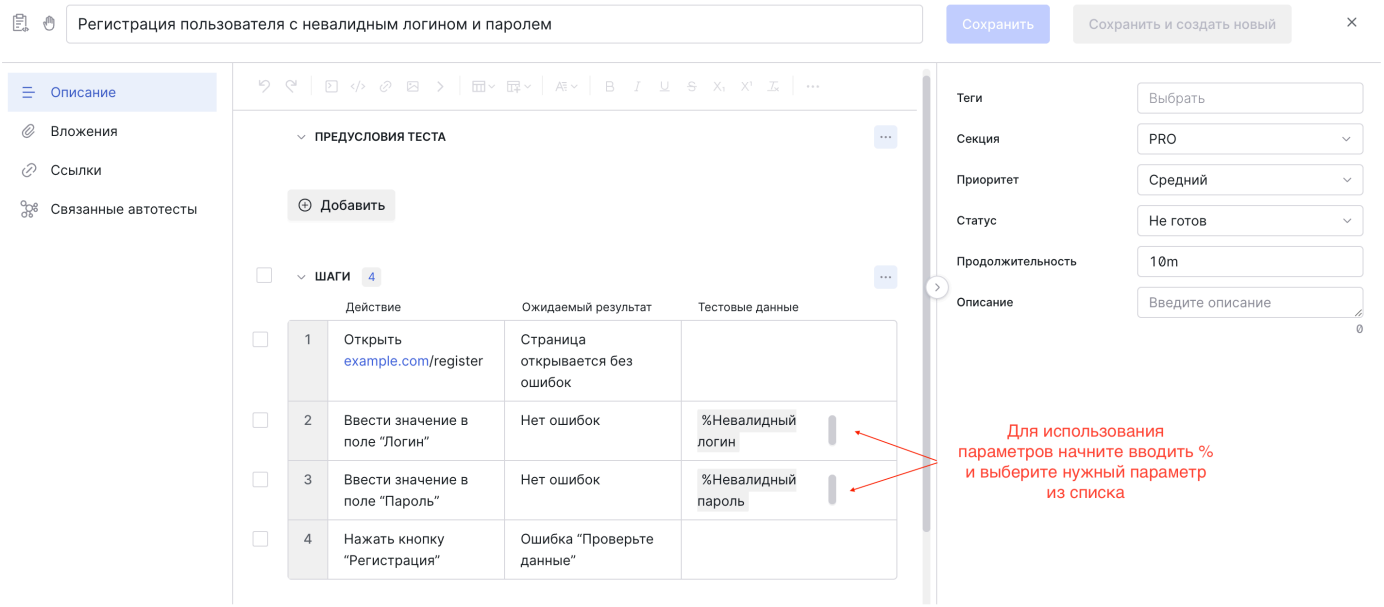



3. Создайте параметр "Невалидный логин" по аналогии.

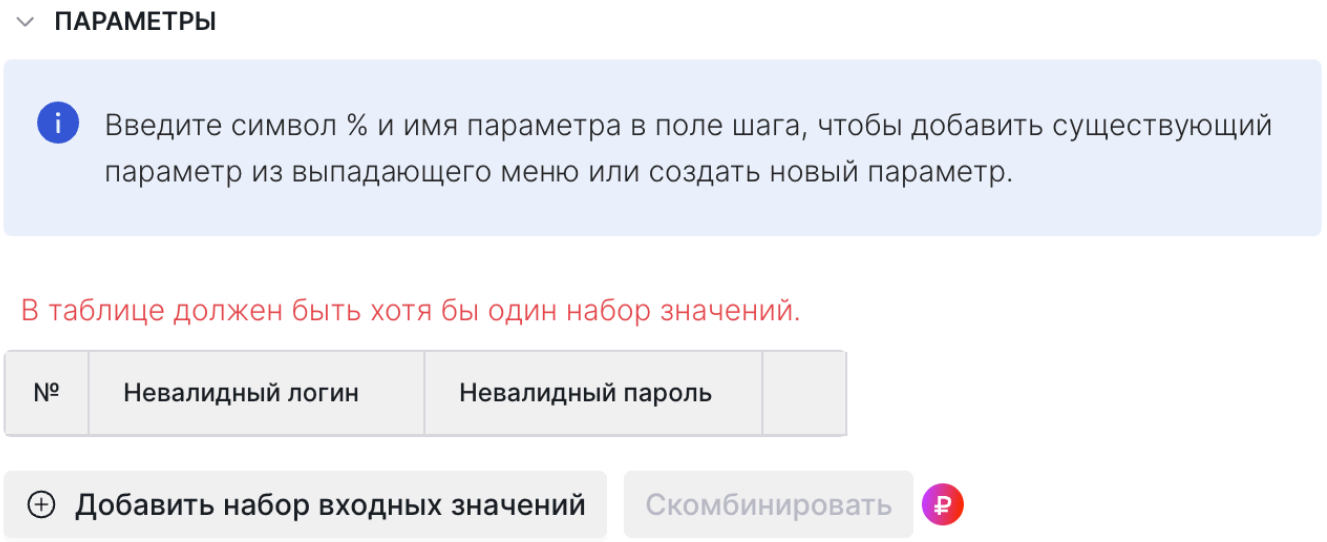
4. После создания параметров их можно редактировать и удалять.

**Добавление параметров в кейс**

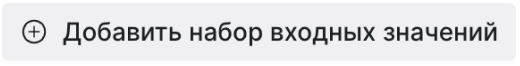
1. Создайте новый кейс "Регистрация пользователя с невалидным логином и паролем".




2. Перейдите в область "Параметры". Она должна выглядеть примерно так.

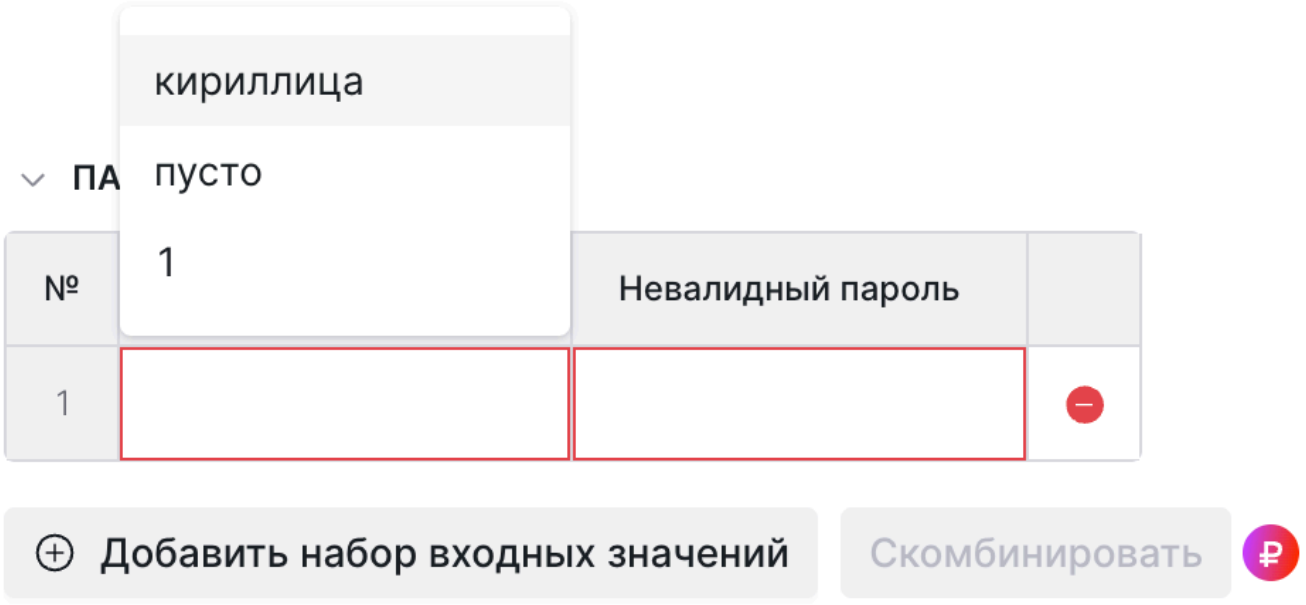



3. Теперь нажмите




.

4. Выберите комбинации нужных значений. В дальнейшем каждая строка станет отдельным тестом. На платных тарифах это можно делать автоматически кнопкой "Скомбинировать".



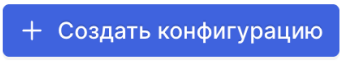

5. У меня получилось примерно так.

6. Сохраните кейс. Мы посмотрим, как это будет работать в тестовом прогоне.

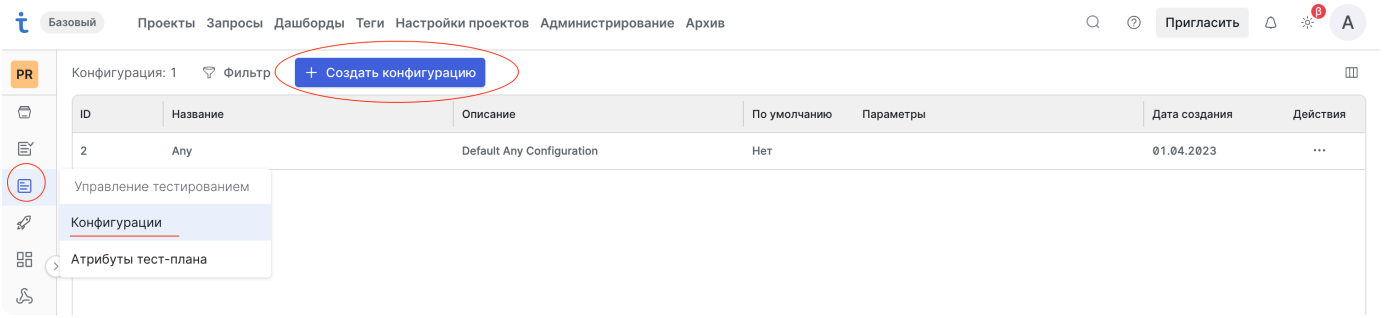
**Создание конфигурации**

**Конфигурации в Test IT** – это параметры, которые позволяют описывать тестовые окружения, например, ОС, браузеры, сервера. Они используются в тест-планах, о которых мы поговорим совсем скоро.

1. Перейдите во вкладку "Конфигурации" и нажмите



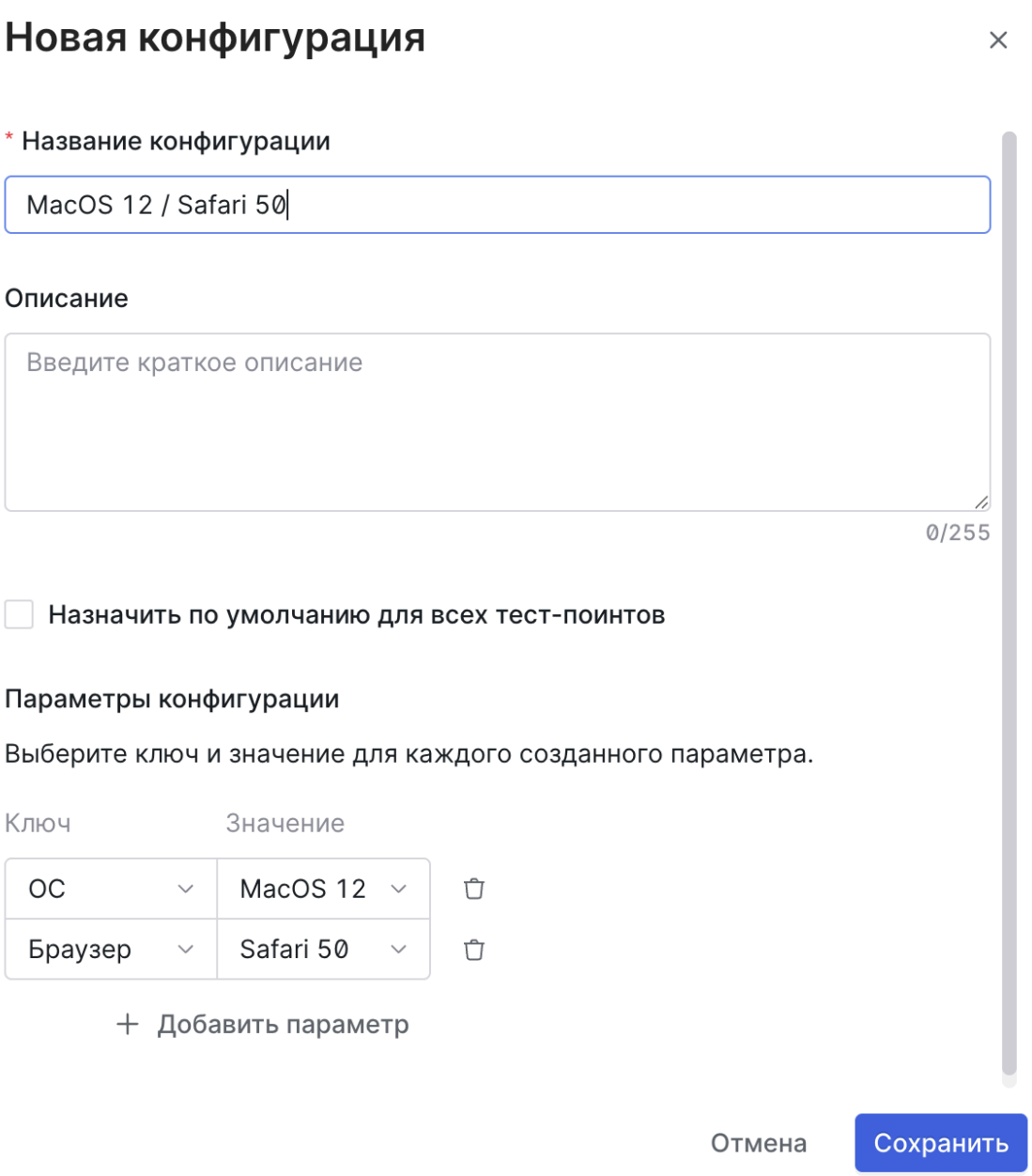

.



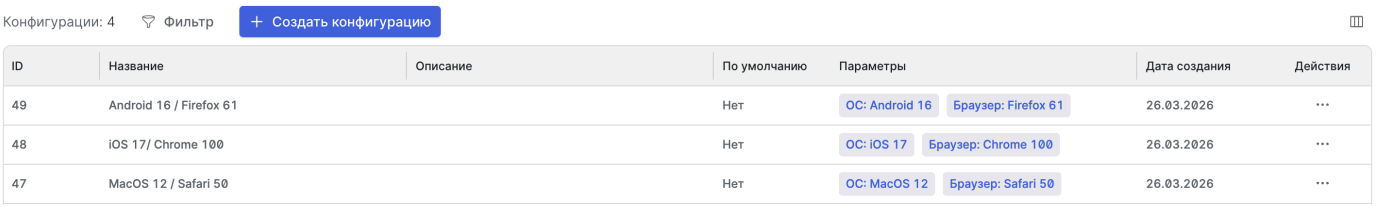

2. Заполните обязательные атрибуты. Ключи и значения могут браться либо из параметров, которые мы создавали в прошлом шаге, либо вы можете добавить новые.

Вы можете включать новую конфигурацию во все создаваемые вами наборы тестов по умолчанию. Для этого активируйте опцию **Назначить по умолчанию для всех тест-поинтов**.

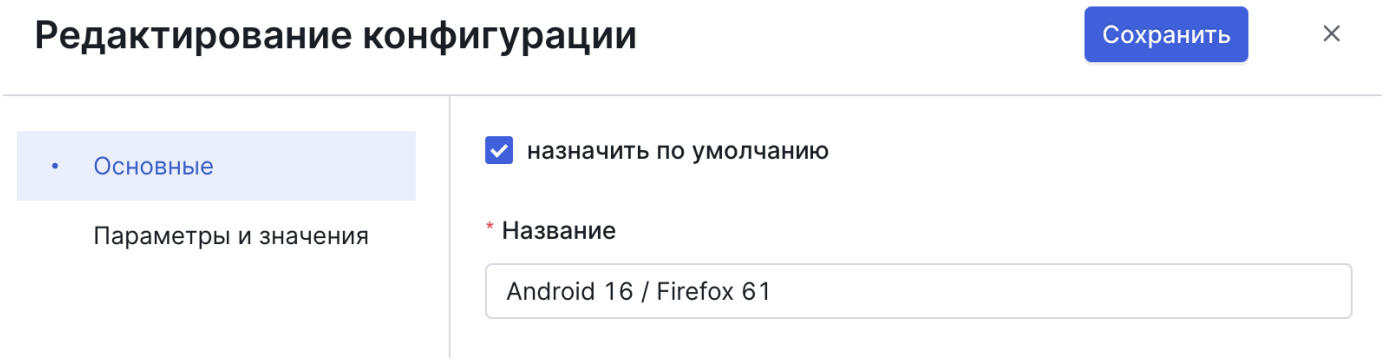
На примере ниже конфигурация операционной системы MacOS и браузера Safari.




3. Создайте еще несколько конфигурация самостоятельно. У меня получился такой набор:




4. Выберите одну из конфигураций и сделайте ее конфигурацией по умолчанию. Если этого не сделать, то при создании тест-плана будет ошибка.



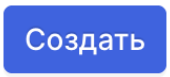

5. Теперь вы готовы к завершающему этапу: созданию тест-плана и тестовому прогону.

## Создание тест-плана и прогона

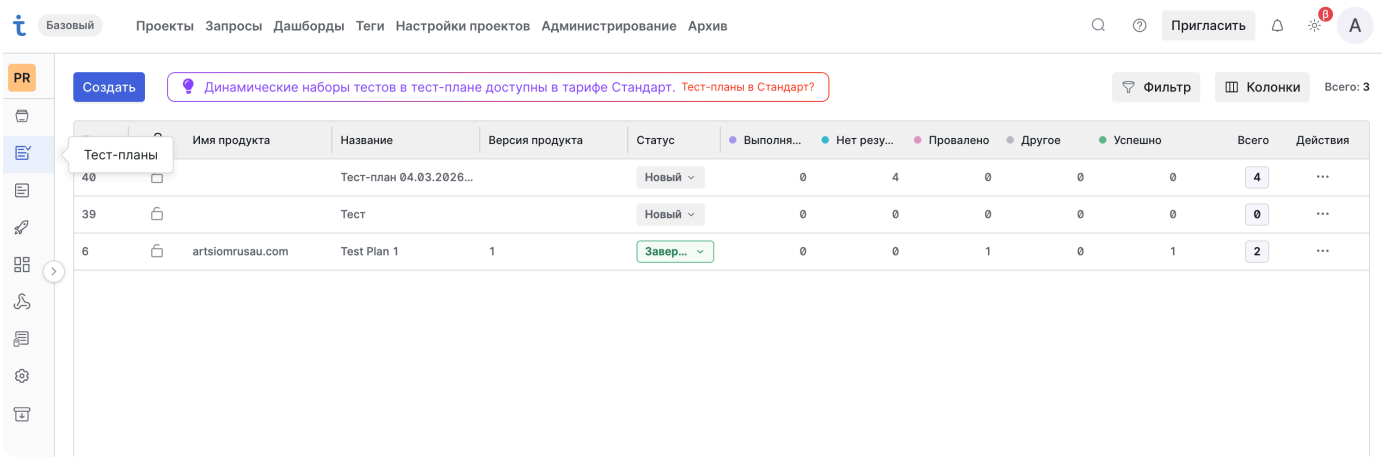
**Тест-план в Test IT** – это набор тестов, привязанный к определенному событию, который можно использовать в тестовом прогоне. Например, тест-план для регрессии, тест-план для smoke тестирования. Он может включать в себя кейсы из разных секций (наборов).

*Перед тем, как перейти к выполнению, убедитесь, что в ваши кейсы добавлена конфигурация.*

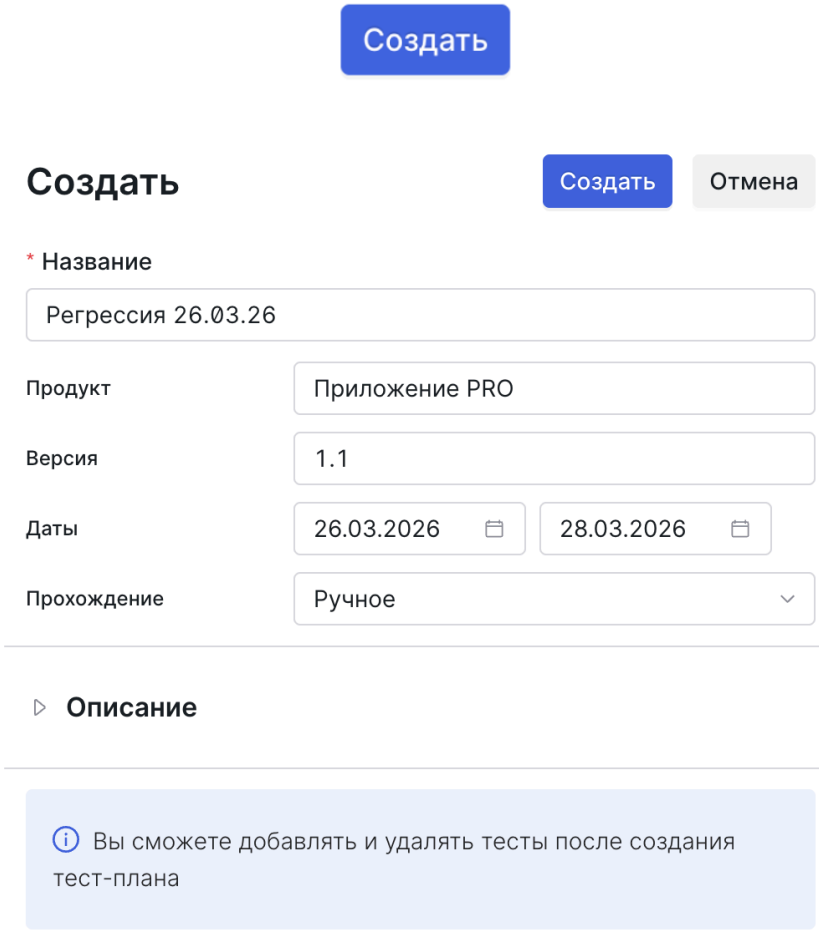
1. Перейдите во вкладку "Тест-планы" и нажмите



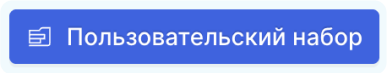

.



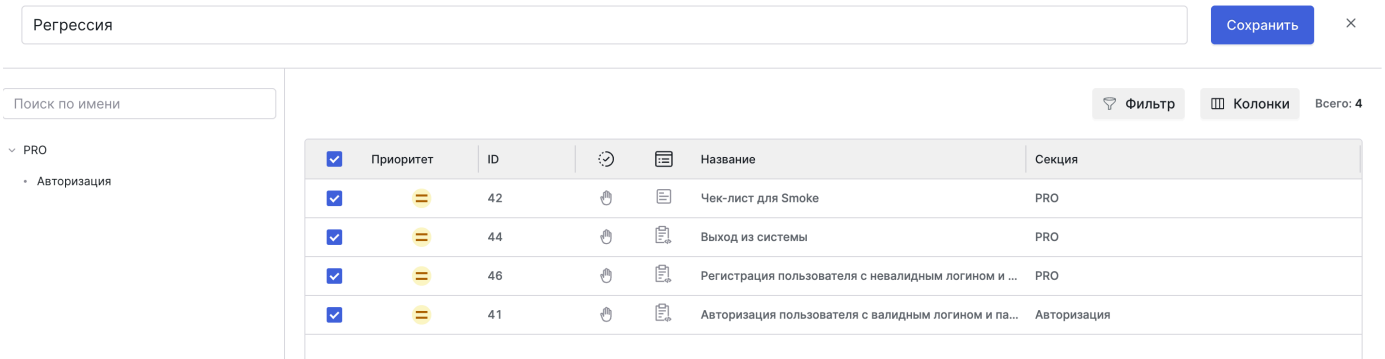

2. Заполните обязательные атрибуты. Обратите внимание, что  вы определяете период работы с планом. После заполнения нажмите



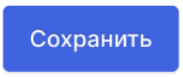

3. Откройте созданный план




и добавьте в него тест-кейсы и чек-лист.

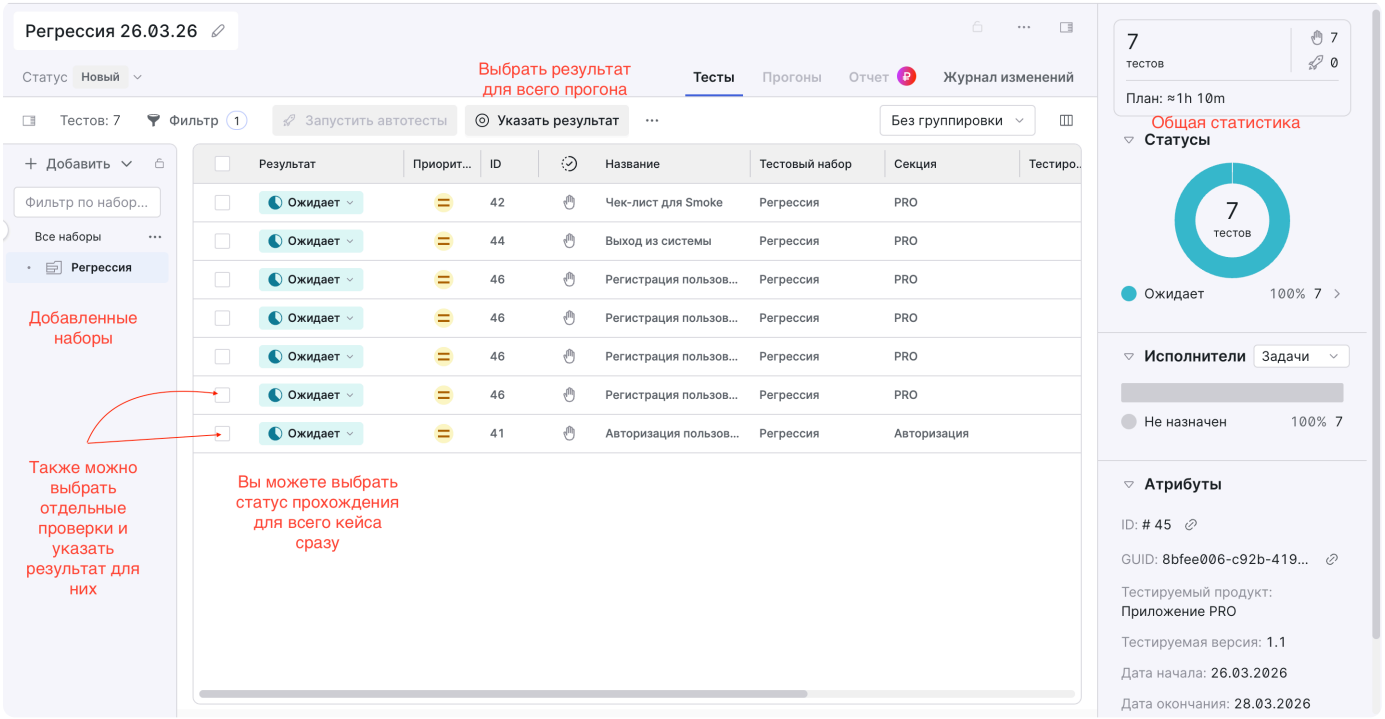



4. После выбора нажмите

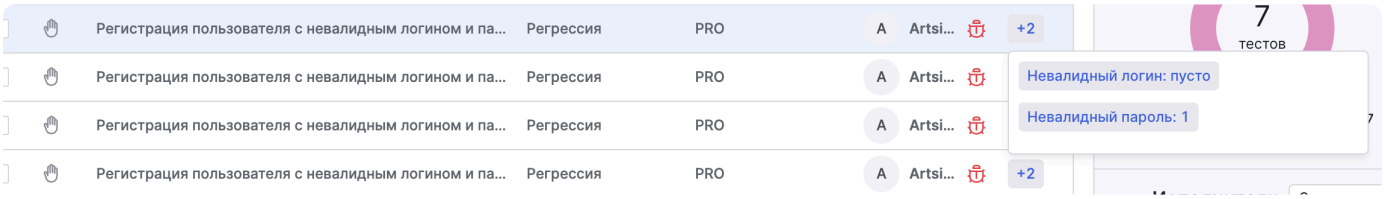


.

5. После добавления тестов вы можете приступить к тестированию и прогону.




Обратите внимание, что на тест-кейс с параметрами создалось 4 отдельные проверки. Параметры можно посмотреть как на странице тест-плана, так и внутри проверки.



**46 Регистрация пользователя с невалидным логином и паролем** Открыть

Описание
История
Вложения
Комментарии
Ссылки

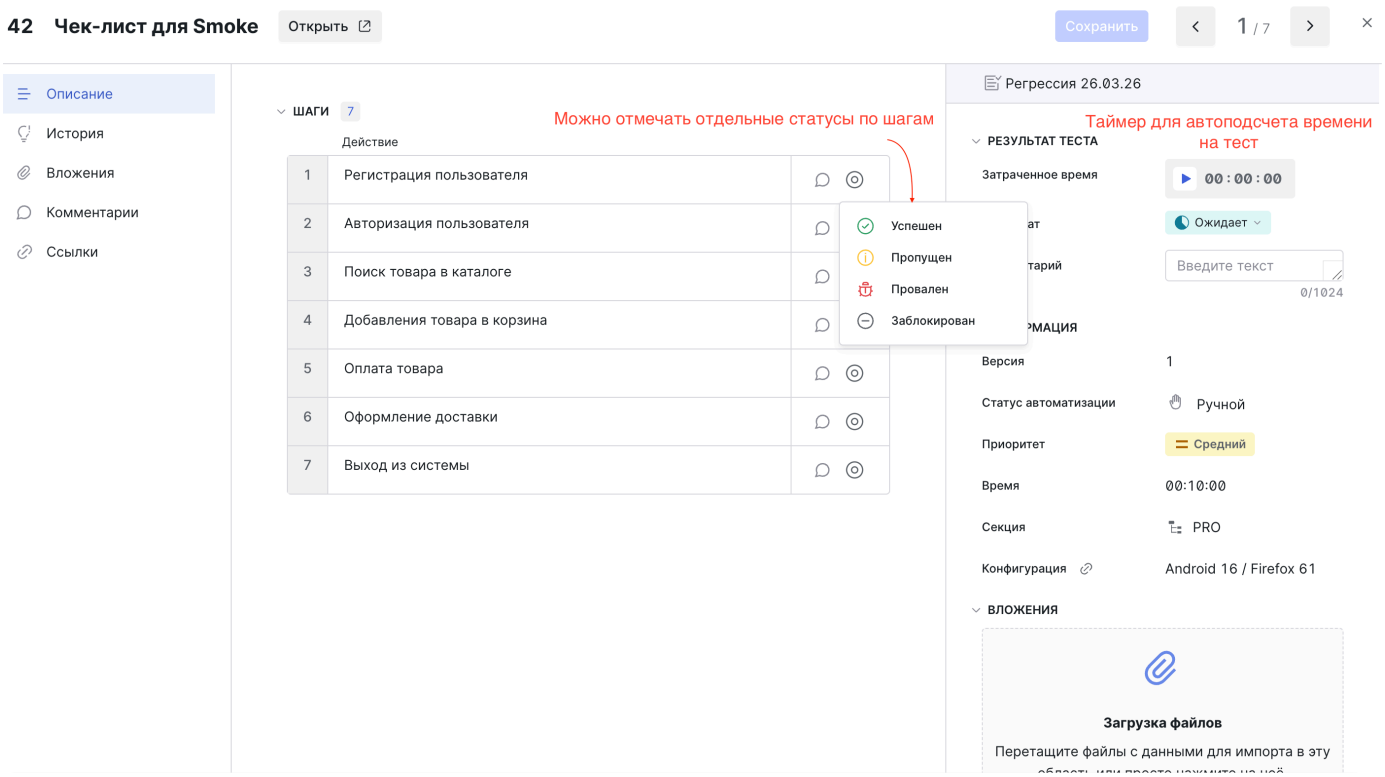
ПРЕДУСЛОВИЯ ТЕСТА

ШАГИ 4

| | Действие | Ожидаемый резуль... | Тестовые данные |
|---|---|---|---|
| 1 | Открыть example.com/register | Страница открывается без ошибок | |
| 2 | Ввести значение в поле “Логин” | Нет ошибок | пусто |
| 3 | Ввести значение в поле “Пароль” | Нет ошибок | 1 |
| 4 | Нажать кнопку “Регистрация” | Ошибка “Проверьте данные” | |

ПОСТУСЛОВИЯ ТЕСТА

6. При открытии чек-листа или кейса вы можете отмечать прогресс его выполнения.




7. В рамках тестового прогона вы проходите по всем проверкам и отмечаете результаты:

1. *Успешен* – тест прошел.
2. *Пропущен* – тест пропускается в прогоне, например, не хватило времени и он с низким приоритетом.
3. *Провален* – фактический результат не совпал с ожидаемым. Нужно завести дефект. Внутри Test IT контакты заводить нельзя, однако можно настроить интеграцию с другими приложениями.
4. *Заблокирован* – тест заблокирован из-за дефекта или неготовности функции.

8. После завершения тестирования на платных тарифах можно сформировать отчет.

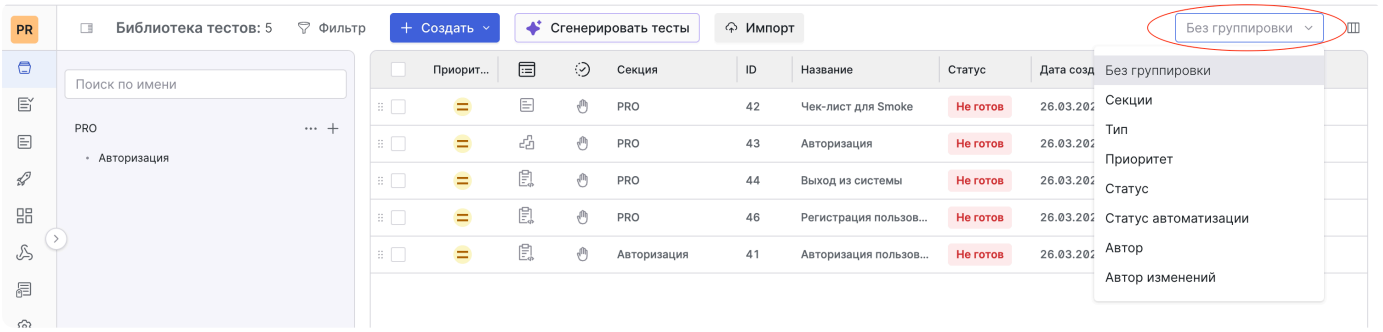
9. Поздравляю! Вы освоили базовые функции Test IT и готовы использовать его в своей практике!

## Дополнительные возможности

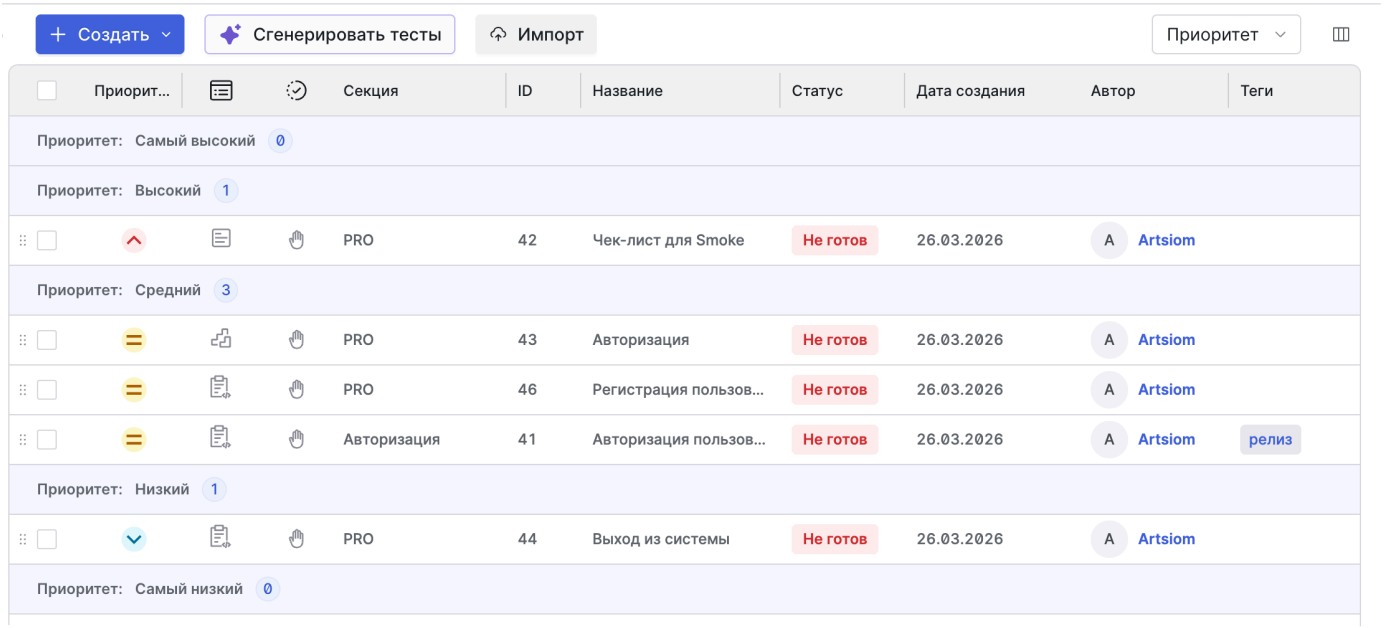
### Поиск, фильтрация, группировка

Одной из важных функций любой системы по управлению документации является поиск и фильтрация для формирования выборок: для регрессии, Smoke, для отслеживания прогресса, тестирования нужных модулей, сортировки и так далее.

### Группировка

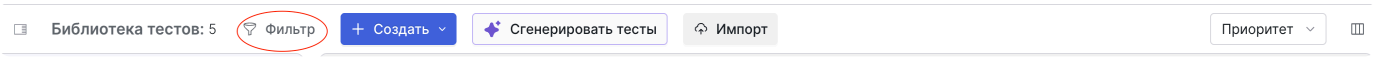



Например, при выборе группировки по приоритету, вы сразу увидите самые важные задачи.

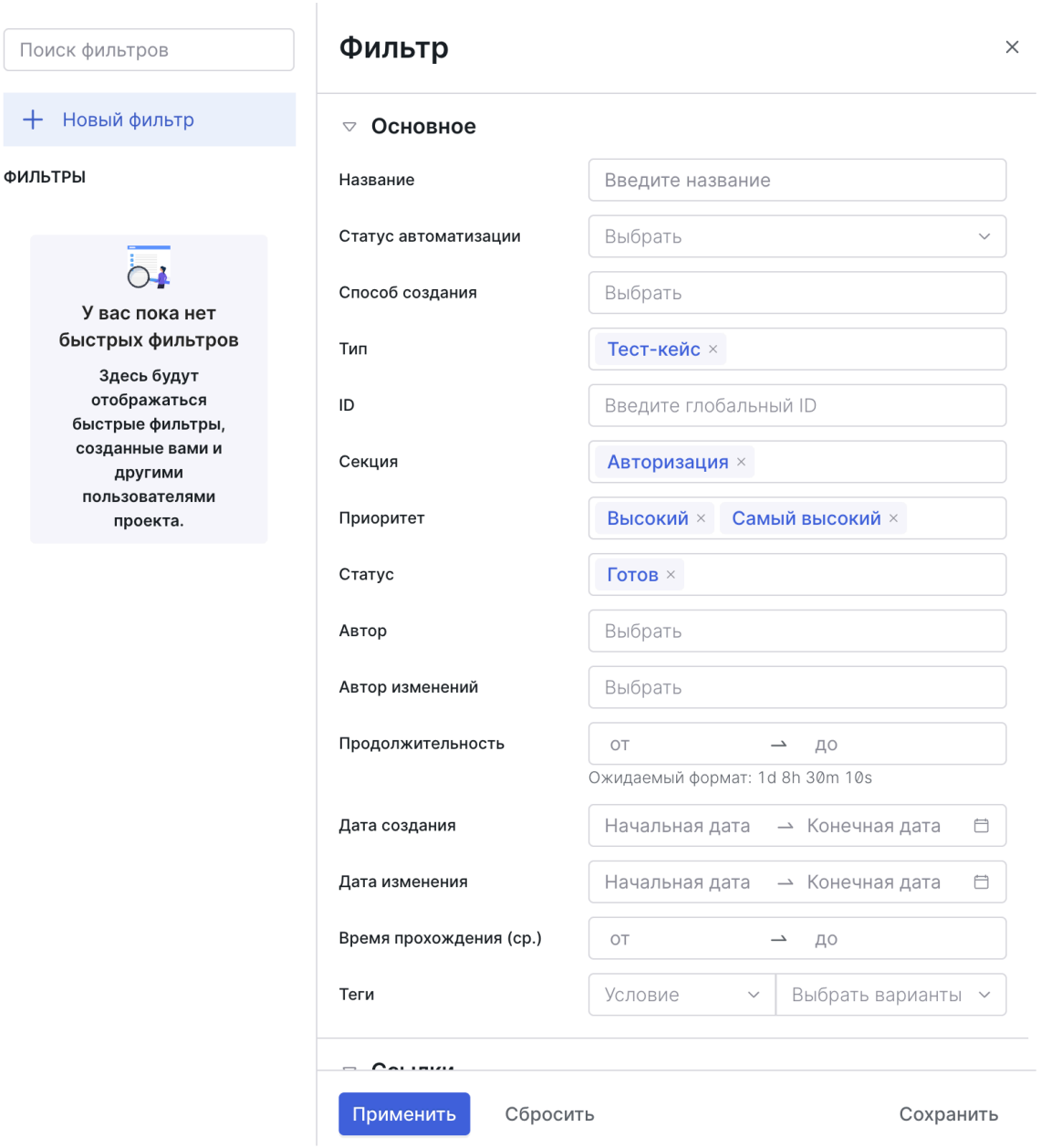



## Поиск и фильтрация

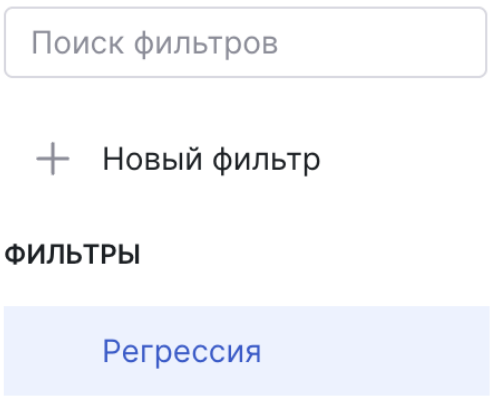
1. Нажмите на кнопку "Фильтр".



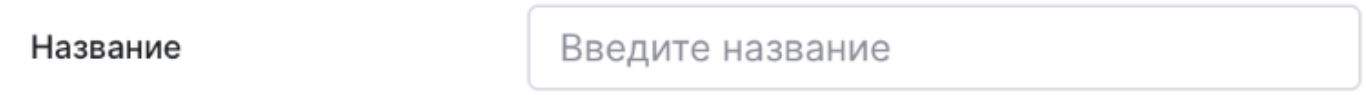

2. Представим, что вам надо сделать универсальный фильтр для поиска тестов в регрессионный набор. Отметим только тест-кейсы с высоким приоритетом, которые уже готовы и относятся к авторизации. Вы можете сразу применить или сохранить фильтр. Предлагаю сделать второй вариант.




После сохранения вы сможете выбрать этот фильтр из левого меню и применить его.



Также в фильтрации вы можете запускать поиск по названию:

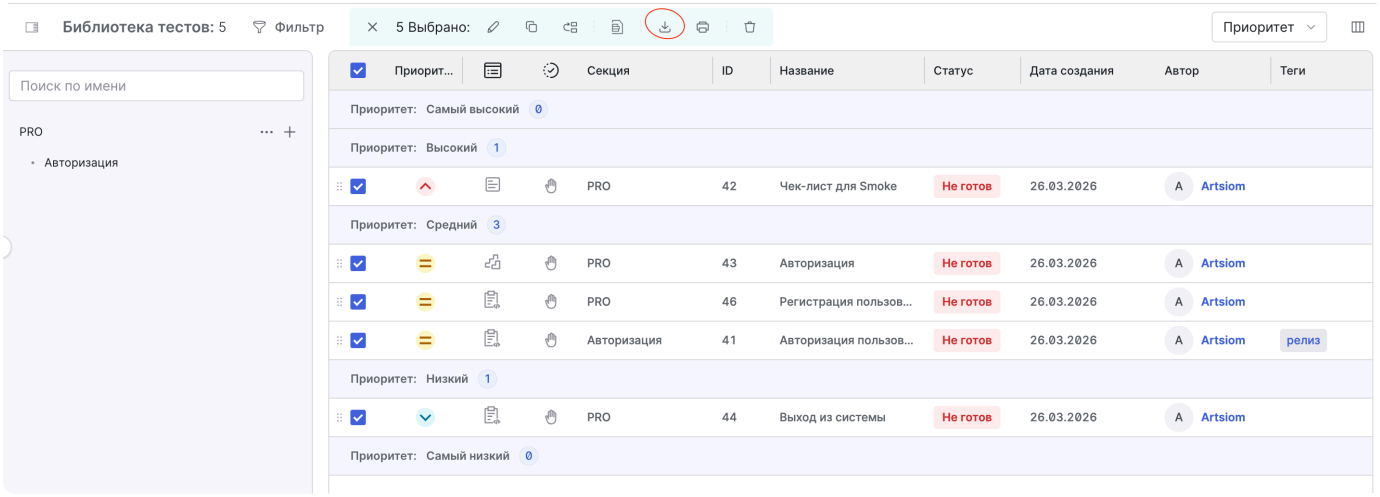



## Экспорт и импорт

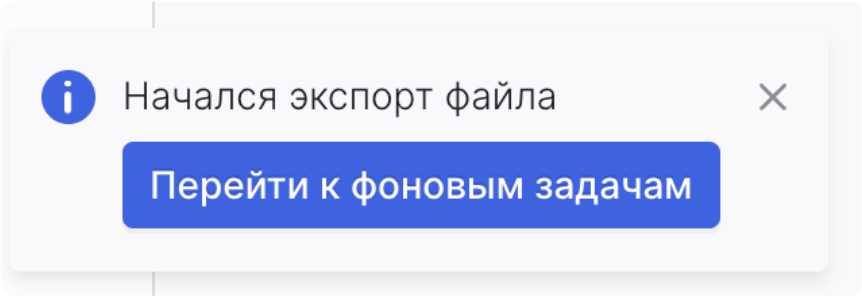
Экспорт (выгрузка) документация может быть полезна при миграции в другие системы и для отчетности.

1. Во вкладке "Библиотека тестов" отметьте нужные кейсы и нажмите

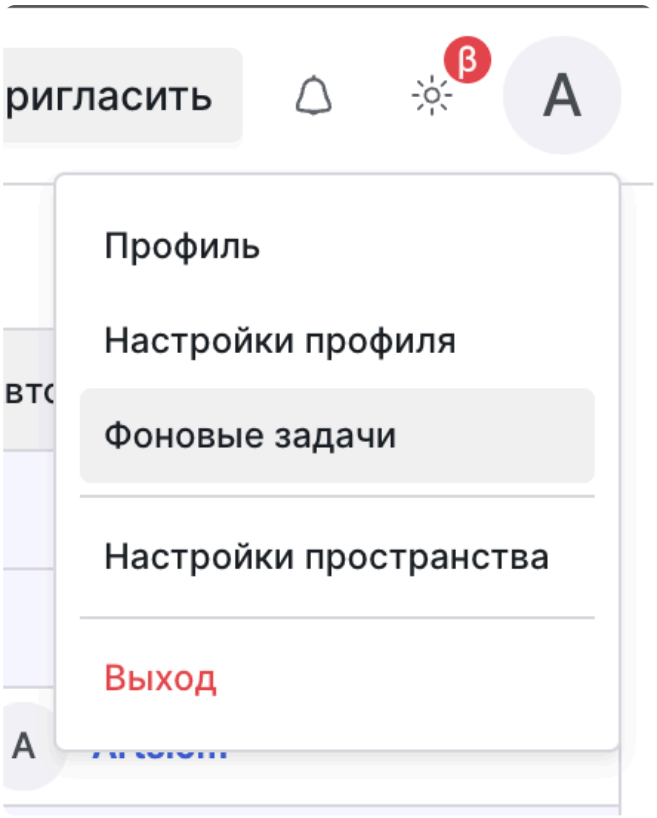
.



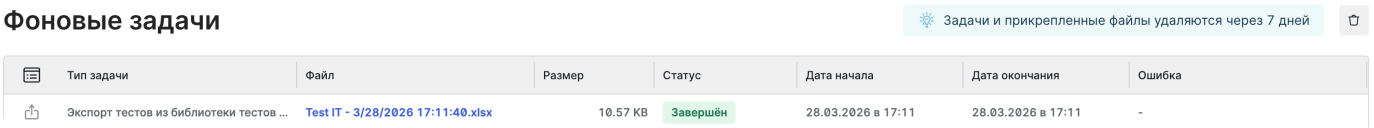

2. Запуститься фоновая задача, результат который можно найти, если при нажатии на кнопку в этом окне:




Или в меню профиля:

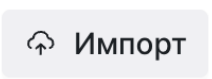



3. Сохраните файл на ваш компьютер.




> Лайфхак: если вы хотите получить вашу документацию в pdf, то вместо экспорта нажмите кнопку "Печать".
>
>
> 
>
> Далее все кейсы откроются в новом окне. Нажмите Ctrl + P и сохраните файл в pdf вместо печати.

4. Также вы можете сделать **импорт (загрузку) кейсов** из файла xlsx в Test IT. В библиотеке тестов нажмите

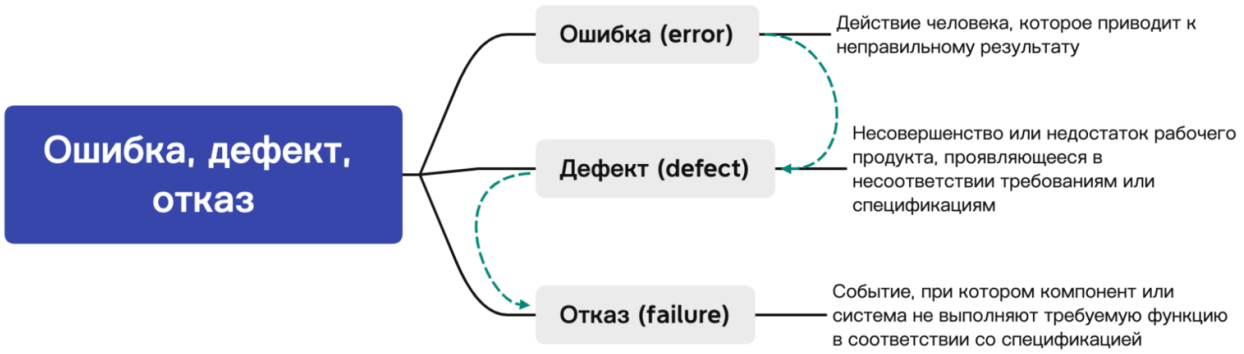



и загрузите свой файл. Для правильного импорта нужно самостоятельно настроить конфигурационный файл. В курсе мы не будем это разбирать.

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

# 23. Отчет о дефекте. Виды улучшений. Жизненный цикл дефекта

### Отличия ошибки, дефекта и отказа




**Отчет о дефекте (defect report, bug report)** – документирование возникновения, характера и состояния дефекта [ISTQB Glossary]

### Атрибуты отчета о дефекте

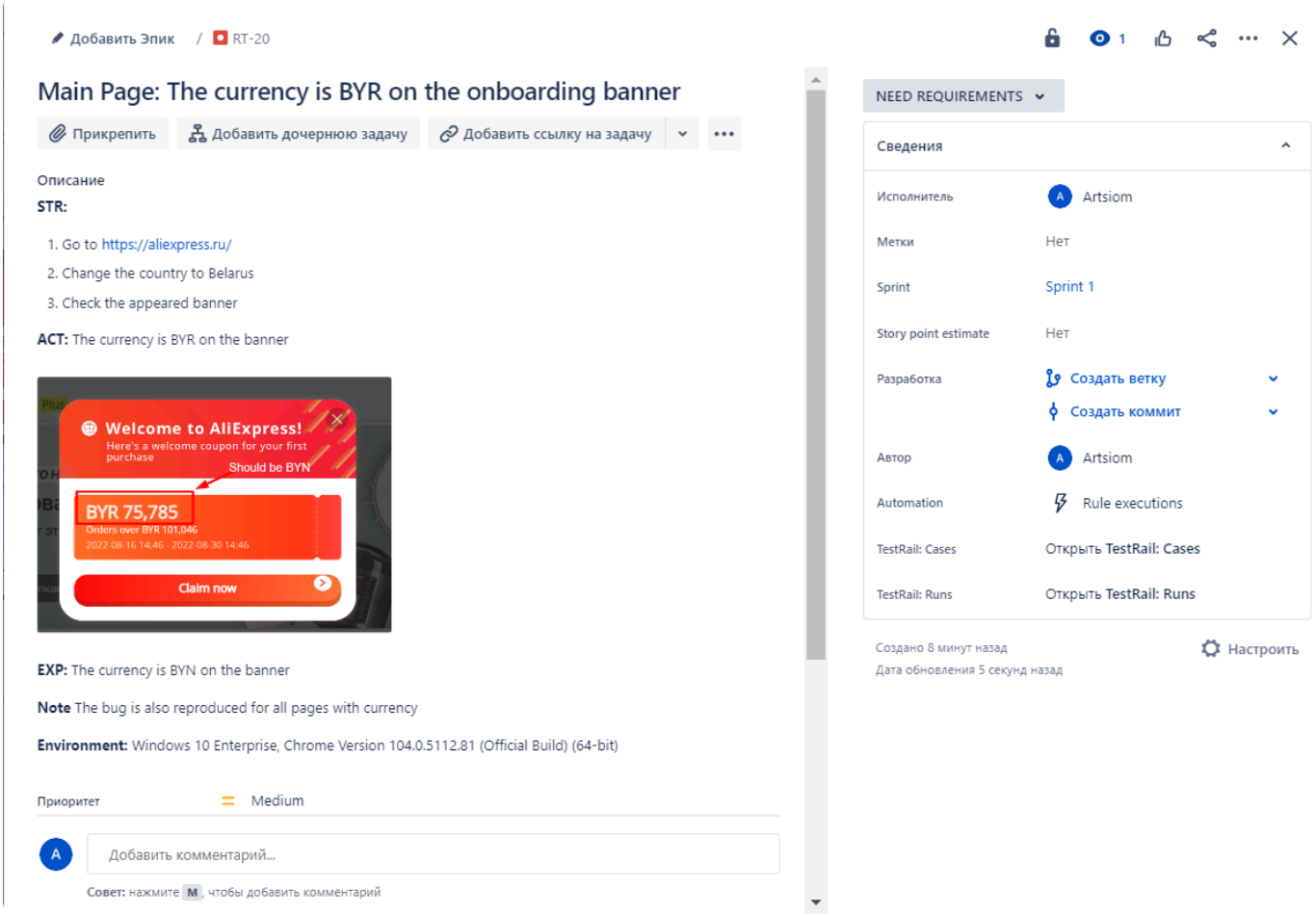
- **Идентификатор:** уникальный номер отчета. Присваивается автоматически в багтрекинговой системе (BTS, bug tracking system)
- **Краткое описание (summary):** отвечает на три вопроса: что, где, когда? Что произошло? Где это произошло? При каких условиях?
- **Подробное описание (description):** информация о дефекте в развернутом виде + фактический и ожидаемый результат + ссылка на требование
- **Шаги по воспроизведению (steps to reproduce, STR)**
- **Окружение (environment):** информация о среде, на которой был обнаружен баг (версия ОС, браузера, мобильного устройства)
- **Важность (severity)** - степень ущерба, который наносится проекту существованием дефекта. Серьезность для разрабатываемого ПО. Примеры на следующем шаге.
- **Срочность (priority) показывает, как быстро дефект должен быть устранён.** Примеры на следующем шаге.
- **Комментарий (comments, additional info)** дополнительные данные о дефекте, возможность упомянуть исполнителя

- **Вложения (attachments)** подтверждение дефекта: фото, видео, логи, тестовые данные, архивы и так далее

Также в дефектах обычно есть **следующие поля:**

1. Статус в жизненном цикле
2. Информация об исполнителе
3. Ссылки на связанные артефакты

### Пример отчета о дефекте




**Важность (severity)** - степень ущерба, который наносится проекту существованием дефекта. Серьезность для разрабатываемого ПО.

- *Критическая (critical)* — существование дефекта приводит к масштабным последствиям катастрофического характера
- *Высокая (major)* — существование дефекта приносит ощутимые неудобства многим пользователям в рамках их типичной деятельности
- *Средняя (medium)* — существование дефекта слабо влияет на типичные сценарии работы пользователей, и/или существует обходной путь

достижения цели
- *Низкая (minor)* — существование дефекта редко обнаруживается незначительным процентом пользователей и (почти) не влияет на их работу

**Срочность (priority) показывает, как быстро дефект должен быть устранён.**

1. *Наивысшая (ASAP, as soon as possible)* срочность указывает на необходимость устранить дефект настолько быстро, насколько это возможно.
2. *Высокая (high) с*рочность означает, что дефект следует исправить вне очереди, т.к. его существование или уже объективно мешает работе, или начнёт создавать такие помехи в самом ближайшем будущем.
3. *Обычная (normal)* срочность означает, что дефект следует исправить в порядке общей очерёдности. Такое значение срочности получает большинство дефектов.
4. *Низкая (low)* срочность означает, что в обозримом будущем исправление данного дефекта не окажет существенного влияния на повышение качества продукта.

### Популярный вопрос про Priority и Severity

Один из популярных вопросов на интервью: привести примеры багов с высоким приоритетом и низкой серьезностью и наоборот.

**Высокий приоритет и низкая серьезность**

Такое сочетание бывает, когда баг на функционал влияет незначительно, но зато на пользовательский опыт влияет очень сильно. Также в эту категорию попадают баги, не влияющие на программу, но требующие исправления.

Вот пара примеров:

1. Кнопки перекрывают друг друга. Они кликабельны, но визуальное впечатление портится.
2. Логотип компании на главной странице содержит орфографическую ошибку. На функционал это вообще не влияет, но портит пользовательский опыт. Этот баг нужно исправить с высоким приоритетом, несмотря на то, что на продукт он влияет минимально.

**Высокая серьезность и низкий приоритет**

Такое сочетание бывает у багов, которые возникают в отдельных функциях программы. Эти баги не позволяют пользоваться системой, при этом обойти их невозможно. Но сами функции, содержащие эти дефекты, конечным потребителем используются  редко.

**Примеры:**

1. Домашняя страница сайта ужасно выглядит в старых браузерах. Перекрывается текст, не загружается логотип. Это мешает пользоваться продуктом, поэтому серьезность бага высокая. Но так как очень мало пользователей открывают сайт при помощи устаревшего браузера, такой баг получает низкий приоритет.
2. Допустим, у нас есть приложение для банкинга. Оно правильно рассчитывает ежедневный, ежемесячный и ежеквартальный отчет, но при расчете годового возникают проблемы. Этот баг имеет высокую степень серьезности. Но если сейчас формирование годовой отчетности не актуально, такой дефект имеет низкий приоритет: его можно исправить в следующем релизе.

**Советы по созданию вложений**

1. Используйте программы для записи экрана и фото: SnagIt, ShareX, Lightshot, Monosnap
2. Не рисуйте от руки, все уже есть в программе
3. Захватывайте часть экрана, по которой без проблем можно найти искомый объект
4. Локализуйте баг на скрине, используя стрелки, красные рамки, надписи
5. Называйте вложения либо заголовком самого отчета, либо его идентификатором
6. Отключайте звук при записи видео (если это только не звуковой баг)
7. Не показывайте на вложении личные данные
8. При тестировании веб на скрине должен присутствовать открытый DevTools
9. При тестировании мобильных приложений нужно прикладывать логи
10. Если для воспроизведения дефекта необходимо определенные тестовые файлы и данные, то их тоже стоит приложить
11. Сокращайте продолжительность видео и оставляйте только ключевые моменты

## Жизненный цикл дефекта

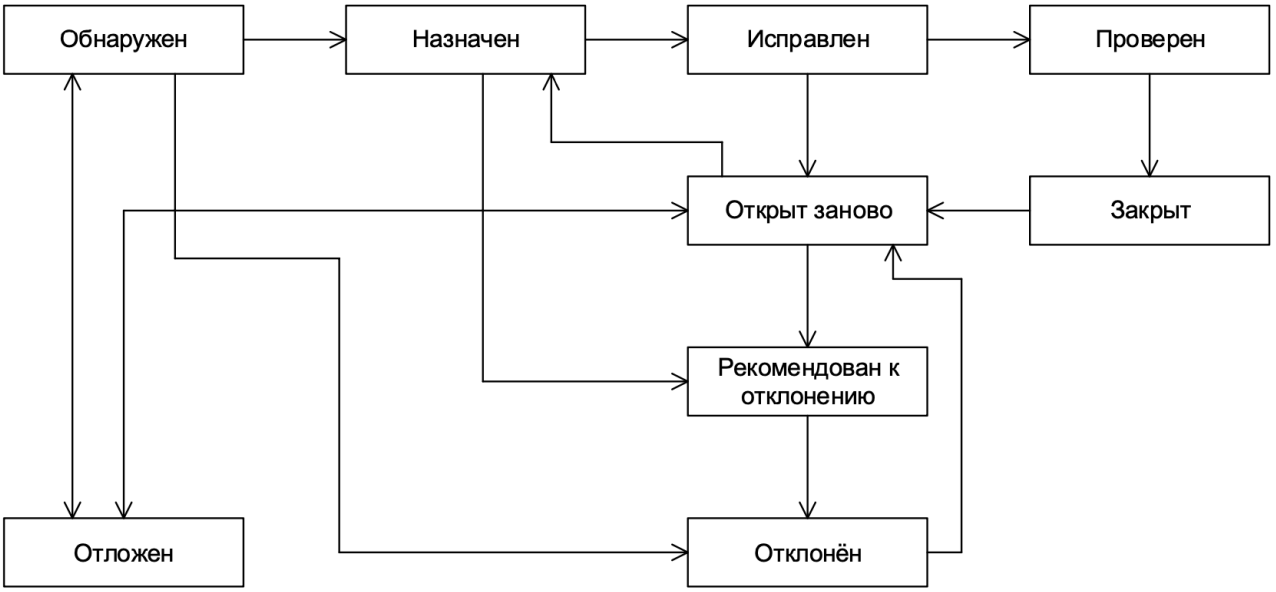


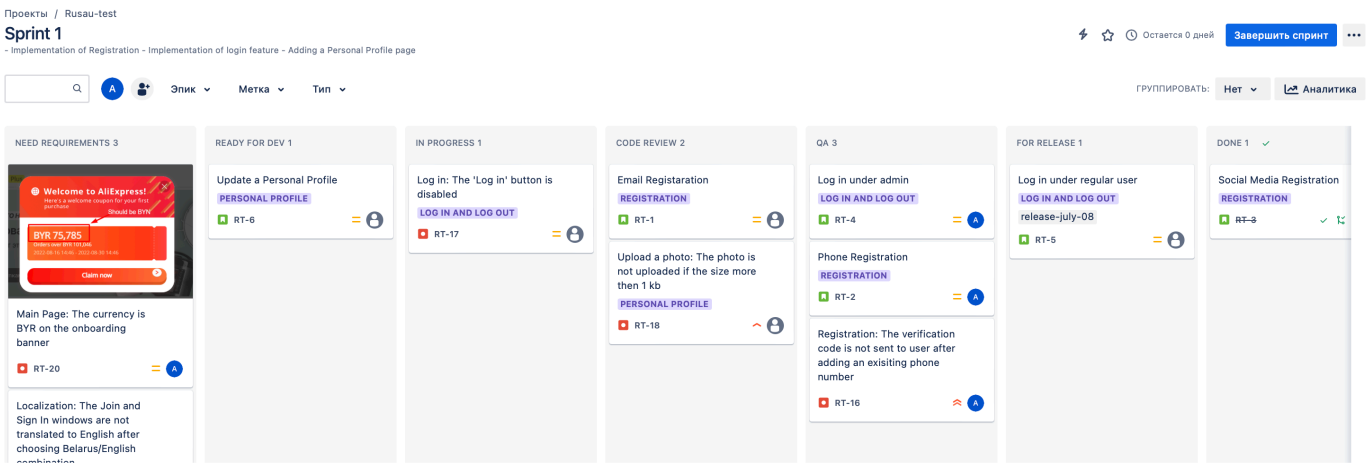

Рисунок 2.5.c — Жизненный цикл отчёта о дефекте с наиболее типичными переходами между состояниями

*Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”*

## Примечание автора курса

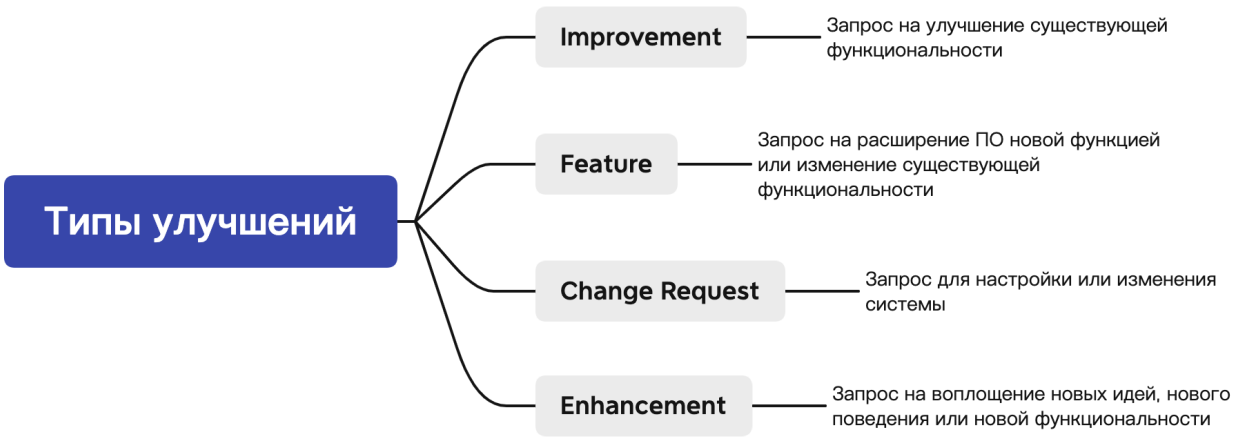
На реальных проектах вы будете работать в багтрекинговых системах по типу Jira на специальных досках. Обычно они представляют собой бэклог текущего спринта, элементы которого находятся в разных статусах.

На доске могут находиться пользовательские истории, задачи (таски) разработчиков и тестировщиков к ним, а также отчеты о дефекте. Жизненный цикл есть не только у дефектов, но и у всех задач и зависит от принятого процесса (workflow) в команде.



*Шпаргалка (адаптирована под десктоп) https://rusau.net/bug-cycle*

## Типы улучшений




## Рекомендации

1. Не все дефекты будут исправлены. Их приоритет зачастую определяет Project/Product Manager.
2. В случае накопления большого количества дефектов в бэклоге необходимо проводить Bug Triage (сортировка багов). Это собрание со всей команды с целью определения приоритетов по исправлению багов.
3. Обращайте внимание на наличие всех атрибутов. Грамотный отчет = счастливый разработчик :)
4. Все дефекты должны быть задокументированы
5. Несколько дефектов можно объединять между собой, если:

- это визуальные баги, которые находятся в одном модуле
- дефект воспроизводится одинаково для разных полей (например, одинаковая ошибка при валидации).

*В таком случае можно указать тот факт, что дефект воспроизводится в нескольких местах как в описании, так и в комментариях.*

## Полезные материалы

1. **ДИСКЛЕЙМЕР:** В ПРЕЗЕНТАЦИИ УКАЗАНЫ ПРОСТО АТРИБУТЫ БЕЗ ПРИВЯЗКИ К ИХ РАСПОЛОЖЕНИЮ В ОТЧЕТЕ О ДЕФЕКТЕ. КОНКРЕТНЫЕ

ПРИМЕРЫ ВЫ ПОСМОТРИТЕ УЖЕ В СЛЕДУЮЩИХ ЛЕКЦИЯХ. Сначала пишут шаги, потом результаты.

2. Общая презентация
3. Шпаргалка (адаптирована под десктоп) https://rusau.net/bug
4. Старые версии уроков https://youtu.be/rqcI-xLzpnc
5. https://youtu.be/6YrgKBTzb5o
6. https://youtu.be/p4SwlO9XDH0
7. Ссылка на схему из старой версии урока "Отчет о дефекте" https://drive.google.com/drive/u/3/folders/1toLPOMPC6y8cl5gsVOYKLXK-uESDQP2h
8. Схема из старой версии урока про "Улучшения" https://drive.google.com/file/d/1zfPmPglPNfzaz3HyPi-dStz0u6kcP1VG/view?usp=sharing
9. Программа подготовки к ISTQB CTFL (п.1.2.3. Ошибки, дефекты и отказы) https://www.rstqb.org/ru/istqb-downloads.html
10. Схема с жизненным циклом дефекта - https://www.softwaretestinghelp.com/bug-life-cycle/
11. Библия QA. Дефекты и ошибки - https://vladislaveremeev.gitbook.io/qa_bible/obshee/defekty-i-oshibki
12. Bug reporting for dummies, или Шпаргалка молодого тестировщика https://dou.ua/lenta/articles/bug-reporting-for-dummies-ili-shpargalka-molodogo-testirovshika/
13. Техника написания хороших кратких Summary баг-репортов https://svyatoslav.biz/education/bug-reports-summary/

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

# 24. Работа в Jira (версия 2026)

### Прочитайте перед просмотром

> Видео-часть в курсе может содержать интерфейс более ранних версий приложения. В текстовой части наиболее актуальная информация.

**Вы можете выбрать любую систему из предложенных для выполнения заданий на курсе.** Настоятельно рекомендую посмотреть видео про Youtrack, так как оно самое свежее и содержит примеры отчетов о дефекте, рекомендации по их созданию.

Также во многих вакансиях есть JIra, так как это самый популярный инструмент для управления проектами.  Поэтому урок по работе с ней, а также видео про JQL, лучше посмотреть.

Если вы научитесь работать в одной системе, то без проблем сможете переключиться на другую, так как принципы взаимодействия и функции там одинаковые. Отличается только интерфейс.

Для работы в Jira и Youtrack из России и Беларуси, скорее всего, понадобится VPN.

**Jira** – это система по управлению проектами, в том числе используется для создания отчетов о дефекте. Именно поэтому в корпоративной среде такие системы называют **багтрэкинговыми**.

Разработана компанией Atlassian и является самым популярным решением на рынке. В России и Беларуси для работы с ней может понадобиться VPN.

В рамках данного урока мы разберем основные функции Jira, которые могут пригодиться тестировщику каждый день.

Все остальное можно прочитать в официальной документации.

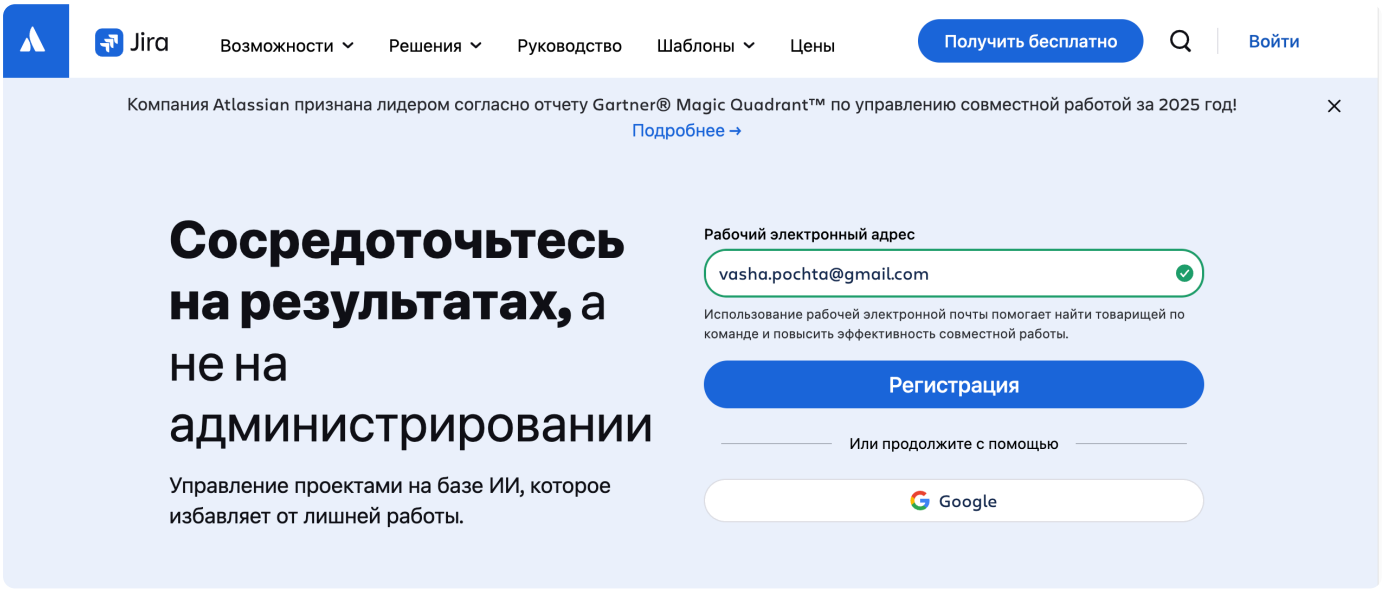
### Давайте зарегистрируемся в ней и создадим первый проект

1. Откройте https://www.atlassian.com/ru/software/jira и измените язык на русский в правом нижнем углу страницы

.

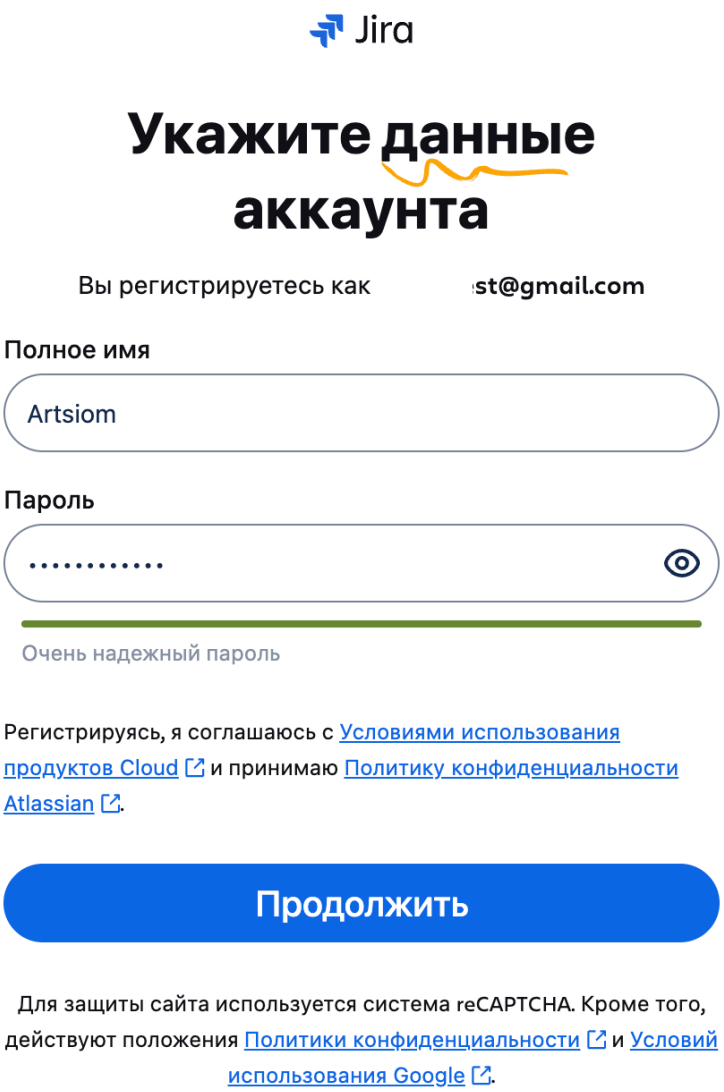
2. Введите вашу почту и нажмите

. Используйте почты, которые заканчиваются на .com.



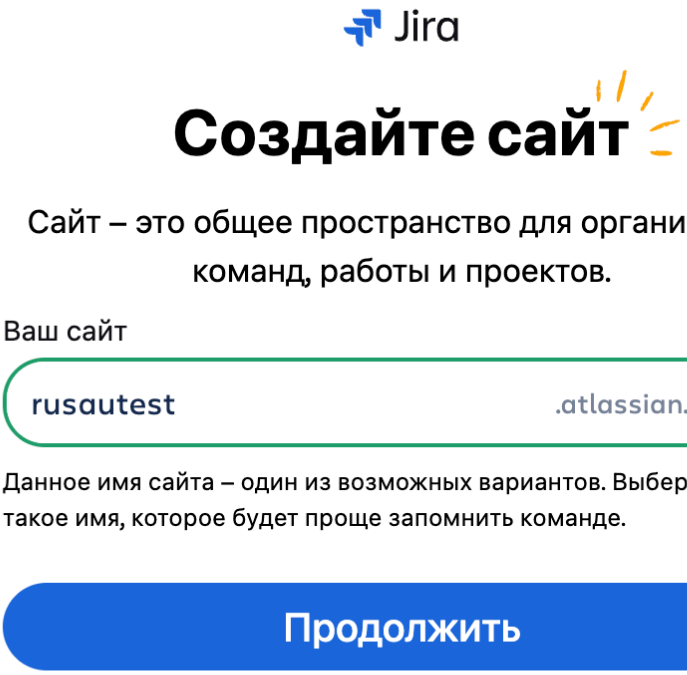

3. Введите имя и придумайте пароль. Нажмите

.



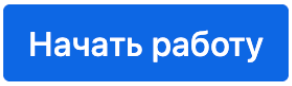

4. Система автоматически предложит домен для вашего пространства. Нажмите

.

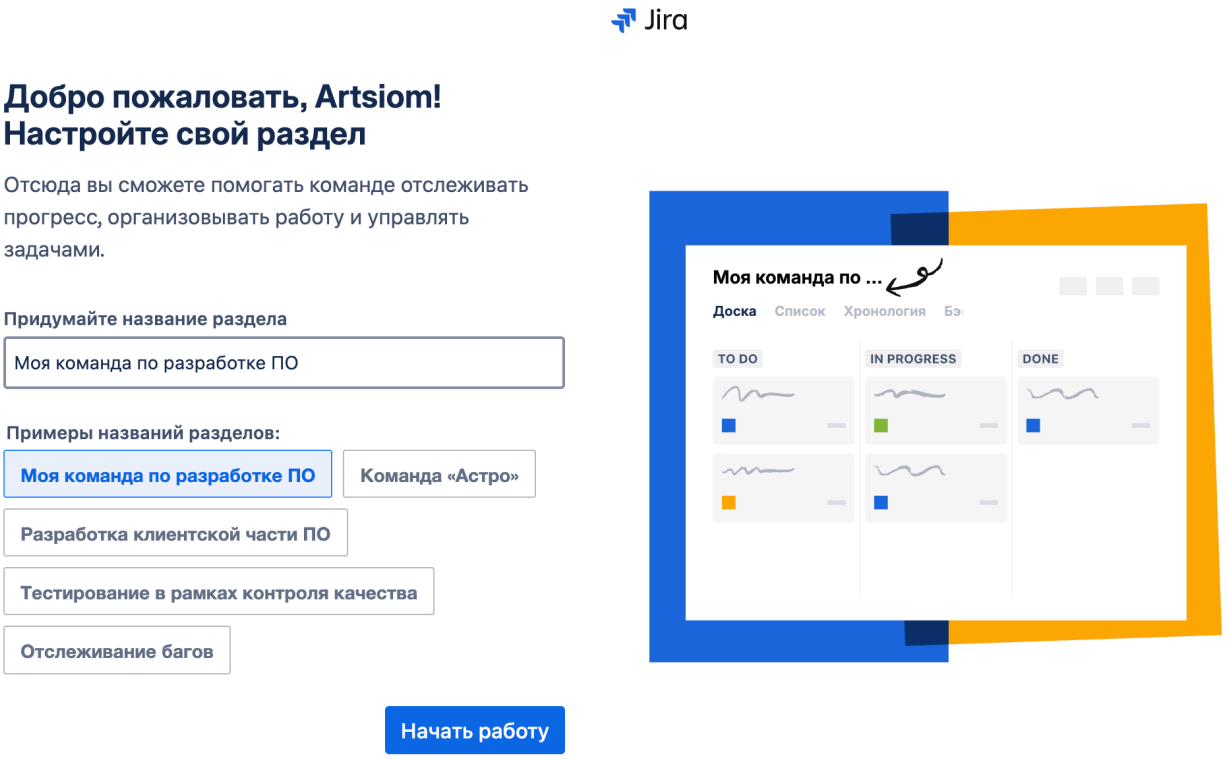



5. Дождитесь конца подготовки пространства. Далее выберете вашу сферу и цели использования. Они могут быть любыми.

6. Придумайте название раздела, или оставьте значение по умолчанию. Нажмите



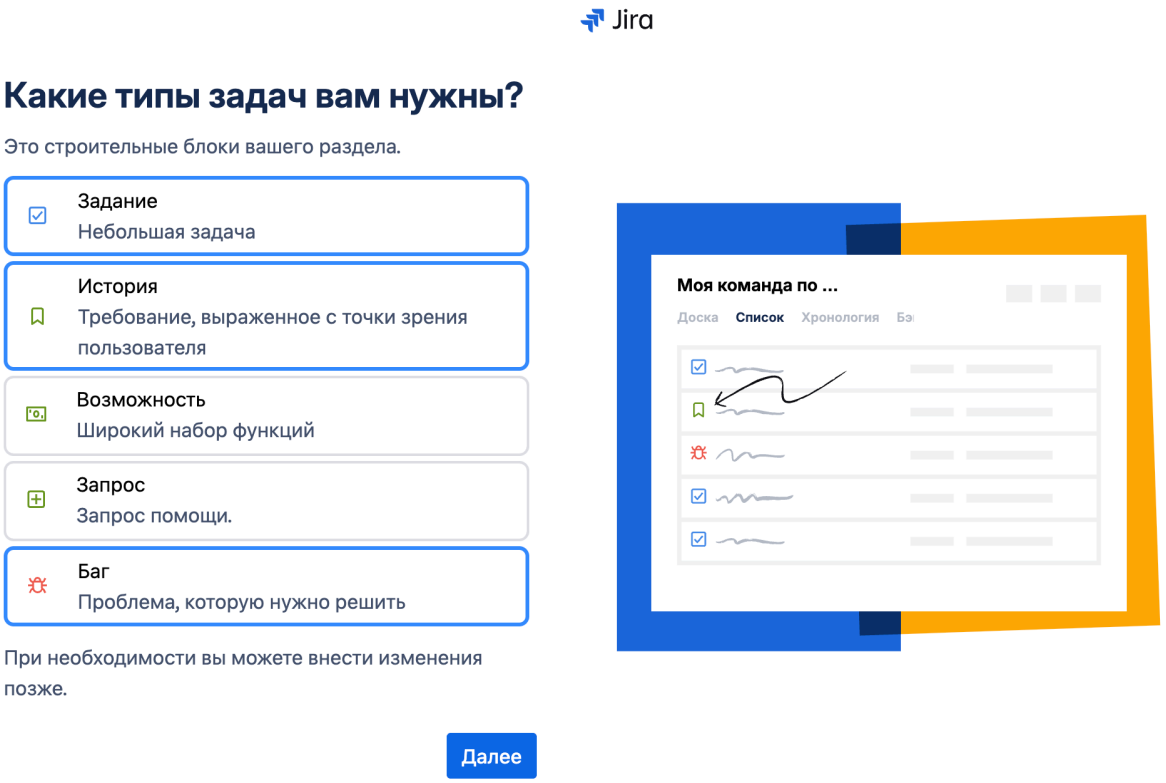

.



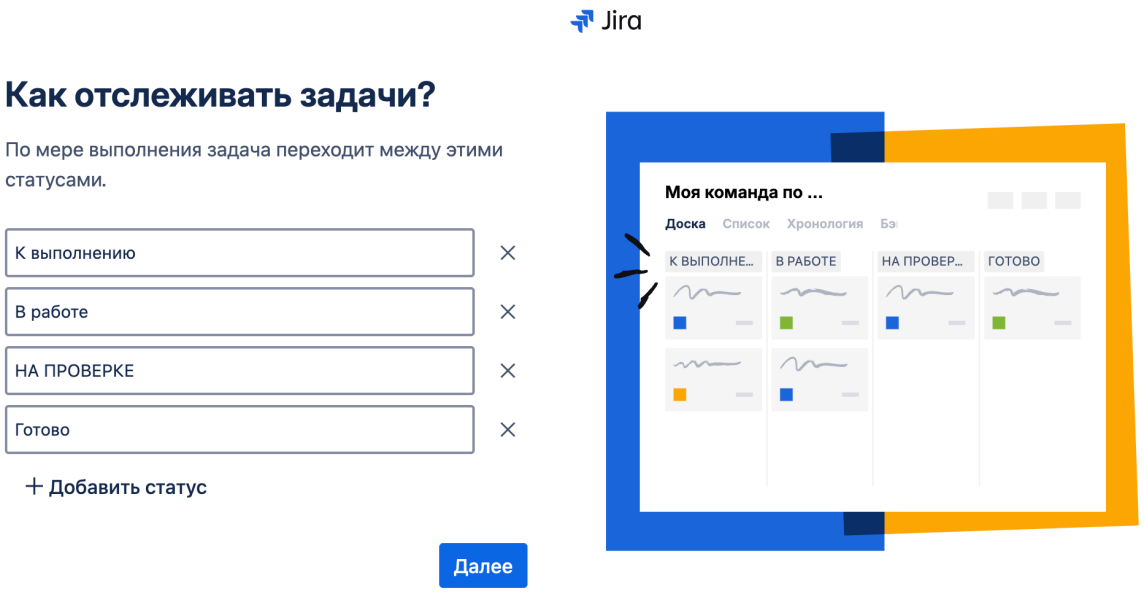

7. Выберите типы задач, как на скрине. Нажмите

Далее

.



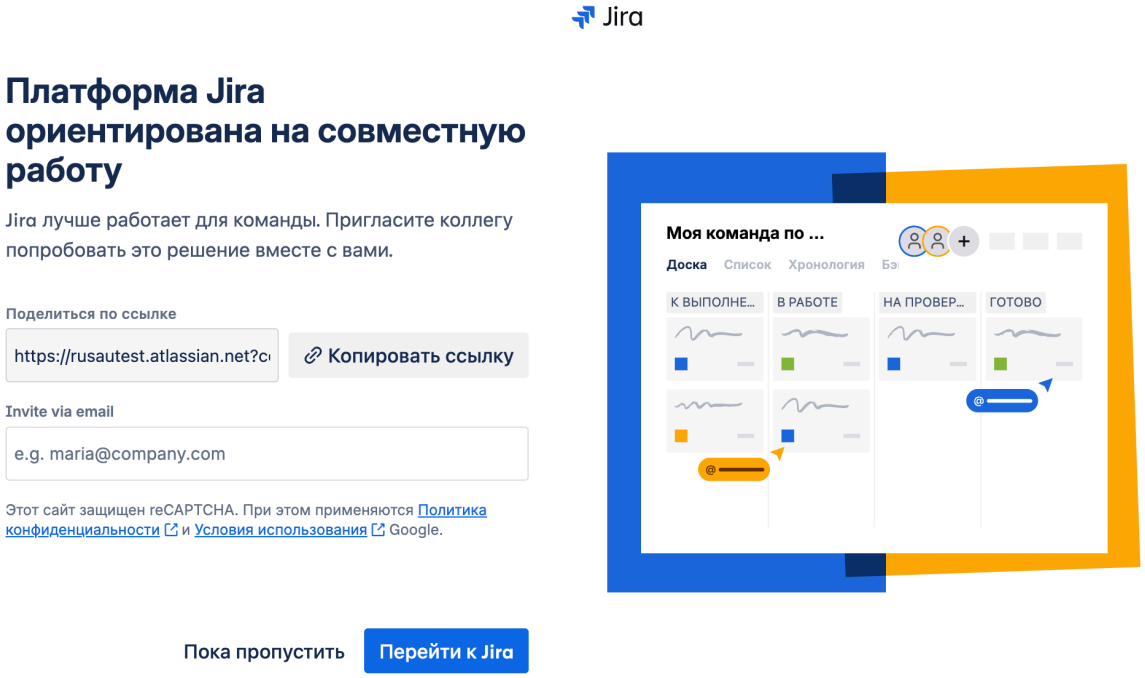

8. Вы можете добавить новые статусы для задач. Я оставлю значения по умолчанию.



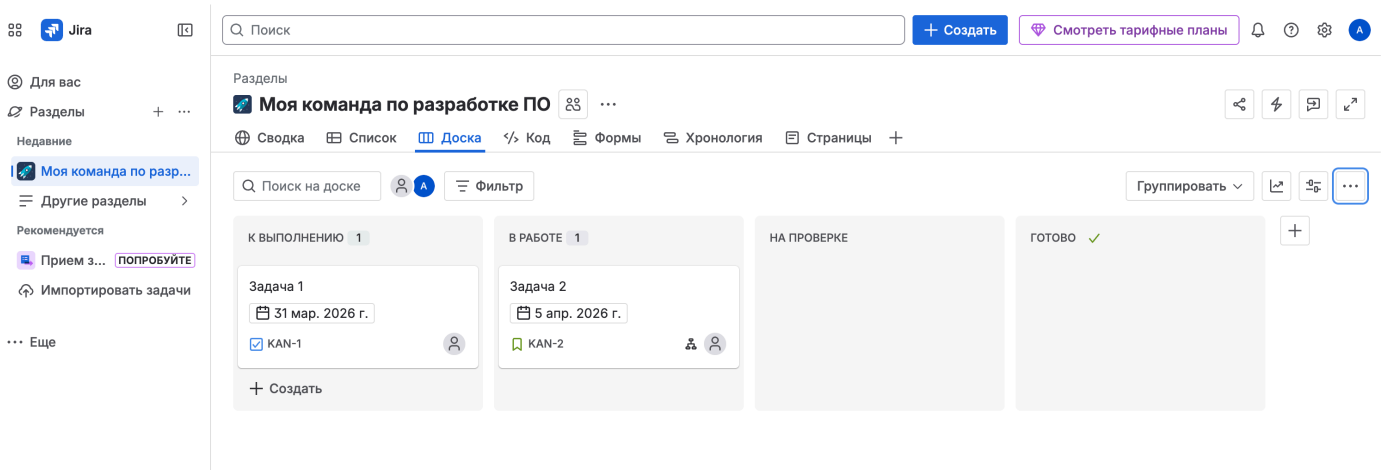
9. На последнем этапе вы можете добавить до 5 человек в команду на бесплатном тарифе. Этот шаг можно пропустить и сразу перейти к

Перейти к Jira

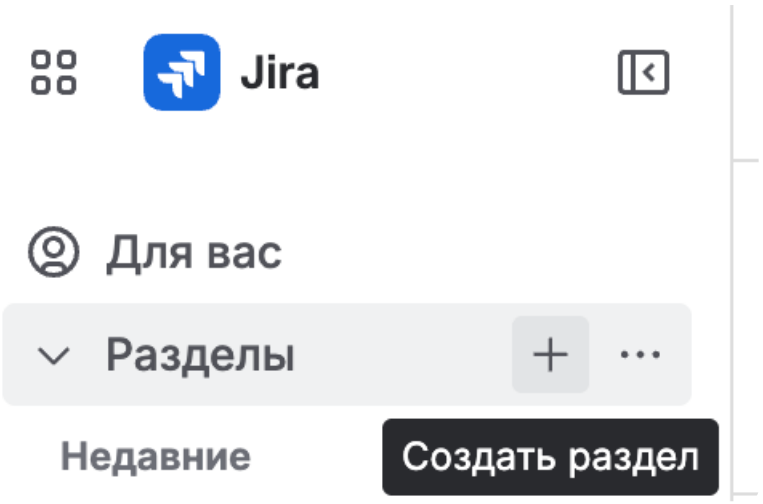
.

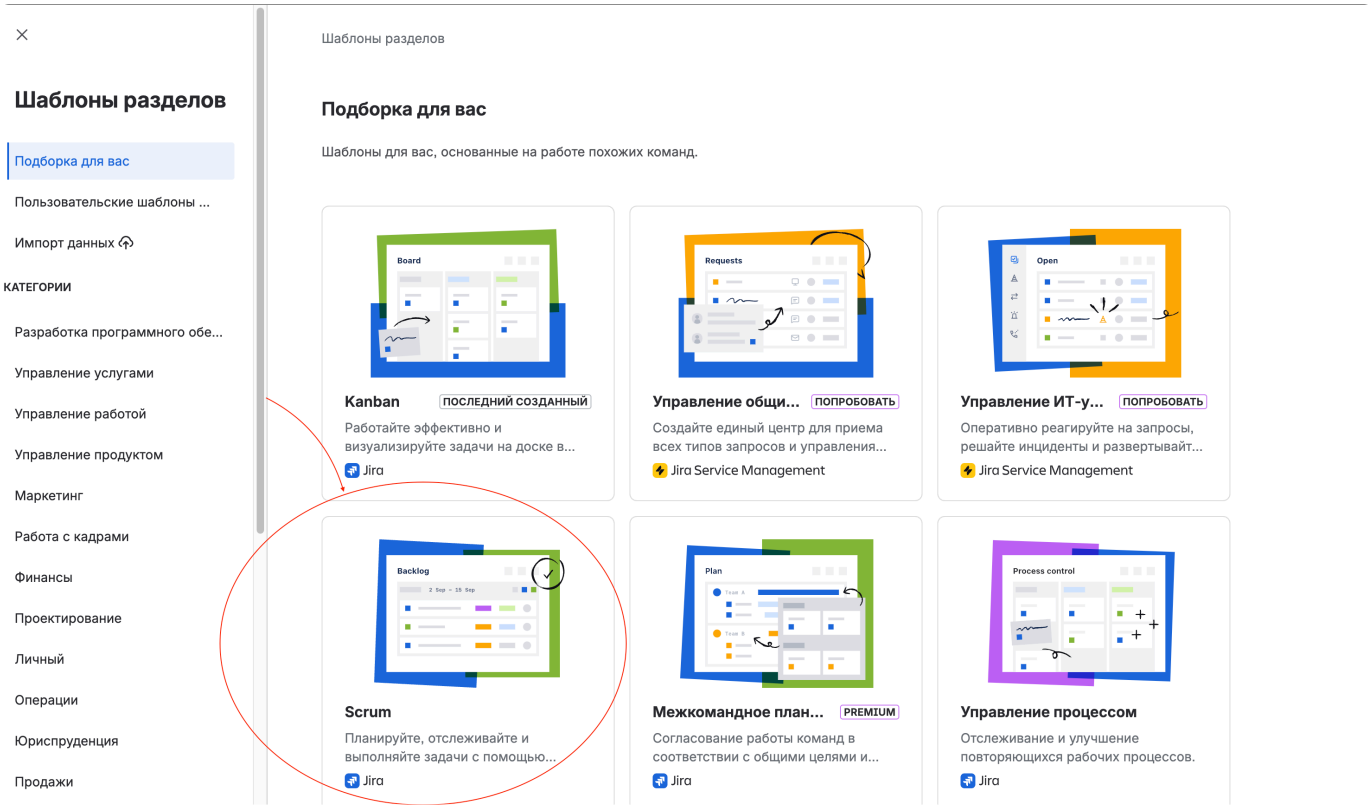


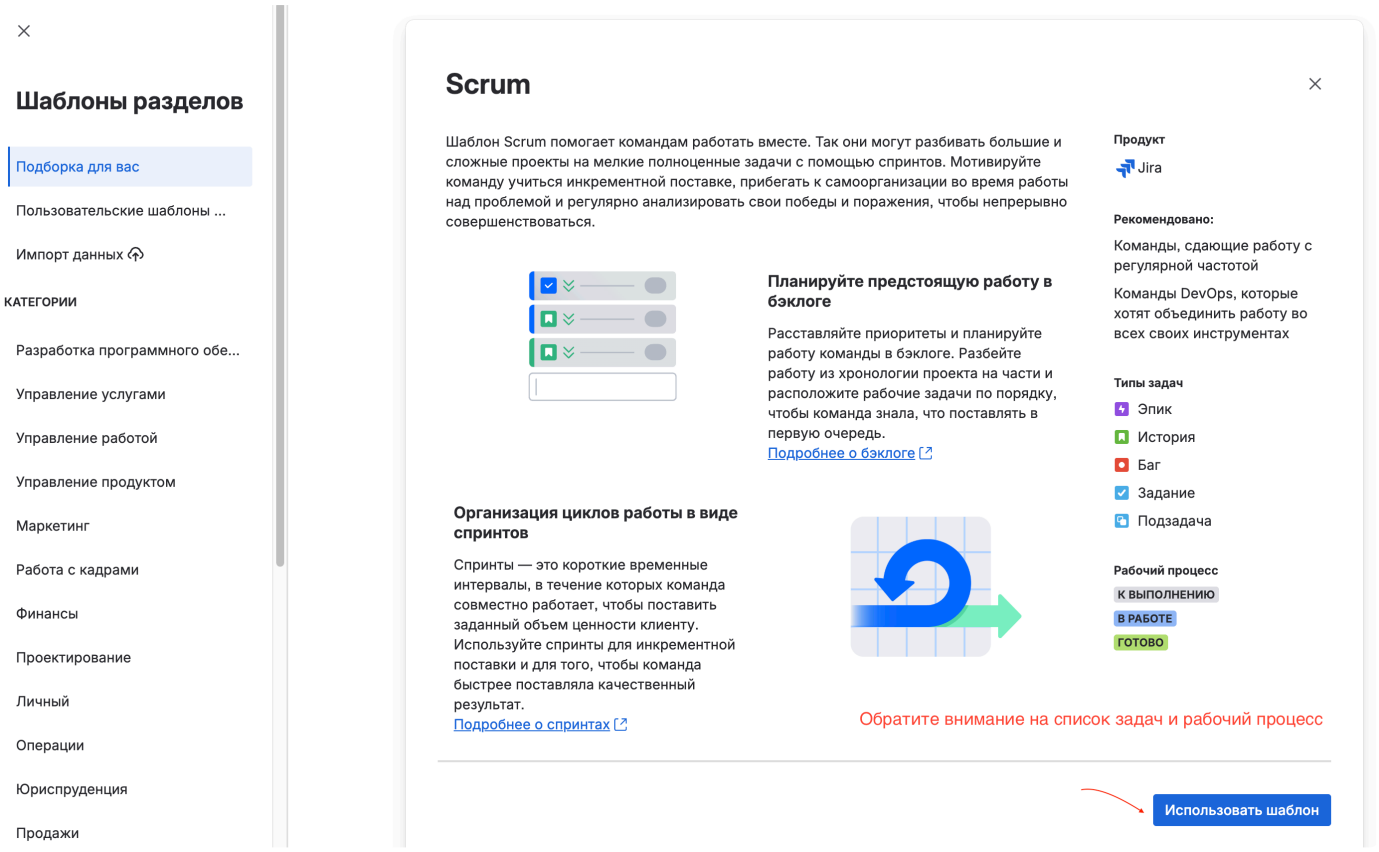

10. Поздравляю! Ваш первый проект создан!

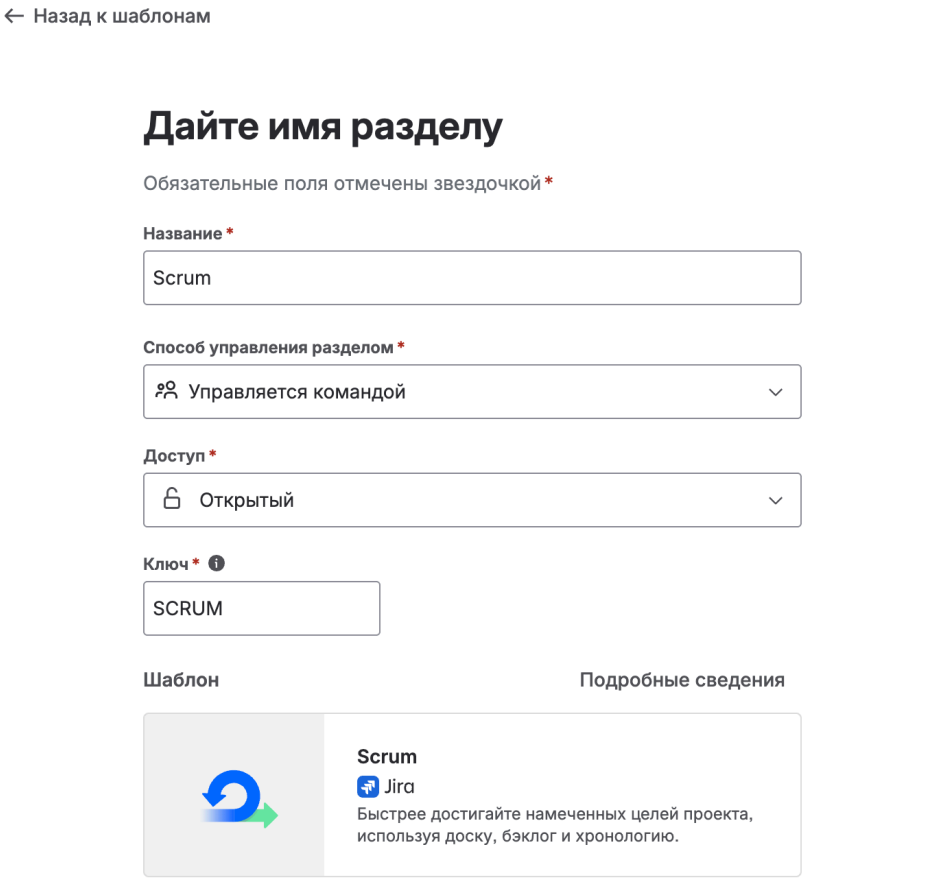



11. По умолчанию первый раздел (пространство для работы) будет выполнен с учетом подхода Kanban. Вы всегда можете создать новый раздел, используя шаблоны. Давайте создадим новый раздел в фреймворке Scrum.

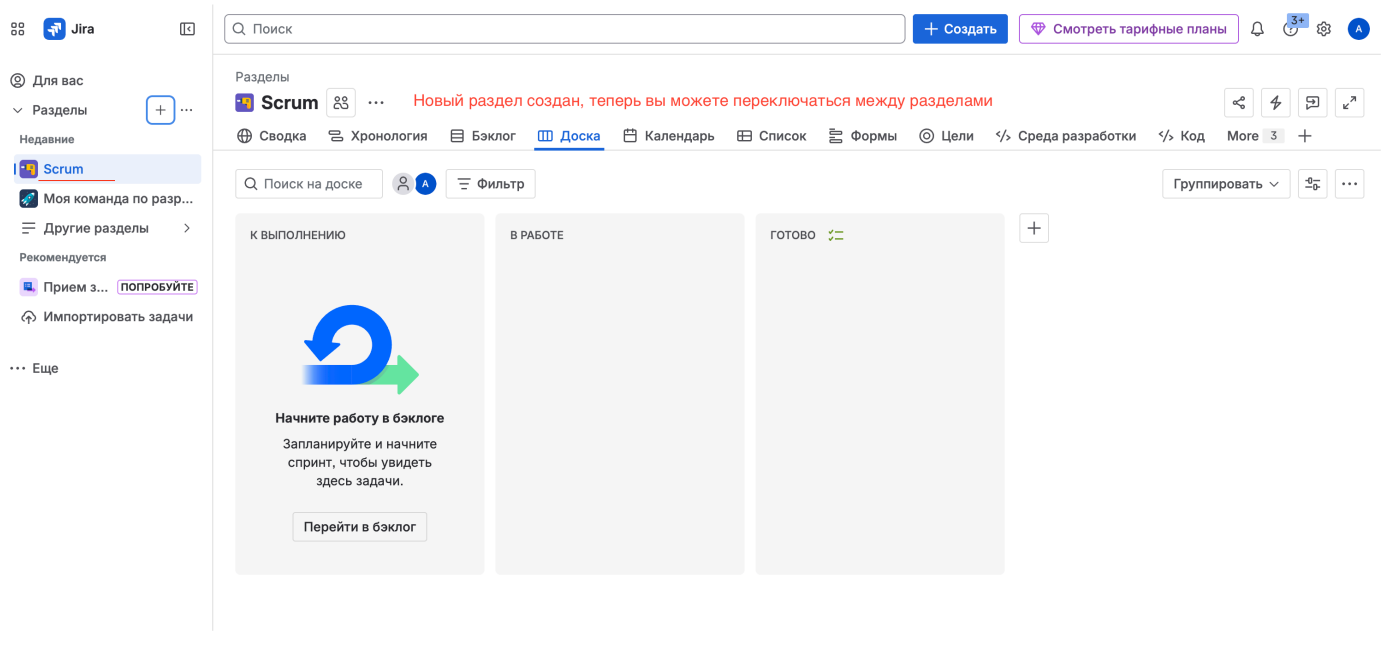

Jira
Для вас
Разделы
Недавние
Создать раздел


Шаблоны разделов
Подборка для вас
Пользовательские шаблоны ...
Импорт данных
КАТЕГОРИИ
Разработка программного обе...
Управление услугами
Управление работой
Управление продуктом
Маркетинг
Работа с кадрами
Финансы
Проектирование
Личный
Операции
Юриспруденция
Продажи
Шаблоны разделов
Подборка для вас
Шаблоны для вас, основанные на работе похожих команд.
Board
Kanban
ПОСЛЕДНИЙ СОЗДАННЫЙ
Работайте эффективно и визуализируйте задачи на доске в...
Jira
Requests
Управление общи...
ПОПРОБОВАТЬ
Создайте единый центр для приема всех типов запросов и управления...
Jira Service Management
Open
Управление ИТ-у...
ПОПРОБОВАТЬ
Оперативно реагируйте на запросы, решайте инциденты и развертывайт...
Jira Service Management
Backlog
Scrum
Планируйте, отслеживайте и выполняйте задачи с помощью...
Jira
Plan
Межкомандное план...
PREMIUM
Согласование работы команд в соответствии с общими целями и...
Jira
Process control
Управление процессом
Отслеживание и улучшение повторяющихся рабочих процессов.
Jira


Шаблоны разделов
Подборка для вас
Пользовательские шаблоны ...
Импорт данных
КАТЕГОРИИ
Разработка программного обе...
Управление услугами
Управление работой
Управление продуктом
Маркетинг
Работа с кадрами
Финансы
Проектирование
Личный
Операции
Юриспруденция
Продажи
Scrum
Шаблон Scrum помогает командам работать вместе. Так они могут разбивать большие и сложные проекты на мелкие полноценные задачи с помощью спринтов. Мотивируйте команду учиться инкрементной поставке, прибегать к самоорганизации во время работы над проблемой и регулярно анализировать свои победы и поражения, чтобы непрерывно совершенствоваться.
Планируйте предстоящую работу в бэклоге
Расставляйте приоритеты и планируйте работу команды в бэклоге. Разбейте работу из хронологии проекта на части и расположите рабочие задачи по порядку, чтобы команда знала, что поставлять в первую очередь.
Подробнее о бэклоге
Организация циклов работы в виде спринтов
Спринты — это короткие временные интервалы, в течение которых команда совместно работает, чтобы поставить заданный объем ценности клиенту. Используйте спринты для инкрементной поставки и для того, чтобы команда быстрее поставляла качественный результат.
Подробнее о спринтах
Продукт
Jira
Рекомендовано:
Команды, сдающие работу с регулярной частотой
Команды DevOps, которые хотят объединить работу во всех своих инструментах
Типы задач
Эпик
История
Баг
Задание
Подзадача
Рабочий процесс
К ВЫПОЛНЕНИЮ
В РАБОТЕ
ГОТОВО
Обратите внимание на список задач и рабочий процесс
Использовать шаблон



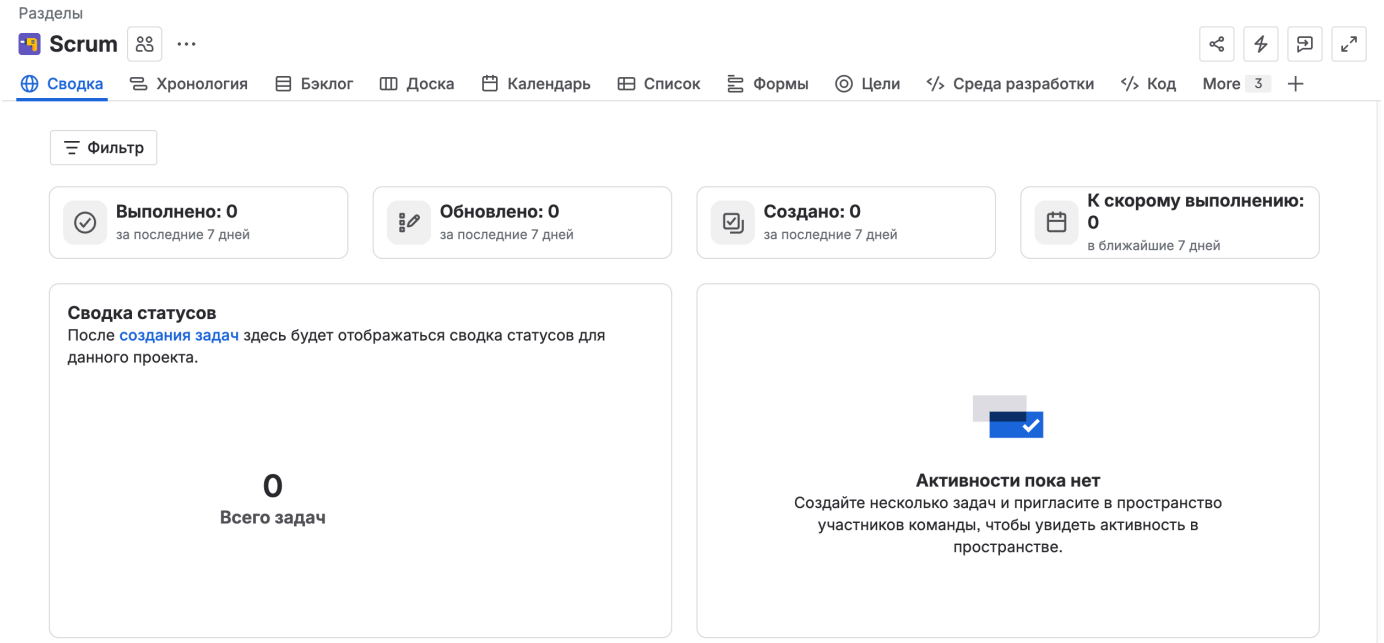

В шаге 2 нажмите "Далее".



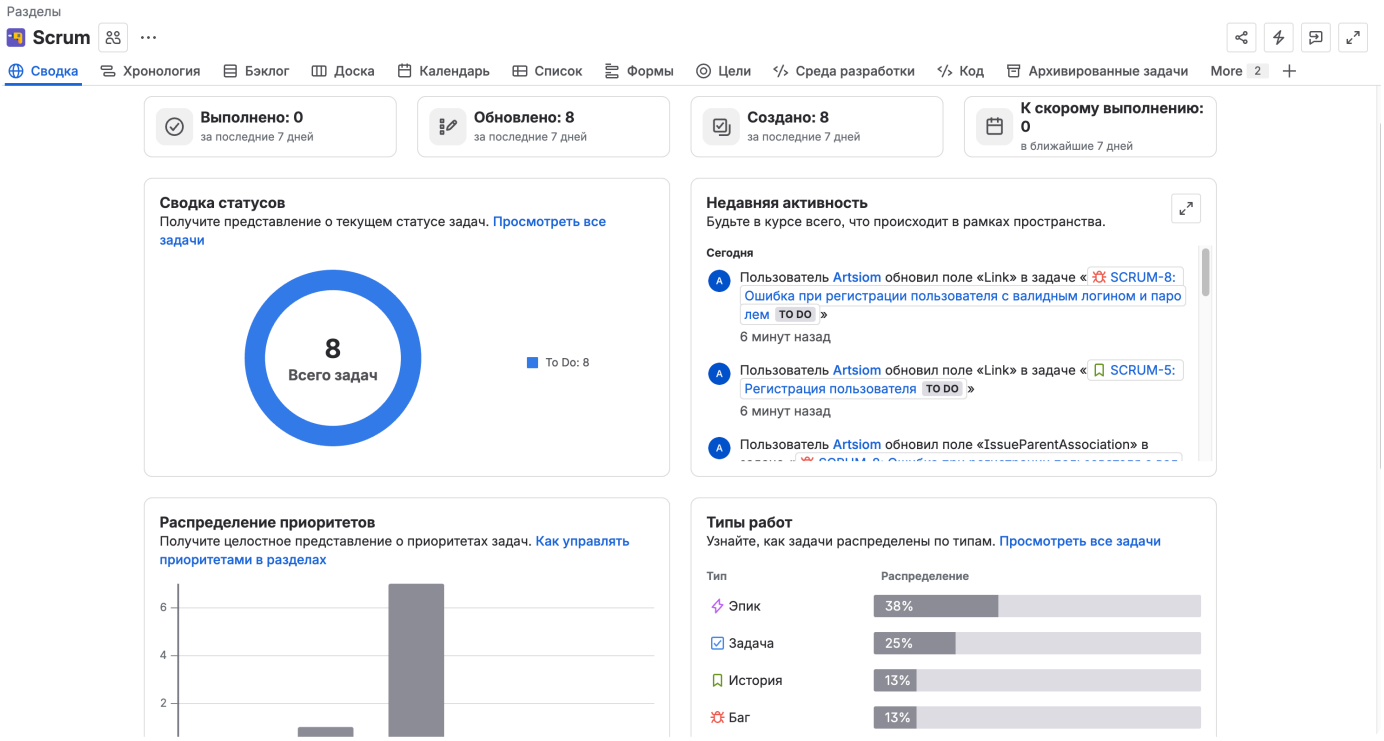
## Интерфейс раздела

Продолжаем работу в разделе "Scrum" и рассмотрим все вкладки внутри него.

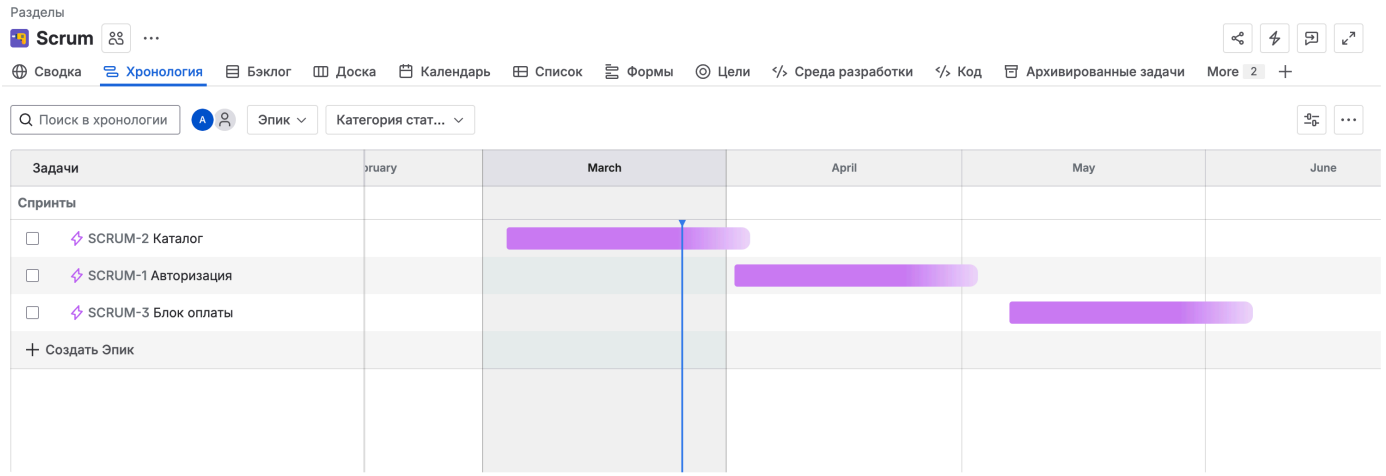
1. **Вкладка "Сводка"** – в ней содержится общая информация по разделу: количество задач, статусы, активности и так далее. По мере добавления сущностей в систему она будет меняться. Информацию отсюда менеджерам удобно использовать для отчетности.



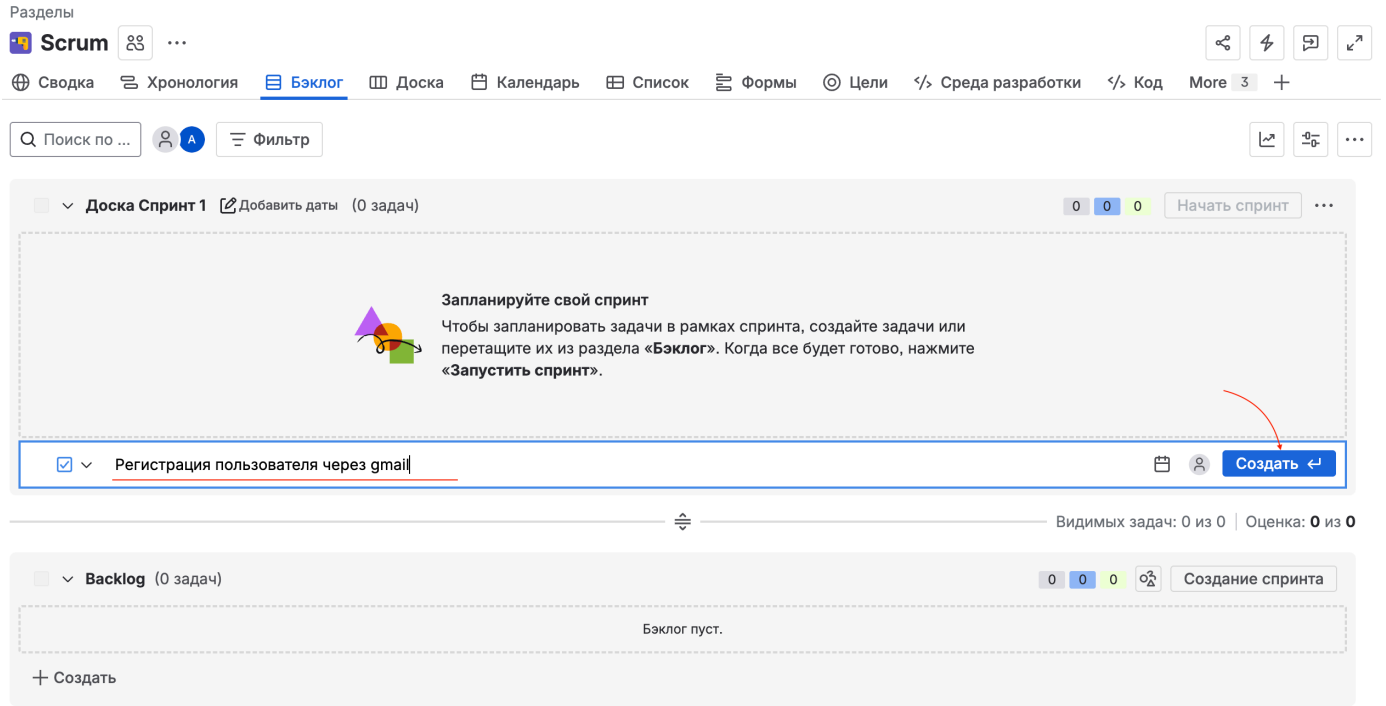

Вот так она будет выглядеть после начала работы:



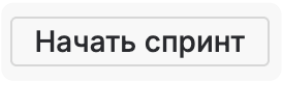
2. **Вкладка "Хронология"** – позволяет видеть дорожную карту развития проекта. Для наглядности я добавил несколько эпиков с датой начала работы по ним. **Эпики** – это крупные задачи в Agile, которые разбиваются на более мелкие пользовательские истории для поэтапной реализации.



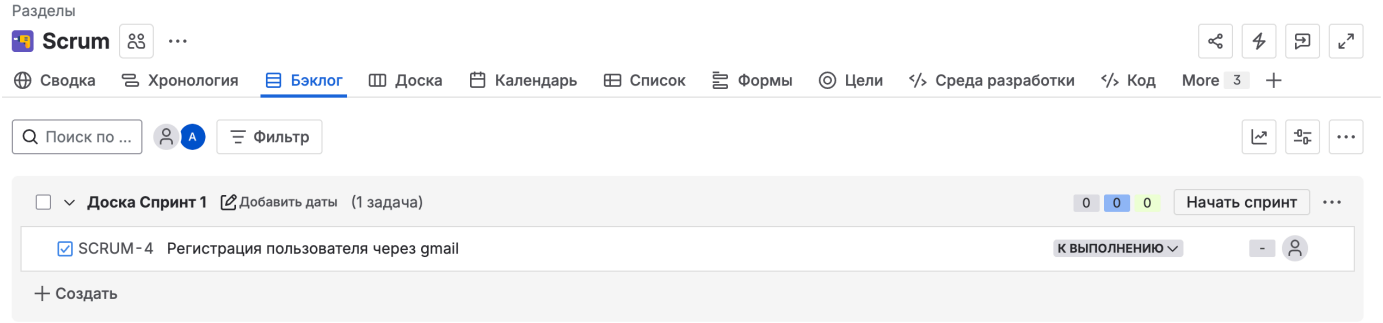

3. Вкладка "Бэклог" – представляет собой все задачи, которые нужно реализовать за всю историю проекта, ранее мы проходили с вами продуктовый бэклог в Scrum. Отсюда выбираются задачи в бэклог спринта. Создайте одну задачу, чтобы вкладка "Доска" из следующего пункта не была пустой.




Нажмите

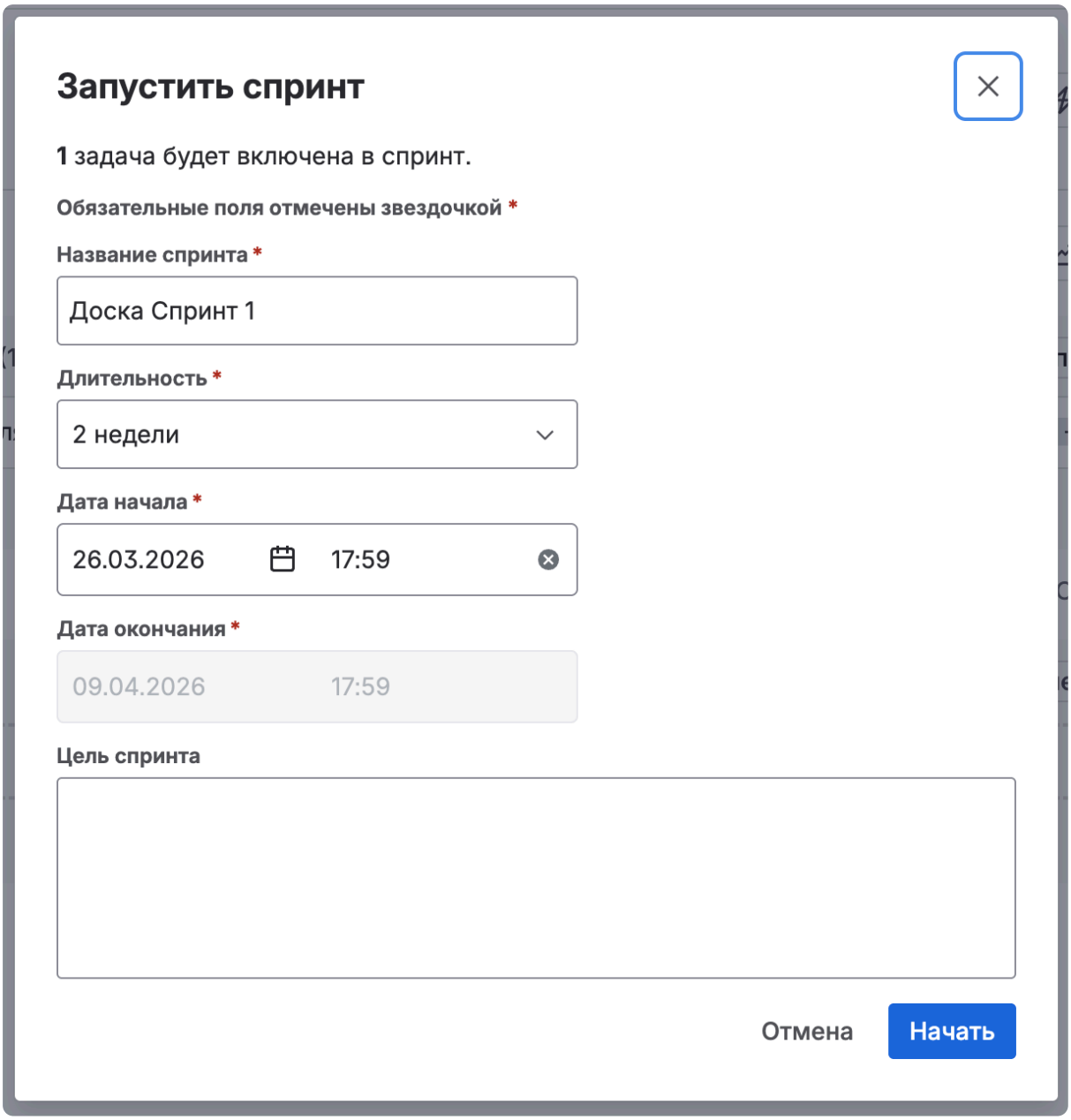


.

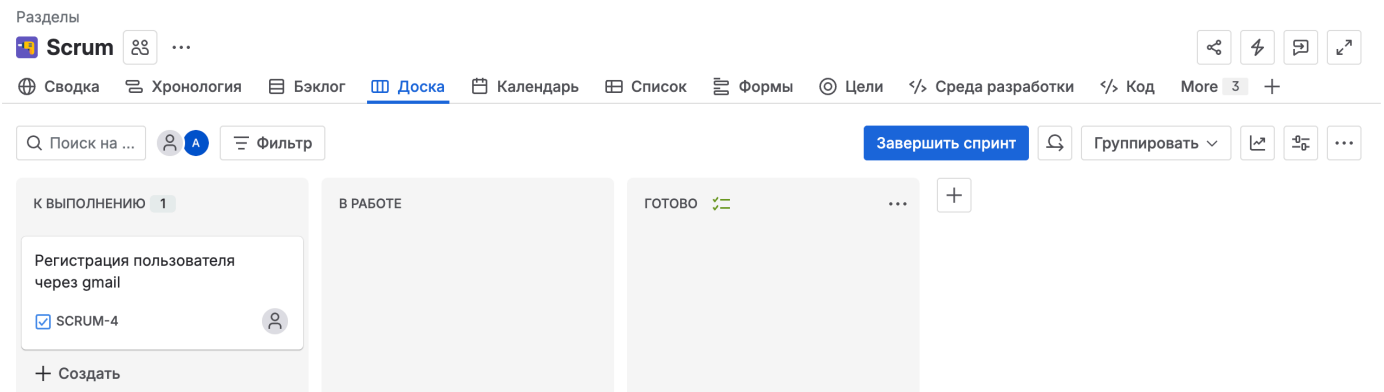



Нажмите

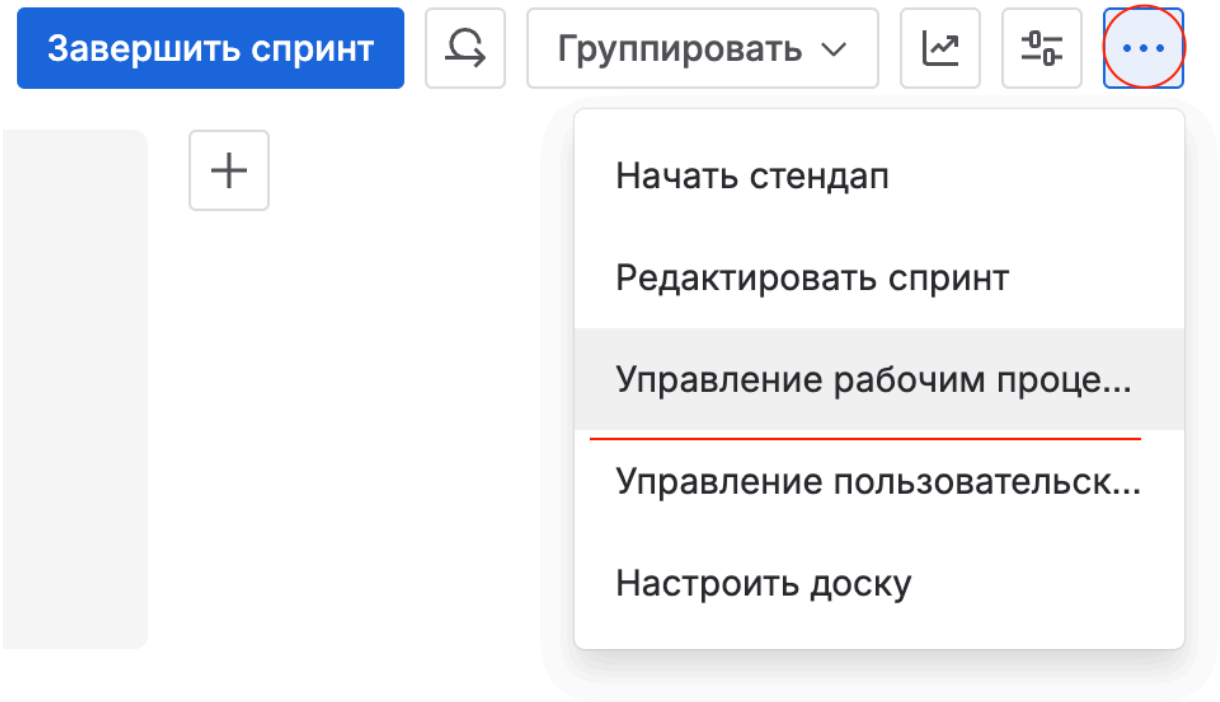
. Вас автоматически переведет на вкладку "Доска".

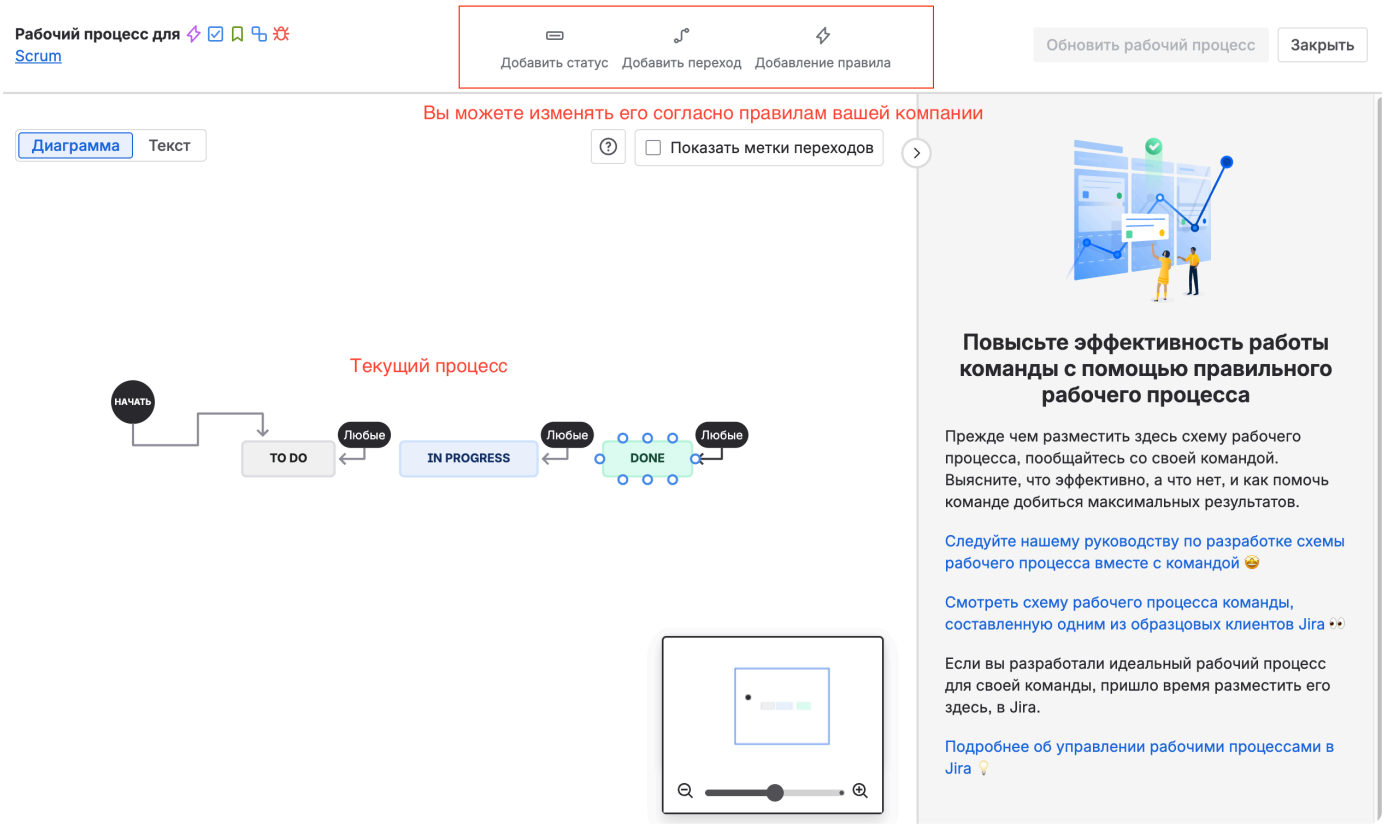



4. **Вкладка "Доска"** – содержит информацию обо всех историях, задачах и багах, которые будут в работе в рамках спринта. Карточки можно передвигать по разным статусам.



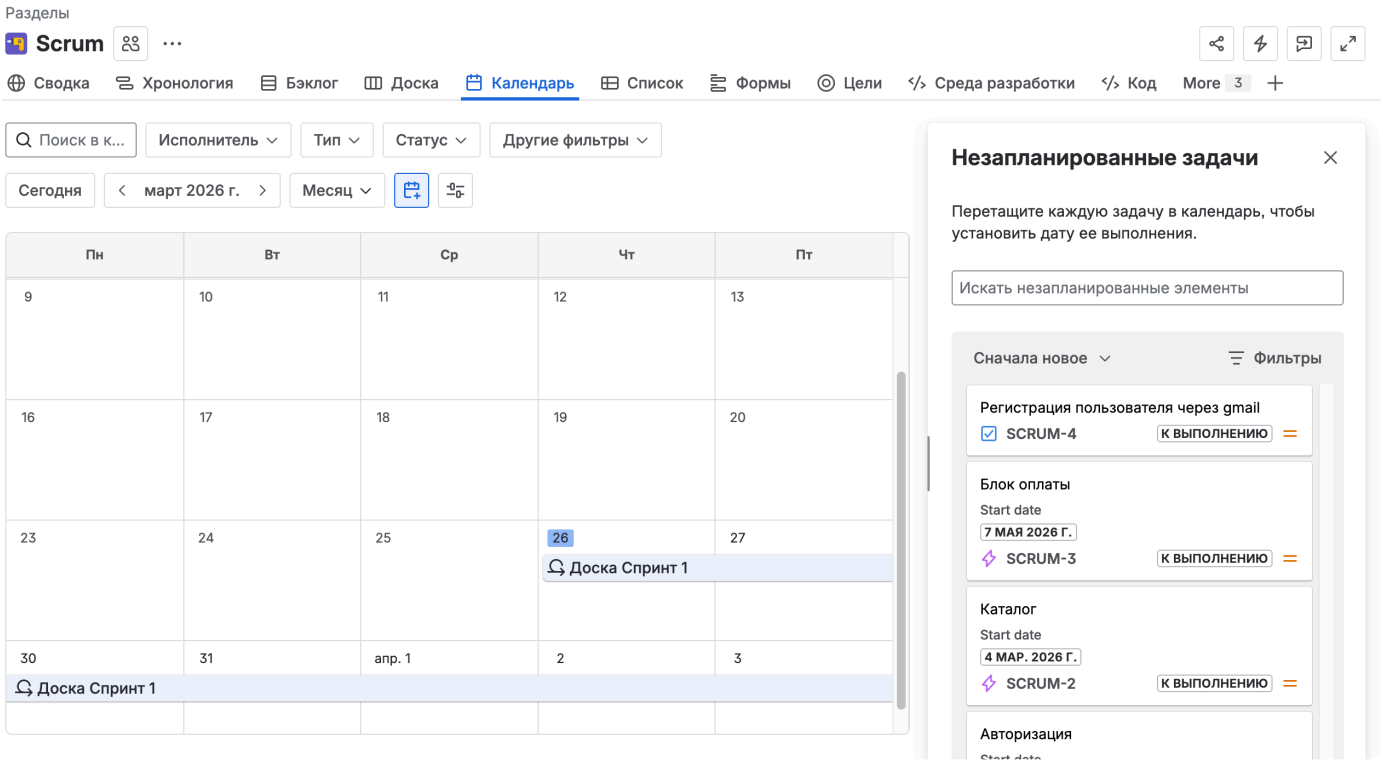
Статусы для каждого проекта будут разные и их можно добавлять через настройку "Управление рабочим процессом". Это делают не тестировщики, а администраторы Jira или менеджеры.



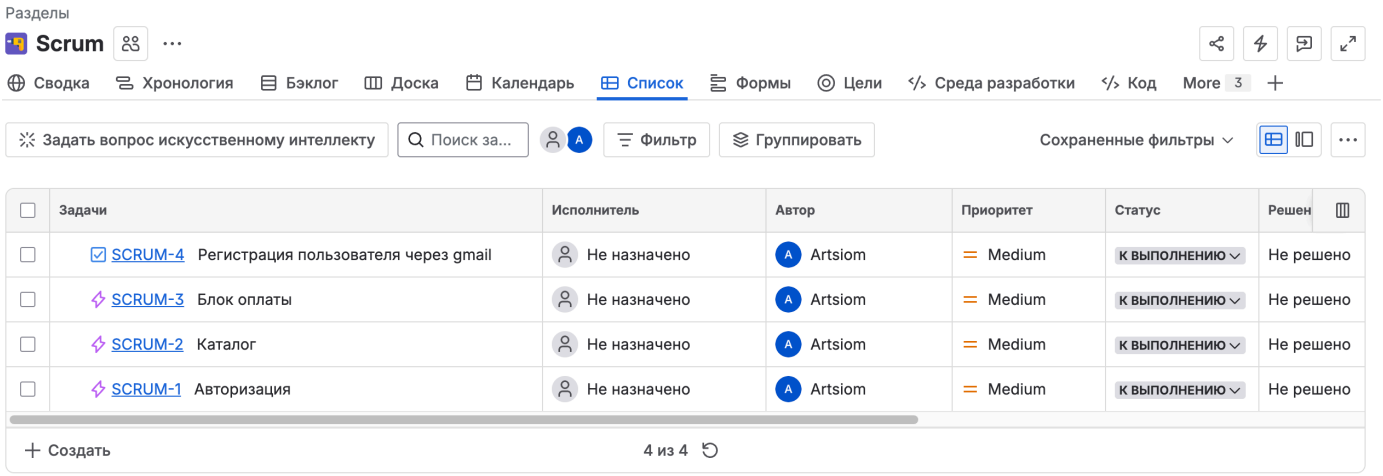




В следующих уроках мы научимся создавать отдельные элементы бэклога.

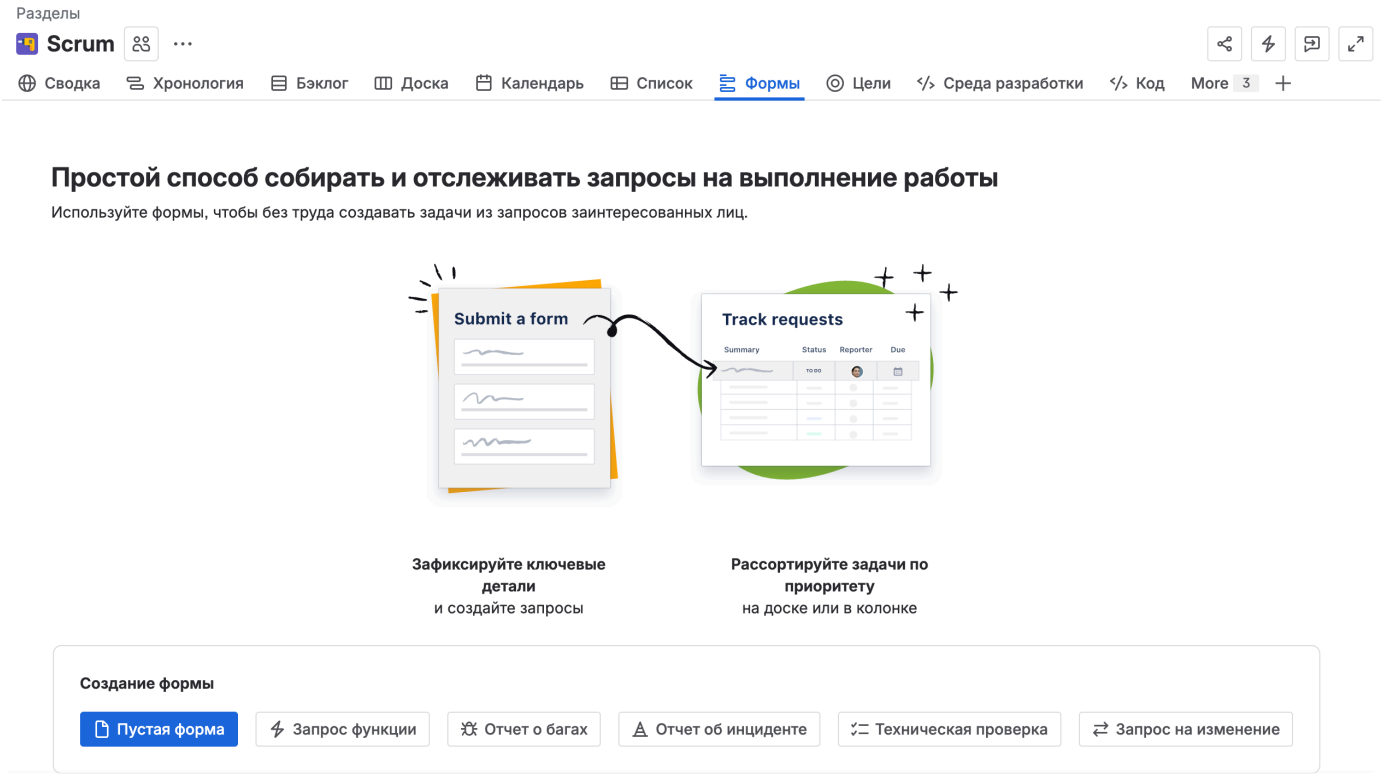
5. **Вкладка "Календарь"** – позволяет отслеживать прогресс по запланированным задачам с привязкой к дате.




6. **Вкладка "Список"** – отображает список всех элементов спринта.



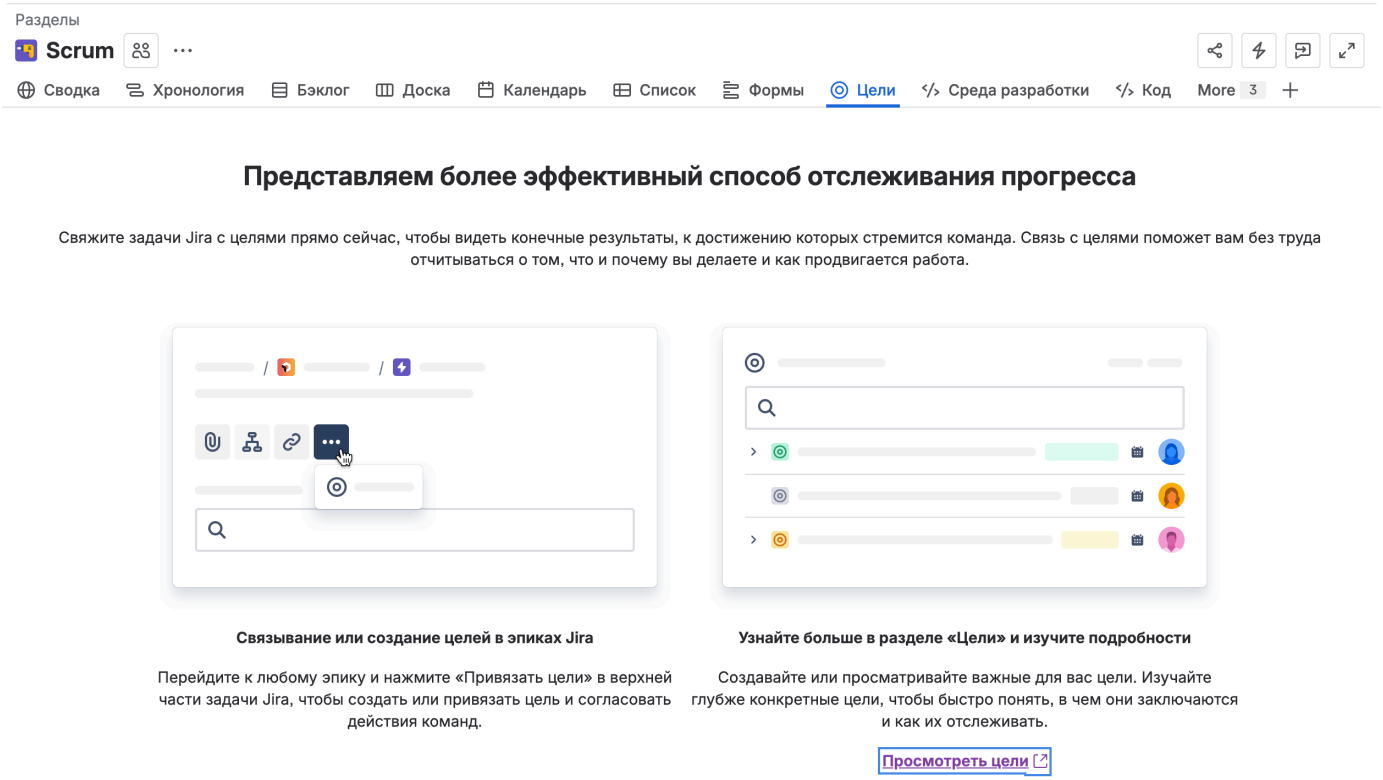

7. **Вкладка "Формы"** – позволяет создавать формы для получения запросов для заинтересованных лиц. Например, через форму пользователь может оставить отчет о дефекте без необходимости быть частью рабочего проекта. В курсе они нам не понадобятся.




8. **Вкладка "Цели"** – позволяет привязывать задачи из Jira к целям, к которым стремится команда. Например, 100% критических багов должно быть исправлено, 100% тестов из регрессионных наборов должны быть автоматизированы, 100% пользовательских историй с высоким приоритетом должны быть разработаны к концу итерации. Цели можно создать, нажав на кнопку

Просмотреть цели

. В рамках курса они нам не понадобятся.



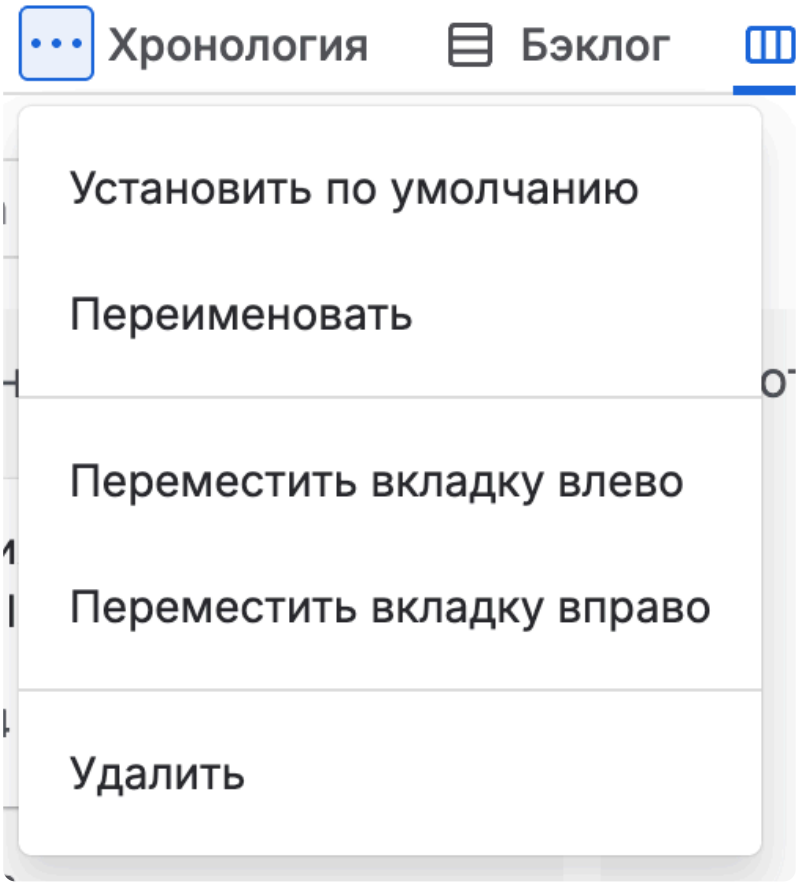

9. **Вкладки "Код" и "Среда разработки"** практически не используются ручными тестировщиками, а нужны для тех, кто работает с кодом.

10. Также вы можете посмотреть дополнительные вкладки, нажав на

или добавить недостающие

. В курсе они нам не понадобятся, но про их назначение можно узнать в документации.

11. Со вкладками можно совершать дополнительные действия. Набор может отличаться.




**Создание элементов бэклога**

С точки зрения тестирования нас интересует *создание отдельных задач и отчетов о дефекте, а также привязка их к историям.*

**Задачи (Tasks)** на проектах чаще всего привязываются к пользовательским историям и предполагают отдельные активности, например, анализ требований, создание тест-кейсов, тестирование истории.

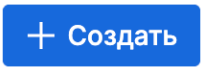
**Отчеты о дефекте (баг-репорты, Bugs)** содержат подробную задокументированную информацию о проблеме. Они чаще всего привязываются к историям. Подробнее о структуре отчета говорили в прошлом уроке.

Все 3 типа элементов мы учли в прошлых шагах. В зависимости от принятого процесса их может быть больше, а сам процесс с ними может отличаться.

**Создание пользовательской истории**

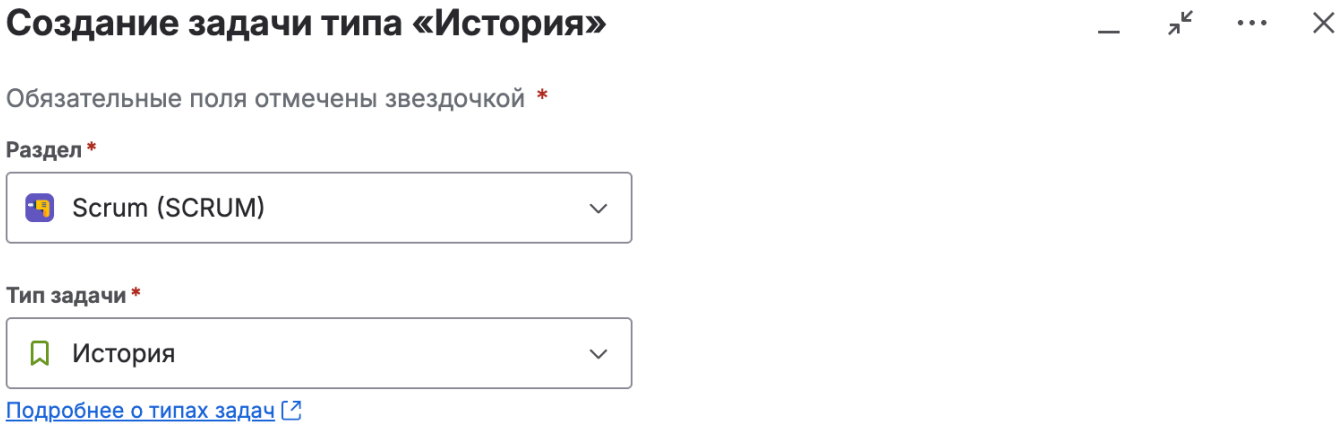
**User Story** создают аналитики, продуктовые менеджеры или владельцы продукта. Подробнее о них мы говорили в уроке 2.13. Давайте перенесем логику уже знакомого требования в Jira.

1. Нажмите

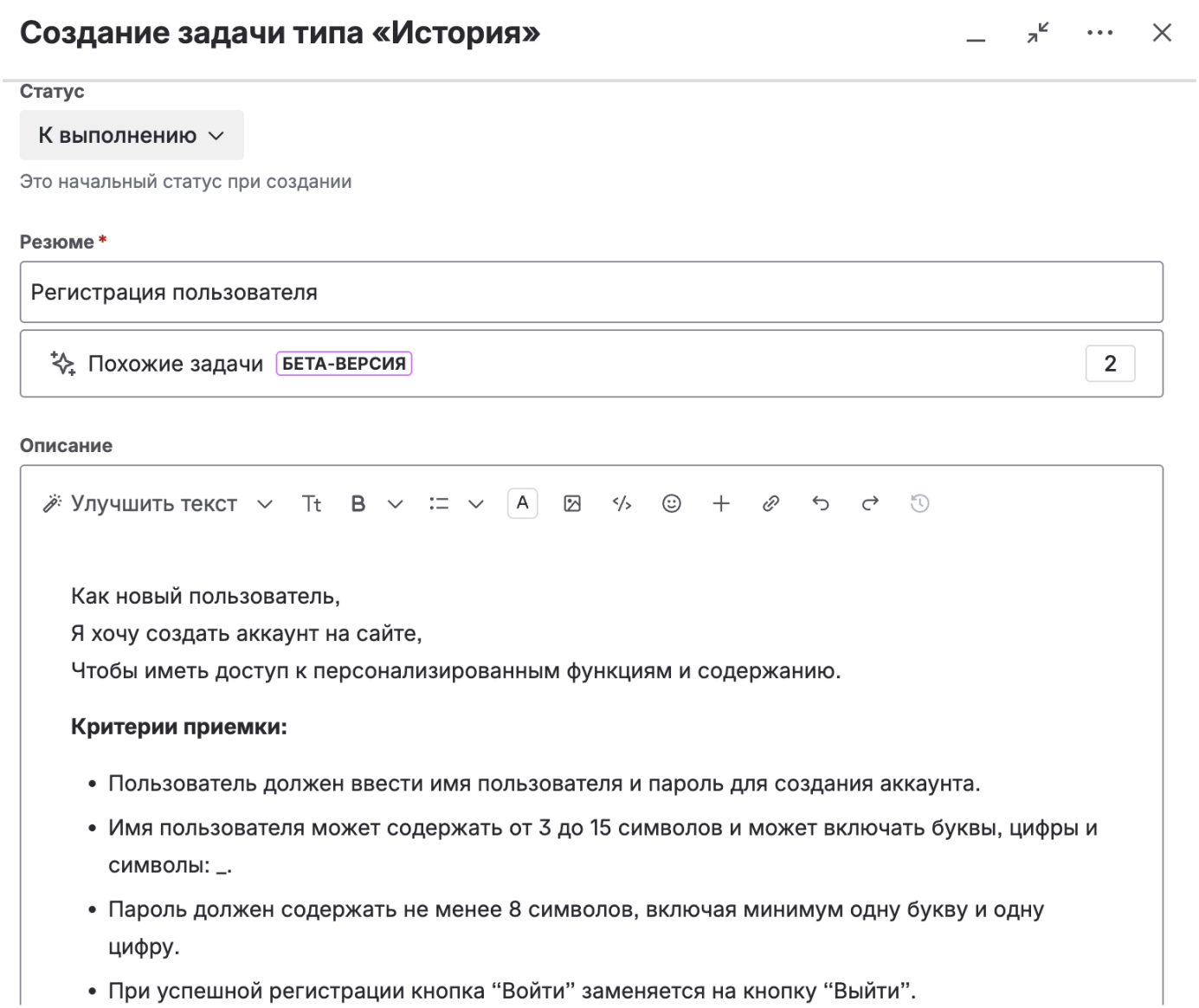



.

2. Убедитесь, что выбран тип задачи "История".

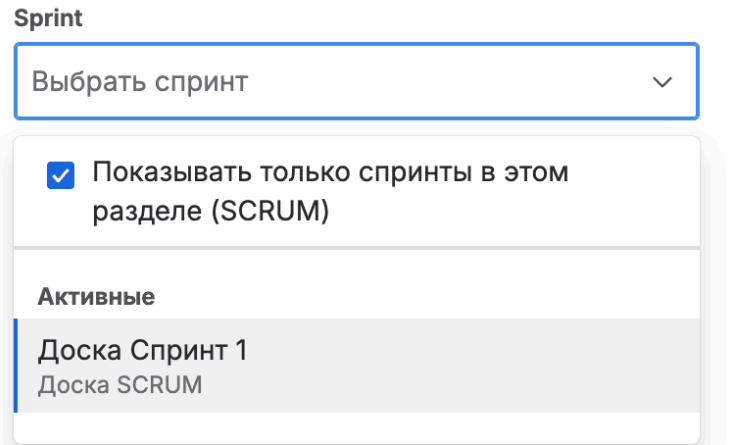



3. Перенесите заголовок истории и описание из Kaiten в Jira.




*Обратите внимание, что текст можно форматировать и автоматические редактировать через опцию "Улучшить текст".*

Обязательно выберите текущий спринт, чтобы история попала на доску.

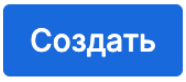


Если вы забудете сделать,  то задачу можно будет найти во вкладках "Бэклог" или "Список" и изменить спринт там.

Далее вы можете заполните остальные атрибуты. Я не буду останавливаться на каждом, так как их назначение понятно из названия.

Обращу внимание на Story point estimate, где указывается количество стори-поинтов для командной оценки сложности истории. Подробно говорили в уроке 2.11.

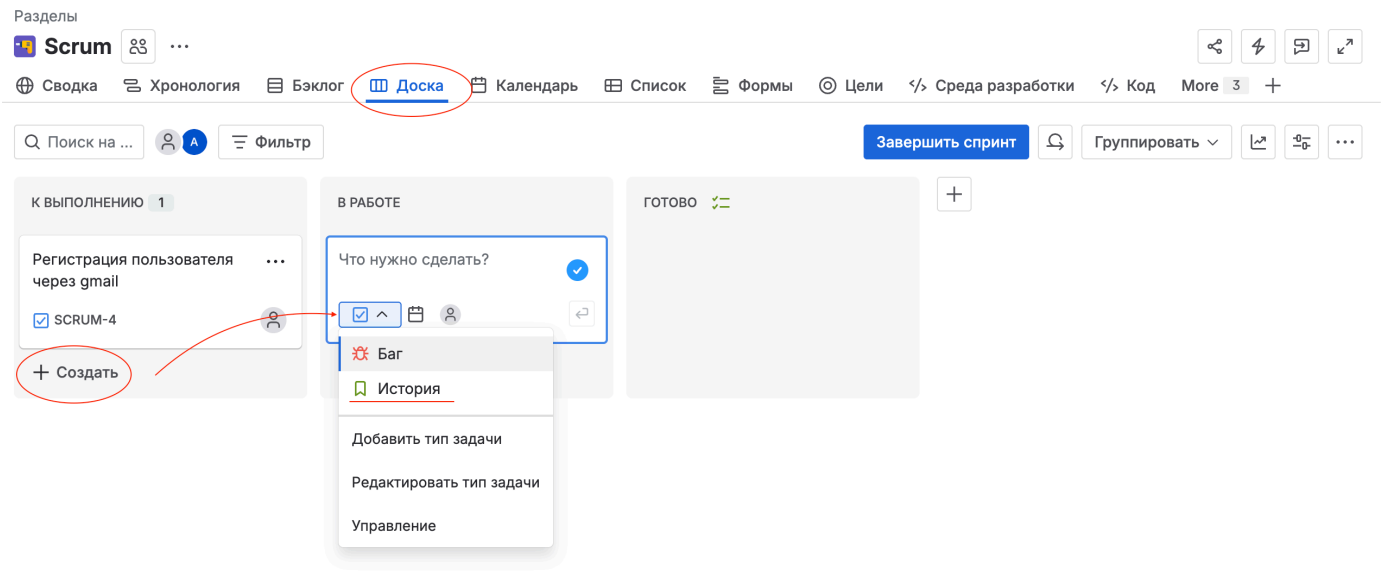
4. Нажмите




. Если вы хотите создать несколько историй, то выберите опцию

.

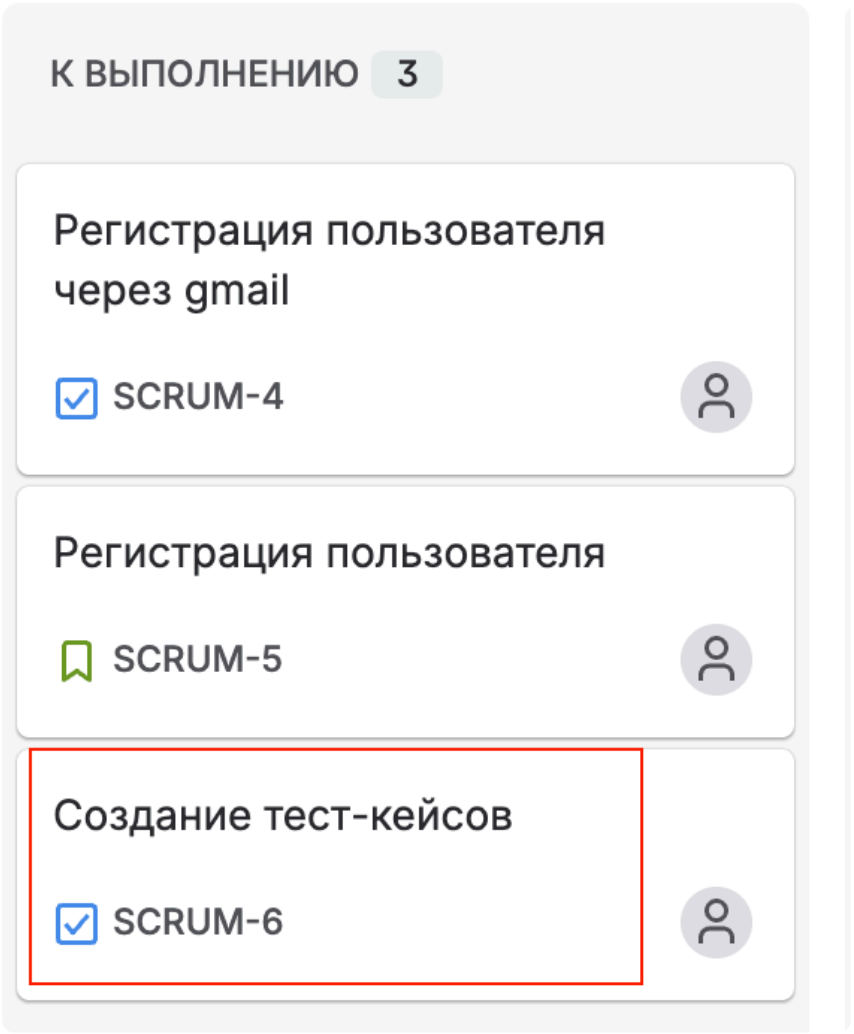
5. Для быстрого создания истории без атрибутов можно использовать этот способ.



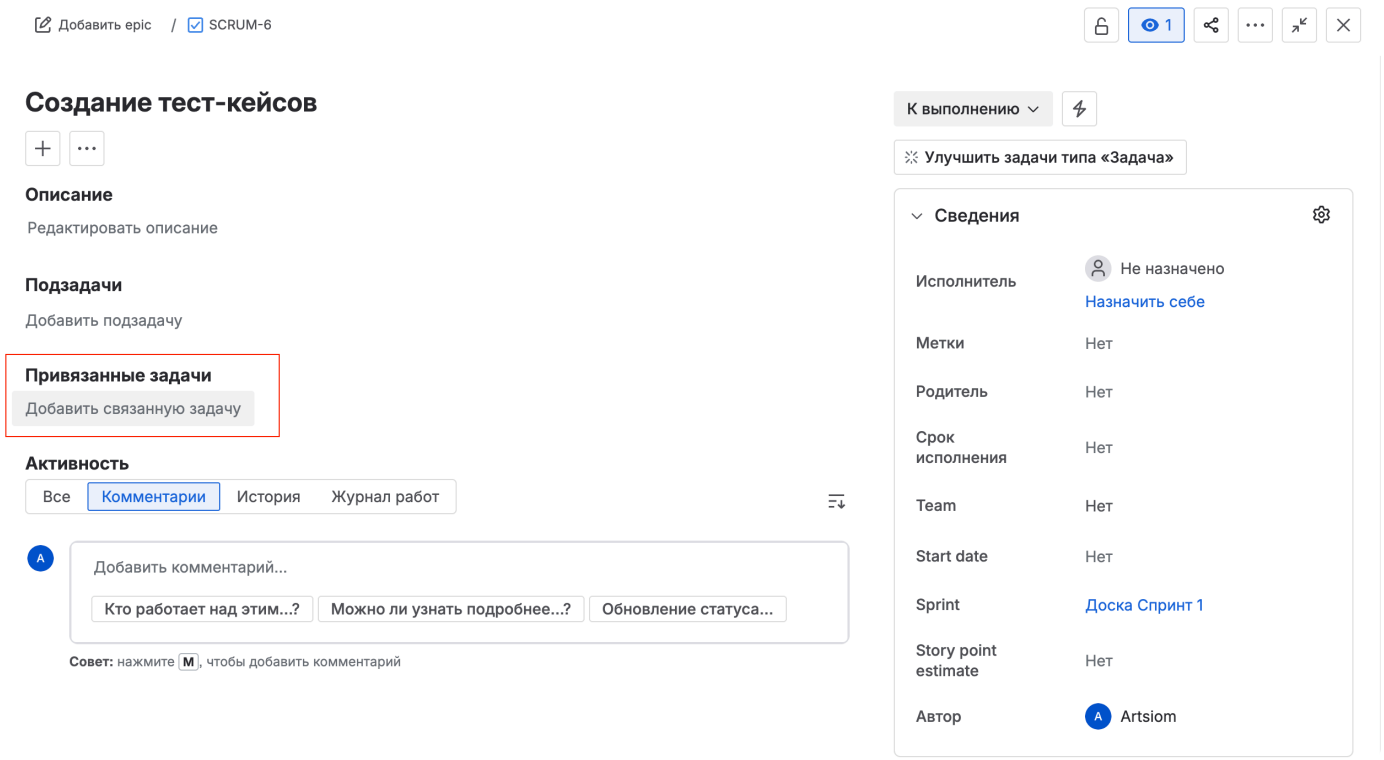

## Создание задачи

Процесс создания задачи ничем не отличается от пользовательской истории. Попробуйте сделать это самостоятельно.

1. Создайте задачу "Создание тест-кейсов".



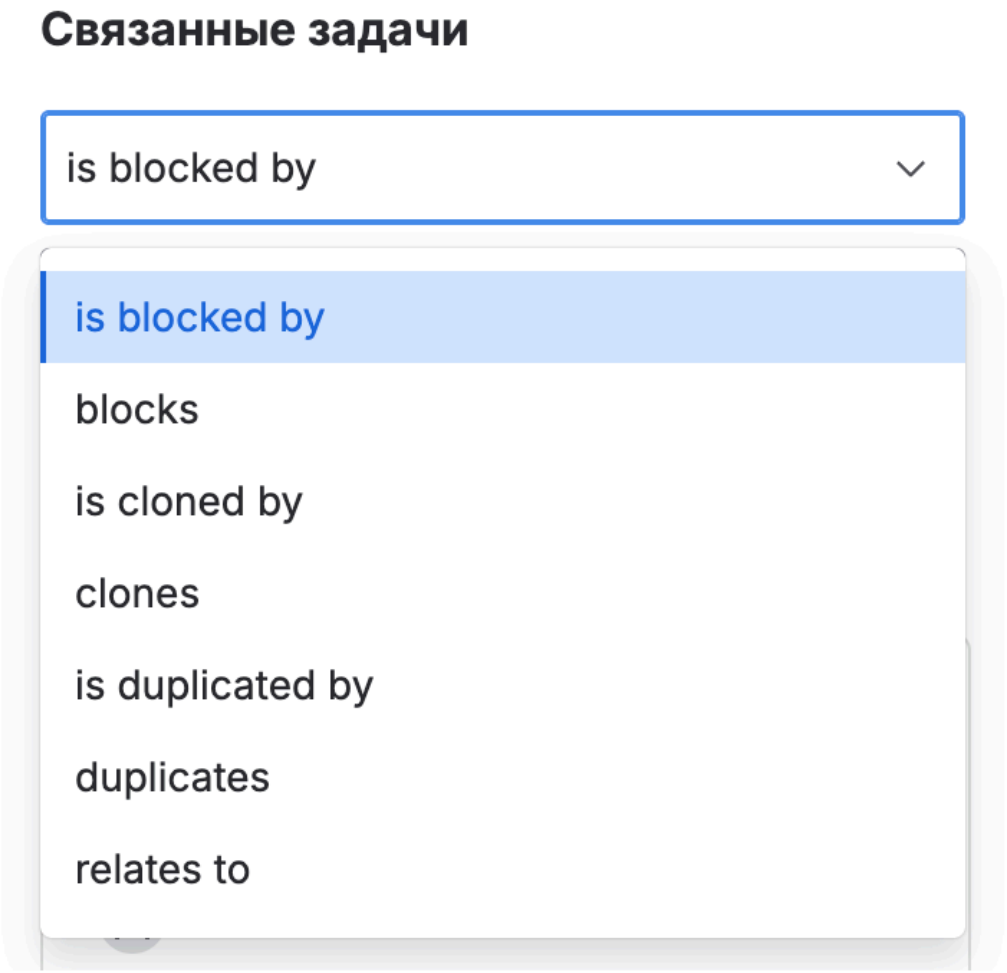

2. Задачи должны быть к чему-то привязаны. В нашем случае создадим связь к истории "Регистрация пользователя". Нажмите на задачу для редактирования.



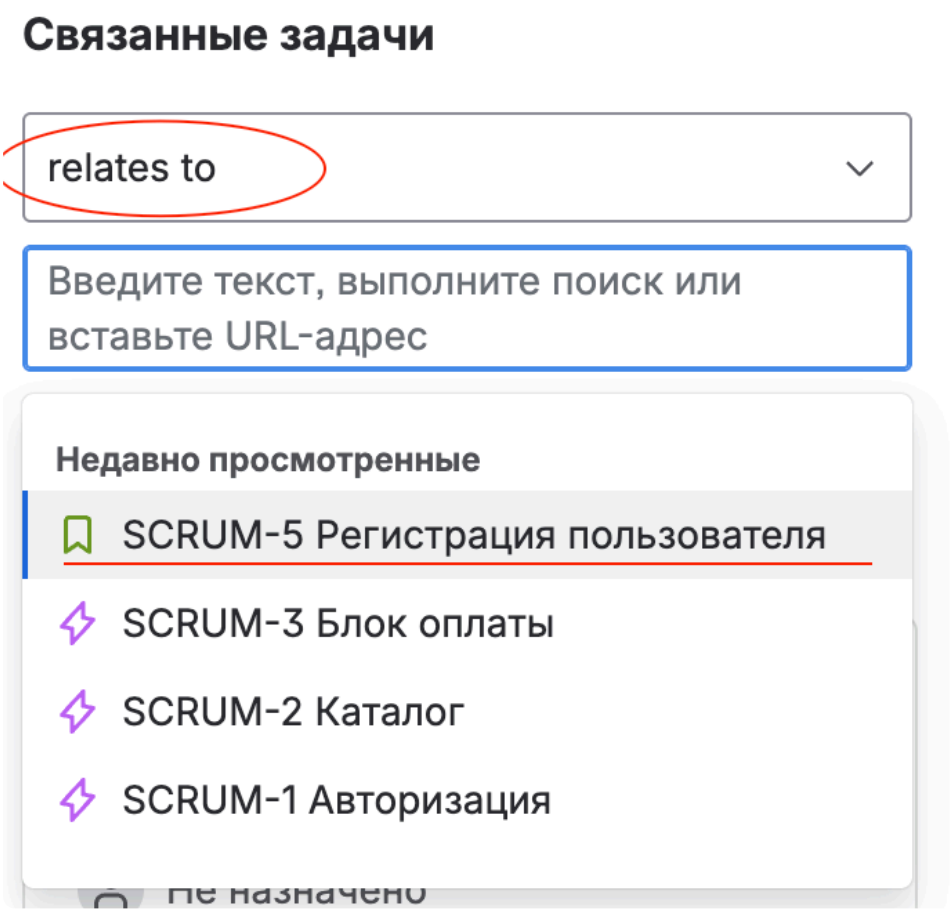

3. Выберите один из типов связей.

- Блокирует/блокируется
- Клонирует/клонируется
- Дублирует/дублируется
- Связано с

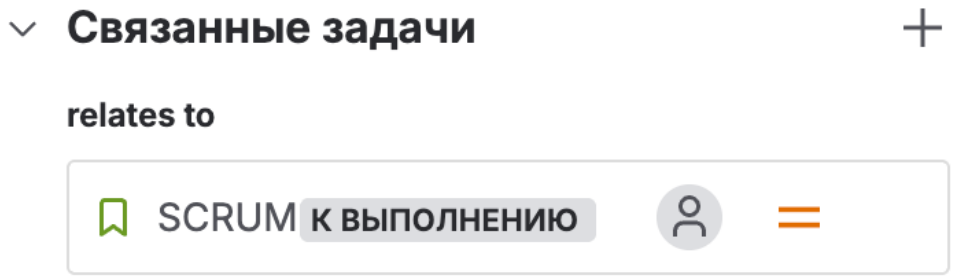
Клонирование – это **создание** новой копии, а дублирование – это **установление связи** между уже существующими одинаковыми задачами.



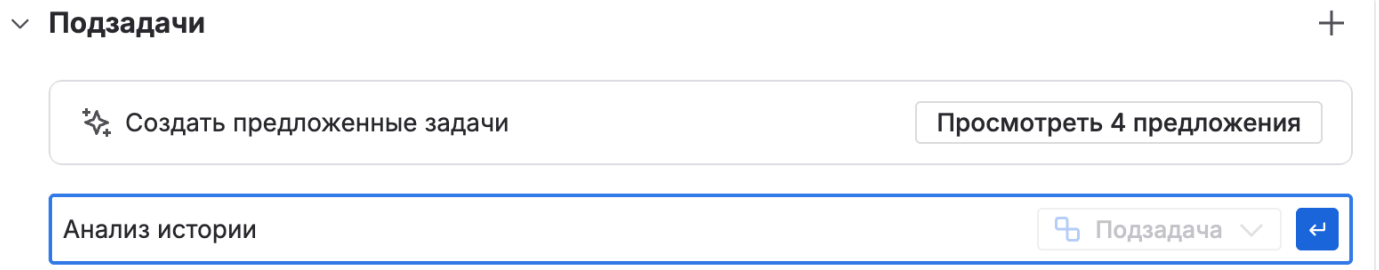

Нам нужна опция relates to (связано с). И далее нужно выбрать историю. И нажать "Связать".

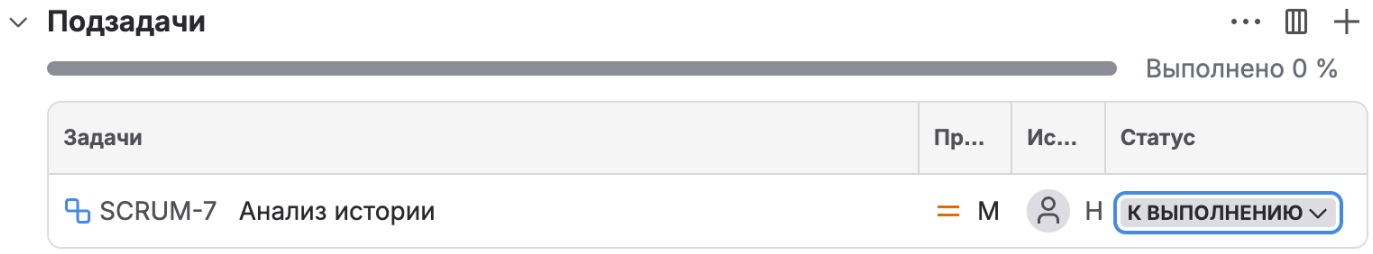



Теперь информация о связи появится в атрибутах задачи и истории.




4. Помимо отдельных задач в виде карточек, вы можете создавать подзадачи. Например, можно создать подзадачу "Анализ истории".



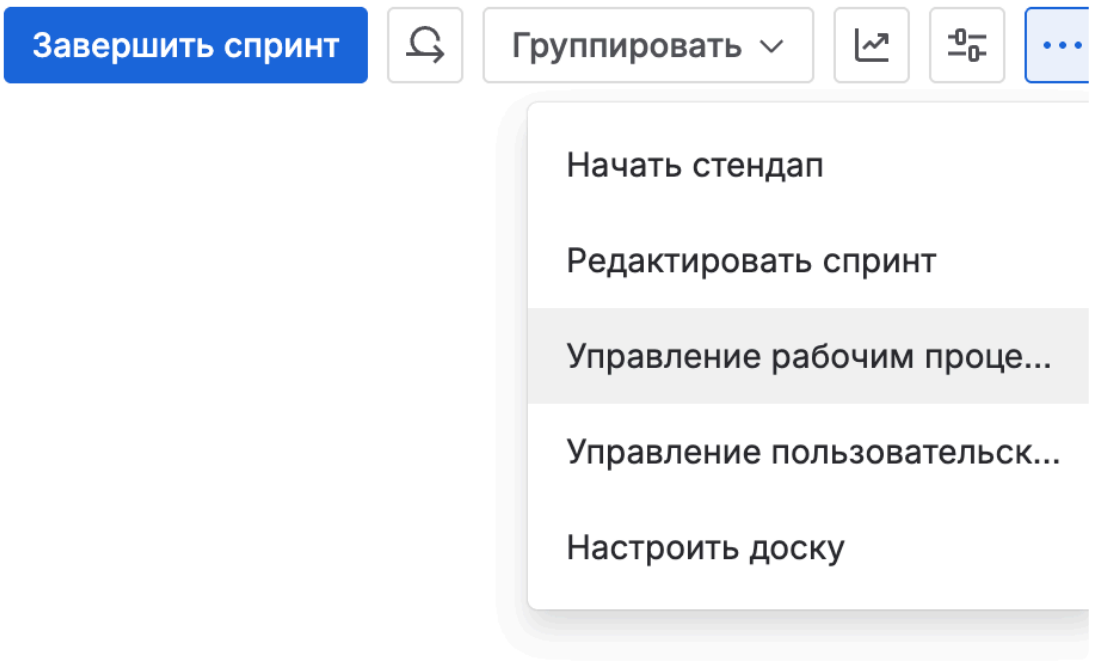



**Создание бага (отчета о дефекте)**

У нас уже есть тип задачи – Баг. По умолчанию в нем есть не все атрибуты, поэтому нам нужно их добавить.

На момент обновления урока я не нашел прямой ссылки на добавление полей конкретно для бага, поэтому расскажу про свой способ. Возможно, это баг или ограничение бесплатного тарифа.

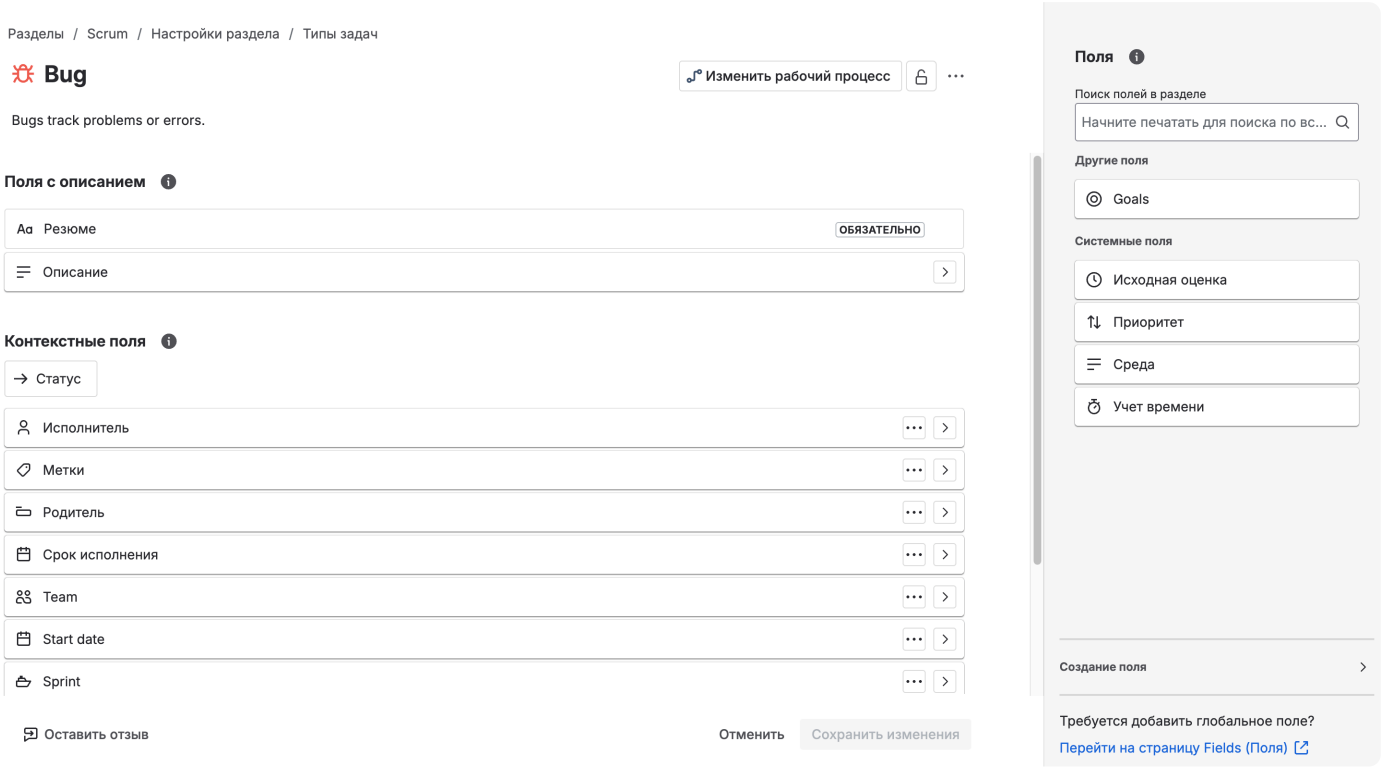
1. Зайдите в настройку "Управление рабочим процессом".




2. Нажмите "Закрыть" в открывшемся окне. У вас откроется панель для добавления полей в задачу "Task", но нам нужен Bug.

Поэтому измените в url идентификатор задачи. В моем случае было `весь url/10007`, а баг был на `весь url/10008`. Может быть у вас будет 10009 или 10010.

3. Если вы все сделаете правильно, то у вас появится панель для изменения полей бага. Нас интересует "Приоритет" и "Среда" из правого меню. Нажмите на них для автоматического добавления в карточку. Также вы можете создать поля через меню "Создание поля". На проектах это настраивают менеджеры или администраторы Jira, а не рядовые тестировщики.




4. Сохраните изменения.

5. Создайте задачу "Баг" и обратите внимание, что в ней появились все нужные атрибуты.

## Создание задачи типа «Баг»

**Резюме** *

Ошибка при регистрации пользователя с валидным логином и паролем

Похожие задачи БЕТА-ВЕРСИЯ 2

**Среда**

Улучшить текст

MacOS 12.4, Chrome v. 146.0.7680.80 (Official Build) (arm64)

For example operating system, software platform and/or hardware specifications (include as appropriate for the work item).

**Приоритет**

High

Подробнее об уровнях приоритетов

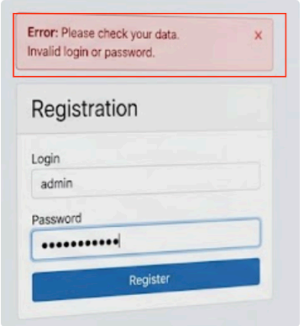
Описание

Улучшить текст

**Шаги:**

1. Открыть example.com
2. Ввести admin в поле “Логин”
3. Ввести admin_password в поле “Пароль”
4. Нажать кнопку “Регистрация”

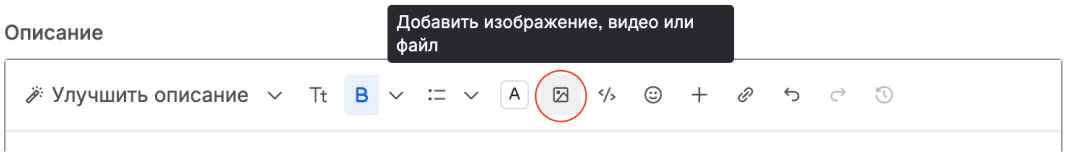
**Фактический результат:** Ошибка “Проверьте свои данные” при вводе валидного пароля и логина.




**Ожидаемый результат:** Пользователь успешно зарегистрирован.

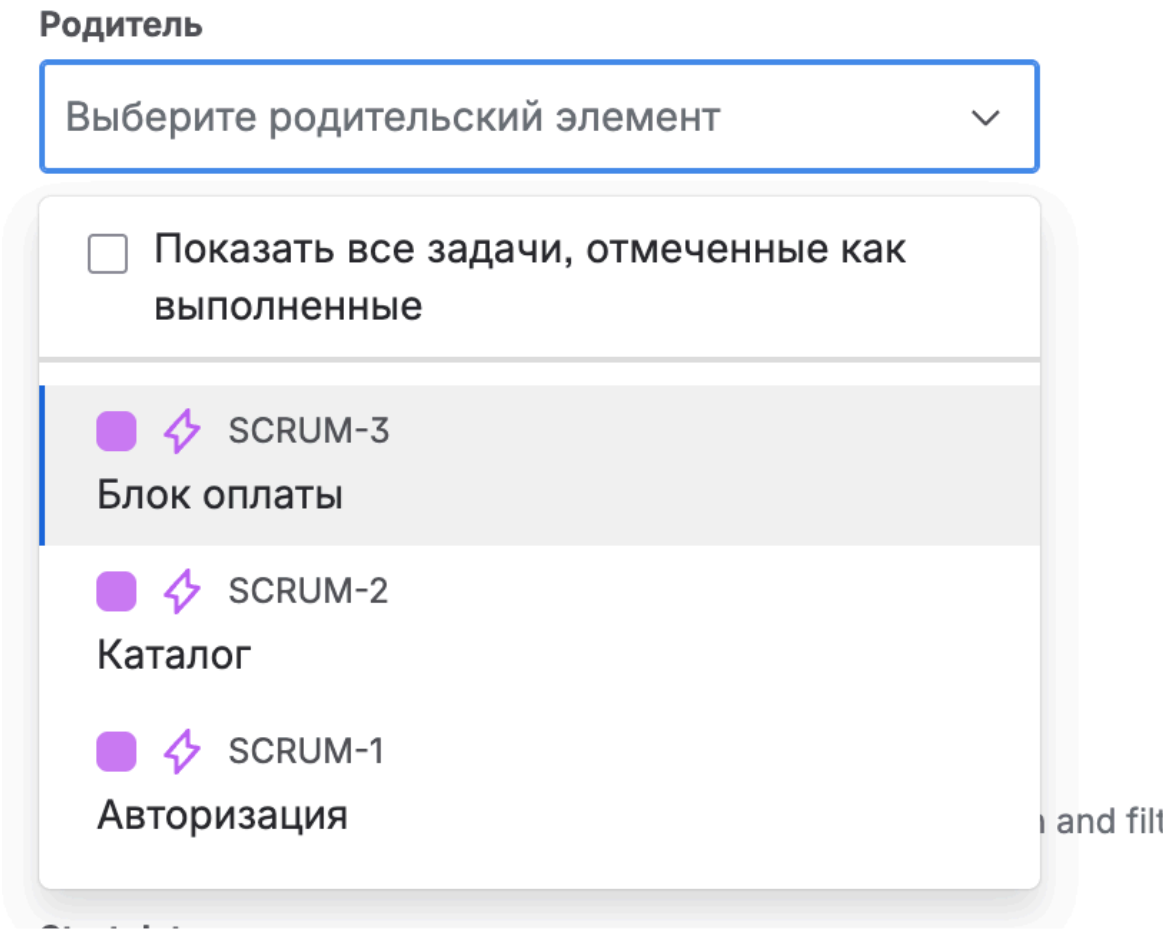
Скрины к багу можно добавлять через буфер обмена (Ctrl + C / Ctrl + V)

Для загрузки других файлов используйте

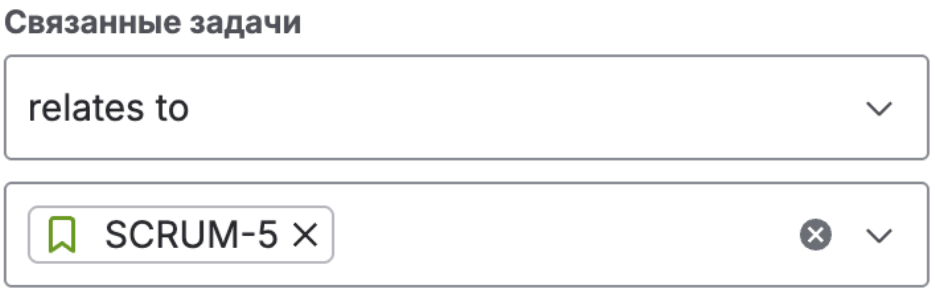



.

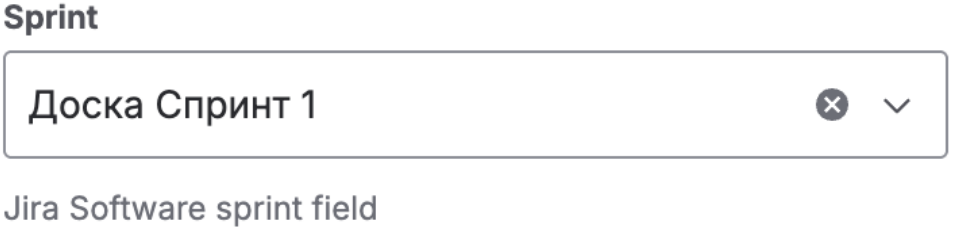
В текущей реализации родительский элемент Эпик. Это также можно настраивать, но мы не будем это менять.




Дополнительно мы добавим связь Relates to к Истории.




6. Не забудьте указать текущий спринт, чтобы баг попал на доску.



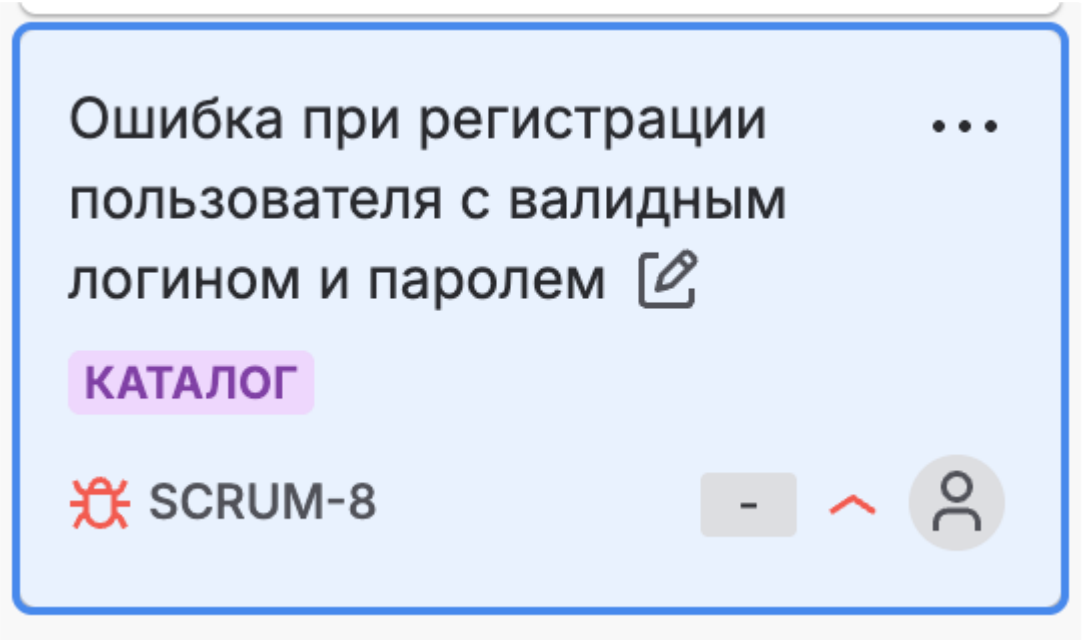

7. Нажмите

Создать

.

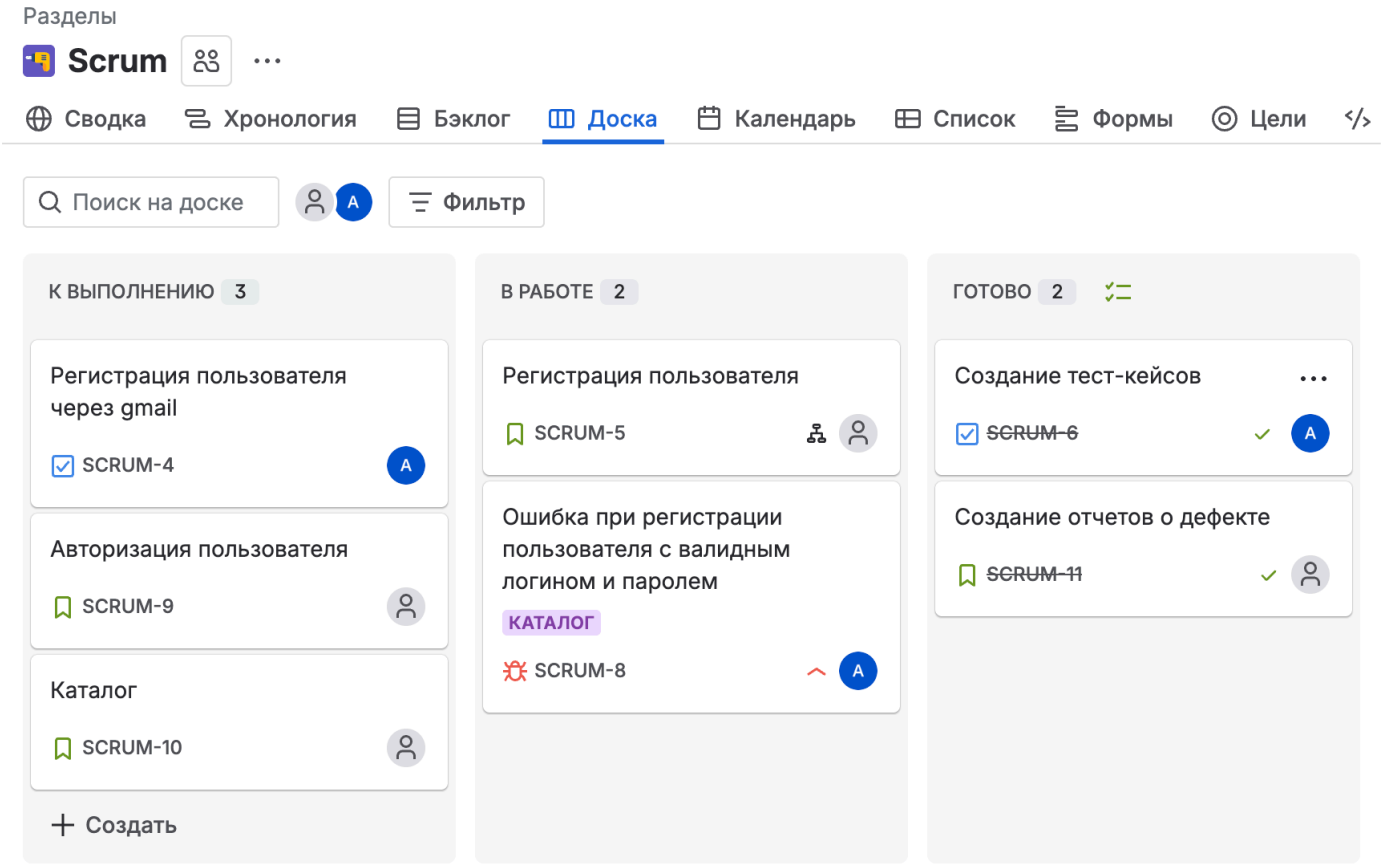
8. Теперь на доске появился дефект. Родительский Эпик отображается фиолетовым, также в углу есть приоритет.

> На реальных проектах приоритет есть во всех типах задач. Серьзность (Severity) используется очень редко.




**Алгоритм работы с Jira**

1. Формируется бэклог продукта (вкладка "Бэклог"), из которого в спринт попадают отдельные задачи (бэклог спринта, вкладка "Доска").



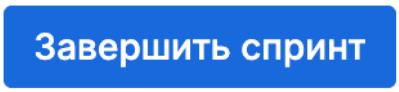

2. Каждый участник команды берет на себя задачи: разработчики берут задачи на создание кода и привязывают их к истории, тестировщики берут на себя задачи по анализу и тестированию историй.

3. В разный момент итерации, одни и те же задачи назначаются на разных людей. Это может быть сделано самостоятельно или менеджером.

4. По мере выполнения задач они меняют статус и передвигаются по доске.

5. Когда задача готова, то она переносится в последний столбец и закрывается.

6. В конце спринта он завершается

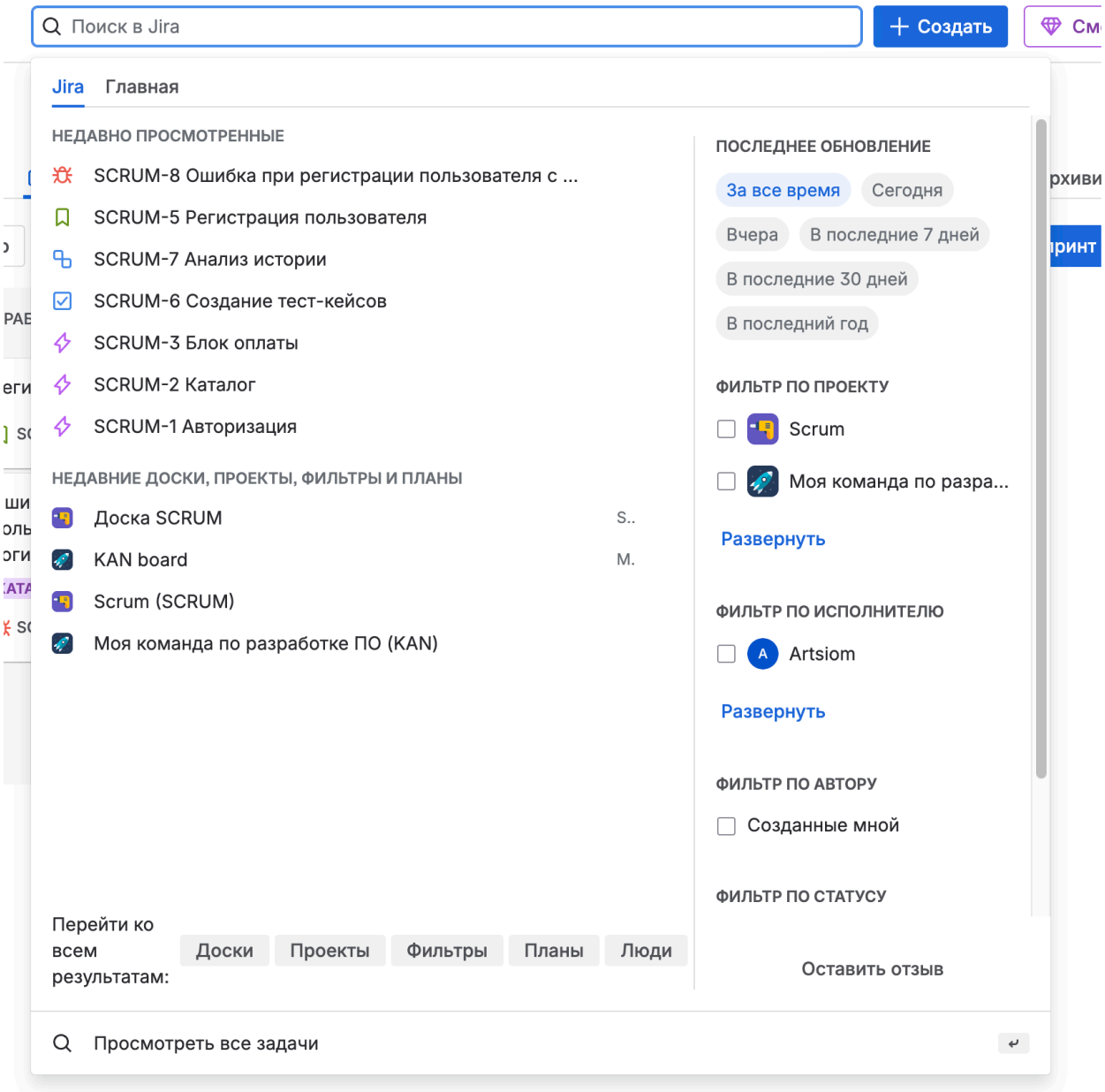



и в работу берутся новые задачи из бэклога. Начинается новый спринт.

**Поиск и фильтрация**

Как и для работы с кейсами, умение искать нужные баги и анализировать их – очень важная часть работы тестировщика. Например, для поиска дубликатов багов, для анализа текущего спринта, для поиска ответственных, для отчетов и др.

1. Вы можете искать задачи в любом месте проекта через верхнее меню в упрощенном виде, однако количество полей для фильтрации там ограничено.

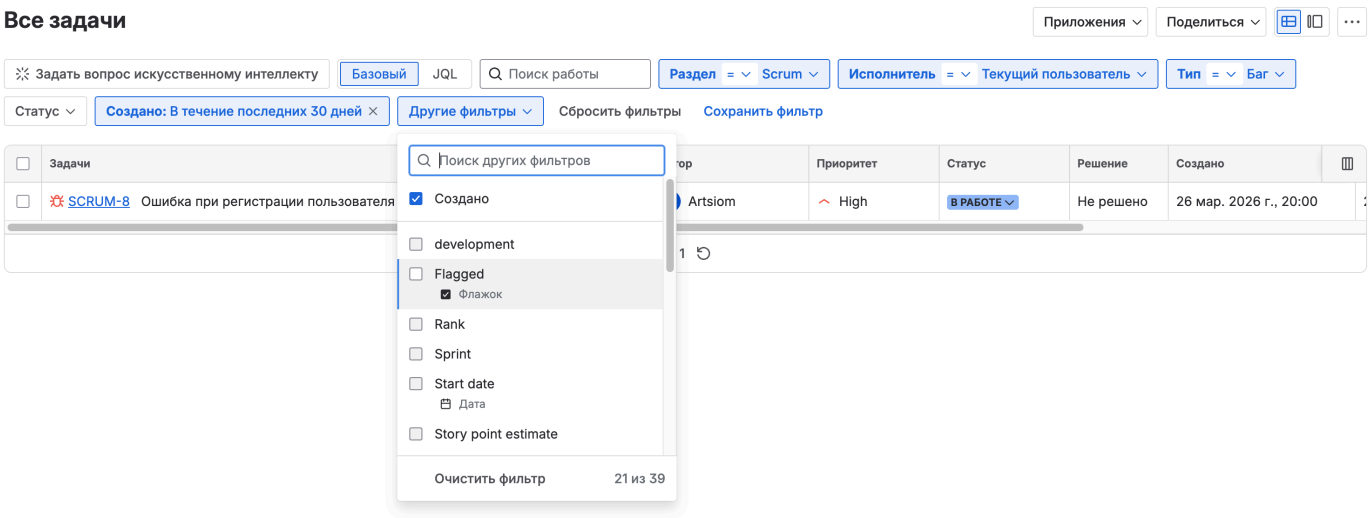



2. Вы можете нажать Enter, чтобы просмотреть все задачи или на кнопку внизу этого окна

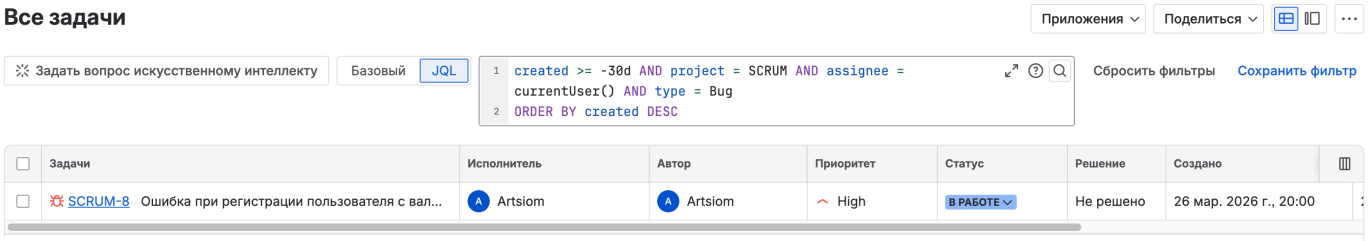
, чтобы получить доступ к полным возможностям для фильтрации.

У вас есть два способа для фильтрации: **базовый** и через специальный **язык запросов JQL**. Также с недавних пор вы можете задать вопрос ИИ, чтобы он нашел нужные баги.

- При **базовом поиске** вы выбираете фильтры вручную. Если каких-то фильтров нет, то их можно добавить.

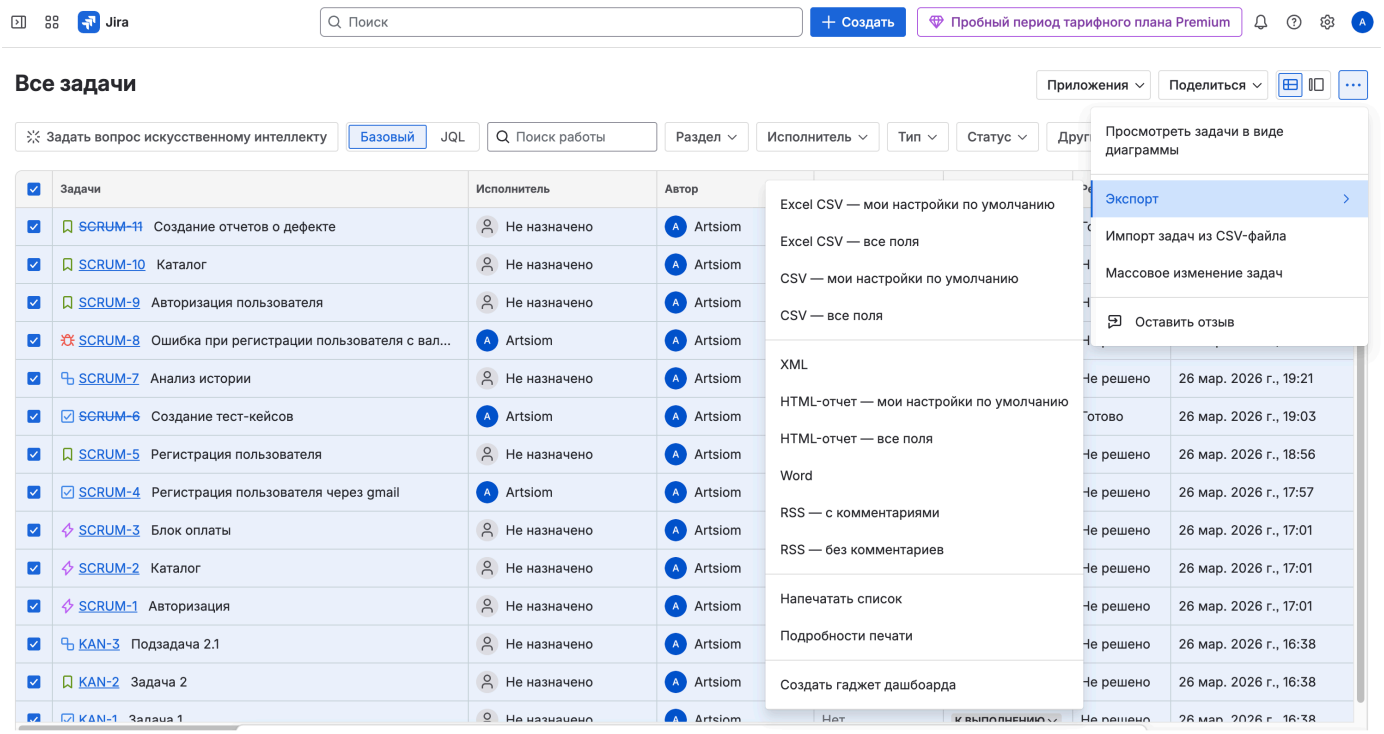



- При **выборе JQL** используется язык, похожий на SQL для баз данных. Если вы нажмете на JQL при выбранных фильтрах в базовом поиске, то он автоматически превратиться в нужный запрос. Подробнее об использовании этого языка можно прочитать здесь.




## Экспорт и импорт задач

1. Для экспорта (выгрузки) задач в окне со всеми задачами выберите соответствующее действие и нужный выходной формат.




> Если вам нужны отчеты о дефекте в pdf, то выберете опцию "Подробности печати". Далее Ctrl + P и сохраните файл в PDF.

2. Импорт задач (загрузка в Jira) возможен только из CSV файла.

3. Обратите внимание, что в контекстном меню доступно "Массовое изменение задач", в котором вы можете вносить изменения не в отдельную задачу, а в целую группу. В качестве домашнего задания попробуйте изменить несколько задач таким способом.

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

# 25. Работа в Youtrack (версия 2026)

**YouTrack** – это система по управлению проектами, в том числе используется для создания отчетов о дефекте. Именно поэтому в корпоративной среде такие системы называют **багтрэкинговыми**.

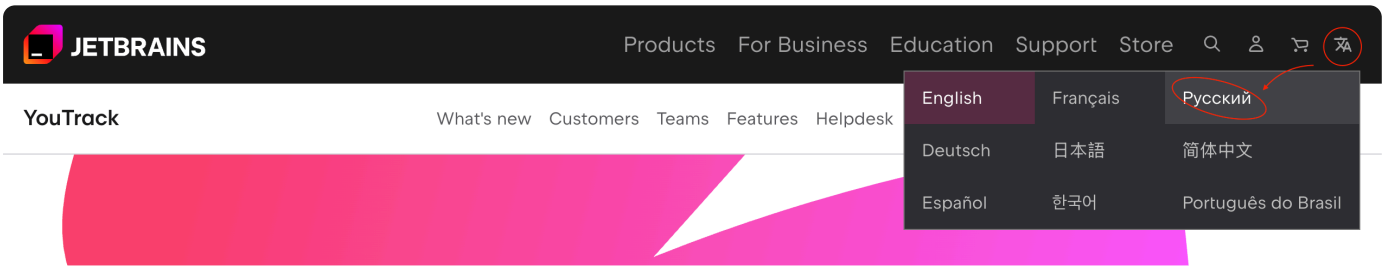
Разработана компанией JetBrains и является одним из популярных решений на рынке. В России и Беларуси для работы с ней может понадобиться VPN.

В рамках данного урока мы разберем основные функции Youtrack, которые могут пригодиться тестировщику каждый день.

Все остальное можно прочитать в официальной документации.

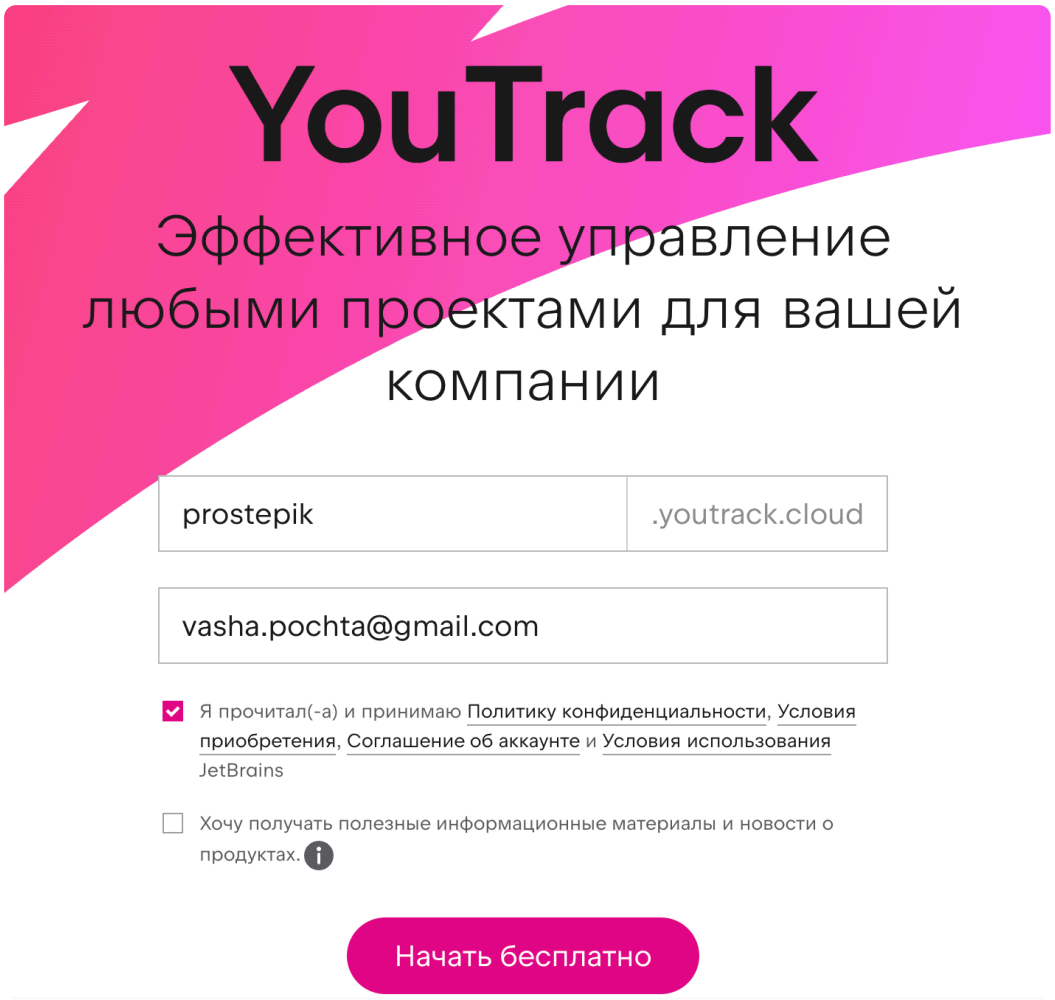
**Давайте зарегистрируемся и создадим свой проект**

1. Перейдите на сайт и измените язык на русский.

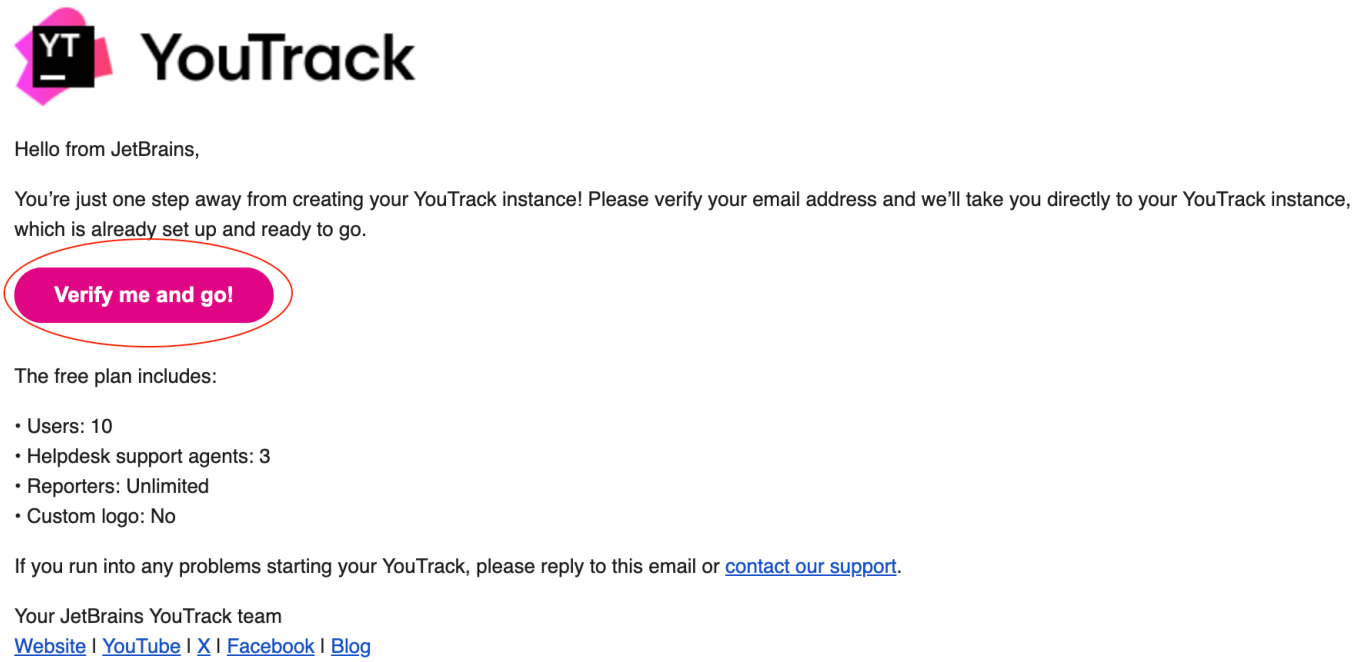



2. Придумайте название для сервера (instancename), введите почту и согласитесь с политиками. Нажмите

Начать бесплатно



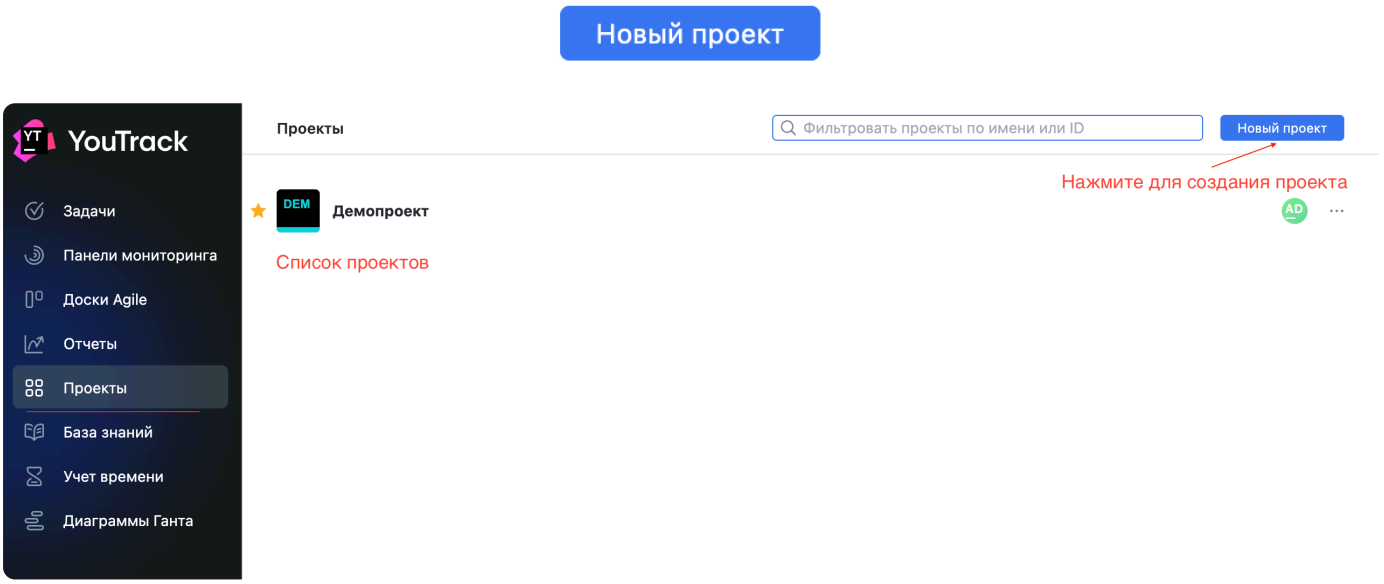

3. Подтвердите свою почту в письме. На бесплатном тарифе вы можете работать с 10 пользователями на одном проекте. После вас переведет на стартовую странице сервера.



4. Придумайте пароль для администратора. Если нужно, то смените место хранения данных и язык системы на другой.

5. Ответьте на все вопросы первичной анкеты и начните работу с сервером. После этого в Youtrack будет создан демо-проект. Он представляет собой задачи для обучения работы в системе. Вы можете пройти шаг за шагом каждую из них, но на курсе мы будем работать в своем проекте и по своему плану.

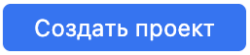
6. Для создания нового проекта перейдите во вкладку "Проекты". Нажмите




7. Как и в Jira, вам предложат уже заготовленные шаблоны и настроенные процессы для разных задач. Предлагаю выбрать Scrum. После выбора нажмите

.

8. Придумайте название проекта. ID проекта создастся автоматически и будет всегда в ID задач, например, PRO-1. Также вы можете добавить: число для отсчета первой задачи (каждая новая будет +1), описание проекта, дополнительную организацию. Нажмите



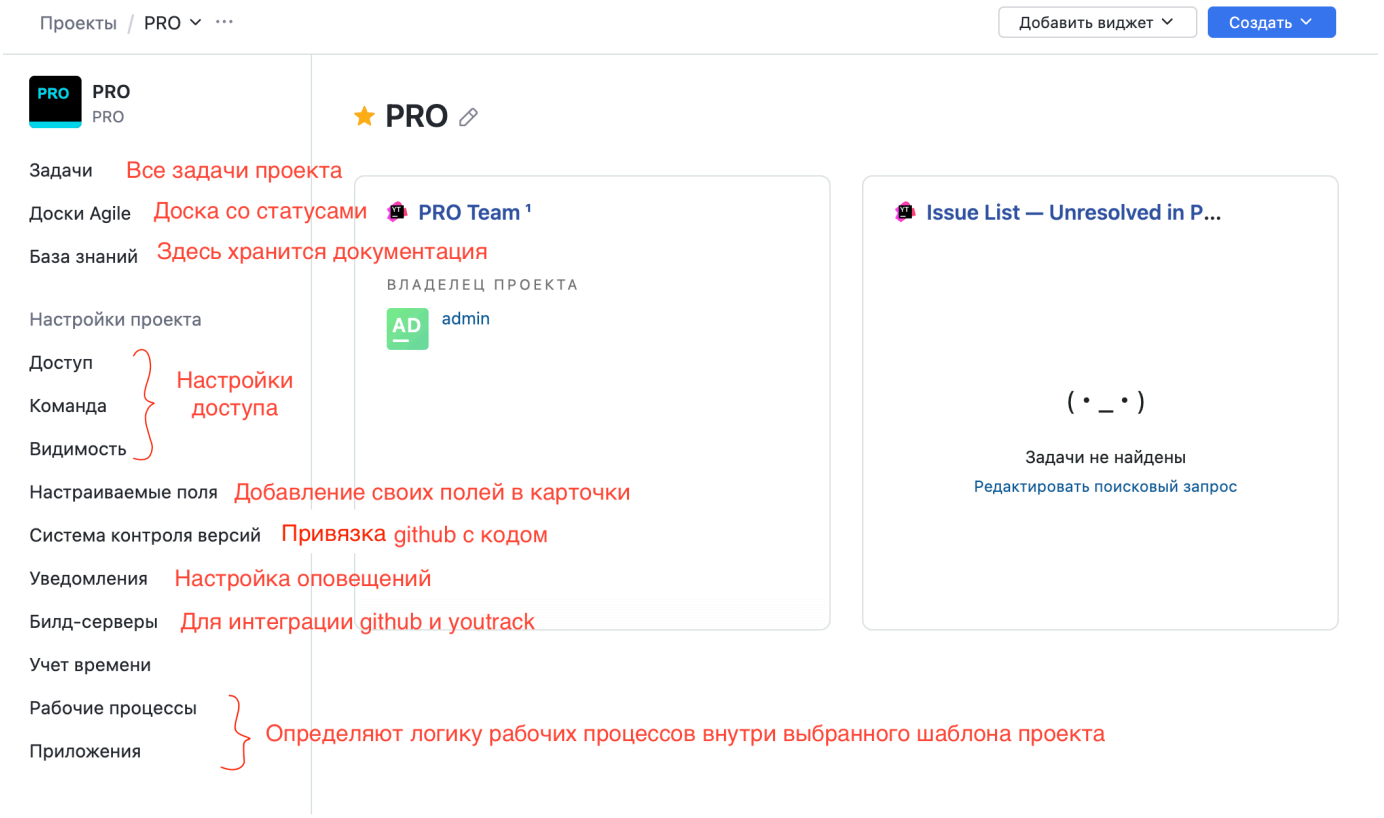

.

9. Далее вы можете добавить людей в проект и включить учет времени. Я пропущу

.

10. После этого вы увидите страницу своего проекта. В рамках следующих шагов мы не будем останавливаться на каждой вкладке, а сделаем акцент

только на самом главном: Задачи, Доски, База знаний, Настраиваемые поля. Остальные настройки нужны для менеджеров и администраторов YouTack.




**Все вкладки в Youtrack**

- **Задачи:** Огромный список всех тикетов. Здесь удобнее всего искать конкретную задачу через фильтры и поисковую строку.
- **Панели мониторинга (Dashboards):** Ваш персональный "центр управления". Сюда выводятся виджеты: графики, календари или списки критических багов, чтобы видеть всё важное на одном экране.
- **Доски Agile:** Визуальное пространство для работы (Scrum или Kanban). Те самые колонки, по которым вы перетаскиваете карточки в течение дня.
- **Отчеты:** Статистика и аналитика. Здесь можно посмотреть, как быстро команда закрывает задачи (Burndown chart) или кто из сотрудников перегружен.
- **Проекты:** Список всех ваших рабочих областей. Тут можно посмотреть общую информацию о проекте и настроить его "под себя".

## Документация и время

- **База знаний:** Внутренняя "Википедия" команды. Здесь хранятся инструкции, регламенты, ТЗ и описания фич (функций).
- **Учет времени:** Тайм-трекинг. Здесь фиксируется, сколько часов было потрачено на задачи (план vs факт).
- **Диаграммы Ганта:** Планирование на календаре. Показывает сроки этапов и как задержка одной задачи сдвигает весь проект.

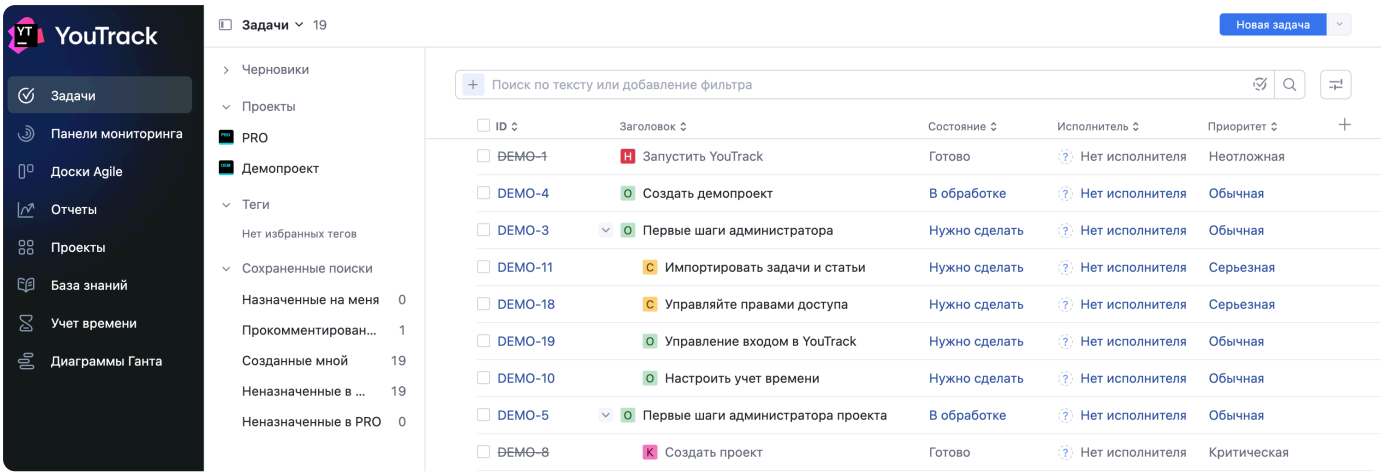
**Системные разделы**

- **Создать (+):** Быстрая кнопка для создания новой задачи или статьи в базе знаний.
- **Администрирование:** Настройка прав доступа, пользователей и полей задач (доступно только админам).
- **Справка:** Официальная документация YouTrack.
- **Уведомления:** Сюда приходят сообщения о комментариях, тегах и изменениях в ваших задачах.
- **admin (Профиль):** Ваши личные настройки: аватар, пароль, тема интерфейса (светлая/темная) и уведомления на почту.

## Вкладка "Задачи"

Прежде чем мы перейдем к созданию задач, я бы хотел рассмотреть общий интерфейс программы.

**Вкладка "Задачи"** – здесь отображаются задачи со всех проектов, доступ к которым у вас есть. Так как сейчас вы сами создали сервер, то доступ есть ко всему. На реальных проектах их будет меньше,

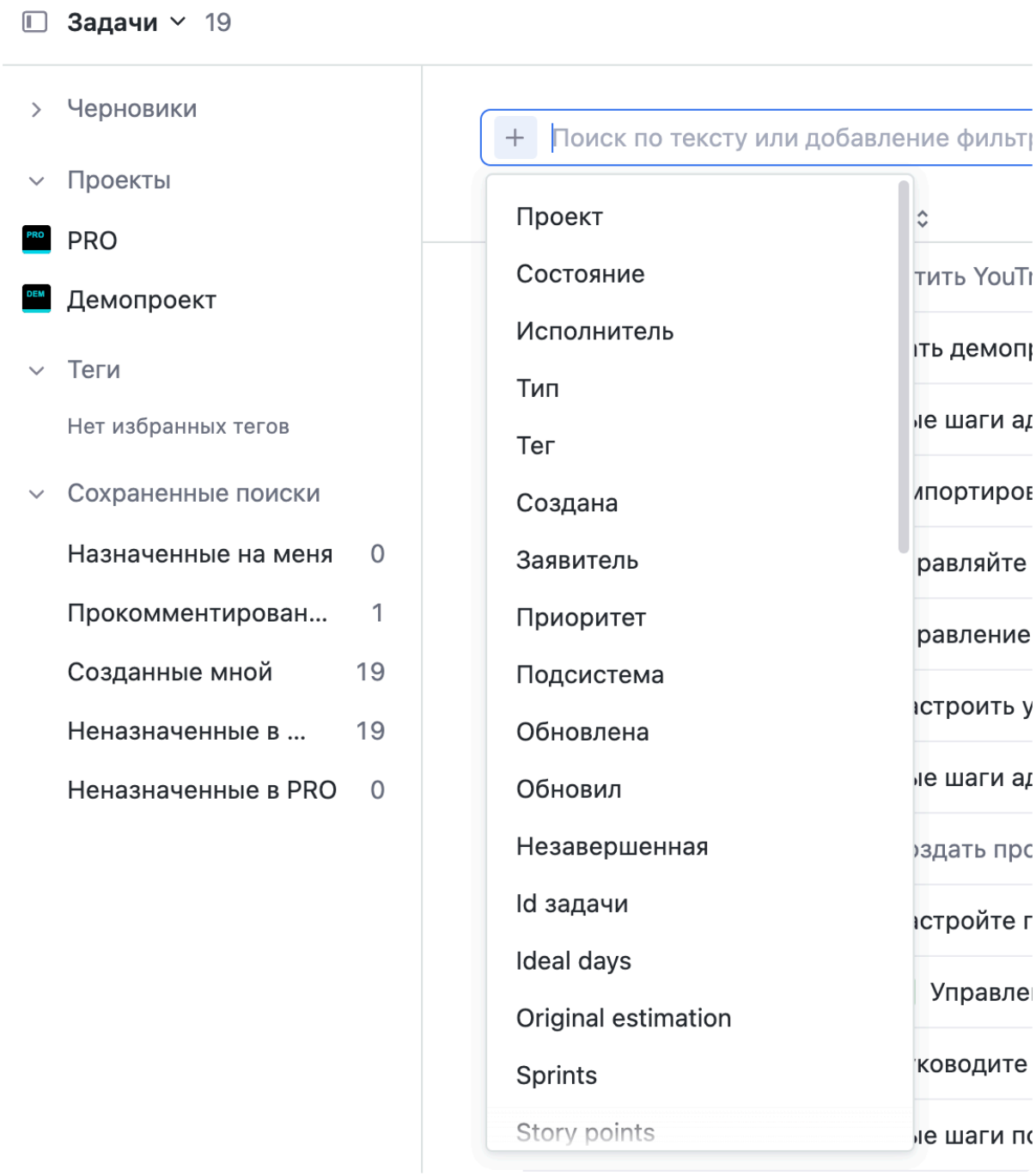



## Поиск задач

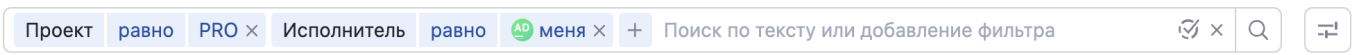
Здесь нас интересует поиск задач. Вы можете делать его, используя простой или расширенный формат.

- В **простом формате** нужные фильтры добавляются через кнопку

. Он выбран по умолчанию.

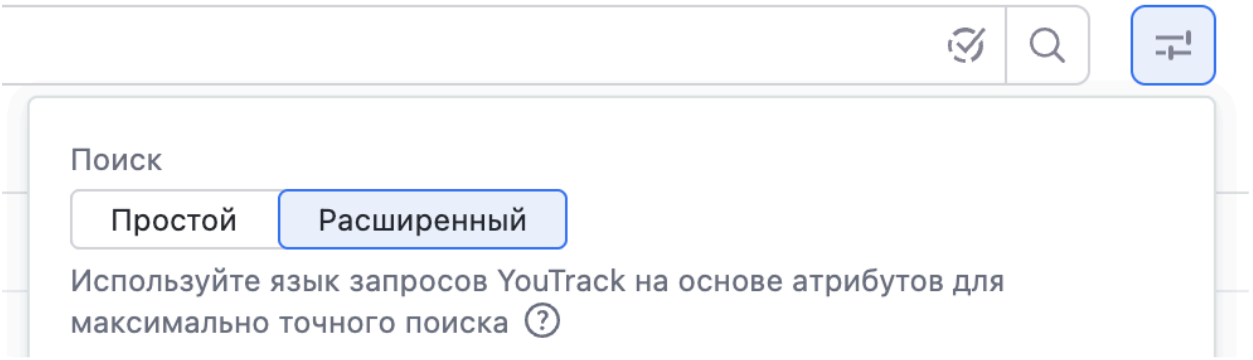



После выбора и назначения условия должен получиться такой запрос.

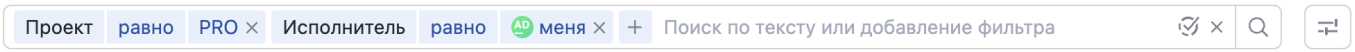



- **Расширенный формат** включается через меню

. Для поиска используется специальный [язык запросов](#).




После его активации запрос:




Изменит вид на:

Вы можете сохранить поиск

Сохранить как новый поиск

, чтобы использовать его для формирования нужных наборов, например, баги по регрессии, текущему релизу и так далее.

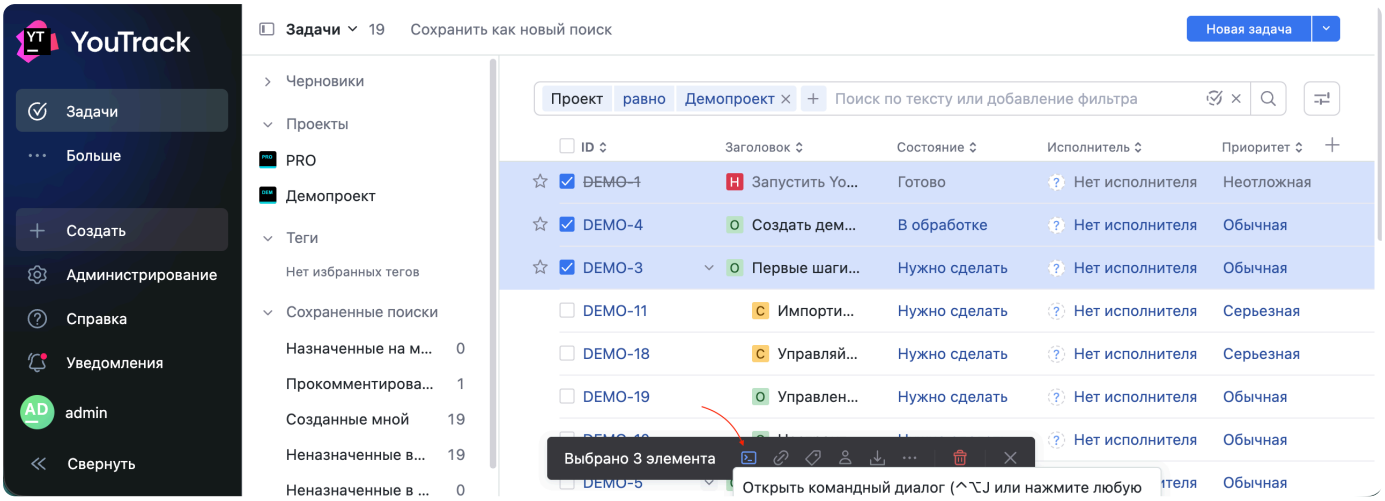
**Массовая работа с задачами**

Вы можете изменять несколько задач одновременно.

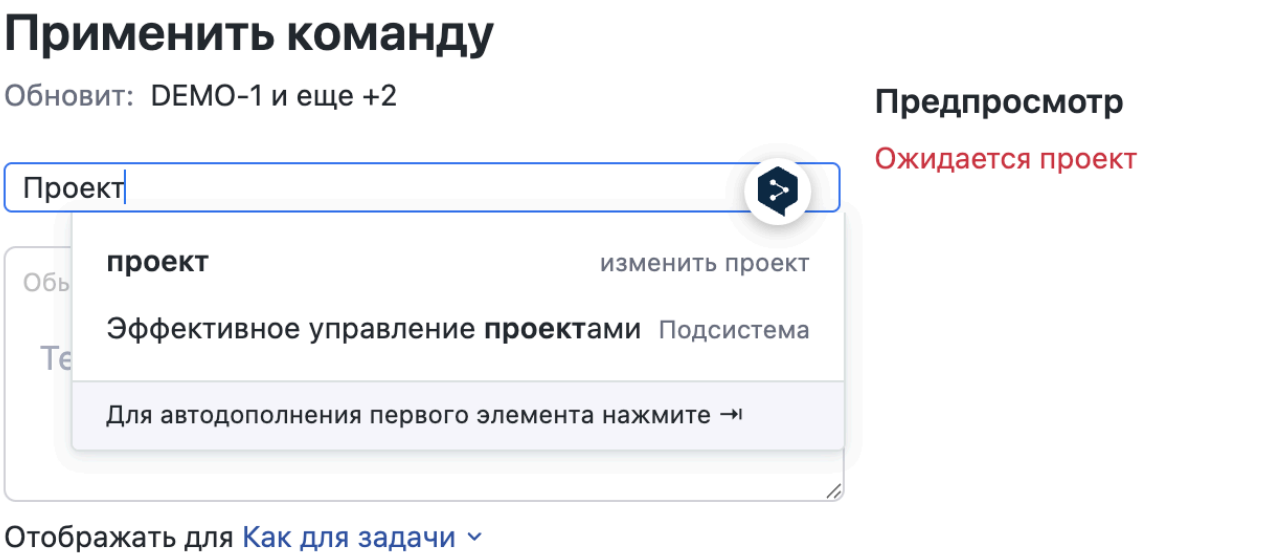
1. Во вкладке "Задачи" выберите нужные объекты.

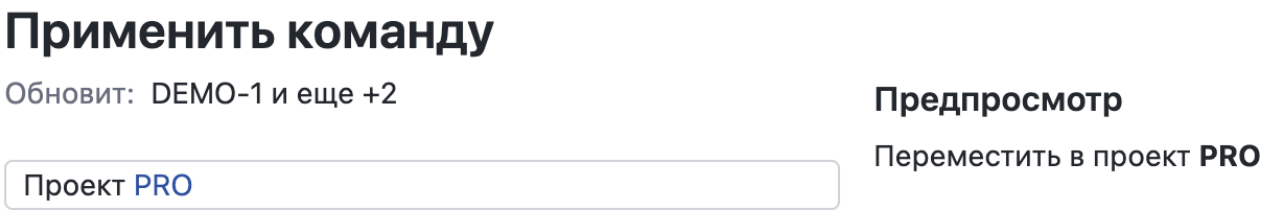
2. Нажмите

для изменения задач через командный диалог.

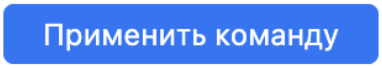



3. Например, если вы хотите изменить значение атрибута, то введите его название и новое значение. Давайте заменим проект. Youtrack будет подсказывать, что вводить и подсвечивать ошибки. Обратите внимание, что, в зависимости от выбранного языка, запрос может изменяться.



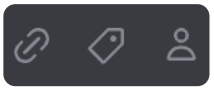




Нажмите



. Теперь проект в задачах изменился.

4. Также вы можете добавлять связи между задачами, тэги для улучшения прослеживаемости, изменять исполнителей этими кнопками



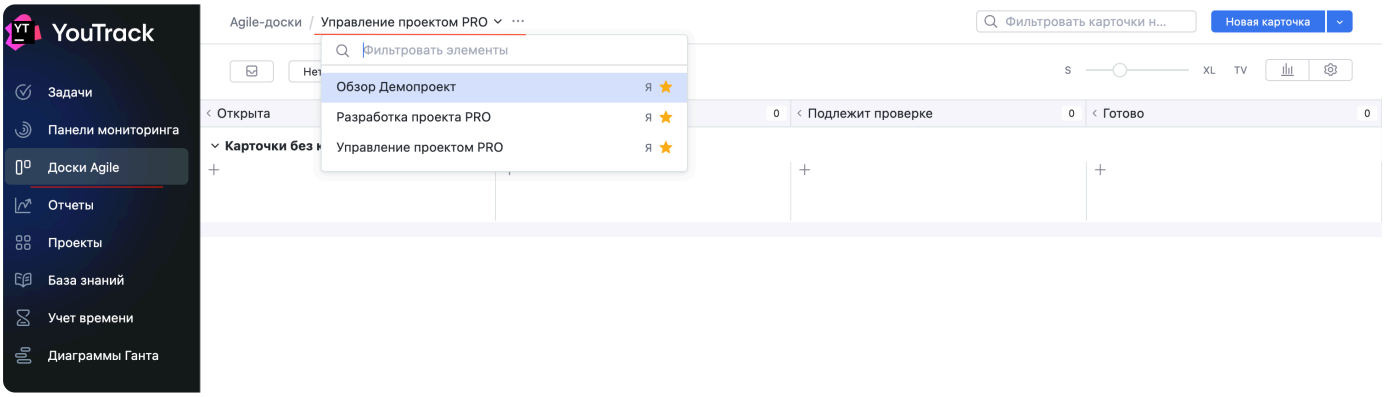
.

5. Для экспорта задач (сохранения в файл, например, XLSX) используйте эту кнопку

. Нужен для переноса данных в другие системы, для отчетов.

## Вкладка "Доски Agile"

**Вкладка "Доски Agile"** визуализирует бэклог всех задач, которые находятся в разных статусах.




Для вашего проекта автоматически создалось **две доски:**

### 1. Разработка проекта PRO

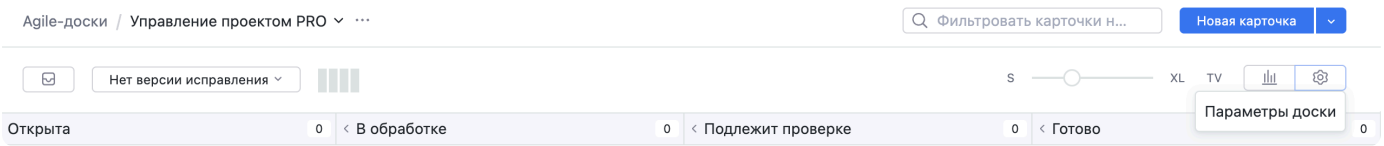
- **Для кого:** Для технической команды (разработчиков, QA, дизайнеров).
- **Особенности:**
  - **Колонки:** Отражают технические этапы: *«В работе», «Code Review», «Тестирование», «Готово»*.
  - **Задачи:** Здесь отображаются мелкие подзадачи (Subtasks), баги и конкретные тикеты на разработку.

- **Свимлейны (горизонтальные строки):** Могут быть настроены по исполнителям, чтобы каждый разработчик видел свою «полосу» работы.

**2. Управление проектом PRO**

- **Для кого:** Для менеджеров проектов (PM), владельцев продукта (PO) и стейкхолдеров.
- **Особенности:**
  - **Уровень:** Здесь отображаются **Эпики (Epics)** или **User Stories**.
  - **Колонки:** Могут быть более общими: *«Бэклог», «В реализации», «Релиз»*.
  - **Свимлейны:** Часто настроены по приоритетам или по версиям (релизам), чтобы менеджер видел общую картину готовности продукта к запуску.

Представление каждой доски, принцип группировки задач, статусы назначаются индивидуально для каждого проекта в "Параметры доски": менеджерами и администратором. Тестировщики это не делают, поэтому мы не будем это рассматривать в уроке.

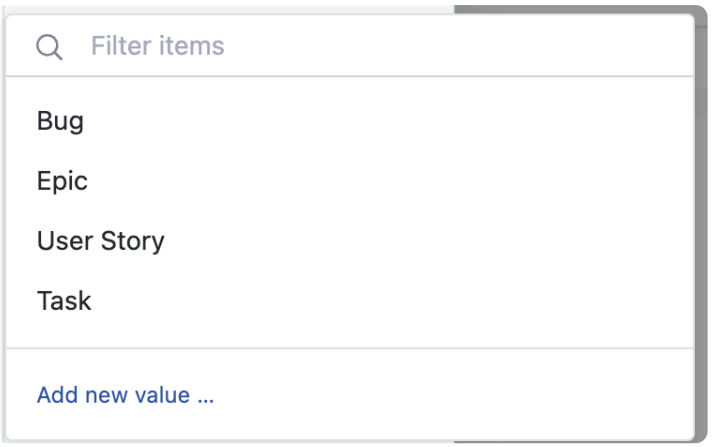



**Типы карточек**

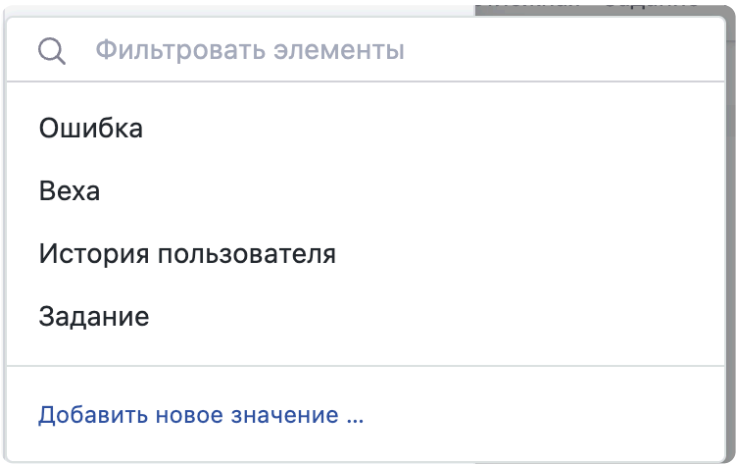
В отличие от Jira, где все сущности называются задачами, в Youtrack используются карточки. Название типов задач на русском языке может отличаться. И это может запутать. Поэтому на реальных проектах лучше просить приводить их к общепринятым стандартам.

Для точного перевода и наглядности покажу, как выглядят одни и те же задачи на русском и английском.

**Английская версия**




**Русская версия**




Мы продолжим работу без изменений, чтобы вам было проще и мы не нарушили маппинг в процессах, т.е. при смене языка у вас все будет работать:

- Bug = Ошибка
- Epic = Веха
- User Story = История пользователя
- Task = Задание

Но вы всегда можете добавить новый тип через опцию

Добавить новое значение ...

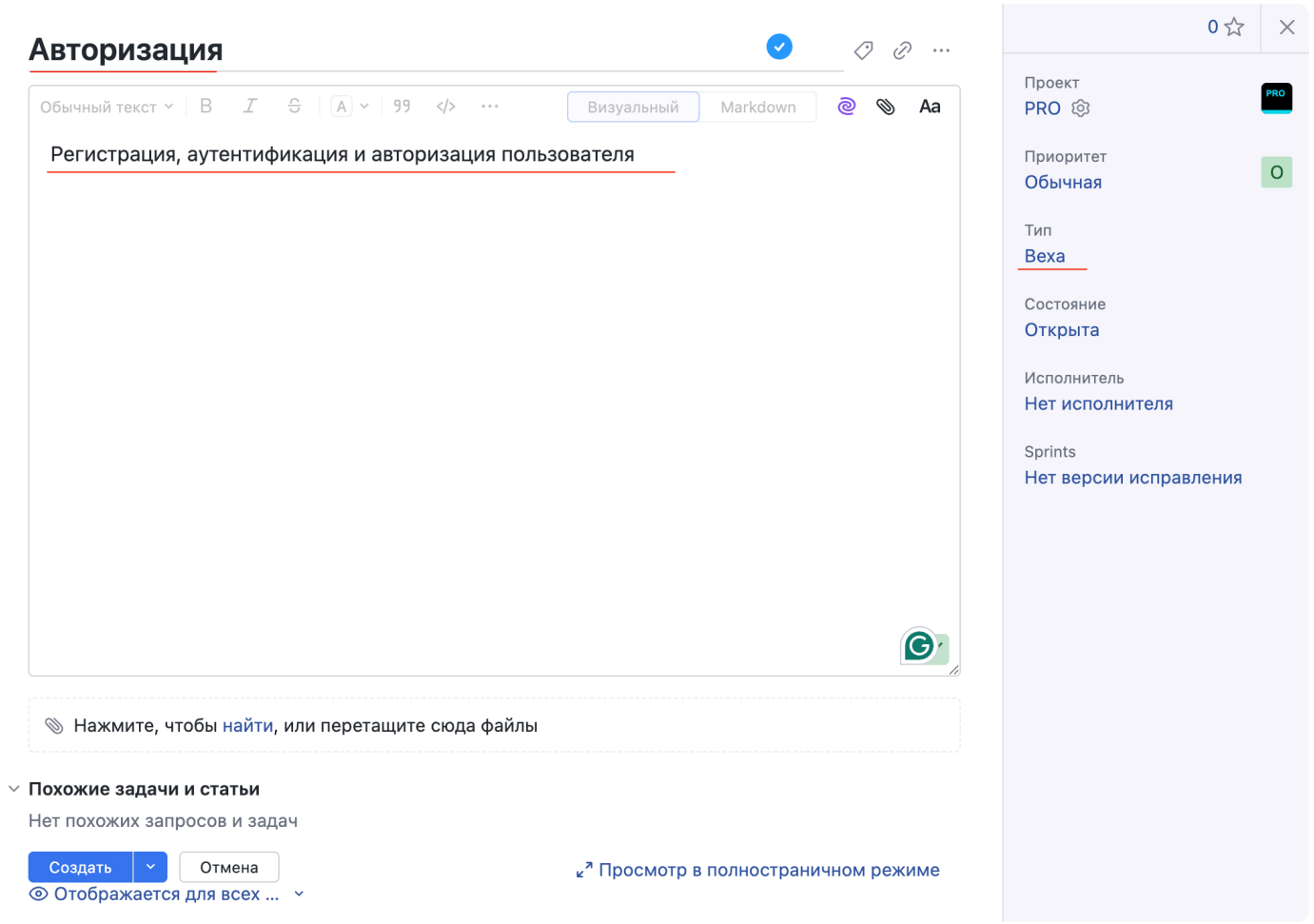
### Создание карточки Epic (Веха)

1. Перейдите во вкладку "Доски Agile" > доска "Разработка проекта PRO". Нажмите "Начать спринт".

2. Нажмите

.

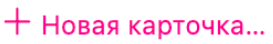
3. Создайте Epic (Веха): "Авторизация". Нажмите "Создать".



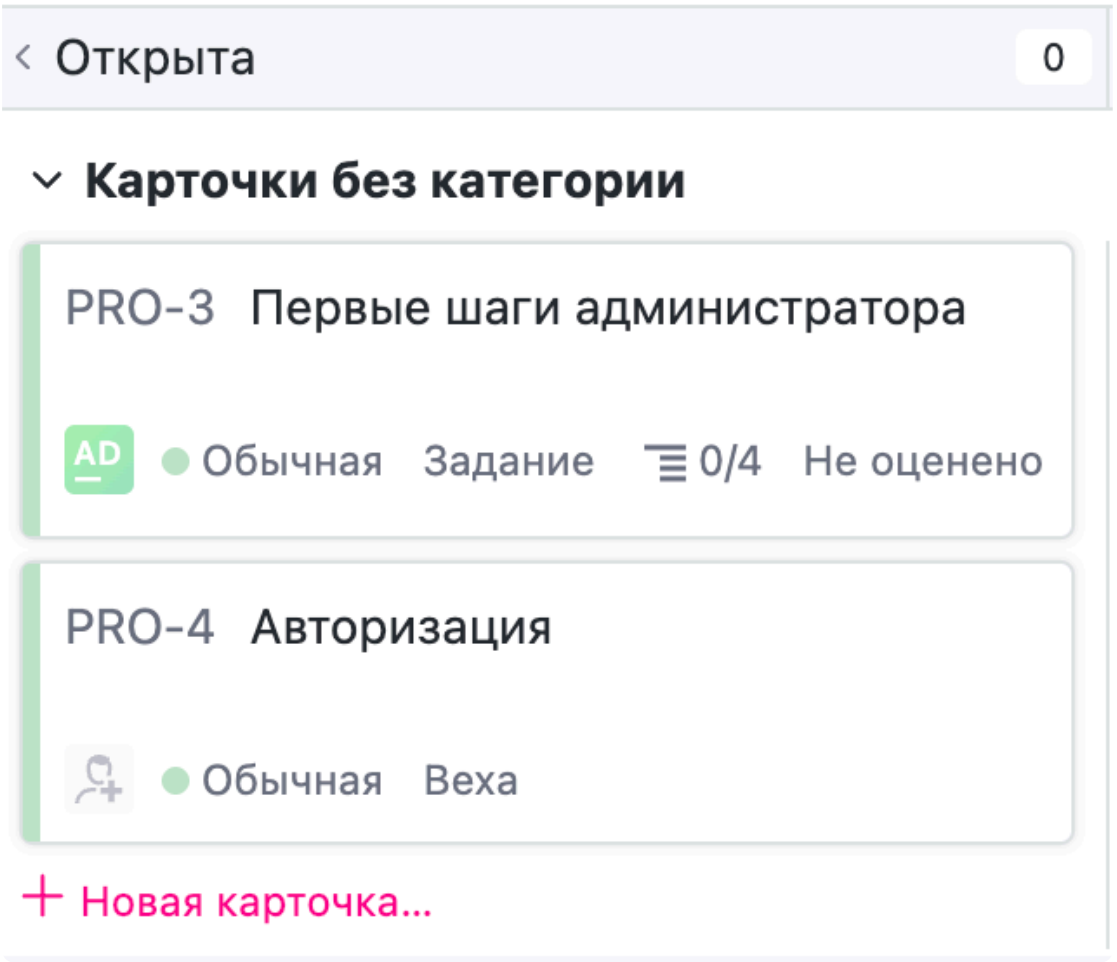

Напоминаю, что эпик – это крупная функциональность или проект, который нельзя сделать за один присест (спринт). В него входит несколько пользовательских историй.

> У каждой карточки и типа задачи есть свой набор полей. Их можно менять в настройках проекта > "Настраиваемые поля". Для текущего урока нам достаточно значений по умолчанию.

4. Новая карточка появилась в столбце "Открыта". Далее ее переносят в другие столбцы по мере готовности. Обратите внимание, что вы можете сразу создавать карточки в нужном столбце, используя кнопку

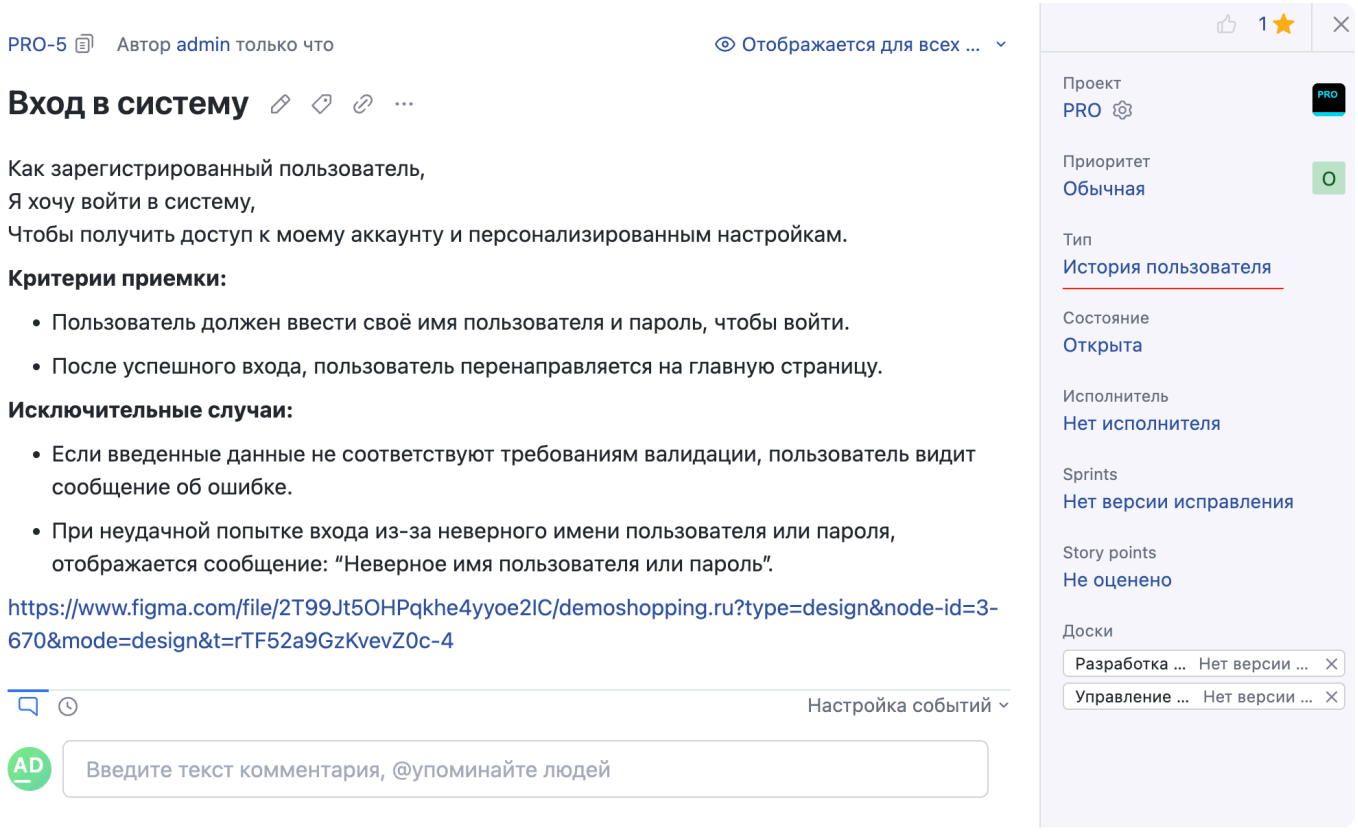


.

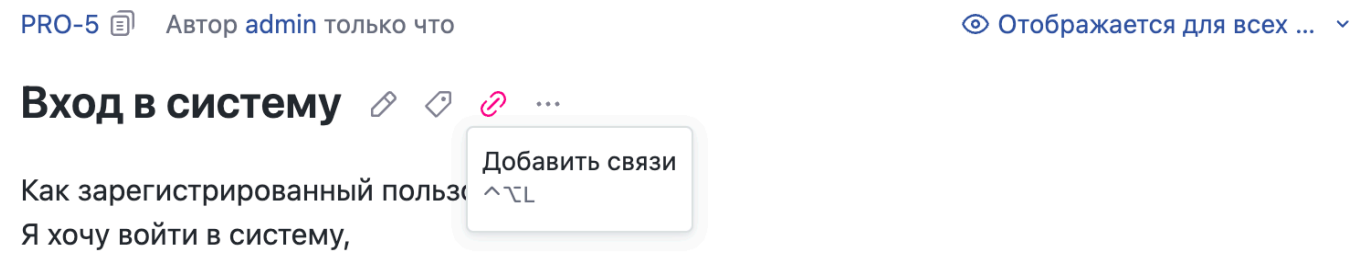



**Создание карточки User Story (История пользователя). Создание связей**

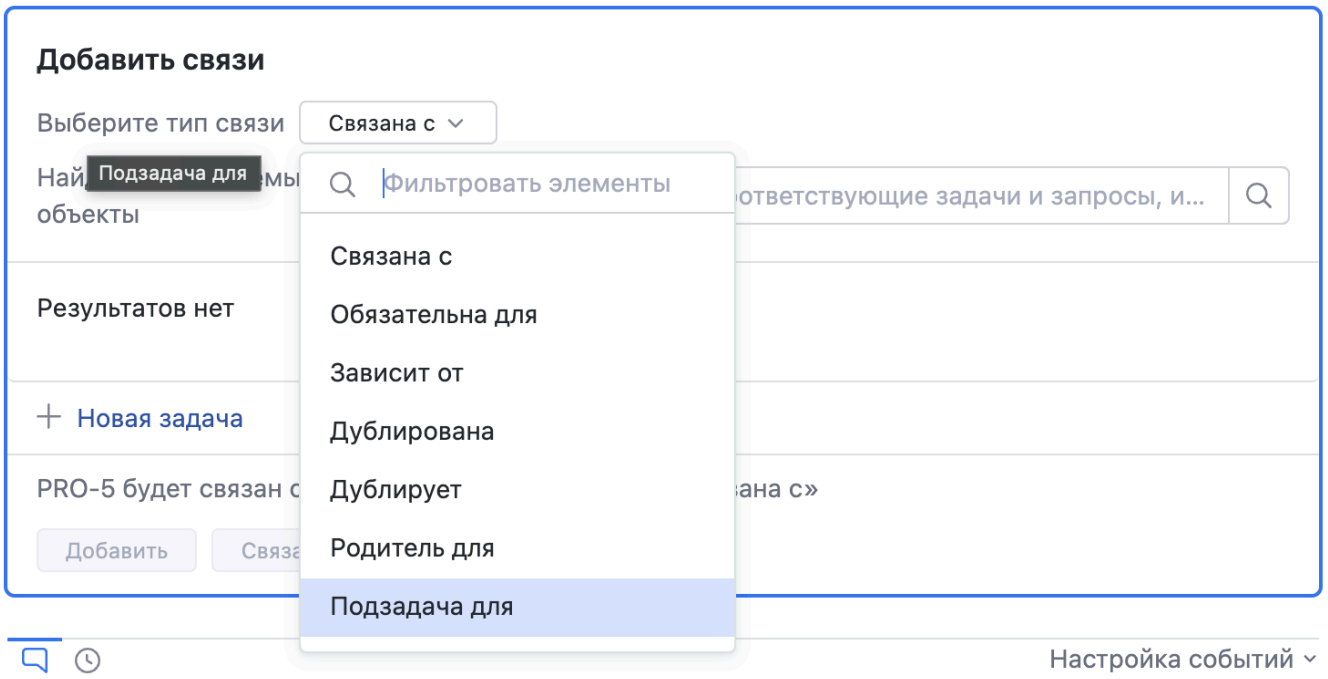
1. Самостоятельно перенесите пользовательскую историю (User Story) из Kaiten в Youtrack




2. Истории обычно привязываются к Epic или Feature. В нашей системе есть только эпики, поэтому давайте создадим связь с ним. Для этого нажмите на кнопку со скрина:



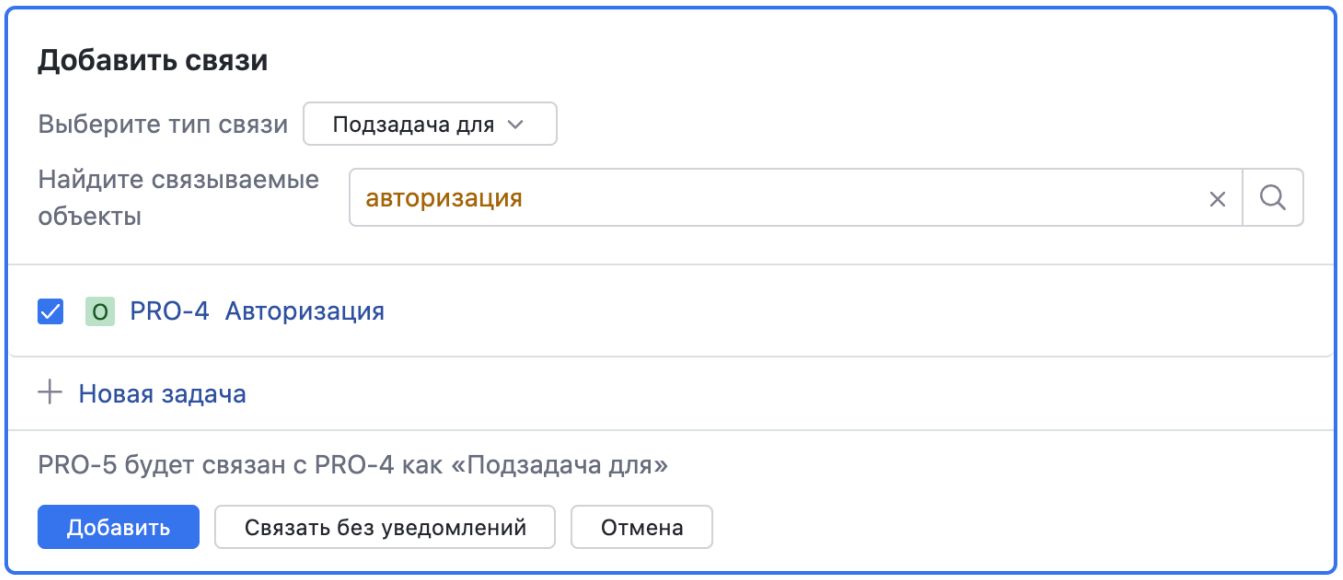

7. Выберите нужный тип связи для истории. В нашем случае это будет "Подзадача для".




Связи помогают YouTrack понимать, как задачи влияют друг на друга.

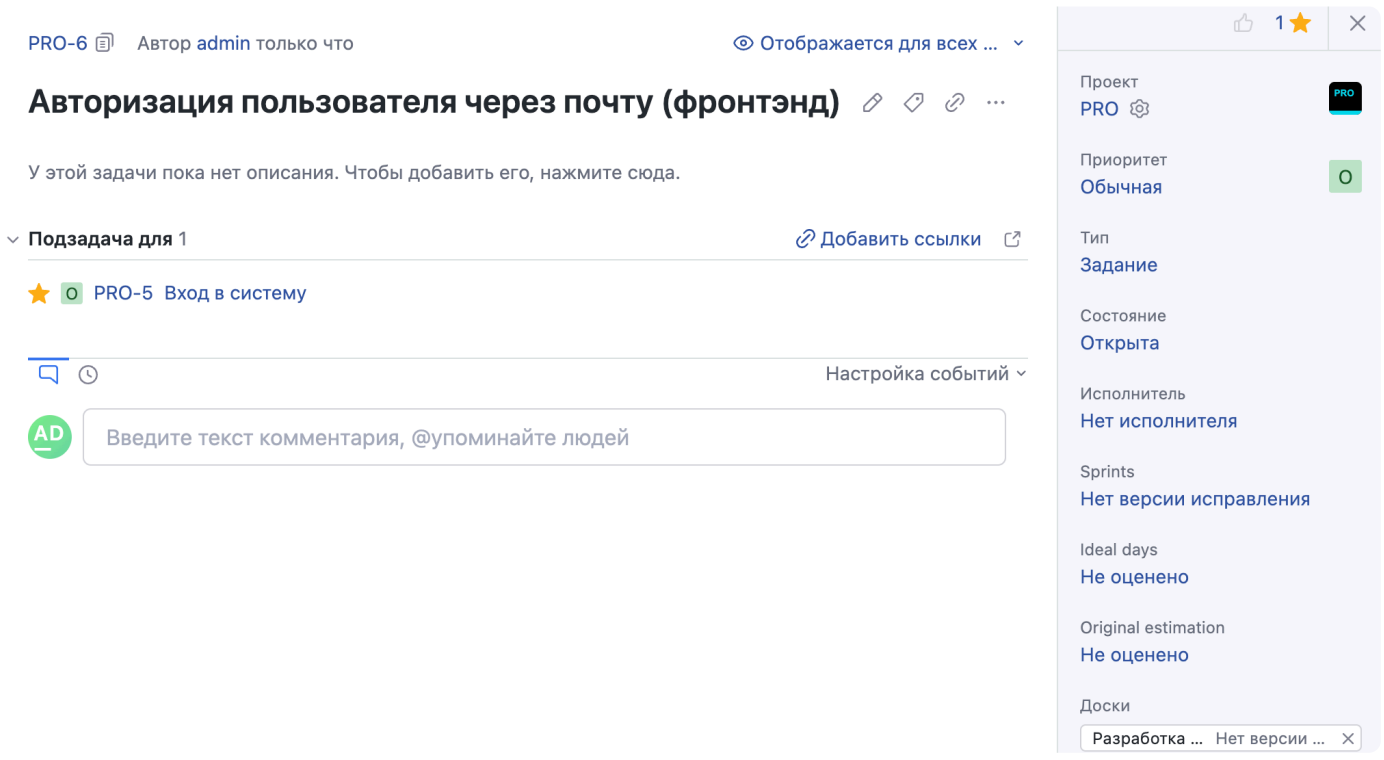
- **Связана с:** Самая простая связь. Задачи просто имеют отношение друг к другу (например, "посмотри еще вот этот тикет для контекста"), но не блокируют работу.
- **Обязательная для:** Текущая задача является "блокером". Пока вы её не закроете, нельзя будет закрыть ту, с которой она связана.
- **Зависит от:** Обратная связь. Текущую задачу нельзя завершить, пока не будет готова та, от которой она зависит.
- **Дублирована:** Эту задачу кто-то уже создал раньше. Эта карточка – копия, её обычно закрывают.
- **Дублирует:** Текущая задача сообщает, что она повторяет другую, уже существующую задачу.
- **Родитель для:** Текущая задача – главная (например, Эпик или История). Внутри неё будут находиться более мелкие части.
- **Подзадача для:** Текущая задача – это мелкий этап внутри большой задачи. Она "дочка»"по отношению к другой.

3. Найдите нужный эпик "Авторизация" и нажмите "Добавить".




4. Теперь в истории появилась связь.

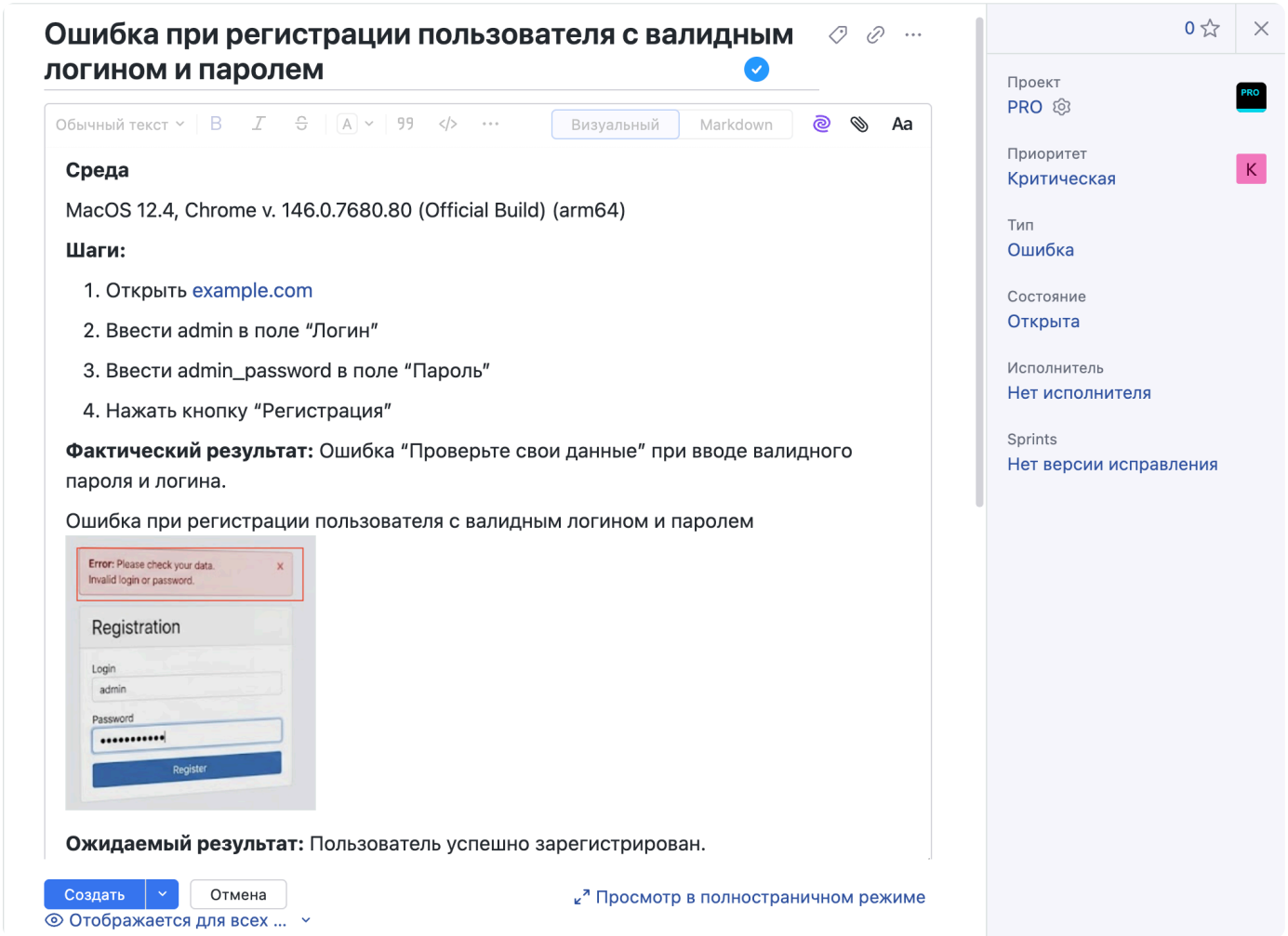
5. По аналогии можете создать **"Задание" (Task)** – это конкретное действие, которое нужно совершить разработчику, дизайнеру или маркетологу. Задача – это атомарная единица работы. В нашем процессе она будет подзадачей для пользовательской истории.



## Создание карточки Bug (Ошибка)

1. Создайте гипотетический баг в системе. Как и в Jira изображения можно добавлять Ctrl + C / Ctrl + V. Остальные файлы через скрепку в правом верхнем угле

. Также не забудьте добавить информацию о тестовой среде, приоритет и другие атрибуты, если они нужны.




Также я добавил связь к эпику "Авторизация" :

2. Если каких-то атрибутов нет, то их можно настраивать через настройки проекта > "Настраиваемые поля", либо как в примере на скрине (*атрибут*

*"Среда"*) добавлять прямо в описание дефекта.

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

# 26. Матрица трассировки и метрики в тестировании

**Матрица трассируемости** (traceability matrix, RTM) - двумерная таблица, описывающая связь двух сущностей (например, требований и тестовых сценариев).

Таблица позволяет производить прямую и обратную трассировку от одной сущности к другой, обеспечивая таким образом возможность определения покрытия и оценки влияния предполагаемых изменений.

Базовая матрица служит для визуальной оценки тестового покрытия, где указаны требования и тест-кейсы, проверяющие их.

На пересечение ставится метка, благодаря которой можно принять решение об оптимизации количества кейсов для проверки требования.

## Пример матрицы трассируемости

| | TC 1 | TC 2 | TC 3 | TC 4 |
|---|---|---|---|---|
| REQ 1 | | + | | |
| REQ 2 | + | | | |
| REQ 3 | | + | + | |
| REQ 4 | | | | + |

### Популярные метрики, которые служат для оценки качества продукта и эффективности команды

**Дефекты**

1. **Общее количество найденных багов:** Это основная метрика, показывающая количество выявленных дефектов в программном обеспечении. Например, если тестировщики обнаружили 100 багов, это число будет отражать общую эффективность тестирования.

2. **Общее количество неисправленных багов на проде:** Эта метрика показывает количество багов, которые еще не были исправлены в продакшн-версии продукта.
3. **Баги в разрезе приоритета/серьезности:** Эта метрика классифицирует баги по уровню их важности и влиянию на систему, например, критические, высокие, средние и низкие.
4. **Баги в разрезе модуля:** Это показывает распределение багов по различным модулям или компонентам системы. Это помогает определить, какие части системы наиболее уязвимы.
5. **Неподтвержденные дефекты:** Отношение числа дефектов, статус которых еще не подтвержден, к общему количеству дефектов.
6. **Повторно открытые дефекты:** Это отношение количества багов, которые были повторно открыты после их первоначального исправления, к общему числу дефектов.

**Кейсы**

1. **Общее количество написанных кейсов:** Общее число разработанных тест-кейсов.
2. **Общее количество автотестов:** Количество тестов, автоматизированных для повторного использования.
3. **Процент автоматизированных смоук-кейсов:** Доля автоматизированных тестов из общего числа смоук-тестов.
4. **Процент автоматизированных регрессионных кейсов:** Аналогично, но для регрессионных тестов.
5. **Процент автоматизированных кейсов с высоким приоритетом:** Доля автоматизированных тестов с высоким приоритетом.
6. **Процент покрытия требований мануальными тестами:** Отношение числа мануальных тест-кейсов к общему числу требований к программному обеспечению.

**Оценка трудозатрат**

1. **Общее количество времени на регрессионное тестирование:** Общее время, затраченное на регрессионное тестирование.
2. **Количество времени, затраченное на регрессионное тестирование с учетом автоматизации:** Время, затраченное на регрессионное тестирование, но с учетом использования автоматизированных тестов.

3. **Количество времени, затраченного на смоук тестирование:** Общее время, затраченное на смоук тестирование.
4. **Точность эстимации:** Это соотношение между оценочным и фактическим временем, затраченным на тестирование. Это помогает улучшить точность будущих оценок.

**Дополнительные метрики**

1. **Оценка качества кода разработчика:** Это соотношение между количеством дефектов, найденных в коде конкретного разработчика, и общим количеством дефектов.
2. **Удовлетворенность пользователей:** Оценка, полученная на основе опросов пользователей и оценок в магазинах приложений. Это помогает понять, насколько хорошо продукт соответствует ожиданиям пользователей.

**Рекомендации**

1. Все метрики должны быть согласованы до их применения
2. Основные из них должны попадать в отчет по результатам тестирования и презентоваться команде
3. Обязательное отслеживание в динамике, все метрики просчитываются каждую итерацию и анализируются

**Полезные материалы**

1. Общая презентация
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/matrix
3. Старая версия урока https://youtu.be/OyCnB2LvAtQ
4. Файл с шаблоном матрицы и перечнем метрик https://docs.google.com/spreadsheets/d/11UvpZkypbMmhlF0IyArLPOKnaWRc5gvJ
5. Библия QA. Матрица трассируемости (RTM - Requirement Traceability Matrix) https://vladislaveremeev.gitbook.io/qa_bible/testovaya-dokumentaciya-i-artefakty-test-deliverablestest-artifacts/matrica-trassiruemosti-rtm-requirement-traceability-matrix

ТЕСТОВАЯ ДОКУМЕНТАЦИЯ

# 27. Оценка трудозатрат в тестировании. Виды эстимации

**Оценка затрат на тестирование** (test estimation) – аппроксимация объема трудозатрат, связанная с различными аспектами тестирования. [ISTQB Glossary]

*Перевожу: расчет времени, затраченного на тестирование.*

### Популярные техники оценки трудозатрат (эстимации)

1. Пальцем в небо (метод проб и ошибок)
2. Процентное отношение к разработке (например, на 1 тестировщика приходится 2-3 разработчика; значит на тестирование тратится в 2-3 раза меньше времени, чем на разработку)
3. Процентное распределение, при котором на все фазы SDLC выделяется определенный процент времени. Время выделенное на тестирование, разделяется на отдельные фазы STLC.
4. Эстимация, основанная на предыдущем опыте
5. Структурная декомпозиция, при которой крупные задачи разделяются на более мелкие, которые легко оценить
6. Эстимация по трем точкам

*Дополнительную информацию по эстимации можно прочесть в ISTQB CTFL 2023.*

### Простой пример эстимации

*Вводные данные:* команда состоит из 3 человек, в регрессионном наборе 30 кейсов, на прохождение одного кейса тратится 10 минут, обычно 20% тестов находят новые баги и нужен ретест. Для простоты подсчета не будем учитывать время на написание документации и уточнение требований.

1. Определяем время на регрессионное тестирование: (30 * 10) / 3 = 100 минут
2. Определяем время на ретест: 100 * 0.2 = 20 минут
3. Примерное время на тестирование 120 минут

4. Обязательно нужно учесть риски: недоступность членов команды, проблемы с ПО, внештатное увеличение количества багов. Заложим на это еще 10% времени.

Итого общее время на тестирование: **132 минут**

### Эстимация по трем точкам

Наиболее точным методом оценки трудозатрат является оценка по трем точкам. Давайте усовершенствуем предыдущий расчет.

Рассчитывается по следующей формуле:

**(O + (4 × M) + E)/6, где**

**O** - оптимистичная оценка, когда все идет по плану

**M** - наиболее вероятная оценка, с учетом возникающих проблем

**E** - пессимистичная оценка, когда все идет не по плану (также может фигурировать **P**)

Также рассчитывается стандартное отклонение: **(E − O)/6**

На нашем примере это будет выглядеть так: **(100 + (4*120) + 132)/6 = 118.7 минут**

Финальный вариант: **118.7 ± 5.3 минут**

### Рекомендации

1. Сохраняйте результаты по времени, которое вы затратили на тестирование. Для этого есть отдельные поля в TMS и/или отдельная таблица, которую нужно завести
2. Эстимация включает в себя не только время, которые вы потратили на само тестирование, но и время на придумывание кейсов, их оформление, общение с аналитиками
3. Оценка трудозатрат всего проекта складывается из работы всех участников команды, а не только отдела разработки. Об этом нужно

напоминать
4. Всегда держите в голове риски и время, которое будет необходимо на повторное тестирование. Возвращаемся к п.1, так как аналитика, статистика и прогнозирование не может существовать без ваших данных
5. Полученную оценку нужно периодически обновлять
6. Будьте готовы к тому, что на незрелых проектах существует только одна оценка: пальцем в небо :)
7. В рамках набора тренажеров на Stepik я разработал специальный инструмент для автоматического расчета времени на тестирования по трем точкам. Может быть полезен на первых порах.

**Полезные материалы**

1. Общая презентация
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/estimation
3. Старая версия урока https://youtu.be/CfHBhmtES1g
4. Статья о методах эстимации (на английском) + шаблон для декомпозиции https://www.guru99.com/an-expert-view-on-test-estimation.html
5. Статья 1 о методах эстимации (на русском) https://coderlessons.com/tutorials/akademicheskii/izuchite-metody-otsenki/metody-otsenki-testirovanie
6. Статья 2 о методах эстимации (на русском) https://tmguru.ru/baza-znanij/protsess-testirovaniya/planirovanie/otsenka-trudozatrat-na-testirovanie/
7. Библия QA. Техники оценки тестов/оценка трудозатрат на тестирование (Test Estimation) https://vladislaveremeev.gitbook.io/qa_bible/obshee/tekhniki-ocenki-testov-ocenka-trudozatrat-na-testirovanie-test-estimation

# 28. Вступительное слово перед модулем "Тестирование веб-приложений"

**Вступительное слово перед стартом модуля "Тестирование веб-приложений"**

Сегодня мы начинаем погружение в тестирование отдельного домена, а именно веб-приложений. Наряду с мобильным тестированием это направление является одним из самых популярных и перспективных для специалистов в области обеспечения качества.

**Основные навыки и знания, необходимые для веб-тестировщика, которые мы закрепим в этом модуле:**

- Понимание клиент-серверной архитектуры, структуры HTTP-запросов и ответов, умение их анализировать
- Работа с инструментами разработчика внутри браузера. В данном курсе их мы будем изучать на примере браузера Chrome и DevTools. Особое внимание следует уделить интерфейсу, отдельным вкладкам и их применению для целей тестирования.
- Основы HTML и CSS для упрощения тестирования фронтенда
- Особенности тестирования различных веб-форм и элементов
- Особенности тестирования графического интерфейса

Тестирование бэкенда, а именно API, мы будем изучать в следующем модуле на примере инструмента Postman.

Параллельно изучению теоретической части модуля, я советую взять реальное веб-приложение и протестировать его.

Создать чек-лист и тест-кейсы, не забывая про техники тест-дизайна, провести тестирование с использованием DevTools, задокументировать отчеты о дефектах.

# 29. Как работает Internet? IP, MAC, URL, DNS

Чтобы уверенно чувствовать себя в профессии тестировщика, важно понимать, как устроен Интернет на базовом уровне. Особенно если ты тестируешь веб-приложения, взаимодействуешь с API, работаешь с DevTools или сетевыми логами. Ниже я собрал для тебя краткое пояснение, которое поможет «разложить по полочкам» техническую основу Сети.

**Что такое Интернет?**

Интернет — это не сайт и не браузер. Это глобальная сеть, которая соединяет миллиарды компьютеров по всему миру. Компьютеры обмениваются данными с помощью маршрутизаторов, кабелей и протоколов связи. Ключевая задача — передавать данные от одного устройства к другому, независимо от расстояния между ними.

**Как работает соединение?**

Если у тебя дома стоит роутер, ты уже подключен к Интернету. Он передаёт сигнал на модем, который связывается с провайдером. Провайдер подключён к другим провайдерам и маршрутизаторам — в итоге получается огромная «сеть сетей». Так данные, например, от твоего браузера доходят до нужного сервера, и обратно.

**Что такое IP-адрес и доменное имя?**

Каждому устройству в Интернете назначается уникальный IP-адрес, например 192.0.2.172. Но такие адреса неудобны для запоминания, поэтому придумали доменные имена, вроде google.com, которые проще в использовании. Специальная система DNS (Domain Name System) помогает «перевести» имя в IP-адрес, чтобы браузер знал, куда отправлять запрос.

**Интернет и Веб — это не одно и то же**

Важно понимать разницу: **Интернет** — это инфраструктура, а **Веб (World Wide Web)** — это один из сервисов, работающий поверх этой инфраструктуры. Он состоит из веб-серверов, браузеров, сайтов и протокола HTTP. То есть, заходя на сайт, ты используешь Веб, а не весь Интернет.

**Что такое Интранет и Экстранет?**

- **Интранет** — это частная сеть организации. Например, внутренняя документация, общие диски, таск-трекеры, которые доступны только сотрудникам.
- **Экстранет** — похож на Интранет, но с доступом для внешних партнёров: клиентов, подрядчиков, заказчиков. Обычно используется для обмена файлами, документацией или совместной работы.

Обе эти сети используют ту же самую инфраструктуру, что и Интернет, но ограничивают доступ только для авторизованных пользователей.

**Зачем это всё знать тестировщику?**

- Чтобы понимать, **почему может не открываться сайт**: проблема с DNS, IP, маршрутизатором или даже настройками прокси.
- Чтобы **читать DevTools вкладку Network** и понимать, откуда и куда уходит запрос.
- Чтобы отличать **баг на клиенте от бага на сервере**.
- Чтобы уверенно работать с кэшем, куками, прокси и даже тестировать API напрямую.

Очень подробно о том, как работает интернет, можно прочитать здесь в самом видеоуроке. В конспекте я сконцентрируюсь только на основных сущностях.

## MAC-адрес

**MAC-адрес —** уникальный идентификатор, присваиваемый каждой единице активного оборудования или некоторым их интерфейсам в компьютерных сетях Ethernet.

По сути это номер сетевой карты, аппаратный номер оборудования. Если у устройства есть выход в интернет значит в нем обязательно есть сетевая карта.

MAC-адрес состоит из шести групп по два символа, разделённых двоеточиями, например, 00:1B:44:11:3A:B7.

## IP-адрес

**IP-адрес (IP - Internet Protocol) -** уникальный адрес устройства в сети.

Есть две версии: IPv4 и IPv6

Адреса IPv4 записываются в виде четырёх чисел, разделённых точками, например: 192.168.2.10. Это 32-битный адрес.

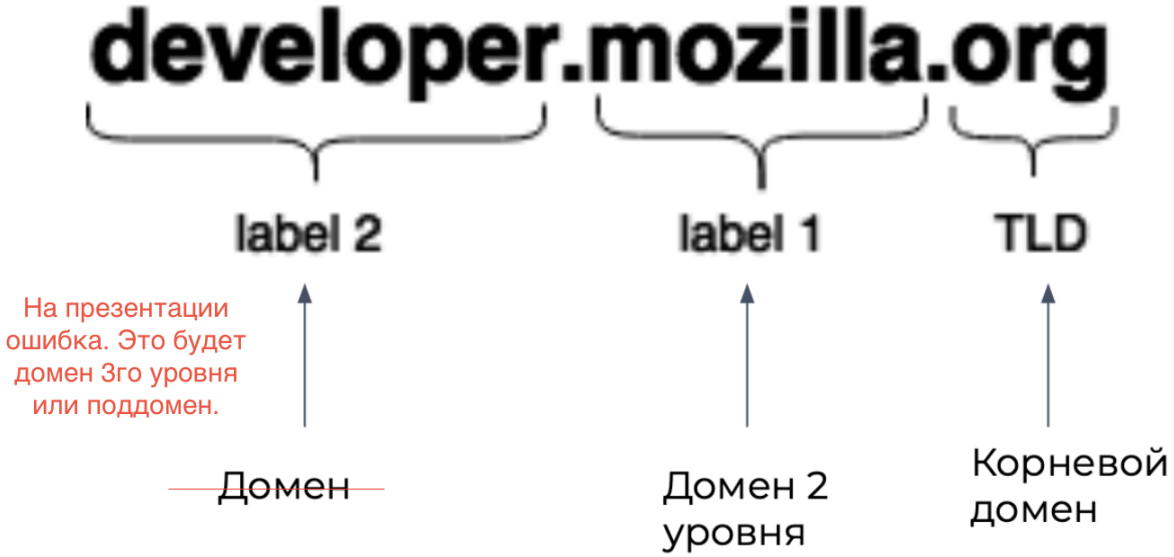
Ввиду ограниченного количества IPv4 в 1999 году была представлена 128-битная версия адреса IPv6. Например, 3ffe: 1900: fe21: 4545: 0000: 0000: 0000: 0000.

Также разделяют статические и динамические адреса.

*Статические адреса* назначаются одному устройству и не меняются.

*Динамические адреса* будут меняться при каждом новом подключении к сети.

IP-адреса имеют псевдонимы – доменные имена. Именно их вы видите в браузере.

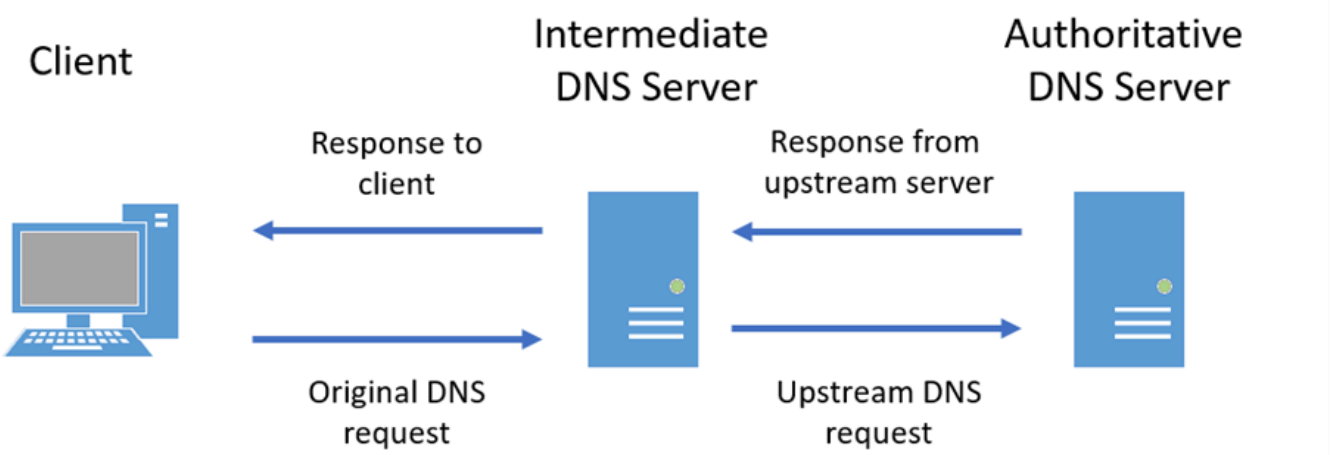



## DNS-сервер (Domain Name System)

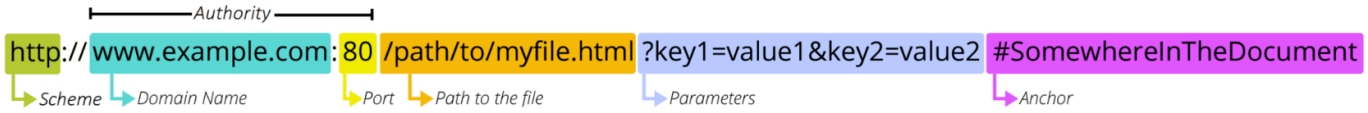
**DNS** — это система, которая позволяет браузеру найти запрошенный пользователем сайт по имени домена.

**Работу DNS-сервера можно описать так:**

1. Ввод адреса целевого сайта в адресную строку в браузере
2. Обращение к DNS-кэшу (временное хранилище) на вашем компьютере или у провайдера: знает ли он какому IP соответствует домен. Если такое имя есть, то вы переходите на целевой ресурс.
3. Если в кэше этой информации нет, то компьютер обращается к DNS-серверу. После чего вы получите соответствующий IP-адрес. Существует несколько уровней DNS-серверов, про которые вы можете почитать здесь




**URL (Uniform Resource Locator)** - адрес, который выдан уникальному ресурсу в интернете.




1. **Протокол:** HTTP, HTTPs, mailto, FTP
2. **Доменное имя:** см. выше
3. **Порт:** определяет доступ к ресурсам на сервере и представляет собой число. Аналогичным образом работают порты на вашем компьютере, которые выделяются под конкретное приложение.
4. **Путь:** адрес ресурса на сервере.

5. **Параметры:** всегда стоят после знака ?. Используются для дополнительных команд. На практике вы будете сталкиваться с ними при фильтрации, например, товаров в интернет-магазине.
6. **Якорь:** всегда идет после знака #. Переводит вас на определенную часть того же ресурса. Например, часть статьи или определенная минута видео.

**Модель OSI и TCP/IP** представляет собой набор протоколов, которые служат для обмена информацией между компьютерами в локальной сети и интернет. Данные модели описывают работу сетевых устройств

## Модель OSI

| Модель | | | | | |
|---|---|---|---|---|---|
| Уровень (layer) | | Тип данных (PDU[15]) | Функции | Примеры | Оборудование |
| Host layers | 7. Прикладной (application) | Данные | Доступ к сетевым службам | HTTP, FTP, POP3, SMTP, WebSocket | Хосты (клиенты сети), Межсетевой экран |
| | 6. Представления (presentation) | | Представление и шифрование данных | ASCII, EBCDIC, JPEG, MIDI | |
| | 5. Сеансовый (session) | | Управление сеансом связи | RPC, PAP, L2TP, gRPC | |
| | 4. Транспортный (transport) | Сегменты (segment) / Датаграммы (datagram) | Прямая связь между конечными пунктами и надёжность | TCP, UDP, SCTP, Порты | |
| Media[16] layers | 3. Сетевой (network) | Пакеты (packet) | Определение маршрута и логическая адресация | IPv4, IPv6, IPsec, AppleTalk, ICMP | Маршрутизатор, Сетевой шлюз, Межсетевой экран |
| | 2. Канальный (data link) | Биты (bit)/ Кадры (frame) | Физическая адресация | PPP, IEEE 802.22, Ethernet, DSL, ARP, сетевая карта. | Сетевой мост, Коммутатор, точка доступа |
| | 1. Физический (physical) | Биты (bit) | Работа со средой передачи, сигналами и двоичными данными | USB, RJ («витая пара», коаксиальный, оптоволоконный), радиоканал | Концентратор, Повторитель (сетевое оборудование) |

Если ты когда-нибудь слышал про модель OSI, но так и не понял, зачем она нужна — ты не один. Это не модель, с которой ты будешь работать ежедневно, но она поможет понять, *как именно данные проходят по сети от одного устройства к другому*, и почему, например, баг в одном уровне может не касаться другого.

Модель OSI делит сетевое взаимодействие на 7 уровней, от физического до прикладного. Зачем так сложно? Чтобы разделить ответственность и

упростить диагностику проблем. Если условно “лампочка не горит” — не нужно разбирать всю электростанцию, достаточно понять, какой “уровень” отвечает за сбой.

Вот как это применимо для новичка:

- **Физический уровень** — это кабели, разъёмы, Wi-Fi-сигналы. Если у пользователя не горит индикатор на роутере — проблема может быть здесь. Это то, что буквально “несёт” сигнал.
- **Канальный уровень** — отвечает за передачу кадров (например, Ethernet), проверяет, дошли ли данные без ошибок. Если у вас не работает локальная сеть, но всё подключено, проверяй этот уровень. Он же управляет MAC-адресами.
- **Сетевой уровень** — логическая маршрутизация: IP-адреса, маршрутизаторы, ICMP-пинги. Например, если ping до сайта не проходит — проверяй этот уровень.
- **Транспортный уровень** — надёжная доставка данных (например, TCP) или быстрая, но без гарантий (например, UDP). Если данные “пропадают” или приходят не в том порядке — смотри сюда.
- **Сеансовый уровень** — управляет установкой и завершением сессий. Если соединение постоянно обрывается — возможна причина здесь.
- **Уровень представления** — шифрование, кодировка. Если сайт не открывается из-за проблем с сертификатом или кодировкой, ищи здесь.
- **Прикладной уровень** — то, с чем ты напрямую взаимодействуешь: HTTP, FTP, SMTP. Если веб-приложение не работает, скорее всего, баг именно тут.

Понимание модели OSI помогает *не гадать, где проблема*, а системно сужать область поиска. В работе тестировщика это особенно полезно при проверке API, отладке сетевых проблем и работе с DevTools или Postman.

*Работа тестировщика концентрируется на прикладном уровне, а именно протокол HTTP, о котором мы поговорим дальше.*

### Модель TCP/IP

Перед тобой — распределение протоколов по уровням модели TCP/IP. Эта модель — не альтернатива OSI, а её практический аналог, которым в реальности пользуются при работе с сетями. Особенно в инженерной и прикладной практике.

**Распределение протоколов по уровням модели TCP/IP**

| | |
|---|---|
| **Прикладной** (Application Layer) | напр., HTTP, RTSP, FTP, DNS |
| **Транспортный** (Transport Layer) | напр., TCP, UDP, SCTP, DCCP<br>*(RIP, протоколы маршрутизации, подобные OSPF, что работают поверх IP, являются частью сетевого уровня)* |
| **Сетевой (Межсетевой)** (Network Layer) | Для TCP/IP это IP<br>*(вспомогательные протоколы, вроде ICMP и IGMP, работают поверх IP, но тоже относятся к сетевому уровню; протокол ARP является самостоятельным вспомогательным протоколом, работающим поверх канального уровня)* |
| **Уровень сетевого доступа (Канальный)** (Link Layer) | Ethernet, IEEE 802.11, WLAN, SLIP, Token Ring, ATM и MPLS, физическая среда и принципы кодирования информации, T1, E1 |

## Чем отличается TCP/IP от OSI?

Модель OSI состоит из 7 уровней и часто используется как обучающий стандарт. Она помогает понять, как данные обрабатываются и передаются в теории. В реальной разработке, при работе с API, DevTools, сетевыми утилитами и т.п., ты будешь сталкиваться с моделью TCP/IP. Она проще — в ней 4 уровня:

1. **Прикладной уровень (Application Layer)**

   Всё, что связано с интернет-сервисами, с чем ты напрямую взаимодействуешь: HTTP, FTP, DNS, SMTP и т.п.

   → Пример: ты отправляешь запрос через Postman — это HTTP-запрос, который обрабатывается на прикладном уровне.

2. **Транспортный уровень (Transport Layer)**

   Отвечает за надёжную (TCP) или быструю (UDP) доставку данных между точками.

   → Пример: TCP обеспечивает, чтобы весь JSON от API дошёл целиком. UDP — как в потоковом видео, где потеря нескольких пакетов некритична.

3. **Сетевой уровень (Network Layer)**

Обеспечивает маршрутизацию — как дойти от одного компьютера к другому в интернете. Протокол IP — главный здесь.

→ Пример: когда ты делаешь ping, работает ICMP на сетевом уровне.

4. **Канальный уровень (Link Layer)**

   Это всё, что касается локальной передачи данных: Ethernet, Wi-Fi, MAC-адреса.

   → Пример: твой Wi-Fi адаптер или сетевая карта обрабатывают канальный уровень.

**Какая модель используется чаще?**

В реальной жизни — **модель TCP/IP**, потому что именно она лежит в основе всех современных сетей и протоколов. Модель OSI используется скорее как «учебный стандарт» — чтобы тебе было проще разобраться с тем, как устроена сеть по слоям.

Важно: протоколы в TCP/IP часто охватывают сразу несколько уровней OSI. Поэтому, когда ты работаешь с API или браузером, не ищи, к какому точно уровню OSI это относится — просто знай, что HTTP, например, работает на прикладном уровне TCP/IP и использует TCP/IP-стек для доставки.

**Как применять это знание?**

На собеседованиях часто спрашивают, чем отличается TCP от UDP, как работает DNS, почему API иногда не отвечает. Все эти вопросы завязаны на понимание уровней и протоколов TCP/IP. Используй эту таблицу как шпаргалку, чтобы лучше понимать, что происходит «под капотом» при работе с сетевыми запросами.

*Также советую запомнить разницу между различными транспортными протоколами, это любят спрашивать на интервью.*

# Транспортные протоколы

| TCP | UDP | QUIC (надстройка над UDP от Google) |
|---|---|---|
| Надежность за счет рукопожатия (handshake) | Ненадежный. Результат не гарантирован | Надежный |
| Упорядоченность | Неупорядоченность | Упорядоченность |
| Тяжелый и медленный | Легкий и быстрый | Легкий и быстрый |
| Электронные сообщения, обмен файлами, загрузка страниц | Звонки в мессенджерах, стриминг, онлайн-игры | HTTP 3 |

Ссылки на первоисточники вы можете найти в презентации к уроку.

### Полезные материалы

1. Презентация
2. Шпаргалка (адаптирована под десктоп) https://rusau.net/web
3. Старая версия урока https://youtu.be/6gZ2OKkKzjw
4. Как устроен интернет? https://developer.mozilla.org/ru/docs/Learn/Common_questions/Web_mechanics/How_does_the_Internet_work
5. Доменное имя https://developer.mozilla.org/ru/docs/Learn/Common_questions/Web_mechanics/What_is_a_domain_name
6. DNS https://learn.microsoft.com/ru-ru/azure/sentinel/normalization-schema-dns
7. Что такое URL? https://developer.mozilla.org/ru/docs/Learn/Common_questions/Web_mechanics/What_is_a_URL
8. Модель OSI
9. Модель TCP/IP
10. Библия QA. Идентификация ресурсов в сети (Identifying resources on the Web) https://vladislaveremeev.gitbook.io/qa_bible/seti-i-okolo-nikh/identifikaciya-resursov-v-seti-identifying-resources-on-the-web

ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЙ

# 30. Клиент-серверная архитектура. Балансировщики

### Клиент-серверная архитектура

**Клиент-серверная архитектура** - вычислительная или сетевая архитектура, в которой задания или сетевая нагрузка распределены между поставщиками услуг, называемыми серверами, и заказчиками услуг, называемыми клиентами.

Фактически клиент и сервер - это программное обеспечение. Обычно эти программы расположены на разных вычислительных машинах и взаимодействуют между собой через вычислительную сеть посредством сетевых протоколов, но они могут быть расположены также и на одной машине. Программы-серверы ожидают от клиентских программ запросы и предоставляют им свои ресурсы в виде: данных (например, загрузка файлов посредством HTTP, FTP, BitTorrent, потоковое мультимедиа или работа с базами данных); сервисных функций (например, работа с электронной почтой, общение посредством систем мгновенного обмена сообщениями или просмотр web-страниц во всемирной паутине).

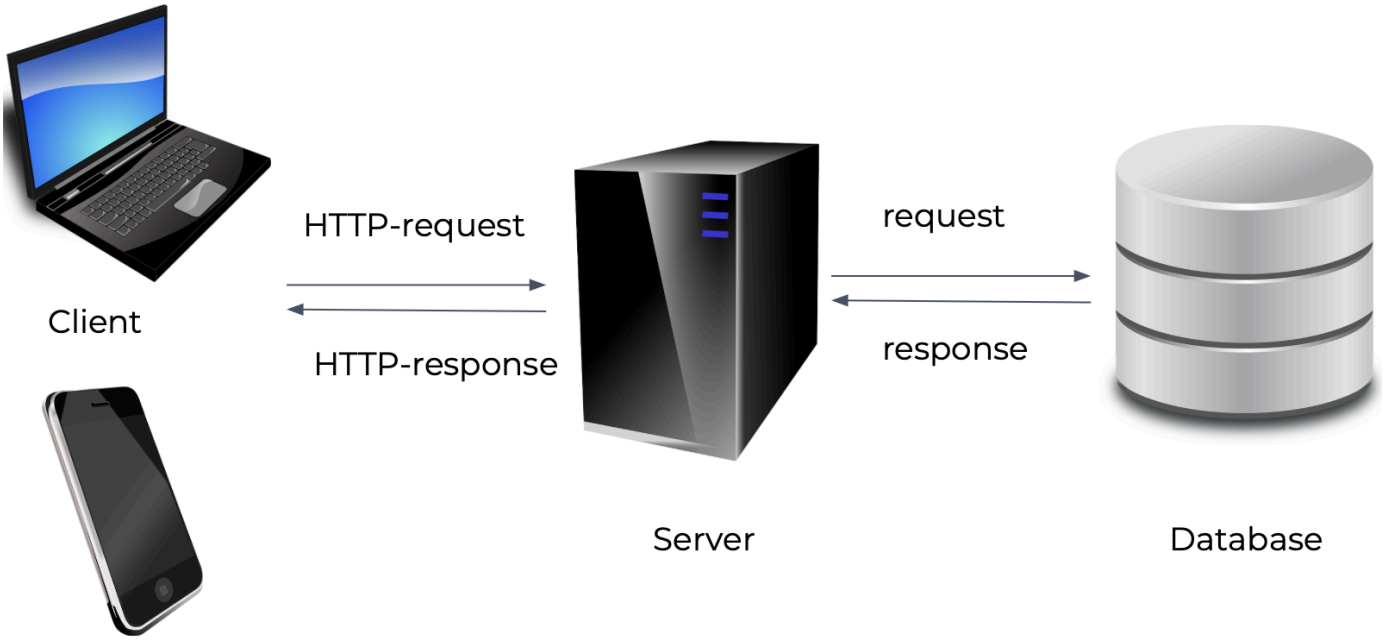
Поскольку одна программа-сервер может выполнять запросы от множества программ-клиентов, ее размещают на специально выделенной вычислительной машине, настроенной особым образом, как правило, совместно с другими программами-серверами, поэтому производительность этой машины должна быть высокой. Из-за особой роли такой машины в сети, специфики ее оборудования и программного обеспечения, её также называют сервером, а машины, выполняющие клиентские программы, соответственно, клиентами.

*Источник: Библия QA*

### Как работает данная модель простыми словами?

1. Клиент (браузер, мобильное приложение) отправляет HTTP-запрос к серверу. *Пользователь ввел запрос в поисковую строку интернет-магазина*
2. Сервер обрабатывает запрос и присылает HTTP-ответ клиенту. *Сервер магазина обработал запрос и обратился к базе данных в поиске искомого товара (см. шаг 3). После этого сервер вернул массив карточек товаров в ответе и пользователь увидел их на клиенте (в браузере).*

3. Если необходимо получить данные, которые сохраняются в базе данных, то сервер обращается к базе и получает ответ от нее. *Сервер базы данных обработал запрос и выдал серверу идентификаторы товаров, которые подходят поисковому запросу.*




Такая модель получила название трехуровневой клиент-серверной архитектуры. В некоторых источниках ее могут называть двухуровневой.

### Тонкий и толстый клиент

В зависимости от того, кто берет на себя основную нагрузку в модели клиент-сервер, выделяют тонкий и толстый клиент.

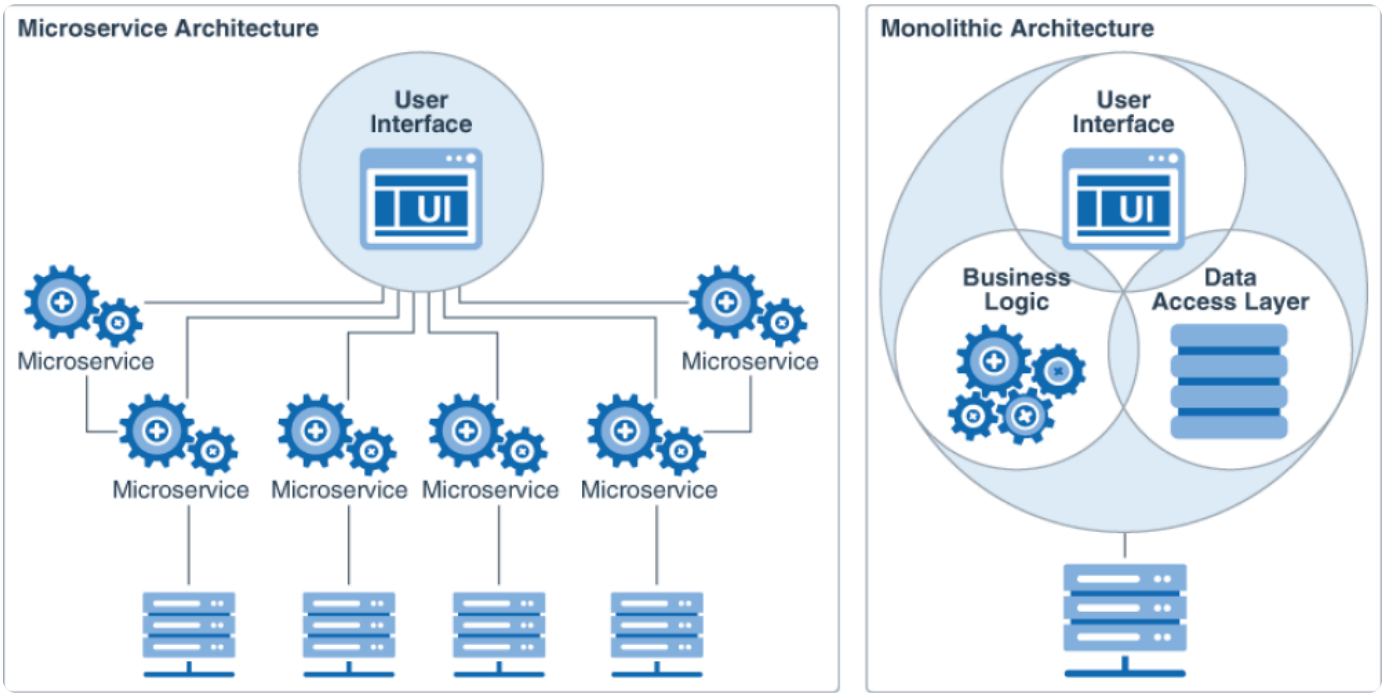
| Тонкий клиент | Толстый клиент |
|---|---|
| Браузер, мобильное веб приложение, тонкий клиент 1С | ПК, толстый клиент 1С, нативное мобильное приложение |
| Задачи по обработке информации переносятся на сервер | Задачи по обработке информации обрабатываются на клиенте |
| Основная бизнес-логика и обработка данных происходит на сервере | Сервер служит в качестве хранилища данных |
| Работа только онлайн | Работа офлайн |

## Монолиты и микросервисы

Многие начинающие тестировщики слышат термины “монолит” и “микросервисы”, но не до конца понимают, как они влияют на тестирование. Здесь важно не просто выучить определения, а понять, как это отразится на вашей повседневной работе.

**Монолитная архитектура** — это когда всё приложение — от интерфейса до базы данных — собрано в один большой блок. Такой подход проще для старта, но может стать громоздким, когда проект растёт. Любое изменение в коде требует пересборки и повторного тестирования всей системы. С другой стороны, отладка и настройка проще — всё в одном месте

**Микросервисная архитектура** — это, по сути, набор независимых мини-приложений, каждое из которых делает свою работу. Это гибко и масштабируемо, но увеличивает сложность тестирования. Вместо одной системы у нас десятки сервисов, и нужно следить, чтобы все они правильно “разговаривали” между собой.




Источник картинки

Что это значит для тестировщика:

- **При монолите** ты чаще работаешь с целым приложением, тестируя его как одно целое. Баги легче воспроизвести и отследить.

- **При микросервисах** ты сталкиваешься с множеством команд, разными API, логами из разных мест и постоянной необходимостью проверять, не нарушилось ли что-то при интеграции.

Особое внимание нужно уделять:

- **Интеграционному тестированию** — чтобы убедиться, что микросервисы взаимодействуют как положено.
- **Нагрузочному тестированию** — чтобы понять, не “сыпется” ли вся архитектура при большом количестве запросов.
- **Контрактному тестированию** — чтобы убедиться, что сервисы не нарушают договорённости (контракты) о формате передаваемых данных.

## Примеры монолитной архитектуры

### Пример 1: Сайт портфолио фрилансера

- Всё — от формы обратной связи до административной панели — собрано в одном приложении.
- Один сервер, одна база данных, один deploy.
- При обновлении кода на сервер загружается сразу всё.

**Что делает тестировщик:** проверяет UI, отправку форм, базу, авторизацию, но всё — в пределах одной системы, без взаимодействия между частями через API.

### Пример 2: Внутренний CRM для небольшой компании

- Сотрудники используют веб-приложение для управления заказами, звонками и клиентами.
- Все функции находятся внутри одного кода — одна команда разработчиков.
- При ошибке в одной части (например, фильтр заказов) может сломаться весь интерфейс.

**Что делает тестировщик:** тестирует сквозные пользовательские сценарии, проверяет данные в базе, делает Smoke/Regression – всё в пределах одной

системы.

### Примеры микросервисной архитектуры

**Пример 1: Онлайн-магазин (например, Wildberries, Ozon)**

- Один сервис отвечает за каталог товаров.
- Отдельный сервис — за корзину и оформление заказа.
- Ещё один — за оплату и взаимодействие с платёжной системой.
- Все общаются между собой через REST API или сообщения (например, Kafka).

**Что делает тестировщик:** проверяет работу каждого сервиса по отдельности (юнит и интеграционные тесты), потом — взаимодействие между ними: заказ → оплата → уведомление → склад.

**Пример 2: Финтех-платформа**

- Есть микросервис управления пользователями (регистрация, вход).
- Сервис верификации личности (паспорт, биометрия).
- Сервис транзакций (переводы, история).
- Сервис оповещений (email, push, SMS).

**Что делает тестировщик:**

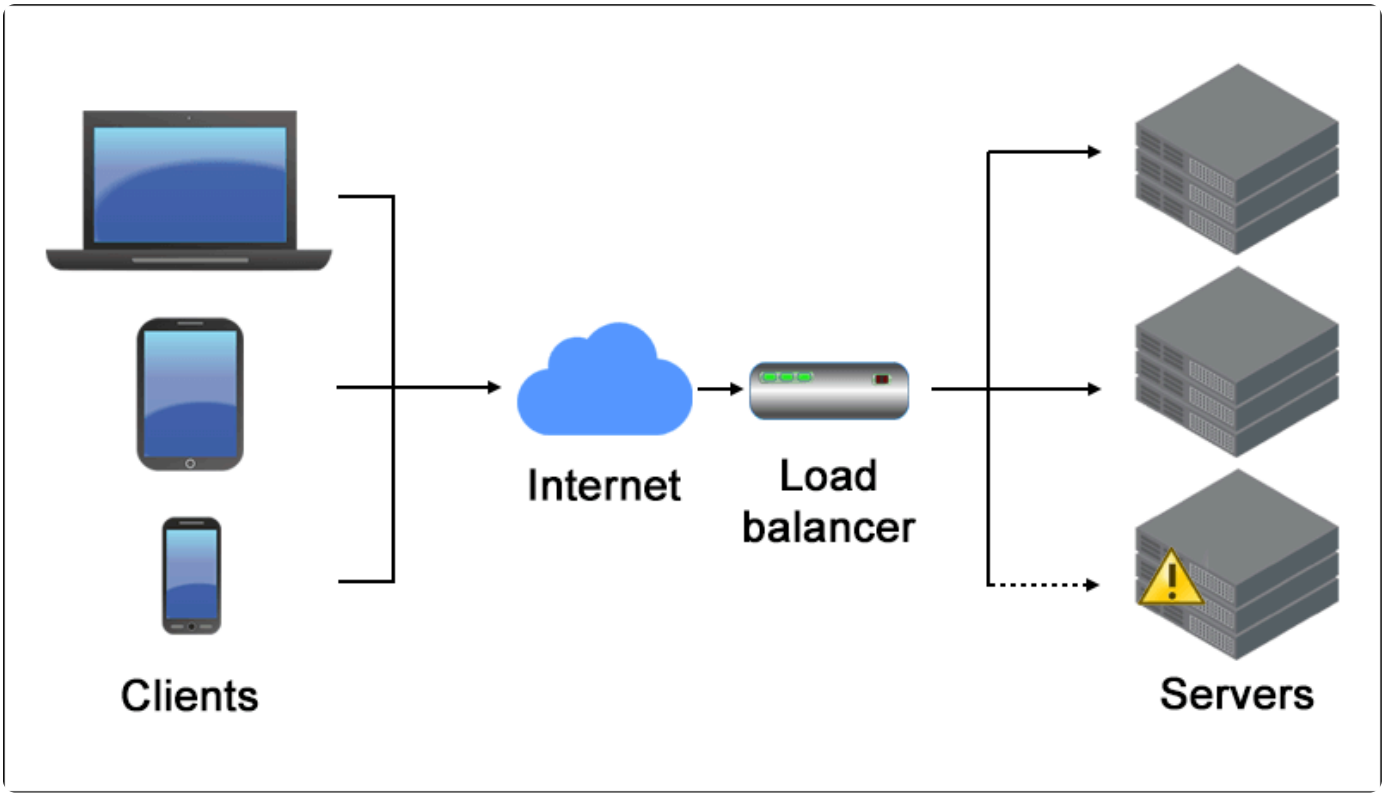
- Работает с mock’ами (заглушками), чтобы проверять сервисы изолированно.
- Пишет Postman-коллекции или автотесты на взаимодействие API.
- Проверяет логи, согласованность данных между сервисами, отказоустойчивость одного компонента при падении другого.

### Балансировщики нагрузки

### (информация для общего развития, редко спрашивают на собеседовании)

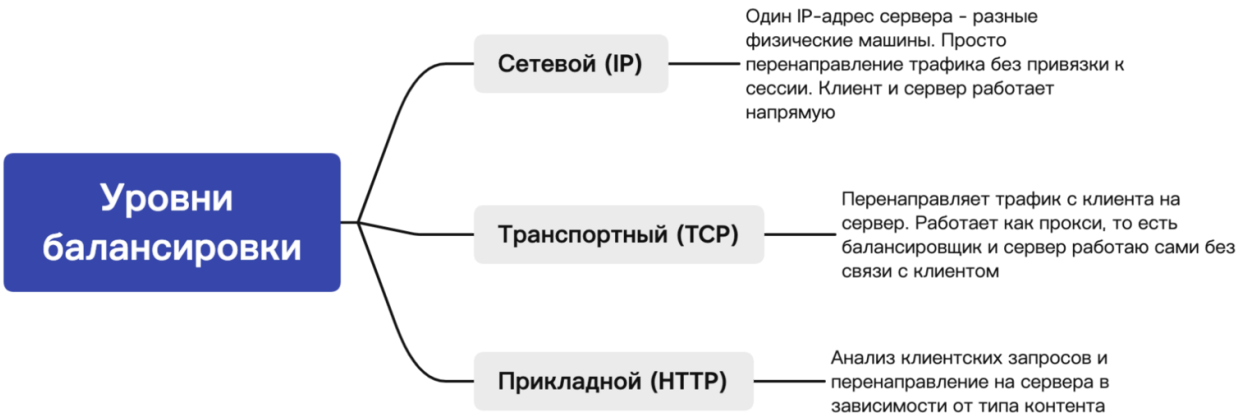
Балансировщик нагрузки — это устройство, которое распределяет трафик между несколькими серверами, чтобы обеспечить оптимальную производительность системы и избежать перегрузки отдельных серверов.

**Схема работы**




На примере, один из серверов недоступен и балансировщик перенаправляет нагрузку на другие.

Различают несколько **уровней балансировки:**



- Тестирование балансировщика нагрузки включает проверку его способности правильно распределять трафик, обеспечивать доступность и отказоустойчивость сервисов.
- Необходимо убедиться, что в случае сбоя одного из серверов, балансировщик корректно перенаправляет трафик на работающие серверы.
- Также важно тестировать, как балансировщик справляется с различными объемами трафика, что позволяет оценить его масштабируемость и производительность.

**Полезные материалы**

1. Презентация
2. Старая версия https://youtu.be/00z-6hyIvG0
3. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1ALqmpNvOJZF7CPFAVBodYGpHJXQrWYVn
4. Что такое сервер приложения? https://habr.com/ru/post/646651/
5. Библия QA. Клиент - серверная архитектура (Client-Server Architecture) https://vladislaveremeev.gitbook.io/qa_bible/seti-i-okolo-nikh/klient-servernaya-arkhitektura-client-server-architecture
6. Чек-лист тестирования веб-приложений https://habr.com/ru/post/542422/
7. Очень хороший чек-лист по тестированию веб-приложений https://habr.com/ru/company/space307/blog/668974/

ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЙ

# 31. HTTP-протокол. HTTP-методы. Запросы и ответы. Коды ответов

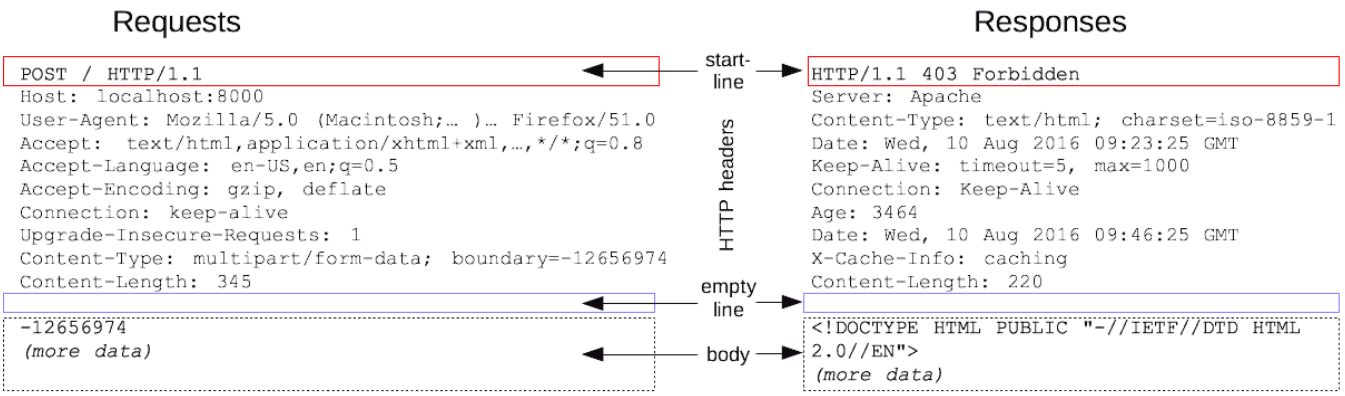
**HTTP (HyperText Transfer Protocol)** — протокол прикладного уровня передачи данных, изначально — в виде гипертекстовых документов в формате HTML, в настоящее время используется для передачи произвольных данных. [Источник]

Когда клиент хочет взаимодействовать с сервером, являющимся конечным сервером или промежуточным прокси, он выполняет следующие шаги:

1. Открытие TCP-соединения: служит для отправки запросов и получения ответов
2. Отправка HTTP-сообщения (запроса и ответа)
3. Чтение ответа с сервера
4. Закрытие TCP-соединения или отправка следующих запросов в нем же

Это так называемый **HTTP-поток.**

Для общения клиента и сервера в сети используются **HTTP-сообщения: запросы ответы.** Давайте рассмотрим их структуру.




Анализировать запросы и ответы можно через специальные инструменты разработчика в браузере, которые вы можете вызвать, нажав клавишу F12 в браузере и перейдя во вкладку Network. Подробнее о работе с ними поговорим в следующих уроках.

Вне зависимости от того, ввели ли вы сами необходимую информацию в

адресную строку или запустили внутренний процесс на самой странице (например, применили фильтр) отправляется запрос к серверу, который выглядит так.

**Request – запрос от клиента**

1. **Стартовая строка (start line)** содержит информацию о методе, которое говорит нам о совершаемом действии. Например, GET - получение информации от сервера, POST - загрузка информации на сервер.
2. **Цель запроса**, чаще всего это URL.
3. **Версия HTTP-протокола**
4. **Headers (заголовки запроса),** которые несут в себе служебную информацию. Например, информация о языке страницы, кодировке, браузере.
5. **Пустая строка (empty line)**
6. **Тело запроса (body)**, в которых содержится полезная нагрузка для отправки на сервер. В методе GET тело игнорируется.

**Response – ответ от сервера**

1. **Стартовая строка** содержит: версию протокола, статус-код об успешности запроса, пояснение для кода. Если все прошло хорошо и запрос обработан, вы увидите код 200. О других группах мы поговорим в следующих шагах.
2. **Headers (заголовки ответа),** которые несут в себе служебную информацию. Например, дополнительную информацию о сервере и ответе.
3. **Тело ответа (body)**, в которых содержится результат отправки полезной нагрузки на сервер.

Подробнее можно прочитать здесь

**HTTP-методы**

**HTTP-метод -** глагол или существительное, описывающее требуемое действие.

Примеры методов, которые применяются:

1. GET - поиск данных
2. POST - загрузка данных
3. PUT - полное обновление данных

4. PATCH - частичное обновление данных
5. DELETE - удаление данных
6. OPTIONS - описание параметров соединения
7. и т.д.

Подробнее про каждый метод можно прочитать здесь и мы будем работать с ними в заданиях и модуле, посвященных Postman.

**CRUD** — акроним, обозначающий четыре базовые функции, используемые при работе с базами данных: создание, чтение, модификация, удаление [Википедия]. Все методы можно разделить отдельные группы:

**C** - POST, PUT
**R** - GET
**U** - PUT, PATCH
**D** - DELETE

Также данные операции можно переложить на права пользователя (permission model).

### Отличия GET и POST

GET и POST являются самыми популярными методами, поэтому стоит разобраться в их отличиях.

| GET | POST |
|---|---|
| Получение данных с сайта | Отправка данных на сайт |
| Отправляется через URL | В теле HTTP-запроса |
| Тела запроса нет, но его можно добавить. В таком случае оно просто игнорируется | Есть тело запроса |
| Ограничено длиной адресной строки (до 2048 символов) | Не ограничено (могут быть ограничения сервера) |
| Данные видны всем (в адресной строке, в истории) | Данные можно увидеть только с помощью программ |
| Страницу можно добавить в Закладки | Нельзя |
| Страница может быть кэширована и проиндексирована | Нельзя |

*Источник: [https://guruweba.com/html/metody-get-i-post-ispolzovanie-i-otlichiya/](https://guruweba.com/html/metody-get-i-post-ispolzovanie-i-otlichiya/)*

## Свойство идемпотентности (в презентации есть ошибки, ниже правильный вариант)

## Коды ответа (response status codes)

**Код ответа (состояния)** показывает, был ли успешно выполнен определённый запрос. Коды группируются в отдельные классы:

| 100 Informational | 200 Successful | 300 Redirection | 400 Client errors | 500 Server error |
|---|---|---|---|---|
| Устарели, запоминать примеры не надо | 200 OK<br>201 Created | 301 Moved Permanently<br>304 Not Modified | 400 Bad Request<br>401 Unauthorized<br>403 Forbidden<br>404 Not Found<br>418 I'm a teapot | 500 Internal Server Error<br>501 Not Implemented<br>502 Bad Gateway<br>503 Service Unavailable<br>504 Gateway Timeout |

Наиболее важные коды ответа для тестировщика: ошибки клиента и сервера. Всегда анализируйте ответы на наличие этих кодов.

Рекомендую разобраться в разнице таких методов, как 401 и 403, 502 и 504. Также обратите внимание на 418, он шуточный, но иногда могут спросить ради прикола.

Знать все коды не надо (их очень много!), но добавленные в таблицу рекомендую запомнить и разобраться в том, что они обозначают. Ознакомиться с ними можно здесь

**Версии HTTP-протокола**

| | HTTP 1.1 | HTTP 2.0 | HTTP 3.0 |
|---|---|---|---|
| **HTTP-поток** | TCP | TCP | QUIC |
| **Параллельность отправки запросов** | Последовательные запросы в 1 TCP | Мультиплексирование | Мультиплексирование |
| **Headers compression** | Нет | Есть | Есть |
| **Приоритет запросов** | Нет | Да | Да |
| **Формат** | Text | Binary | Binary |
| **Server Push** | Нет | Да | Да |

Подробнее

**HTTP и HTTPs**

1. **HTTPs** — это расширение для протокола HTTP, которое делает его безопасным
2. **HTTPs = HTTP + TLS** (криптографический протокол)

**SSL-сертификат** - цифровое удостоверения сайта

Как и для протокола HTTP у HTTPs есть свои версии:

1. HTTPS 1.1 = HTTP + TLS + TCP
2. HTTPS 2 = HTTP + TLS + TCP
3. HTTPS 3 = HTTP + (TLS + QUIC)

Советую дополнительно изучить этот ресурс.

## Полезные материалы

1. Презентация
2. Старая версия урока https://youtu.be/OA3jExxIkew
3. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1ShQHFxCAkJ5XQGkd39TddeCohMGP6Uu4
4. Из чего состоит HTTP-протокол? https://developer.mozilla.org/ru/docs/Web/HTTP/Overview
5. Про HTTPs в комиксах https://howhttps.works/
6. Библия QA. HTTP https://vladislaveremeev.gitbook.io/qa_bible/seti-i-okolo-nikh/http
7. Библия QA. Эталонные модели OSI и TCP/IP https://vladislaveremeev.gitbook.io/qa_bible/seti-i-okolo-nikh/etalonnye-modeli-osi-i-tcp-ip

ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЙ

# 32. Кэш, куки, хранилища. Веб-сайт, приложения, сервис

**Веб-страница, веб-сайт, веб-приложение, веб-сервис, веб-сервер**

Существует достаточно много сущностей, которые содержат в себе приставку веб. Советую разобраться в них, чтобы в дальнейшем не путаться и говорить на одном языке с командой.

**Веб-страница (web page)** - документ, который может быть отображён веб-браузерами, такими как: Firefox, Google Chrome, Microsoft Internet Explorer / Edge или Safari от Apple. [Источник]

*Это страница, на которой вы находитесь прямо сейчас. У нее есть уникальный адрес URL и она написана на языке HTML.*

**Веб-сайт (website)** - коллекция веб-страниц под одним доменным именем, связанных между собой какими-либо способами. [Источник]

*Пример, страница-лендинг по продаже курса, которая несет на себе только информационную функцию и не выполняет сложную бизнес-логику.*

**Веб-приложение (web application)** - программное обеспечение, которое запускается в веб-браузере. [Источник]

*Пример, онлайн-банкинг, в котором вы можете осуществлять сложные операции.*

**Веб-сервер (web server) -** компьютер, предоставляющий компьютерное и программное обеспечение, необходимое для функционирования веб-сайта. [Источник]

*Подробно поговорили о нем в предыдущих шагах урока.*

**Веб-сервис (web service) -** реализация абсолютно четких интерфейсов обмена данными между различными приложениями, которые написаны не только на разных языках, но и распределены на разных узлах сети. [Источник]

*Это реализация API (интерфейс программирования приложения), с помощью которого два и более веб-приложения могут взаимодействовать между собой. Подробнее эту тему будем разбирать в модуле "Тестирование API".*

**Кэш (cache)** — промежуточный буфер с быстрым доступом к нему, содержащий информацию, которая может быть запрошена с наибольшей вероятностью. [Источник]

*Пример: вы используете ресурс с большим количеством картинок. Чтобы каждый раз не загружать данные с сервера, картинки загружаются на ваш компьютер и каждый раз при обращении к ресурсу загружаются с него.*

Различают серверный и клиентский кэш.

**Клиентский кэш**

Когда вы заходите на сайт, ваш браузер (клиент) может сохранить копии определенных данных, таких как изображения, стили страницы и скрипты, на вашем компьютере или мобильном устройстве. В следующий раз, когда вы посетите тот же сайт, браузер сначала проверит, есть ли эти данные в кэше, и если да, то использует их оттуда, вместо того чтобы снова загружать их с сервера. Это сокращает время загрузки страницы и уменьшает количество данных, передаваемых через сеть.

**Серверный кэш**

Серверный кэш работает похожим образом, но хранит данные на стороне сервера. Когда сервер получает запрос на данные, которые он уже обрабатывал недавно, он может быстро предоставить их из своего кэша, не выполняя затратных операций, таких как обращение к базе данных или выполнение сложных вычислений. Это снижает нагрузку на сервер и ускоряет отклик на запросы клиентов.

**При тестировании кэширования** мы должны проверить, что данные кэшируются правильно и не устаревают. Например, если на сайте обновили изображение, кэш должен это учитывать и предоставить новую версию. Также важно убедиться, что кэш не сохраняет чувствительные данные, которые не должны быть легкодоступны.

Это важная тема, поскольку правильное кэширование может значительно улучшить производительность приложения и уменьшить его нагрузку, в то время как неправильное может привести к ошибкам и устареванию данных.

**Cookies** - небольшой фрагмент данных, который сервер отправляет браузеру пользователя, который хранится на компьютере пользователя. [Источник]

**Куки часто используются для:**

1. Управления сеансом (логины, корзины для виртуальных покупок)
2. Персонализации (пользовательские предпочтения)
3. Трекинга (отслеживания поведения пользователей)

*Примеры: вы добавили товар в корзину, информация об этом сохранилась в куки. Даже если вы перезагрузите страницу, то данные в корзине не изменятся и будут содержать добавленные товары.*

*Когда вы входите на сайт, куки помогают сайту "помнить" вас при следующих посещениях с помощью специального токена, который в них сохраняется.*

**Виды куки:**

- Сессионные куки (session cookie) - такие cookie удаляются при закрытии клиента, то есть существуют только на протяжении текущего сеанса.
- Постоянные куки (permanent cookies) удаляются не с закрытием клиента, а при наступлении определённой даты или после определённого интервала времени.

Тестировщикам необходимо проверять, что куки правильно устанавливаются и обновляются, и что они не хранят чувствительные данные в незашифрованном виде.

**LocalStorage** — это вид веб-хранилища, который позволяет сайтам сохранять данные в браузере пользователя на неопределенный срок. Эти данные не отправляются на сервер при каждом запросе, как куки, и остаются доступными после перезапуска браузера. LocalStorage часто используется для сохранения настроек пользователя или данных форм. Тестировщики должны проверять, что данные в LocalStorage не потеряются и не станут недоступными после обновлений сайта.

**SessionStorage** похож на LocalStorage, но с одним ключевым отличием: данные сохраняются только в течение сессии браузера, то есть они исчезают при закрытии вкладки или окна. Это полезно для хранения информации, которая должна оставаться постоянной во время одного визита на сайт, но не должна сохраняться между сессиями. Тестировщикам важно убедиться, что SessionStorage четко отслеживает сессии и правильно очищается.

Подробнее о механизме работы каждого хранилища я рассказываю в видео-уроке.

### Полезные материалы

1. Презентация
2. Различные сущности в Web https://developer.mozilla.org/ru/docs/Learn/Common_questions/Web_mechanics/Pages_sites_servers_and_search_engines
3. Кэширование https://developer.mozilla.org/ru/docs/Web/HTTP/Caching
4. Cookies https://developer.mozilla.org/ru/docs/Web/HTTP/Cookies
5. Web Storage API https://developer.mozilla.org/ru/docs/Web/API/Web_Storage_API/Using_the_Web_Storage_API

ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЙ

# 33. Основы HTML, CSS и JS (версия 2026)

### Основы фронтенда для тестировщика: HTML, CSS и JavaScript

Для успешного тестирования графического интерфейса (GUI) QA-инженеру необходимо понимать архитектуру клиентской части веб-приложения. Мы не будем учиться писать код с нуля, но разберем, как читать его, находить уникальные локаторы для автотестов и точно локализовать дефекты.

Современный фронтенд строится на взаимодействии трех технологий: HTML (структура), CSS (визуальное представление) и JavaScript (логика и поведение).

### HTML (HyperText Markup Language) – Структура сайта

HTML – это язык гипертекстовой разметки. Он определяет иерархию элементов на странице (DOM-дерево) и их расположение (верстку). Сохраняется в файле `index.html`.

- **Где тестировать:** В DevTools на вкладке **Elements**. *Нажмите в браузере F12, чтобы ее увидеть. Подробнее в следующем уроке.*

Для тестировщика HTML представляет интерес с точки зрения семантики и поиска локаторов.

**Ключевые теги:**

- `<html>`: Это корень веб-страницы, в который вкладываются все остальные элементы.
- `<head>`: Здесь располагается информация, не отображаемая напрямую на странице – такая как заголовок веб-страницы `<title>` и ссылки на стилевые файлы.
- `<body>`: Всё, что пользователь видит на странице, находится здесь – тексты, изображения, ссылки.
- `<p>` : этот тег используется для разметки текстовых абзацев на веб-странице.
- `<div>` и `<span>`: Универсальные блочные и строчные контейнеры. Часто используются для группировки элементов.

- <h1> – <h6>: Заголовки. Важно для тестирования SEO (тег <h1> должен быть один на страницу).
- <form>: Форма. Объединяет поля ввода для отправки данных.
- <input>: Поле ввода. Ключевой тег для тестировщика. Важно проверять атрибут type (например, type="password", type="checkbox", type="email"), так как он включает встроенную браузерную валидацию.
- <a>: Ссылка. Атрибут href указывает, куда она ведёт. Например, <a href="https://example.com">Перейти на сайт</a>.
- <img> с атрибутом src: Для вставки изображений. src указывает путь к изображению, например, <img src="image.jpg" alt="Описание">.
- <button>: Интерактивная кнопка для выполнения действий.

**Ключевые атрибуты (Основа для поиска локаторов):**

- id: Уникальный идентификатор элемента. Самый надежный локатор для автоматизации.
- class: Класс элемента, используется для привязки CSS-стилей. Менее надежен для автотестов, так как может часто меняться.
- data-* (например, data-testid или data-qa): Кастомные атрибуты, которые разработчики специально добавляют для тестировщиков. Это "золотой стандарт" в AQA, так как они не меняются при редизайне.
- disabled / readonly: Атрибуты состояния. Тестировщик может удалить их через DevTools (Edit as HTML), чтобы проверить, есть ли валидация на стороне сервера, если клиентскую защиту удалось обойти.

## CSS (Cascading Style Sheets) – Визуальное представление

CSS отвечает за стилизацию HTML-узлов: позиционирование, размеры, цвета, адаптивность (Media Queries). Находится в отдельном файле styles.css или прямо в index.html в теге <styles>.

- **Где тестировать:** В DevTools на вкладке **Elements**, панели **Styles** (исходные правила) и **Computed** (итоговые вычисленные

значения). *Нажмите в браузере F12, чтобы ее увидеть. Подробнее в следующем уроке.*

**Что тестирует QA в CSS:**

- **Box Model (Блочная модель):** Понимание того, как формируется размер элемента (ширина контента + внутренние отступы `padding` + рамка `border` + внешние отступы `margin`). Сверяется с макетами в Figma.
- **Состояния элементов (Pseudo-classes):** Стили, которые применяются при определенных условиях (`:hover` — наведение, `:active` — клик, `:focus` — выделение клавиатурой).
- **Свойства видимости (Частая причина багов):**
  - `display: none;` — элемент полностью удален из потока документа (на него нельзя кликнуть, автотест его не найдет).
  - `visibility: hidden;` — элемент невидим, но физически занимает место на странице.
- **Z-index и позиционирование:** Свойство `z-index` определяет порядок перекрытия элементов по оси Z (глубине). Баг, когда выпадающее меню перекрыто соседним блоком или не нажимается – это проблема с z-index.
- **Переполнение (Overflow):** Свойство, определяющее поведение контента, который не помещается в блок (обрезать, добавить скролл или скрыть). Тестируется путем ввода длинных строк в поля имени или названия.

## JavaScript (JS) – Логика и интерактивность

JavaScript – это язык программирования, который управляет поведением страницы на стороне клиента без необходимости ее полной перезагрузки. В современных веб-приложениях Single Page Applications (SPA), написанных на React, Vue или Angular, весь HTML генерируется именно с помощью JS. Сохраняются в отдельных файлах с расширением `.js` или прямо в `index.html` в теге `<scripts>`.

- **Где тестировать:** Во вкладке **Console** (логи и ошибки), **Network** (сетевые запросы), **Sources** (исходный код и дебаггер) и панели **Event**

**Listeners**. *Нажмите в браузере F12, чтобы ее увидеть. Подробнее в следующем уроке.*

**Что делает JS и как это тестировать:**

1. **Манипуляция DOM-деревом:** JS может "на лету" удалять, добавлять или изменять HTML-теги и CSS-классы.
    - *Кейс:* Вы нажали "Добавить в корзину". JS добавляет счетчик на иконку корзины. Если счетчик не появился, проблема может быть в логике рендеринга JS.
2. **Асинхронные запросы (AJAX / Fetch):** JS отправляет данные на бэкенд и получает ответы в фоновом режиме.
    - *Кейс:* Локализация бага. Если после нажатия кнопки "Оплатить" ничего не происходит, QA открывает консоль (поиск ошибок синтаксиса JS) и вкладку Network (поиск упавших запросов к API).
3. **Обработка событий (Event Handling):** JS "слушает" действия пользователя (клики, свайпы, нажатия клавиш).
    - *Кейс:* Если кнопка не реагирует на нажатие, тестировщик проверяет вкладку `Event Listeners` в Elements, чтобы убедиться, что к кнопке вообще привязан обработчик события `click`.
4. **Клиентская валидация:** Быстрая проверка данных до отправки на сервер (например, проверка сложности пароля).
    - *Кейс:* JS должен блокировать кнопку отправки, если email введен без `@`.
5. **Работа с состоянием (State Management):** JS сохраняет токены, сессии и настройки пользователя в браузере (Local Storage, Cookies). Тестировщик манипулирует этими хранилищами на вкладке Application.

**Вы должны понимать цикл: "Действие пользователя -> Событие JS -> Запрос к API (Network) -> Ответ сервера -> Изменение HTML (DOM)".**

**Зачем тестировщику эти знания на практике?**

- **Тестирование графического интерфейса.** Обычно тестировщик использует разработанный дизайнером макет и сравнивает его с уже

готовой веб-страницей. Вы сможете соотнести стили (расположение, размеры, цвета, шрифты, отступы и т.д.) из обоих ресурсов и правильно протестировать готовое приложение.

- **Точная локализация дефектов:** Вы перестанете писать абстрактные баг-репорты вида *"Кнопка не работает"*. Вы сможете указать: *"При клике на `<button class="submit-btn">` падает ошибка TypeError в Console, сетевой запрос не инициируется"*. Это кардинально сокращает время исправления бага.
- **Разделение ответственности (Frontend vs Backend):** Вы сможете четко определять, кто должен чинить баг. Если в Network запрос ушел и сервер ответил `200 OK` с правильными данными, но на экране они не появились – баг заводится на frontend-разработчика.
- **Тестирование безопасности и граничных условий:** Умение изменять HTML-атрибуты (`maxlength`, `disabled`) "наживую" позволяет проверить, защищен ли бэкенд от некорректных данных, отправленных в обход пользовательского интерфейса.
- **Кроссбраузерное тестирование.** Некоторые стили могут не работать в определенных браузерах и нужно уметь проводить отладку готовых страниц.
- **Фундамент для автоматизации (AQA):** Написание скриптов на Selenium, Playwright или Cypress полностью построено на знании структуры DOM и умении составлять уникальные CSS-селекторы и XPath, так как они определяют в тестах с каким элементом взаимодействовать.

### Пример исполняемого кода

Давайте создадим классический сценарий, с которым тестировщики сталкиваются каждый день: **поле ввода с кнопкой и проверкой (валидацией) данных**.

Ниже представлен полный код простой веб-страницы. Вы можете скопировать этот текст, вставить его в обычный Блокнот, сохранить файл под именем `test.html` и открыть его в любом браузере (например, в Chrome). Он будет работать.

В этом примере я объединил HTML, CSS и JS в один файл и снабдил каждую строку подробным комментарием.

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>Пример для QA</title>

    <style>
        /* Селектор по классу (начинается с точки). Применяется к блоку формы
*/
        .form-container {
            border: 2px solid #ccc; /* Серая рамка толщиной 2 пикселя */
            padding: 20px;          /* Внутренний отступ от рамки до контента
*/
            width: 300px;           /* Жесткая ширина блока (Box Model) */
            font-family: Arial;     /* Тип шрифта */
        }

        /* Селектор по тегу. Применяется ко всем полям ввода на странице */
        input {
            padding: 10px;
            width: 90%;
            margin-bottom: 10px;    /* Внешний отступ снизу, чтобы кнопка не
прилипала */
        }

        /* Селектор по ID (начинается с решетки #). Применяется только к
одной кнопке */
        #submit-btn {
            background-color: blue; /* Синий цвет фона */
            color: white;           /* Белый цвет текста */
            padding: 10px 20px;
            border: none;           /* Убираем стандартную рамку браузера */
            cursor: pointer;        /* Меняем курсор на "руку" при наведении
*/
        }
```

```
        /* Состояние (Pseudo-class). Сработает только при наведении мыши
(Hover) */
        #submit-btn:hover {
            background-color: darkblue; /* Цвет меняется на темно-синий */
        }

        /* Динамический класс. JS добавит его, если пользователь введет
ошибку */
        .error-text {
            color: red;             /* Текст ошибки должен быть красным */
            display: none;          /* ПО УМОЛЧАНИЮ СКРЫТО (тестировщик этого
не видит) */
        }
    </style>
</head>
<body>

    <div class="form-container">

        <h2>Вход в систему</h2>

        <input type="text" id="username" placeholder="Введите имя">

        <button id="submit-btn">Отправить</button>

        <p id="error-message" class="error-text">Ошибка: Поле не может быть
пустым!</p>
    </div>

    <script>
        // ШАГ 1. Находим элементы на странице (по их ID) и сохраняем их в
память браузера (переменные)
        const button = document.getElementById('submit-btn'); // Нашли кнопку
        const inputField = document.getElementById('username'); // Нашли поле
ввода
        const errorMessage = document.getElementById('error-message'); //
Нашли скрытый текст ошибки
```

```
        // ШАГ 2. Вешаем "Слушатель событий" (Event Listener) на кнопку.
        // Говорим браузеру: "Когда по переменной button кликнут ('click'),
запусти эту функцию".
        button.addEventListener('click', function() {

            // ШАГ 3. Получаем то, что пользователь ввел в поле (свойство
value)
            const userValue = inputField.value;

            // ШАГ 4. Клиентская валидация (Проверка условия)
            if (userValue === '') {
                // ЕСЛИ ПОЛЕ ПУСТОЕ:
                // Обращаемся к стилям (style) текста ошибки и меняем скрытое
состояние на блочное (видимое)
                errorMessage.style.display = 'block';

                // Дополнительно красим рамку поля ввода в красный цвет для
наглядности
                inputField.style.border = '2px solid red';
            } else {
                // ЕСЛИ ПОЛЕ ЗАПОЛНЕНО:
                // Прячем ошибку обратно
                errorMessage.style.display = 'none';

                // Возвращаем стандартную рамку полю ввода
                inputField.style.border = '1px solid #ccc';

                // Выводим системное уведомление браузера об успехе
                alert('Данные успешно отправлены: ' + userValue);
            }
        });
    </script>

</body>
</html>
```

**Как тестировщик работает с этим кодом на практике:**

1. **Взаимодействие:** Тестировщик нажимает кнопку с пустым полем. Срабатывает JavaScript (`if (userValue === '')`), который принудительно меняет CSS-свойство `display: none` на `display: block`. Текст появляется.

2. **Проверка через DevTools (Elements):** Если текст ошибки вылез за пределы серой рамки формы, тестировщик открывает вкладку *Elements*, находит `<p id="error-message">` и проверяет вкладку *Styles*. Скорее всего, нужно добавить отступы или изменить `width`.

3. **Обход логики (Console):** Тестировщик хочет проверить, что будет, если отправить форму в обход кнопки. Он открывает вкладку *Console* и пишет скрипт: `document.getElementById('username').value = 'Test';`. Значение поля подменится напрямую через JS.

4. **Автоматизация (AQA):** Чтобы написать автотест для этого сценария, инженер не будет привязываться к русскому тексту "Отправить". Он заглянет в HTML и скажет программе (например, Selenium): `driver.findElement(By.id("submit-btn")).click();`. Это самый надежный способ.

## Полезные материалы

1. Презентация
2. Старая версия https://youtu.be/IOk-S5h8rpc
3. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1n_ub_qIMiWSPTBh5EuwW4-_dFuWejNTx
4. Урок по HTML https://youtu.be/XiQ9rjaa2Ow
5. Урок по CSS https://youtu.be/Tfjd5yzCaxk
6. Быстрый и бесплатный курс для изучения основ HTML и CSS https://htmlacademy.ru/courses/297
7. Туториалы по HTML
8. Туториалы по CSS
9. Туториалы по JS

ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЙ

# 34. Chrome DevTools (версия 2026)

В этом уроке мы познакомимся с одним из главных инструментов веб-тестировщика – **Chrome DevTools** (Инструменты разработчика).

**Chrome DevTools** – это набор инструментов, встроенный прямо в браузер Google Chrome. На самом деле такие инструменты есть в любом браузере, но в данном уроке мы поговорим о самом популярном.

Разработчики используют его для создания и отладки веб-сайта и веб-приложения.

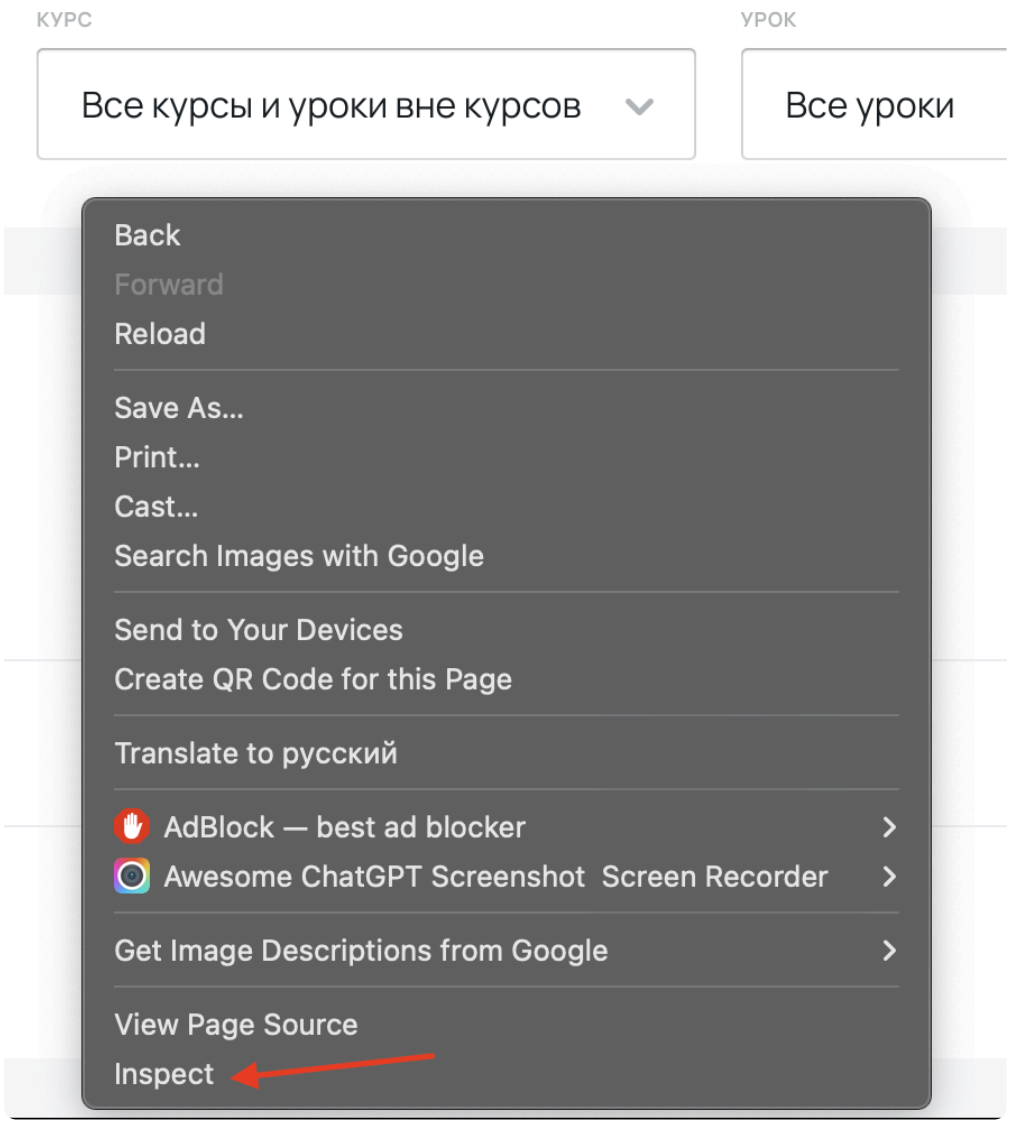
Тестировщики с его помощью могут заглянуть "под капот" веб-страницы, перехватить сетевые запросы, сымитировать медленный интернет и проверить, как сайт отреагирует на нестандартные действия.

Для работы с DevTools не нужно ничего устанавливать, он уже есть в вашем браузере.

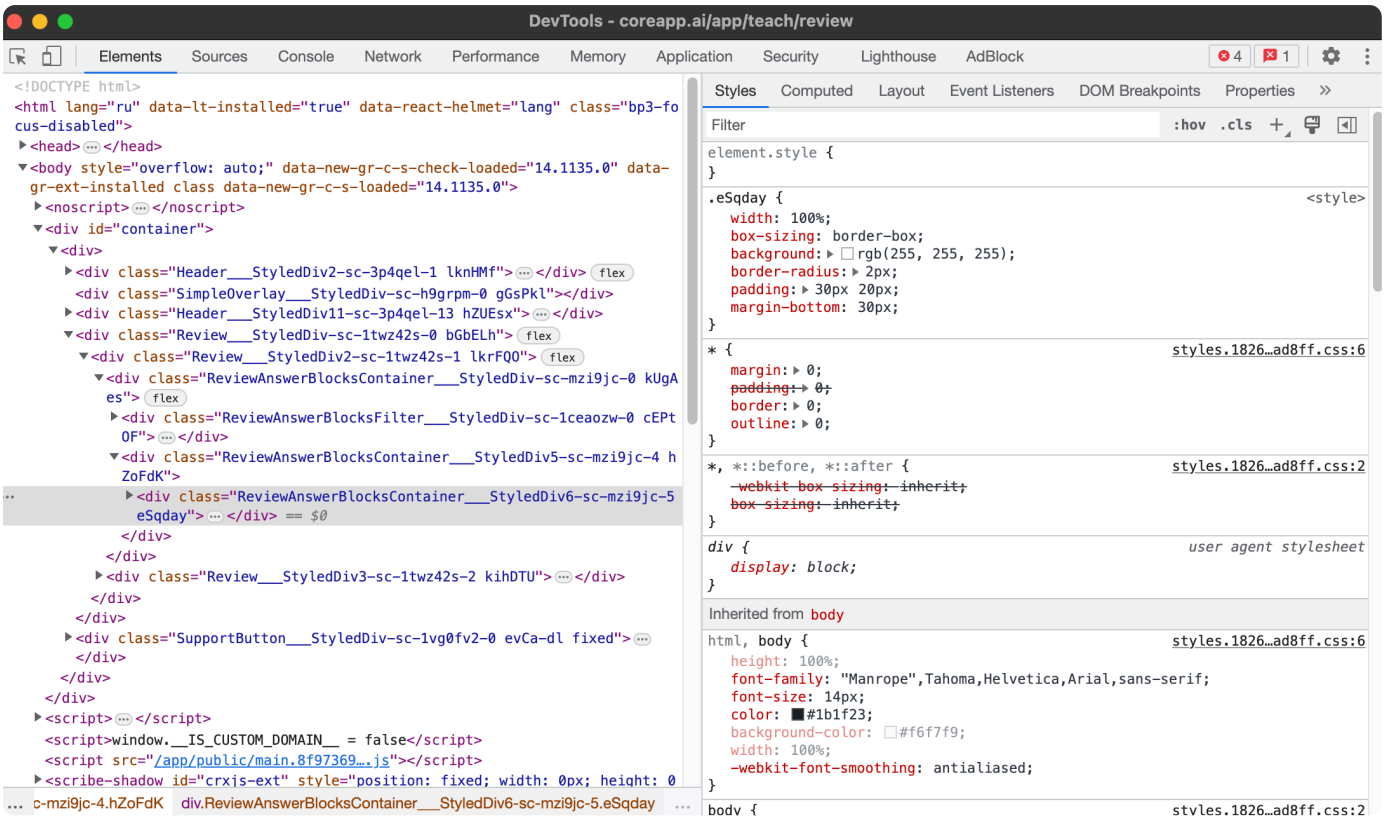
> Далее мы рассмотрим все вкладки внутри этого инструмента, однако некоторые инструменты могу быть избыточными для новичка. Я отметил словом "Основная" то, на что нужно обратить внимание в первую очередь при изучении.

**Как открыть DevTools:**

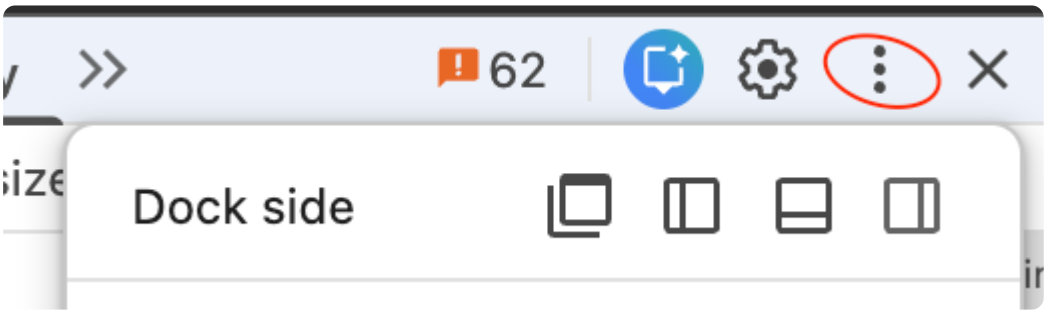
- **Способ 1:** Нажмите клавишу `F12` на клавиатуре (на некоторых ноутбуках `Fn + F12`).
- **Способ 2:** Сочетание клавиш `Ctrl + Shift + I` (для Windows/Linux) или `Cmd + Option + I` (для Mac).
- **Способ 3:** Кликните правой кнопкой мыши по любому элементу на странице и выберите пункт **"Посмотреть код"** (Inspect).




После вызова интерфейс инструментов разработчика выглядит примерно так:




Также вы можете выбрать отображение DevTools в браузере из меню в правом верхнем углу: в отдельном окне (удобно, если вы работаете на нескольких экранах), слева, справу или внизу.




## Полезные материалы

1. Старая версия урока https://youtu.be/a3vRm9neKhs
2. Ссылка на документацию Chrome DevTools https://developers.google.com/web/tools/chrome-devtools
3. Библия QA. Рендеринг в интернете (Rendering on the Web) https://vladislaveremeev.gitbook.io/qa_bible/seti-i-okolo-nikh/rendering-v-internete-rendering-on-the-web

## Вкладка Elements (основная)

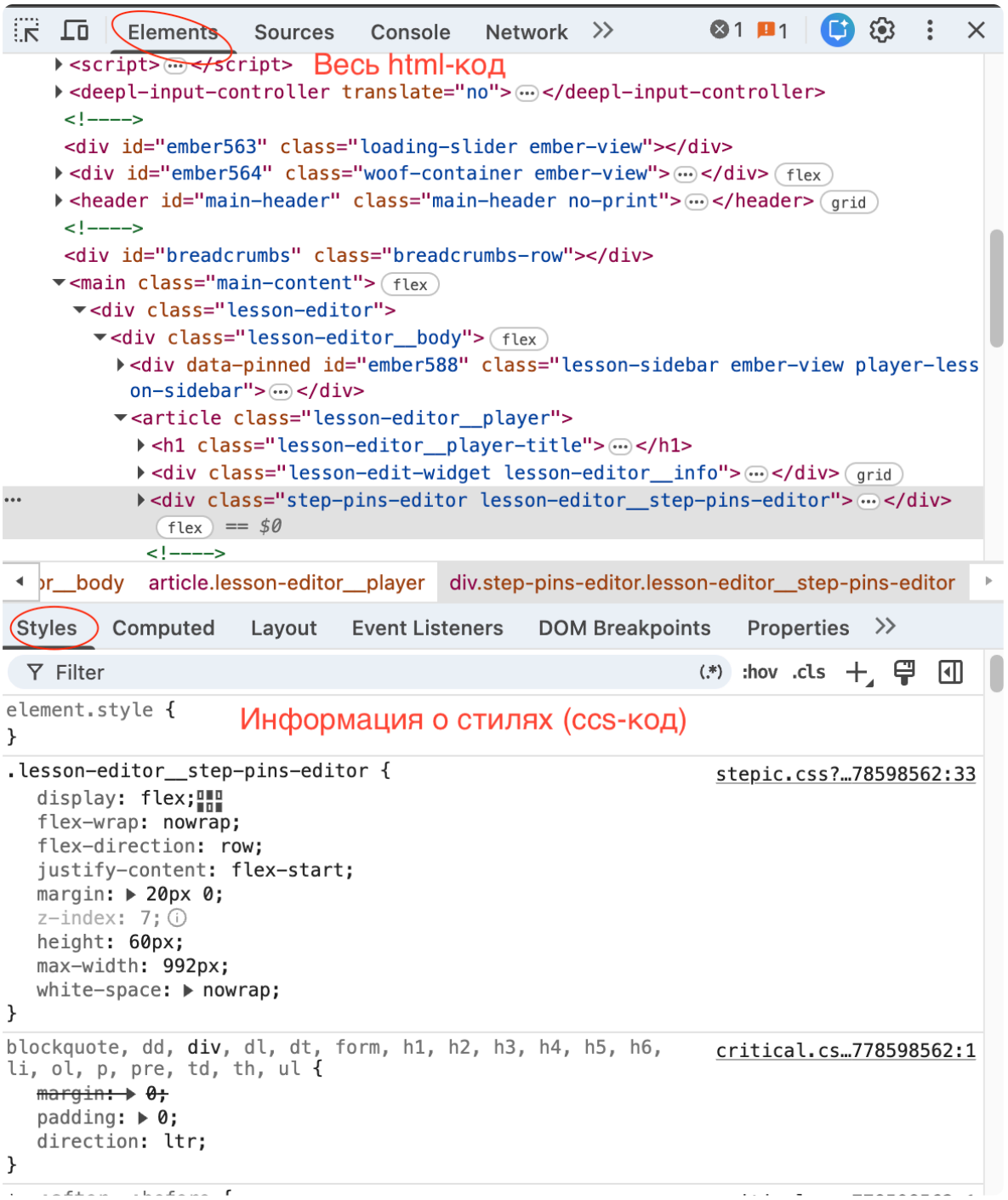
Вкладка **Elements** (Элементы) позволяет просматривать и изменять DOM-дерево (HTML-структуру) и CSS (стили во вкладке Styles) страницы в реальном времени. *Именно здесь мы проводим основное тестирование графического интерфейса (GUI).*

**Зачем это тестировщику в реальной работе :**

1. **Проверка тэгов и атрибутов.** Вы можете находить нужные элементы и взаимодействовать с ними.
2. **Проверка стилей.** Цвета, размеры, расположение элементов и многое другое.
3. **Проверка верстки на "переполнение" (Overflow):** Что будет, если имя пользователя будет состоять из 100 символов? Разорвет ли это карточку профиля? Нам не нужно реально регистрировать такого пользователя в базе данных. Мы можем просто подменить текст прямо в браузере.
4. **Проверка недоступных (disabled) кнопок:** Часто кнопка "Оплатить" неактивна до заполнения формы. Тестировщик может удалить атрибут `disabled` в коде, чтобы проверить, защищен ли запрос на сервер, если пользователь попытается кликнуть кнопку обходным путем.
5. **Поиск скрытых элементов:** Иногда элементы интерфейса перекрывают друг друга из-за ошибок стилей. Во вкладке Elements можно найти "пропавшую" кнопку и понять, почему ее не видно.
6. **Сбор локаторов для автотестов:** Если вы захотите автоматизировать рутину, именно здесь вы будете искать уникальные селекторы (пути) для каждого элемента страницы, чтобы научить робота кликать по ним.

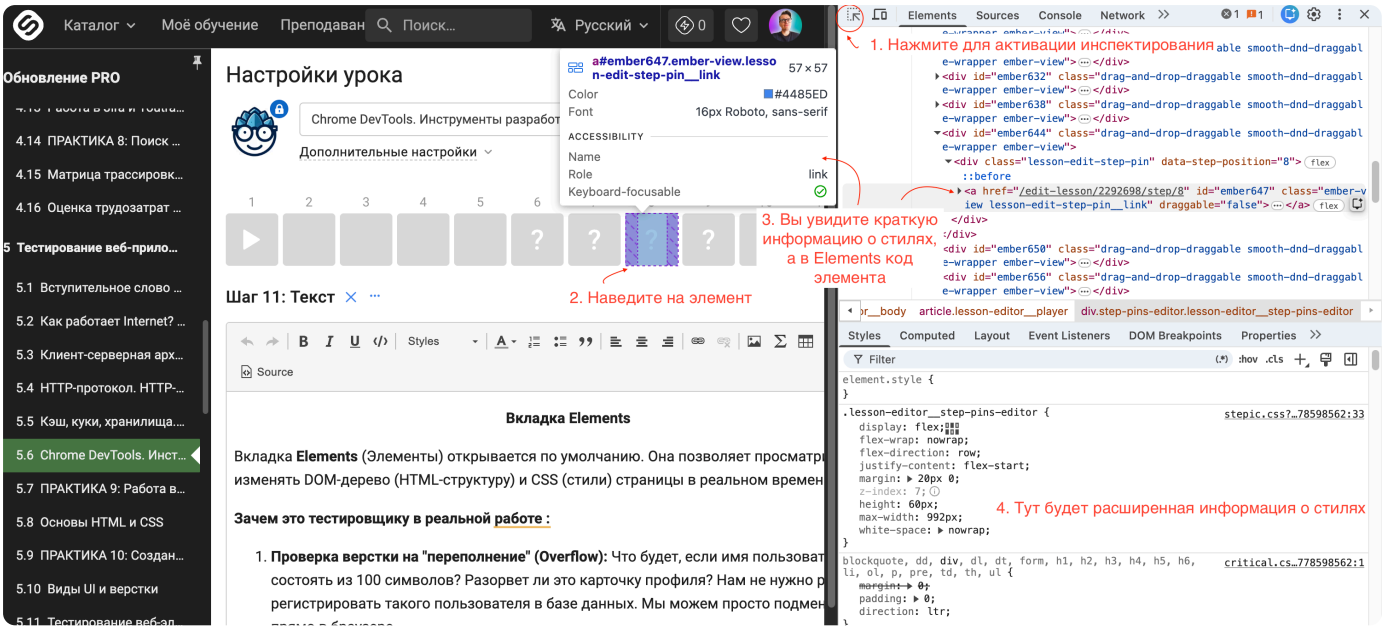
### Как работать с основной вкладкой Elements?

Откройте DevTools любым способом. Вкладка Elements с HTML-страницы откроется по умолчанию. Сразу под или рядом с ней будет доступна информация о стилях в Styles.



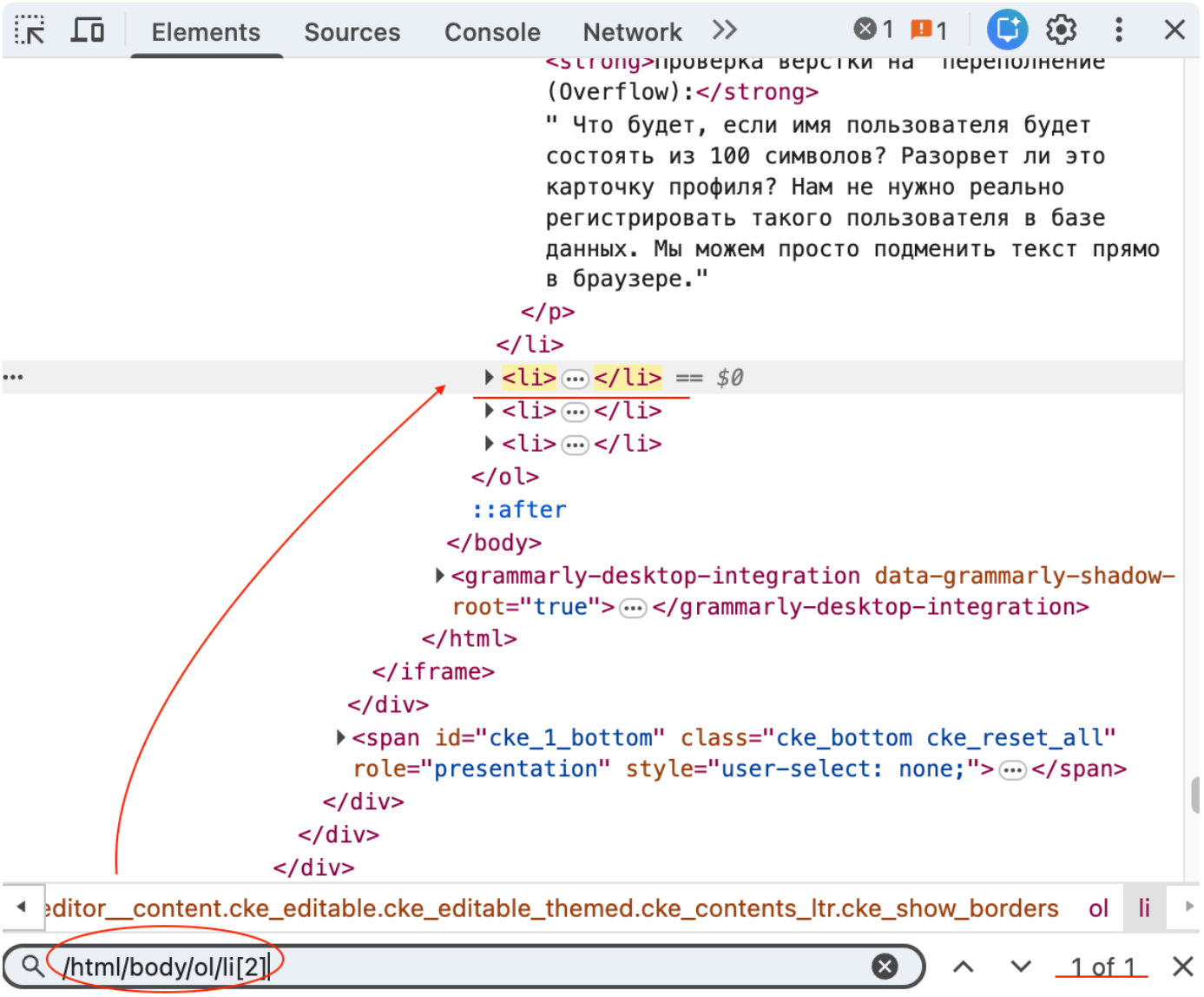

Теперь рассмотрим разные случаи взаимодействия с ними.

**1. Поиск элементов через инспектирование.** Вы можете находить нужный элемент, наводя на него на странице. Это базовая задача тестировщика при тестировании GUI. Информацию отсюда должна совпадать с макетом в Figma.

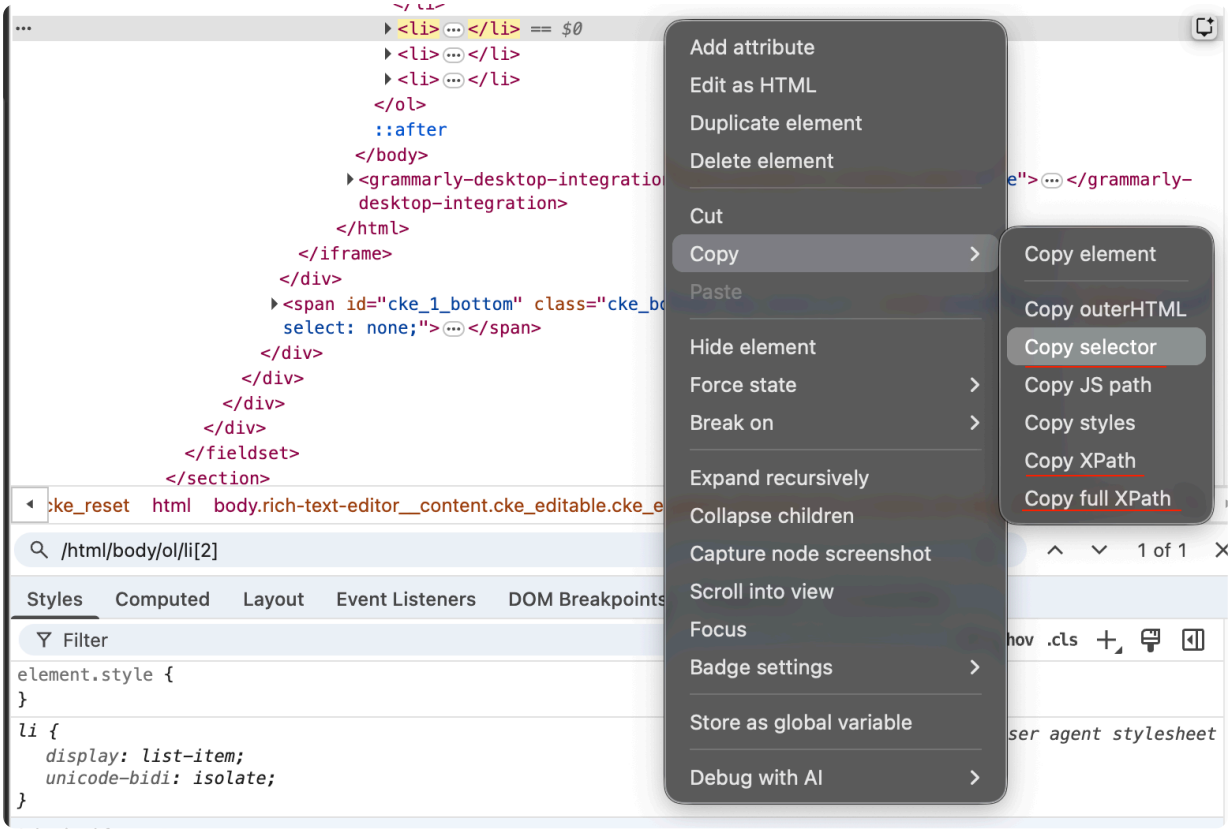



**2. Поиск элементов по локаторам (XPath и CSS-селекторы)** Это базовая операция для тех, кто планирует заниматься автоматизацией или просто хочет быстро найти нужный блок. Для дальнейших задач в этом курсе не понадобится.

- Кликните в любую часть DOM-дерева и нажмите `Ctrl + F` (или `Cmd + F` на Mac).
- Внизу появится строка поиска. Сюда можно вводить обычный текст, но главное: здесь мы проверяем наши CSS-селекторы и XPath.
- Например, введите `/html/body/ol/li[2]`. DevTools подсветит найденную кнопку желтым цветом и покажет количество совпадений (например, "1 of 3"). Цель для автотестов: всегда добиваться уникального совпадения ("1 of 1").

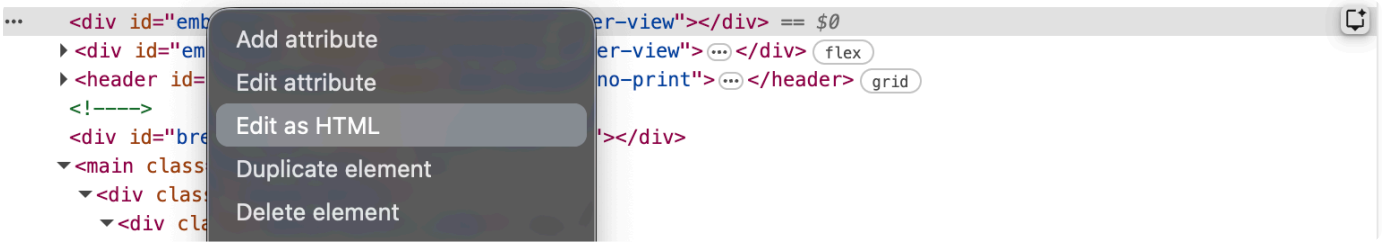



- Если вы знаете, что именно вам нужно, селектор можно копировать прямо в DOM-дереве. Выберите нужный элемент > ПКМ > Copy > любой вариант из Copy selector, XPath, full XPath.




**3. Редактирование HTML в реальном времени.** Иногда нужно изменить структуру элемента, его содержимое, добавить ему новый атрибут или убрать существующий.

- Нажмите правой кнопкой мыши по нужному тегу в DOM-дереве.
- Выберите пункт **Edit as HTML** (Редактировать как HTML).




- В активной строке можно дописать любой код. Например, добавить атрибут `required` к полю ввода, чтобы проверить, как сработает валидация формы, если пользователь попытается отправить ее пустой.

- Кликните в любом пустом месте вне поля редактирования, чтобы применить изменения.

- Используя другие опции контекстного меню вы можете дублировать или удалять элементы.
- Также можно два раза нажать на нужно месте, чтобы отредактировать элемент.

```
<div id="ember563" class="loading-slider ember-view"></div> == $0
```

- Вы можете менять элементы местами, просто зажимая их левой кнопкой мыши в DOM-дереве и перетаскивая выше или ниже. Это полезно для проверки того, как поведет себя верстка, если поменять местами карточки товаров или блоки меню.

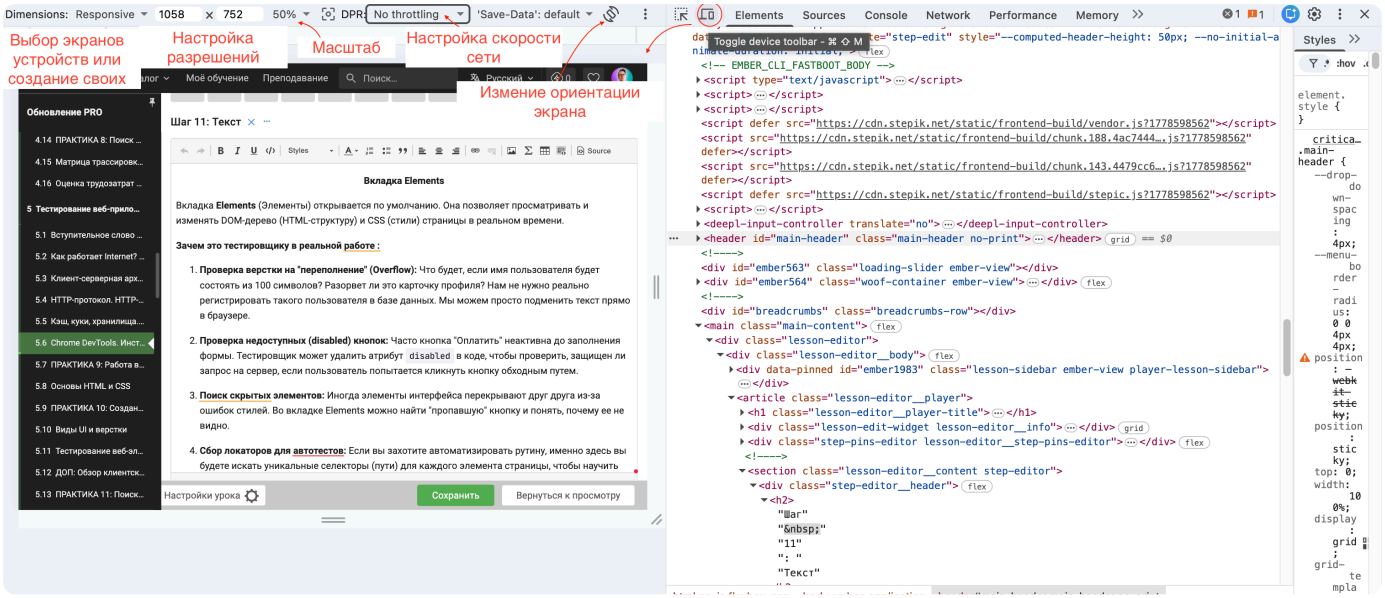
> Помните, что эти изменения после обновления страницы пропадут, так как мы не меняем код на сервере, а при каждом запросе он обновляется. Так что не бойтесь экспериментировать.

**5. Проверка элементов страницы на разных размерах экрана.**

Здесь вы можете задать разрешение экрана, чтобы проверить отображение содержимое страницы на экране телефона, планшета, компьютера, а также при разных размерах окна браузера. Для запуска этого режима нужно нажать

.

Это не полноценная замена тестирования на реальных устройствах, но дает быстрое понимание, как это может выглядеть на них.

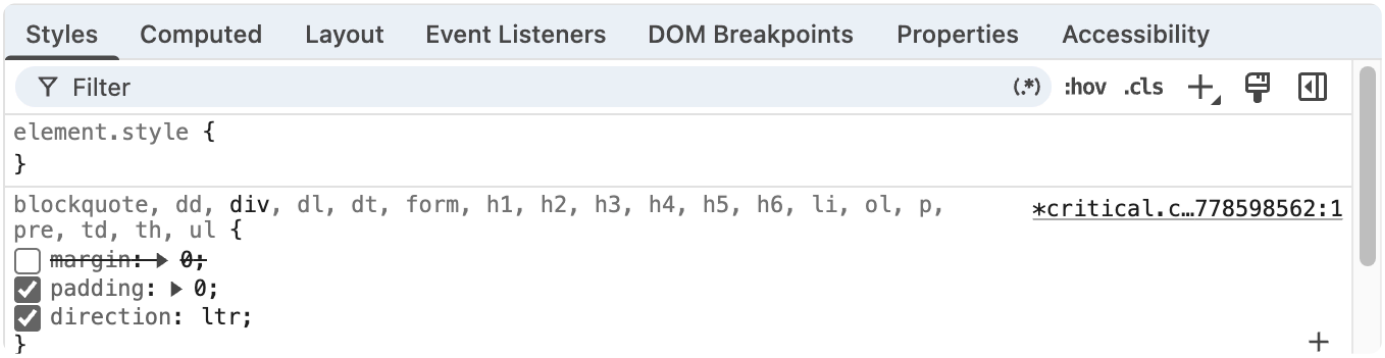



## Как работать со стилями в Elements?

Вам точно пригодятся вкладки Styles и Computed, о них расскажу подробнее с примерами.

**1. Вкладка Styles (Стили)** Здесь отображаются все CSS-правила, которые применяются к выбранному слева элементу.

- **Чекбоксы:** Возле каждого правила (например, `color`, `padding`, `display`) есть галочка. Снимая и ставя ее, вы включаете и выключаете конкретный стиль. Это помогает понять, какое именно правило ломает дизайн.




- **Добавление своих стилей:** Кликните в пустое место внутри любого блока правил, чтобы написать свой CSS-код. Например, напишите `display: none;`, чтобы скрыть элемент.

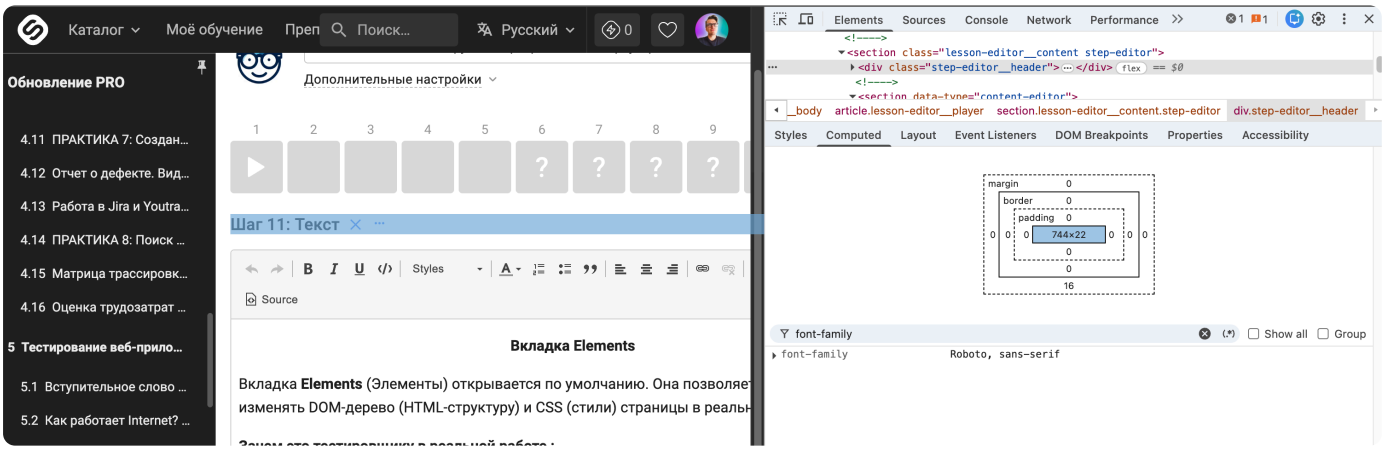
```
blockquote, dd, div, dl, dt, form, h1, h2, h3, h4, h5, h6, li, ol, p,        *critical.c…778598562:1
pre, td, th, ul {
  margin: ▶ 0;
  padding: ▶ 0;
  direction: ltr;
  test: ;
}
```

- **element.style:** В самом верху есть блок `element.style`. То, что вы напишете здесь, применится к элементу напрямую и с наивысшим приоритетом.

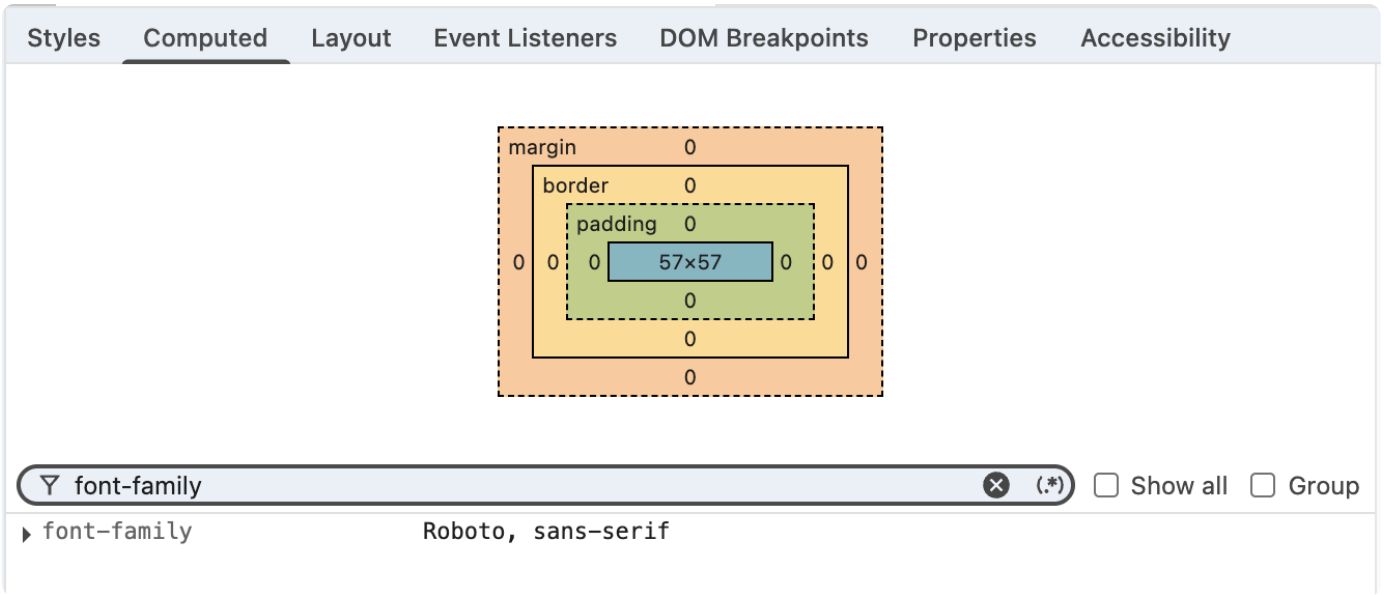
```
element.style {
  : ;
}
```

**2. Вкладка Computed (Вычисленные стили)** Стили могут наследоваться, переопределяться и конфликтовать. Вкладка Styles показывает всю эту борьбу, а вкладка Computed показывает **итоговый результат**.

- **Box Model (Блочная модель):** В самом верху вы увидите схему из цветных прямоугольников. Она точно показывает размеры самого элемента, его внутренние отступы (padding), границы (border) и внешние отступы (margin) в пикселях. Это идеальный инструмент, чтобы проверить, совпадают ли реальные отступы на сайте с макетом в Figma.




- **Фильтр:** Если вам нужно быстро узнать, какой именно шрифт в итоге применился к тексту, введите в строке фильтра "font-family" и вы увидите финальное значение.




**Остальные вкладки в Elements (используются реже)**

**1. Вкладка Layout (Макет)** Эта вкладка помогает визуализировать сложные современные сетки на сайте: Flexbox и Grid.

- Если элемент использует Grid, вы можете поставить галочку рядом с ним во вкладке Layout, и на самой странице поверх элемента появится сетка с линиями и номерами. Это помогает визуально оценить, правильно ли элементы расставлены по ячейкам макета.

**2. Вкладка Event Listeners (Обработчики событий)** Здесь находится JavaScript, привязанный к элементу.

- Вы выбрали кнопку, но по клику ничего не происходит. Откройте Event Listeners и разверните пункт `click`. Если там пусто, значит разработчики забыли "привязать" к кнопке функцию нажатия.

- Здесь можно посмотреть события `focus` (наведение через клавиатуру), `scroll` (прокрутка) и другие. Вы можете временно удалить обработчик (кнопка Remove), чтобы проверить поведение страницы без скрипта.

**3. Вкладка DOM Breakpoints (Точки останова DOM)** Продвинутый инструмент. Позволяет остановить работу страницы в тот момент, когда происходит изменение.

- **Кейс:** На странице есть сообщение об успешной оплате, которое исчезает через 1 секунду. Вы не успеваете его "поймать" в DevTools, чтобы проверить стили.

- **Решение:** Кликаете правой кнопкой мыши по родительскому блоку в HTML-дереве -> `Break on` -> `subtree modifications` (остановка при изменении внутри блока). Вызываете сообщение об оплате. Как только оно появится в коде, браузер поставит страницу "на паузу". Вы сможете спокойно изучить исчезающий элемент во вкладке Elements. Выбранные точки останова будут отображаться в этой вкладке.

**4. Вкладка Properties (Свойства)** Отображает JavaScript-свойства выбранного HTML-узла как объекта. Могут использоваться тестировщиками для глубокой проверки специфичных значений (например, текущего значения свойства `value` у сложных кастомных полей ввода), которые могут не отображаться напрямую в HTML-атрибутах.

**5. Вкладка Accessibility (Доступность)** Важная вкладка для тестирования доступности интерфейса (a11y) для людей с ограниченными возможностями.

- **Дерево доступности:** Показывает, как ваш сайт "видят" программы для чтения с экрана (Screen readers).
- **ARIA-атрибуты:** Здесь проверяется наличие обязательных подписей и ролей для элементов (например, если иконка лупы выполняет роль кнопки поиска, экранная читалка должна понимать, что это кнопка).

**Вкладка Console (основная)**

**Вкладка Console (Консоль) –** это командный пункт и журнал событий веб-страницы. Здесь показываются системные процессы, статусы и сбои сайта и приложения. Здесь отображаются системные сообщения, ошибки в коде, и здесь же мы можем выполнять команды на языке JavaScript в реальном времени.

**Зачем это тестировщику в реальной работе:**

- **Поиск фронтенд-ошибок (Errors):** Если кнопка не нажимается, форма не отправляется или страница зависла, первым делом смотрим сюда. Красные строчки в консоли – главное доказательство бага при заведении отчета для разработчиков.
- **Чтение логов приложения:** Часто разработчики специально настраивают вывод важной информации в консоль (например, ID созданного заказа или ответ от сервера), чтобы вам было проще тестировать сложные сценарии.

Но здесь тоже важно понимать, что консоль доступна и пользователю приложения, так что ничего лишнего на версии для прода быть не должно.

- **Сбор информации для баг-репорта:** Скриншот или скопированный текст ошибки из консоли экономит разработчику часы на поиск причин поломки. Без логов баг-репорт фронтенд-задачи часто считается неполным.
- **Выполнение вспомогательных скриптов:** Тестировщик может написать пару строк кода, чтобы автоматически заполнить длинную анкету из 50 полей тестовыми данными, удалить куки (cookies) или принудительно показать скрытый элемент.
- **Взаимодействие со страницей напрямую:** Вы можете программно кликать по кнопкам или менять текст на сайте с помощью команд, имитируя действия пользователя.

### Как работать со вкладкой Console?

Откройте DevTools и перейдите на вкладку **Console** на верхней панели. Также консоль можно вызвать в виде дополнительной нижней панели на любой другой вкладке (например, находясь в Elements), просто нажав клавишу Esc. Рекомендую всегда держать ее открытой при тестировании.

Теперь рассмотрим основные сценарии работы:

**1. Чтение логов и поиск ошибок.** Как только вы открываете консоль, вы видите различные текстовые сообщения. Они делятся на несколько уровней (уровней важности):

- **Ошибки (Errors):** Подсвечиваются красным цветом. Это критические сбои в работе JavaScript или ошибки загрузки файлов (например, картинка не найдена на сервере – ошибка 404). Именно они чаще всего приводят к багам.
- **Предупреждения (Warnings):** Подсвечиваются желтым цветом. Обычно это не баги, а сигналы от браузера, что разработчики используют устаревший код или что-то работает неоптимально.
- **Информация (Info / Logs):** Стандартный текст, который выводится самими разработчиками для отладки и мониторинга.

- **Verbose (Подробные логи):** Самый детальный уровень вывода, включающий низкоуровневые системные данные браузера и скрытые сообщения (например, `console.debug()`). В повседневной работе этот уровень обычно выключен, чтобы не создавать информационный шум. Тестировщик включает его только по прямой просьбе разработчика для сбора расширенных данных при сложных "плавающих" дефектах.

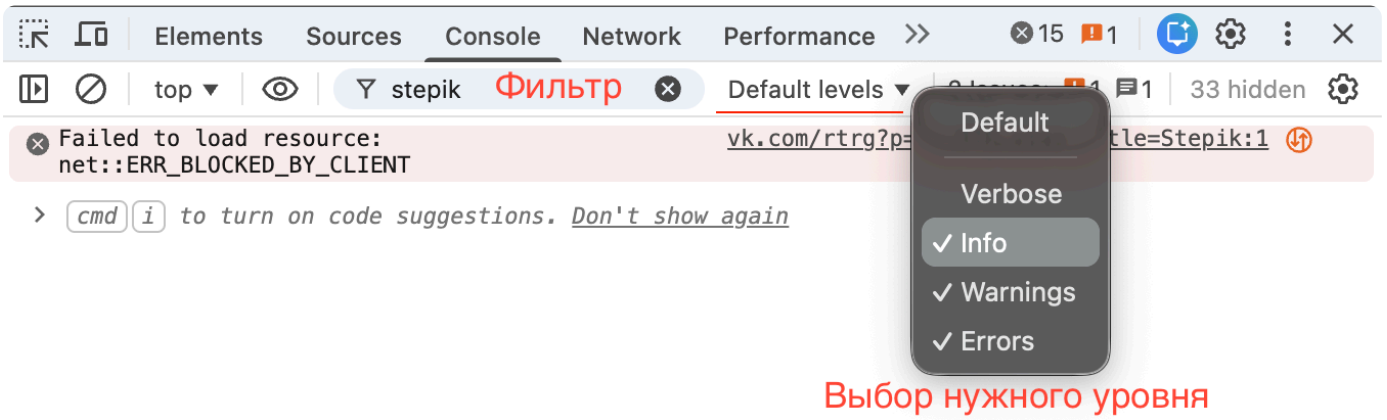
*Примеры желтых предупреждений, которые НЕ нужно заводить как баги:*

- `DevTools failed to load source map` — браузер просто не нашел специальный файл для удобной отладки кода. Обычный пользователь этого никогда не увидит, на работу сайта это не влияет.
- Предупреждения со словами `SameSite attribute` или `Third-party cookie will be blocked` — браузер заранее предупреждает разработчиков о будущих изменениях в политиках безопасности.

> *Совет:* Если кнопка нажимается и все работает согласно требованиям, то можно игнорировать желтые строки.

**2. Фильтрация сообщений.** На сложных сайтах консоль может за секунду заполниться сотнями строк, в которых легко потерять нужную ошибку.

- В верхней части вкладки Console найдите строку **Filter** (Фильтр). Сюда можно ввести любое слово (например, `error` или `payment`), чтобы оставить только сообщения с этим текстом.
- Справа от строки фильтра находится выпадающий список **All levels** (Все уровни) или **Default levels**. Кликните на него.

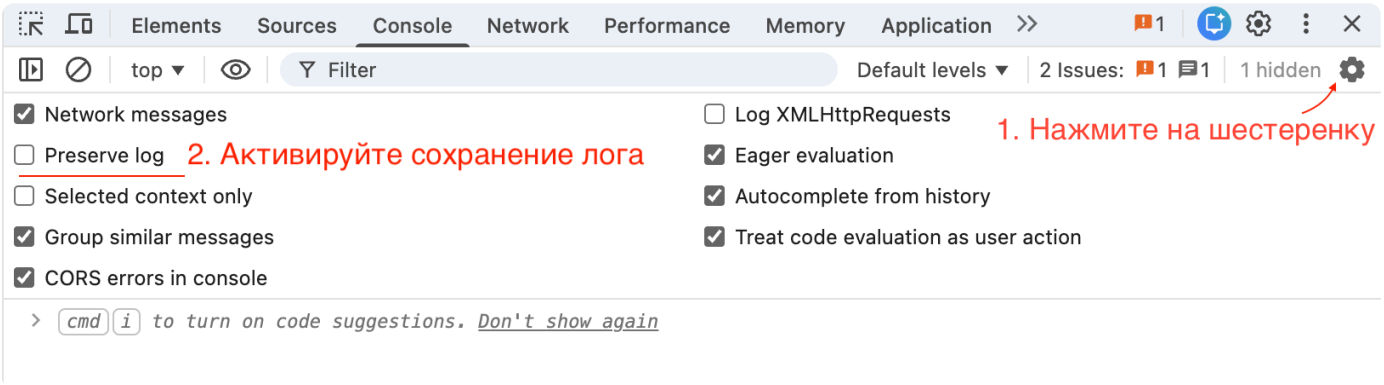



- Снимая и ставя галочки (Verbose, Info, Warnings, Errors), вы можете оставить на экране только то, что нужно. Например, для баг-репорта

тестировщики часто оставляют только галочку на **Errors**, чтобы скопировать чистый лог красных ошибок без лишнего "мусора".

**3. Настройка Preserve log (Сохранение лога).** Это очень важная функция для тестирования.

- **Кейс:** Вы нажимаете кнопку "Оплатить", происходит ошибка, сайт мгновенно перезагружается или перенаправляет вас на другую страницу. Консоль по умолчанию очищается при каждой перезагрузке, и вы не успеваете скопировать текст ошибки.
- **Решение:** В верхней панели консоли (рядом с фильтром, либо в настройках консоли под значком шестеренки ⚙️) найдите и поставьте галочку **Preserve log** (Сохранять журнал).




- Теперь история консоли не будет стираться при переходах между страницами, и вы сможете спокойно пролистать лог наверх и найти ошибку, которая произошла до перезагрузки.

**4. Очистка консоли.** Если логов стало слишком много и они мешают начать новый сценарий тестирования:

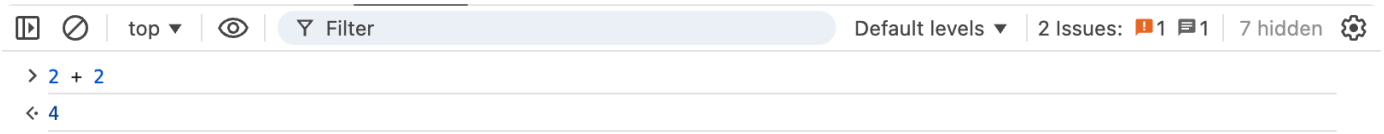
- Нажмите кнопку с иконкой перечеркнутого круга (Clear console)

в левом верхнем углу вкладки.

- Или нажмите сочетание клавиш `Ctrl + L` (или `Cmd + K` на Mac).
- Или кликните правой кнопкой мыши в любом месте консоли и выберите **Clear console**.

**5. Выполнение JavaScript-кода.** В самом низу консоли есть строка ввода со значком >. Это место, где вы можете отправлять команды браузеру.

- Кликните в эту строку.
- Введите простую математическую операцию, например `2 + 2`, и нажмите `Enter`. Консоль сразу выдаст результат: `4`.




- Введите команду `alert('Тест пройден');` и нажмите `Enter`. На сайте появится всплывающее системное окно с вашим текстом.

> *Полезный кейс:* Введите `document.cookie` и нажмите `Enter`, чтобы увидеть все куки, которые сайт сохранил в вашем браузере.

> *Важно:* Если вам нужно перенести строку при написании длинного кода, зажмите `Shift + Enter`. Обычный `Enter` сразу выполняет команду.

**6. Использование системной переменной $0 (Взаимодействие с Elements).** Консоль и вкладка Elements тесно связаны, что позволяет быстро управлять выбранными узлами DOM.

- Перейдите во вкладку Elements и выделите любой элемент (например, кнопку).
- Рядом с выделенным HTML-тегом появится серая подсказка `== $0`. Это значит, что браузер сохранил ссылку на этот элемент во временную переменную `$0`.
- Перейдите обратно в Console (или откройте ее снизу через `Esc`).
- Введите в строку `$0` и нажмите `Enter`. Консоль выведет HTML-код этой кнопки.

> *Практическое применение:* введите команду `$0.click()` и нажмите `Enter`. Кнопка на сайте будет нажата программно. Это работает, даже если элемент был визуально перекрыт другим прозрачным блоком и вы не могли кликнуть по нему курсором мыши. Это надежный способ проверить работоспособность функции обработчика события в обход ограничений графического интерфейса.

### Как отличить баг сайта от "ложной" ошибки (Расширения и блокировщики)

Новички часто пугаются обилия красных строк в консоли на совершенно рабочих сайтах. Важно понимать: в консоль сыпется всё, что происходит в браузере, включая работу ваших личных плагинов. В 90% случаев шум на реальных проектах создают сторонние программы, а не тестируемый сайт.

Как понять, что красная ошибка не наша и баг заводить не нужно:

- **Блокировщики рекламы (AdBlock, uBlock и др.):** Если вы видите красную ошибку с текстом `net::ERR_BLOCKED_BY_CLIENT`, а рядом упоминаются скрипты Яндекс.Метрики, Google Analytics, пиксели соцсетей или рекламные баннеры – это просто ваш блокировщик рекламы запретил их загрузку. Сайт работает правильно, просто плагин выполнил свою работу.
- **Ошибки браузерных расширений:** Справа от каждой строчки с ошибкой указана ссылка на файл, в котором она произошла. Если эта ссылка начинается с `chrome-extension://...` — это сломалось одно из ваших расширений (например, переводчик, VPN, менеджер паролей или антивирус), а не код разработчика продукта.

> **Золотое правило тестировщика:** Чтобы избежать траты времени на расследование чужих ошибок, всегда проводите тестирование в режиме **Инкогнито** (где по умолчанию выключены все ваши расширения) или создайте отдельный, абсолютно чистый профиль в браузере специально для работы.

### Вкладка Network (основная)

**Вкладка Network (Сеть)** показывает всё общение вашего браузера (фронтенда) с сервером (бэкендом). Здесь фиксируется каждый отправленный запрос и каждый полученный ответ: будь то загрузка картинки, отправка формы регистрации или получение списка товаров.

**Зачем это тестировщику в реальной работе:**

- **Локализация багов (Разделение ответственности):** Вы нажали кнопку "Сохранить профиль", но ничего не произошло. Кто виноват? Фронтенд-

разработчик (кнопка сломана) или бэкенд-разработчик (сервер вернул ошибку)? Вкладка Network поможет найти ответ.

- **Проверка отправляемых данных (Payload):** Вы ввели в поле цены "100", но в базу данных ушло "100.00" или вообще пустота. Здесь вы проверяете, какие именно данные уходят на сервер при отправке форм.
- **Имитация слабого интернета и оффлайна (Throttling):** Как поведет себя приложение, если пользователь едет в метро и интернет внезапно пропал? В Network можно искусственно замедлить или отключить соединение.
- **Поиск проблем с производительностью:** Сайт грузится 10 секунд? Здесь вы сразу увидите, что разработчик случайно загрузил на главную страницу неоптимизированную картинку размером 25 МБ.
- **Сбор HAR-файлов для сложных багов:** При плавающих сетевых ошибках тестировщики выгружают весь лог запросов в специальный файл (HAR), чтобы бэкенд-разработчики могли воспроизвести проблему.

**Как работать со вкладкой Network?**

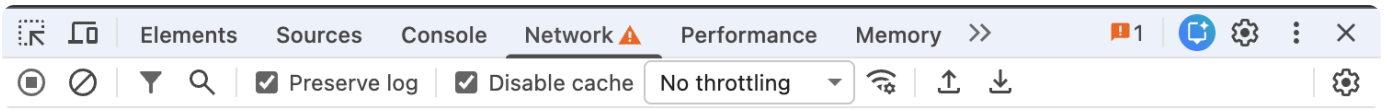
Откройте DevTools и перейдите на вкладку Network. Важное правило: **запись сетевой активности начинается только тогда, когда открыта вкладка Network**. Если вы открыли ее уже после загрузки страницы, список будет пуст. Обязательно обновите страницу.

Теперь рассмотрим основные сценарии работы:

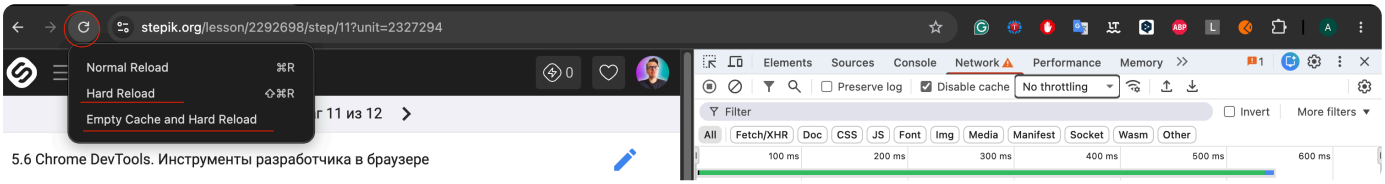
### 1. Работа с сессиями и кэшем

Во вкладке Network есть встроенные инструменты для управления этим:

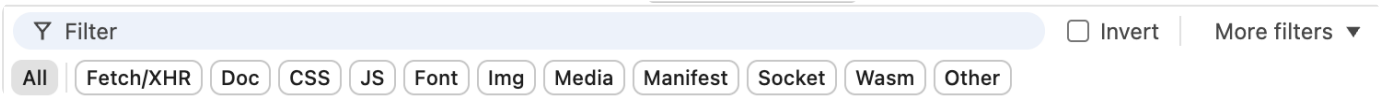
- **Disable cache (Отключить кэш):** В верхней панели есть чекбокс. Если он включен, браузер не будет кэшировать файлы (картинки, стили, скрипты), пока открыт DevTools. Это идеальный способ тестировать верстку и новый функционал, не удаляя кэш вручную каждый раз.
- **Preserve log (Сохранять журнал):** Как и в Консоли, галочка Preserve log в Network не дает браузеру стереть список запросов, если страница перезагрузилась или вас перекинуло на другой экран после оплаты.




- **Hard Reload (Жесткая перезагрузка):** Если вам нужно точечно сбросить кэш только для текущей страницы, не нужно чистить всю историю браузера. При открытом DevTools кликните правой кнопкой мыши по иконке обновления страницы (слева от адресной строки) и выберите "Очистить кэш и жестко перезагрузить" (или `Ctrl + F5` / `Cmd + Shift + R`).




**2. Фильтрация запросов и типы ресурсов** При загрузке современного сайта могут выполняться сотни запросов. Чтобы не искать нужный вручную, используем панель фильтров (под строкой Filter).




- **Типы запросов (Кнопки-фильтры):**
  - **All:** Показывает абсолютно все запросы.
  - **Fetch/XHR:** Главный фильтр тестировщика. Оставляет только запросы данных к API (бэкенду). Здесь вы ищете ошибки при сохранении форм, авторизации или загрузке списков.
  - **Doc (Document):** Исходный HTML-документ страницы.
  - **CSS / JS / Font / Img / Media:** Фильтры по стилям, скриптам, шрифтам, картинкам и медиафайлам (видео/аудио). Полезно, если нужно проверить, загрузился ли конкретный шрифт или видео.
  - **Manifest:** Конфигурационный файл для PWA (прогрессивных веб-приложений), который делает сайт похожим на мобильное приложение.
  - **Socket (WebSockets):** Запросы постоянного соединения. Используются для чатов, биржевых котировок или уведомлений в реальном времени.

    - **Wasm (WebAssembly):** Скомпилированный бинарный код для сложных вычислений в браузере (встречается редко).
    - **Other:** Запросы, которые не попали в другие категории (например, некоторые типы аналитики).

- **Поле Filter.** Позволяет фильтровать запросы по ключевым словам.
- **Чекбокс Invert (Инвертировать):** Находится рядом со строкой поиска. Если вы введете в поиск слово yandex (чтобы найти скрипты аналитики) и поставите галочку Invert, список покажет **все запросы, кроме** тех, что содержат слово yandex. Это удобно для отсеивания "мусорных" логов.

**3. Чтение деталей запроса (Инспектирование)**

Кликните на любой запрос в списке. Справа откроется дополнительная панель с деталями. В ней нас интересуют четыре основные вкладки:

- **Headers (Заголовки):** Здесь находятся сами заголовки и основные части запроса/ответа. Например, **Request URL** (куда ушел запрос), **Request Method** (GET – получить данные, POST – создать или отправить, PUT, PATCH – обновить, DELETE – удалить) и самое главное – **Status Code** (Код ответа). Если там зеленый код 200 (OK) или 201 (Created), то всё отлично. Если красный 400 (Bad Request), 404 (Not Found) или 500 (Internal Server Error), то это может быть багом.
- **Payload (Полезная нагрузка):** Эта вкладка появляется только если вы что-то отправляете на сервер (например, при POST-запросах). Здесь в удобном виде показано, какие именно данные (логин, пароль, сумма) ушли из браузера.
- **Preview (Предпросмотр):** Отформатированный и удобный для чтения ответ от сервера. Если сервер прислал JSON-файл со списком товаров, здесь он будет показан в виде красивого раскрывающегося списка.
- **Response (Ответ):** Сырой, неформатированный ответ сервера. То же самое, что в Preview, но в виде сплошного текста.
- **Initiator (Инициатор):** Отвечает на вопрос "Кто отправил этот запрос?". Показывает цепочку вызовов (Call Stack). Если на сайте "стреляет" странный запрос с ошибкой, здесь можно кликнуть на ссылку и DevTools

перекинет вас во вкладку Sources прямо на ту строку JavaScript-кода, которая этот запрос сгенерировала.

- Вкладка **Timing** разбивает время жизни запроса на миллисекунды: сколько времени ушло на поиск в DNS, установку защищенного соединения (SSL), ожидание ответа от сервера (TTFB - Time To First Byte) и само скачивание данных (Content Download). Если запрос шел 5 секунд, именно здесь вы докажете разработчику, что 4.9 секунды браузер просто ждал ответа от "медленного" бэкенда (высокий TTFB).
- **Cookies (Куки):** Показывает, какие куки были отправлены вместе с этим конкретным запросом (Request Cookies) и какие куки сервер попросил сохранить в ответ (Response Cookies). Незаменимо при тестировании авторизации и сессий.

### 4. Имитация скорости сети (Throttling)

Пользователи не всегда сидят на быстром Wi-Fi. Тестировщик обязан проверять, не ломается ли интерфейс при долгой загрузке.

- В верхней панели Network найдите выпадающий список со значением **No throttling** (Без ограничений).
- Кликните на него. Вы увидите готовые пресеты: **Fast 3G**, **Slow 3G** и **Offline**.
- Выберите `Offline` и попробуйте нажать кнопку на сайте или обновить страницу. Посмотрите, показывает ли интерфейс заглушку "Нет интернета" или приложение просто молча зависает (что является багом).

### 5. Обзорная шкала времени (Overview)

В самом верху вкладки Network (сразу под фильтрами) находится Обзорная шкала времени. Это визуальный график, который показывает общую интенсивность сетевого трафика с момента начала записи.

> *Примечание: если у вас нет этого графика, нажмите на значок шестеренки ⚙️ в правом верхнем углу Network и поставьте галочку **Show overview**.*

**Зачем это тестировщику в реальной работе и как этим пользоваться:**

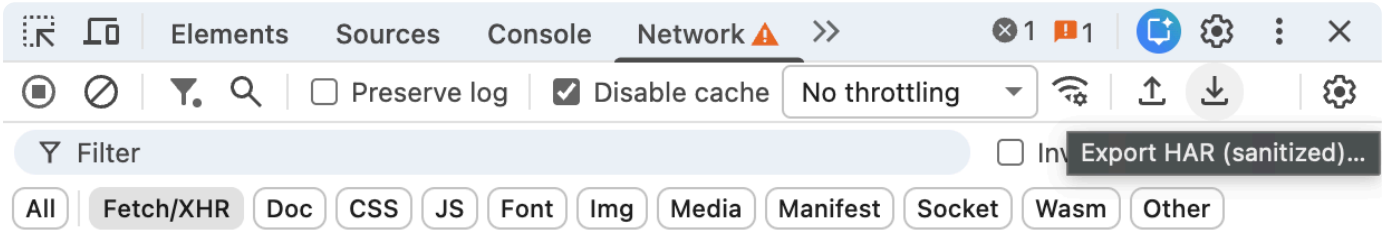
- **Выделение временного диапазона (Изоляция запросов):** Это главная и самая полезная функция графика Overview.
    - *Кейс:* Вы 5 минут заполняли огромную анкету на сайте, в фоне постоянно грузились какие-то метрики и картинки (накопилось 500 запросов). Вы нажимаете кнопку "Отправить" и получаете ошибку. Как найти нужный запрос среди 500 других?
    - *Решение:* Посмотрите на график Overview в момент нажатия кнопки там появился всплеск активности (новые полоски). Зажмите левую кнопку мыши на графике и выделите только этот короткий отрезок времени (последние пару секунд). Таблица запросов внизу мгновенно отфильтруется и покажет **только те запросы**, которые произошли в этот выделенный промежуток. Чтобы сбросить выделение, просто дважды кликните по графику.
- **Поиск паразитного трафика (Фоновые запросы):** Вы ничего не делаете на странице, а на графике Overview каждые 5 секунд появляются новые пики загрузки. Вы выделяете этот пик и видите, что фронтенд постоянно "дергает" сервер запросами `status` или отправляет данные аналитики. Если разработчик сказал, что отключил фоновый опрос сервера, этот график поможет быстро доказать обратное.
- **Чтение вертикальных линий:** Сквозь график проходят вертикальные линии, отмечающие важные события жизненного цикла страницы:
    - **Синяя линия (DOMContentLoaded):** Момент, когда браузер скачал весь HTML-скелет страницы (без учета тяжелых картинок).
    - **Серые/Черные линии:** Моменты отрисовки (Paint). Например, когда браузер впервые показал хоть какой-то контент на экране (First Contentful Paint).
    - **Красная линия (Load):** Событие полной загрузки (когда скачались все картинки и сторонние виджеты).

> *Важно:* иногда красной линии нет вообще. Это не ошибка DevTools, а особенность современных сайтов (SPA). Если страница устанавливает постоянное соединение (WebSockets) или бесконечно подгружает видеопоток, браузер считает, что загрузка формально еще "не завершена", и красная линия не появляется.

**5. Экспорт логов в HAR**

Если вы нашли сложный баг на бэкенде, разработчику понадобится полная картина: какие запросы ушли, в каком порядке и с какими заголовками. Скриншотов тут недостаточно.

- Нажмите **Export HAR** для сохранения вашей сессии.




- На компьютер скачается файл в формате `.har`. Просто прикрепите его к вашему баг-репорту в Jira или Test IT. После разработчик сможет скачать его себе и импортировать в браузер для отладки. *Это один из самых эффективных инструментов для правильной коммуникации разработчика и тестировщика.*

**6. Полезные функции контекстного меню (Правый клик по запросу)** Если нажать правой кнопкой мыши по любому запросу в списке, откроется мощное меню инструментов. Вот самые важные из них для тестировщика:

- **Clear browser cache / Clear browser cookies:** Быстрый способ точечно очистить кэш или куки прямо здесь, не уходя в настройки браузера.
- **Copy -> Copy as cURL (bash):** Копирует запрос со всеми его заголовками, куками и скрытыми параметрами в виде консольной команды. Разработчик вставляет эту команду в свой терминал и мгновенно воспроизводит ваш баг без необходимости открывать браузер. Также эта удобная функция для отправки этого запроса через командную строку или в Postman. Об этом дальше на курсе.
- **Block requests / Block request domain (Блокировка запросов):** Позволяет искусственно запретить загрузку конкретного файла или

целого домена.

    - *Кейс:* Что будет, если у пользователя не загрузится файл стилей (CSS) или зависнет скрипт виджета онлайн-консультанта? Сломает ли это кнопку оплаты? Блокируем запрос и проверяем отказоустойчивость интерфейса. Снять блокировку можно в появившейся внизу вкладке Network request blocking.

- **Throttle requests:** Позволяет замедлить скорость интернета не для всей вкладки целиком, а для одного конкретного запроса.
- **Override headers / Override content:** Продвинутые функции (связанные с Local Overrides из вкладки Sources). Позволяют "на лету" подменить ответ от сервера (например, сервер прислал статус 404, а вы подменяете его на 200) или изменить тело ответа (изменить цену товара в JSON), чтобы проверить, как фронтенд отреагирует на нестандартные данные, не прося бэкендера менять базу данных.

## Вкладка Sources

**Вкладка Sources (Источники) –** это файловая система веб-страницы и встроенная среда отладки (дебаггер). Вкладка Elements показывает финальный результат сборки страницы, а Sources показывает все исходные файлы (картинки, шрифты, скрипты, стили), из которых эта страница состоит.

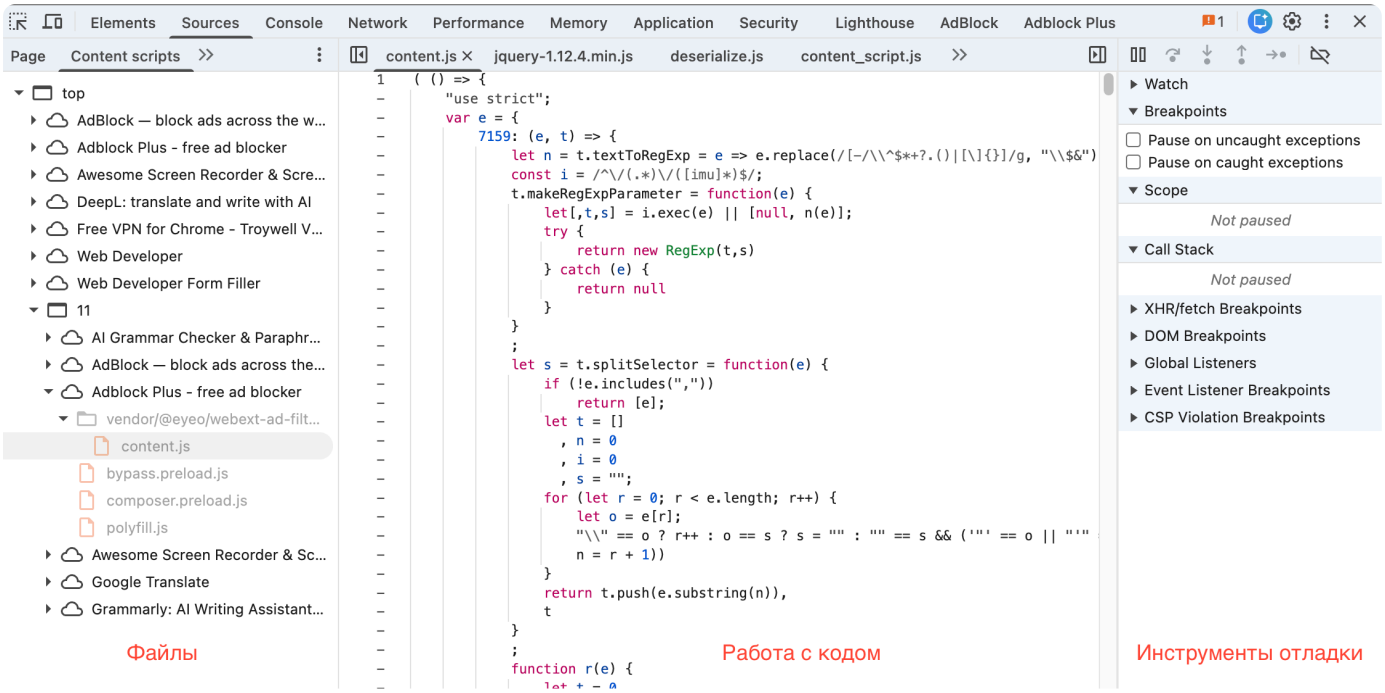
**Зачем это тестировщику в реальной работе:**

- **Доступ к исходникам (Assets):** Иногда нужно проверить, в каком именно разрешении или формате загружается логотип или фоновое изображение, или убедиться, что подтянулся правильный файл шрифта.
- **Использование сниппетов (Snippets):** Вы можете один раз написать (или попросить у разработчиков) полезный JavaScript-код, сохранить его прямо в браузере и запускать в один клик. Это идеально для автоматизации рутины (например, скрипт, который разом заполняет все обязательные поля в анкете).
- **Поимка исчезающих элементов (Пауза работы браузера):** Часто на сайте есть всплывающие подсказки (тултипы) или анимации загрузки (спиннеры), которые исчезают так быстро, что вы не успеваете проверить их верстку в Elements. В Sources можно поставить страницу "на паузу".

- **Локальная подмена файлов (Overrides):** Продвинутый навык. Позволяет изменить код файла (например, подменить картинку или текст) и сохранить эти изменения даже после обновления страницы. Полезно для проверки гипотез без привлечения разработчиков.

### Как работать со вкладкой Sources?

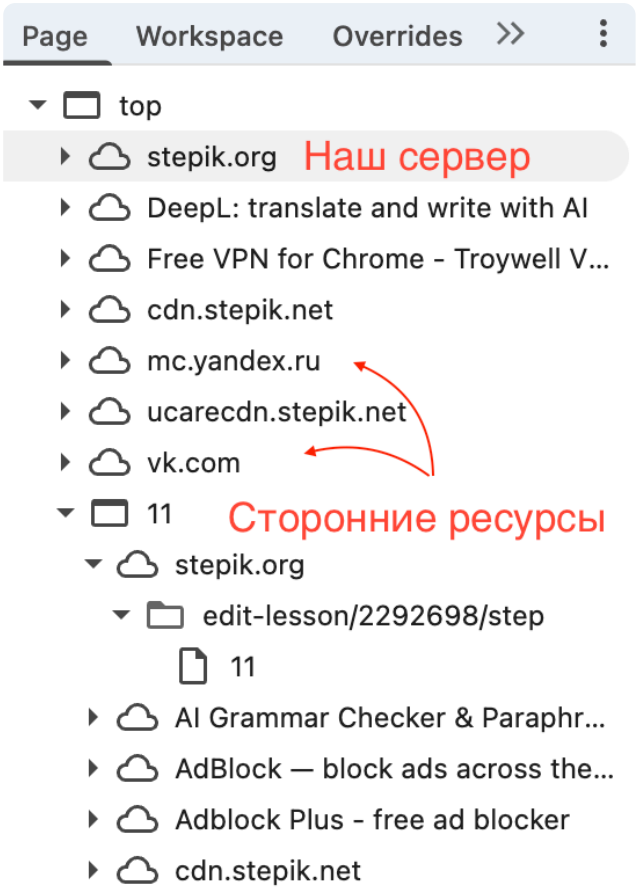
Откройте DevTools и перейдите на вкладку Sources. Интерфейс здесь визуально разделен на три части: левая панель (навигация по файлам), центральная панель (редактор кода) и правая панель (инструменты отладки).




Теперь рассмотрим основные сценарии работы:

**1. Навигация по загруженным файлам.** В левой панели откройте вкладку Page (Страница).

- Здесь в виде дерева папок отображаются все ресурсы, которые загрузил браузер для отображения текущей страницы.
- Папки сгруппированы по доменам. То есть вы четко видите, какие файлы пришли с вашего сервера, а какие – со сторонних ресурсов (например, шрифты от Google Fonts или скрипты аналитики).

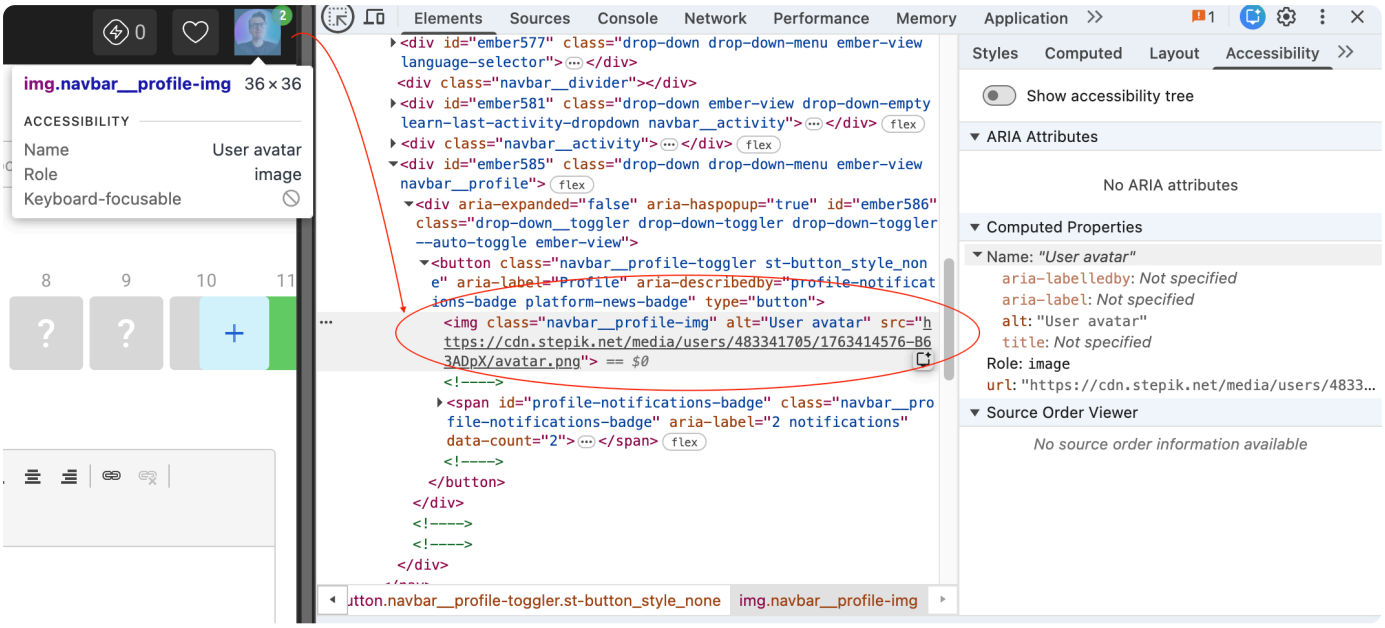


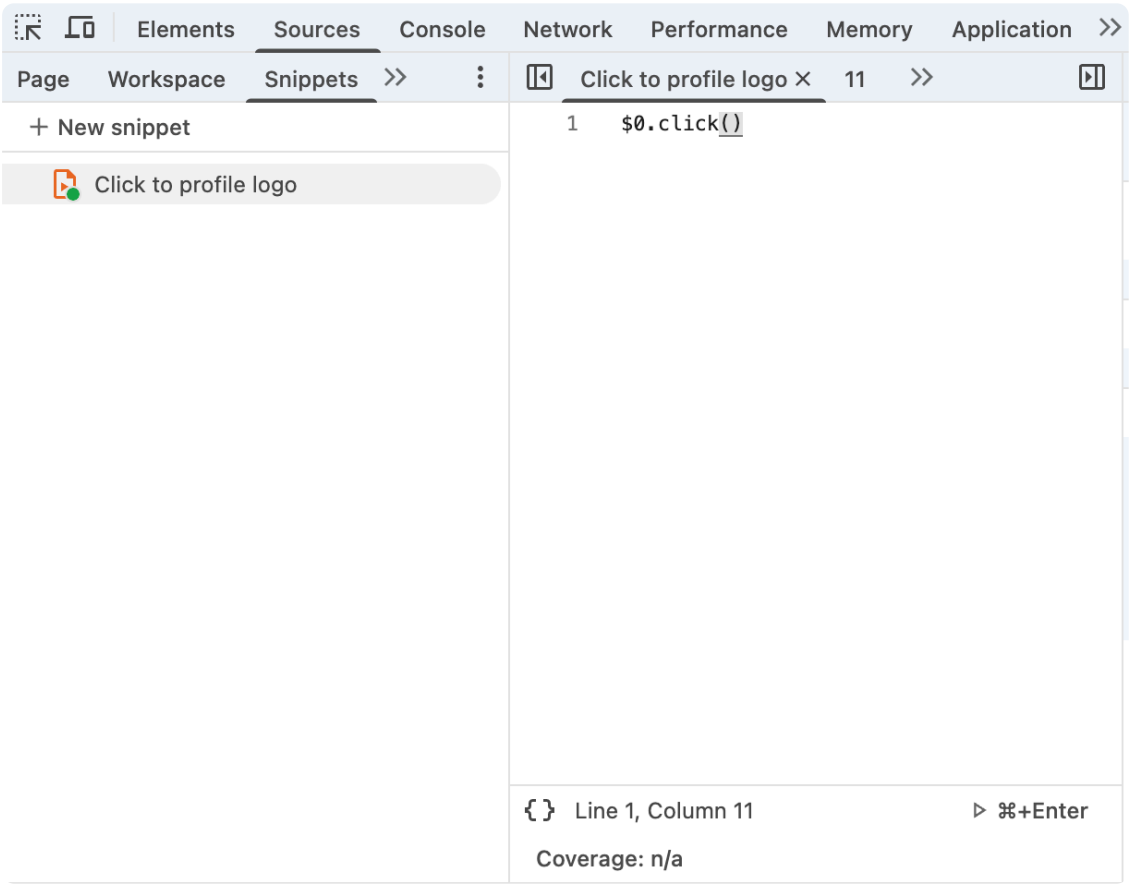

- Вы можете раскрывать эти папки, находить нужные картинки (они откроются в центральном окне), проверять их реальный размер, скачивать или копировать их прямые URL-адреса.

2. **Сохранение и запуск скриптов (Snippets).** Это ваша личная библиотека мини-автотестов прямо в DevTools.

- В левой панели нажмите на значок стрелочек >> (More tabs) рядом со словом Page и выберите **Snippets** (Сниппеты).
- Нажмите **+ New snippet** (Новый сниппет).
- В левом меню появится новый файл (например, `Script snippet #1`). Нажмите на него правой кнопкой мыши, выберите Rename и назовите понятно, например `Click on profile logo`.
- В центральном окне (редакторе) вставьте нужный JavaScript-код. Давайте повторим трюк из консоли из прошлого шага: сначала выберите нужный элемент в Elements (пусть это будет лого вашего профиля на Stepik), в нем

появится == `$0`. После вернитесь в Sources и вставьте этот код в сниппет `$0.click()`



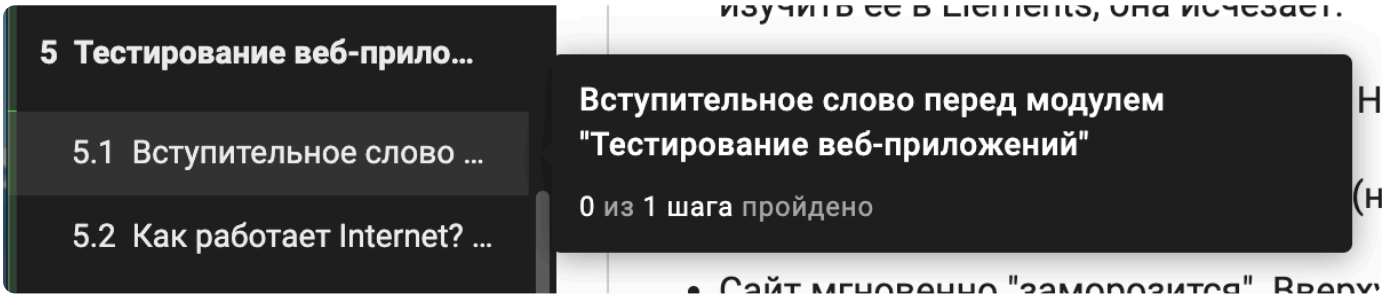




- Чтобы запустить этот код, нажмите сочетание клавиш `Ctrl + Enter` (`Cmd + Enter` на Mac) или кликните по кнопке "Play" (Run snippet) в правом нижнем углу центральной панели.

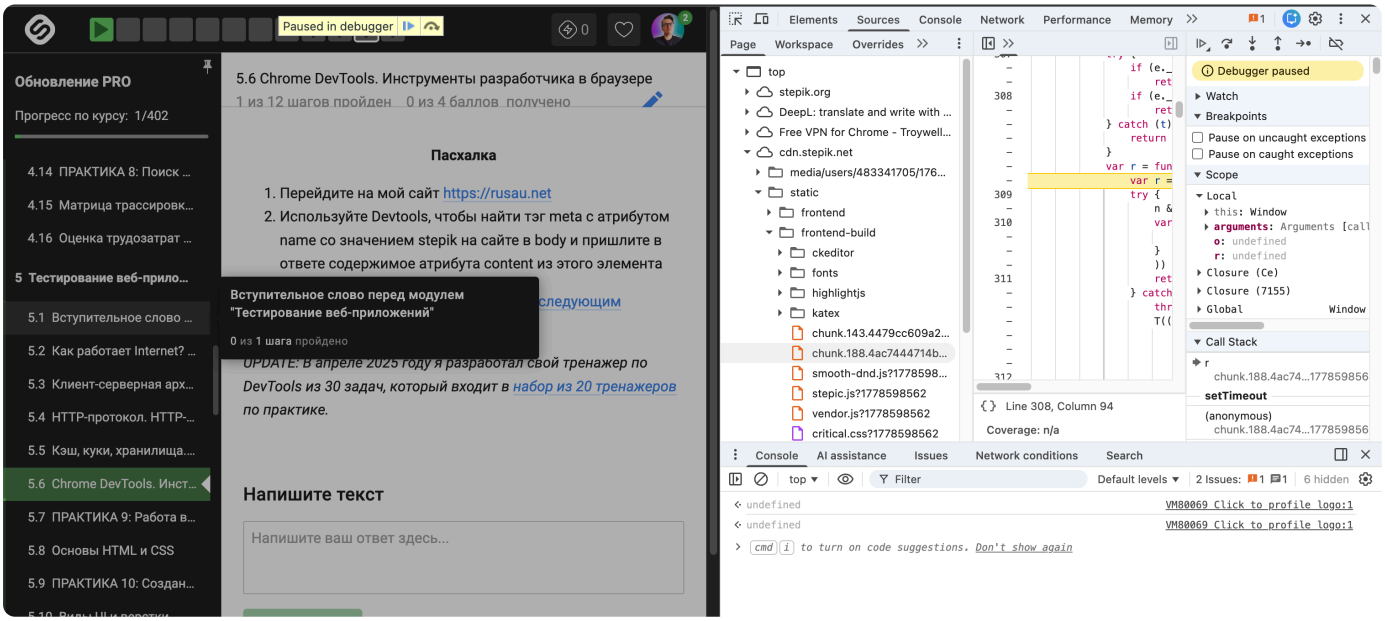
> *Преимущество:* Эти скрипты сохраняются в вашем браузере навсегда. Вы можете закрыть вкладку, перезагрузить компьютер, и на следующий день ваш `Fill_Form` будет готов к работе.

**3. Остановка выполнения страницы (Пауза).** Самый популярный инструмент для проверки динамичных элементов.

- *Кейс:* При наведении на урок появляется красивая всплывающая подсказка, но как только вы уводите курсор, чтобы изучить ее в Elements, она исчезает.
- *Решение:* Откройте вкладку Sources. Наведите курсор на любой урок, чтобы подсказка появилась.




- Нажмите клавишу **F8** на клавиатуре (на Мас иногда `Fn + F8` или `Cmd + \`).
- Сайт мгновенно "заморозится". Вверху экрана появится плашка *Paused in debugger*.




- Теперь вы можете спокойно перейти во вкладку Elements: подсказка никуда не исчезнет, вы сможете не спеша проверить ее стили, цвета и

отступы. Если не выйдет навести на нее курсор при инспектировании, просто найдите ее через поиск в Elements.

- Чтобы снять паузу, снова нажмите F8 или синюю кнопку "Play" на плашке.

**4. Глобальный поиск по всем файлам проекта.** Иногда нужно найти, где именно запрятан определенный текст ошибки или цвет.

- Находясь во вкладке Sources, нажмите сочетание клавиш `Ctrl + Shift + F` (`Cmd + Option + F` на Mac).
- Внизу откроется панель **Search** (Поиск).
- Введите туда искомое слово. DevTools просканирует все загруженные CSS, HTML и JS файлы и покажет точные строки, где упоминается этот текст.

5. **Локальные переопределения (Local Overrides)** – продвинутый уровень.

- *Кейс:* Вы хотите проверить, как будет выглядеть страница, если заменить главное промо-изображение на другое, но у вас нет доступа к серверу. И вы хотите, чтобы при нажатии F5 (обновлении страницы) ваш тестовый вариант не пропадал (в отличие от редактирования в Elements).
- *Решение:* В левой панели выберите вкладку **Overrides**.
- Нажмите **+ Select folder for overrides** и выберите любую пустую папку на вашем компьютере (туда браузер будет сохранять измененные файлы).
- Разрешите браузеру доступ к этой папке (появится всплывающее окно вверху страницы).
- Теперь перейдите обратно во вкладку Page, найдите нужный CSS-файл или картинку, нажмите правой кнопкой мыши и выберите **Save for overrides**.
- Внесите изменения в файл в центральном редакторе кода и сохраните (`Ctrl + S`).
- Обновите страницу. Браузер вместо файла с сервера подтянет ваш измененный файл с жесткого диска. Рядом с файлом в левой панели появится фиолетовый значок, сигнализирующий, что файл переопределен.

> *Важно:* Не забывайте отключать Overrides (снимать галочку *Enable Local Overrides*) после окончания тестирования, иначе вы рискуете долго искать несуществующие баги, глядя на свои же старые локальные изменения.

### Вкладка Performance

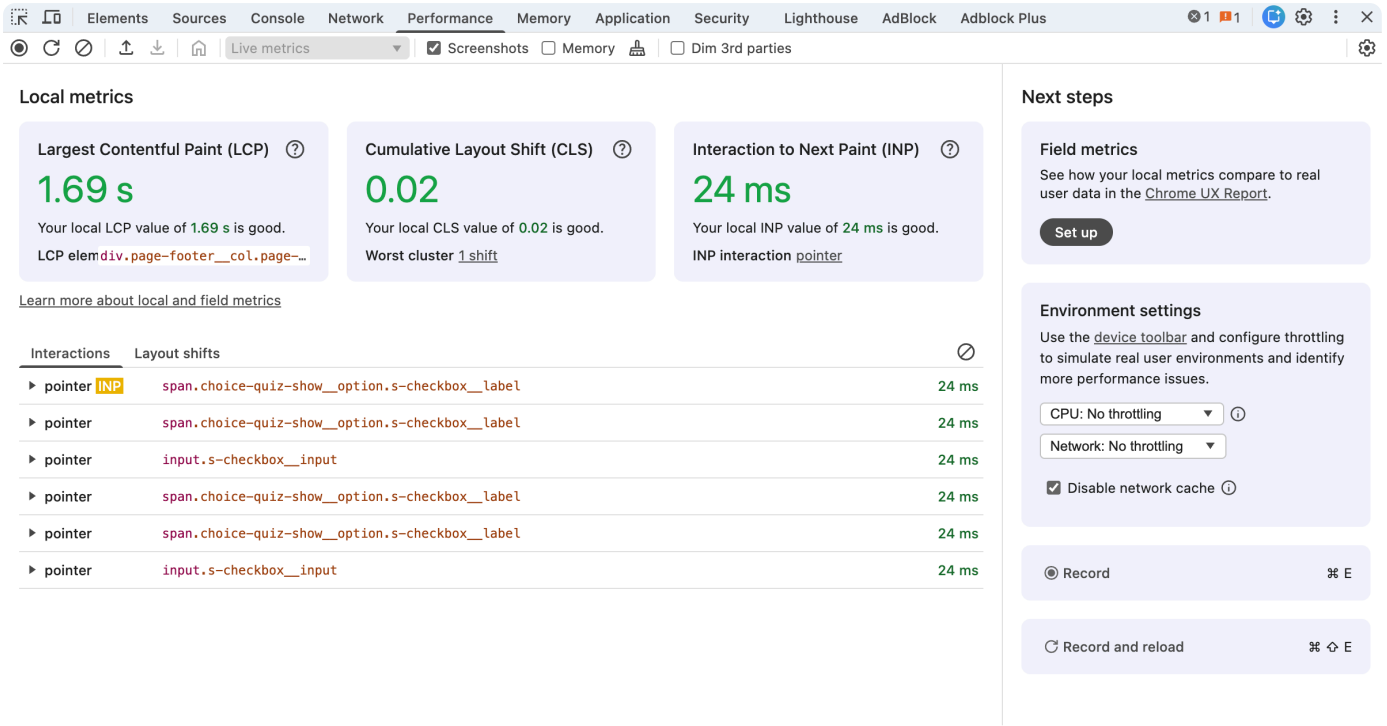
**Вкладка Performance (Производительность)** – это вкладка показывает, как быстро сам браузер обрабатывает файлы: рисует интерфейс, выполняет JavaScript и реагирует на действия пользователя. В последних версиях Chrome эта вкладка была сильно переработана и теперь делает упор на **Core Web Vitals** – ключевые метрики пользовательского опыта от Google.

**Зачем это тестировщику в реальной работе:**

- **Тестирование интерфейса на "слабых" устройствах:** У вас может быть мощный рабочий ноутбук, но большинство пользователей открывает сайт с недорогих смартфонов. Здесь вы можете искусственно "замедлить" свой компьютер, чтобы найти баги производительности.
- **Проверка Core Web Vitals:** Эти метрики напрямую влияют на то, как высоко сайт будет отображаться в поиске Google. Тестировщик помогает убедиться, что новый релиз не обвалил эти показатели в красную зону.
- **Анализ "прыгающей" верстки:** Выявление элементов (рекламы, всплывающих окон), которые сдвигают контент во время загрузки и раздражают пользователей.
- **Оценка отзывчивости (INP):** Поиск ответа на вопрос: "Почему я нажал на кнопку добавления в корзину, а интерфейс отреагировал только через полсекунды?".

### Как работать со вкладкой Performance?

Откройте DevTools и перейдите на вкладку **Performance**. Стартовый экран вкладки представляет собой удобный дашборд (аналитическая панель).




Рассмотрим основные блоки и сценарии работы с ней:

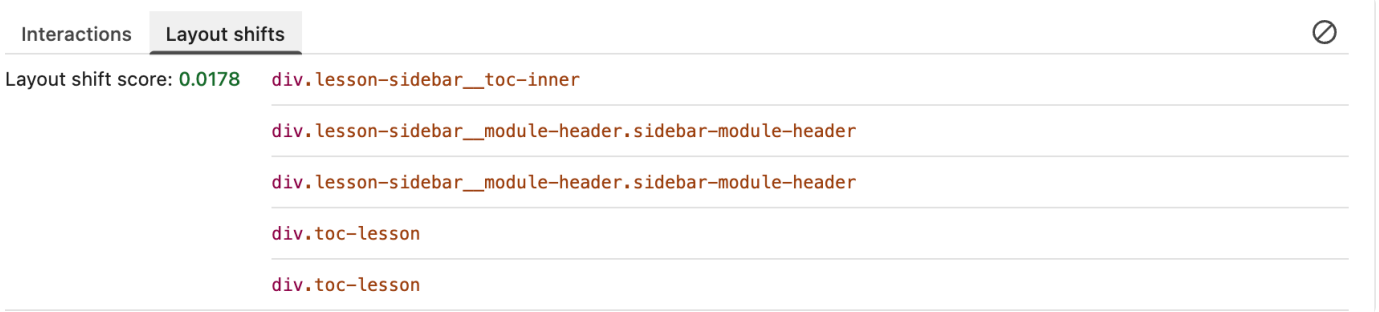
### 1. Анализ локальных метрик (Local metrics)

Прямо на стартовом экране браузер показывает три главные метрики загрузки текущей страницы (результаты работы  на вашем устройстве). Результаты будут трактоваться со скрина выше:

- **Largest Contentful Paint (LCP):** Время отрисовки самого крупного видимого элемента на экране (например, главного рекламного баннера или заголовка статьи).
    - *Пример из практики:* Надпись `Your local LCP value of 1.69 s is good` (Показатель 1.69 сек – это хорошо. Зеленая зона до 2.5 сек).
    - *LCP element:* DevTools сразу подсказывает, какой именно элемент браузер посчитал самым большим (в случае на скрине – это подвал страницы `div.page-footer__col.page-footer__CC-note`). Если страница грузится долго, вы сразу знаете, какую картинку или блок нужно оптимизировать.
- **Cumulative Layout Shift (CLS):** Совокупный сдвиг макета. Оценивает визуальную стабильность страницы. Бывало ли у вас: вы хотите кликнуть по ссылке, но в этот момент сверху подгружается картинка или

рекламный баннер, страница съезжает вниз, и вы кликаете не туда? Это и есть плохой (высокий) CLS, который очень раздражает пользователей.

- *Разбор примера:* Значение `0.02` на карточке – это отличный показатель (норма до 0.1). В разделе *Worst cluster* (худшая группа сдвигов) показано `1 shift` (был один незначительный сдвиг).
- **Вкладка Layout shifts:** Прямо под карточками метрик, по соседству с Interactions, находится вкладка **Layout shifts**. Если показатель CLS оказался в красной зоне, именно здесь тестировщик ищет виновника.
- *Как читать таблицу:* На скриншоте зафиксирован конкретный сдвиг с оценкой `Layout shift score: 0.0178`. Справа красным шрифтом перечислены HTML-элементы (например, `div.lesson-sidebar__toc-inner` и `div.toc-lesson`), которые в этот момент сместились на экране.




> При завдении бага можно скопировать из этой таблицы точные селекторы сместившихся элементов и передать разработчику. Чаще всего баг лечится тем, что разработчик заранее прописывает жесткую высоту (height) для блоков с картинками или рекламой, чтобы они не расталкивали соседний контент по мере загрузки.

- **Interaction to Next Paint (INP):** Отзывчивость интерфейса. Эта метрика измеряет визуальную задержку: сколько времени прошло с момента вашего клика (или нажатия клавиши) до момента, когда браузер смог отрисовать ответ на этот клик (например, показать галочку в чекбоксе или открыть выпадающее меню).
  - *Как это работает:* Если вы только открыли страницу, метрика будет пустой, так как вы еще ничего не нажали. Вам нужно начать взаимодействовать с сайтом.

    - *Разбор примера со скриншота:* После кликов по элементам, появилась цифра `24 ms` (зеленый цвет означает, что это отличный показатель, норма до 200 миллисекунд). Поле `INP interaction pointer` подсказывает, что метрика зафиксировала взаимодействие типа "pointer" (клик мышью или тап по экрану).

- **Вкладка Interactions:** Прямо под карточками метрик находится таблица с подробной историей. На вкладке **Interactions** DevTools записывает каждый ваш клик. Вы можете точно увидеть, по какому именно HTML-элементу вы попали (например, `input.s-checkbox__input` или `span.choice-quiz...`) и сколько времени интерфейс думал над каждым конкретным нажатием.

> *Поиск "слабого звена":* Самая долгая задержка в этом списке автоматически помечается желтым бейджиком **INP** (как на первой строке вашего скриншота). Если сайт начнет тормозить, вам не придется гадать, какая именно кнопка вызывает зависание, то DevTools сам укажет на нее этим значком, и вы сможете приложить эту точную информацию к баг-репорту.

### 2. Настройки окружения (Environment settings)

Здесь можно имитировать различные настройки для имитации мобильного устройства.

- **Network (Сеть):** Аналог функции Throttling из вкладки Network. Позволяет имитировать `Slow 3G` или `Fast 3G`.
- **CPU (Процессор):** По умолчанию стоит `No throttling`. Кликните сюда и выберите `4x slowdown` (замедление в 4 раза) или `6x slowdown`. Браузер искусственно ограничит свои вычислительные мощности.
- **Disable network cache:** Галочка для отключения кэша сети. Обязательно включайте ее при замерах стартовой загрузки страницы, чтобы тестирование было объективным.

### 3. Полевые метрики (Field metrics / Next steps)

Внизу панели есть блок `See how your local metrics compare to real user data in the Chrome UX Report` (Посмотрите, как ваши локальные метрики соотносятся с данными реальных пользователей).

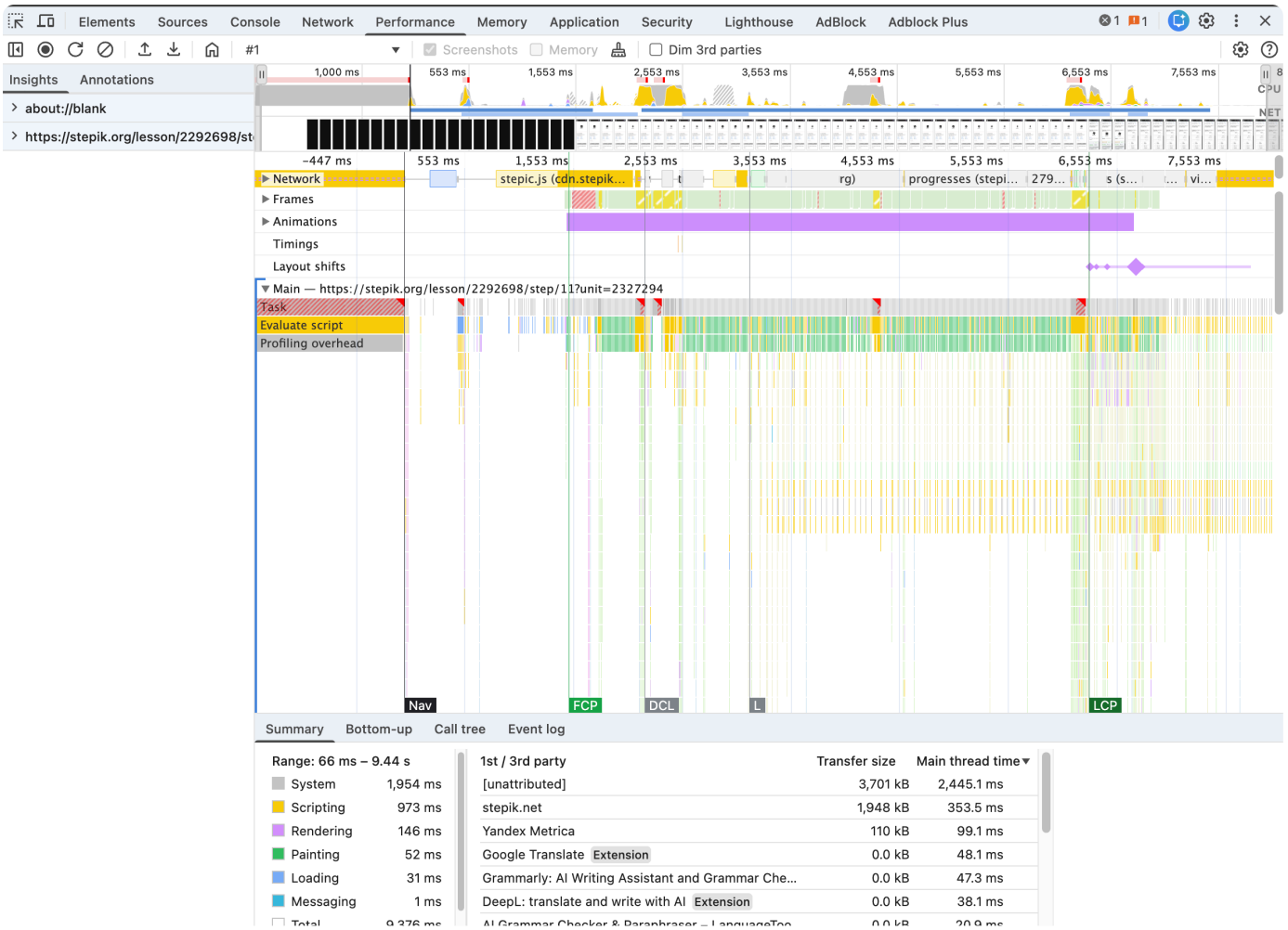
- *Что это значит:* Ваши локальные 1.69 секунды – это результат на вашем рабочем интернете. Если вы нажмете **Set up** (Настроить), DevTools подтянет базу данных Google (CrUX) и покажет средние значения для этой страницы у реальных пользователей из интернета. Это помогает понять: страница быстро работает только у вас, или у пользователей тоже всё хорошо.

### 4. Запись профиля производительности (Record)

Если метрики в красной зоне и вам нужно собрать детальные доказательства для разработчика, необходимо записать профиль загрузки. Для этого есть две кнопки в самом низу:

- **Record (Кнопка записи или `Ctrl+E` / `Cmd+E` на Mac):** Используется для записи конкретного сценария. Нажмите кнопку -> совершите действие на сайте (например, откройте сложный фильтр товаров) -> нажмите Stop. Вы получите огромный график (Flame chart) того, какие скрипты съели ресурсы процессора в этот момент.
- **Record and reload (Запись и перезагрузка):** Идеально для проверки стартовой скорости. Нажмите ее, и DevTools сам обновит страницу, запишет весь процесс ее отрисовки с нуля и остановит запись, когда сайт полностью загрузится.

В конце вы получите примерно такой график профиля производительности. Для общего развития я напишу его расшифровку, однако в реальной практике вам это, скорее всего, не пригодится на ежедневной основе.




Интерфейс разделен на три логические части: обзорная шкала сверху, детальные треки (дорожки) по центру и панель сводки внизу.

**1. Обзорная шкала (Timeline Overview)** Самый верхний график показывает загрузку процессора (CPU) и сети (NET) на протяжении всей записи.

- Выделяя мышью небольшой участок на этой верхней шкале (зажмите левую кнопку и потяните), вы приближаете (зумируете) центральный график. Анализировать весь 10-секундный профиль разом невозможно, нужно приближать конкретные пики активности.

**2. Трек Main (Основной поток) – главная зона поиска багов** Под вкладками Network, Frames и Timings находится секция **Main**. Это визуализация того, чем занимался процессор. Браузер работает в один поток, поэтому, если он занят сложными вычислениями, пользователь не может кликать по кнопкам (интерфейс "зависает").

- **График пламени (Flame chart):** Задачи отображаются в виде перевернутых "сосулек". Верхний блок – это главная функция, а блоки под

ней представляют более мелкие функции, которые она вызвала.

- **Красные треугольники (Long Tasks):** Это ваш главный ориентир! Посмотрите на скриншот: на дорожке Main (в строке Task) видно множество красных засечек. DevTools автоматически помечает так "Длинные задачи" (Long Tasks) – любые скрипты, выполнение которых занимает больше 50 миллисекунд.

> **Кейс для тестировщика:** Если пользователь жалуется, что при скролле страница тормозит (лагает), вы делаете запись, находите огромный блок с красным треугольником, кликаете по нему и передаете разработчику информацию о том, какой именно скрипт блокирует интерфейс.

**3. Метки событий (Тайминги)** Обратите внимание на вертикальные линии с цветными квадратиками в нижней части графика (сразу над панелью Summary). Это моменты, когда сработали ключевые метрики, которые мы видели на стартовом дашборде:

- **FCP (First Contentful Paint, зеленый):** Момент, когда на белом экране появился первый текст или фон.
- **DCL (DOMContentLoaded, серый):** HTML-документ загружен.
- **L (Load, серый):** Страница загрузилась полностью со всеми картинками.
- **LCP (Largest Contentful Paint, зеленый):** Отрисовался самый большой элемент на странице (на скрине видно, что это произошло ближе к 7-й секунде).

**4. Вкладка Summary (Сводка) и вкладка Bottom-Up** В самом низу экрана находится панель деталей. Если кликнуть в пустое место на графике, она покажет общую сводку за всю запись.

- **Цветовое распределение (Pie Chart):** Слева показано, на что ушло время.
    - Желтый (Scripting) – выполнение JavaScript. На скриншоте это 973 мс.
    - Фиолетовый (Rendering) – расчет стилей и макета страницы.
    - Зеленый (Painting) – физическая отрисовка пикселей на экране монитора.

> *Кейс:* Если страница зависает, и вы видите, что 80% времени занимает желтый цвет, значит, проблема в перегруженном коде фронтенда. Если огромный фиолетовый кусок — проблема в неоптимизированных CSS-стилях и прыгающей верстке.

- **Анализ сторонних скриптов (Важное правило из реальной практики):** Посмотрите на таблицу справа в блоке Summary. DevTools показывает, какие именно сервисы съели процессорное время. Мы видим там код самого сайта (stepik.net), аналитику (Yandex Metrica)... и браузерные расширения: **Google Translate, Grammarly, DeepL**!

> Это наглядное доказательство того, почему тестировать производительность нужно **строго в режиме Инкогнито** без расширений. На данном скриншоте расширения по проверке грамматики и перевода активно сканировали текст сайта, нагружая процессор и искажая чистые метрики приложения.

## Вкладка Memory

**Вкладка Memory (Память)** – это инструмент для диагностики того, сколько оперативной памяти (RAM) потребляет ваш сайт. Она показывает, насколько приложение "прожорливо".

В современных веб-приложениях (**SPA** – Single Page Applications, таких как Jira, Figma или банковские дашборды) пользователь может работать часами без обновления страницы. Если разработчик допустил ошибку в коде, старые данные не будут удаляться из памяти. Оперативная память переполнится, сайт начнет жутко тормозить, и в конце вкладка браузера просто "упадет" с системной ошибкой "Aw, Snap!". Это называется **утечкой памяти (Memory Leak)**.

**Зачем это тестировщику в реальной работе:**

- **Проверка стабильности при долгом использовании:** Выяснить, почему после 30 минут активной работы на сайте интерфейс начинает реагировать с задержкой в несколько секунд.
- **Анализ "Отсоединенных элементов" (Detached DOM):** Вы открыли модальное окно (например, корзину), а затем закрыли его. Визуально окно исчезло, но из-за ошибки в скрипте оно навсегда осталось висеть в оперативной памяти компьютера, занимая место.

- **Предоставление доказательств утечки:** Разработчику недостаточно слов "сайт тормозит". Ему нужен файл снимка памяти, чтобы увидеть, какие именно объекты (массивы данных, картинки или куски HTML) не удаляются браузером.

**Как работать со вкладкой Memory?**

1. Откройте DevTools и перейдите на вкладку **Memory**.

> **Важное правило подготовки:** Тестирование памяти проводится **ИСКЛЮЧИТЕЛЬНО в режиме Инкогнито** (без расширений) или в чистом профиле браузера. Расширения (переводчики, блокировщики рекламы) потребляют свою память и внедряют свои скрипты в вашу страницу. Если вы не отключите их, вы найдете сотни утечек памяти, которые принадлежат вашим плагинам, а не вашему продукту.

Рассмотрим основной алгоритм работы:

**1. Инструмент Heap Snapshot (дословный перевод: Снимок кучи)** Это основной инструмент. Снимок делает мгновенную "фотографию" всей оперативной памяти, которую сайт использует прямо сейчас.

- В меню по умолчанию выбран пункт **Heap snapshot**.
- В самом верху слева найдите **иконку**

**(Collect garbage)**. Это принудительный вызов "Сборщика мусора" браузера – встроенной программы, которая удаляет из памяти всё, что уже точно не нужно странице. *Тестировщику нужно нажимать эту корзину перед каждым снимком*, чтобы получить чистый результат без временного мусора.

- Нажмите кнопку **Take heap snapshot**

(Сделать снимок).

- В левом меню появится файл `Snapshot 1` и его размер (например, 15 MB).

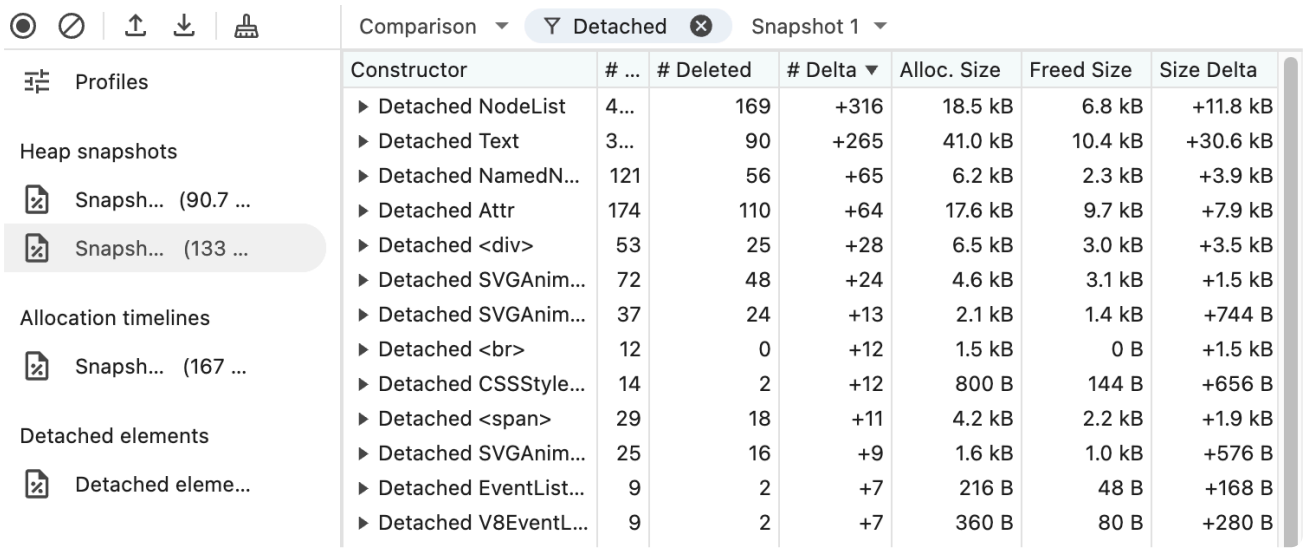
**2. Практический кейс: Поиск утечки памяти (Метод трех шагов)** Это классический алгоритм тестировщика для поиска "зависших" в памяти окон и компонентов.

1. **Базовый снимок:** Откройте тестируемую страницу. Нажмите корзину (Collect garbage) -> нажмите **Take snapshot**. У вас появился `Snapshot 1`.
2. **Сценарий:** Выполните действие на сайте. Например, откройте сложную форму регистрации, заполните пару полей, а затем **закройте** ее (вернитесь в исходное состояние).
3. **Финальный снимок:** Снова нажмите

   (это очень важно, чтобы браузер удалил закрытую форму из памяти) -> нажмите **Take snapshot**. У вас появится `Snapshot 2`.
4. **Сравнение:** Выберите в левом меню `Snapshot 2`. В верхней панели над таблицей данных измените выпадающий список с **Summary** (Сводка) на **Comparison** (Сравнение). Справа от него выберите для сравнения `Snapshot 1`.

   
5. **Анализ:** Теперь таблица показывает только то, что **добавилось** в память между первым и вторым снимком.
6. В строке фильтра (Class filter) введите слово `Detached` (Отсоединенные).
7. *Результат:* Если вы видите список элементов `Detached HTMLDivElement` или `Detached DOM node`, и их размер измеряется в мегабайтах, то вы нашли утечку памяти. Вы закрыли форму, но она осталась висеть в фоне. Вы можете нажать кнопку **Save profile**

   в левом меню рядом со снимком и прикрепить этот файл к баг-репорту.




| Constructor | # ... | # Deleted | # Delta ▼ | Alloc. Size | Freed Size | Size Delta |
|---|---|---|---|---|---|---|
| ▸ Detached NodeList | 4... | 169 | +316 | 18.5 kB | 6.8 kB | +11.8 kB |
| ▸ Detached Text | 3... | 90 | +265 | 41.0 kB | 10.4 kB | +30.6 kB |
| ▸ Detached NamedN... | 121 | 56 | +65 | 6.2 kB | 2.3 kB | +3.9 kB |
| ▸ Detached Attr | 174 | 110 | +64 | 17.6 kB | 9.7 kB | +7.9 kB |
| ▸ Detached  | 53 | 25 | +28 | 6.5 kB | 3.0 kB | +3.5 kB |
| ▸ Detached SVGAnim... | 72 | 48 | +24 | 4.6 kB | 3.1 kB | +1.5 kB |
| ▸ Detached SVGAnim... | 37 | 24 | +13 | 2.1 kB | 1.4 kB | +744 B |
| ▸ Detached <br> | 12 | 0 | +12 | 1.5 kB | 0 B | +1.5 kB |
| ▸ Detached CSSStyle... | 14 | 2 | +12 | 800 B | 144 B | +656 B |
| ▸ Detached  | 29 | 18 | +11 | 4.2 kB | 2.2 kB | +1.9 kB |
| ▸ Detached SVGAnim... | 25 | 16 | +9 | 1.6 kB | 1.0 kB | +576 B |
| ▸ Detached EventList... | 9 | 2 | +7 | 216 B | 48 B | +168 B |
| ▸ Detached V8EventL... | 9 | 2 | +7 | 360 B | 80 B | +280 B |

8. Когда вы сравниваете два снимка памяти (Snapshot 1 и Snapshot 2), таблица раскладывает изменения по следующим колонкам:

- **New (Новые):** Количество элементов этого типа, которые были созданы между первым и вторым снимком. *То, что сайт создает новые элементы при кликах – это нормально.*
- **Deleted (Удаленные):** Количество элементов, которые браузер ("сборщик мусора") успешно удалил за это же время.
- **Delta (Дельта количества):** Чистая разница (`# New` минус `# Deleted`).
    - Если значение с плюсом (например, +28), значит элементов стало больше. Если с минусом или ноль, то сайт отлично убирает за собой.
- **Alloc. Size (Выделенный размер):** Объем оперативной памяти, который забрали себе новые элементы (из колонки `# New`).
- **Freed Size (Освобожденный размер):** Объем памяти, который вернулся компьютеру после уничтожения старых элементов (из колонки `# Deleted`).
- **Size Delta (Дельта размера):** Итоговое изменение объема памяти (`Alloc. Size` минус `Freed Size`). Это ваш главный индикатор веса.

### Несколько сценариев по работе с утечками памяти

**Сценарий 1: Здоровое приложение (Утечек нет)**

- **Что видим:** Колонка `Alloc. Size` показывает рост, но и `Freed Size` показывает примерно такие же цифры. Столбцы `# Delta` и `Size Delta`

болтаются около нуля (или показывают небольшой плюс из-за кэширования, который перестает расти на 3-й раз).

- **Вердикт:** Бага нет. Приложение потребляет память для работы, но исправно ее освобождает.

**Сценарий 2: Классическая утечка (Баг)**

- **Что видим:** Вы закрыли все окна на сайте, но колонка `# New` огромная, а `# Deleted` равна `0` (или очень мала). Соответственно, `# Delta` постоянно показывает плюс, а `Size Delta` неуклонно растет в килобайтах или мегабайтах с каждым вашим кликом. В колонке `Freed Size` – пустота.
- **Вердикт:** Заводим дефект. Сборщик мусора не может удалить объекты. Память выделяется (`Alloc. Size`), но не освобождается (`Freed Size`).

**Сценарий 3: Разовая загрузка тяжелых данных (Не баг)**

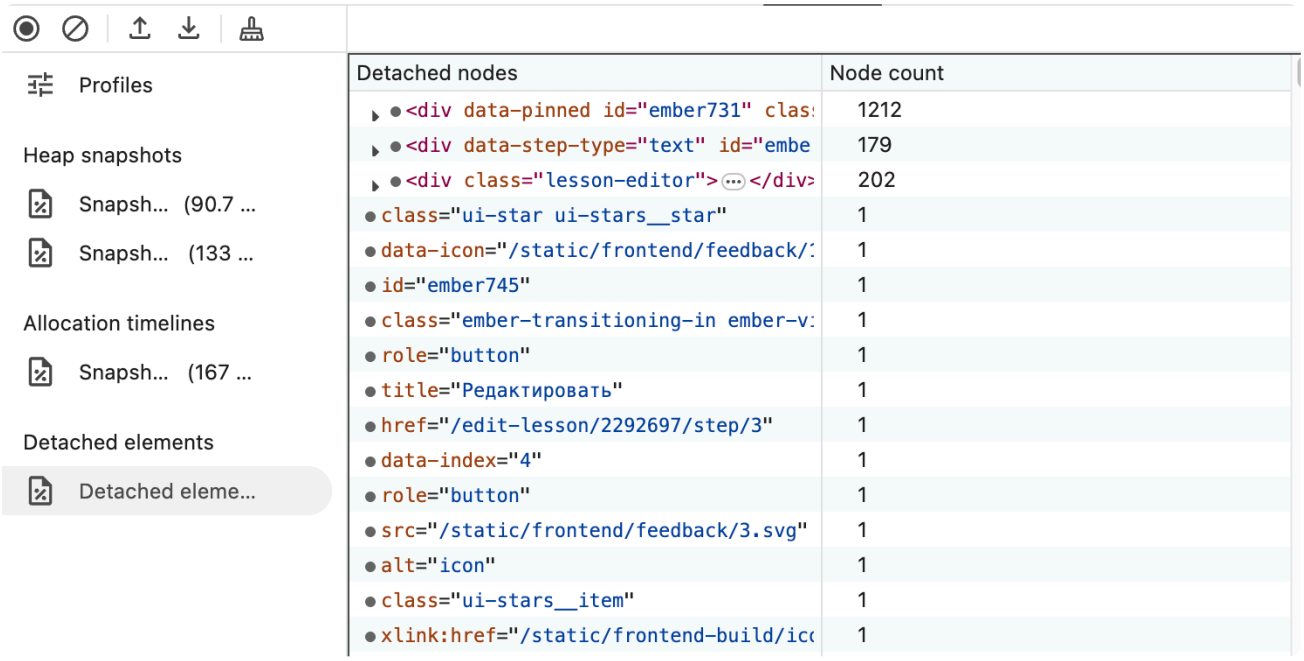
- **Что видим:** Вы перешли на новую страницу с огромным графиком. `Size Delta` резко подскочила на `15 MB`. Но при обновлении данных на этом графике дельта больше не растет.
- **Вердикт:** Бага нет. Это просто "тяжелая" страница, которая единоразово забрала память для отрисовки интерфейса. Утечка – это всегда *неконтролируемый бесконечный рост*, а не просто большой изначальный вес.

**3. Инструмент Detached Elements.** Также для поиска таких элементов есть отдельная опция **Detached Elements.**

1. **Для ее активации вернитесь в Profiles > выберите Detached Elements > нажмите**

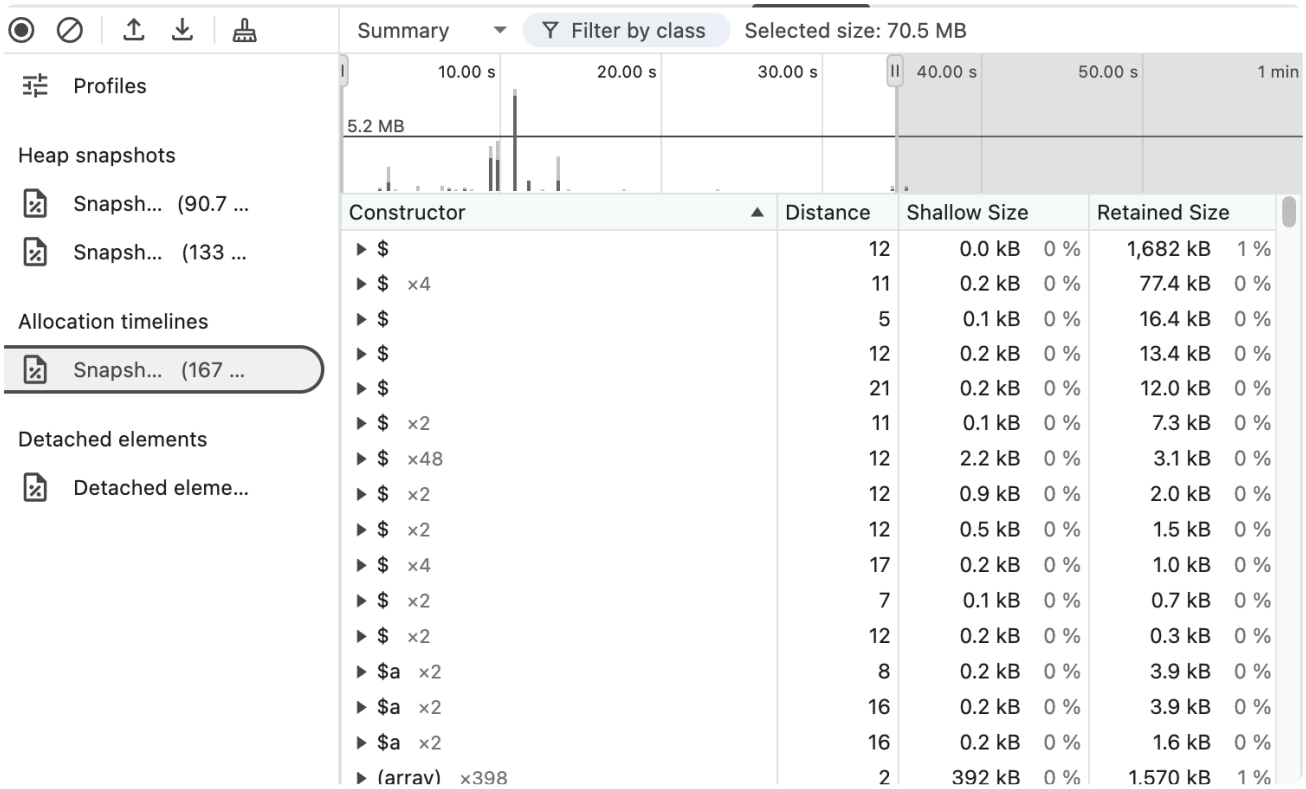
внизу.
2. Далее вы получите список, в котором не будет указан размер элемента, однако можно изучать другие закономерности. Например, на скрине ниже вы можете увидеть, что в первой строке в памяти зависло `1212` одинаковых элементов `div` с `id="ember731"`. Чуть ниже – `202` элемента с классом `lesson-editor`. Оставить в памяти одну случайную иконку (как нижние строки со значением `1`) – это мелкий недочет. Но когда в ОЗУ "мертвым грузом"

висит 1200+ кусков редактора уроков, то это похоже на критическую утечка. Даже если каждый из них весит крохи, их количество забьет процессор. Это готовый баг.




**3. Инструмент Allocation instrumentation on timeline (Запись выделения памяти во времени)** Heap Snapshot – это фотография, Allocation instrumentation on timeline – видеозапись.

- Выберите пункт `Allocation instrumentation on timeline` (вторая радиокнопка) и нажмите **Start**.
- Начните активно пользоваться сайтом: скроллите, открывайте вкладки, добавляйте товары.




- Сверху появится временная шкала с синими и серыми столбиками.
    - **Синие столбики:** Память, которая была выделена под ваши действия и *до сих пор находится в ОЗУ*.
    - **Серые столбики:** Память, которая была выделена, но затем успешно удалена "сборщиком мусора".

> *Кейс:* Если вы видите, что при каждом скролле каталога появляются новые синие столбики, которые не становятся серыми (даже если нажать иконку мусорного бака), это явный сигнал о том, что страница скоро зависнет. Вы можете увидеть синие столбцы, например, обновляя ленту новостей в любой соцсети.

**4. Инструмент Allocation Sampling (Сэмплирование выделений памяти)** По своей сути это "облегченная" версия предыдущего инструмента (Timeline).

- *В чем разница:* Обычная запись Timeline фиксирует абсолютно каждую операцию с памятью, из-за чего браузер начинает сильно тормозить в процессе самой записи. Сэмплирование (Sampling) работает иначе: оно делает "замеры" через равные промежутки времени.

- *Зачем это тестировщику (Кейс):* Если вам нужно протестировать сложный дашборд или онлайн-игру на предмет утечек памяти на протяжении **длительного времени** (например, 10–15 минут использования), обычный Timeline просто "повесит" ваш браузер. В этом случае вы выбираете Allocation Sampling, нажимаете Start, спокойно пользуетесь приложением 15 минут, а затем нажимаете Stop. DevTools выдаст вам список JavaScript-функций, отсортированный по тому, какие из них "съели" больше всего памяти за эту долгую сессию.

### Вкладка Application

**Вкладка Application (Приложение)** — это локальное хранилище или база данных веб-сайта прямо в вашем браузере. Здесь хранится всё, что веб-приложение запоминает о вас между обновлениями страницы или даже после перезагрузки компьютера: данные для авторизации, пользовательские настройки, недооформленные корзины товаров и системные файлы для работы в офлайн-режиме.

**Зачем это тестировщику в реальной работе:**

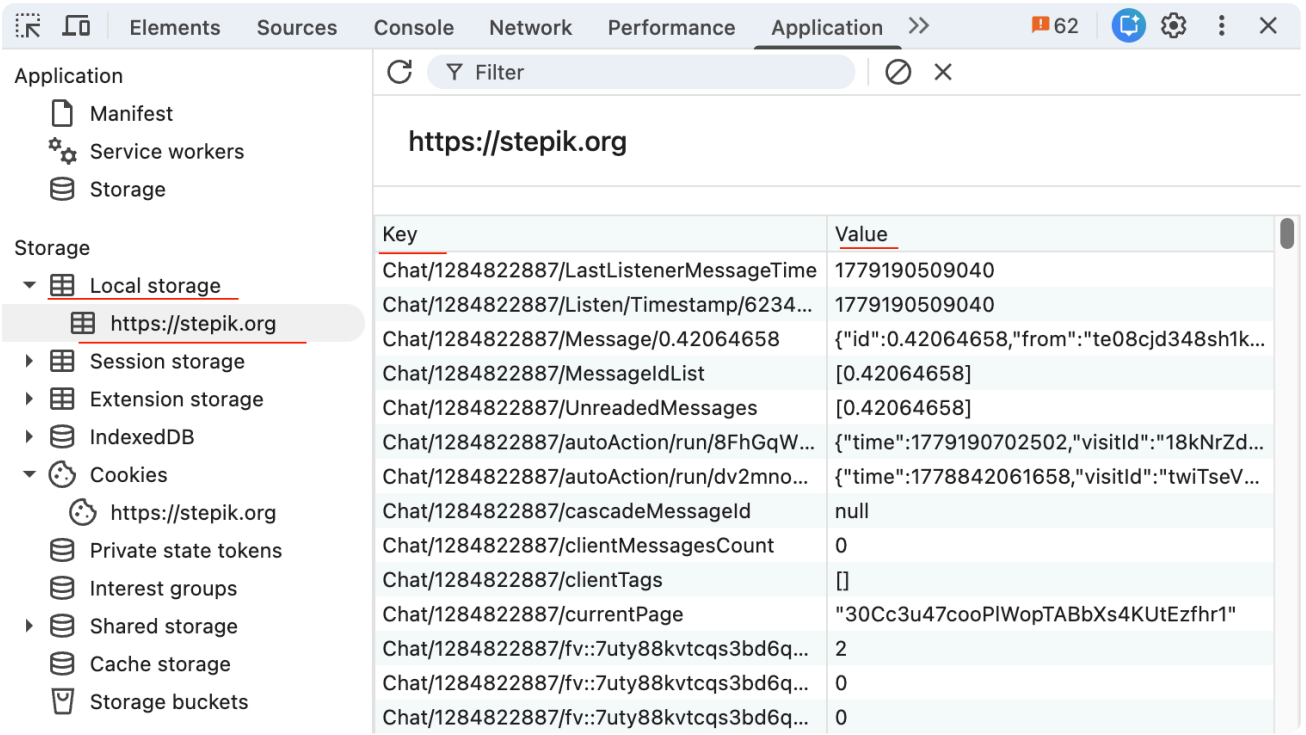
- **Проверка авторизации и безопасности:** Именно здесь хранятся токены доступа (ключи, по которым сервер понимает, что вы вошли в систему). Тестировщик может удалить их, чтобы проверить, заблокирует ли сайт доступ к приватному контенту.
- **Тестирование пользовательских состояний:** Изменение параметров (например, переключение темы оформления или флага "пользователь прошел обучение") напрямую в хранилище, чтобы проверить реакцию интерфейса, не прокликивая весь путь в приложении с нуля.
- **Полный сброс состояния (Hard Reset):** Очистка абсолютно всех данных сайта одним кликом, чтобы протестировать его поведение для "нового" пользователя.
- **Решение проблем с зависшим кэшем:** Поиск причин, почему разработчик выкатил обновление, а вы всё ещё видите старую версию дизайна или старую логику.

### Как работать со вкладкой Application?

Откройте DevTools и перейдите на вкладку Application. В левом боковом меню вы увидите длинный список разделов, сгруппированных по категориям (Storage, Background Services и т.д.). Рассмотрим самые важные инструменты для QA-инженера:

**1. Local Storage (Локальное хранилище)** Это постоянное хранилище браузера. Данные здесь не удаляются при закрытии вкладки или перезапуске ПК. Они хранятся до тех пор, пока вы или код сайта их не удалите.

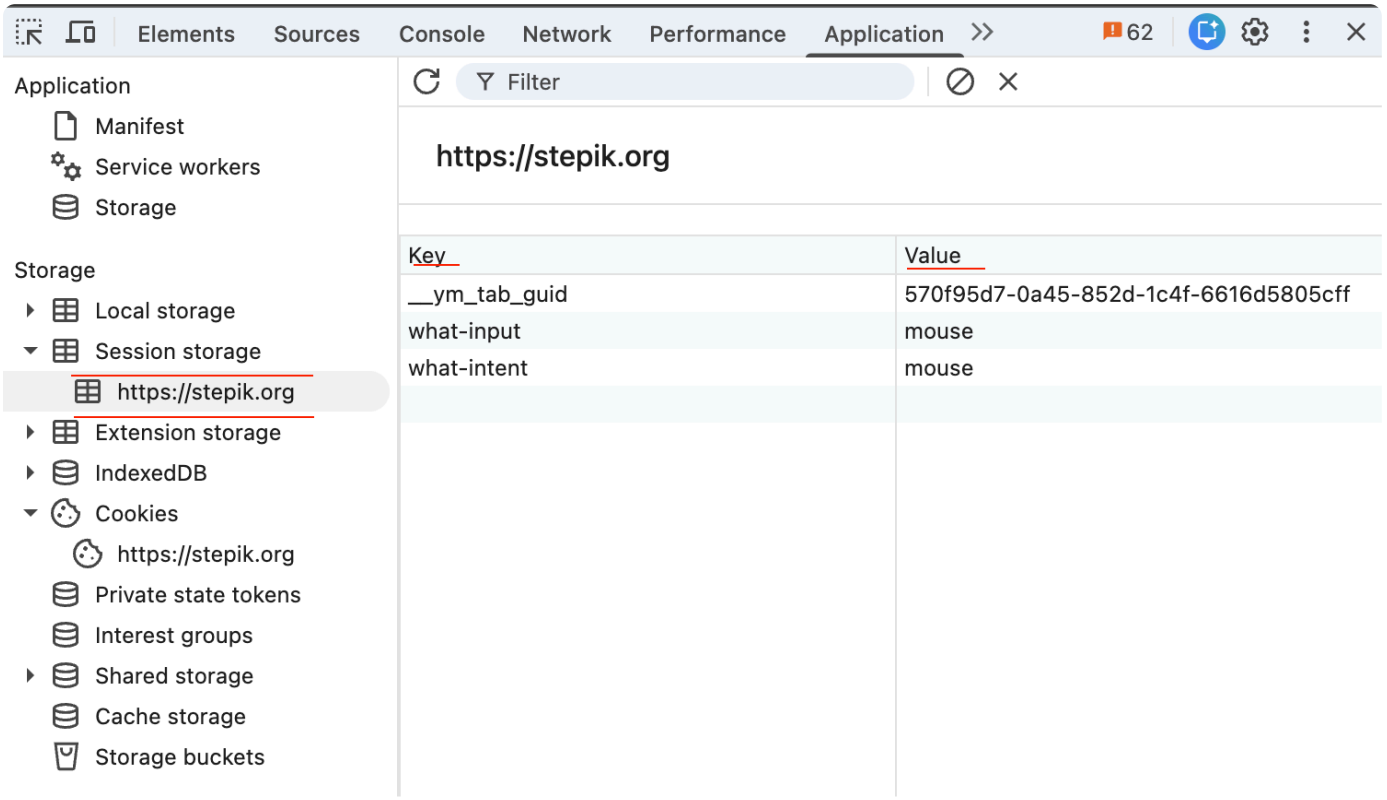
- Разверните пункт **Local Storage** в левом меню и кликните по адресу тестируемого сайта.
- Справа откроется таблица с колонками **Key** (Ключ) и **Value** (Значение).




- **Чтение значений:** Здесь можно найти понятные ключи, например `theme` со значением `dark` или `cookie_banner_accepted` со значением `true`.
- **Редактирование:** Дважды кликните по любому значению, чтобы изменить его. Представьте, что у вас есть признак, который отвечает за приветственный онбординг (знакомство с приложением). В нем можно поменять `true` на `false` у ключа. Далее стоит обновить страницу и проверить, покажет ли сайт всплывающие подсказки заново.
- **Удаление:** Выделите строку и нажмите клавишу `Delete` (или иконку крестика). Это отличный способ проверить, корректно ли приложение

восстанавливается при потере части локальных настроек.

**2. Session Storage (Сессионное хранилище)** Работает точно так же, как Local Storage, но с одним принципиальным отличием: данные живут только пока открыта конкретная вкладка браузера. Закрыли вкладку и всё стерлось.

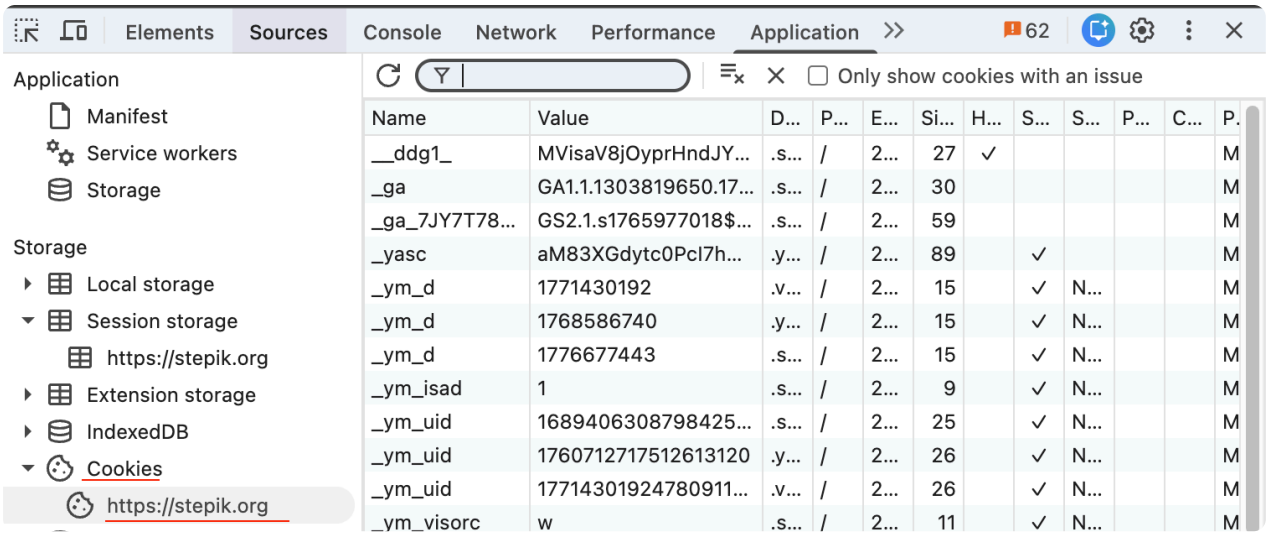



- Тестировщики проверяют здесь временные состояния. Например, при заполнении сложной многошаговой анкеты (шаг 1 из 3), промежуточные данные часто сохраняются сюда, чтобы не потеряться при случайном обновлении страницы (F5).

- **Кейс проверки изоляции:** Вы можете скопировать URL многошаговой формы, на которой уже заполнили половину полей, открыть ее в **новой** вкладке и проверить, подтянулись ли данные. Если они лежали в Session Storage, новая форма должна быть пустой (так как сессия строго привязана к своей вкладке).

**3. Cookies (Файлы куки)** Самый старый и строгий способ хранения данных. В отличие от Local Storage, куки автоматически отправляются на сервер при каждом вашем запросе.

- Разверните пункт **Cookies** и выберите домен вашего сайта.

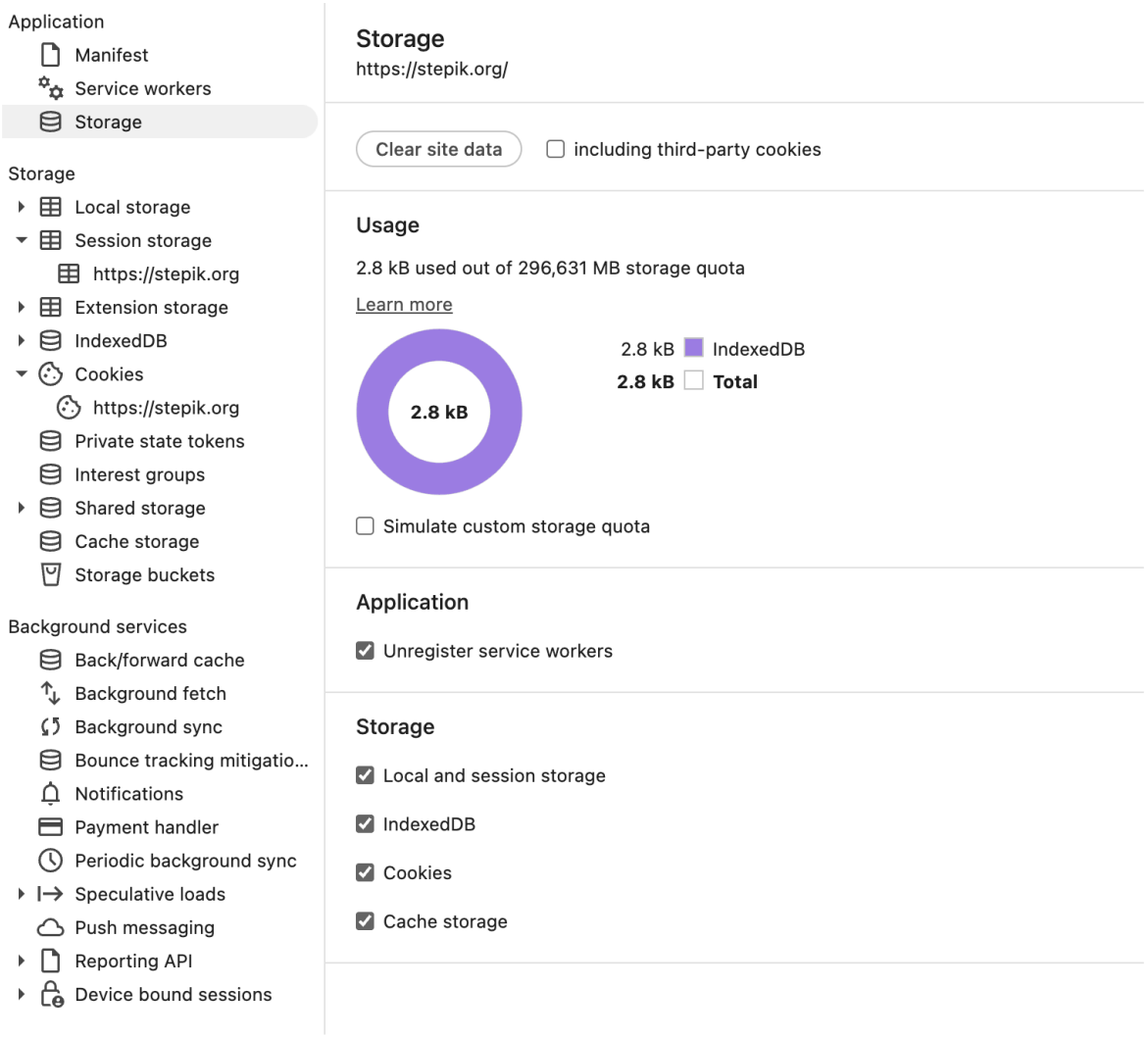



- **Кейс разлогина:** Найдите куки, отвечающие за авторизацию (обычно это ключи со словами `session`, `auth`, `token`), выделите строку и удалите ее. Обновите страницу. Если сайт не выбросил вас на форму входа и позволил совершить покупку, то это серьезный баг безопасности.

- **Проверка флагов безопасности:** Обратите внимание на колонки в таблице куки. Если в ключе хранится секретный токен доступа, там обязательно должны стоять галочки в колонках **HttpOnly** (запрещает вредоносным скриптам украсть этот куки) и **Secure** (разрешает передавать этот куки только по безопасному протоколу HTTPS). Если разработчик забыл поставить эти флаги – это уязвимость, можно заводить дефект.

**4. Storage -> Clear site data (Глобальная очистка)** Это кнопка для сброса всех хранилищ.

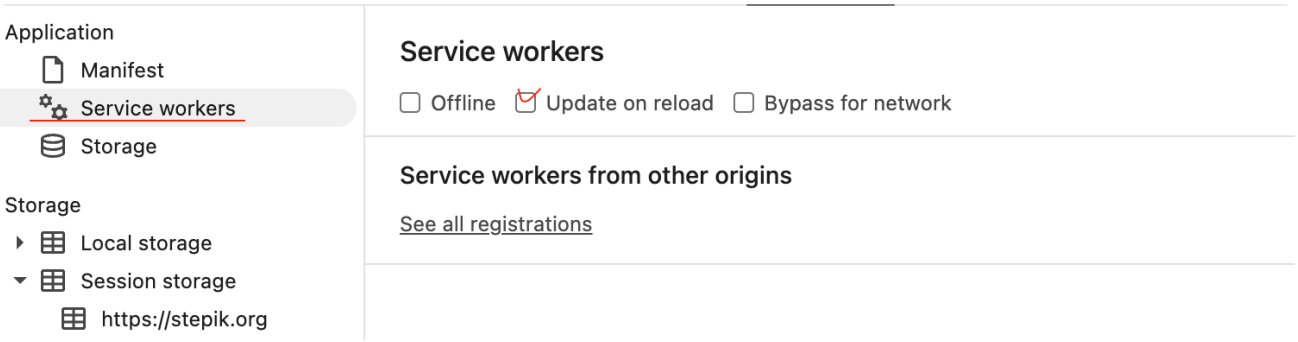
- В левом меню в самом верху (раздел Application) выберите пункт **Storage**.




- В правой панели появится диаграмма, показывающая, сколько места занимает сайт на вашем диске.
- Чуть ниже находится список с галочками (Local and session storage, Cookies, Cache storage).
- Убедитесь, что нужные галочки стоят, и нажмите кнопку **Clear site data** (Очистить данные сайта).
- После этого просто обновите страницу. Вы станете для сайта абсолютно новым, "чистым" пользователем. Это экономит массу времени по сравнению с ручной очисткой кэша через общие настройки браузера.

**5. Service Workers (Служебные воркеры)** Скрипты, которые работают в фоновом режиме. Они нужны для отправки push-уведомлений и загрузки приложения при отсутствии интернета (офлайн-режим).

- **Кейс "зависшего обновления":** Вы тестируете новую фичу, бэкенд выкатил обновление, но вы упорно видите старый интерфейс. Даже `Ctrl+F5` не помогает. Скорее всего, старый код намертво закэшировался в Service Worker.
- Перейдите в раздел **Service Workers** в левом меню.
- Поставьте галочку **Update on reload** (Обновлять при перезагрузке) в верхней части панели. Теперь браузер будет принудительно скачивать самую свежую версию воркера при каждом нажатии F5.




**Вкладка Security**

**Вкладка Security (Безопасность)** показывает, насколько безопасно соединение между браузером пользователя и сервером сайта. В первую очередь эта вкладка отвечает за проверку HTTPS-соединения и валидность SSL-сертификатов.

**Зачем это тестировщику в реальной работе:**

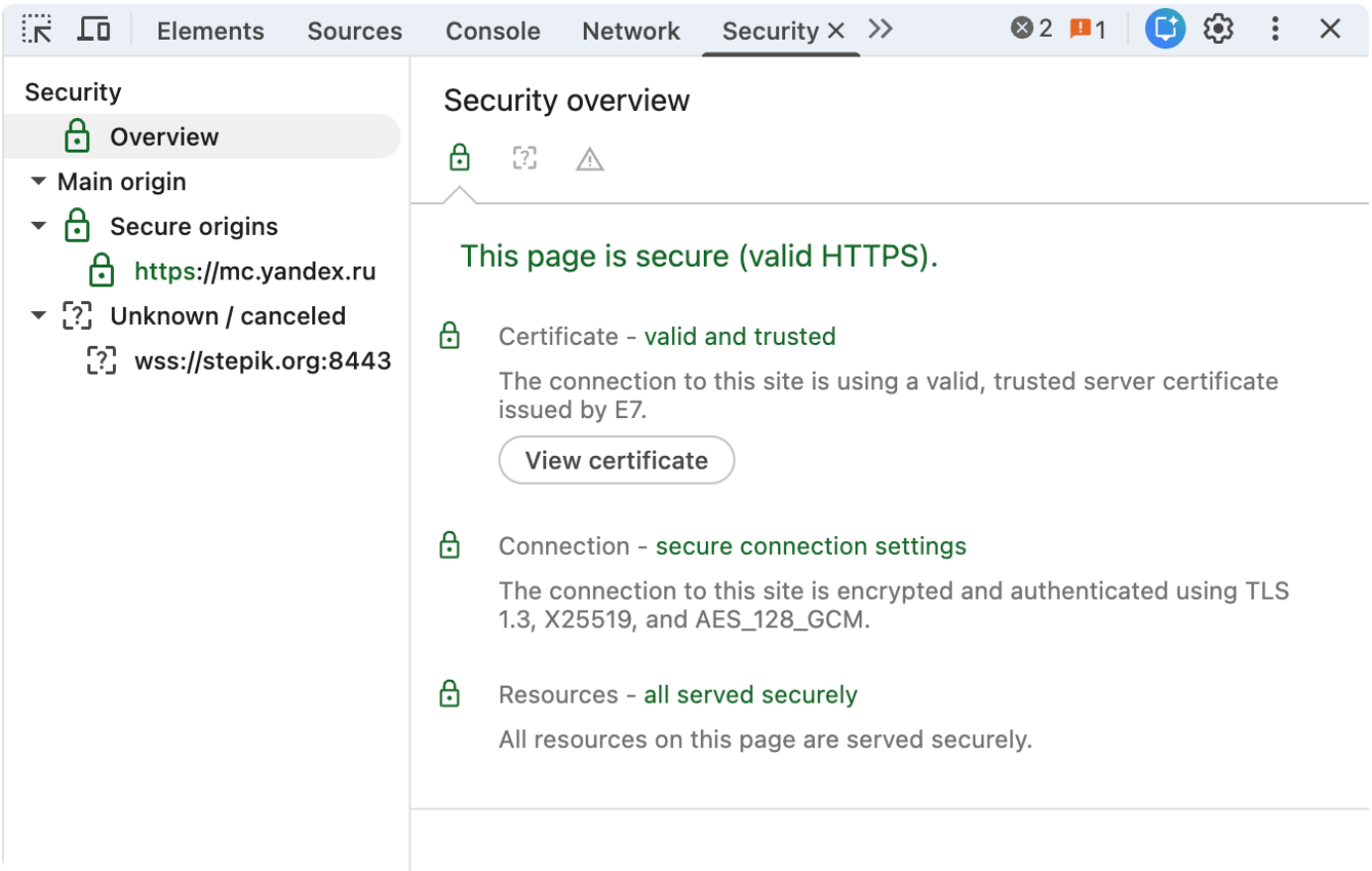
- **Проверка безопасности передачи данных:** Убедиться, что пароли, данные банковских карт и личная информация пользователей передаются в зашифрованном виде (по протоколу HTTPS), а не в открытом тексте.
- **Контроль срока действия сертификатов:** SSL-сертификаты имеют срок годности. Если он истечет, браузер заблокирует доступ к сайту, показав пользователю красное окно "Подключение не защищено". Тестировщик может заранее заметить истекающий сертификат на тестовом или боевом стенде.
- **Поиск "смешанного контента" (Mixed Content):** Сайт работает по безопасному HTTPS, но часть картинок или скриптов пытается загрузиться по небезопасному HTTP. Браузер принудительно заблокирует такие файлы,

и интерфейс "сломается" (например, пропадут иконки). Вкладка Security сразу укажет на эту проблему.

- **Проверка соответствия требованиям безопасности:** На банковских или государственных проектах есть строгие требования к версиям протоколов шифрования (например, запрещено использовать устаревший TLS 1.0). Здесь это можно быстро проверить.

**Как работать со вкладкой Security?**

Откройте DevTools и перейдите на вкладку **Security**. Стартовый экран (Security Overview) сразу дает общую оценку текущей страницы.




Рассмотрим основные сценарии работы для QA-инженера:

**1. Чтение общего статуса безопасности (Overview)** Как только вы открываете вкладку, вы видите главный статус:

- **This page is secure (valid HTTPS):** Зеленый статус. Всё отлично, соединение зашифровано, сертификат подлинный.

- **This page is not secure:** Красный статус. Соединение не зашифровано (используется обычный HTTP) или сертификат недействителен. Для любого современного сайта, где есть авторизация, это критический дефект.
- **This page is secure, but has mixed content:** Желтый статус (или предупреждение). Сам сайт безопасен, но часть файлов подтягивается из небезопасных источников.

**2. Проверка данных сертификата (View certificate)** Под главным статусом находится блок **Certificate**.

- Нажмите кнопку **View certificate** (Посмотреть сертификат). Откроется системное окно вашего компьютера с деталями.
- *Кейс для тестировщика:* Найдите в этом окне поле **Validity Period** (Действителен с ... по ...). Если вы тестируете проект и видите, что сертификат на тестовом стенде (или, что еще хуже, на проде) истекает через 3 дня, то немедленно заводите задачу или сообщайте системным администраторам (DevOps). Это спасет проект от внезапного падения.

**3. Анализ деталей подключения (Connection)** В блоке **Connection** написана техническая информация о том, как именно браузер договорился с сервером о шифровании.

- *Что здесь искать:* Обратите внимание на версию протокола (например, `TLS 1.2` или `TLS 1.3`). Если вы тестируете финансовое приложение, в требованиях (ТЗ) часто прописано, что старые версии TLS не поддерживаются. Если вы видите здесь `TLS 1.0` или `Obsolete connection settings` – это баг уязвимости.

**4. Поиск смешанного контента (Mixed Content) и панель Origins** В левом боковом меню вкладки Security вы увидите разделы со словом **Origin** (Источник). Там перечислен главный домен (Main origin) и все сторонние ресурсы, с которых сайт грузит данные (аналитика, шрифты, картинки).

- *Кейс:* Пользователь загрузил аватарку, но на странице вместо нее отображается "битая" иконка. Во вкладке Console могут быть общие ошибки, но вкладка Security даст точный ответ.

- Если на сайте есть проблема смешанного контента, в панели Origins небезопасный источник будет подсвечен красным или желтым.
- В правой панели появится предупреждение о Mixed Content и кликабельная ссылка **View requests in Network Panel** (Посмотреть запросы на панели Network).
- Нажмите на нее! DevTools автоматически перекинет вас во вкладку **Network** и отфильтрует список так, чтобы показать **только те файлы**, которые вызвали ошибку безопасности. Вы просто копируете URL этого "проблемного" файла и отправляете разработчику с описанием: *"Аватарка загружается по HTTP вместо HTTPS, поэтому браузер ее блокирует"*.

### Вкладка Lighthouse

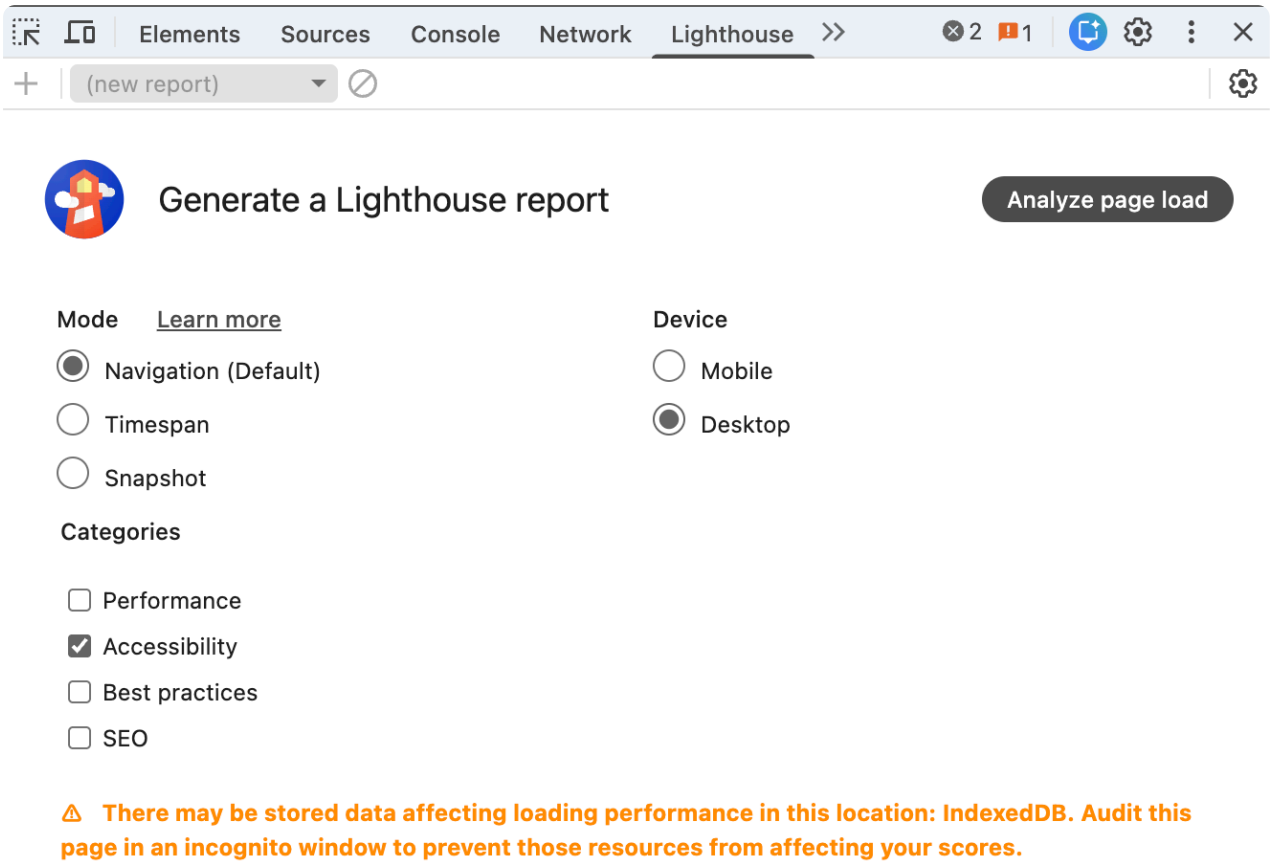
Вкладка Lighthouse (Маяк) – это встроенный автоматический аудитор вашего сайта. Он прогоняет серию автоматических тестов и выдает готовый отчет с оценками от 0 до 100 и конкретными рекомендациями по исправлению проблем.

**Зачем это тестировщику в реальной работе:**

- **Быстрая проверка нефункциональных требований:** Вам не нужно быть гуру SEO или экспертом по доступности для людей с ограниченными возможностями (a11y), чтобы найти базовые ошибки. Инструмент сам укажет, где в коде не хватает обязательных тегов или где кнопки слишком мелкие для нажатия пальцем.
- **Понятные метрики для бизнеса:** Менеджеры и заказчики любят понятные цифры. Зеленые показатели (90-100) в отчете – отличный аргумент, что релиз готов. Красные цифры – железный аргумент отправить задачу на доработку.
- **Готовые подсказки для баг-репортов:** Инструмент указывает прямые рекомендации по улучшению: "Сожмите вот эти 3 картинки (ссылки прилагаются), это сэкономит 2 секунды загрузки". Вы просто копируете эту информацию в баг-трекер.

### Как работать со вкладкой Lighthouse?

Откройте DevTools и перейдите на вкладку **Lighthouse**.




> **Важное правило:** Как и в случае с производительностью и памятью, запускайте аудит **только в режиме Инкогнито** (или в чистом профиле). Любое включенное браузерное расширение испортит ваши оценки, добавив свой код на страницу.

Рассмотрим алгоритм работы от запуска до составления отчета:

### 1. Настройки генерации отчета (Configuration)

Перед запуском нужно указать, в каких условиях будет проводиться аудит.

- **Mode (Режим):**
  - *Navigation (Навигация):* Загрузка страницы с нуля. Используется в 90% случаев для оценки стартовой загрузки.
  - *Timespan (Период времени):* Вы нажимаете Start, кликаете по сайту, нажимаете Stop. Аудит оценивает только ваши действия.
  - *Snapshot (Снимок):* Оценка текущего состояния страницы без перезагрузки (удобно для аудита открытых модальных окон).

- **Device (Устройство):** Mobile (Мобильное) или Desktop (ПК).

> *Важно:* Выбор Mobile автоматически применяет Throttling (замедление сети и процессора), чтобы сымитировать реальный смартфон, а также меняет размер окна браузера (Viewport).

- **Categories (Категории):** Выберите, что именно вы хотите проверить.
    - *Performance (Производительность):* Замеряет метрики LCP, CLS и другие.
    - *Accessibility (Доступность):* Проверяет цвета, контрастность, поддержку скринридеров (инструментов, которые читают вслух текст с сайта).
    - *Best Practices (Лучшие практики):* Проверяет безопасность, использование устаревшего кода и ошибки в консоли.
    - *SEO (Поисковая оптимизация):* Проверяет, "понятен" ли ваш сайт поисковым роботам Google или Яндекс.

### 2. Запуск аудита

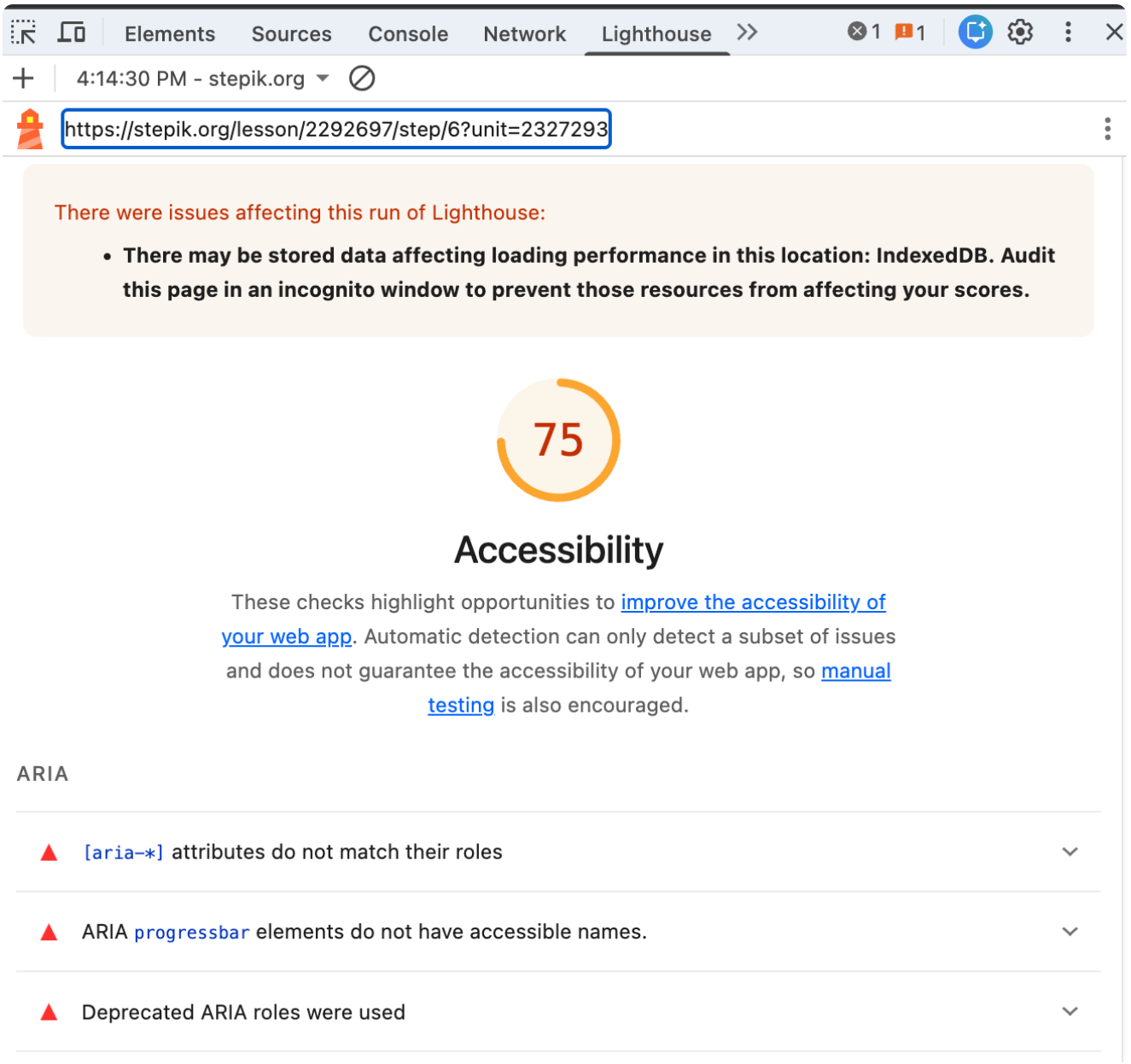
Выберите режим *Navigation*, устройство *Desktop*, оставьте все галочки в категориях и нажмите кнопку **Analyze page load** (Анализировать загрузку страницы).

Браузер начнет сам перезагружать страницу, менять ее масштаб и собирать данные. **Ничего не нажимайте и не переключайтесь на другие вкладки**, пока идет аудит, иначе результаты будут искажены.

### 3. Чтение отчета и поиск багов

После окончания проверки появится итоговый экран. В самом верху вы увидите круговые диаграммы с баллами от 0 до 100.

- **Зеленая зона (90–100):** Всё отлично, можно не трогать.
- **Желтая зона (50–89):** Есть проблемы, требующие внимания.
- **Красная зона (0–49):** Критические дефекты, которые нужно исправлять.



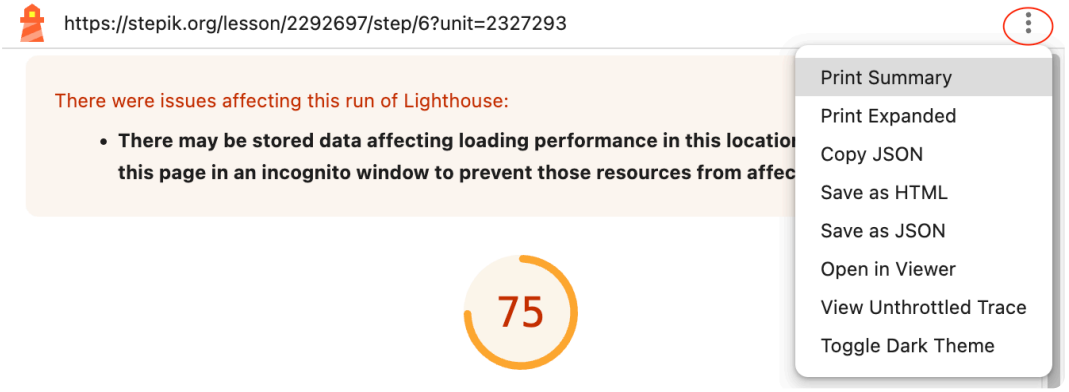

Прокрутите отчет ниже, чтобы увидеть детализацию по каждой категории. Внутри каждой секции есть блок **Failed audits (Непройденные проверки)** — это ваш источник багов:

- **Пример из секции Accessibility:** Вы видите ошибку `Background and foreground colors do not have a sufficient contrast ratio`. Раскройте ее. Lighthouse покажет конкретный кусок текста на странице, где серый шрифт сливается со светло-серым фоном и его невозможно прочитать.
- **Пример из секции SEO:** Ошибка `Links do not have descriptive text`. Раскройте список. Инструмент покажет ссылки вида "Узнать больше", которые не описывают, куда они ведут. По правилам SEO, ссылка должна быть понятной (например, "Узнать больше о тарифах").

- **Пример из секции Performance:** Предупреждение `Serve images in next-gen formats`. Инструмент прямо укажет, какие картинки загружены в устаревших форматах JPG/PNG, и предложит заменить их на WebP для ускорения работы.

### 4. Экспорт отчета

Если вы нашли много проблем, то их можно передать команде целиком. В правом верхнем углу отчета Lighthouse нажмите на иконку с тремя точками (меню настроек отчета).




Выберите **Save as HTML** (Сохранить как HTML) или **Save as JSON** (Сохранить как JSON). Вы получите готовый файл со всеми метриками и раскрывающимися списками ошибок, который можно просто прикрепить к задаче в Jira или Test IT.

**Итоговая информация по всем вкладкам**

**Elements (Элементы)**:

- Позволяет инспектировать и редактировать HTML-код и стили CSS (отдельная вкладка *Styles*) в реальном времени, а также видеть структуру DOM-дерева.
- *DOM-дерево* - структура HTML или CSS-документа в древовидной форме.
- Используется в тестировании фронтенда и графического интерфейса путем сравнения мокапа (дизайна страницы) и готовой страницы. Обычно на мокапе указаны требования к размеру, форме, цвету, шрифту и прочим стилям. Нужно убедиться, что на финальной странице frontend-разработчик реализовал их верно.

**Console (Консоль)**:

- Используется для выполнения JavaScript-кода, отладки ошибок и отображения вывода console.log() и других сообщений.
- Для тестировщика важно наблюдать за логом предупреждений и ошибок. Если говорить про автоматизаторов, то эту вкладку удобно использовать для дебаггинга (поиск ошибок и их исправления) в автотестах.

**Sources (Исходники)**:

- Позволяет отлаживать JavaScript-код, устанавливать точки остановки, просматривать стек вызовов, исследовать переменные и многое другое.

**Network (Сеть)**:

- Предоставляет информацию о сетевом трафике, включая запросы, ответы, загрузку ресурсов, время выполнения и размер файлов.
- Самая главная вкладка для тестировщиков. Позволяет анализировать отправленные запросы и ответы, статус-коды, время обработки, размер запроса и ответа, дублирующие запросы. В случае обнаружения дефекта лучше всего использовать сохранение данных о запросе и ответе в специальном формате HAR, который позволит разработчику увидеть ту же самую картину на своем браузере, что и вы. Принцип работы есть в видео-уроке.

**Performance (Производительность)**:

- Используется для профилирования производительности веб-приложения, анализа времени выполнения и выявления узких мест.
- Позволяет сделать "скриншот" работы веб-приложения с учетом основных активностей по загрузке и времени обработки в виде диаграмм и графиков. Может использоваться для анализа производительности.

**Memory (Память)**:

- Позволяет анализировать использование памяти веб-приложения, искать утечки памяти и оптимизировать работу приложения.

**Application (Приложение)**:

- Предоставляет информацию о хранилище данных, такие как куки, локальное хранилище, IndexedDB и др. Вы можете просматривать и редактировать данные.

- Описанные выше данные могут использоваться в специфических проверках, связанных с хранилищами данных. Например, правильно ли сохраняются куки, тестирование аутентификации, работа с данными и т.д.
- Там же можно очищать кэш и куки

**Security (Безопасность)**:

- Помогает обнаруживать потенциальные уязвимости и проблемы безопасности на веб-сайте, такие как смешанный контент, устаревшие протоколы и др.
- Используется для тестирования безопасности. Содержит информацию про протоколы шифрования и актуальность сертификатов для работы по HTTPs.

**Lighthouse (Маяк)**:

- Этот инструмент работает поверх вкладки Audits и предоставляет более подробные анализы и рекомендации для улучшения веб-сайта.
- Можно использовать для экспресс-аудита ресурса в рамках тестирования доступности, производительности, SEO-оптимизации и др.

**Несколько обязательных правил по тестированию веб-приложений:**

1. **Держите DevTools открытым на вкладке Network (Сеть) и Console.** Это позволит вам сразу видеть, если какой-то запрос к серверу упал с ошибкой, даже если визуально на сайте ничего не сломалось.

2. **Используйте галочку Disable cache (Отключить кэш).** Во вкладке Network есть чекбокс `Disable cache`. Если он включен, браузер не будет кэшировать файлы (картинки, стили, скрипты), **пока открыт DevTools**. Это идеальный способ тестировать верстку и новые функции, не удаляя кэш вручную каждый раз.

3. **Применяйте Hard Reload (Жесткая перезагрузка).** Если вам нужно точечно сбросить кэш только для текущей страницы, не нужно чистить всю историю браузера. При открытом DevTools кликните правой кнопкой мыши по иконке обновления страницы (слева от адресной строки) и выберите **"Очистить кэш и жестко перезагрузить"** (Empty Cache and Hard Reload). Либо используйте горячие клавиши `Ctrl + F5` (`Cmd + Shift + R` на Mac).

4. **Чередуйте режимы для разных проверок:**

    - **Обычное окно браузера:** Используйте для тестирования "возвращающихся" пользователей (проверка того, что сессия не "протухла", корзина сохранилась, а настройки профиля не сбились).
    - **Режим Инкогнито (`Ctrl + Shift + N`):** Используйте для тестирования "с чистого листа" (регистрация новых пользователей, проверка работы сайта без ваших рабочих куки-файлов и без влияния ваших браузерных расширений). При закрытии окна Инкогнито все данные сессии удаляются автоматически.

# 35. ПРАКТИКА 10: Работа в DevTools

**Работа в тренажере**

Для практики использования DevTools предлагаю использовать данный тренажер из 30 задач, который я специально разработал для своих продуктов.

Пример решения от меня для этого задания не нужен. Если у вас возникнут затруднения по выполнению, то для каждого задания есть подсказка.

**Тренажер доступен по ссылке https://www.rusau.net/trainer-13**

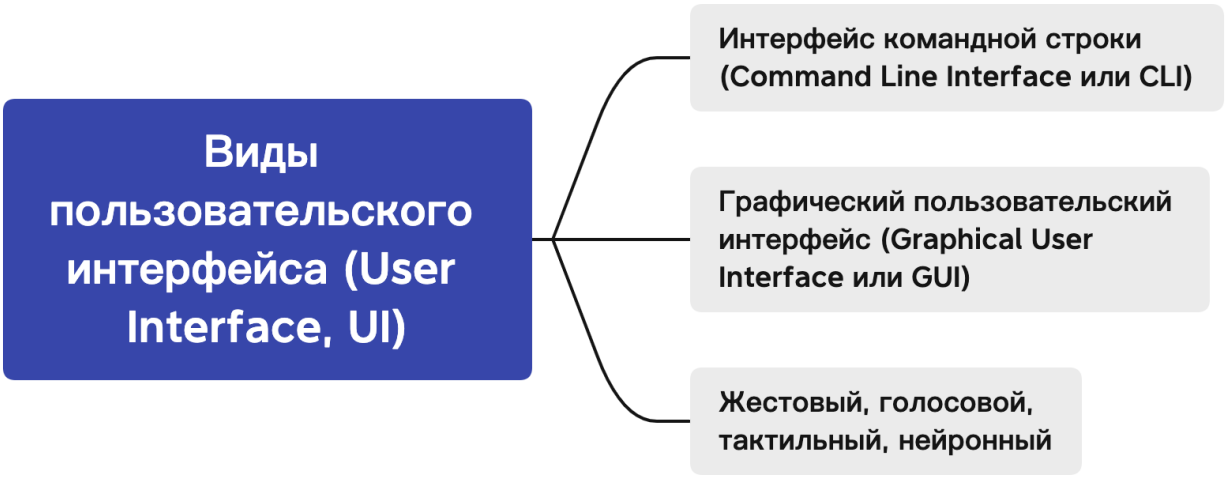
**Пароль для начала работы: *devtools***

*Это лишь один из примеров тренажеров, а у меня их более 20 в отдельном наборе на Stepik. Изучить состав набора можно здесь.*

ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЙ

# 36. Виды UI и верстки (версия 2026)

**Виды пользовательского интерфейса**




Источник

Если вы новичок в тестировании веб-приложений, важно понимать, как устроена верстка сайта. От этого зависит, как интерфейс будет отображаться на разных устройствах и на какие уязвимые места обращать внимание при поиске багов.

Вот основные виды верстки, с которыми вы можете столкнуться на практике, и способы их определить по коду.

### 1. Фиксированная верстка

Это сайт с жестко заданной шириной. Как правило, он выглядит нормально только на экранах с нужным разрешением. На маленьких экранах появляется горизонтальный скролл, а на больших – интерфейс "прилипает" к левому краю или центрируется, оставляя огромные пустые поля по бокам.

- **Как определить по коду:** В CSS главного контейнера (например, `<body>` или `<div class="container">`) задана строгая ширина в пикселях.
    - *Пример в CSS:* `width: 1024px;` или `width: 1200px;`

- **Что проверять тестировщику:** Появление горизонтальной полосы прокрутки (скролла) при уменьшении окна браузера. Наличие горизонтального скролла на современных сайтах почти всегда считается дефектом.

**2. Резиновая верстка**

Ширина элементов задается в процентах. Контент подстраивается под размер окна браузера: сайт пропорционально "тянется" или сжимается.

- **Как определить по коду:** В CSS ширина блоков указана в процентах (%) или относительных единицах ширины экрана (vw - viewport width).
    - *Пример в CSS:* `width: 100%;` или `width: 50vw;`
- **Что проверять тестировщику:** Поведение блоков на экстремально узких и очень широких экранах. Часто появляются проблемы с читаемостью: текстовые строки становятся слишком длинными на широком мониторе, а изображения непропорционально растягиваются.

**3. Табличная верстка**

Устаревший подход, где каркас страницы строится с помощью HTML-таблиц. Сейчас практически не используется в современных веб-проектах, но повсеместно встречается в HTML-письмах (email-рассылках) и очень старых корпоративных системах (админках).

- **Как определить по коду:** В исходном коде (вкладка Elements) структура строится не на блоках `<div>`, а на табличных тегах.
    - *Пример в HTML:* Использование конструкции `<table>`, `<tr>` (строка) и `<td>` (ячейка) для позиционирования контента.
- **Что проверять тестировщику:** Таблицы очень хрупкие. Если в одну из ячеек добавить слишком длинный текст без пробелов или большое изображение, "поехать" может вся структура таблицы.

**4. Блочная верстка (div-верстка)**

Исторический преемник табличной верстки. Элементы структурируются с помощью блочных тегов и CSS-свойств обтекания или позиционирования.

- **Как определить по коду:** Обилие тегов `<div>` в HTML и использование классических свойств позиционирования.
    - *Пример в CSS:* `display: block;`, `float: left;`, `position: absolute;` или `position: relative;`.
- **Что проверять тестировщику:** Корректность отступов (`margin` и `padding`), порядок элементов в потоке. Особое внимание: наложению блоков друг на друга при изменении масштаба страницы в браузере (например, при 150%).

**5. Адаптивная (и отзывчивая) верстка**

Современный стандарт. Сайт подстраивается под разные экраны (мобильный, планшет, десктоп), кардинально меняя структуру. Например, на ПК вы видите меню в шапке, а на мобильном телефоне оно прячется в кнопку "бургер".

- **Как определить по коду:** В CSS активно используются медиа-запросы (Media Queries). Они применяют разные правила при достижении контрольных точек ширины экрана (breakpoints).
    - *Пример в CSS:* `@media (max-width: 768px) { .menu { display: none; } }` (Этот код скроет меню, если ширина экрана меньше 768 пикселей).
- **Что проверять тестировщику:** Переключение между разрешениями (сужайте окно браузера плавно и следите за моментами перестроения блоков). Обязательно проверяйте кликабельность "бургер-меню" на мобильных устройствах, скрытие лишних элементов и читаемость шрифтов.

**6. Flex-верстка (Flexbox)**

Современный инструмент CSS для гибкого выравнивания элементов. Позволяет легко распределять пустое пространство и выравнивать блоки по вертикали и горизонтали (в рамках одной оси).

- **Как определить по коду:** У родительского контейнера задано свойство Flex.
    - *Пример в CSS:* `display: flex;` вместе со свойствами выравнивания, такими как `justify-content: center;` или `align-items: center;`.

- **Что проверять тестировщику:** Поведение элементов при увеличении объема контента. Например, если в карточку товара добавить очень длинное название, тестировщик проверяет: не выдавливает ли этот текст соседние элементы и переносится ли он на новую строку (за это отвечает свойство `flex-wrap: wrap;`).

### 7. Grid-верстка (CSS Grid)

Самый мощный современный стандарт для создания макетов. В отличие от Flexbox, который работает только в одном измерении (строка ИЛИ столбец), Grid позволяет строить сложные двумерные сетки (строки И столбцы одновременно), напоминая невидимую шахматную доску.

- **Как определить по коду:** У главного контейнера задано свойство сетки.
    - *Пример в CSS:* `display: grid;`, а также свойства, задающие размеры колонок: `grid-template-columns: 1fr 1fr 1fr;` (создает три равные колонки) и `gap: 20px;` (расстояние между блоками).
- **Что проверять тестировщику:** Поведение сетки при динамических данных. Например, если в ряду должно быть 3 карточки товара, а база данных вернула только 2, сетка не должна сломаться, а пустая ячейка должна корректно зарезервировать место. Также важно проверять размеры "дыр" (gap) между элементами на узких экранах.

**Рекомендации**

1. Сайт всегда нужно проверять на адаптивность и валидность данных (больше для веб-дизайнеров и веб-разработчиков).
2. Для последнего можно использовать валидатор
3. Битые ссылки - расширение для Chrome Check My Links

*Конкретные примеры тестирования графического интерфейса есть в следующих уроках.*

**Другие полезные расширения и инструменты**

1. Perfect Pixel
2. Responsive Viewer
3. Web Developer Form Filler
4. ColorZilla
5. EditThisCookie

6. Clear Cache
7. Loom
8. Redux DevTools

## Презентация

Презентация

# 37. Тестирование веб-элементов и веб-форм (версия 2026)

### Основные инструменты для тестирования GUI у тестировщика

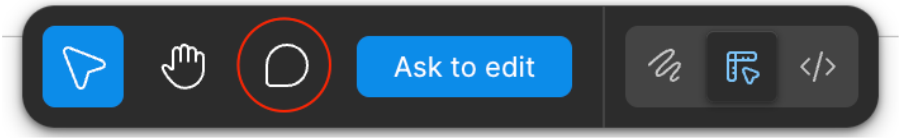
- Требования
- Макет (чаще всего в Figma)
- Сам веб-сайт или приложение с открытым DevTools (вкладки Elements, Styles, Computed)

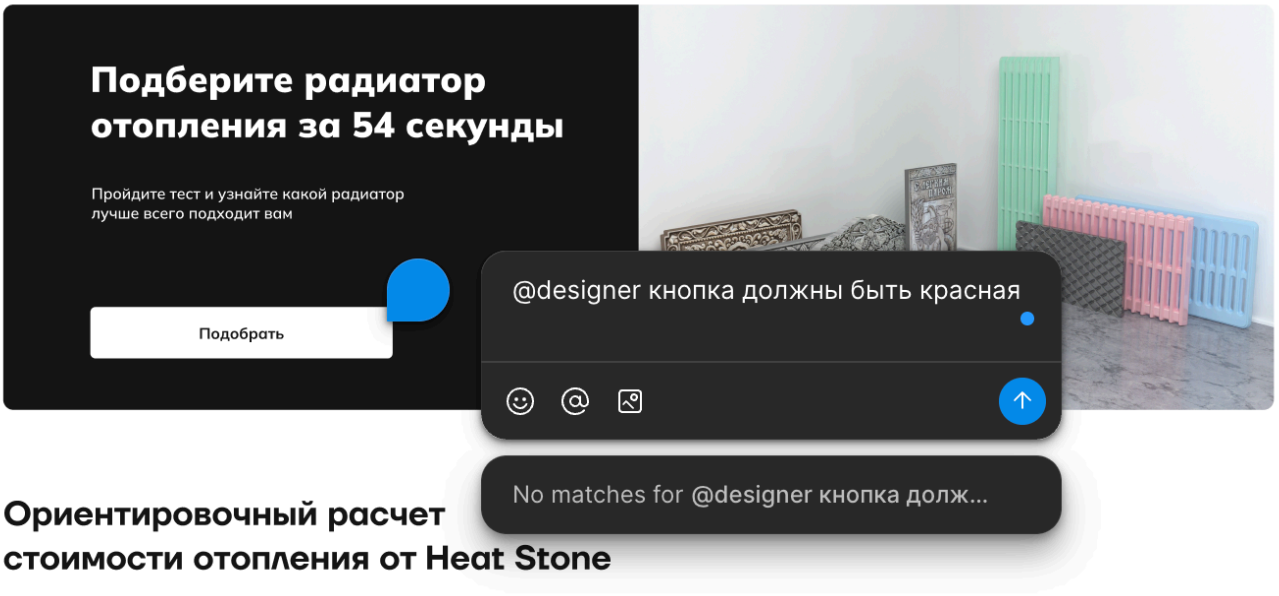
### Алгоритм работы

1. До получения готового приложения нужно провести анализ требований и сравнить их с реализацией на макете.

2. Если обнаружены противоречия, то нужно оставить комментарий в багрекинговой системе к пользовательской истории с упоминанием ответственных через @.

3. В Figma это можно сделать через нажатие на кнопку **Comment** внизу страницы > навести на нужный элемент > ЛКМ > добавить комментарий (через @ можно упомянуть дизайнера).






4. После согласования макета и получения рабочего приложения, вы можете приступить к самому тестированию.

5. Во вкладке с сайтом откройте *Devtools > Elements > Styles или Computed*, а в соседних вкладках откройте требования и макет. Подробнее о работе этих вкладок вы можете узнать в уроке Chrome DevTools.

6. Для перемещения по макету в Figma, используйте режим **Hand tool**

.

7. Для изучения информации по отдельному элементу используйте режиме **Move**

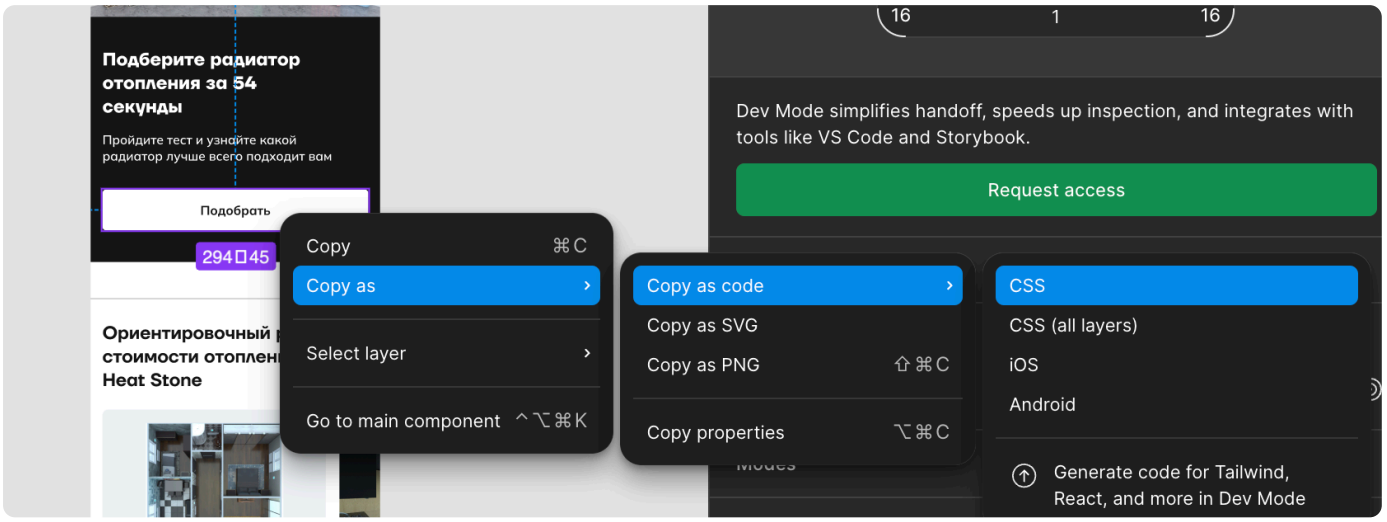
.

8. Выбрав нужный элемент (иногда нужно двойное нажатие), сравните информацию о нем в Figma и во вкладке Computed, они должны совпадать.

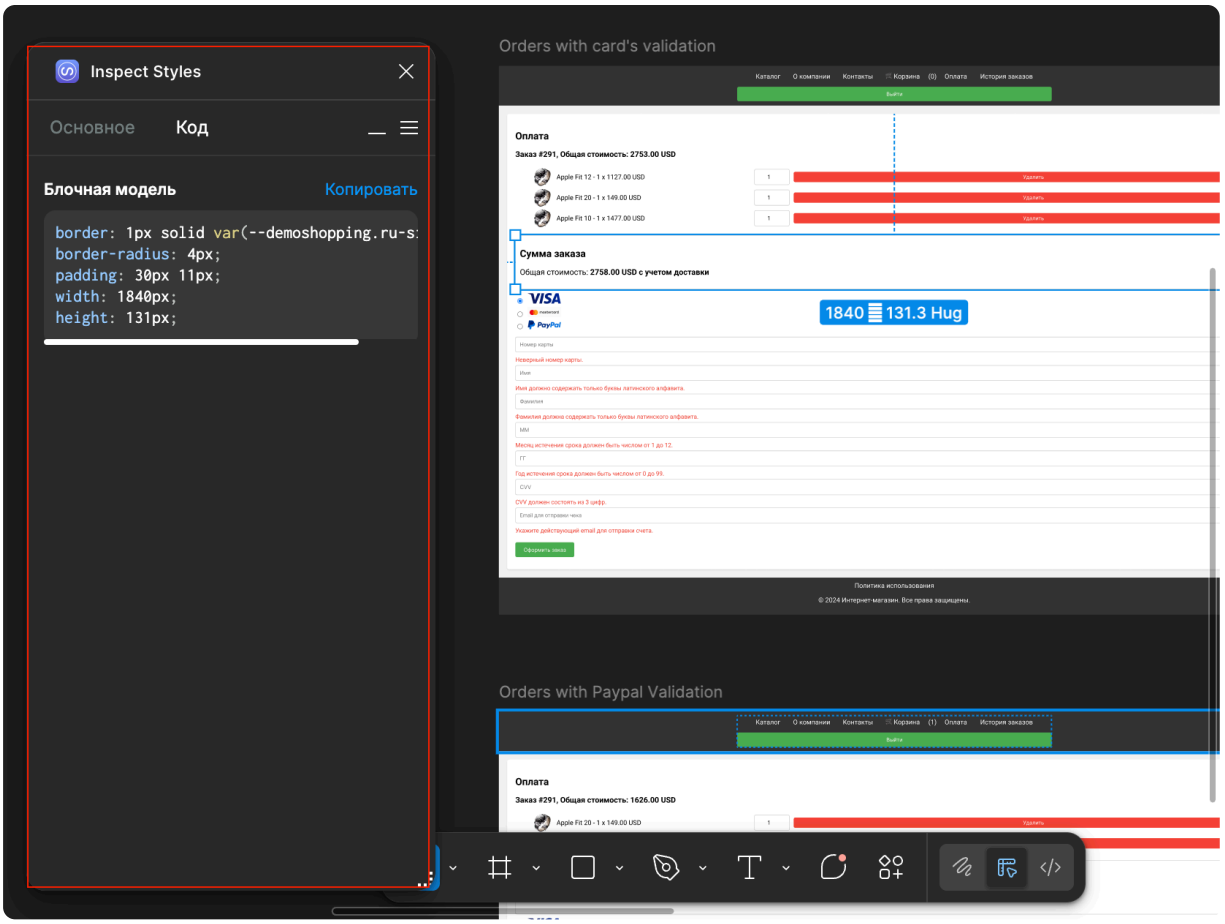
9. Раньше в Figma можно было видеть CSS-стили бесплатно, но сейчас это платная функция и специальный режим Dev Mode.

*Есть 2 способа это обойти:*

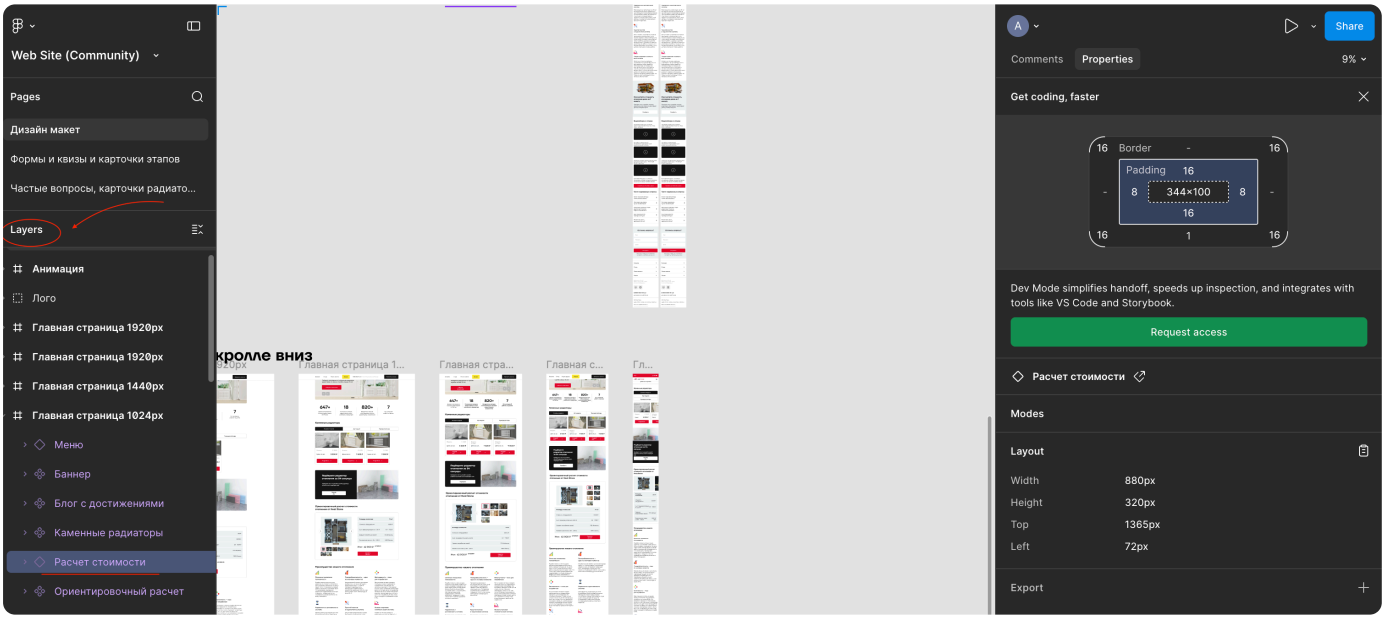
- *Способ 1:* ***Выберите нужный элемент > ПКМ > Copy as CSS***. Далее вставить в любой текстовый редактор и сравнить с кодом в Styles или Computed.




- *Способ 2:* **использовать бесплатный плагин для проекта [Inspect Styles]**. На странице плагина нажать ***Open in*** *> выбрать нужный проект > дождаться автоматического перенаправления на страницу проекта*. Плагин будет доступен в отдельном окне. При выборе нужного элемента, вы увидите информацию обо всех стилях.




9. Также вы можете искать нужные элементы в меню **Layers**.




10. В случае несоответствия вы должны завести баг.

> Если у вас много визуальных багов на одной странице, то их можно объединить в один отчет о дефекте.

## Основные типы проверок GUI и веб-форм

В первую очередь советую ориентироваться на видео-версию уроков. Данный перечень – это лишь часть всех проверок, которые можно осуществлять.

Всегда помните, что сначала нужно проводить позитивное тестирование с валидными данными, а уже после использовать негативные кейсы и вызывать ошибки. Обязательно проверяйте валидацию как на клиенте (в браузере), так и на сервере (через API или базу данных), так как клиентскую защиту легко обойти.

**Тестирование текстовых полей (Input/Textarea):**

- *Примеры:* Проверка граничных значений (ввод максимального и минимального допустимого количества символов), копирование и вставка текста превышающего лимит, ввод пробелов в начале и конце строки (проверка трима), проверка ввода спецсимволов и эмодзи.

**Тестирование безопасности полей ввода (XSS и SQLi):**

- *Примеры:* Ввод базовых SQL-инъекций (например, `OR 1=1`), ввод выполняемых JS-скриптов (например, `<script>alert('Bug')</script>`), проверка экранирования HTML-тегов при их отображении на странице после сохранения. Такие проверки лучше согласовывать с командой и никогда не тестировать на ресурсах, которые вы используете каждый день вне работы.

**Тестирование полей для URL:**

- *Примеры:* Ввод ссылок без указания протокола (http/https), ввод локальных IP-адресов, проверка слишком длинных URL, обработка ссылок с кириллическими доменами (Punycode).

**Тестирование загрузки файлов (File Upload):**

- *Примеры:* Загрузка файлов с невалидными или подмененными расширениями (например, .exe переименованный в .jpg), попытка загрузить пустой файл (0 байт) или файл превышающий лимит, проверка загрузки через "Drag-and-Drop" (перетаскивание), прерывание сети во время загрузки тяжелого файла.

**Тестирование полей ввода номеров телефонов:**

- *Примеры:* Ввод номеров в разных международных форматах, попытка скопировать и вставить буквы в цифровое поле, проверка работы автоматической маски ввода (например, когда скобки и тире подставляются сами).

**Тестирование полей для email:**

- *Примеры:* Ввод email без символа "@" или точки, использование пробелов внутри адреса, проверка очень длинных доменных зон, проверка чувствительности к регистру.

**Тестирование полей для ввода паролей:**

- *Примеры:* Проверка маскирования символов (ввод отображается точками или звездочками), работа иконки "показать/скрыть пароль", проверка запрета на копирование введенного пароля из поля, проверка индикатора сложности пароля.

**Тестирование динамических элементов и состояния кнопок:**

- *Примеры:* Двойной клик по кнопке "Отправить" (проверка защиты от дублирования запросов), появление лоадеров (спиннеров) на время ожидания ответа от сервера, визуальная блокировка кнопки до заполнения всех обязательных полей.

**Многостраничные формы (Wizard-формы):**

- *Примеры:* Сохранение черновика при переходе на следующий шаг, корректное возвращение на предыдущий шаг без потери введенных данных, попытка перейти на финальный шаг по прямой ссылке в обход первого шага.

**Тестирование календаря (Datepicker):**

- *Примеры:* Выбор нелогичных дат (например, дата рождения в будущем или бронь отеля в прошлом), обработка високосных годов (29 февраля), попытка вручную написать дату буквами в поле в обход визуального виджета календаря.

**Тестирование чекбоксов и радио-кнопок:**

- *Примеры:* Клик по самому тексту (лейблу) рядом с чекбоксом (он должен нажиматься, а не только маленький квадратик), проверка состояния по умолчанию, взаимоисключающий выбор в радио-группах (нельзя выбрать два пола одновременно).

**Тестирование выпадающих списков (Dropdowns):**

- Примеры: Прокрутка очень длинных списков, работа поиска внутри выпадающего меню, корректная обрезка слишком длинного текста опции (чтобы не ломалась верстка), выбор опции с помощью стрелок на клавиатуре.

**Тестирование капчи (CAPTCHA):**

- *Примеры:* Истечение времени действия капчи (подождать 10 минут перед отправкой), работа кнопки обновления картинки, проверка аудио-капчи для слабовидящих.

**Адаптивность (Responsive Design) и масштабирование:**

- *Примеры:* Проверка наслоения элементов при масштабе страницы 150% или 200%, проверка работы элементов в портретной и ландшафтной ориентациях на мобильных устройствах, тестирование сжатия элементов при сужении окна браузера.

**Доступность (Accessibility) и Usability:**

- *Примеры:* Навигация по всей форме исключительно с помощью клавиши Tab (фокус должен двигаться логично сверху вниз), проверка контрастности цвета текста и фона, наличие визуального выделения (Focus State) активного поля ввода.

**Полезные материалы**

1. Старая версия https://youtu.be/ZCdw3Js887E
2. Схема из урока https://drive.google.com/drive/u/3/folders/1-ncxu18_9RNEr6UFwvKlrU5RPy-nYWPC
3. Чек-лист для тестирования веб-форм. Можно сразу использовать в работе https://docs.google.com/spreadsheets/d/1aW9C4fauN8wER7D4Vvxw8T2KqSfAg53F/edit?usp=sharing&ouid=114291358875751094296&rtpof=true&sd=true
4. Большой гайд по графическим элементам https://guides.kontur.ru/

5. Тестирование GUI. Базовое руководство https://qarocks.ru/gui-testing/
6. Отличное видео с обзором веб-элементов https://www.youtube.com/watch?v=ECpB6lSyeZM
7. Примеры багов и улучшений для форм https://habr.com/ru/post/721160/
8. Figma + прототип сайта https://www.figma.com/file/fqZlcMyvIY1CJCJiPUjPnb/Web-Site-(Community)?node-id=186-78&t=iPY5P8MkJHdfho3x-0
9. Примеры HTML, CSS и JS реализации веб-элементов и форм https://www.w3schools.com/howto/

ТЕСТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЙ

# 38. ДОП: Обзор клиентской части приложения на курсе

### Полезные материалы

1. Ссылка на репозиторий с исходным кодом приложения

ТЕСТИРОВАНИЕ API

# 39. Вступительное слово перед модулем "Тестирование API"

**Вступительное слово перед стартом модуля "Тестирование API. Работа в Postman"**

Помимо тестирования frontend, о котором мы много говорили в предыдущем модуле, существует и тестирование backend, скрытого от пользователя, где осуществляется "основная магия" для работы нашего приложения. Поэтому нам нужно уметь тестировать и его.

Плюс у тестировщиков не всегда есть доступ к графическому интерфейсу и нам необходимо проверять серверную валидацию, что нельзя сделать через встроенные в браузер инструменты.

В новом модуле мы рассмотрим одну из частей этого направления тестирования: тестирование API (тестирование веб-сервисов).

Существует несколько реализаций веб-сервисов, самые популярные из них - REST и SOAP.

Мы научимся тестировать их с помощью инструмента Postman.

**В данном модуле мы:**

- Узнаем про типы веб-сервисов и применяемых форматов
- Научимся читать API-документацию на примере Swagger
- Научимся различать идентификацию, аутентификацию и авторизацию
- Разберемся с интерфейсом, основными функциями Postman
- Напишем простые автотесты
- Создадим тестовые данные и цепочки запросов
- Узнаем про перехват трафика
- Освоим подходы написания тестовой документации

***Если по окончанию модуля вам нужно будет еще больше практики тестирования REST API, то ее можно найти в моем углубленном курсе по Postman в формате игры и сеттинге легенд и мифов Древней Греции.***

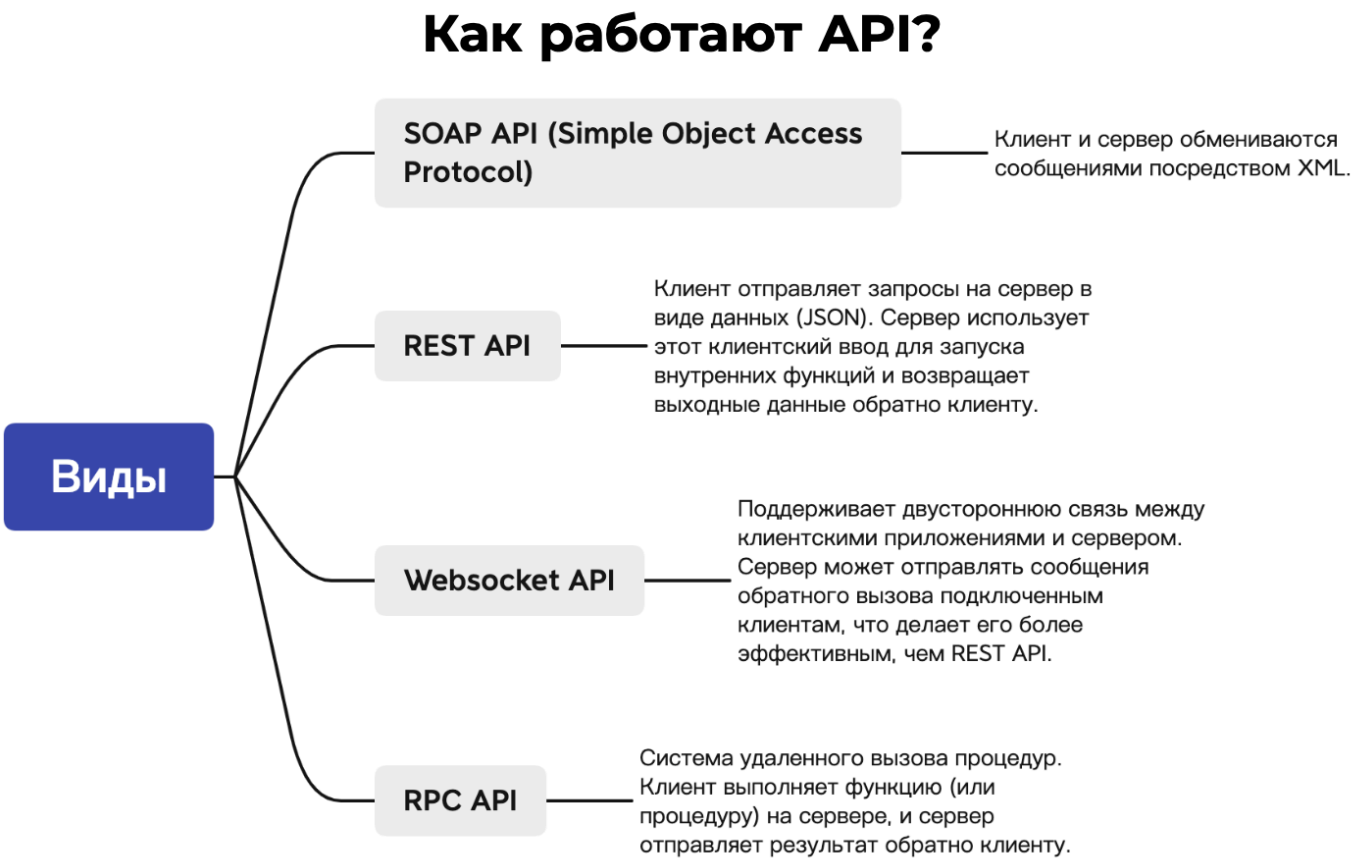
POSTMAN
GET
POST
SEND
BODY
BODY
HEADERS

ТЕСТИРОВАНИЕ API

# 40. Виды API. SOAP и REST

**Веб-сервис (web service)** - реализация абсолютно четких интерфейсов обмена данными между различными приложениями, которые написаны не только на разных языках, но и распределены на разных узлах сети.

**API (Application Programming Interface)** – это механизмы, которые позволяют двум программным компонентам взаимодействовать друг с другом, используя набор определений и протоколов.




## SOAP

**SOAP** (Simple Object Access Protocol) — это **протокол** передачи структурированных сообщений между системами по сети. Чаще всего используется с HTTP. SOAP определяет, **как** отправить запрос и получить ответ.

📌 Представьте его как *курьера*, который передаёт сообщение от клиента к серверу.

**WSDL** (Web Services Description Language) — это **описание сервиса**, написанное на XML. Оно говорит: *вот какие методы есть у сервиса, какие параметры можно передавать, и в каком формате будет ответ*.

📌 Это как *техническое задание* на сервис: что можно вызывать и как именно.

**XML** (eXtensible Markup Language) — это **язык разметки**, с помощью которого структурируются данные внутри SOAP-сообщения. Он читаем человеком и машиной.

📌 Это как *форма с полями*, где указаны имя, дата, сумма и т.д.

**Пример XML**

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
</bookstore>
```

**XSD** (XML Schema Definition) — это **набор правил**, по которым проверяется XML-документ: какие поля обязательны, какого они типа, в каком порядке.

📌 Это как *инструкция к заполнению формы*: например, “поле ‘email’ обязательно, не более 50 символов, без пробелов”.

Для запоминания:

- **SOAP** — *передаёт* сообщение
- **WSDL** — *описывает* сервис и его методы
- **XML** — *упаковывает* сами данные
- **XSD** — *контролирует* корректность этих данных

Если вы тестируете SOAP API, вам нужно уметь читать WSDL, анализировать XML-ответы и, при необходимости, сверяться с XSD, чтобы понять, валидны ли данные. Это всё входит в арсенал ручного тестировщика при работе с интеграциями.

## WSDL

Для описания функциональности веб-сервиса используется файл, написанный на WSDL (Web Services Description Language), основанный на XML, в котором содержится информация о получаемых и отправляемых данных, форматах сообщений, методах.

WSDL является обязательным элементом любого SOAP-сервиса, без которого он не может существовать.

В тестировании вы будете в первую очередь работать с файлами WSDL. Об особенностях их практического использования мы поговорим на следующих уроках.

**Пример WSDL (приведен в ознакомительных целях, на данный момент нет смысла запоминать определение каждого тэга)**

```
<?xml version="1.0"?>
<wsdl:definitions xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
                  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
                  xmlns:tns="http://www.example.com/wsdl"
                  targetNamespace="http://www.example.com/wsdl">

    <wsdl:types>
        <!-- Описание типов данных -->
    </wsdl:types>

    <wsdl:message name="MyRequest">
        <!-- Описание структуры запроса -->
    </wsdl:message>

    <wsdl:message name="MyResponse">
        <!-- Описание структуры ответа -->
    </wsdl:message>

    <wsdl:portType name="MyPortType">
        <wsdl:operation name="MyOperation">
            <wsdl:input message="tns:MyRequest"/>
            <wsdl:output message="tns:MyResponse"/>
        </wsdl:operation>
    </wsdl:portType>

    <wsdl:binding name="MyBinding" type="tns:MyPortType">
        <soap:binding style="document"
transport="http://schemas.xmlsoap.org/soap/http"/>
        <wsdl:operation name="MyOperation">
            <soap:operation soapAction="http://www.example.com/MyAction"/>
            <wsdl:input>
```

```
                <soap:body use="literal"/>
            </wsdl:input>
            <wsdl:output>
                <soap:body use="literal"/>
            </wsdl:output>
        </wsdl:operation>
    </wsdl:binding>

    <wsdl:service name="MyService">
        <wsdl:port name="MyPort" binding="tns:MyBinding">
            <soap:address location="http://www.example.com/MyService"/>
        </wsdl:port>
    </wsdl:service>

</wsdl:definitions>

<wsdl:definitions>: Корневой элемент, определяющий WSDL-документ.
<wsdl:types>: Определяет типы данных, используемые в сообщениях.
<wsdl:message>: Определяет структуру данных для запроса и ответа.
<wsdl:portType>: Описывает набор операций, которые можно выполнить.
<wsdl:operation>: Конкретная операция, доступная через веб-сервис.
<wsdl:binding>: Определяет протокол и формат данных для операций и сообщений.
<soap:binding>: Указывает, что привязка использует SOAP.
<wsdl:service>: Определяет веб-сервис.
<wsdl:port>: Определяет конечную точку для веб-сервиса.
```

### Основные правила и ошибки в синтаксисе в XML

Иногда задание на поиск ошибок в XML дают на собеседовании или перед ним. Поэтому нам важно знать особенности синтаксиса этого языка.

```
1. Обязательное наличие корневого элемента:

Ошибка: <tag1></tag1><tag2></tag2>
Правильно: <root><tag1></tag1><tag2></tag2></root>

2. Наличие закрывающих тегов:

Ошибка: <tag>This is a paragraph.<tag>
```

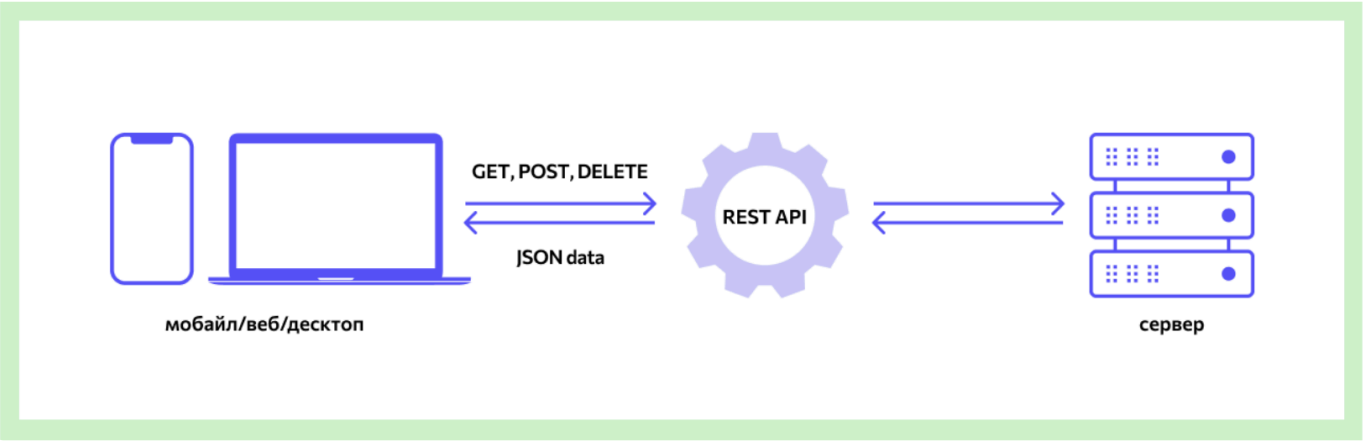
```
Правильно: <tag>This is a paragraph.</tag>

3. Чувствительность к регистру:

Ошибка: <Tag>example</tag>
Правильно: <tag>example</tag>

4. Правильное вложение:

Ошибка: <b><i>Text</b></i>
Правильно: <b><i>Text</i></b>

5. Кавычки в значениях атрибутов:

Ошибка: <tag attribute=value></tag>
Правильно: <tag attribute="value"></tag>

6. Использование специальных значений для некоторых спецсимволов:

Ошибка: <tag> 5 < 10 </tag>
Правильно: <tag> 5 &lt; 10 </tag>
```

Подробнее о правилах можно прочитать здесь

## Rest API

**Representational State Transfer (REST)** — это архитектурный стиль, который определяет условия работы API.

# Архитектура Rest




Когда вы слышите, что API “соответствует REST”, или RESTful API, это означает, что он построен с учётом **шести ограничений**, которые делают архитектуру масштабируемой, простой и удобной в поддержке. Вот краткое объяснение каждого из них – только то, что важно знать тестировщику:

### 1. Единый интерфейс (Uniform Interface)

Это ключевой принцип REST. Все запросы к API должны быть предсказуемыми и единообразными. Используются стандартные HTTP-методы (GET, POST, PUT, DELETE) и единообразные URL-адреса.

📌 Пример: /users/123 должен означать одного и того же пользователя во всех частях системы.

### 2. Отсутствие состояний (Stateless)

Каждый запрос должен быть **независим**: сервер не должен “помнить”, что было до этого. Вся необходимая информация (например, токен авторизации) должна передаваться в каждом запросе.

📌 Как тестировщику: проверь, можно ли сделать запрос без предварительного шага (например, без логина), и что будет, если отправить невалидный токен.

### 3. Кеширование (Cacheable)

Сервер может помечать ответы как пригодные для кэширования, чтобы клиент или прокси могли не обращаться к серверу повторно.

📌 При тестировании важно понимать, где работает кэш и когда он сбрасывается (например, после PUT или DELETE).

### 4. Клиент-сервер (Client–Server)

Клиент и сервер разделены: клиент не знает, как устроен сервер, а сервер — как устроен клиент. Это повышает независимость и гибкость.

📌 Это значит, что фронт и бэкенд можно тестировать по отдельности, а REST API должен быть независим от интерфейса.

### 5. Многоуровневая система (Layered System)

Между клиентом и сервером могут быть промежуточные звенья: прокси, шлюзы, балансировщики. API не должен “зависеть” от того, сколько слоёв между ними.

📌 Тестировщику важно понимать, что может быть кэширование на прокси или модификация запроса на промежуточных слоях.

### 6. Код по требованию (Code on Demand, опционально)

Сервер может отправлять исполняемый код клиенту (например, скрипт JavaScript), но это не обязательное требование.

📌 Почти не используется в классических REST API, так что можете не зацикливаться.

REST не является строгим стандартом, но эти ограничения — ориентиры, к которым стремятся. Если API нарушает их слишком сильно — скорее всего, он просто “JSON over HTTP”, а не настоящий REST.

## JSON

**JSON (JavaScript Object Notation)** — это легковесный формат обмена данными, который удобен для чтения и написания для человека, а также для анализа и генерации машинами. Его синтаксис основан на языке JavaScript, но сам JSON существует независимо и может использоваться с другими языками программирования.

### Синтаксис JSON

**JSON (JavaScript Object Notation)** состоит из следующих элементов:

1. **Объекты**: Коллекции пар ключ/значение, заключенные в фигурные скобки `{}`. Например: `{"name": "John", "age": 30}`

2. **Массивы**: Упорядоченные списки значений, заключенные в квадратные скобки `[]`. Например: `["apple", "banana", "cherry"]`. Если вам нужно заключить несколько объектов в один JSON, то в таком случае тоже нужно использовать массивы Например: `[{"name": "John", "age": 30},{"name": "Liza", "age": 55}]`
3. **Значения**: Могут быть строками (в двойных кавычках), числами, объектами, массивами, булевыми значениями (`true` или `false`), или `null`. Например: `"Hello World"`, `123`, `true`, `null`.
4. **Строки**: Последовательности символов в двойных кавычках. Например: `"example string"`
5. **Числа**: Целые или с плавающей точкой. Например: `100`, `20.5`

**Пример JSON**

```
{
  "name": "John Doe",
  "age": 30,
  "isEmployed": true,
  "address": {
    "street": "123 Main St",
    "city": "Anytown"
  },
  "phoneNumbers": ["123-456-7890", "456-789-0123"]
}
```

- `"name"`, `"age"`, и `"isEmployed"` — это ключи, соответствующие строковому имени, числовому возрасту и булевому значению занятости.
- `"address"` — это вложенный объект с ключами `"street"` и `"city"`.
- `"phoneNumbers"` — это массив телефонных номеров.

Далее на курсе мы будем учиться тестировать REST API посредством специального инструмента - Postman.

**Как тестировать API?**

1. Запросить документацию.

а. В SOAP она есть по умолчанию в виде WSDL (читается через парсеры в SoapUI, Wizdler).
б. Для REST описывается в Swagger или других системах

2. Изучить доступные методы

3. Создать коллекции в Postman и (если надо) спроектировать позитивные и негативные тест-кейсы

4. Запустить процесс тестирования (SoapUI, Postman)

**Что нужно проверять?**

1. Обязательные/необязательные значения
2. Отправка различных типов данных
3. Статус-коды
4. Ответ от сервера (заголовки, тело, ключи, значения, типы данных)
5. Авторизация и разрешения
6. Время ответа

**Полезные материалы**

1. Обязательно посмотрите еще одно мое видео про API https://youtu.be/kUPWQMalWNk
2. Презентация
3. Старые версии урока
4. https://youtu.be/_cfmDnIIQTU
5. https://youtu.be/VqjaDULOYOM
6. Схема из старых версий урока https://drive.google.com/drive/u/3/folders/1RYbvze2c1ULfArT4jWViIqzufISBZ6Tn
7. https://drive.google.com/drive/u/3/folders/1A8frWIUvHlgP5KYoUxFyfh-AbUT1mstM
8. Что такое API? https://aws.amazon.com/ru/what-is/api/
9. Одно из самых лучших видео о тестировании REST API https://youtu.be/UdMrvh-egqk
10. Что такое JSON? https://habr.com/ru/post/554274/
11. Библия QA. REST/SOAP/gRPC https://vladislaveremeev.gitbook.io/qa_bible/seti-i-okolo-nikh/rest-soap-grpc
12. Информация о тестовых площадках для отработки навыков тестирования API https://okiseleva.blogspot.com/2017/04/users-soap-rest.html

13. Платформы для практики:
14. http://users.bugred.ru/
15. https://reqres.in/
16. https://petstore.swagger.io/
17. https://swapi.dev/
18. https://pokeapi.co/
19. Тест-кейсы и пример чек-листа для тестирования API https://is.gd/kKDQ5O
20. Список полезных статей и видео для изучения тестирования API https://habr.com/ru/post/667634/
21. Примеры ресурсов с документацией в Swagger
22. https://reverb.com/swagger#/articles
23. https://developers.themoviedb.org/3/account/get-account-details
24. https://codesandbox.io/examples/package/swagger-ui
25. https://codesandbox.io/embed/y34so?codemirror=1
26. Примеры открытой тестовой документации
27. https://developer.atlassian.com/cloud/trello/rest/api-group-actions/
28. https://dev.twitch.tv/docs/api/
29. https://api.trello.com
30. https://dev.vk.com/reference/json-schema
31. https://dev.vk.com/reference
32. Для тех, кто сейчас потерял доступ к API-документации для тестовой площадки http://users.bugred.ru/ вам просто нужно зайти с кредами из статьи в Jira
33. Далее просто перейти по этой ссылке и все будет работать. Тренируйтесь на здоровье https://testbase.atlassian.net/wiki/spaces/USERS/pages/592511089/SOAP+REST
34. Туториал по XML https://www.w3schools.com/xml/default.asp
35. Туториал по JSON https://www.w3schools.com/js/js_json_intro.asp

ТЕСТИРОВАНИЕ API

# 41. Установка и обзор функций Postman (версия 2026)

### Установка Postman

Вы можете работать с Postman прямо в браузере, но я рекомендую установить его на свой компьютер.

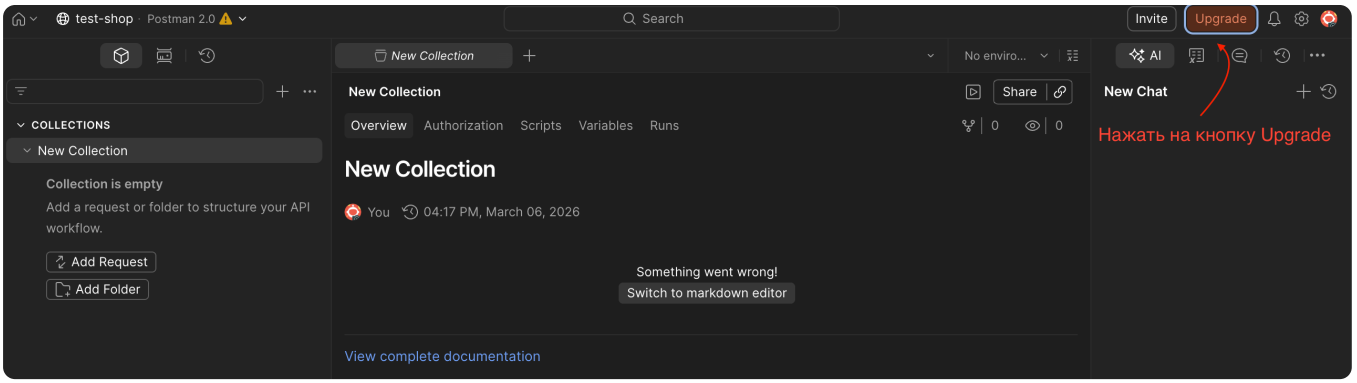
1. Перейдите по ссылке и скачайте установщик.
2. Установите программу, ничего не меняя по ходу установки.
3. Запустите Postman и создайте аккаунт в системе.
4. Ответьте на базовые вопросы для всех новых пользователей.
5. Продолжите с бесплатным планом.
6. Если вы хотите работать в команде с кем-то, то добавьте почты этих людей.
7. Далее вам предложат пройти онбординг (знакомство в Postman с AI). Вы можете продолжить обучение с ним, либо закрыть окно и действовать по моей инструкции.

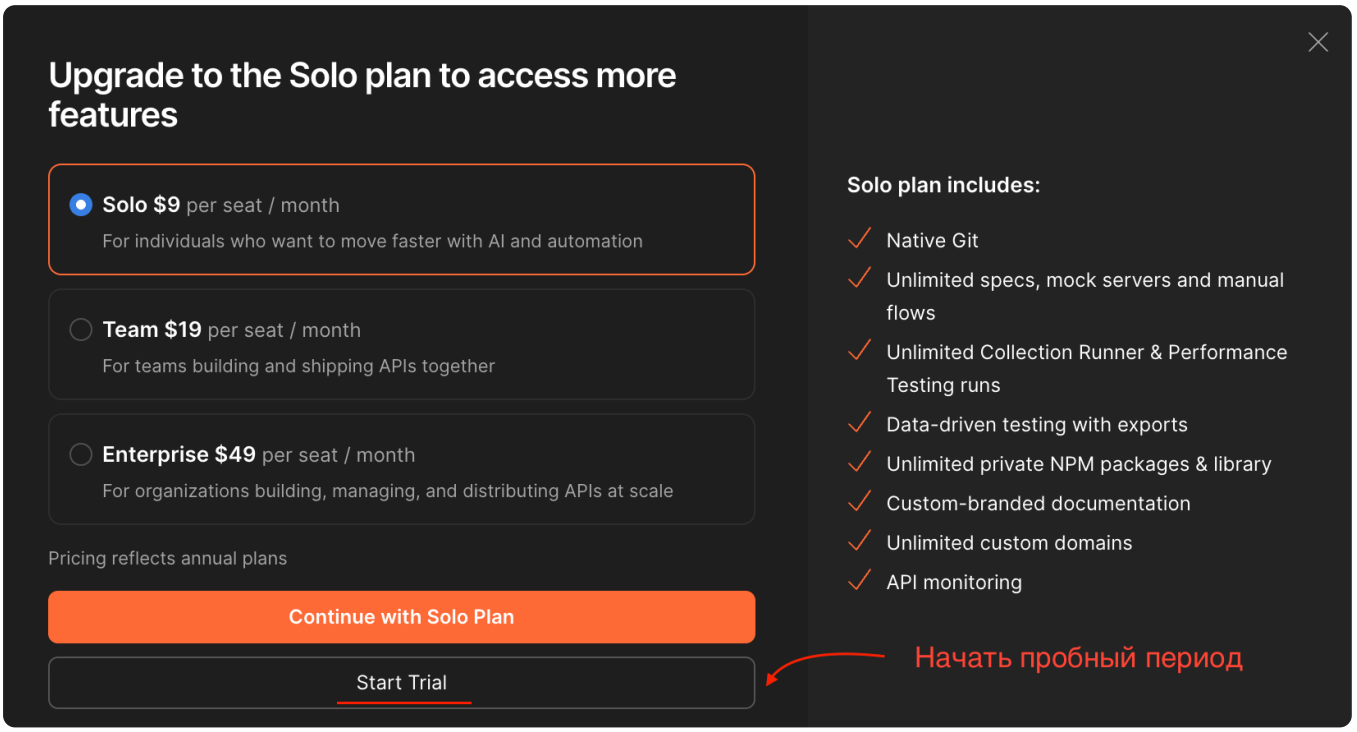
Все дальнейшие уроки в курсе я буду показывать на примере установленного приложения на MacOS, так что если вы работаете в веб-версии или приложении на Windows что-то может отличаться.

Интерфейс Postman очень часто меняется, поэтому некоторые элементы интерфейса на момент вашего обучения могут выглядеть по-другому. Самые критические изменения буду добавлять по ходу их появления.

Всегда используйте официальную документацию в спорных моментах.

> С 1 марта 2026 года ряд функций стали платными, поэтому рекомендую подключить бесплатный пробный период на 30 дней, карта не требуется. Этого времени должно хватить на освоение этого модуля. Ниже показано как это сделать:

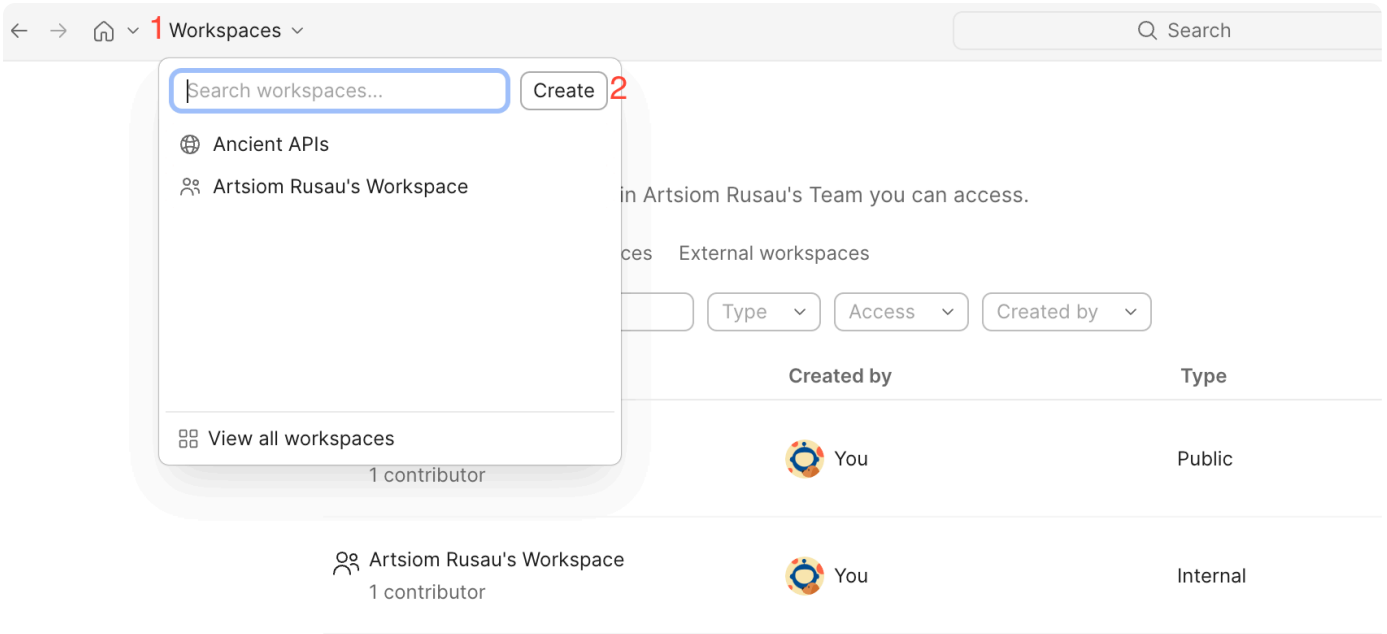


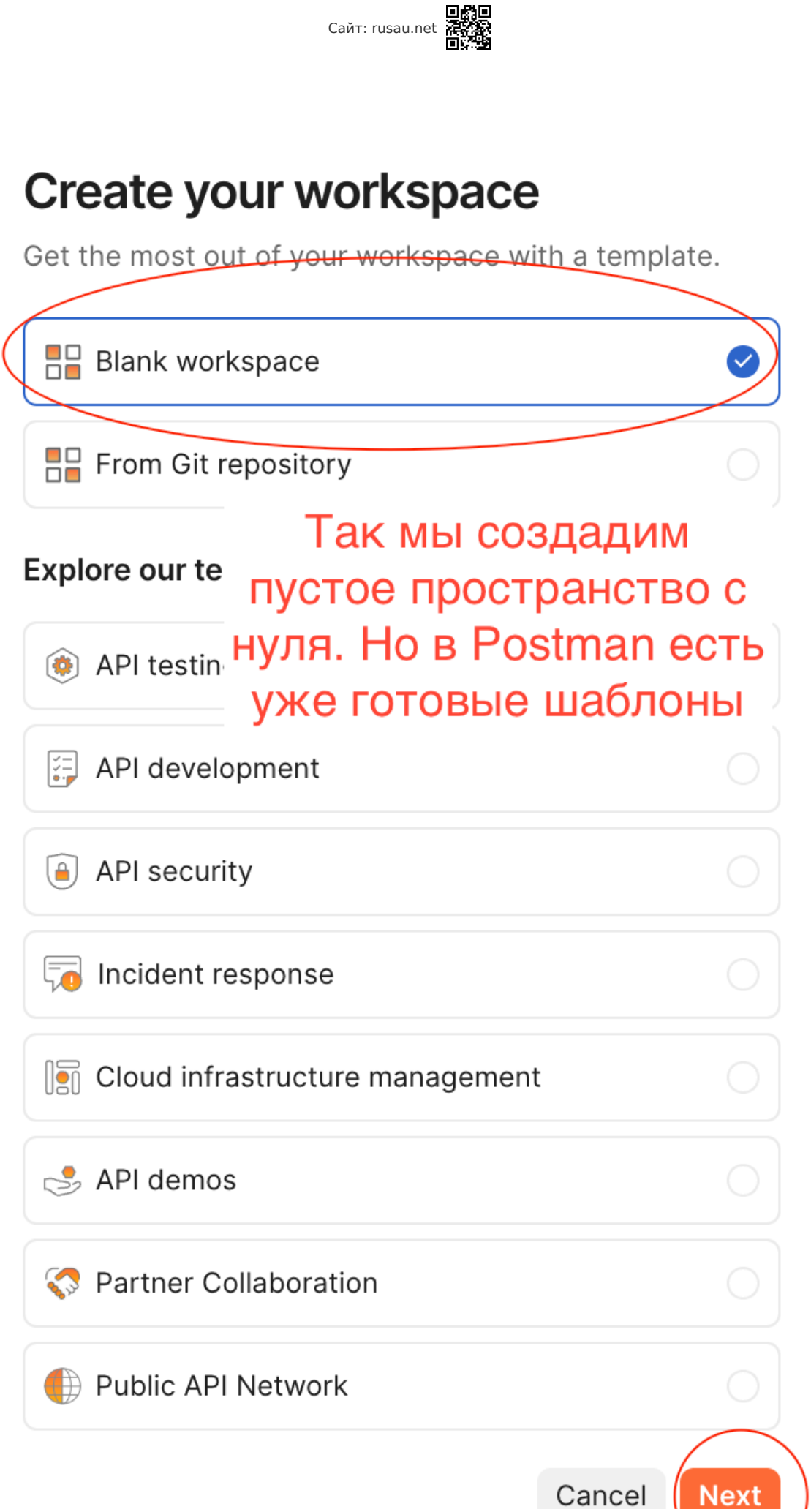



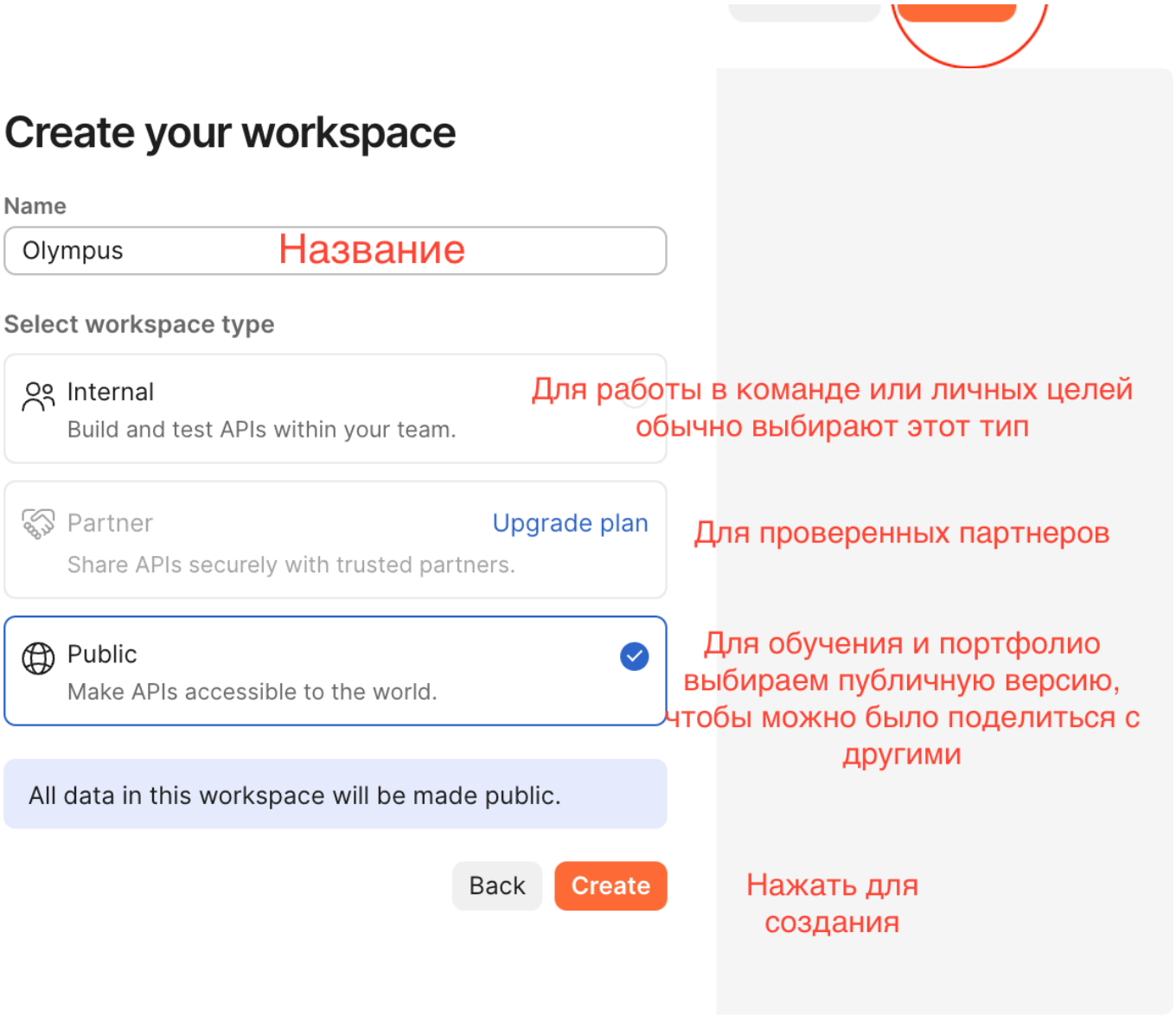

## Создание рабочего пространства (Workspace)

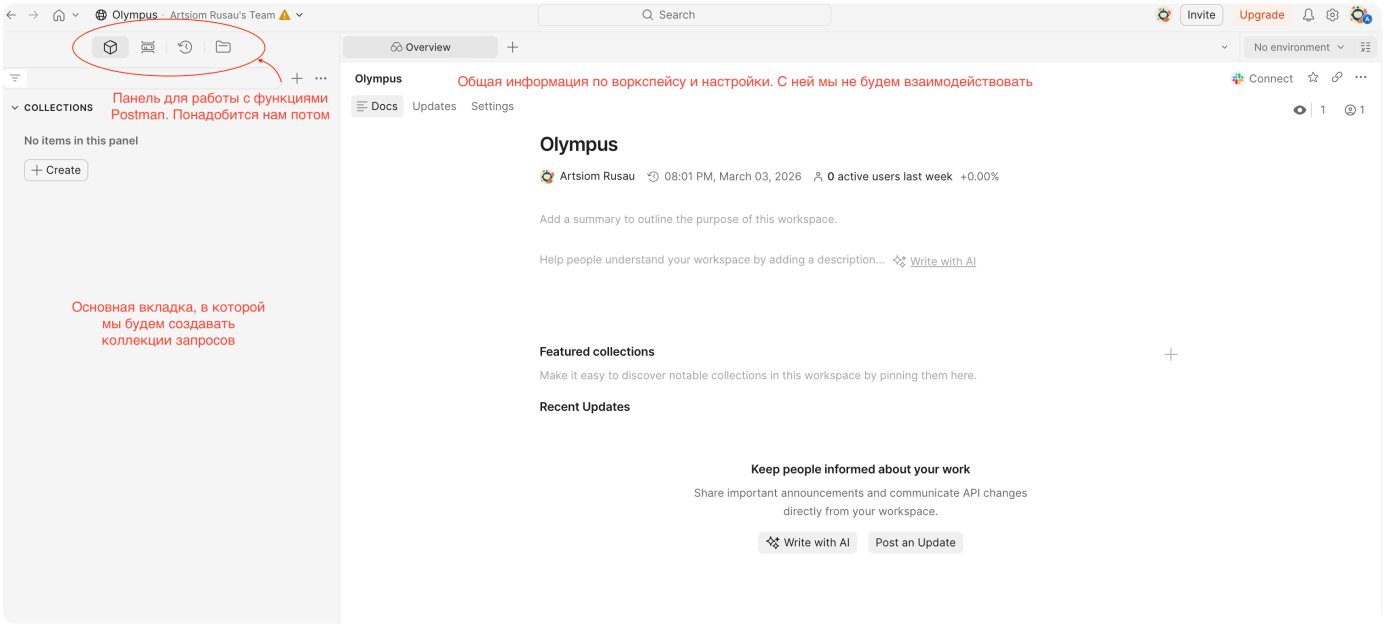
1. **Workspaces:** Это логическое разделение твоих проектов. В Postman ты можешь создавать неограниченное количество воркспейсов.

2. Для создания Workspace необходимо выполнить следующее:


1 Workspaces
Search
Search workspaces...
Create
2
Ancient APIs
Artsiom Rusau's Workspace
View all workspaces
in Artsiom Rusau's Team you can access.
ces
External workspaces
Type
Access
Created by
Created by
Type
You
Public
1 contributor
Artsiom Rusau's Workspace
1 contributor
You
Internal


Create your workspace
Get the most out of your workspace with a template.
Blank workspace
From Git repository
Так мы создадим пустое пространство с нуля. Но в Postman есть уже готовые шаблоны
Explore our te
API testin
API development
API security
Incident response
Cloud infrastructure management
API demos
Partner Collaboration
Public API Network
Cancel
Next



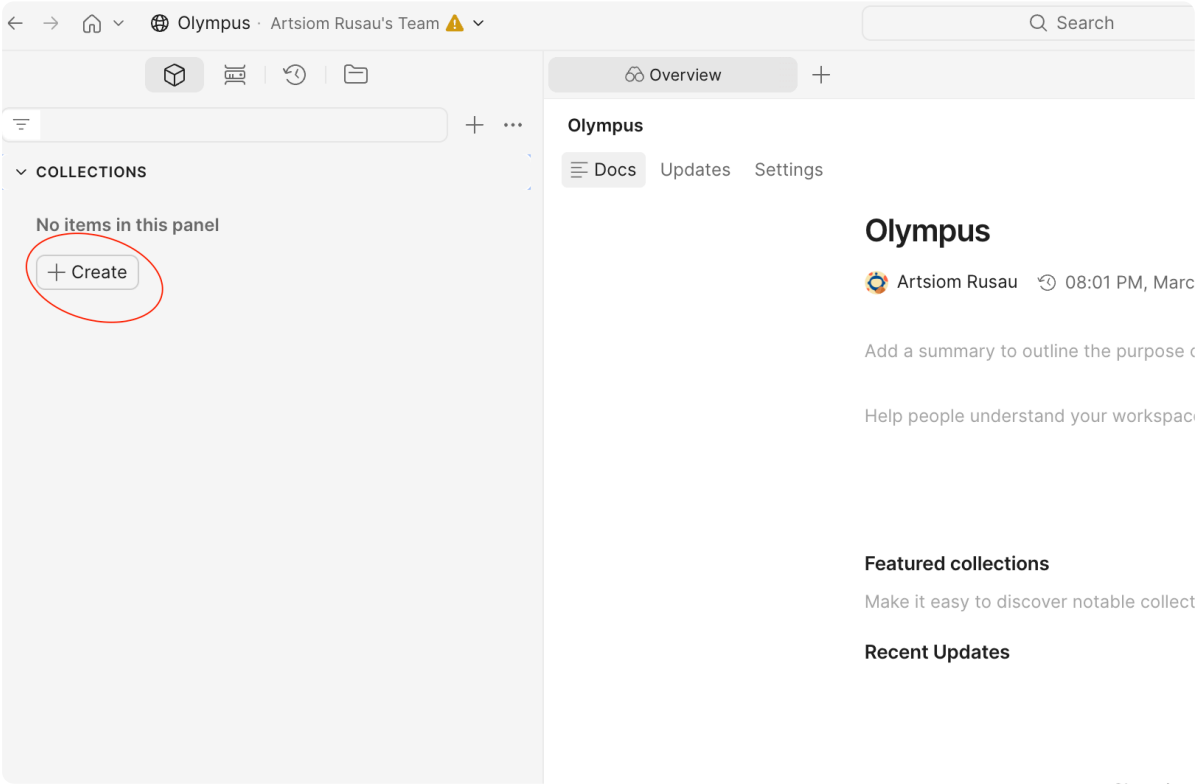

## Коллекции в Postman

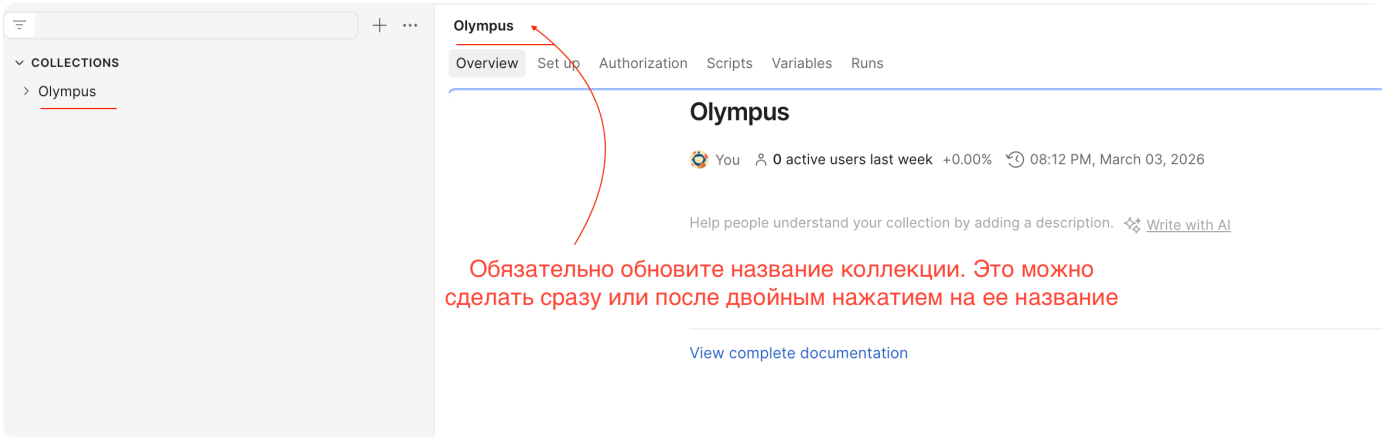
1. **Collections (коллекции)** – это группы запросов. Позволяют организовывать тесты, делиться ими с командой и запускать их пачкой. Если провести аналогию с классическим тестированием, то это тестовые наборы, внутри которых лежат тест-кейсы (запросы).

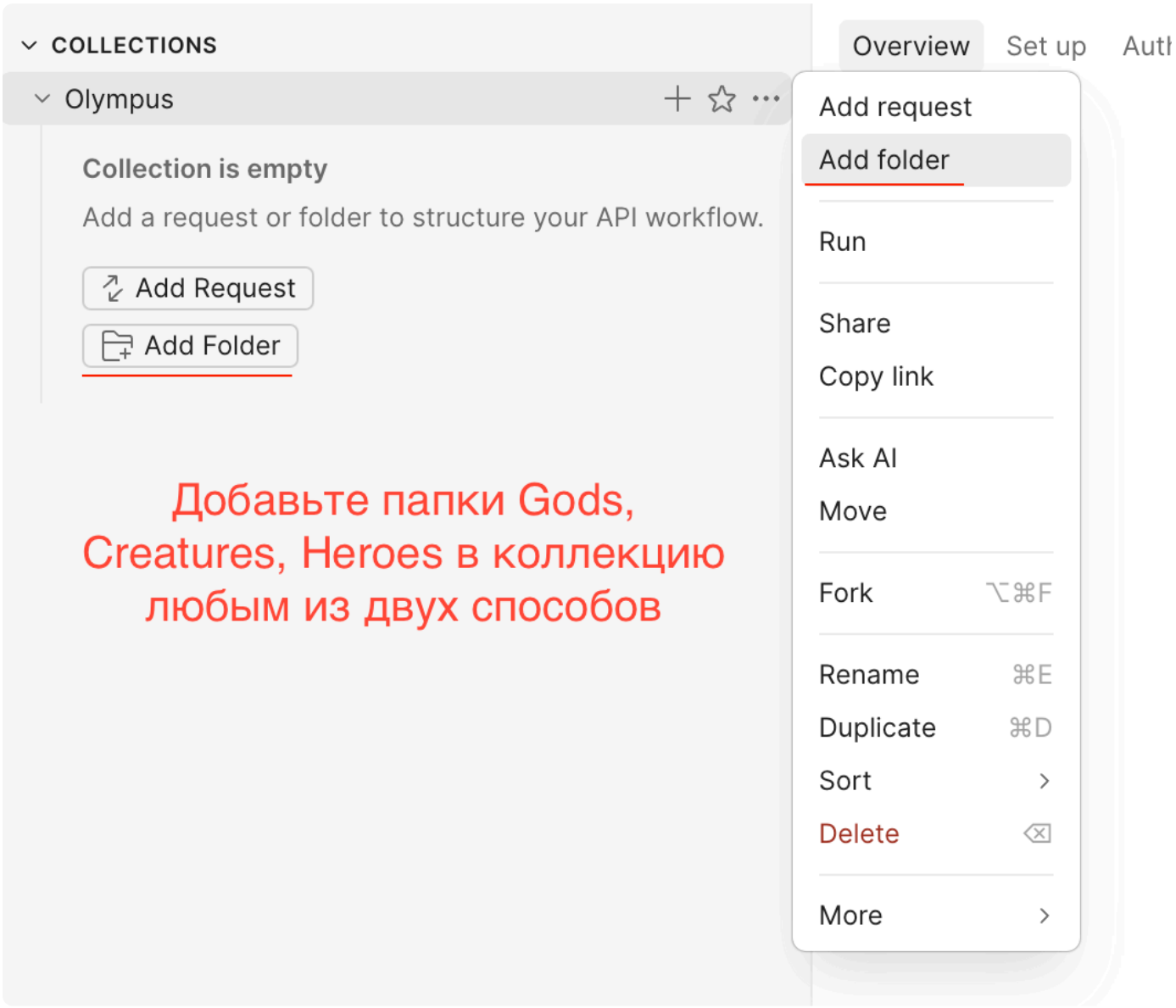
2. Коллекции обычно привязывают к конкретному сервису или приложению. Например, наша коллекция будет содержать информацию о героях Древней Греции. А если говорить о примерах классической разработки, то она может содержать запросы целого интернет-магазина или отдельного сервиса "Оплаты".

3. **Структура папок (Folders):** Внутри коллекции можно создавать папки с запросами. Хорошим тоном считается разделение по функциональным модулям или версиям API (например, v1, v2). Не рекомендую делать папки с привязкой к одному методу, например, папка "GET" или "POST". Их сложно прослеживать.

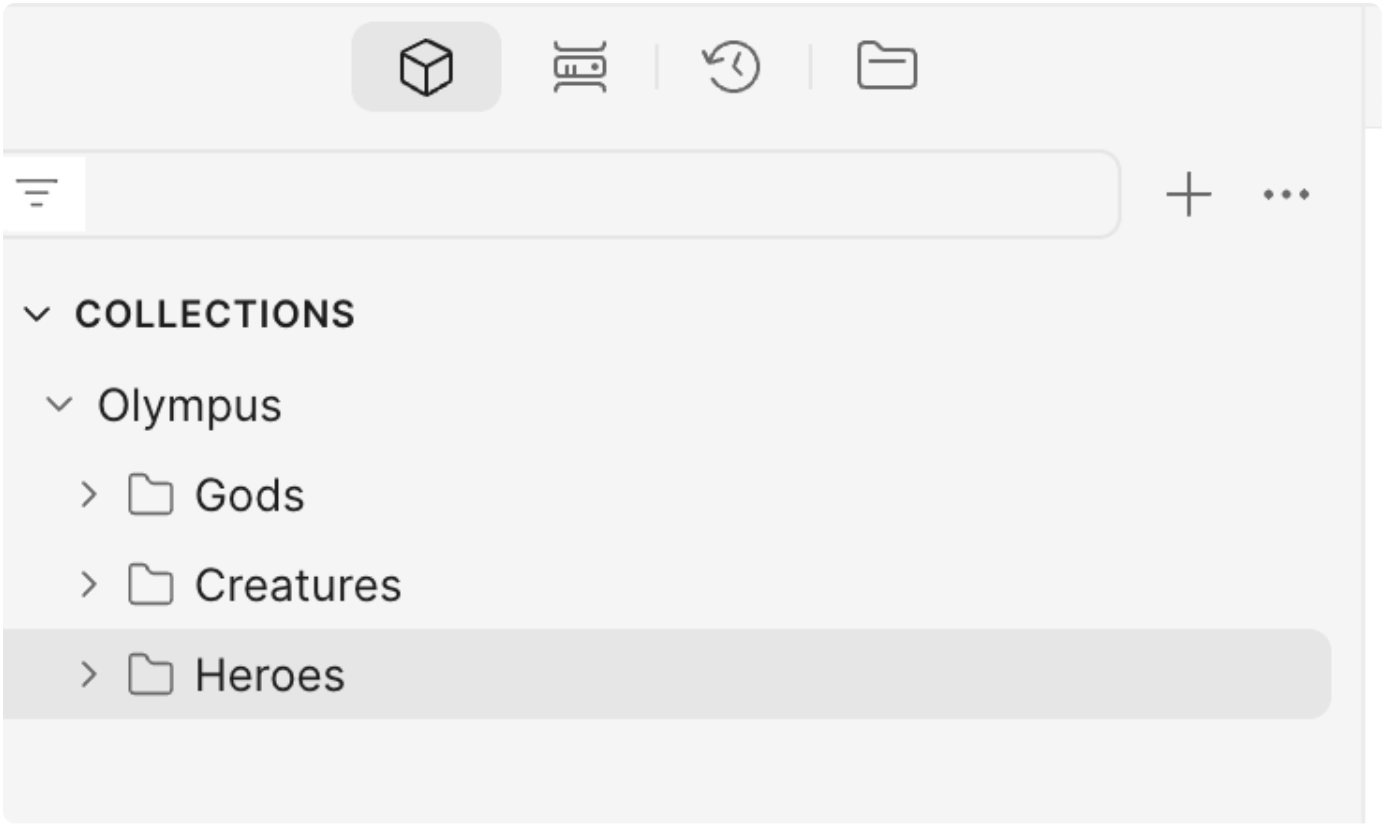
Давайте создадим первую коллекцию в нашем Workspace:






Вот, что должно получится в итоге:




**GET-запрос и параметры**

**GET-запрос** служит для получения информации с сервера. Он не содержит тело запроса и его всегда можно найти в адресной строке браузера. Это может быть как один url, когда дело касается загрузки главной страницы Stepik, так и url + параметры, которые уточняют запрос.

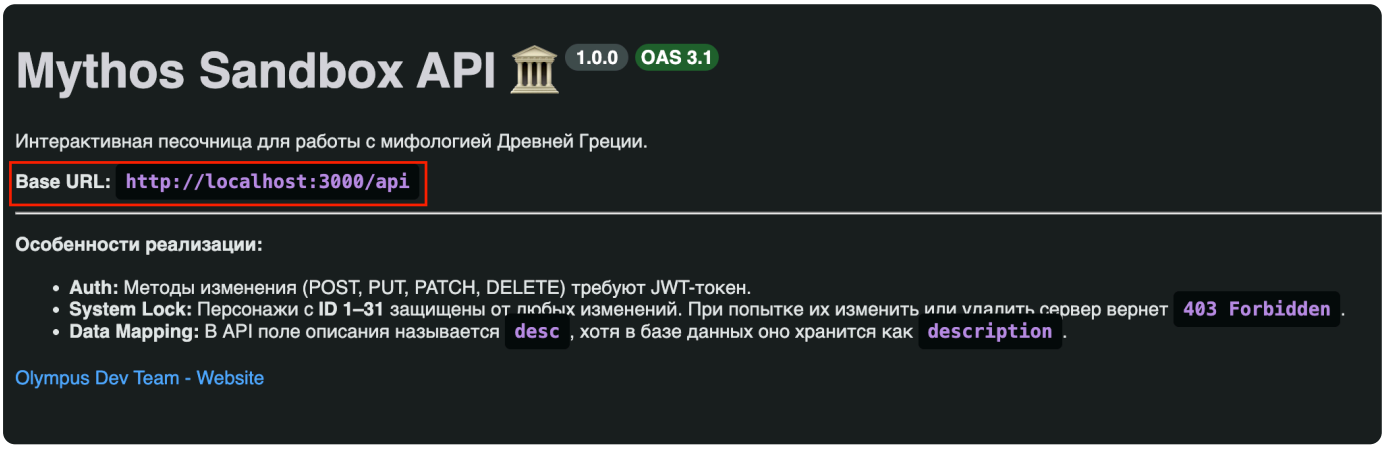
**Параметры запроса –** это способ отфильтровать или отсортировать данные, которые ты получаешь от сервера.

- **Синтаксис:** В URL они добавляются после знака вопроса `?`.
- **Структура:** Параметры всегда идут парами **Ключ (Key) = Значение (Value)**.
- **Множественность:** Если параметров несколько, они разделяются символом амперсанда `&`. Например: `?rank=olympian&weapon=thunderbolt`.

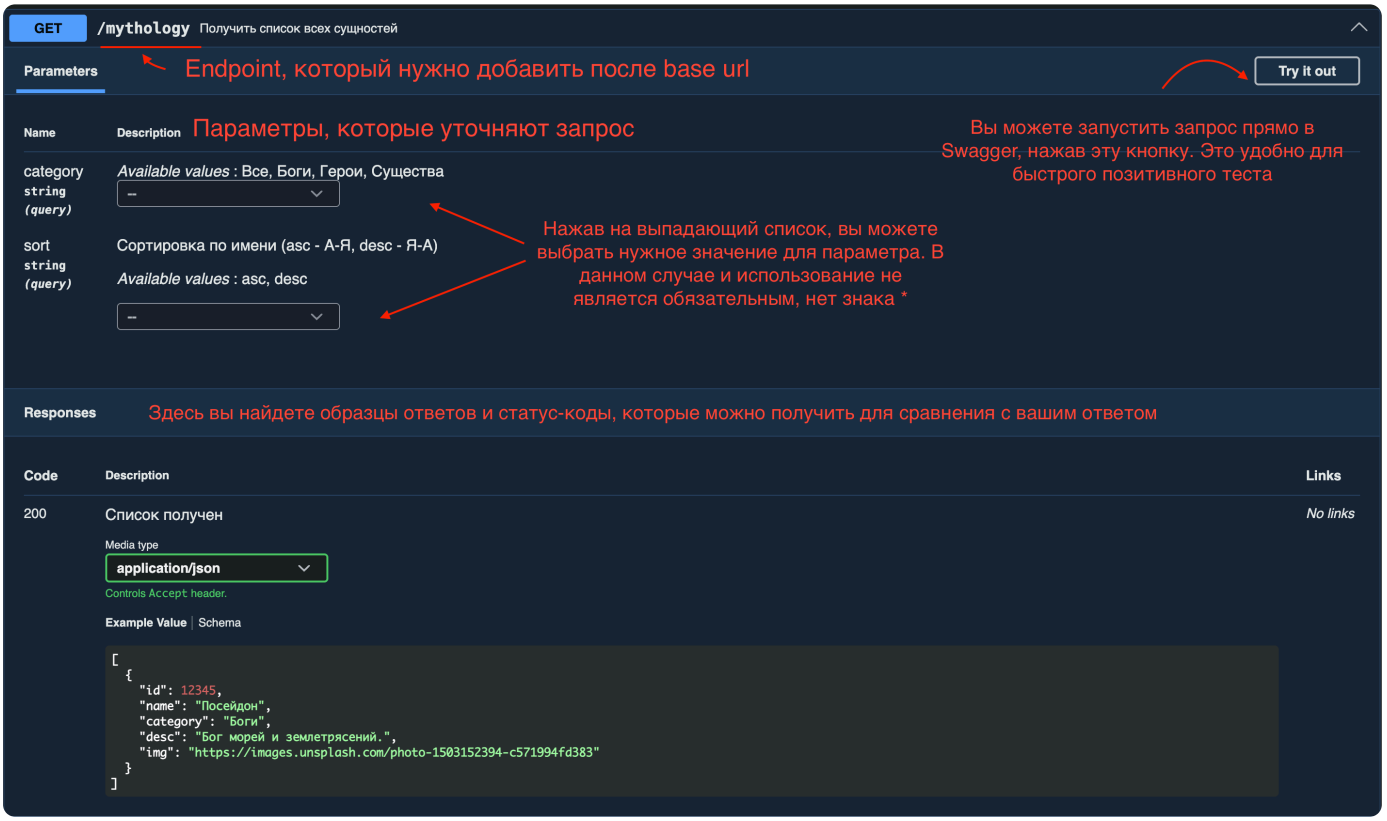
> **В Postman:** Тебе не нужно вписывать их вручную в строку адреса. Под строкой URL есть вкладка **Params**. Там расположена удобная таблица: в колонку **Key** пишем, по какому признаку ищем, а в **Value** – само значение. Обрати внимание: как только ты печатаешь в таблице, Postman сам дописывает это в URL сверху.

*Перед тем, как создать запрос, давайте обратимся в Swagger, чтобы изучить структуру нужного метода.*

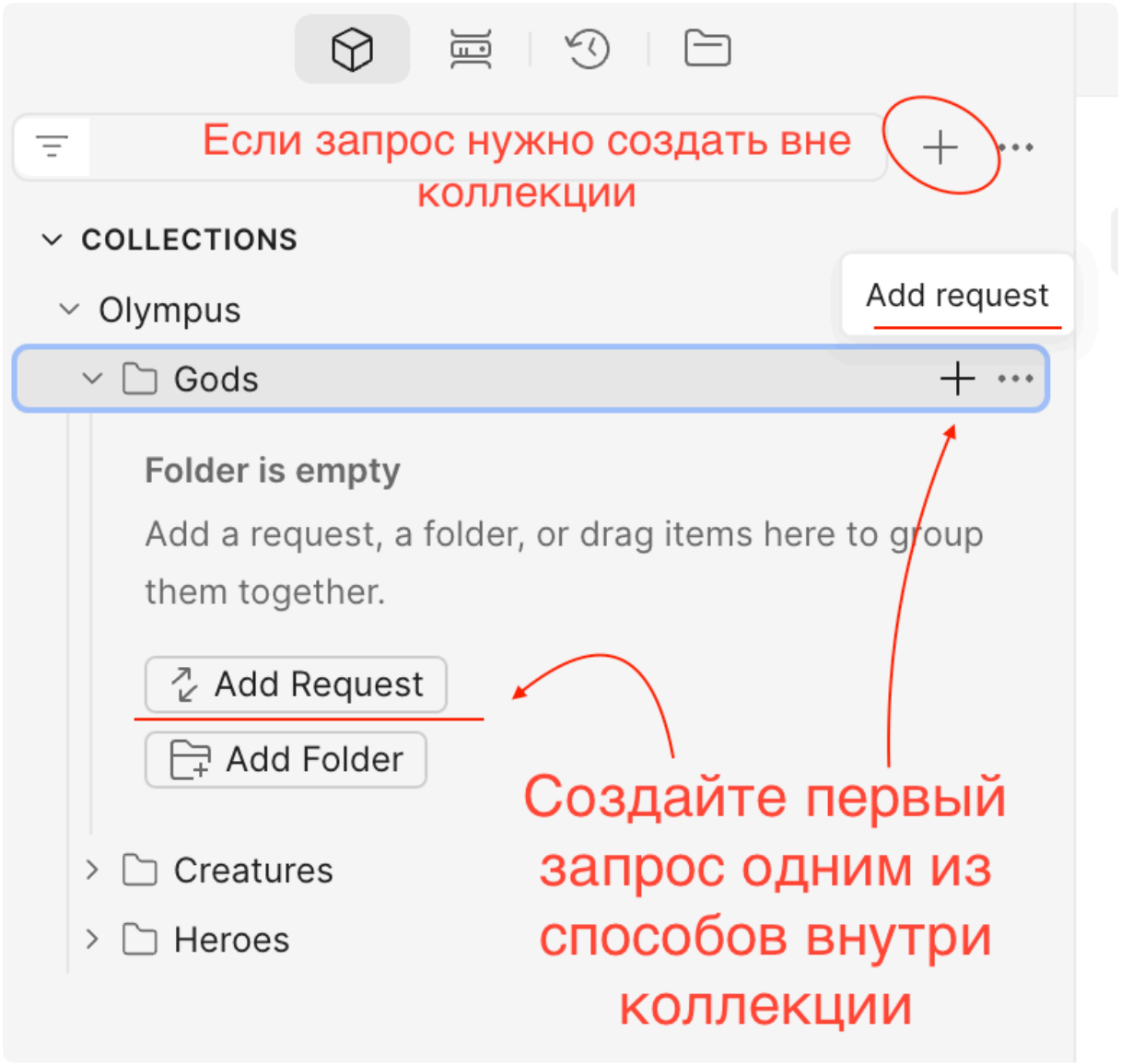
1. Обратите внимание, на **базовый url**. Он нужен для всех запросов. После него в запросе добавляется endpoint, который будет уточнять метод. На скрине сейчас url локального сервера. В нашей версии Swagger он другой: https://api.qasandbox.org/api/ Также вы можете прочитать про особенности реализации отдельных методов.



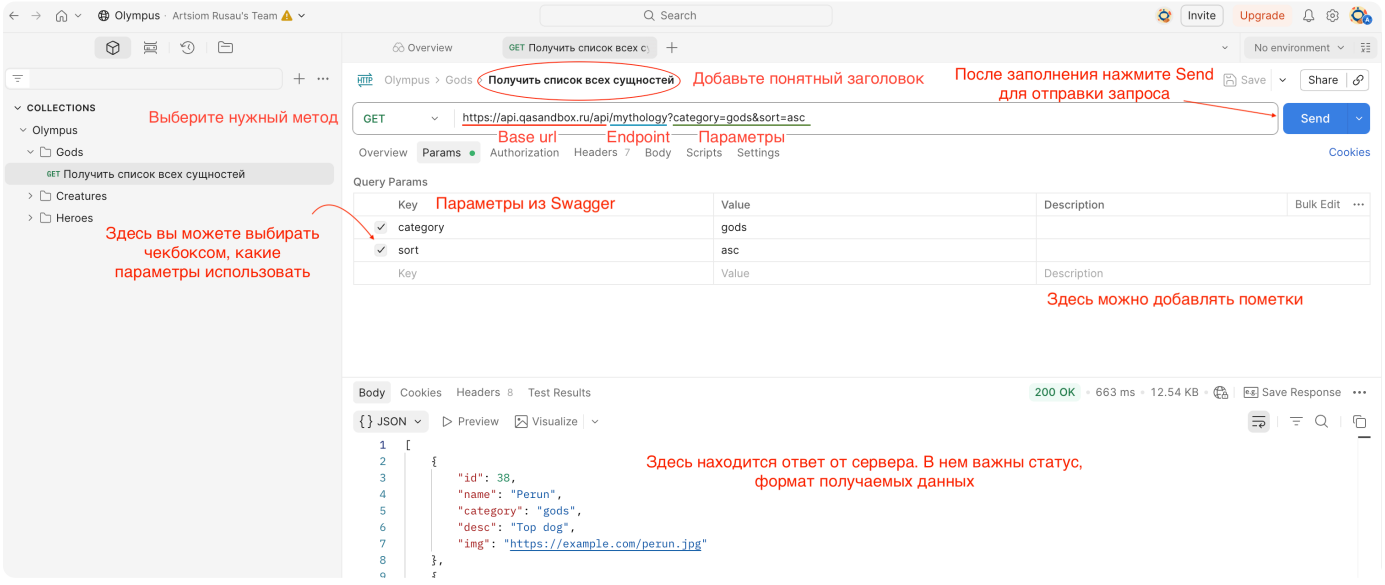

2. Для нашего урока нужен метод GET. Давайте найдем его, пролистав вниз, и изучим документацию.




3. Теперь можно создать свой первый GET-запрос в нашей коллекции:




4. После нажатия на Add request нужно последовательно заполнить все части GET-запроса.



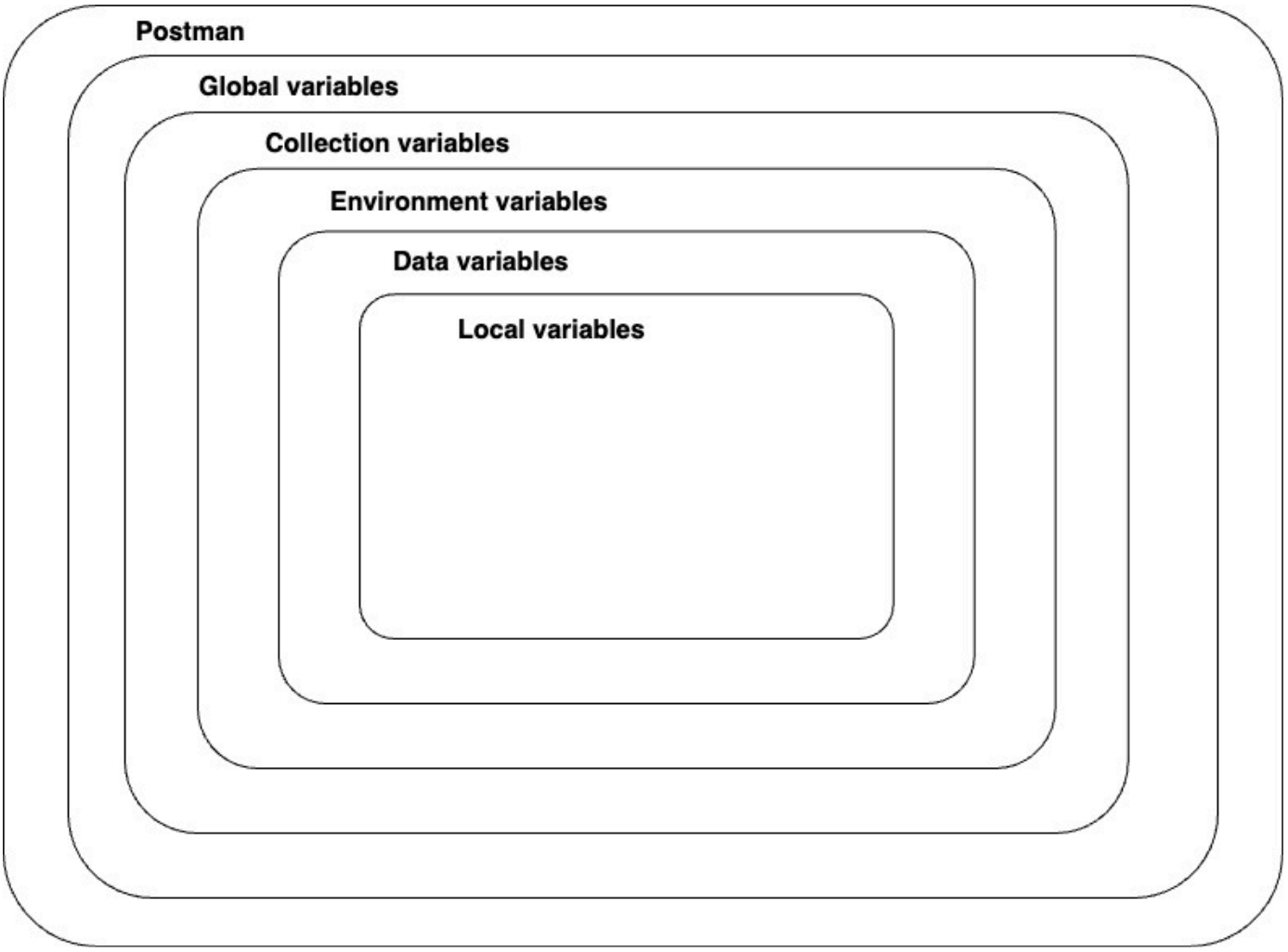
## Переменные

Переменные в Postman позволяют динамически подставлять значения в запросы.

1. **Синтаксис:** Для использования переменной в URL, Params, Headers или Body используй двойные фигурные скобки: `{{название_переменной}}`.
2. **Области видимости (Scopes):**
    - **Global Variables:** Глобальные переменные. Доступны во всем Postman. Используются редко (например, для твоего логина).
    - **Collection Variables:** Переменные коллекции. Живут только внутри одной коллекции. Идеально для `base_url`.
    - **Environment Variables:** Переменные окружения. Самые важные. Меняются в зависимости от выбранного окружения (см. следующий урок).
    - **Data Variables:** Переменные данных. Берутся из внешних файлов (CSV/JSON) при массовом запуске тестов.
    - **Local Variables:** Локальные переменные. Временные, живут только во время выполнения одного запроса.

Также советую обратить внимание эту схему, где показано какой уровень переменных затирает другой уровень, если одинаковые переменные указаны в разных местах (та, что ближе к центру всегда главнее).

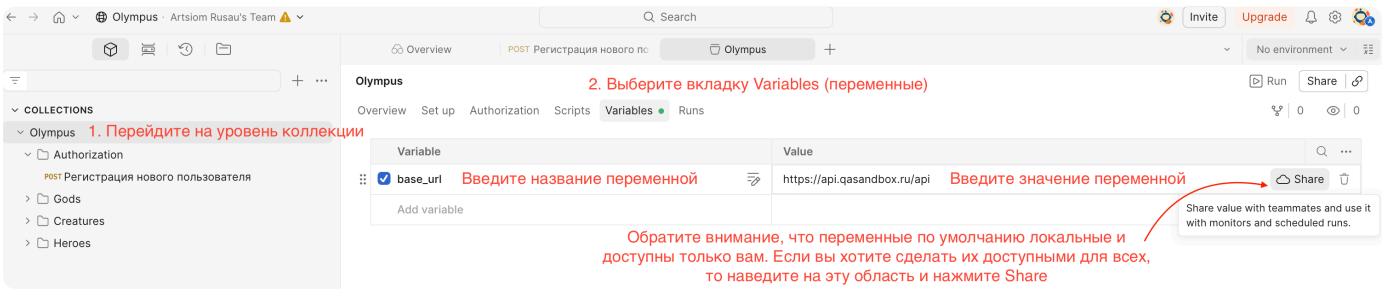
Например, если мы добавим переменную с одним и тем же названием в коллекцию, и в окружение, то переменная окружения будет использоваться вместо переменной коллекции. В следующем уроке это станет понятнее.




## Как это работает в Postman?

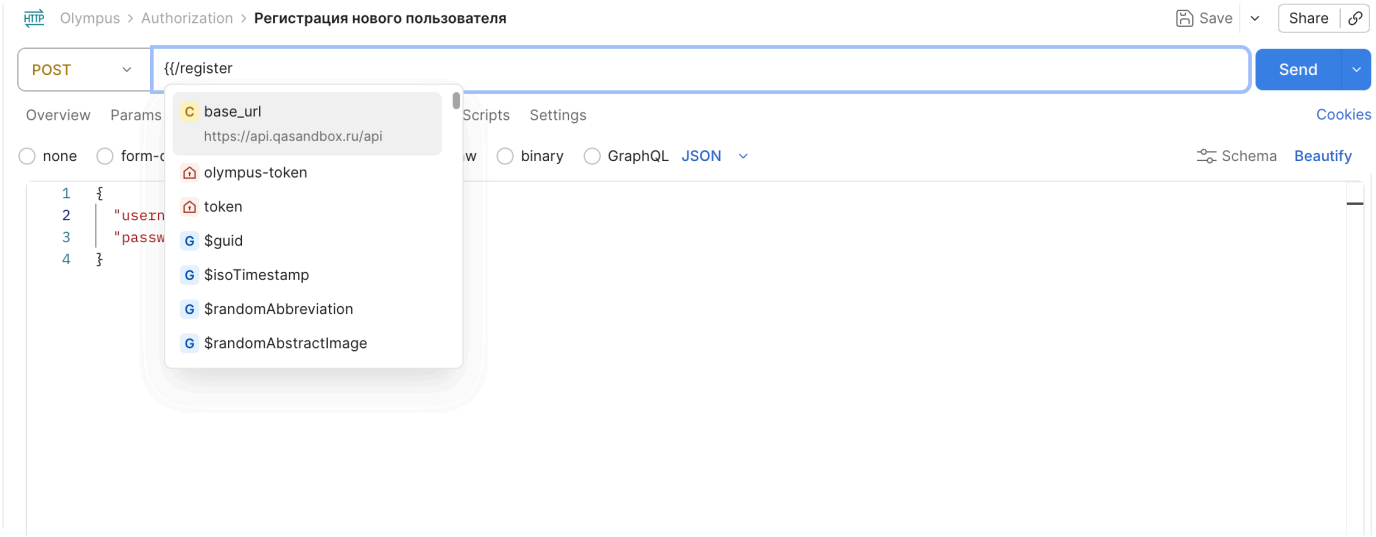
На данный момент у нас есть два запроса: GET на получение сущностей. У него есть часть: `base_url`.

1. Давайте вынесем ее в переменную коллекции и назовем `base_url`.




2. В будущем замените значение базового url во всех запросах на переменную. Переменные всегда находятся в двойных фигурных скобках. Обратите

внимание, что в моем примере базовый url не содержит в конце /. Если вы добавите его, то следует убрать / в endpoint. Postman сам подскажет название переменной, если она уже есть. Просто выберите нужный вариант или продолжите ручной ввод.

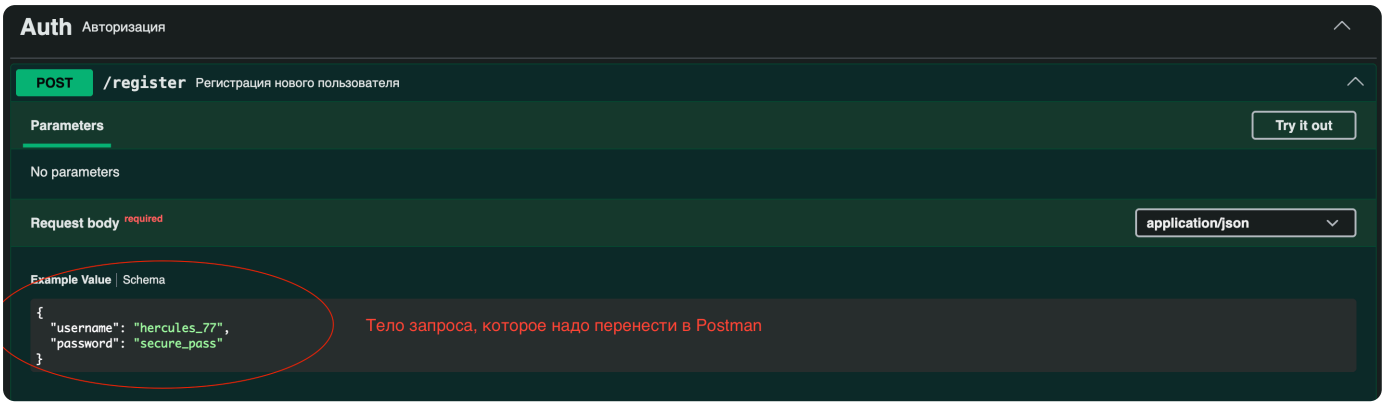


3. Теперь, когда вы захотите поменять базовый url во всех запросах, вам нужно будет сделать это в одном месте, на уровне коллекции с первого скрина.

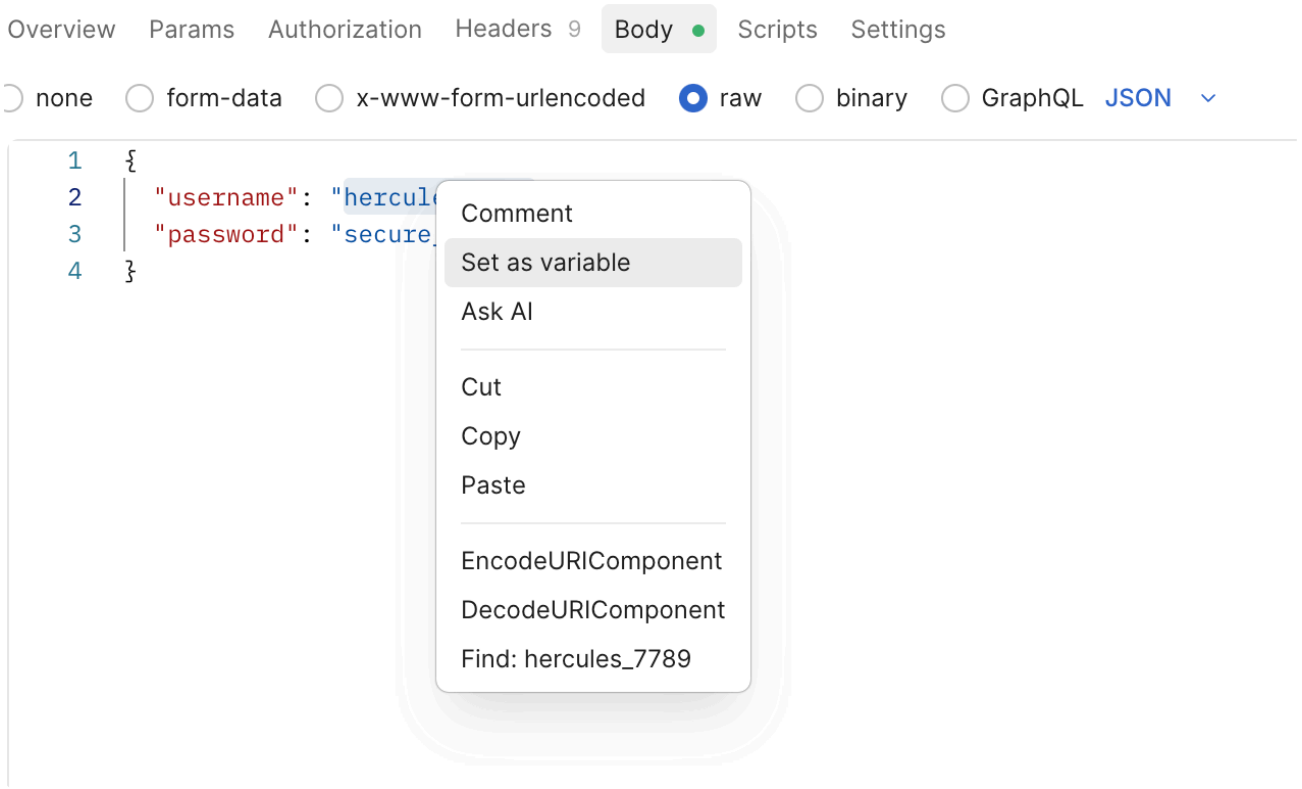
4. Также переменную можно создавать непосредственно из самого запроса: из url, Params, Headers, Body. Нужно просто выделить то значение, который нужно *сделать переменной > ПКМ > Set a variable > Set as New Variable*, выбрать имя и область.

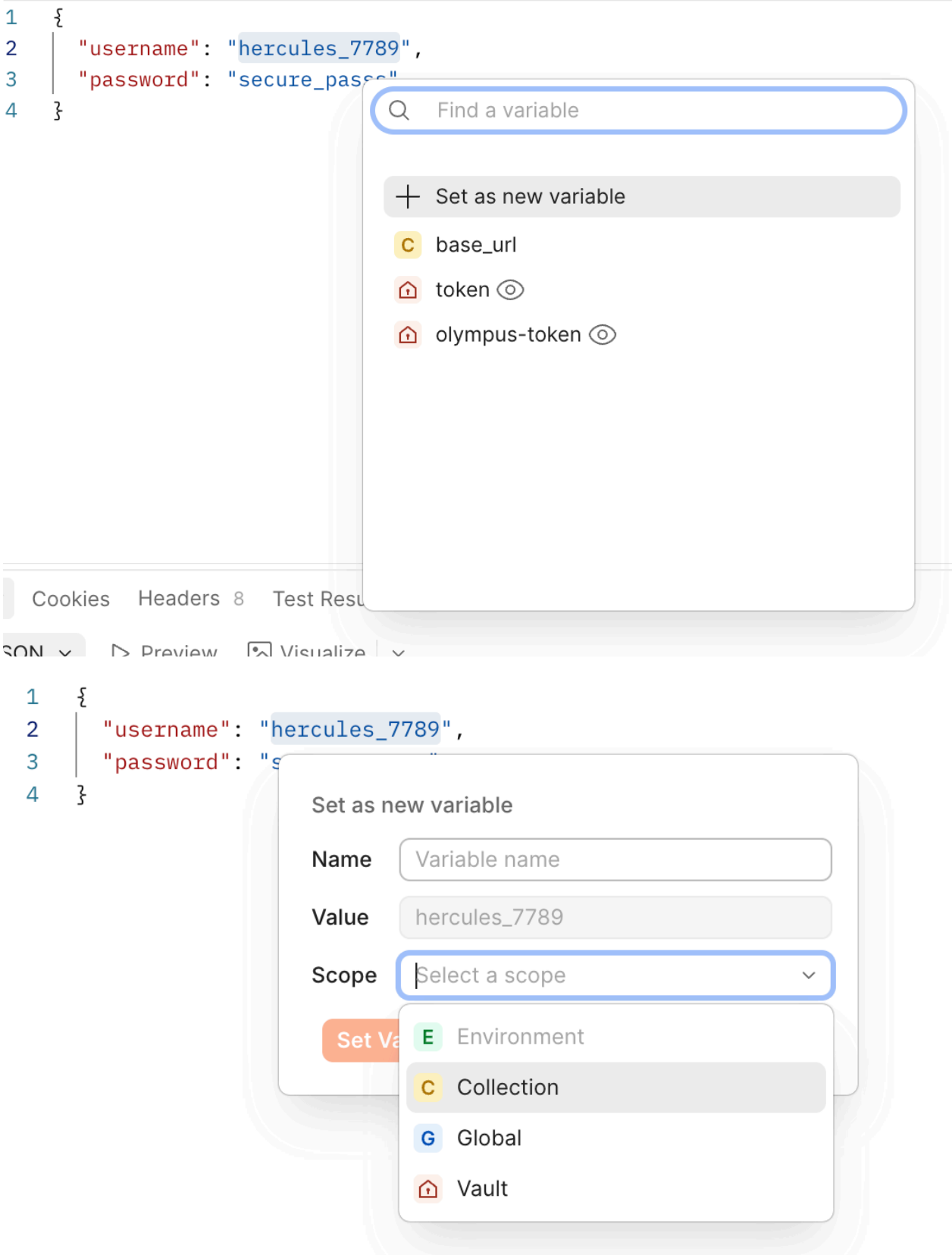
5. Давайте создадим POST-запрос на регистрацию и заменим значение username в теле на переменную.

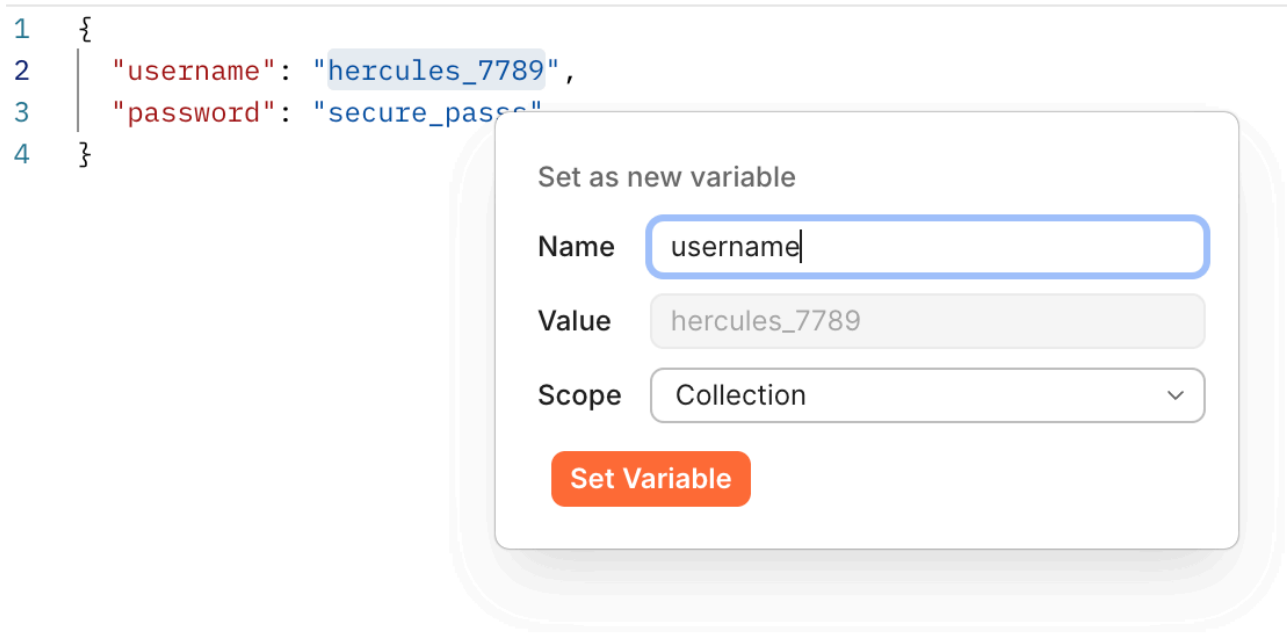
В POST-запросе всегда есть тело, которое можно найти в Swagger. Скопируйте его и вставьте в запрос во вкладку Body > Raw > JSON.



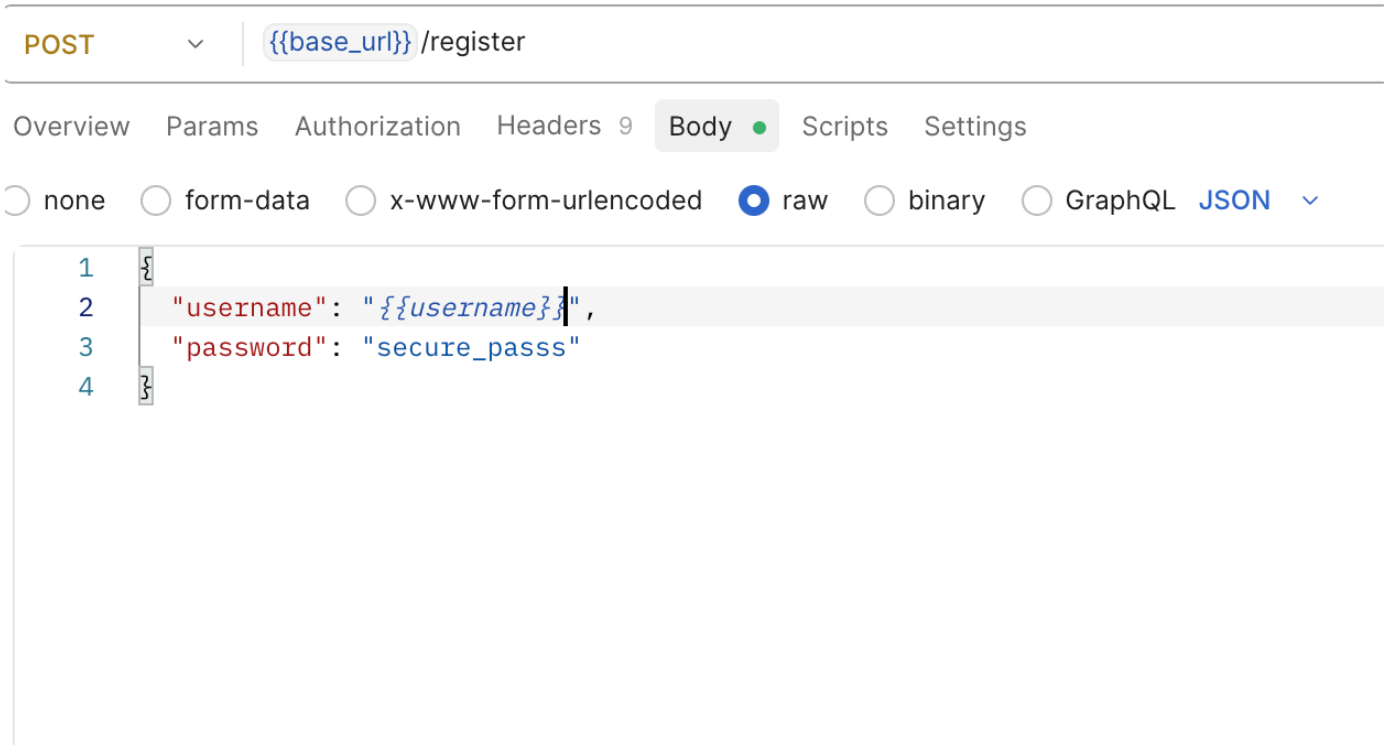

Далее повторите действия со скринов.




{
"username": "hercules_7789",
"password": "secure_pass"
}
Find a variable
Set as new variable
base_url
token
olympus-token
Cookies
Headers 8
Test Resu
Preview
Visualize
{
"username": "hercules_7789",
"password":
}
Set as new variable
Name
Variable name
Value
hercules_7789
Scope
Select a scope
Set Va
Environment
Collection
Global
Vault




После нажатия на **Set Variable** переменная создается на уровне коллекции и автоматически подставляется в тело запроса.




**Обратите внимание, что в значении переменной нет кавычек. Обязательно добавляйте их в само тело, если это формат string.**

## Окружения

**Environment** (окружение) – это набор переменных, который можно активировать "поверх" коллекции.

**Зачем это нужно?** Чтобы не плодить дубликаты запросов для разных серверов (Dev, Test, Prod). То есть мы можем использовать одну и ту же коллекцию, но с разными наборами переменных.

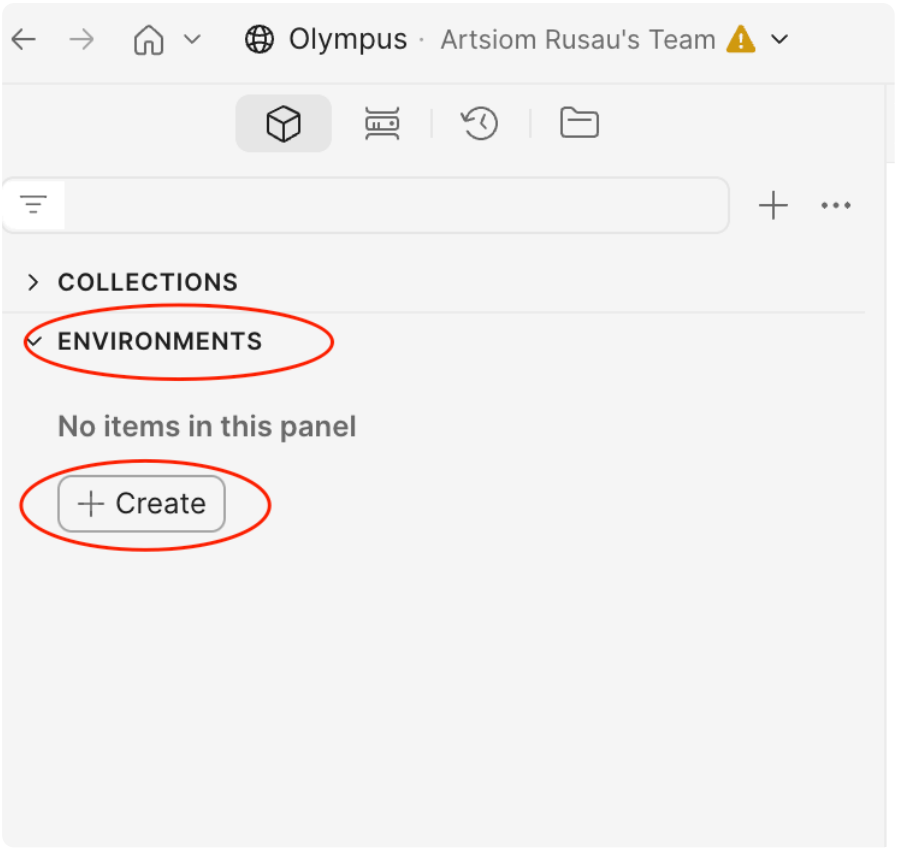
**В реальной разработке есть разные окружения для разных целей:**

- *Local* – локальная среда на твоем компьютере.
- *Dev* – песочница для разработчиков, где они тестируют свой код.
- *Test* – среда для тестировщиков, где мы тестируем код, предоставленный разработчиками.
- *Prod* – среда, где с нашим приложением работает конечный пользователь.

Механика окружений в Postman позволяет менять наборы переменных одним нажатием мышки.

**Давайте создадим такой набор** и посмотрим как это работает:

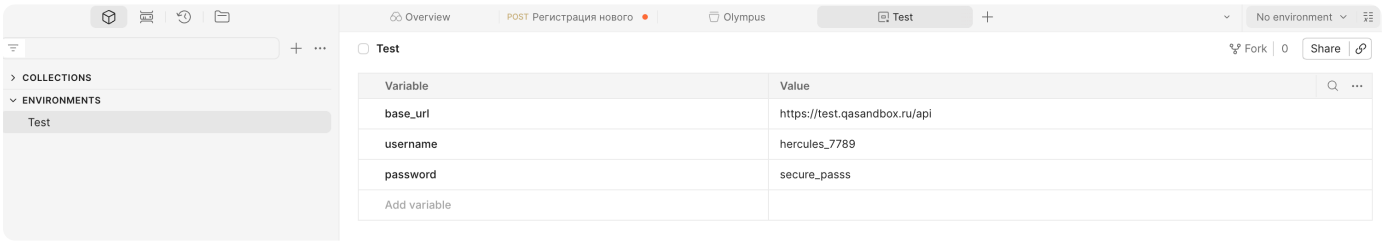
1. На уровне коллекции у нас есть 3 переменные: `base_url`, `username`, `password`. Предположим, что у нас есть отдельный тестовый сервер и мы хотим переместить этот набор туда. Сначала зайдем в панель Environments и нажмем на кнопку Create для создания первого окружения.



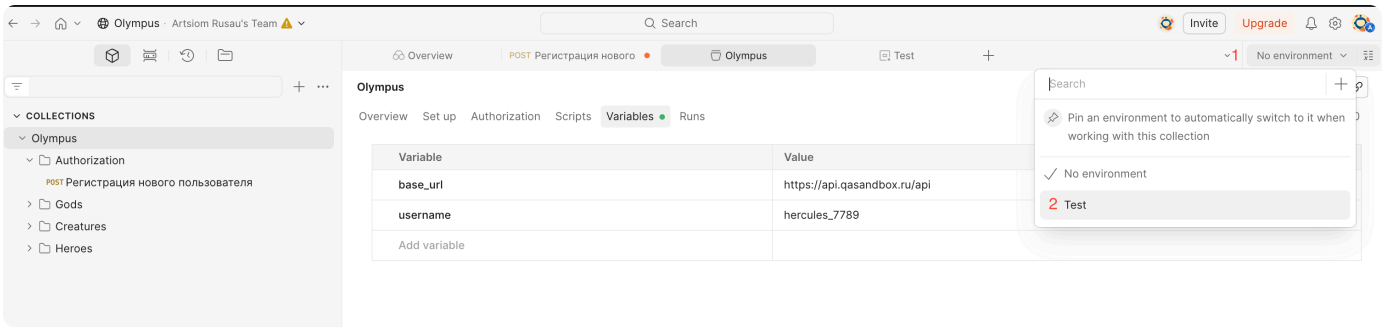

2. Замените название New Environment на Test и добавьте три переменные. Обратите внимание, что значения переменных, как и в коллекции, могут быть локальными и публичными.

*Локальные (Local by default) – доступны только вам, публичные (shared) – доступны всем, у кого есть ссылка на публичную коллекцию.*

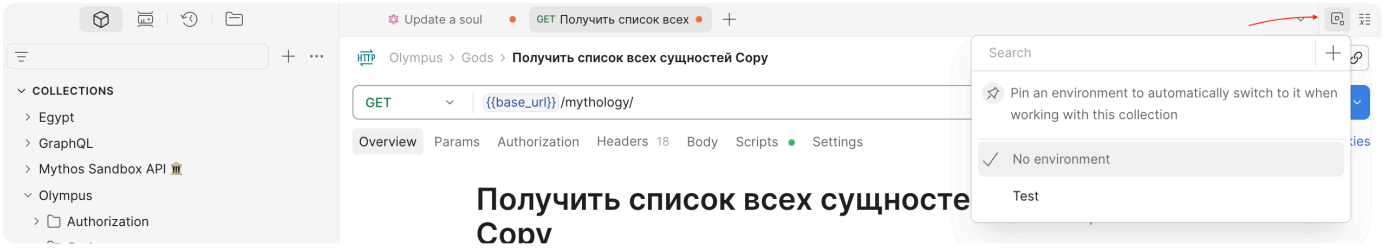
У вас должен получиться примерно такой результат.



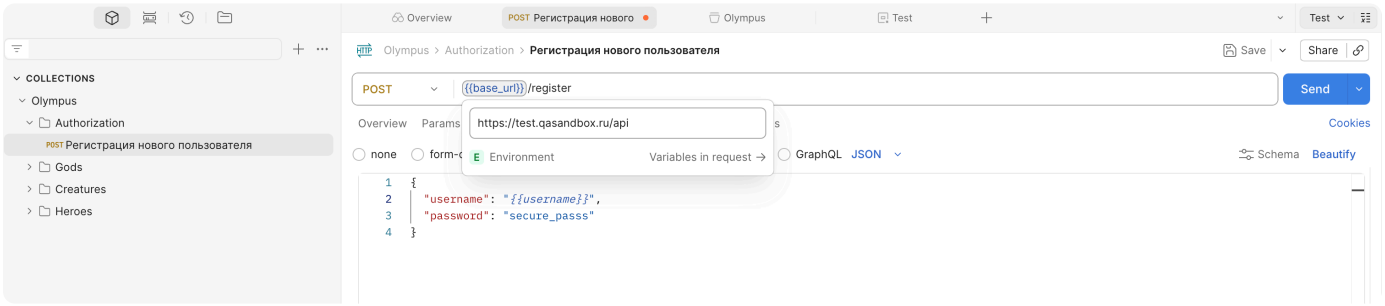

3. Давайте вернемся к нашей коллекции и выберем окружение Test.




Если у вас нет такого селектора, то он может прятаться здесь:




4. Теперь при наведении на переменную, вы увидите новое значение и указание на то, что используется окружение (буква E). Следовательно, переменная окружения `base_url` приоритетнее и заменяет переменную коллекцию `base_url`, которая также сейчас есть в коллекции. Если вы отмените выбор окружения, то снова будет использоваться переменная коллекции.




5. Таким образом вы можете создавать сколько угодно окружений, наборов переменных в них и выбирать нужное в зависимости от задачи!

## Полезные материалы

1. Еще одна версия курса по Postman 2025 https://www.youtube.com/playlist?list=PLKbJd47KcbjtfLcro9sO1jVqK3KXYxb3_
2. И еще одно видео на тему изменений интерфейса https://youtu.be/VlQbh0b5MJA
3. Официальный сайт postman.com
4. Тестовые площадки из урока https://reqres.in/
5. http://users.bugred.ru/

6. https://petstore.swagger.io/
7. https://api.qasandbox.org/swagger/
8. Learning Center by Postman (обучающие материалы) https://learning.postman.com/
9. Переменные в Postman https://learning.postman.com/docs/sending-requests/variables/
10. Полезный гайд по Postman на русском языке https://testengineer.ru/gajd-po-testirovaniyu-v-postman/

ТЕСТИРОВАНИЕ API

# 42. API-документация. Swagger и типы данных (версия 2026)

### Swagger

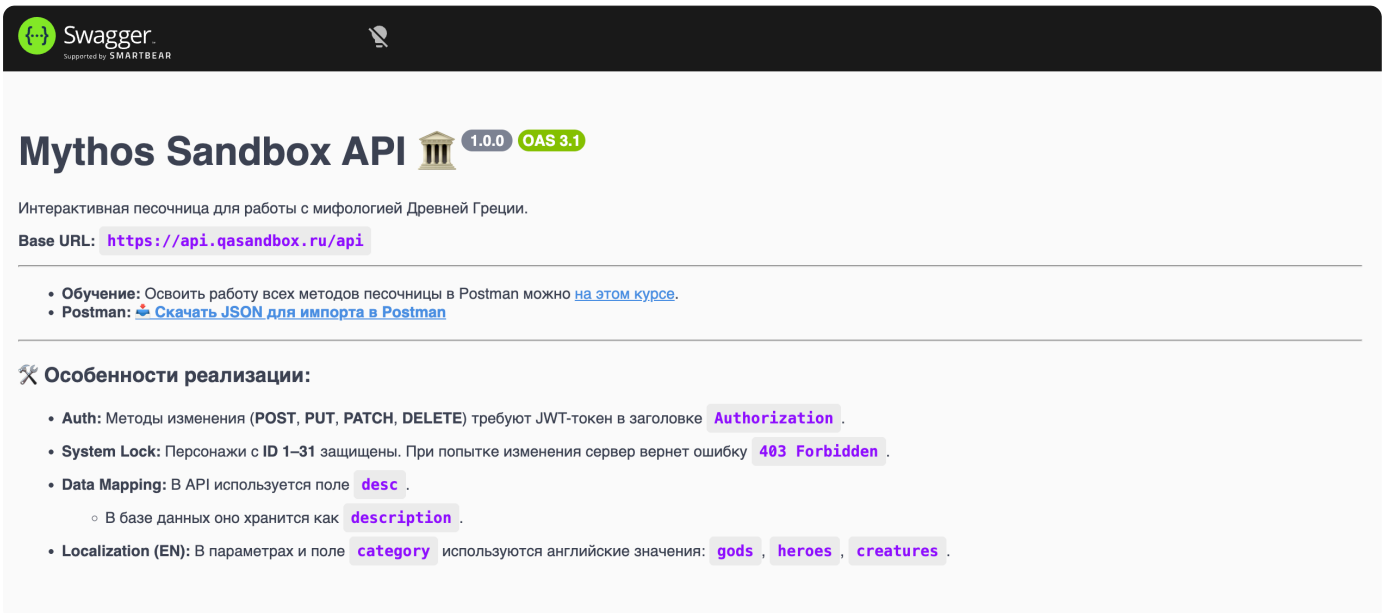
Одним из способов хранения документации API на проекте является Swagger. В нем содержится информация по всем методам, запросам и ответам, которые используются в системе и понадобятся для тестирования.

Разработкой подобной документации на проекте чаще всего занимаются разработчики. Без документации тестировать API очень сложно, поэтому в ваших интересах настоять на ее создании.

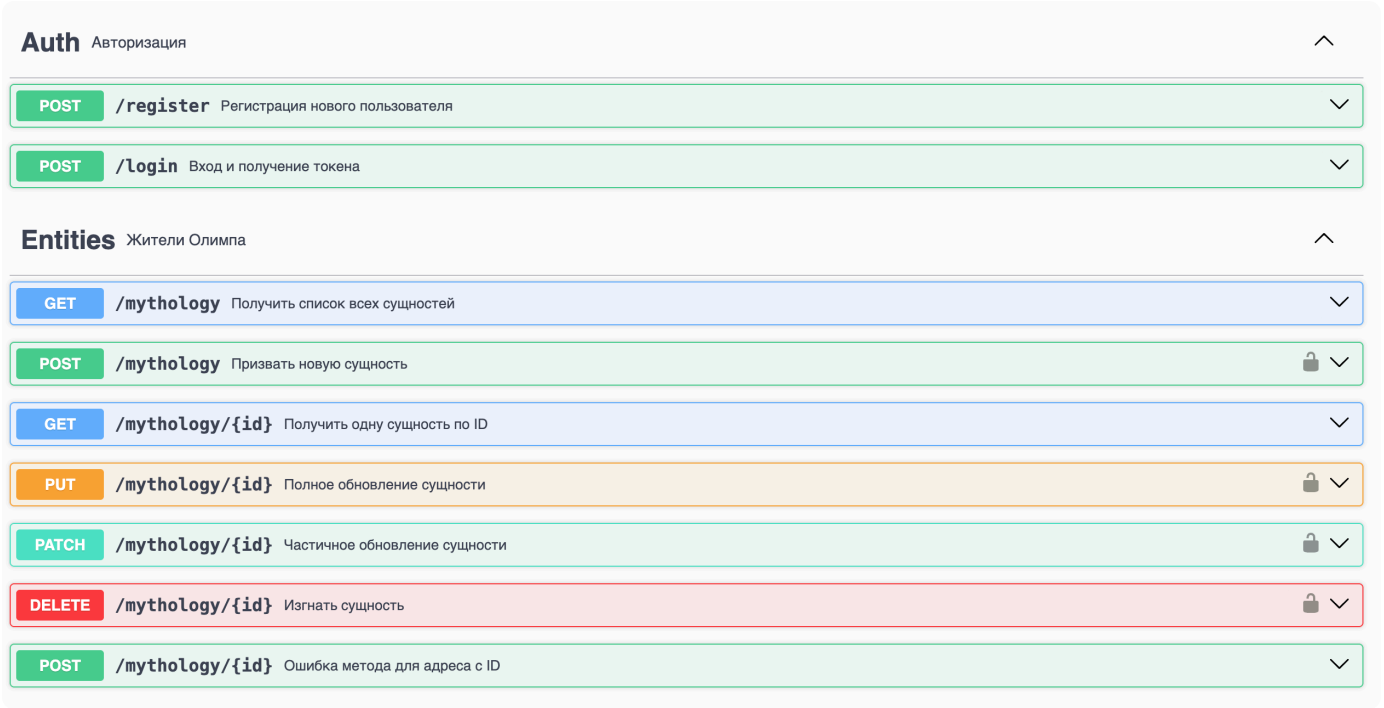
Давайте на примере песочницы Mythos моей разработки познакомимся с интерфейсом и возможностями данного инструмента.

Для начала перейдите по ссылке https://api.qasandbox.org/swagger

1. В шапке вы найдете информацию по **базовому URL**, который нужно использовать для каждого запроса, а также общую информацию о сервисе и настройке авторизации для некоторых методов, если вы захотите их запускать прямо из Swagger.

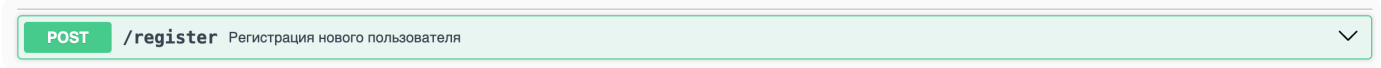



2. Далее вы найдете список сущностей и методов для каждой из них



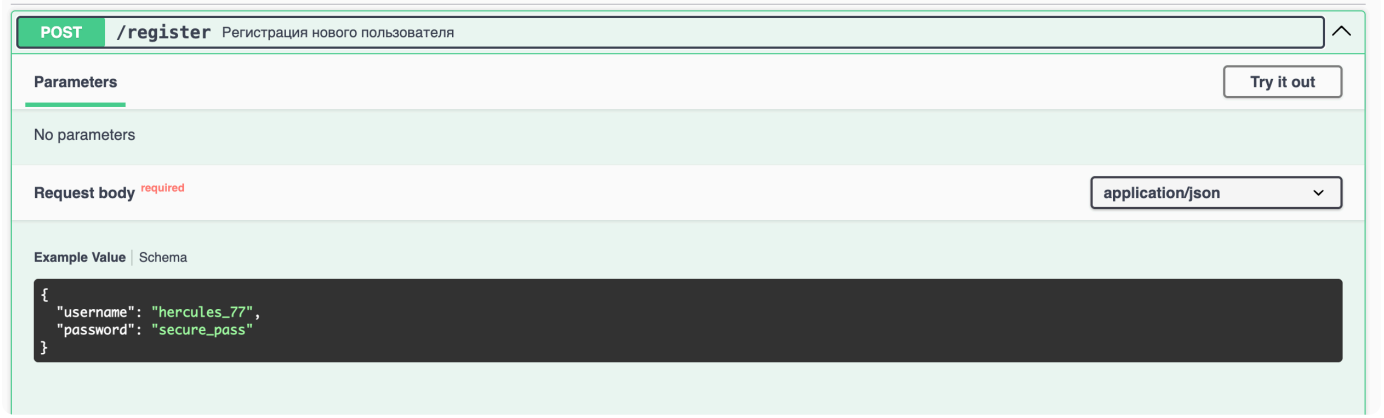

В зависимости от целей тестирования вы выбираете конкретный метод, анализируете и создаете тестовые коллекции в Postman. Давайте рассмотрим описание метода на примере добавления нового питомца

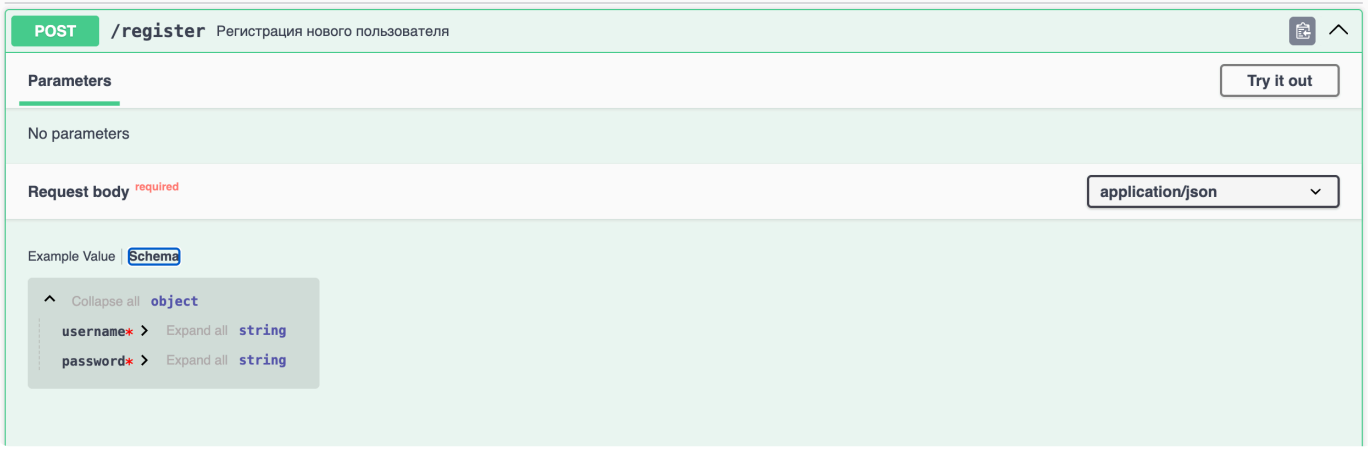
1. В данной строке находится описание метода и **endpoint (ручка)**, который необходимо добавлять к базовому URL для конкретного запроса




2. Далее располагается пример запроса. Для каждого ключа указывается тип данных, который необходимо использовать.

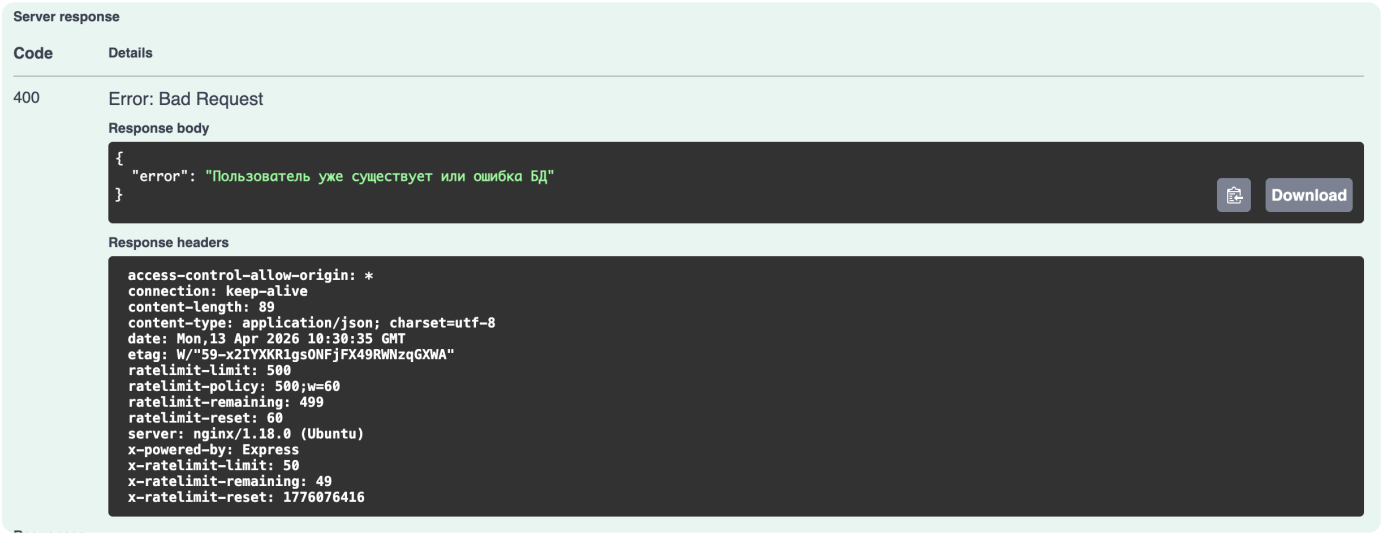
Например, ключ "username" принимает строковый тип данных "string". Тип данных можно посмотреть во вкладке Schema.






Для целей тестирования, вы можете скопировать этот запрос и видоизменить его в Postman. Напоминаю, что для POST необходимо использовать вкладку Body внутри запроса.

3. Чтобы ознакомиться с примером тела ответа нажмите на кнопку Try it out > Execute. Опираясь на его содержимое, вы сможете проверить правильность тела ответа, который будет получен в Postman.




4. Дополнительно вы можете обратить внимание на другие статус-коды и примеры ответов, которые могут быть вызваны после отправки запроса и использовать их для проектирования кейсов.

**Помимо Swagger документация на проекте может быть в виде:**

1. Отдельных документов в базе знаний, Wiki, Confluence
2. Создаваться в Postman для коллекции
3. Redoc в качестве аналога Swagger

Вне зависимости от выбора хранения документации, содержание во всех системах будет похожим: метод, эндпоинт, образец запроса и ответа.

## Основные типы данных в JSON

Формат JSON (JavaScript Object Notation) поддерживает следующие типы данных:

- **Объекты (Objects):** Коллекции неупорядоченных пар «ключ: значение», заключенные в фигурные скобки `{}`.
    - *Пример:* `{"name": "John", "age": 30}`
- **Массивы (Arrays):** Упорядоченные списки значений, заключенные в квадратные скобки `[]`. Если вам нужно поместить несколько самостоятельных объектов в один JSON-документ, для этого как раз используется массив.
    - *Пример списка:* `["apple", "banana", "cherry"]`
    - *Пример массива объектов:* `[{"name": "John", "age": 30}, {"name": "Liza", "age": 55}]`

- **Строки (Strings):** Последовательности символов, которые всегда должны быть заключены в двойные кавычки.
    - *Пример:* `"Hello World"`, `"example string"`
- **Числа (Numbers):** Целые числа или числа с плавающей точкой. Пишутся без кавычек.
    - *Пример:* `123`, `100`, `20.5`
- **Логические значения (Booleans):** Булевы значения, которые могут быть только истинными или ложными. Пишутся строго строчными буквами и без кавычек.
    - *Пример:* `true`, `false`
- **Пустое значение (Null):** Специальное ключевое слово, обозначающее отсутствие данных или пустое значение. Также пишется без кавычек.
    - *Пример:* `null`

> **Что такое "Значения" (Values)?** > В терминологии JSON **значение (value)** – это не отдельный тип данных, а собирательное понятие. Значением в JSON-документе может выступать любой из перечисленных выше типов: строка, число, объект, массив, `true`, `false` или `null`.

### Полезные материалы

1. Swagger - документация https://swagger.io/docs/specification/about/
2. Типы данных в Swagger https://swagger.io/docs/specification/data-models/data-types/
3. Swagger Petstore https://petstore.swagger.io/
4. Redoc Petstore https://redocly.github.io/redoc/#section/Introduction

ТЕСТИРОВАНИЕ API

# 43. cURL в Postman и Swagger (версия 2026)

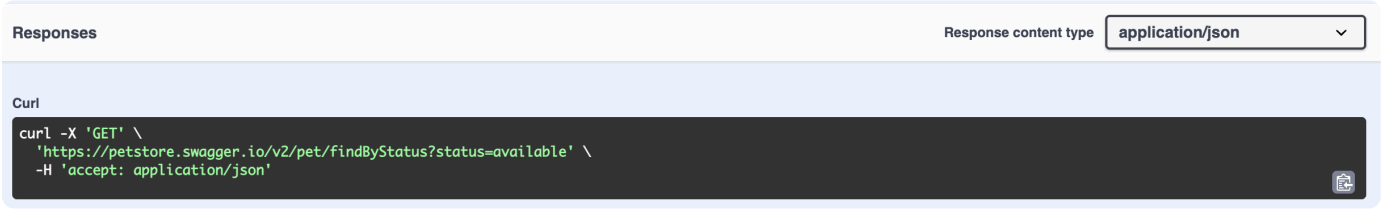
cURL — это командная утилита, которая позволяет отправлять HTTP-запросы из терминала. В тестировании она полезна тем, что позволяет повторить любой запрос без Postman или интерфейса. Например, ты можешь воспроизвести ошибку, не открывая приложение, а просто запустив нужный cURL из консоли.

Мы еще с ней столкнемся в самом конце курса, когда будет проходить bash и его команды.

Swagger позволяет удобно генерировать такие cURL-запросы. Это особенно полезно, если ты хочешь скопировать запрос и вставить его в Postman или сохранить как пример в баг-репорт. Обычно рядом с каждым запросом в Swagger есть кнопка **“cURL”** — она показывает эквивалент команды для терминала.

Ты можешь:

– скопировать cURL из Swagger (для этого сделайте Execute запроса в Swagger с нужными данными и скопируйте cURL из Responses)

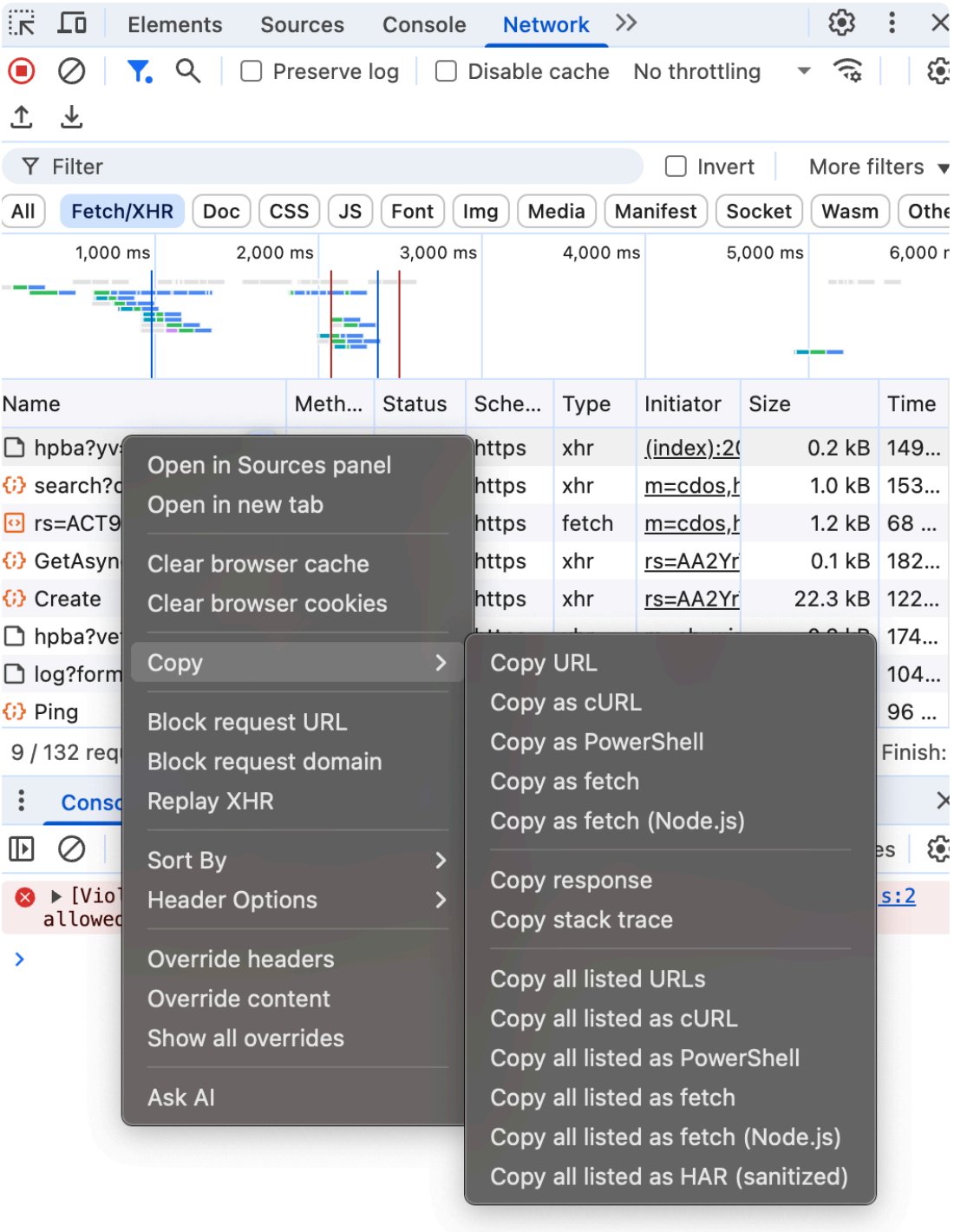



– вставить его в Postman в сам запрос

– после вы сможете выполнить тот же запрос с теми же параметрами и телом

Таким образом ты можешь легко воспроизводить запросы, делиться ими с коллегами и тестировать API вручную.

*Также cURL можно копировать из Devtools во вкладке Network. Это может быть полезно для анализа уже отправленного запроса или понимания, как работает сервер, если на проекте нет документации по типу Swagger.*


Elements
Sources
Console
Network
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
1,000 ms
2,000 ms
3,000 ms
4,000 ms
5,000 ms
Name
Meth...
Status
Sche...
Type
Initiator
Size
Time
Open in Sources panel
Open in new tab
Clear browser cache
Clear browser cookies
Copy
Block request URL
Block request domain
Replay XHR
Sort By
Header Options
Override headers
Override content
Show all overrides
Ask AI
Copy URL
Copy as cURL
Copy as PowerShell
Copy as fetch
Copy as fetch (Node.js)
Copy response
Copy stack trace
Copy all listed URLs
Copy all listed as cURL
Copy all listed as PowerShell
Copy all listed as fetch
Copy all listed as fetch (Node.js)
Copy all listed as HAR (sanitized)

ТЕСТИРОВАНИЕ API

# 44. Идентификация, аутентификация и авторизация (версия 2026)

**Идентификация (Identification)** — процесс, когда клиент (пользователь, приложение) **заявляет** о своей личности. Обычно это передача логина, e-mail или иного уникального идентификатора в запросе.

**Аутентификация (Authentication)** — процесс **проверки** заявленной личности. Клиент доказывает, что он тот, за кого себя выдаёт, с помощью пароля, токена, сертификата и т. п. Статус-код 401 в случае ошибки.

**Авторизация (Authorization)** — процесс **предоставления или отказа** в доступе к конкретному ресурсу или действию на основании прав, которыми обладает уже **аутентифицированный** клиент. Статус-код 403 в случае ошибки.

В тестировании API важно последовательно проверять все три этапа:

1. *Передача идентификатора*
2. *Проверка учётных данных*
3. *Проверка прав доступа*

**Типы аутентификации и их использование в Postman (для ознакомления)**

| Метод | Описание | В Postman: настройка |
|---|---|---|
| **No Auth** | Без механизма безопасности (для публичных или mock-сервисов) | Вкладка **Authorization → Type: No Auth** |

| Метод | Описание | В Postman: настройка |
|---|---|---|
| **API Key** | Специальный ключ (api_key) в заголовке или параметре URL | **Type: API Key**, задаёте имя (например, X-API-KEY) и значение, выбираете «Header» или «Query Params» |
| **Basic Auth** | HTTP-базовая аутентификация: Base64(логин:пароль) | **Type: Basic Auth**, вводите **Username** и **Password** |
| **Bearer Token** | Токен доступа в заголовке Authorization: Bearer <token> | **Type: Bearer Token**, поле **Token.** Token извлекается из ответа после авторизации в переменную и используется в настройках |
| **Digest Auth** | Сложнее базовой: сервер шлёт «nonce», клиент шифрует ответ | **Type: Digest Auth**, вводите **Username**, **Password**, остальные параметры Postman подтягивает автоматически |
| **OAuth 2.0** | Стандарт авторизации через токены (Authorization Code, Client Credentials, ...) | **Type: OAuth 2.0** → **Get New Access Token** → заполняете Client ID/Secret, URL авторизации и токена → **Use Token** |
| **Mutual TLS** | Аутентификация клиентским SSL-сертификатом | В **Settings → Certificates** добавляете домен и путь к сертификату (.crt/.key) |

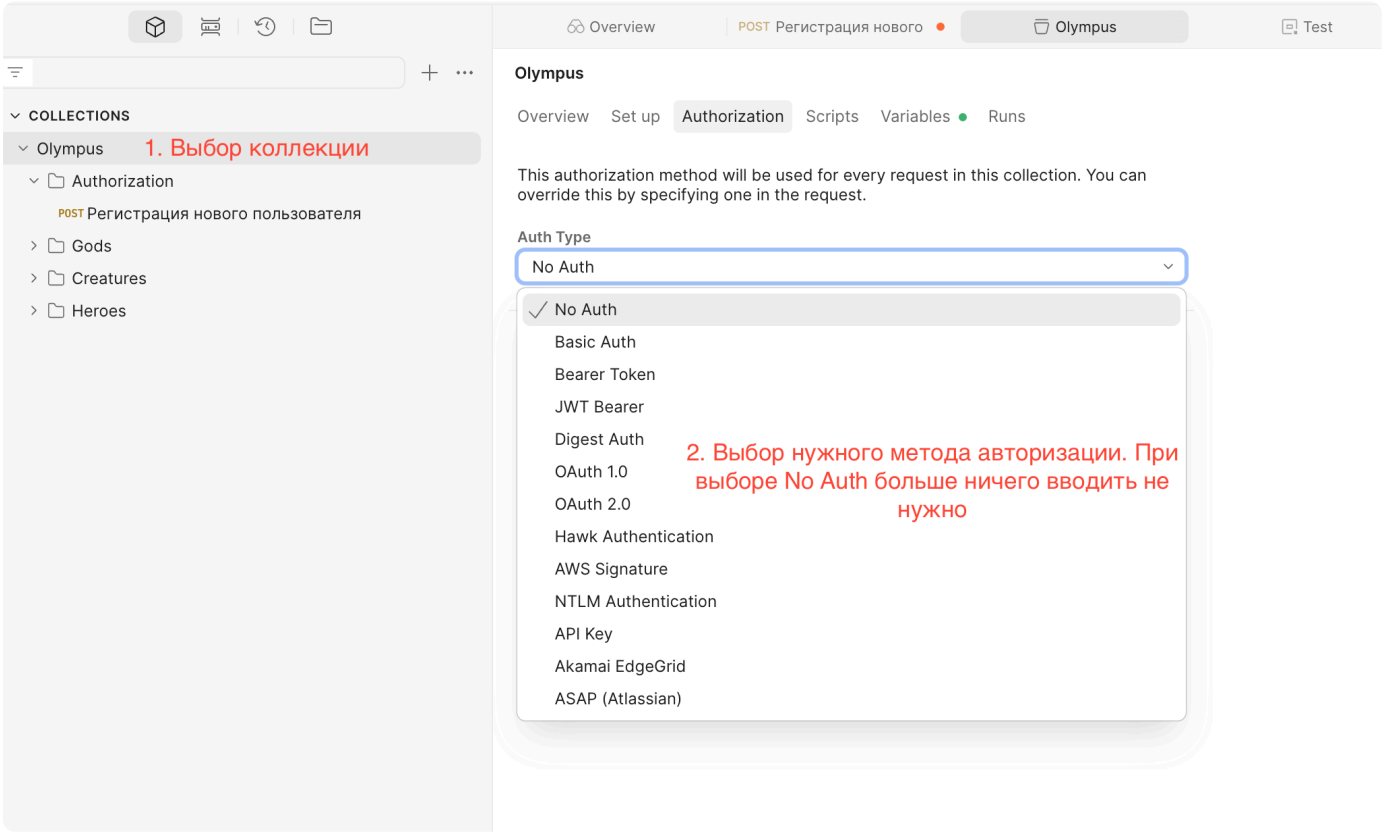
**Базовая авторизация в Postman**

1. **No Auth:** Используется для публичных эндпоинтов. Сервер не проверяет личность отправителя. Например, при отправке запроса GET на получение

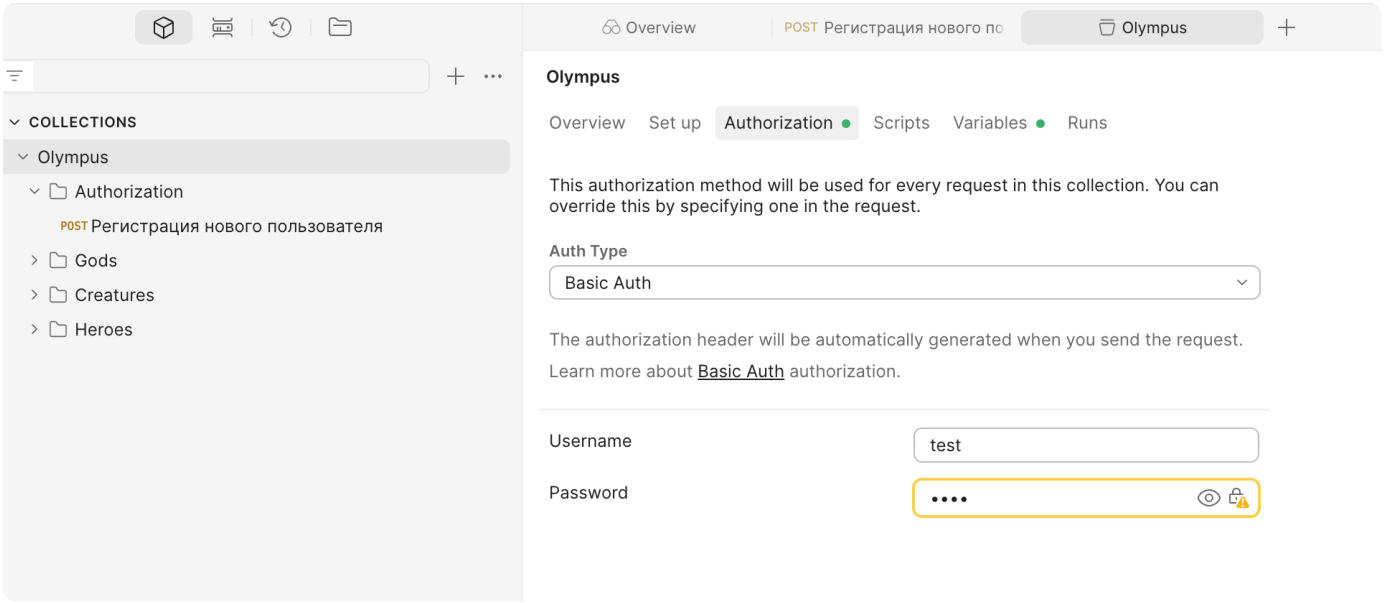
всех сущностей нам не нужны логин и пароль.

2. **Basic Auth (базовая авторизация):** Самый простой метод. Вы передаете `Username` и `Password`.
    - **Как это работает:** Postman берет эти данные, объединяет их через двоеточие (`admin:password123`) и кодирует в формат **Base64**. Затем это помещается в заголовок `Authorization: Basic [код]`.
    - **Важно:** Это НЕ шифрование, а кодирование. Любой, кто перехватит запрос (если нет HTTPS), сможет его прочитать.
    - Это небезопасный способ и в современных проектах он используется редко.

Давайте посмотрим, где можно указать авторизацию в Postman. Это можно сделать на уровне коллекции, папки и запроса. Механизм один и тот же. Он реализуется через вкладку Authorization.




При выборе Basic Auth нужно ввести логин и пароль. По умолчанию выбранный метод авторизации для коллекции будет работать со всеми вложенными в нее папками и запросами. Подробнее об этом в следующем шаге.




## Bearer Token и JWT

**Bearer Token** (дословно "Токен предъявителя") – это самый распространенный метод авторизации в современных REST API. Его логика проста: "Любой, кто предъявит этот токен, получает доступ".

1. **Формат JWT (JSON Web Token):** Чаще всего Bearer-токены представляют собой JWT. Это длинная строка, состоящая из трех частей, разделенных точками: `Заголовок.Полезная_нагрузка.Подпись`. Внутри зашифрована информация о том, кто ты такой и какие права у тебя есть.

2. **Заголовок запроса:** Токен всегда передается в заголовке `Authorization`.

   > **Пример:** `Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...`

3. **Срок жизни (TTL — Time To Live):** В отличие от API-ключа, токен "протухает". У него есть время жизни (например, 3600 секунд). Как только время вышло, сервер вернет **401 Unauthorized**, и тебе придется снова отправить запрос на `Login`, чтобы получить свежее "благословение".

### Как формируется токен?

1. Клиент отправляет запрос на endpoint аутентификации с учётными данными.
2. Сервер их проверяет и в ответе возвращает токен (Bearer), обычно в теле ответа или заголовке.
3. Далее клиент добавляет этот токен в заголовок `Authorization: Bearer <token>` каждого следующего запроса.

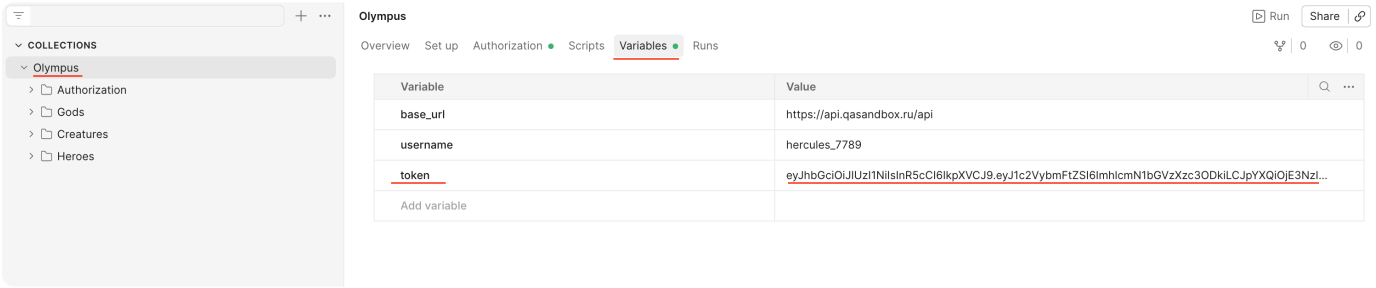
4. Сервер на каждом запросе проверяет токен и либо разрешает доступ, либо возвращает ошибку.

**Как это работает в Postman?**

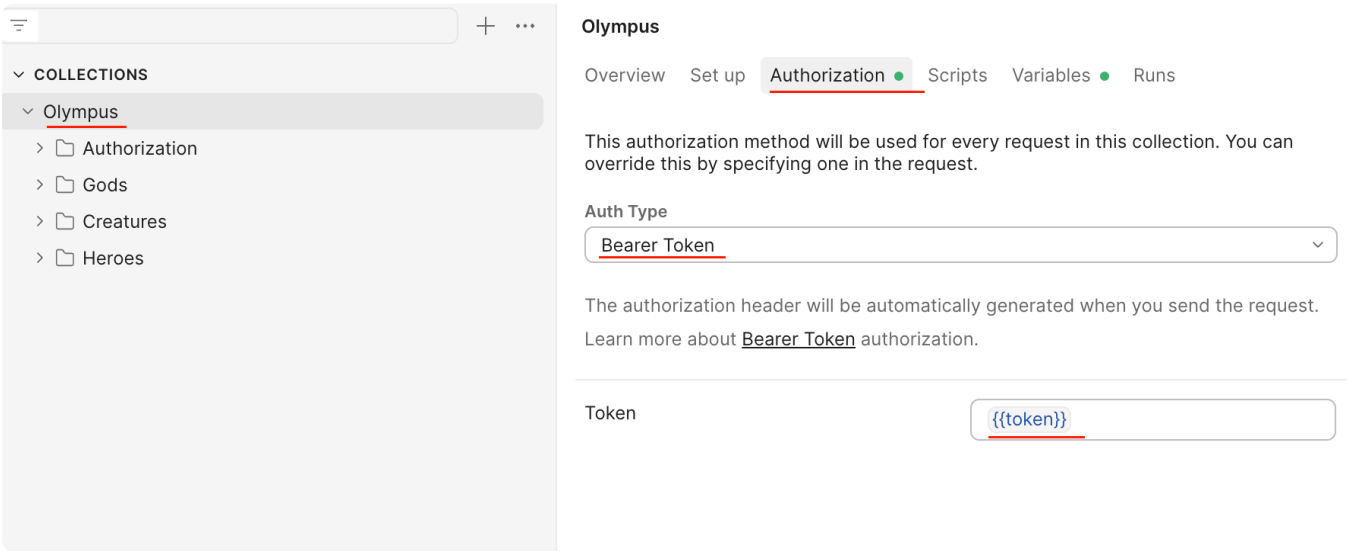
1. У нас уже есть запрос на регистрацию пользователя и сам пользователь. Теперь нужно войти в систему и получить токен авторизации. В предыдущих заданиях мы отправляли запрос POST: "Вход и получение токена". Отправьте его еще раз.

2. После отправки вы получите тело ответа с токеном. Скопируйте значение токена без кавычек.

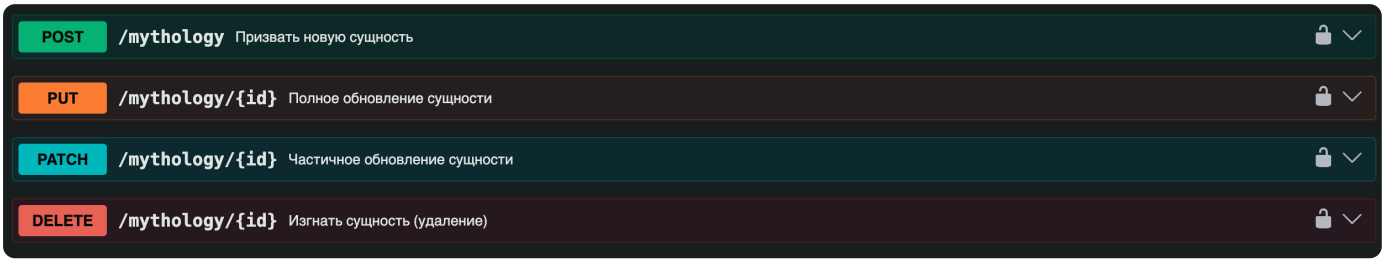
3. Далее перейдите на уровень коллекции > Variables > создайте переменную token и вставьте значение токена туда.



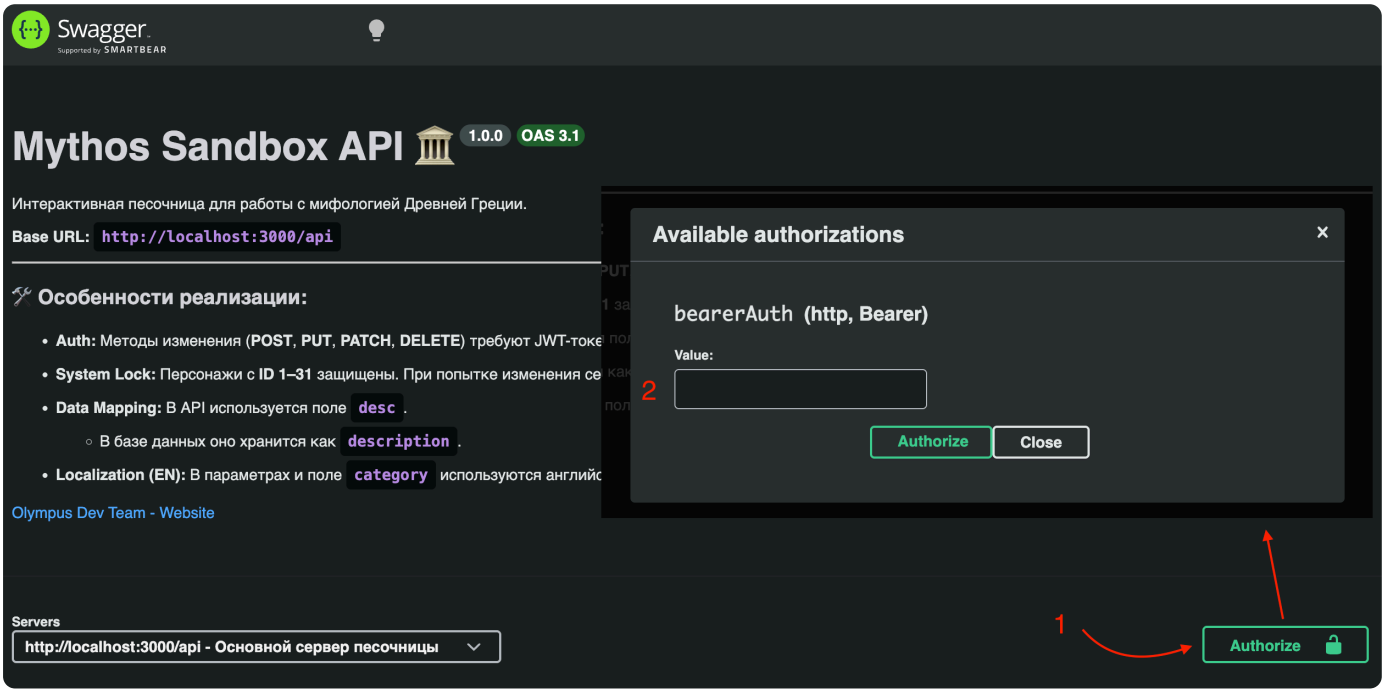

4. После перейдите во вкладку Authorization > выберите значение Bearer Token и в поле token добавьте переменную {{token}}.



5. Теперь при каждом запросе в заголовок Authorization будет добавляться токен авторизации. И с этого момента вы можете выполнять любой запрос из нашей песочницы при выборе данного типа авторизации. Это все запросы с "замком" в Swagger.




6. Чтобы эти запросы выполнялись в Swagger выполните следующие действия.




7. Помните, что у токена есть время жизни, а значит периодически его нужно обновлять.

## Полезные материалы

1. Аутентификация и авторизация https://vladislaveremeev.gitbook.io/qa_bible/seti-i-okolo-nikh/autentifikaciya-i-avtorizaciya-authentication-and-authorization
2. Пример скрипта для bearer token https://drive.google.com/file/d/1z7yDX4N0wjVN9D-ApdxY7tYrZehYucta/view?usp=sharing

ТЕСТИРОВАНИЕ API

# 45. HTTP-методы в Postman. Цепочка запросов (версия 2026)

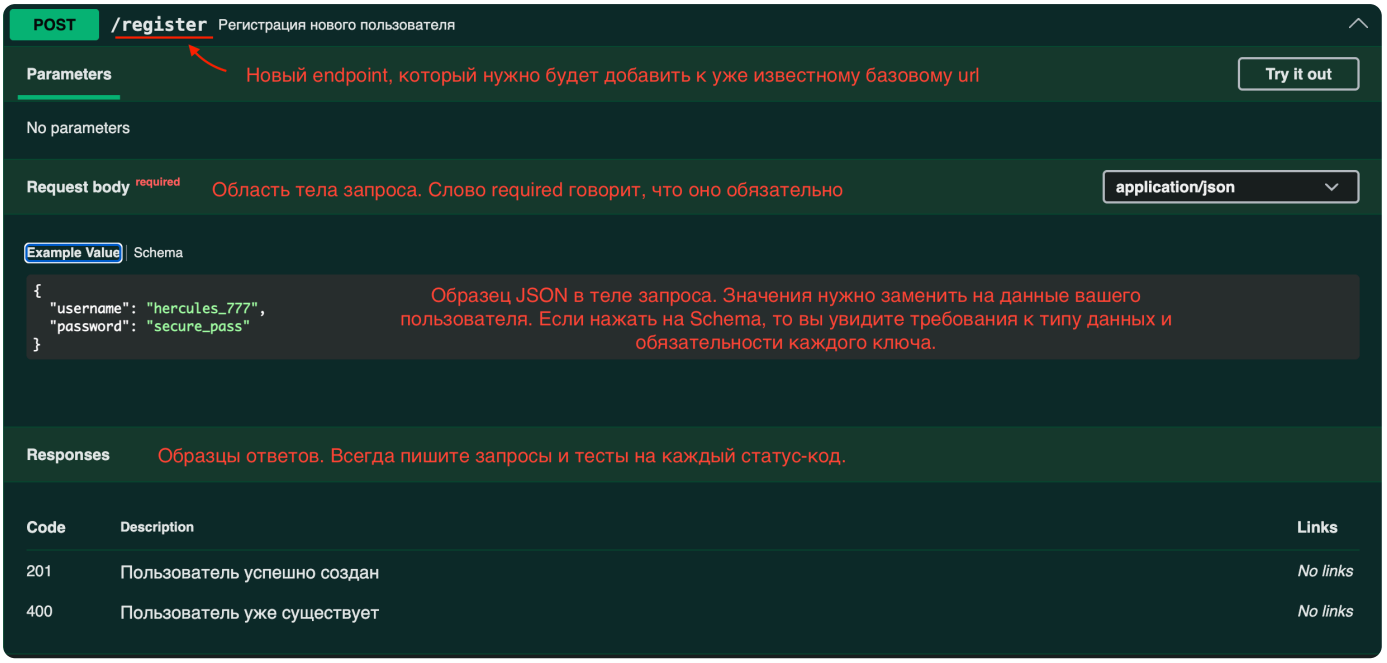
### POST-запрос

Когда ты отправляешь POST-запрос, ты передаешь серверу данные, которых там еще нет.

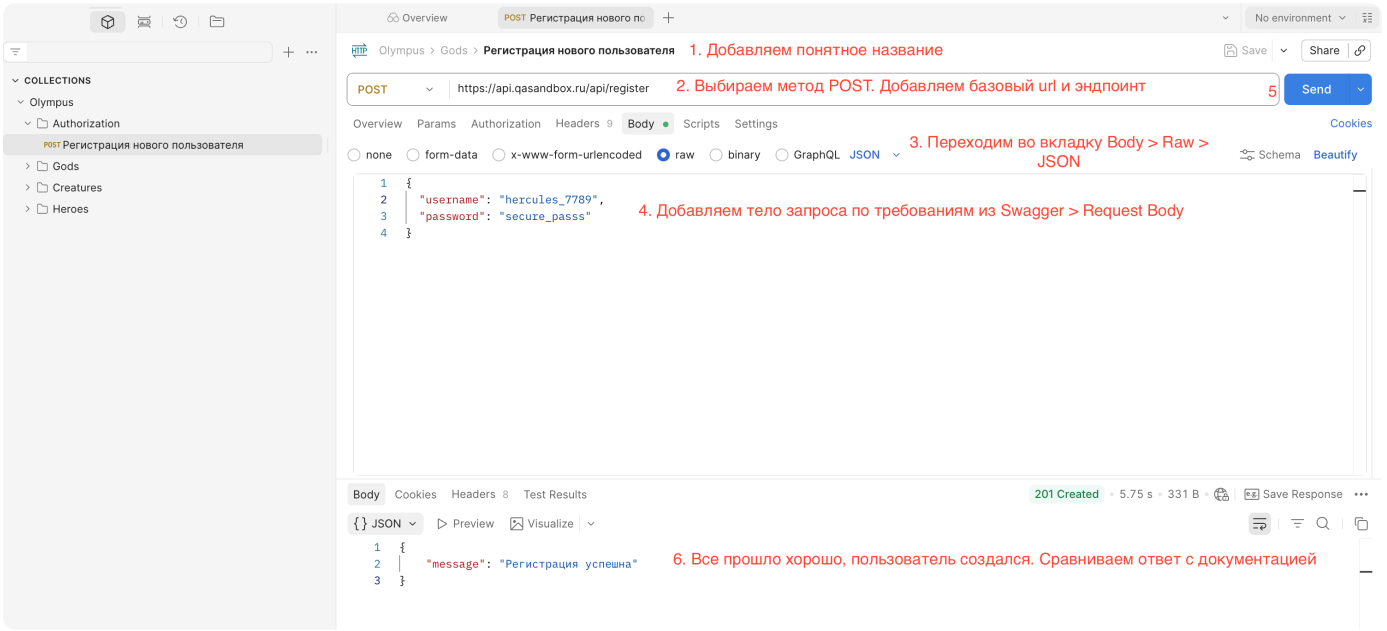
1. **Метод POST:** Используется для создания новых записей. В отличие от GET, здесь данные передаются не в адресной строке, а внутри тела запроса.
2. **Вкладка Body -> raw -> JSON:** путь для добавления тела.
    - **Body:** Вкладка в Postman, где лежит само тело запроса
    - **raw:** Режим, позволяющий писать код вручную.
    - **JSON:** Формат, который понимает сервер. Помни о строгом синтаксисе: ключи и строковые значения **всегда в двойных кавычках**, а пары разделяются **запятыми**. Мы говорили о нем в уроке 1.9.
3. **Автоматические заголовки:** Как только ты выбираешь "JSON" в выпадающем списке, Postman автоматические добавляет скрытый заголовок `Content-Type: application/json`. Это говорит серверу: «Внутри тела – структурированный код, а не просто текст».

**Давай отправим первый POST-запрос в Postman**

1. Вернемся в Swagger и найдем всю нужную информацию для запроса. В данном случае это будет регистрация нового пользователя.




2. Перейдем в Postman и отправим POST-запрос. Для этого создайте новую папку *Authorization и запрос в ней*.




3. Теперь у нас есть первый пользователь, с помощью которого мы можем выполнять все остальные методы после авторизации!

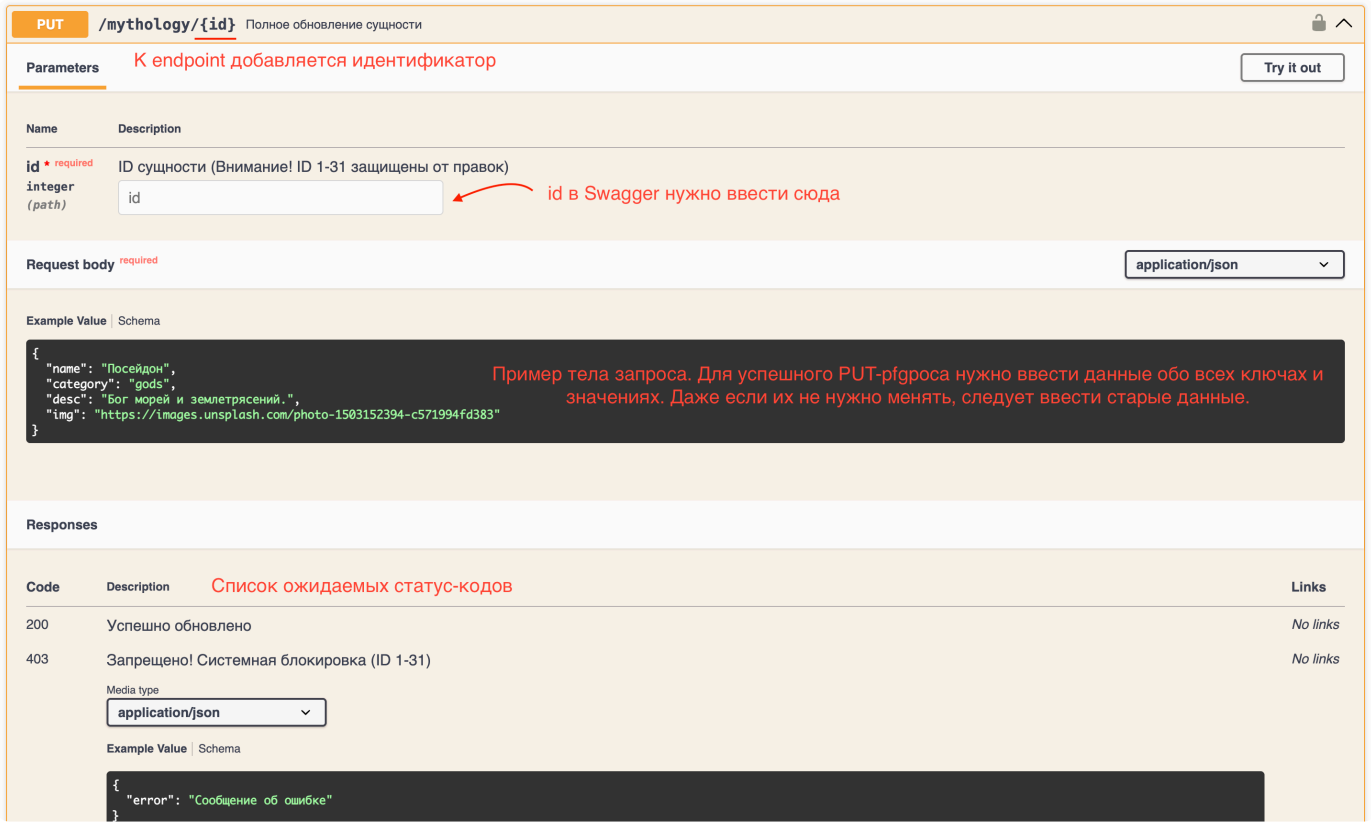
## Как отправить PUT запрос в Postman?

Принцип точно такой же как в POST. Есть url + тело запроса. Но для того, чтобы сервер понимал: какой объект нужно изменить, добавляется идентификатор либо в адресную строку, либо в тело запроса. Все зависит от логики каждого конкретного сервиса.

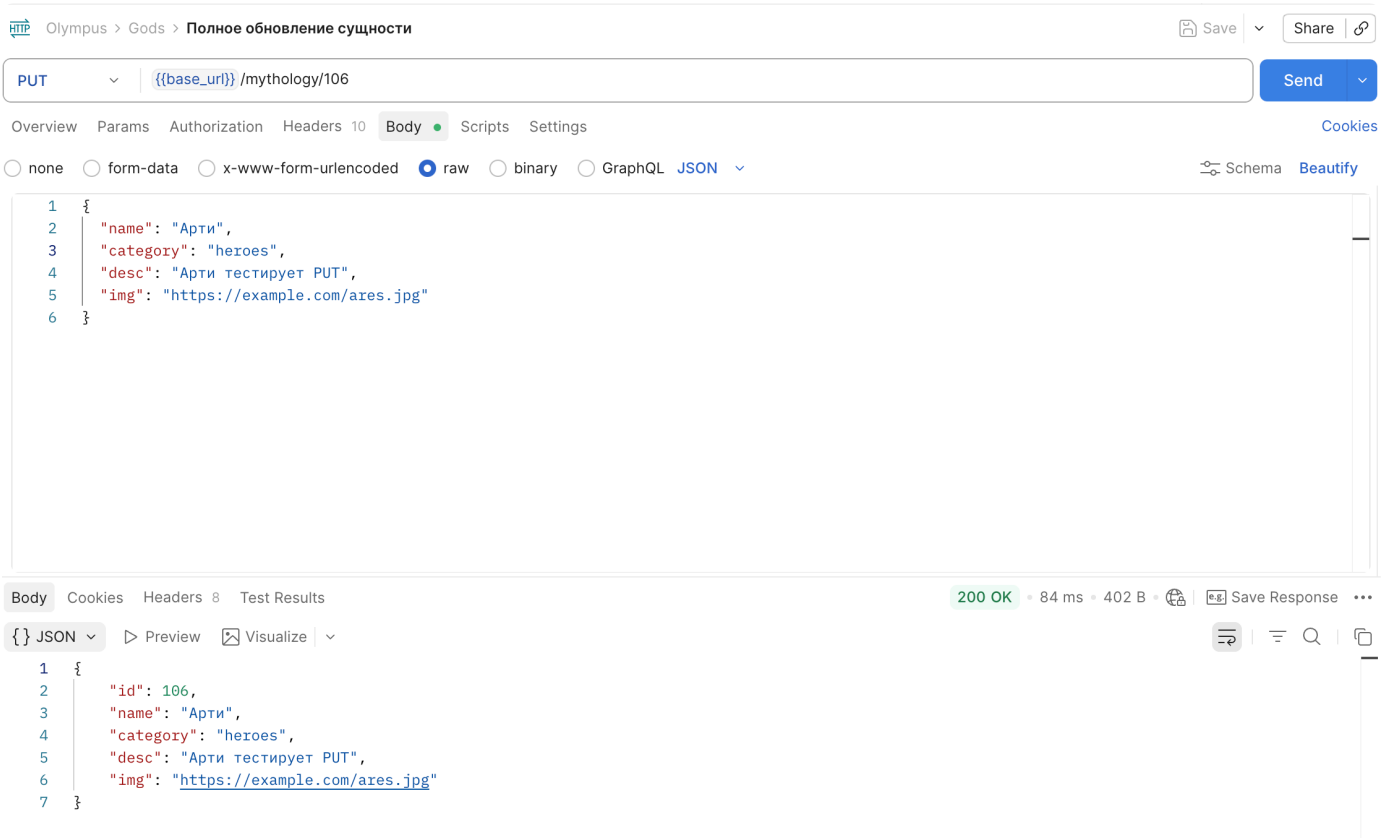
В прошлом уроке ты создал нового героя методом POST. Давай изменим данные этого объекта.

1. Сначала нужно найти id объекта, который нужно изменить. Его можно узнать, используя запрос GET и исследуя полученный массив. Лучше сразу поместить его в переменную *id* для удобства.

2. Далее нужно изучить метод PUT в Swagger.




3. Теперь нужно перенести логику запроса PUT в Postman. Попробуйте сделать это самостоятельно, а потом изучите мой вариант и подсказку. Помните, что для PUT нужна авторизация.



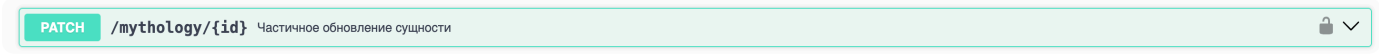

## Как отправить PATCH запрос в Postman?

Принцип точно такой же как в POST. Есть url + тело запроса. Но для того, чтобы сервер понимал: какой объект нужно изменить добавляется идентификатор либо в адресную строку, либо в тело запроса.

В прошлом мы изменили объект. Давай изменим теперь не весь объект, а отдельную пару ключ-значение.

1. Идентификатор у нас уже есть. Мы выяснили его в прошлом шаге.

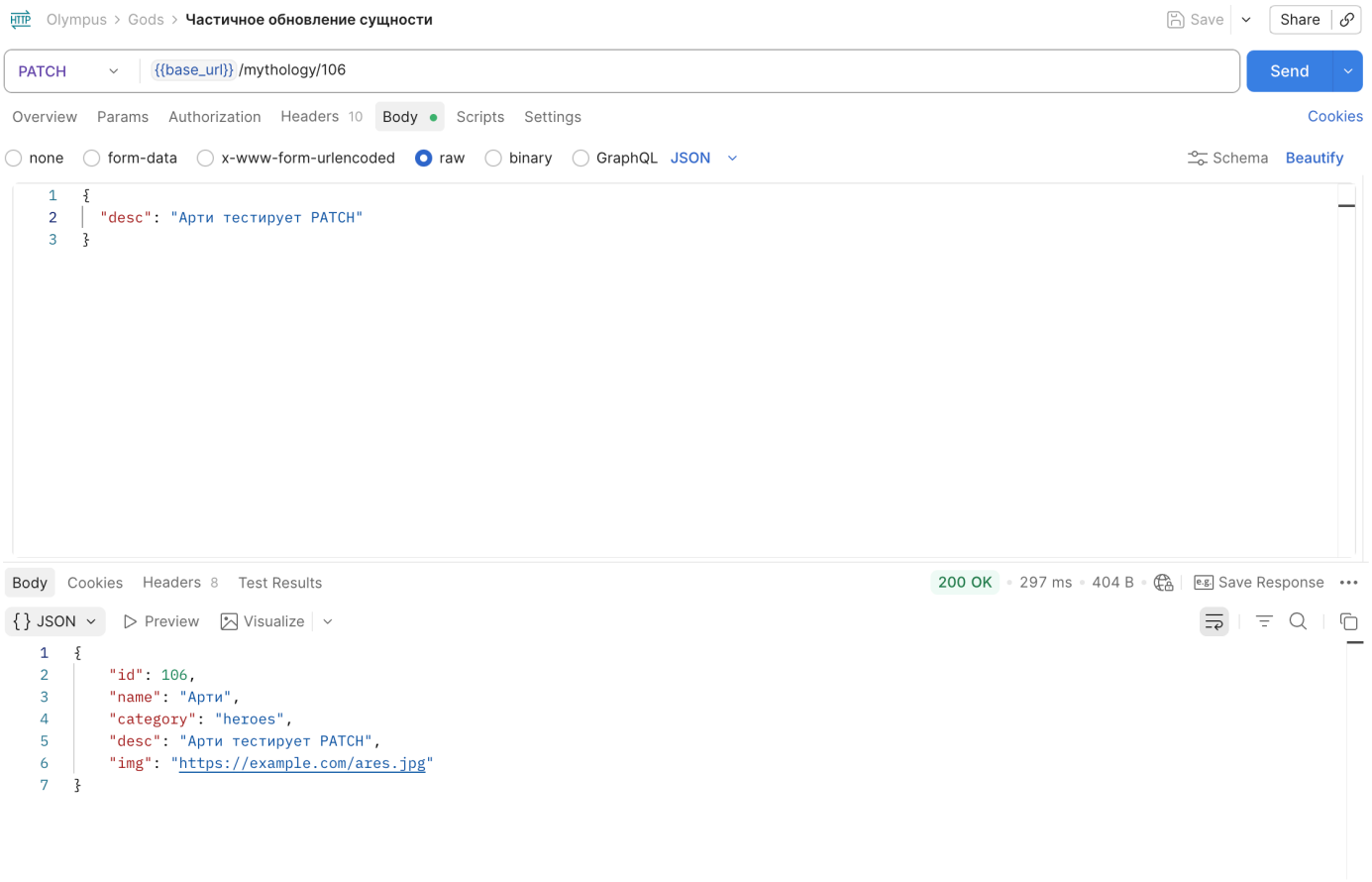
2. Далее нужно изучить метод PATCH в Swagger.




Я не буду останавливаться на всей его структуре, так как она такая же, как и в PUT. Единственное отличие: вы можете менять только определенные пары ключей и значений, а значит вам не нужно использовать весь JSON. Например, если нужно поменять только описание, то достаточно добавить в тело запроса `{"desc": "Новое описание"}`

3. Теперь нужно перенести логику запроса PATCH в Postman. Попробуйте сделать это самостоятельно, а потом изучите мой вариант и подсказку.

Помните, что для PATCH нужна авторизация.

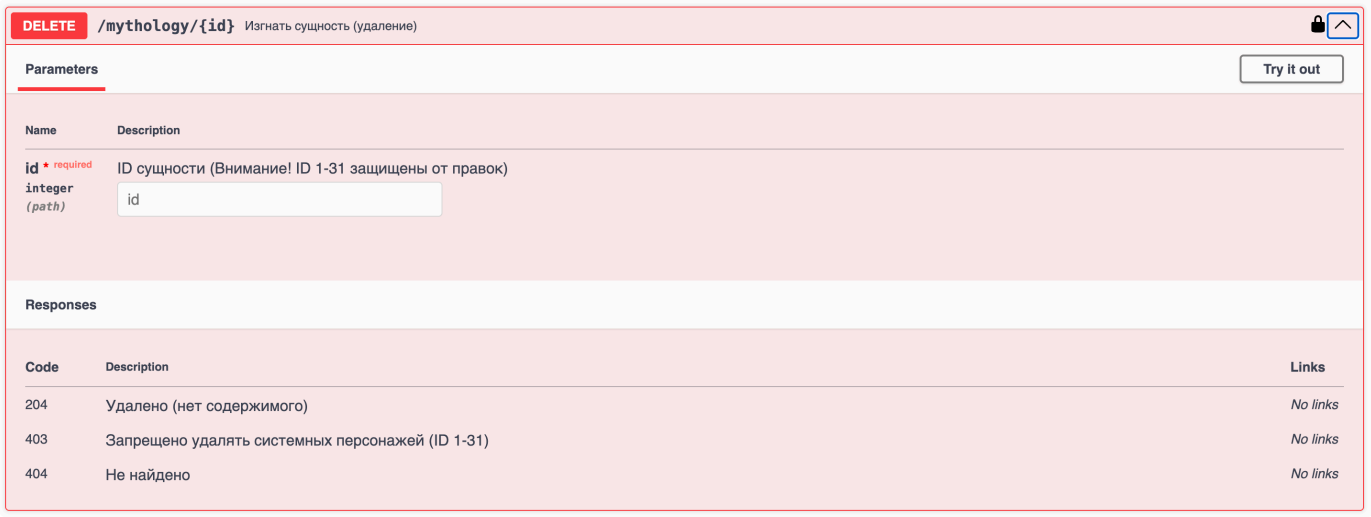



## Как отправить DELETE в POSTMAN?

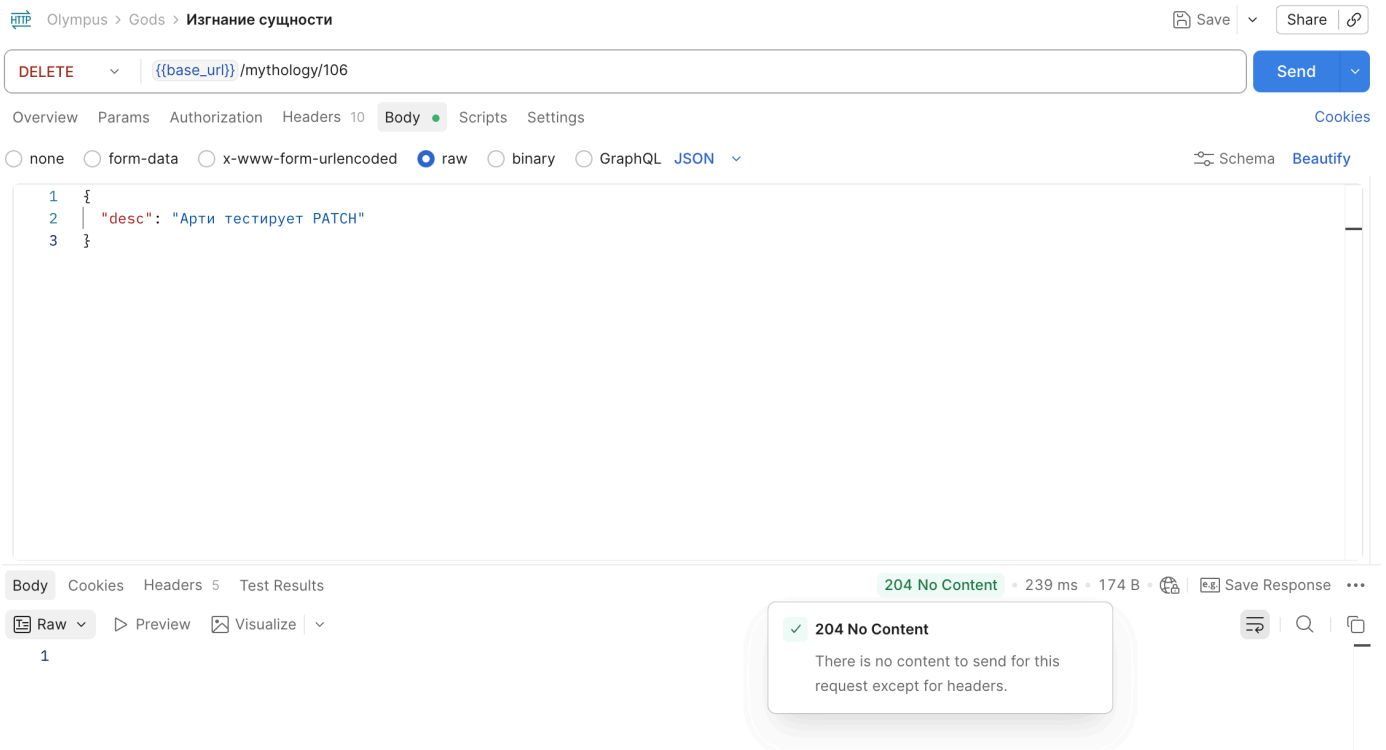
Как и в GET у метода DELETE нет тела запроса. В url нужно просто указать идентификатор удаляемого объекта.

1. Ранее мы создали героя, изменили его через PUT и PATCH, но теперь ему пора на покой.

2. Найдем метод DELETE в Swagger. Вы уже умеете читать такую документацию. Для отправки запроса нам нужен только id, который у вас уже есть.




3. Попробуйте удалить объект самостоятельно, а потом обратитесь к подсказке ниже. Напоминаю, что для отправки запроса нужна авторизация.




Наш герой удален и вы получили 204 статус-код. Попробуйте отправить запрос еще раз и изучите статус-код, который вы получите.

## Chaining (Связывание запросов)

**API Chaining (Цепочки запросов)** – это процесс, когда вы извлекаете данные из ответа одного запроса (например, ID только что созданного пользователя, токен авторизации или номер заказа) и автоматически подставляете эти данные в параметры, заголовки или тело следующих запросов.

Этот процесс можно назвать **динамическим обновлением переменных**. Мы уже делали это для извлечения токена авторизации.

Без этого навыка вы не сможете тестировать реальные пользовательские сценарии, где все шаги взаимосвязаны.

**От переменной до цепочки**

**Шаг 1: Создание сущности** Мы отправляем запрос POST "Призвать новую сущность".

В ответе вы получите примерно такой JSON:

```
{

"id": 37,

"name": "Garrett",

"category": "heroes",

"desc": "Maiores recusandae provident",

"img": "http://placeimg.com/640/480"

}
```

**Шаг 2: Создаем переменную.** Сервер отвечает: `{"id": 37, "name": "Garret", ...}`. Нам нужно перехватить число 37. Мы пишем скрипт, который выполняется сразу *после* получения ответа.

```
// Вкладка "Scripts > Post-response" запроса

// 1. Превращаем сырой ответ сервера в понятный JavaScript-объект
const responseJson = pm.response.json();

// 2. Извлекаем значение для переменной.
// Мы знаем, что она лежит в поле "id".
const hero_id = responseJson.id;
```

```
console.log("Поймана искра нового героя: " + hero_id); // Для отладки в
консоли

// 3. Самое важное: Кладем переменную в Переменную Окружения).
// Мы называем этот сосуд "id". Теперь это значение доступно везде!
pm.environment.set("hero_id", hero_id);

// 4. Хорошим тоном будет сразу проверить, что мы поймали именно число, а не
пустоту.
pm.test("Искра (ID героя) успешно получена и сохранена", function () {
    pm.expect(hero_id).to.be.a('number');
    pm.expect(pm.environment.get("hero_id")).to.eql(hero_id);
});
```

**Шаг 3: Использование цепочки** Теперь можно создать следующий запрос, например, PUT-запрос — "Полное обновление сущности". Нам больше не нужно знать конкретный ID. Мы используем переменную hero_id. Postman сам заглянет в окружение и подставит актуальное значение.

Если вы запустите эту коллекцию в Collection Runner в следующем уроке, первый запрос создаст героя и сохранит его ID, а второй запрос автоматически использует этот ID, для изменения. Цепочка замкнулась!

ТЕСТИРОВАНИЕ API

# 46. ДОП: Обзор серверной части приложения на курсе

### Полезные материалы

1. Ссылка на репозиторий с исходным кодом приложения

ТЕСТИРОВАНИЕ API

# 47. Простые автотесты. Тест-кейсы и чек-листы (версия 2026)

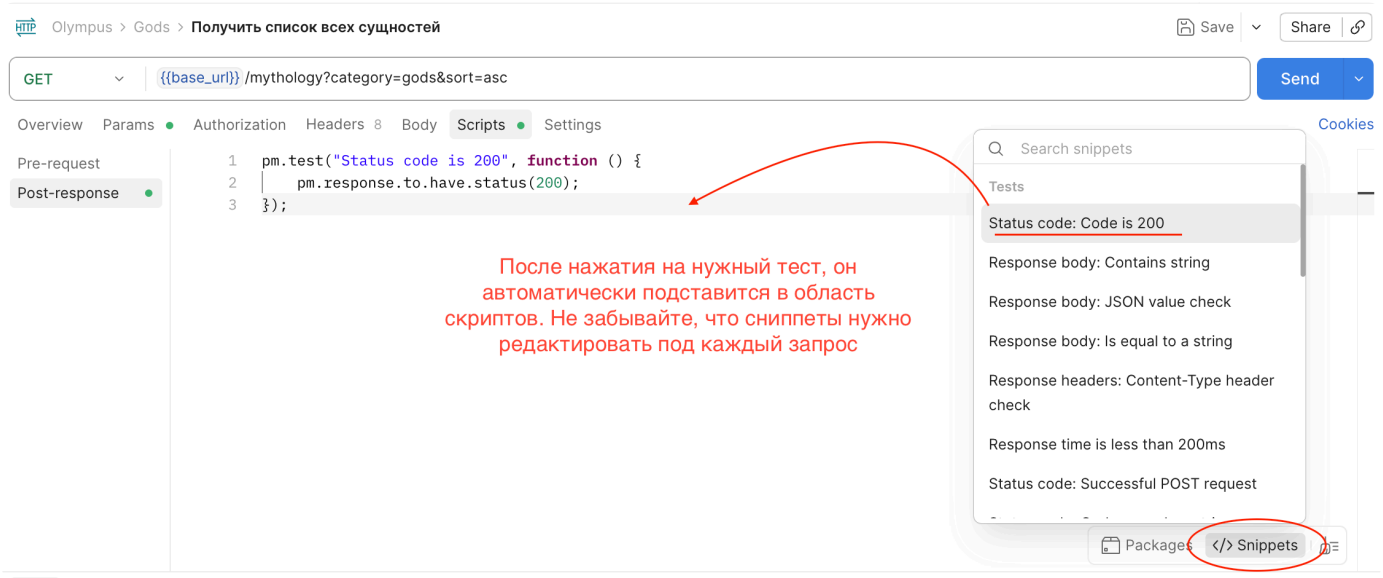
### Простые автотесты

Автотесты в Postman пишутся на языке **JavaScript** в специальной библиотеке Chai. В задачу данного курса не входит обучение этому языку программирования, а только понимание самих тестов и их применению.

Весь процесс тестирования строится вокруг функции `pm.test()`.

*Такие тесты можно легко писать, используя ChatGPT и другие ИИ-агенты, но нам важно не бояться их делать самим.*

*В самом Postman есть заготовленные образцы самых популярных тестов (сниппеты), их можно использовать в качестве отправной точки.*



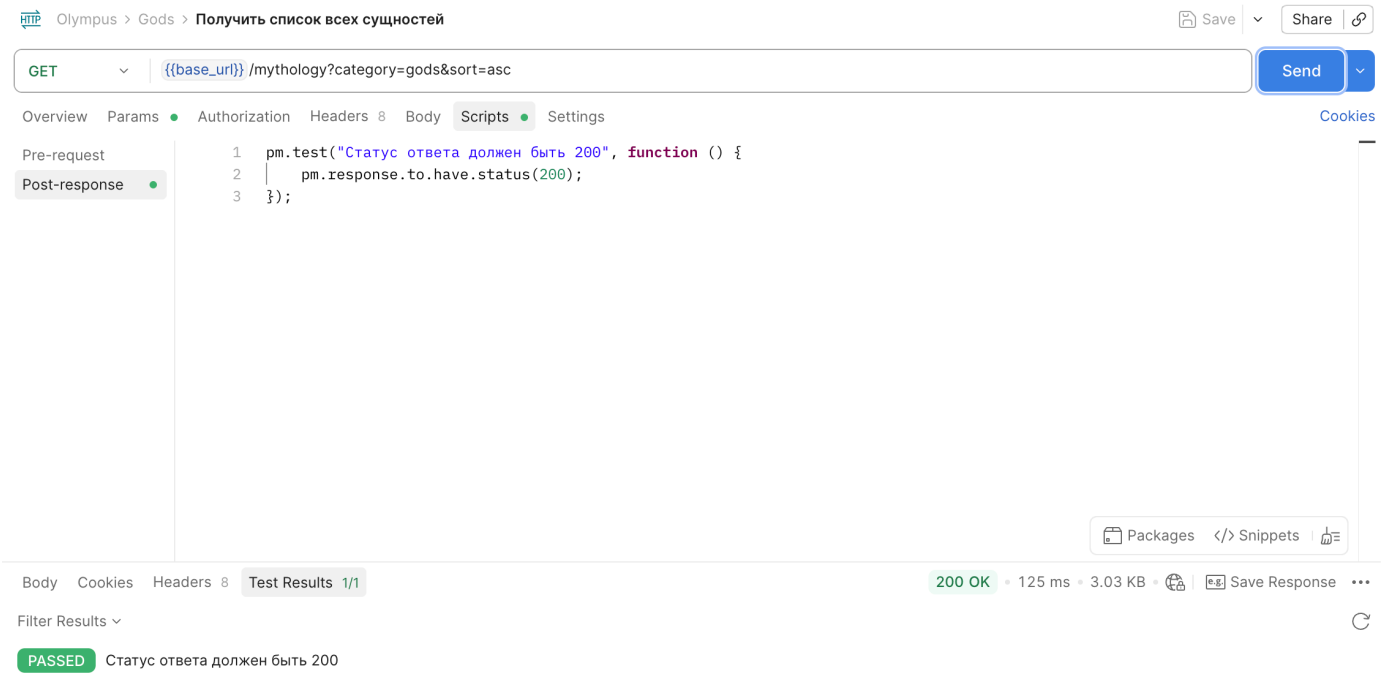

### Тест на статус-код

1. **Выбери запрос и перейди во вкладку Scripts > Post-response:** Код здесь выполняется *после* получения ответа.
2. **Синтаксис теста (меняй статус-код в зависимости от запроса):**

```
pm.test("Статус ответа должен быть 200", function () {
    pm.response.to.have.status(200);
```

```
});
```

3. Просто скопируй код из пункта 2 во вкладку, отправь запрос кнопкой Send и изучи ответ. Если во вкладке Test Results отмечено PASSED, значит тест прошел. Если FAILED, то где-то ошибка.




**Зачем это нужно:** Тестировщик не должен каждый раз смотреть в правый верхний угол экрана. Если у тебя 50 запросов в коллекции, ты просто нажимаешь кнопку, и Postman сам проверяет все статус-коды.

### Парсинг и тест на правильность значения

Чтобы Арти не пришлось перепроверять каждый свиток вручную, он использует встроенные инструменты Кадуцея (Postman).

### 1. Превращение свитка в понятный объект

Ответ приходит запечатанным. Чтобы JavaScript мог прочитать его, нужно выполнить **парсинг – преобразования данных в более удобный формат**:

```
const jsonData = pm.response.json();
// Мы говорим: "Возьми ответ и преврати его в понятный объект jsonData"
```

Мы уже делали это ранее, когда извлекали токен для авторизации в переменную.

### 2. Создание теста

Мы используем библиотеку **Chai**, чтобы написать проверку на "человеческом" языке.

| Команда | Что она делает в мире мифов |
|---|---|
| `pm.test` | Объявление начала судебного разбирательства по делу о данных. |
| `pm.expect` | Мы кладем на одну чашу весов то, что реально пришло в свитке (`jsonData.name`). |
| `.to.eql` | Мы кладем на другую чашу то, что мы ожидали увидеть ("Hercules"). |

**Код проверки на значение в ключе, если возвращается один объект (не подходит для GET из нашего сервиса, будет ошибка):**

```
pm.test("Имя героя в свитке должно быть Геракл", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.name).to.eql("Геракл"); // С новой строки каждый новый ключ
});
```

Если в поле `name` будет "Зевс", Весы Истины качнутся, и тест в Postman загорится красным, сигнализируя об ошибке, несмотря на статус 200.

*"Статус 200 при неправильном теле"* – это пример ложноположительного теста, поэтому всегда стоит несколько раз перепроверить написанный тест до того, как сделать его массовым.

**Код проверки на значение в ключе, если возвращается массив из объектов (вариант для GET-запроса нашего сервиса).**

*Тест проверит, что в нашем массиве хотя бы у одного объекта, есть имя Геракл и возраст = 30:*

```
pm.test("В массиве есть герой с именем Геракл", function () {
    const jsonData = pm.response.json();

    pm.expect(
    jsonData.some(item =>
```

```
        item.name === "Геракл" && //&& значит "и", === значит строгое
равенство
        item.age === 30
    )
 ).to.be.true;
});
```

### Заголовки HTTP-ответа (Response Headers)

**Заголовки** – это метаданные. Если тело ответа – это "посылка", то заголовки — это информация на коробке: кто отправил, что внутри, сколько это весит и как с этим обращаться.

В тестировании мы проверяем их, чтобы убедиться, что сервер работает корректно и безопасно.

#### 1. Безопасность и проверка типа данных

Прежде чем парсить ответ через `pm.response.json()`, стоит проверить **Content-Type**. Если сервер вместо JSON пришлет ошибку в формате HTML, ваш скрипт упадет. Проверка заголовка – это "страховка" вашего автотеста.

#### 2. Некоторые заголовки, которые нужно знать

| Заголовок | Зачем тестировщику? |
|---|---|
| **Content-Type** | Проверяем, что пришел именно `application/json`. Если там `text/html`, значит, что-то пошло не так (например, вылетела 500-я ошибка). |
| **Set-Cookie** | Сервер передает куки, которые браузер (или Postman) сохранит для сессии. Важно проверять при тестировании **логина** или **корзины**. |
| **Cache-Control** | Указывает, можно ли кэшировать ответ. Если мы тестируем динамические данные (например, баланс кошелька), там должно быть `no-cache`. |
| **Authorization** | Чаще это заголовок *запроса*, но в ответе мы можем искать токены (например, `Access-Token`) в кастомных заголовках, если так спроектировано API. |

| Заголовок | Зачем тестировщику? |
| --- | --- |
| **Content-Length** | Помогает быстро проверить, не пришло ли нам пустое тело ответа. |

### Примеры тестов в Postman

Вот пример того, как проверить не только формат, но и важные параметры сессии и кэша.

```
// 1. ПРОВЕРКА ФОРМАТА (Страховка перед основным тестом)
pm.test("Ответ пришел в формате JSON", () => {
    pm.response.to.have.header("Content-Type");
    pm.expect(pm.response.headers.get('Content-
Type')).to.include('application/json');
});

// 2. ПРОВЕРКА КУКИ (Важно для авторизации)
pm.test("Сервер прислал сессионную куку (sessionID)", () => {
    // Проверяем, что заголовок Set-Cookie вообще есть
    pm.response.to.have.header("Set-Cookie");
    // Можно проверить наличие конкретной куки в менеджере кукисов
    pm.expect(pm.cookies.has('sessionID')).to.be.true;
});

// 3. ПРОВЕРКА КЭШИРОВАНИЯ
pm.test("Данные не кэшируются (всегда свежие)", () => {
    // Проверяем, что сервер запрещает кэш для этого эндпоинта
    pm.expect(pm.response.headers.get('Cache-Control')).to.include('no-
cache');
});

// 4. ПРОВЕРКА РАЗМЕРА
pm.test("Тело ответа содержит данные (Content-Length > 0)", () => {
    const length = parseInt(pm.response.headers.get('Content-Length'));
    pm.expect(length).to.be.above(0);
});
```

### Зачем это в реальной работе?

1. **Баги кэширования:** Если вы поменяли имя пользователя, а оно не обновилось, то проверьте `Cache-Control`. Возможно, сервер заставляет браузер показывать старую версию.

2. **Проблемы с авторизацией:** Если после логина пользователя "выкидывает", первым делом смотрим, прилетает ли `Set-Cookie` и корректен ли он.

3. **Безопасность:** Тестировщик может проверить, нет ли в заголовках лишней информации (например, версии сервера `Server: Nginx/1.18.0`), которую могут использовать хакеры.

### Полезные материалы

1. Примеры кейсов https://docs.google.com/spreadsheets/d/1fLgmhpmvJP06d4xzslEnWEgmeTKWH7uTBT_a1Rzph1Q/edit#gid=0
2. Библиотека Chai https://www.chaijs.com/api/bdd/
3. ChatGPT https://chat.openai.com/
4. Notion https://www.notion.so/
5. Статьи о написании скриптов в Postman https://learning.postman.com/docs/writing-scripts/intro-to-scripts/

ТЕСТИРОВАНИЕ API

# 48. Работа с тестовыми данными в Postman (версия 2026)

### Pre-request Scripts (Скрипты перед запросом)

**Pre-request scripts** – это JavaScript-код, который выполняется в Postman непосредственно **перед** тем, как запрос уйдет на сервер.

### Зачем это нужно?

В реальном тестировании серверы часто отклоняют повторяющиеся данные (например, нельзя создать двух пользователей с одинаковым `email` или `id`). Чтобы не менять данные вручную перед каждым нажатием кнопки **Send**, мы используем скрипты для автоматизации.

**Сценарий 1: Генерация случайных данных**

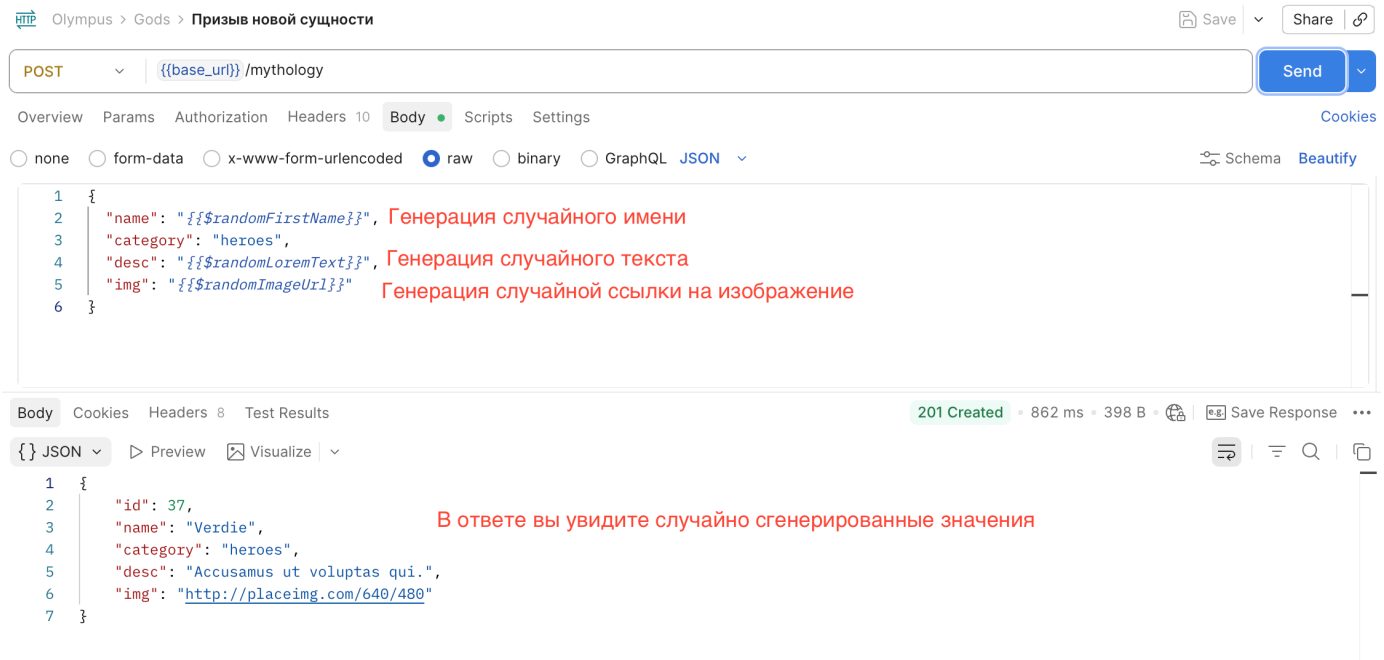
Есть два основных способа сделать данные уникальными: встроенные переменные Postman и чистый JavaScript.

**Вариант А: Встроенные динамические переменные (самый простой)**

Postman уже содержит готовые функции для генерации данных. Их можно писать прямо в теле запроса (Body) в качестве значения или в URL без всякого кода, например:

- `{{$guid}}` — генерирует длинный уникальный ID (UUID).
- `{{$randomInt}}` — случайное число от 0 до 1000.
- `{{$randomEmail}}` — готовый случайный почтовый адрес.
- `{{$randomUserName}}` — случайное имя пользователя.

*Давайте покажу на примере POST "Призвать новую сущность":*



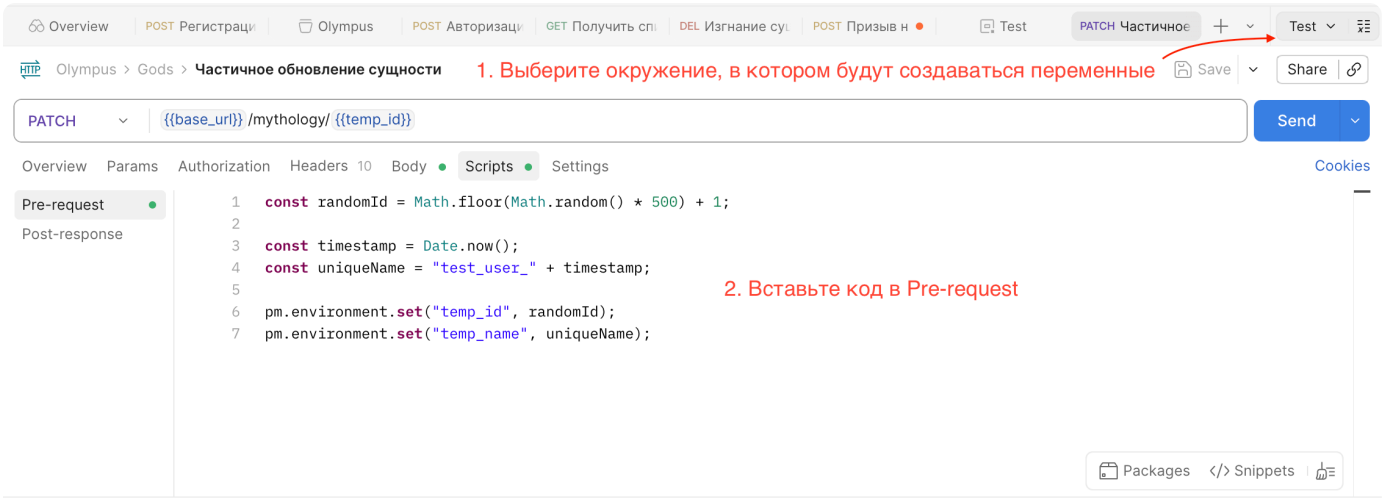

Таких функций очень много. Просто начните вводить $ в значении, чтобы выбрать подходящую.

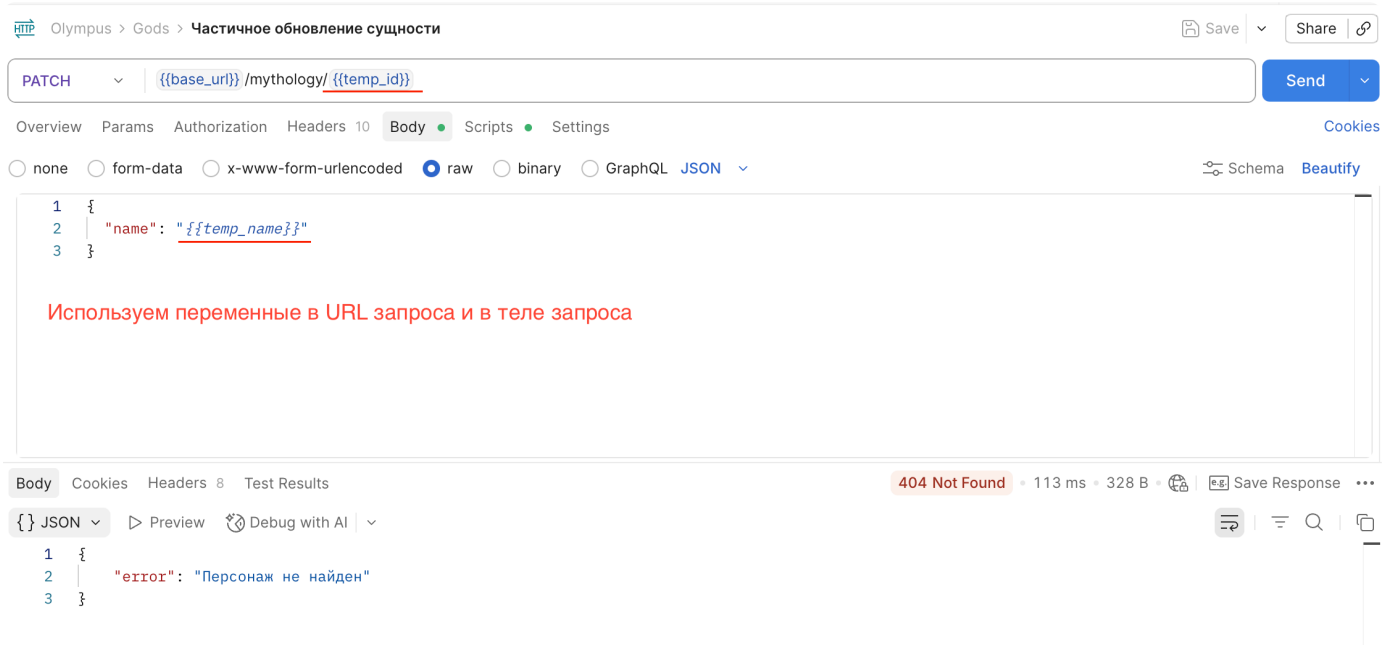
**Вариант Б: Использование JavaScript (вкладка Scripts > Pre-request)**

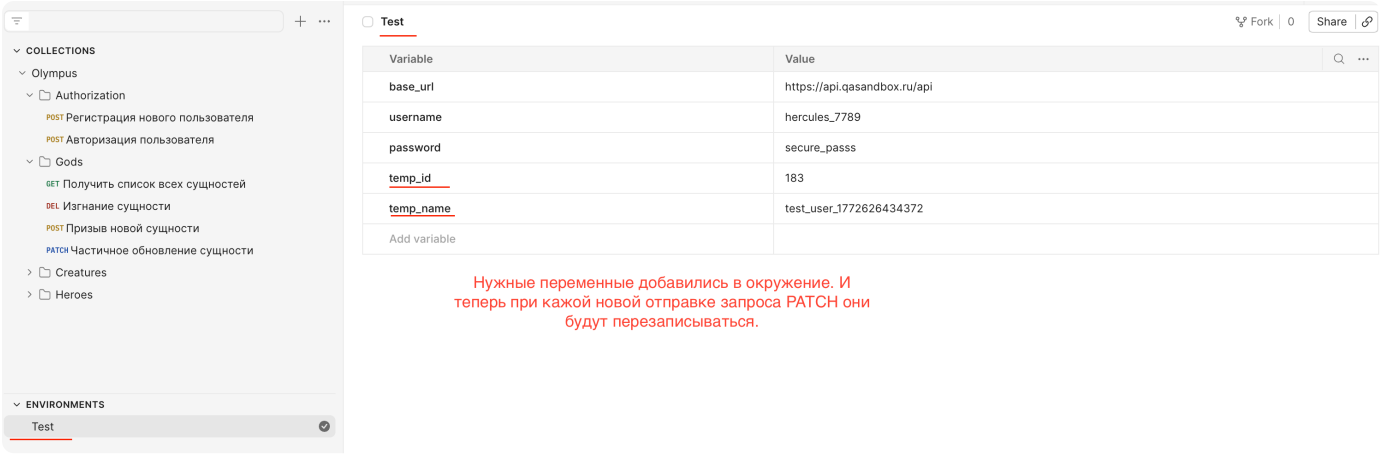
Так как переменные будем помещать в окружение, то заранее выберите то, которое вам нужно.

Если вам нужна сложная логика (например, число строго в диапазоне), используйте JS:

```
// 1. Генерация числа от 1 до 500
const randomId = Math.floor(Math.random() * 500) + 1;

// 2. Генерация строки с текущей датой, чтобы сделать имя уникальным
const timestamp = Date.now();
const uniqueName = "test_user_" + timestamp;

// ТЕПЕРЬ СОХРАНЯЕМ В ПЕРЕМЕННЫЕ:
// Мы кладем эти значения в "окружение", чтобы использовать их в самом
запросе
pm.environment.set("temp_id", randomId);
pm.environment.set("temp_name", uniqueName);
```


Overview
Olympus
Test
Olympus > Gods > Частичное обновление сущности
1. Выберите окружение, в котором будут создаваться переменные
Save
Share
PATCH
{{base_url}} /mythology/ {{temp_id}}
Send
Overview Params Authorization Headers 10 Body Scripts Settings
Cookies
Pre-request
Post-response
const randomId = Math.floor(Math.random() * 500) + 1;
const timestamp = Date.now();
const uniqueName = "test_user_" + timestamp;
pm.environment.set("temp_id", randomId);
pm.environment.set("temp_name", uniqueName);
2. Вставьте код в Pre-request
Packages
Snippets
Body Cookies Headers 8 Test Results
404 Not Found
113 ms
328 B
Save Response
JSON
Preview
Debug with AI
{
"error": "Персонаж не найден"
}


Olympus > Gods > Частичное обновление сущности
Save
Share
PATCH
{{base_url}} /mythology/ {{temp_id}}
Send
Overview Params Authorization Headers 10 Body Scripts Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"name": "{{temp_name}}"
}
Используем переменные в URL запроса и в теле запроса
Body Cookies Headers 8 Test Results
404 Not Found
113 ms
328 B
Save Response
JSON
Preview
Debug with AI
{
"error": "Персонаж не найден"
}




*Остальные сценарии работают по такому же принципу определения переменных до запроса.*

### Сценарий 2: Автоматический логин в Pre-request Script

Не всегда нам нужно запускать все тесты сразу, а значит удобнее сделать авторизацию для одного. Такой скрипт можно вставить на уровне конкретного запроса или на уровне **Коллекции** или **Папки**, чтобы он работал для всех запросов внутри.

Обратите внимание, что для успеха этого запроса у вас уже должны быть добавлены переменные login_url, username и password в окружение, а пользователь должен быть зарегистрирован. Если регистрации нет, то тогда нужно добавить код для этого в скрипт.

```
// 1. Конфигурация данных для логина
const loginUrl = pm.environment.get("login_url") ||
"https://api.example.com/login";
const credentials = {
    username: pm.environment.get("username"),
    password: pm.environment.get("password")
};

// 2. Сама функция запроса токена
pm.sendRequest({
    url: loginUrl,
    method: 'POST',
    header: {
```

```
        'Content-Type': 'application/json'
    },
    body: {
        mode: 'raw',
        raw: JSON.stringify(credentials)
    }
}, function (err, res) {
    if (err) {
        console.error("Ошибка при попытке логина: ", err);
    } else {
        // 3. Парсим ответ и достаем токен
        const responseJson = res.json();
        const token = responseJson.token; // Путь к токену зависит от
структуры вашего API

        if (token) {
            // 4. Сохраняем токен в переменную окружения или коллекции
            pm.environment.set("token", token);

            // 5. Сразу подставляем токен в текущий запрос
            pm.request.headers.add({
                key: 'Authorization',
                value: 'Bearer ' + token
            });

            console.log("Токен обновлен и добавлен в заголовки");
        } else {
            console.warn("Логин прошел успешно, но токен в ответе не
найден");
        }
    }
});
```

### Сценарий 3: Использование библиотеки Lodash (продвинутый рандом)

В Postman встроена библиотека **Lodash**, которая сильно упрощает генерацию данных.

```
const _ = require('lodash');

// Выбрать случайное значение из списка (массива)
const roles = ["admin", "editor", "viewer"];
const randomRole = _.sample(roles);

pm.environment.set("random_role", randomRole);
```

**Почему это важно для тестировщика?**

1. **Стабильность автотестов:** Ваши тесты не будут падать из-за того, что "такой пользователь уже существует".
2. **Экономия времени:** Вам не нужно переписывать ID или даты вручную при каждом прогоне коллекции.
3. **Гибкость:** Вы можете имитировать поведение реальных пользователей, которые вводят разные данные.

**Массовый запуск тестов с Data Files**

В рамках тестового прогона через Runner мы можем использовать файлы с данными, из которых будут подставляться значения переменных. Так им образом мы можем за один раз протестировать много комбинаций.

**Данная функция доступна только на платных тарифах, но вы можете активировать пробный период на 30 дней без привязки карты для ее тестирования.**

1. **Подготовка Data File:** Это внешний файл, который служит источником данных для твоих тестов. Скопируйте данные из примеров и создайте файлы с расширениями `heroes.csv` и `heroes.json`. Оба файла можно забрать здесь.
    - **CSV (Comma-Separated Values):** Первая строка — имена переменных, дальше — значения.

```
name,category,desc,img
Perseus,heroes,Slayer of Medusa,https://images.unsplash.com/photo-
1503152394-c571994fd383
Atalanta,heroes,Huntress of the Calydonian
```

```
Boar,https://images.unsplash.com/photo-1543165365-07232ed12faf
Bellerophon,heroes,Slayer of the
Chimera,https://images.unsplash.com/photo-1605342415160-
c44773838c6b
Hercules,gods,Hero known for his twelve labors and
strength,https://images.unsplash.com/photo-1599707367072-
cd6ada2bc375
Medusa,monsters,Gorgon whose gaze turns onlookers to
stone,https://images.unsplash.com/photo-1634151703901-72993888746c
```

- **JSON (JavaScript Object Notation):** Массив объектов.

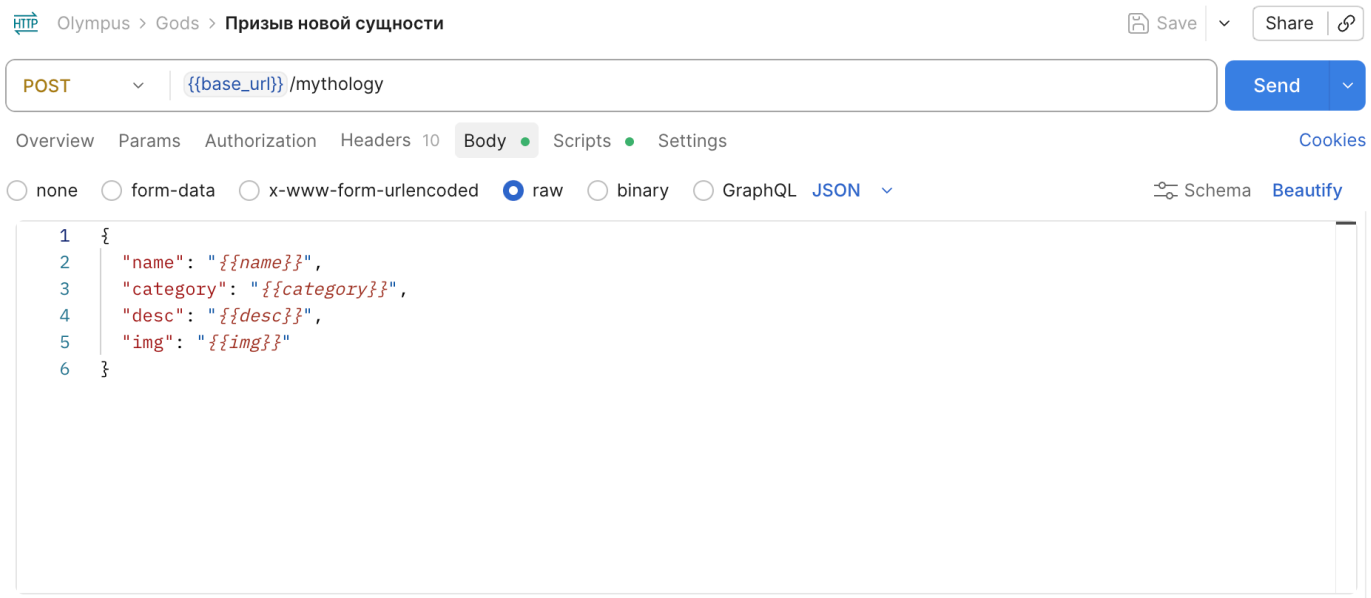
```
[
  {
    "name": "Perseus",
    "category": "heroes",
    "desc": "Slayer of Medusa",
    "img": "https://images.unsplash.com/photo-1503152394-
c571994fd383"
  },
  {
    "name": "Atalanta",
    "category": "heroes",
    "desc": "Huntress of the Calydonian Boar",
    "img": "https://images.unsplash.com/photo-1543165365-
07232ed12faf"
  },
  {
    "name": "Bellerophon",
    "category": "heroes",
    "desc": "Slayer of the Chimera",
    "img": "https://images.unsplash.com/photo-1605342415160-
c44773838c6b"
  },
  {
    "name": "Hercules",
    "category": "gods",
    "desc": "Hero known for his twelve labors and strength",
```

```
    "img": "https://images.unsplash.com/photo-1599707367072-
cd6ada2bc375"
  },
  {
    "name": "Medusa",
    "category": "monsters",
    "desc": "Gorgon whose gaze turns onlookers to stone",
    "img": "https://images.unsplash.com/photo-1634151703901-
72993888746c"
  }
]
```

2. **Настройка Запросов:** Чтобы запросы использовали данные из файла, замени в них конкретные значения на переменные в двойных фигурных скобках `{{...}}`. Обязательно добавь двойные кавычки, если тип данных string `"{{...}}"`.

3. Имена переменных должны совпадать с заголовками в CSV или ключами в JSON.

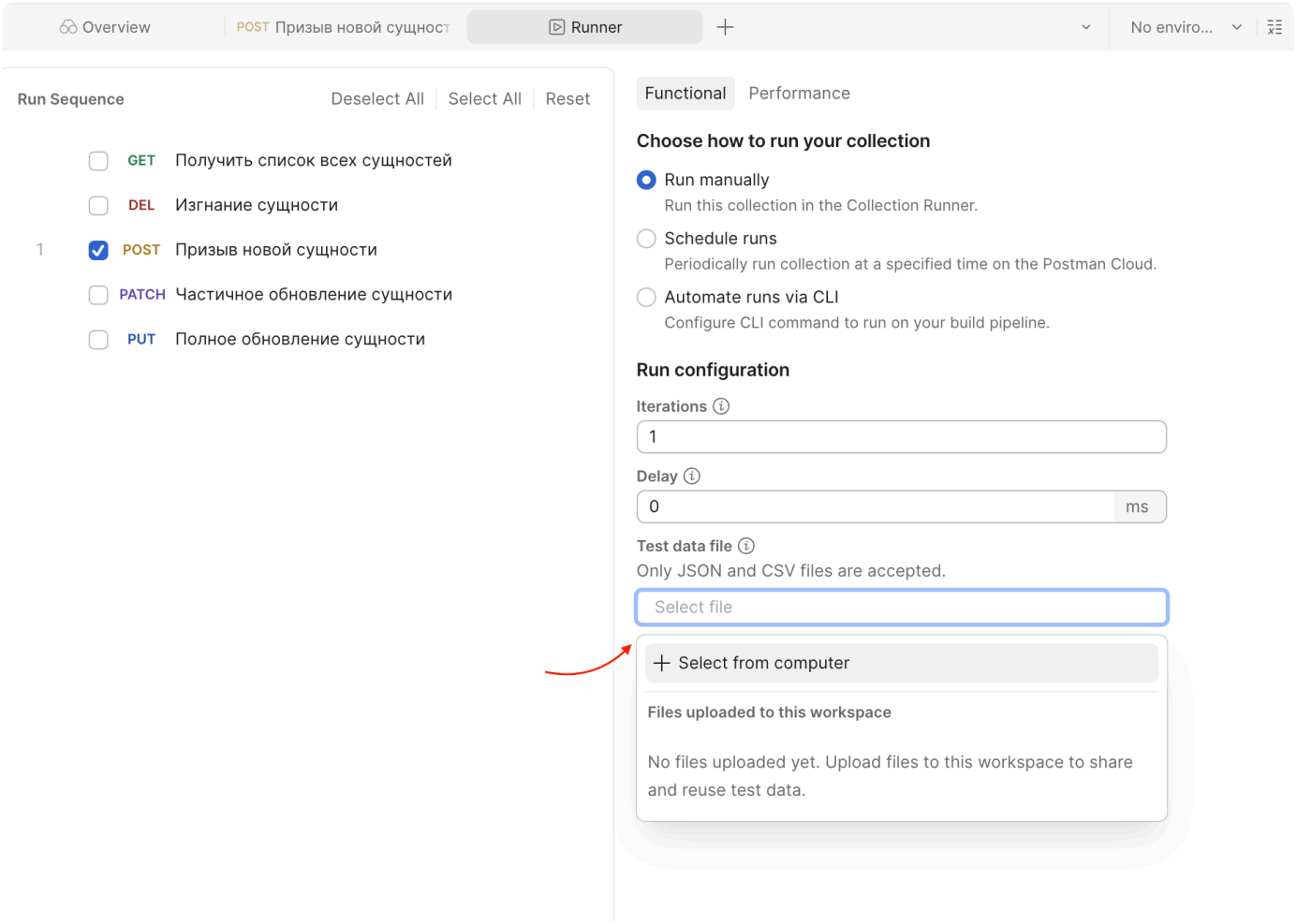
    - **Было в Body:** `"name": "Artichoke the Great"`
    - **Стало в Body:** `"name": "{{name}}"`



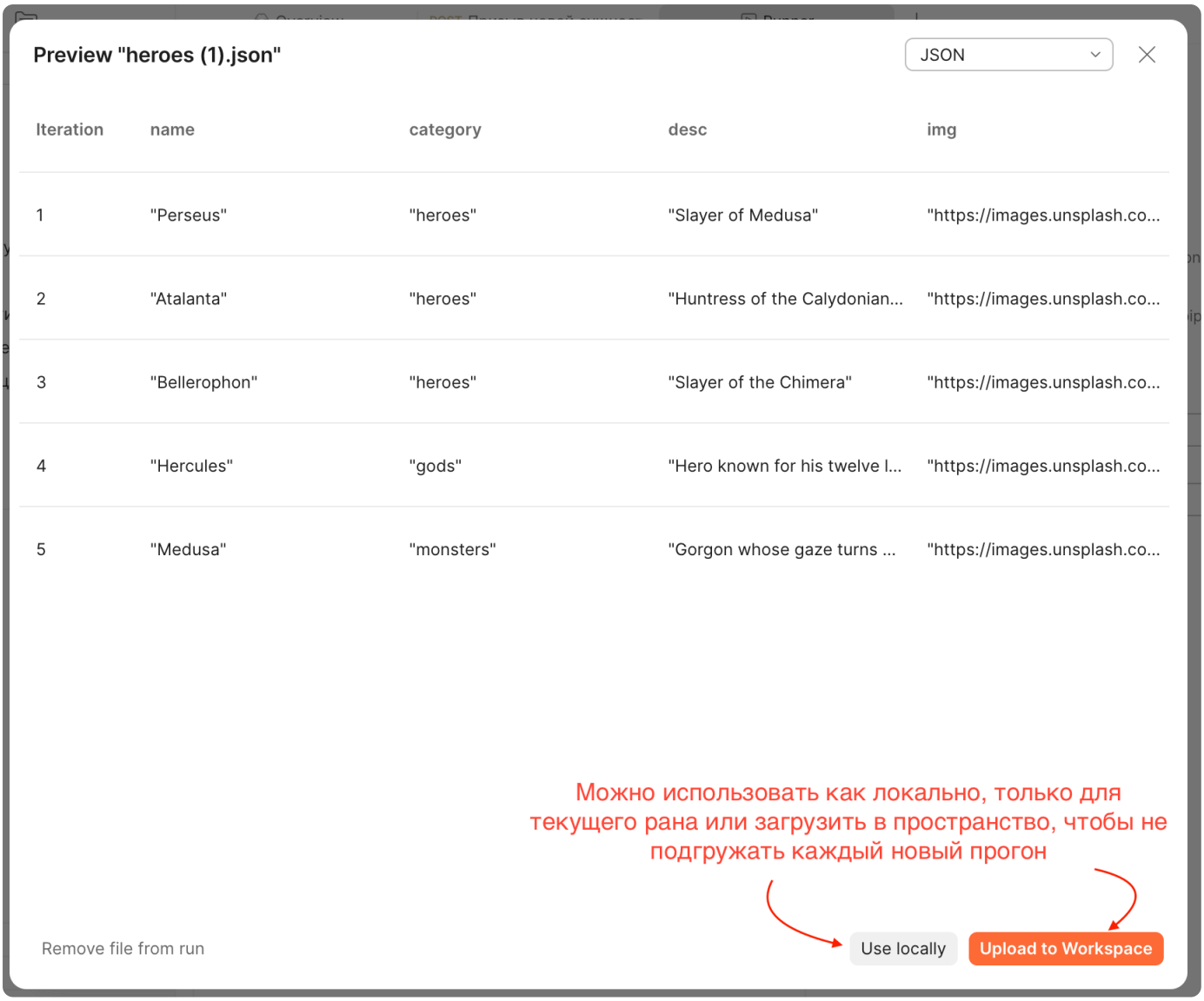

1. **Запуск Ритуала Итераций:**

    - В окне Collection Runner в разделе **Data** нажми **Select File**.

- Выбери свой CSV или JSON файл.




- После загрузки проверьте, что все переменные и их значения на месте. Если что-то не так, исправьте ошибки и еще раз загрузите файл.




- Postman сам посчитает количество строк/объектов и установит нужное число в поле **Iterations**.
- Нажми **Run** и смотри, как твой сценарий выполняется для каждого "героя" из файла.

## Полезные материалы

1. Тестовая коллекция в Postman https://www.postman.com/api-evangelist/workspace/collections/overview

ТЕСТИРОВАНИЕ API

# 49. Перехват трафика в Postman (версия 2026)

### Зачем нужен перехват трафика тестировщику?

Для QA-инженера инструменты разработчика (DevTools) – отличный инструмент. Но когда дело доходит до переиспользования запросов и автоматизации, на помощь приходит перехват трафика, при котором мы можем работать с запросами из других приложений. Postman становится промежуточным сервером (прокси) между вашим клиентом и сервером приложения.

Вот основные сценарии, когда это критически необходимо:

- **Реверс-инжиниринг API:** Если документация (Swagger/OpenAPI) отсутствует или безнадежно устарела, вы можете просто отправить запросы в браузере или приложении. Postman автоматически соберет все отправленные запросы, которые вы можете сохранить в готовую коллекцию.
- **Воспроизведение плавающих багов:** Перехват позволяет захватить точные заголовки (headers), куки (cookies) и тело запроса (body), которые отправляет клиент. Это помогает найти разницу между тем, как работает фронтенд, и тем, что ожидает бэкенд.
- **Быстрое создание тест-кейсов:** Вместо того чтобы вручную собирать каждый запрос по документации, вы записываете сессию пользователя (user journey) и получаете готовую цепочку вызовов для дальнейшего написания автотестов.

### Как настроить перехват трафика в Postman?

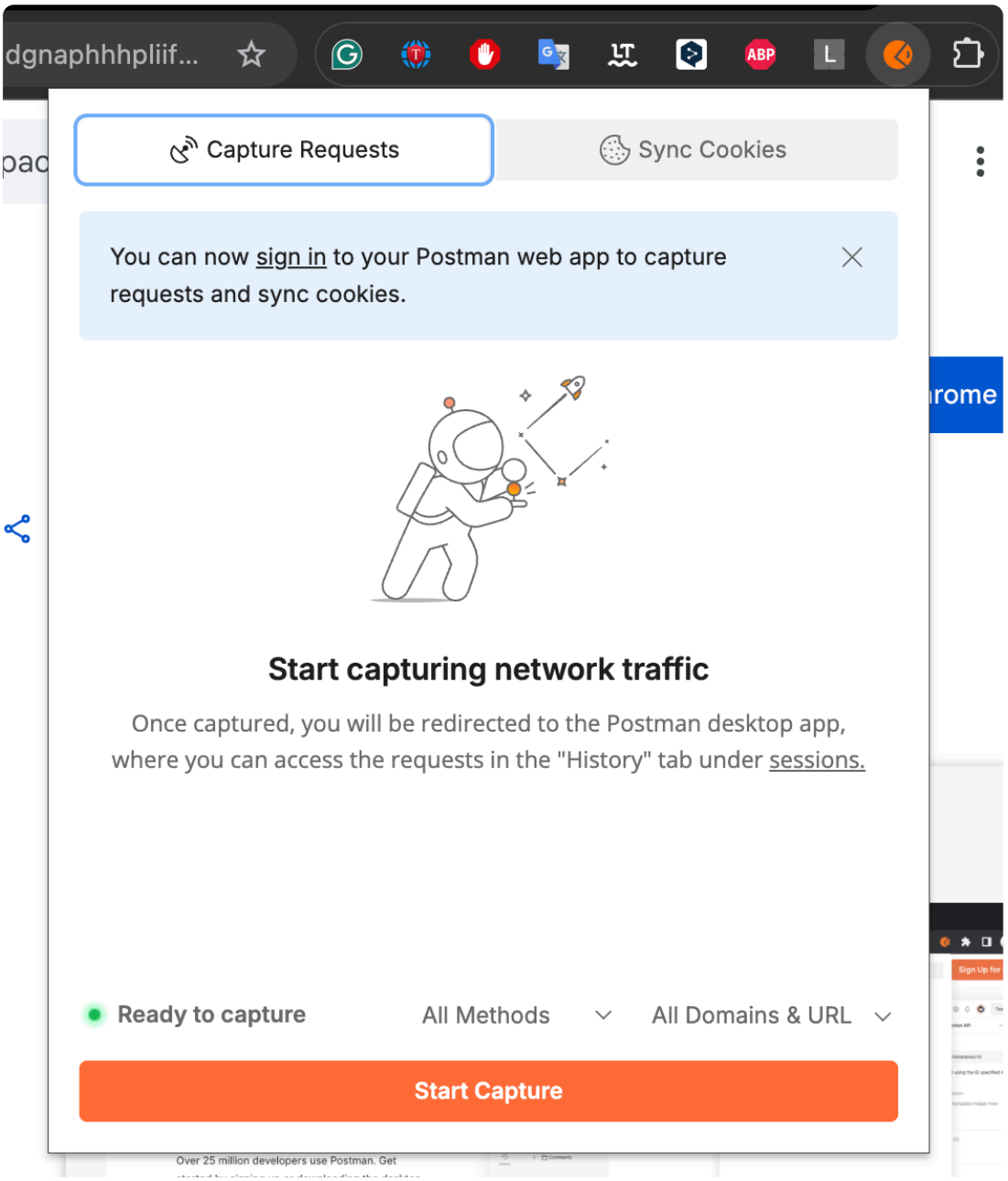
В Postman есть два основных инструмента для перехвата: браузерное расширение **Postman Interceptor** (расширение для браузеров) и **встроенный Proxy**.

### Вариант А: Использование Postman Interceptor в браузере

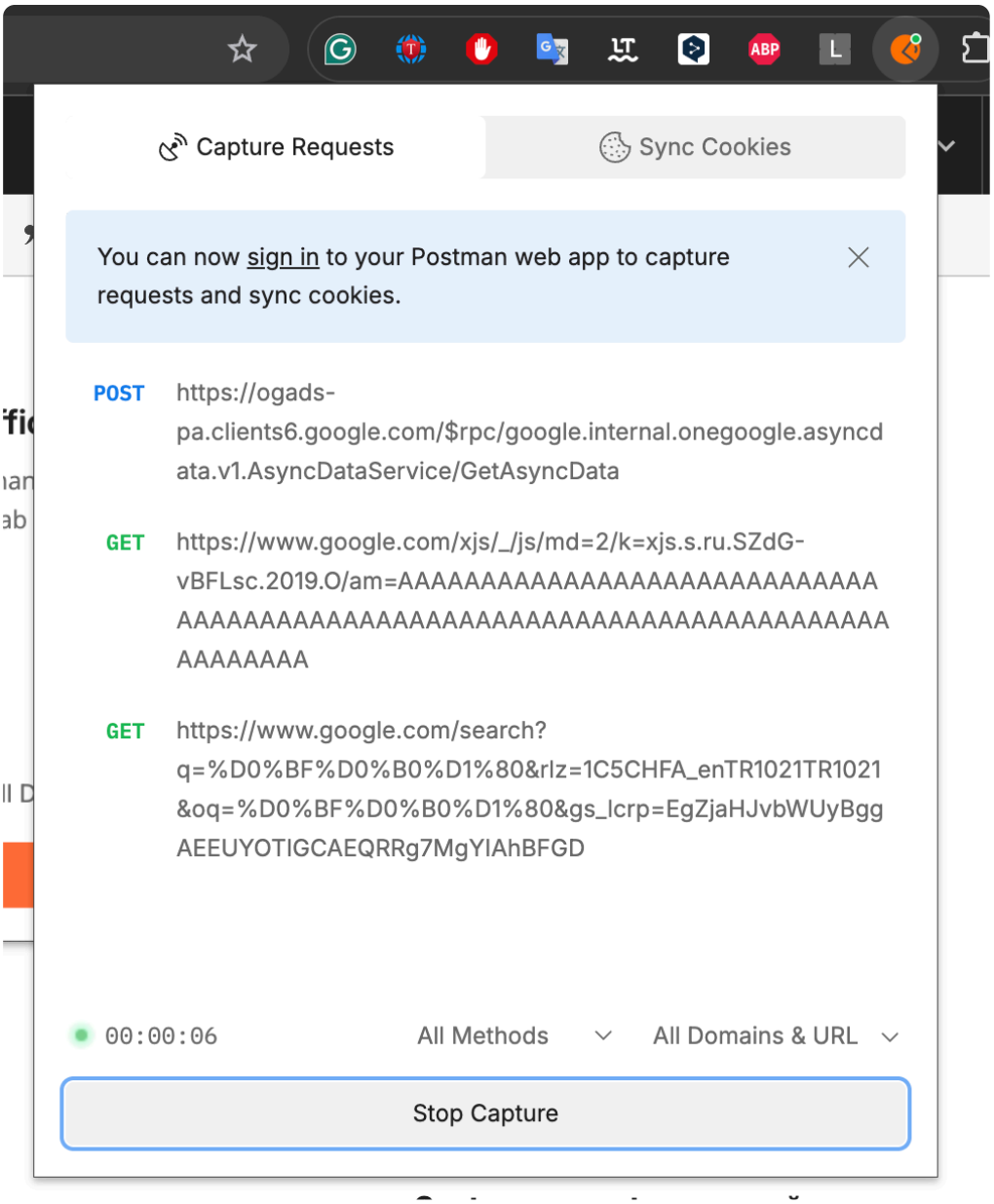
**1. Установите расширение:** Найдите и добавьте "Postman Interceptor" в магазине расширений вашего браузера (Chrome, Edge и др.).

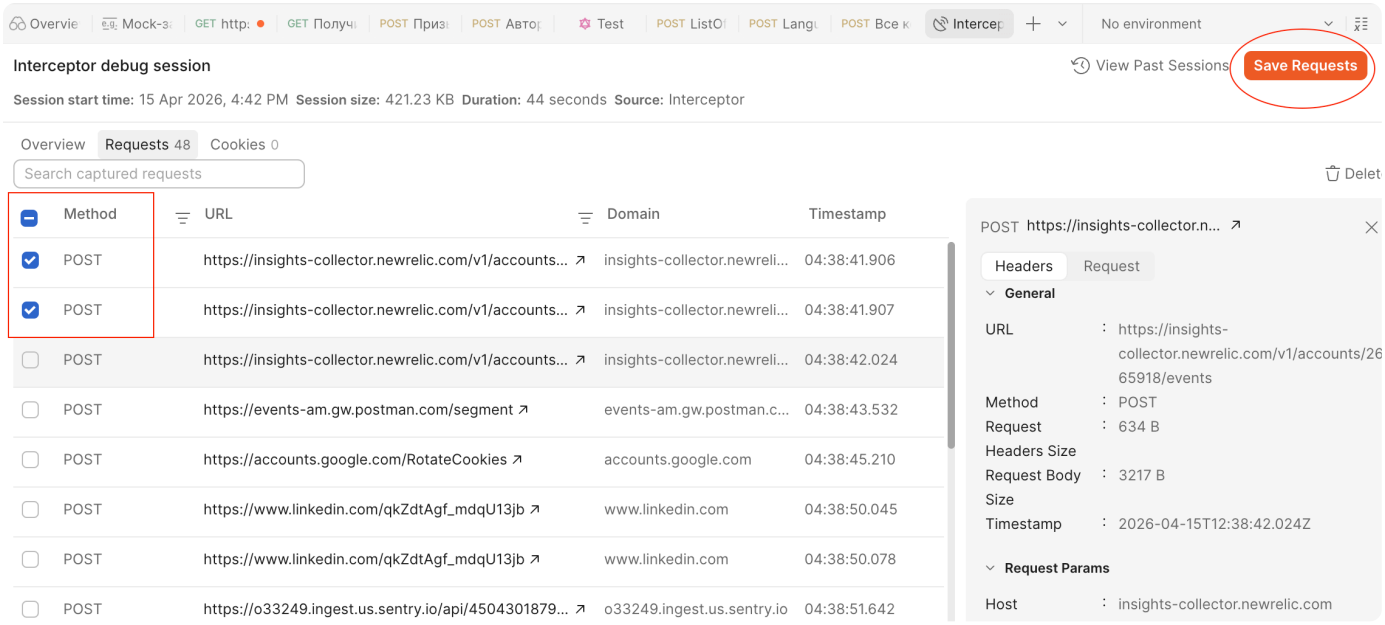
**2. Активируйте расширение:** Кликните на иконку Interceptor на панели самого браузера. Убедитесь, что статус `Ready to capture`. Нажмите

.



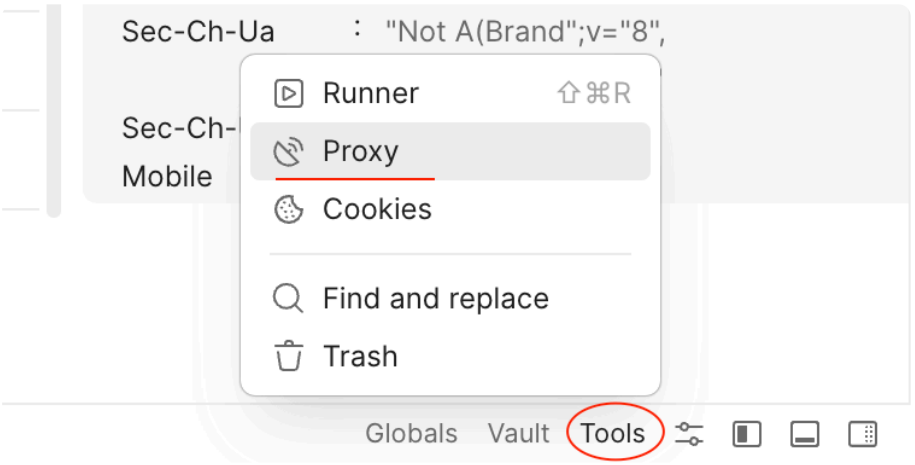

3. Теперь при отправке запросов в браузере они будут отображаться в расширении, пока вы не нажмете. После этого вам предложат открыть Postman и все перехваченные запросы окажутся в вашем Workspace.


Capture Requests
Sync Cookies
You can now sign in to your Postman web app to capture requests and sync cookies.
POST https://ogads-pa.clients6.google.com/$rpc/google.internal.onegoogle.asyncdata.v1.AsyncDataService/GetAsyncData
GET https://www.google.com/xjs/_/js/md=2/k=xjs.s.ru.SZdG-vBFLsc.2019.O/am=AAAA

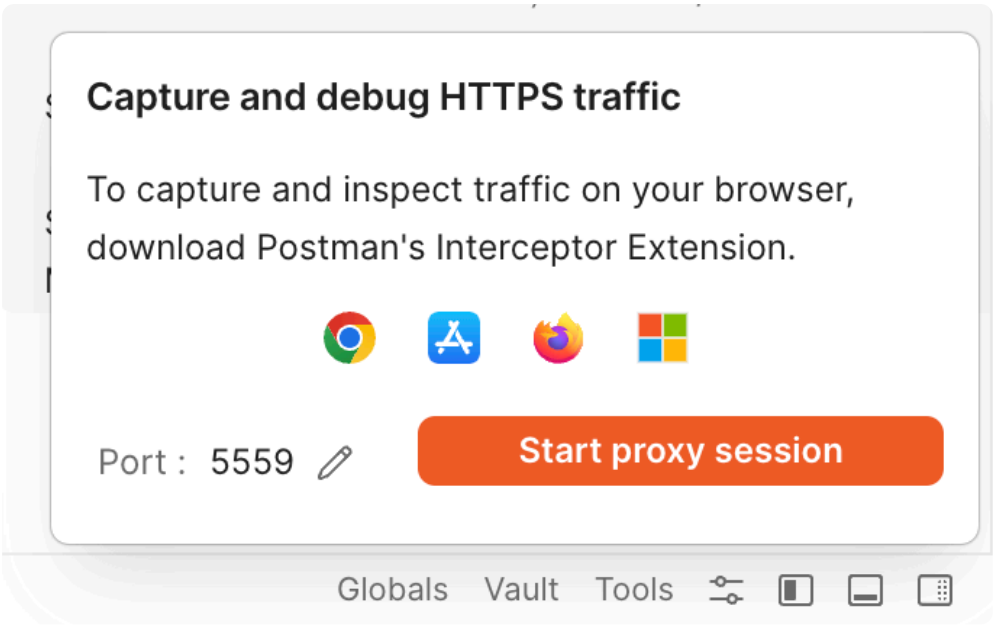



## Вариант Б: Использование встроенного Proxy

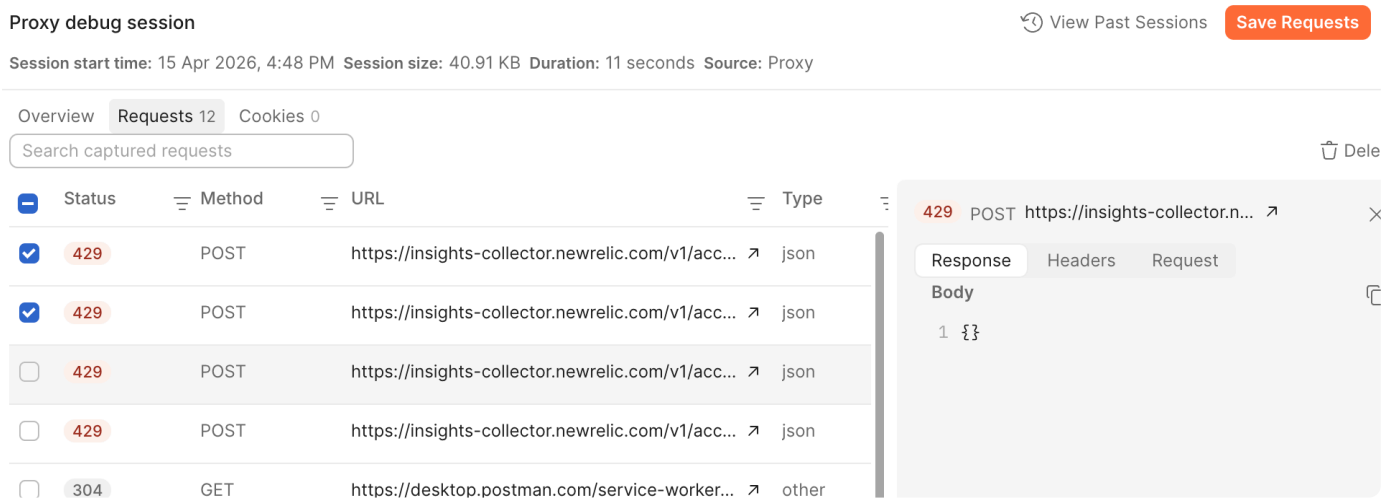
1. В десктопном Postman нажмите на *Tools > Proxy* в нижней панели.




2. По умолчанию Postman предлагает свой порт. При необходимости его можно изменить.




3. В Postman нажмите кнопку `Start Proxy`. Теперь Postman ловит трафик со всех приложений на вашем компьютере. По окончании сессии нажмите `Stop`.




4. В конце сессии вы можете сохранить нужные запросы в коллекцию.

## Полезные материалы

1. Документация Postman по перехвату трафика https://learning.postman.com/docs/sending-requests/capturing-request-data/capturing-http-requests/
2. https://learning.postman.com/docs/sending-requests/capturing-request-data/capturing-https-traffic/
3. https://learning.postman.com/docs/sending-requests/capturing-request-data/interceptor/
4. Postman Interceptor

ТЕСТИРОВАНИЕ API

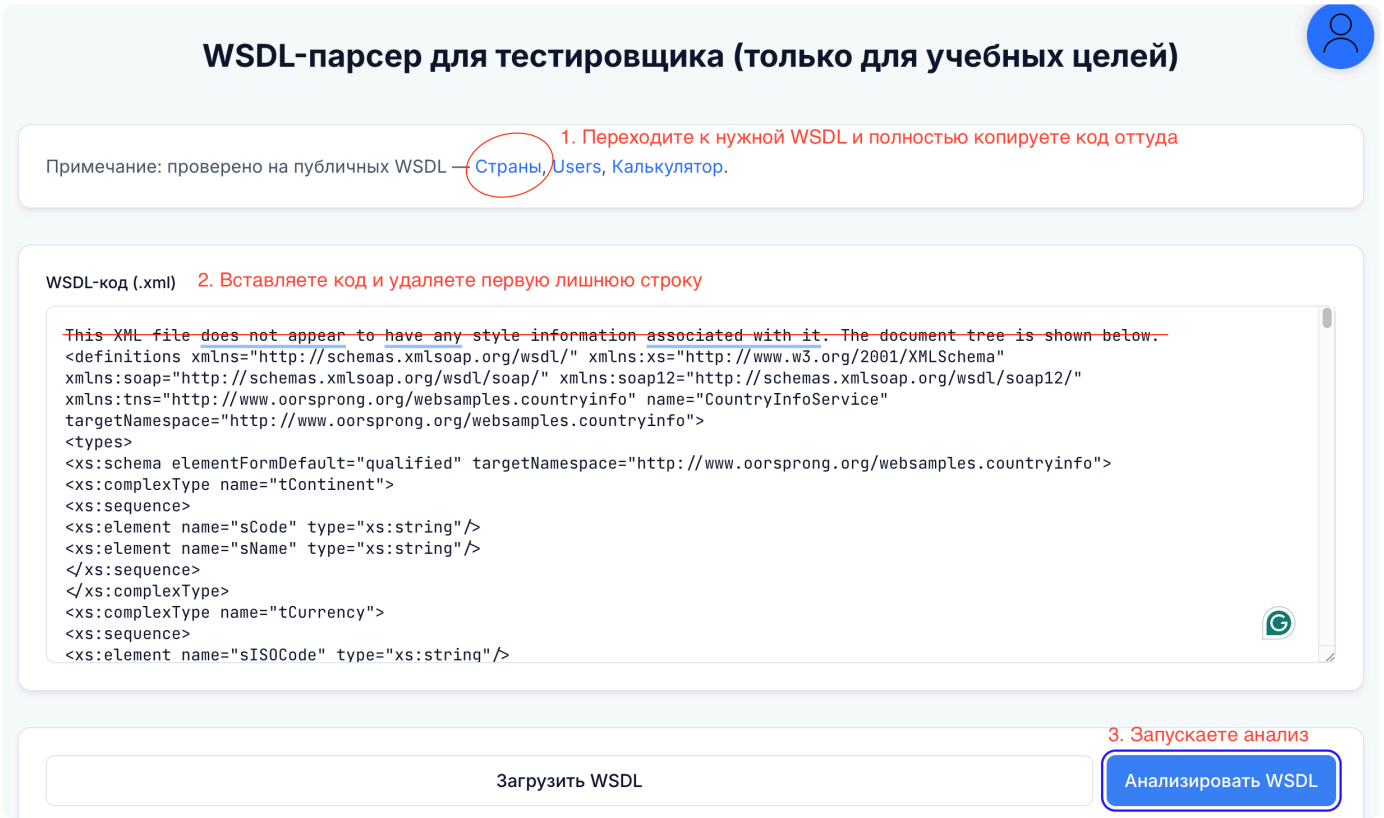
# 50. Тестирование SOAP-сервисов в Postman (версия 2026)

**Есть два способа создания запросов в Postman для Soap: ручной и автоматизированный.**

Давайте рассмотрим на примере сервиса Country Info оба варианта.

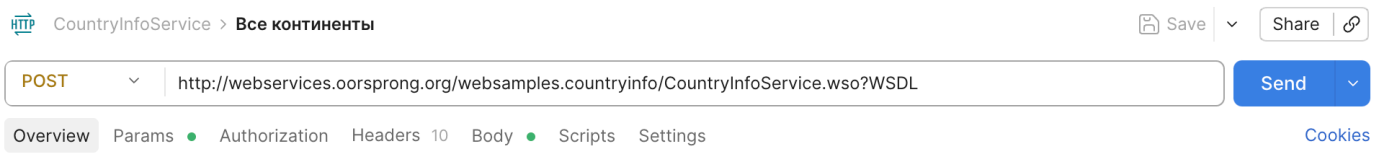
### Способ 1: Ручное создание запроса

1. Сначала вам нужно найти нужный метод, который можно получить либо в SOAP UI, либо в специальных парсерах. Для курса я разработал свой парсер, который работает с несколькими открытыми WSDL.

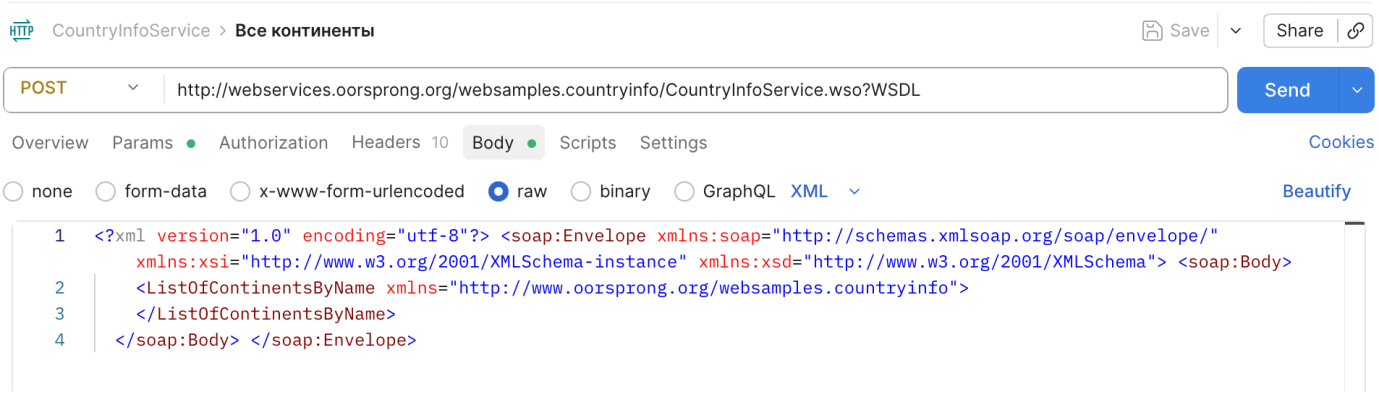


**Найденные методы:**

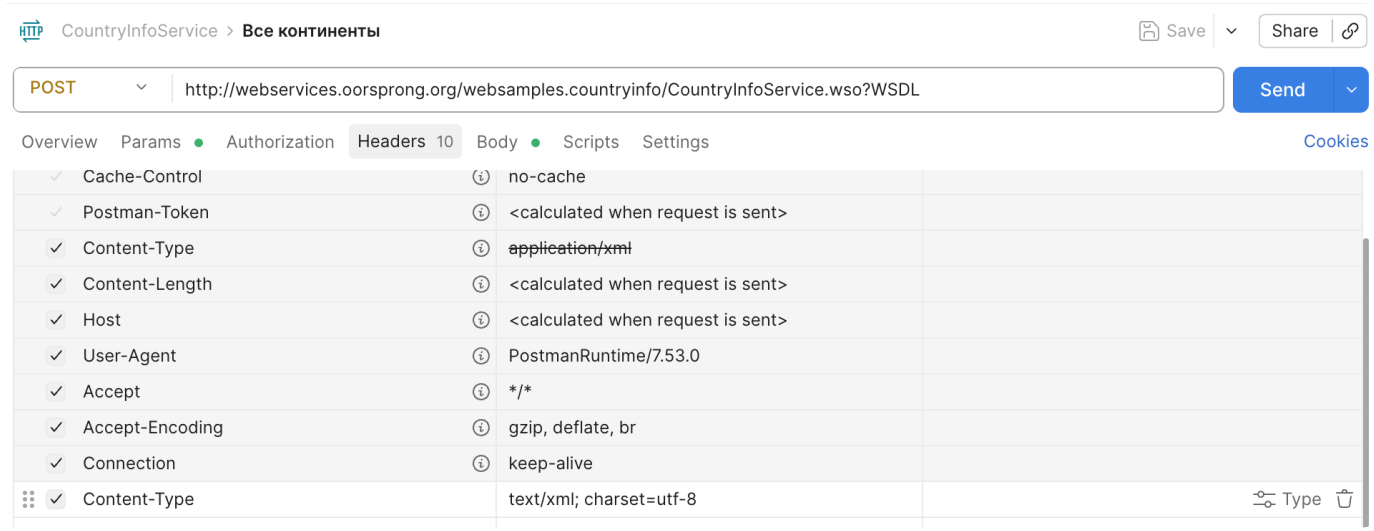
3. Далее вы создаете обычный POST-запрос в Postman, в котором URL будет представлен целевым WSDL.



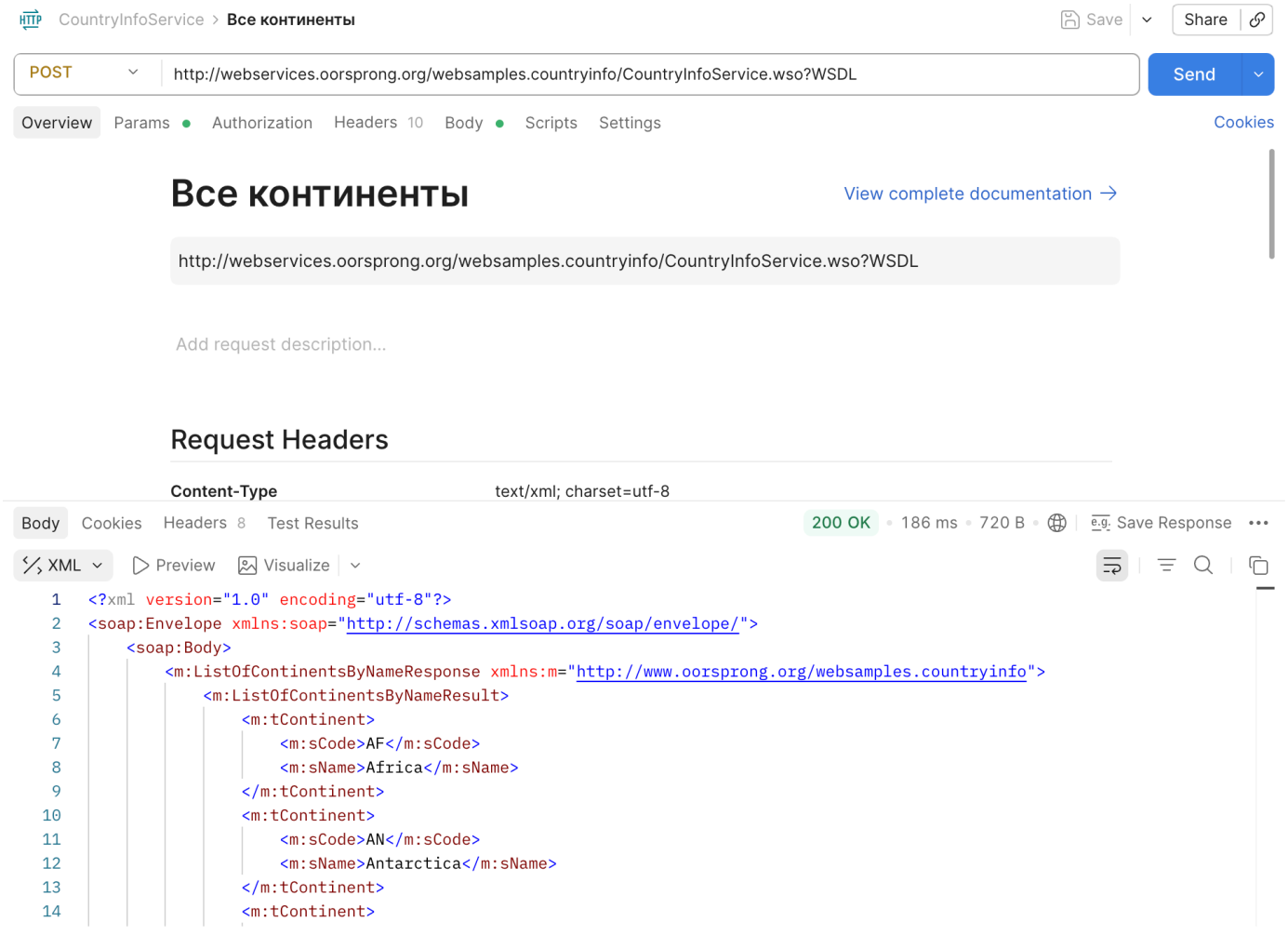

4. В *Body > Raw > XML* добавляете данные из парсера *(шаблон SOAP 1.1 Запроса)*.




5. В Headers нужно добавить значение `text/xml; charset=utf-8` для заголовка `Content-Type`.



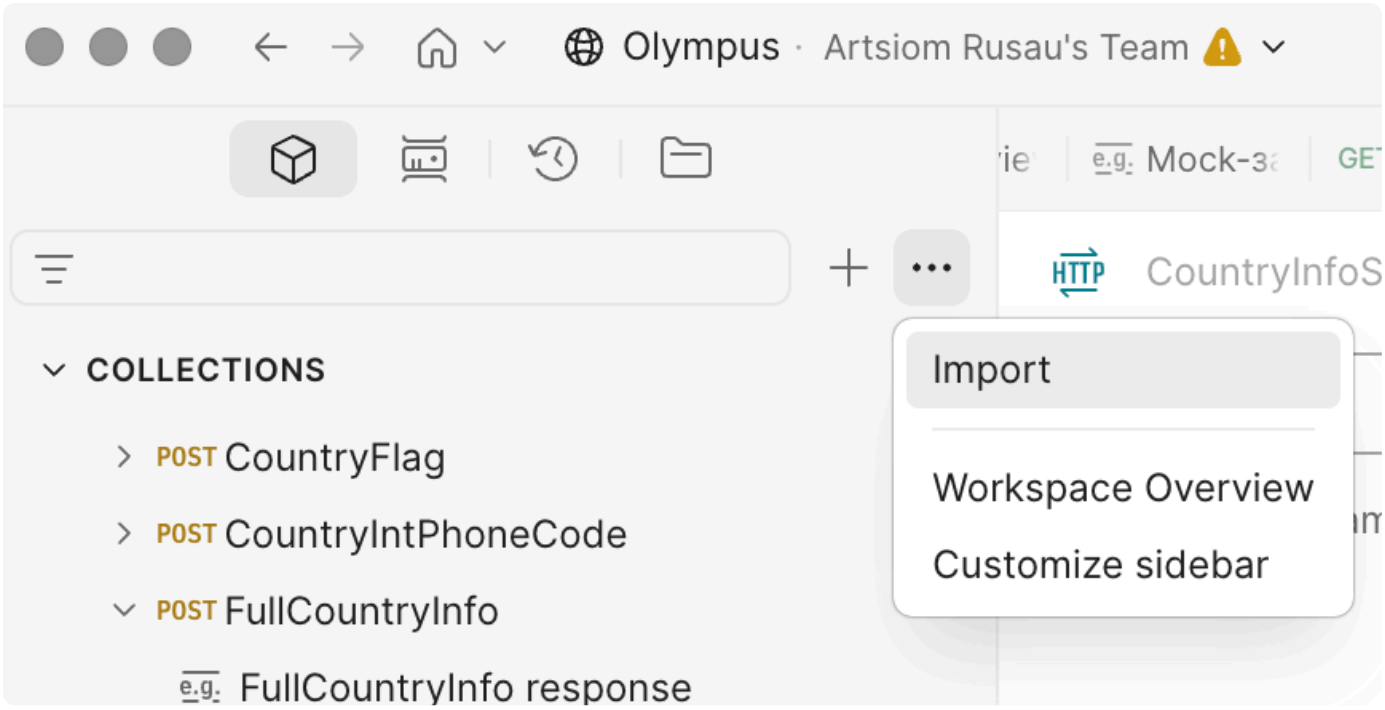

4. Теперь можно отправить запрос!



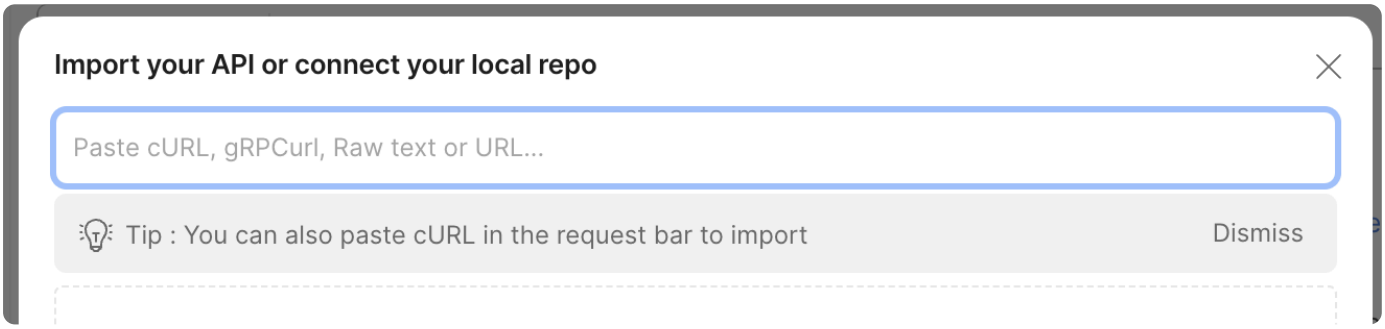

## Способ 2: Автоматическое создание коллекции из SOAP-запросов

1. Скопируйте URL вашего WSDL.

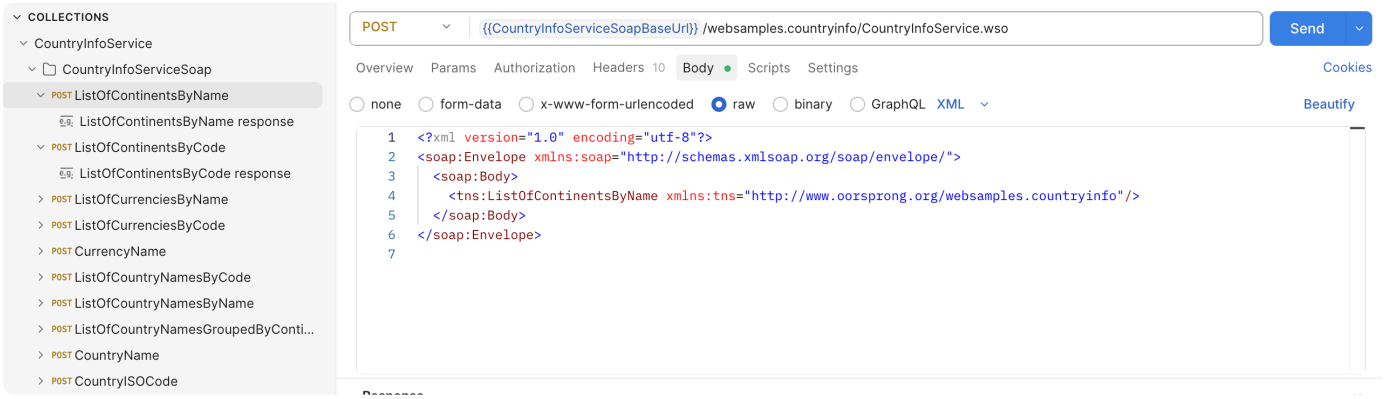
2. Выберите Import в контекстном меню Workspace.




3. Вставьте WSDL-ссылку и дождитесь окончания импорта.




4. Теперь у вас есть готовая коллекция со всеми методами из WSDL, которые можно проверять.




## Полезные материалы

1. Сайт SoapUI https://www.soapui.org/

2. Тестовая WSDL http://webservices.oorsprong.org/websamples.countryinfo/CountryInfoService.wso?WSDL
3. Для удобной работы с WSDL можно использовать расширение в Chrome https://chromewebstore.google.com/detail/boomerang-soap-rest-clien/eipdnjedkpcnlmmdfdkgfpljanehloah
4. Если расширение не работает, ты вы можете использовать инструмент моей разработки. Подробнее здесь

РАБОТА С ДАННЫМИ

# 51. Вступительное слово перед модулем "Работа с данными"

**Вступительное слово перед стартом модуля "Работа с данными"**

Представьте ситуацию, когда вы заходите на сайт своего любимого интернет магазина, а все товары в нем отсутствуют, вы не можете авторизоваться в системе, хотя еще вчера регистрировались в ней. Остались только голые страницы с общей верстkой сайта и больше ничего. Что же случилось?

Возможно, данные ресурса были стерты. Возможно, потерялась связь с сервером базы данных и прочие неприятности.

Данные окружают нас повсюду и приложение состоит из них. Поэтому очень важно уметь работать с ними и включать в процесс тестирования.

**В данном модуле мы узнаем:**

1. Где хранятся данные (БД) и как ими управлять (СУБД)?
2. Как использовать язык структурированных запросов SQL для создания, изменения, удаления и поиска данных на примере MySQL СУБД?
3. Рассмотрим основы работы с нереляционными БД на примере MongoDB.

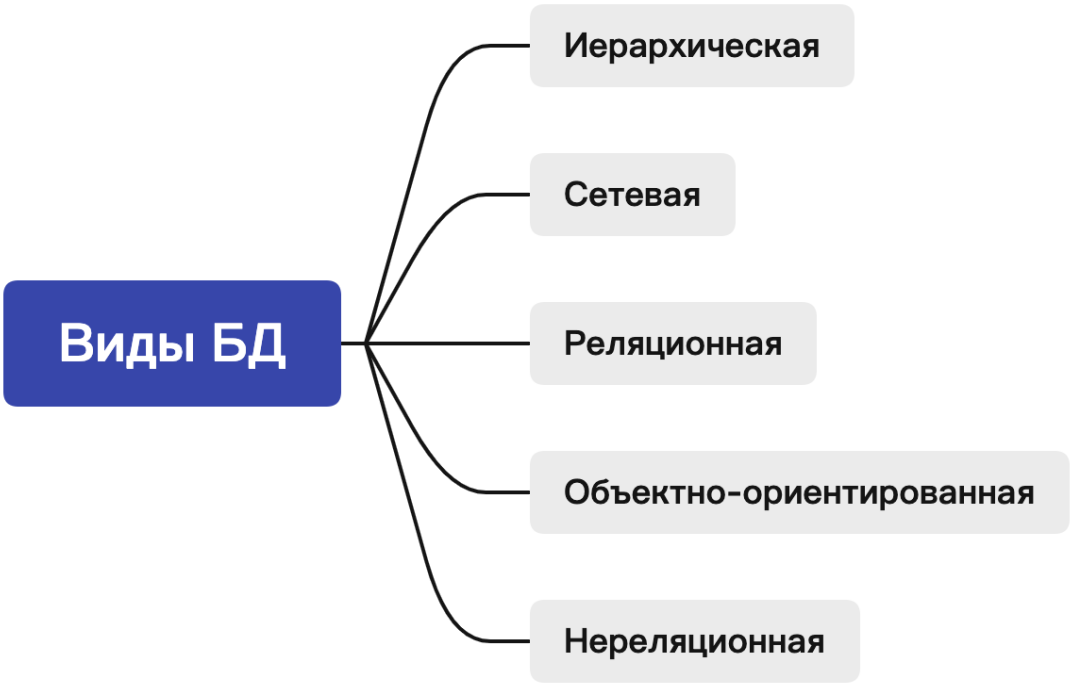
Отдельно хочу отметить несколько **путей применения этих знаний в работе тестировщика**:

1. Быстрое создание тестовых данных внутри БД, без использования графического интерфейса
2. Изменение прав пользователей
3. Исследование причин возникновения дефектов на уровне данных
4. Помощь в решении проблем конечных пользователей, используя слепки (дампы) их баз данных

РАБОТА С ДАННЫМИ

# 52. Базы данных. Типы БД и СУБД. Нормальные формы

**База данных (database, DB)** - упорядоченный набор структурированной информации или данных, которые обычно хранятся в электронном виде в компьютерной системе.
**Система управления базами данных (СУБД, database management system, DBMS)** - интерфейс (ПО) между базой данных и пользователями или программами.




- **Иерархические БД** организованы как дерево, где связи устанавливаются в виде "родитель-потомок".
- **Сетевые БД** имеют более сложную структуру с множественными связями между записями.
- **Реляционные БД (SQL)** основаны на таблицах с данными, связанными через общие ключи. На курсе мы будем работать с ними.
- **Нереляционные БД (известные как NoSQL)** структурируют данные не по табличной схеме, а используют форматы, такие как ключ-значение, документы, колонки или графы.

## Реляционные базы данных (SQL)

**Формат:** таблицы, которые состоят из строк (записей) и столбцов (полей)

**SQL (Structured Query Language)** — это язык программирования, используемый в большинстве реляционных баз данных для запросов, обработки и определения данных, а также контроля доступа.

**Примеры СУБД:** MySQL, Microsoft SQL Server, PostgreSQL, Oracle DB

### Типы связей

- **Один к одному (1:1):** Каждая запись в таблице A соответствует максимум одной записи в таблице B и наоборот.

  *Пример: пользователь — паспорт.*

- **Один ко многим (1:N):** Одна запись в таблице A может быть связана с несколькими записями в таблице B, но каждая запись в таблице B связана только с одной в A.

  *Пример: автор — книги.*

- **Многие ко многим (M:N):** Каждая запись в таблице A может быть связана с несколькими записями в B и наоборот.

  *Реализуется через промежуточную таблицу.*

  *Пример: студенты — предметы.*

### Нормализация баз данных

Тема нормализации может звучать академично, но на практике — это вопрос удобства и надёжности вашей базы данных. Представьте, что вы создаёте систему для интернет-магазина. Если не применять нормализацию, данные

легко могут начать дублироваться, а значит — будет тяжело сопровождать и поддерживать в порядке.

### Что такое нормализация?

Нормализация — это **разделение данных на таблицы** таким образом, чтобы каждая таблица хранила только один тип сущности. Это **уменьшает дублирование** и **повышает целостность** (данные в одном месте — значит, изменили один раз, и всё ок).

### Пример без нормализации (0NF, нулевая форма)

Представим таблицу заказов:

| Заказ_ID | Имя клиента | Товар 1 | Товар 2 | Цена 1 | Цена 2 |
|---|---|---|---|---|---|
| 001 | Анна | Книга | Ручка | 10 | 2 |
| 002 | Иван | Блокнот | 5 | | |

Здесь:

- Много колонок с повторяющимися типами данных (Товар 1, Товар 2…).
- Трудно масштабировать.
- Сложно выполнять SQL-запросы.

### Первая нормальная форма (1NF)

Разбиваем повторяющиеся группы:

| Заказ_ID | Имя клиента | Товар | Цена |
|---|---|---|---|
| 001 | Анна | Книга | 10 |

| Заказ_ID | Имя клиента | Товар | Цена |
|---|---|---|---|
| 001 | Анна | Ручка | 2 |
| 002 | Иван | Блокнот | 5 |

Теперь:

- Каждый атрибут — **атомарный**.
- Удобнее считать сумму заказа.
- Данные структурированы.

### Вторая нормальная форма (2NF)

Выносим клиента в отдельную таблицу, чтобы не дублировать имя:

**Таблица Клиенты:**

| Клиент_ID | Имя клиента |
|---|---|
| 1 | Анна |
| 2 | Иван |

**Таблица Заказы:**

| Заказ_ID | Клиент_ID |
|---|---|
| 001 | 1 |
| 002 | 2 |

**Таблица Состав заказа:**

| Заказ_ID | Товар | Цена |
|---|---|---|
| 001 | Книга | 10 |
| 001 | Ручка | 2 |
| 002 | Блокнот | 5 |

Теперь:

- У каждого заказа — связанный клиент.
- Нет повторения имени клиента.
- Всё зависит от **всего первичного ключа**, а не от части.

### Третья нормальная форма (3NF)

Допустим, у нас появляется колонка “Город клиента” и “Регион”. Если город → регион, то это **транзитивная зависимость**.

Выносим справочник:

**Таблица Города:**

| Город | Регион |
|---|---|
| Минск | Минская обл |
| Гомель | Гомельская |

А в таблице клиентов указываем только Город.

**Что запомнить**

- Нормализация = меньше дубликатов, меньше ошибок.
- Для интервью и реальной работы чаще всего достаточно 1NF, 2NF и 3NF. Но это не так часто спрашивают.
- Если данных немного и читаемость важнее, можно осознанно применять **денормализацию** — например, для отчётов или кэширования.

**Денормализация** — это процесс **умышленного добавления избыточности в данные** (объединение таблиц, дублирование колонок и значений), чтобы упростить и **ускорить** чтение информации.

Чаще всего её применяют:

- в системах с высокой нагрузкой на чтение;
- в хранилищах данных (data warehouse);
- когда скорость важнее чистоты структуры;
- для удобства отображения данных в UI, отчётах, дашбордах.

### Нереляционные базы данных (NoSQL)

**Нереляционная база данных** — это база данных, в которой не используется табличная схема строк и столбцов.

Данные могут храниться как простые пары "ключ — значение", документы JSON или граф, состоящий из ребер и вершин. Чаще всего вы будете работать с БД на основе документов.

**Примеры СУБД:** MongoDB, DynamoDB

### Реальная жизнь и советы

1. Не всегда на проекте тестировщикам выдают доступ к БД. С вашей стороны вы должны настаивать на выдачу таких доступов, так как мы должны тестировать правильность сохранения данных, а также без БД гораздо сложнее тестировать разрешения и права, другие специфические кейсы
2. Обычно права тестировщиков ограничены. Чаще всего на Prod DB не может работать никто, кроме самых доверенных людей

3. Миграция - перемещение данных из одной базы в другой, изменение схемы текущей базы
4. У базы данных должен быть backup (резервная копия)
5. Все операции у DELETE делайте сначала на SELECT (подробнее в следующих уроках)

1. Презентация
2. Старая версия урока https://youtu.be/vhYZYALBL8M
3. Что такое База Данных (БД) https://habr.com/ru/post/555760/
4. https://www.oracle.com/cis/database/what-is-database/
5. Нормализация https://info-comp.ru/database-normalization
6. Нереляционные БД https://learn.microsoft.com/ru-ru/azure/architecture/data-guide/big-data/non-relational-data
7. Книга «Работа с MySQL, MS SQL Server и Oracle в примерах» https://svyatoslav.biz/database_book/
8. Подробнее о связях в реляционных базах данных https://habr.com/ru/post/193380/
9. Библия QA. Тестирование баз данных (Database) https://vladislaveremeev.gitbook.io/qa_bible/testirovanie-v-raznykh-sferakh-oblastyakh-testing-different-domains/testirovanie-baz-dannykh-database

РАБОТА С ДАННЫМИ

# 53. Работа с DBeaver и MySQL server. Создание таблиц (версия 2026)

> На курсе нам не нужно устанавливать свой локальный сервер базы данных, поэтому вы можете ограничиться только DBeaver. Но если вы захотите развернуть базу у себя локально, то вам поможет видео в последнем шаге урока, так как в БД курса вы сможете отправлять только SELECT и JOIN.

**DBeaver** – это универсальный клиент для работы с базами данных (СУБД).

**Зачем DBeaver нужен QA-инженеру:**

- **Проверка сохранения данных:** Вы зарегистрировали пользователя через графический интерфейс (GUI). Как убедиться, что он действительно сохранился, а не просто "повис" в кэше браузера? Нужно зайти в БД и найти эту запись через SQL-запрос.
- **Подготовка тестовых данных:** Быстро изменить баланс тестового пользователя на 1 000 000 рублей или поменять статус заказа с "В обработке" на "Доставлен", минуя долгий процесс прокликивания интерфейса.
- **Универсальность:** Вам не нужно устанавливать разные программы для PostgreSQL, MySQL или Oracle. DBeaver умеет подключаться практически к любой существующей базе данных.

Для нашей работы и большинства задач на реальных проектах используется бесплатная версия – **DBeaver Community Edition (CE)**.

### 1. Установка DBeaver на Windows

Процесс установки на Windows стандартен и не требует сложных настроек.

1. **Скачивание:** Перейдите на официальный сайт программы https://dbeaver.io/download/.
2. В блоке **DBeaver Community Edition** найдите раздел для Windows и выберите ссылку **Windows**. Скорее всего он будет выбран автоматически.
3. **Установка:** Запустите скачанный `.exe` файл.

4. Следуйте инструкциям установщика. На этапе выбора компонентов (Choose Components) оставьте все галочки по умолчанию.

> *Примечание:* DBeaver написан на языке Java, поэтому для его работы нужна среда JRE. Современный установщик для Windows уже включает в себя необходимую версию Java, поэтому вам не придется устанавливать её отдельно. Если ее нет (узнаете по ошибке при первом запуске DBeaver), то просто скачайте по этой ссылке.

1. Нажмите **Install** (Установить), дождитесь окончания процесса и нажмите **Finish**.

### 2. Установка DBeaver на macOS

На компьютерах Mac процесс установки зависит от типа вашего процессора.

1. **Скачивание:** Перейдите на страницу https://dbeaver.io/download/.
2. Если ОС не определилась автоматически, то в блоке **DBeaver Community Edition** найдите раздел для Mac OS X. Вам нужно выбрать правильный `.dmg` файл:
    - **Mac OS X (dmg):** Выбирайте этот вариант, если у вас старый Mac на процессоре **Intel**.
    - **Mac OS X (Apple Silicon):** Выбирайте этот вариант, если у вас современный Mac на чипах **M1, M2, M3 и т.д.** *(Проверить процессор можно, нажав на значок "Яблока" в левом верхнем углу -> "Об этом Mac" / "About This Mac").*
3. **Установка:** Дважды кликните по скачанному `.dmg` файлу. Откроется небольшое окно.
4. Перетащите иконку DBeaver в папку **Applications** (Программы), расположенную в этом же окне.
5. **Обход блокировки безопасности:** Операционная система macOS строго относится к программам, скачанным из интернета.
    - При первом запуске DBeaver (из Launchpad или папки Программы) может появиться предупреждение: *"Невозможно открыть программу DBeaver, так как разработчик не установлен"*.

- Нажмите **Отменить** (Cancel).
- Откройте **Системные настройки (System Settings)** -> **Конфиденциальность и безопасность (Privacy & Security)**.
- Прокрутите вниз. Там появится сообщение о том, что запуск DBeaver был заблокирован. Нажмите кнопку **Подтвердить вход (Open Anyway)**.

> **Продвинутый способ (Для пользователей Homebrew на macOS):** Если вы умеете пользоваться терминалом и у вас установлен пакетный менеджер Homebrew, вы можете установить программу одной командой: `brew install --cask dbeaver-community`

### 3. Первый запуск и важная особенность (Загрузка драйверов)

Когда вы впервые откроете DBeaver, программа может предложить вам создать **Sample Database** (Тестовую базу данных).

- Если вы хотите потренироваться в написании SQL-запросов прямо сейчас, то нажмите **Yes**. Программа создаст локальную базу SQLite с готовыми таблицами. Вы можете ее создать, чтобы потренироваться на запросах, которые будут ограничены для БД курса.
- Если вы планируете сразу подключаться к рабочей базе данных вашего проекта, то нажмите **No**. Подключение к базе курса мы сделаем на следующих шагах.

> **Важное правило для QA (Загрузка JDBC-драйверов):** DBeaver – это просто графическая оболочка. Чтобы программа говорила на языке конкретной базы данных (например, PostgreSQL), ей нужен специальный файл-переводчик — **драйвер**. Когда вы будете первый раз настраивать подключение к новой для вас СУБД, DBeaver автоматически покажет всплывающее окно **"Download driver files"** (Скачать файлы драйвера). Всегда нажимайте **Download (Скачать)**. Программа сама найдет в интернете безопасные официальные файлы, установит их и успешно завершит подключение. Если окно не покажется, значит уже все установлено.

### Подключение к удаленной БД MySQL

Чаще всего тестировщики работают не с локальными базами на своем компьютере, а подключаются к удаленным серверам (тестовым стендам или staging-окружению).

Давайте разберем пошаговый алгоритм создания подключения к MySQL и отправки вашего первого запроса.

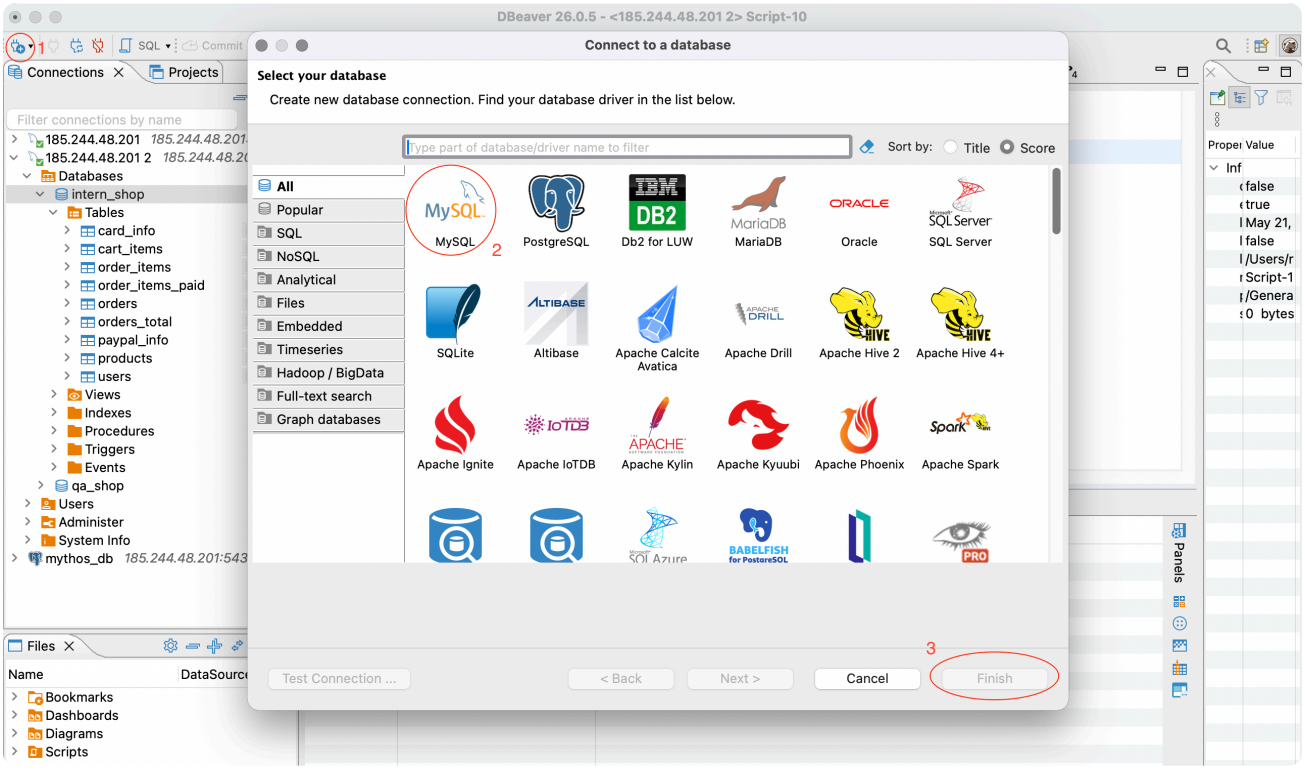
### 1. Создание нового подключения и ввод доступов

Для подключения вам потребуются учетные данные (credentials), которые обычно вам выдают в команде.

1. Откройте DBeaver. В левом верхнем углу нажмите на иконку

   (New Database Connection / Новое подключение).

2. В появившемся окне выберите базу данных **MySQL** (находится на вкладке All) и нажмите **Next** (Далее).




3. Откроется окно настройки соединения (Connection Settings). Заполните поля доступами к БД курса:

- **Server Host (Хост):** *`hayabusa.proxy.rlwy.net`* Это адрес сервера, где находится база.
- **Port (Порт):** *18163 (по умолчанию для MySQL это 3306, если не указано иное).*
- **Database (База данных):** *если не знаете точное имя, можно оставить поле пустым, DBeaver покажет все доступные базы после подключения*. В нашем случае можно оставить пустым.
- **Username (Пользователь):** *`student`*.
- **Password (Пароль):** *`StudentRead2026`*. Оставьте галочку *Save password* (Сохранить пароль), чтобы не вводить его каждый раз при запуске программы.

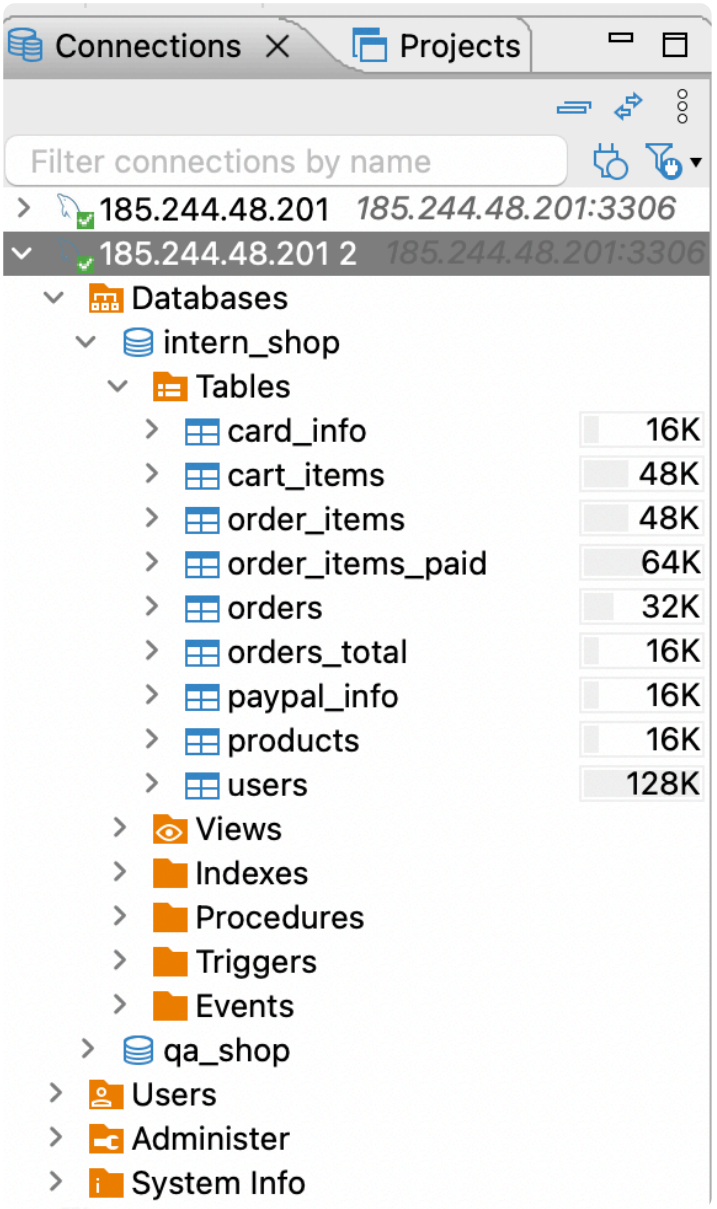
4. *Также вы можете использовать [https://db.demoshopping.ru/](https://db.demoshopping.ru/) для этой задачи с теми же данными для входа. Он не требует установки.*
5. Перед нажатием на **Finish***,* Сначала нажмите кнопку **Test Connection (Тест соединения)** в левом нижнем углу.
   - Если DBeaver предложит скачать драйверы для MySQL, то соглашайтесь (кнопка *Download*).
   - Если появится окно `Connected`, значит доступы верны, сеть открыта, и вы успешно достучались до сервера. Нажимайте **OK**, а затем **Finish**.
   - *Кейс:* Если тест падает с ошибкой `Connection timed out` или `Access denied`, значит либо креды неверны, либо ваш IP-адрес не добавлен в белый список (VPN не включен). Убедитесь, что введено верно, если не может, то с логом ошибки нужно идти к ответственному лицу.

### 2. Работа с интерфейсом: Как смотреть данные без SQL

В левой панели **Database Navigator (Навигатор баз данных)** появилось ваше новое подключение.

1. Кликните по нему дважды или нажмите на стрелочку рядом (разверните дерево).

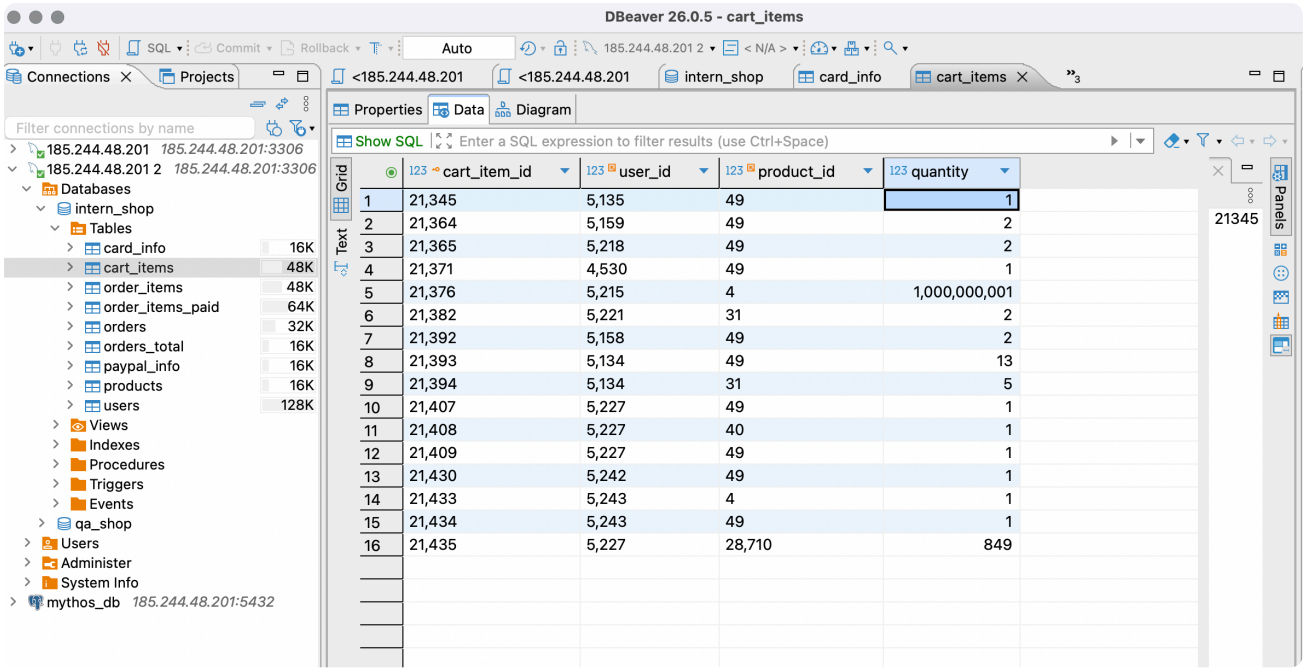
2. Перейдите по пути: **Имя вашего подключения** -> **Databases** -> **Ваша база** -> **Tables (Таблицы)**.




3. Вы увидите список всех таблиц (например, `users`, `orders`, `products`).

4. **Двойной клик по таблице** откроет её в правой части экрана.

5. Перейдите на вкладку **Data (Данные)**. Вы увидите привычную таблицу, похожую на Excel.

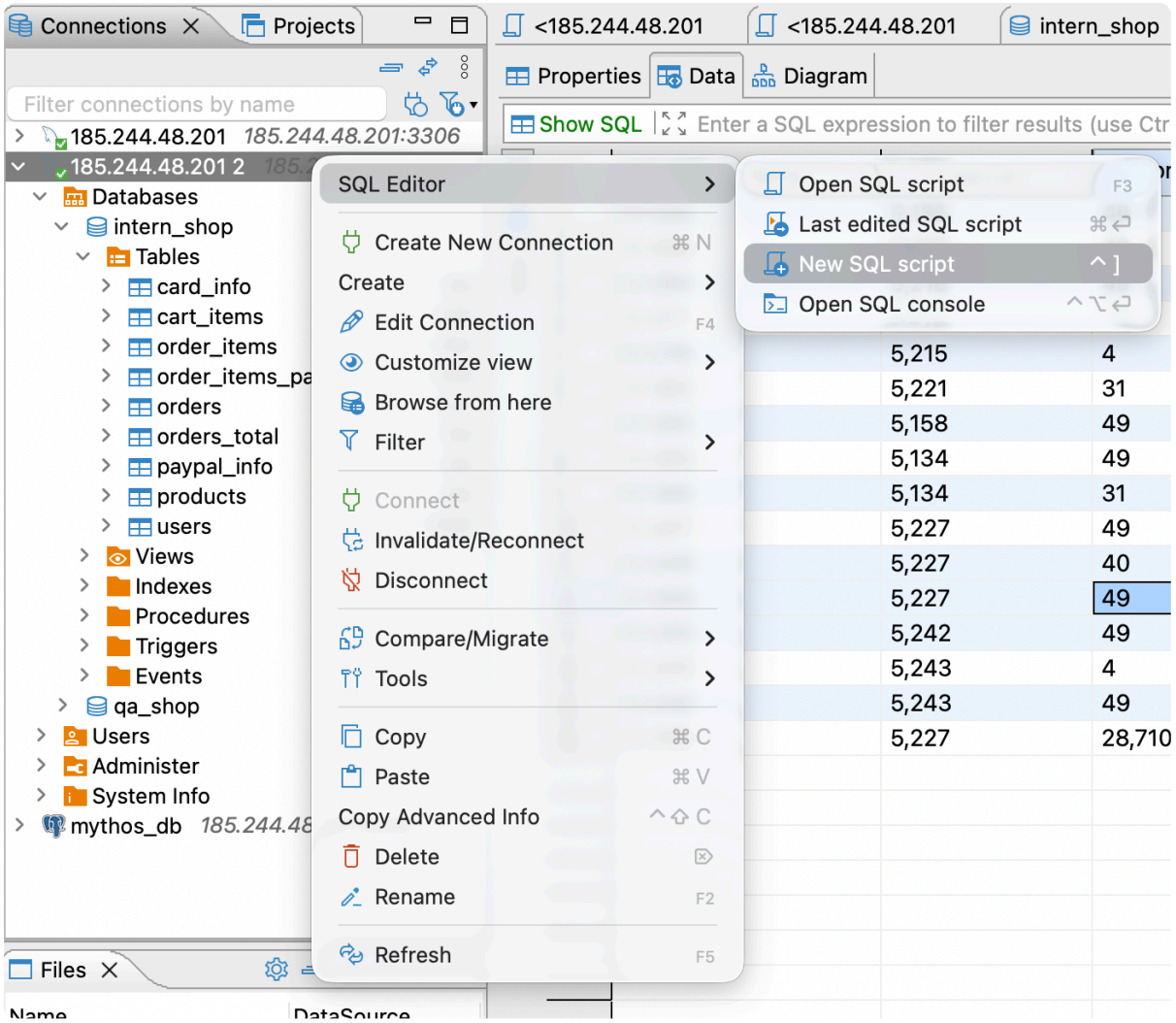



### 3. Написание и выполнение SQL-запросов

Далее в курсе мы будем взаимодействовать с данным через специальные SQL-запросы. Давайте научимся их отправлять в DBeaver.

**Как открыть редактор:**

- Кликните **правой кнопкой мыши** по вашему подключению в левой панели.
- Выберите **SQL Editor -> New SQL script (Редактор SQL -> Новый скрипт SQL)**.




- *Горячая клавиша:* Нажмите F3 (или F4), предварительно выделив подключение.

**Как выполнить запрос:**

1. В открывшемся окне редактора сначала выберите базу, написав следующий запрос:

```
USE intern_shop;
```

2. Теперь можно отправлять реальные запросы к таблицам из БД. Например, выведем 10 последних зарегистрированных пользователей:

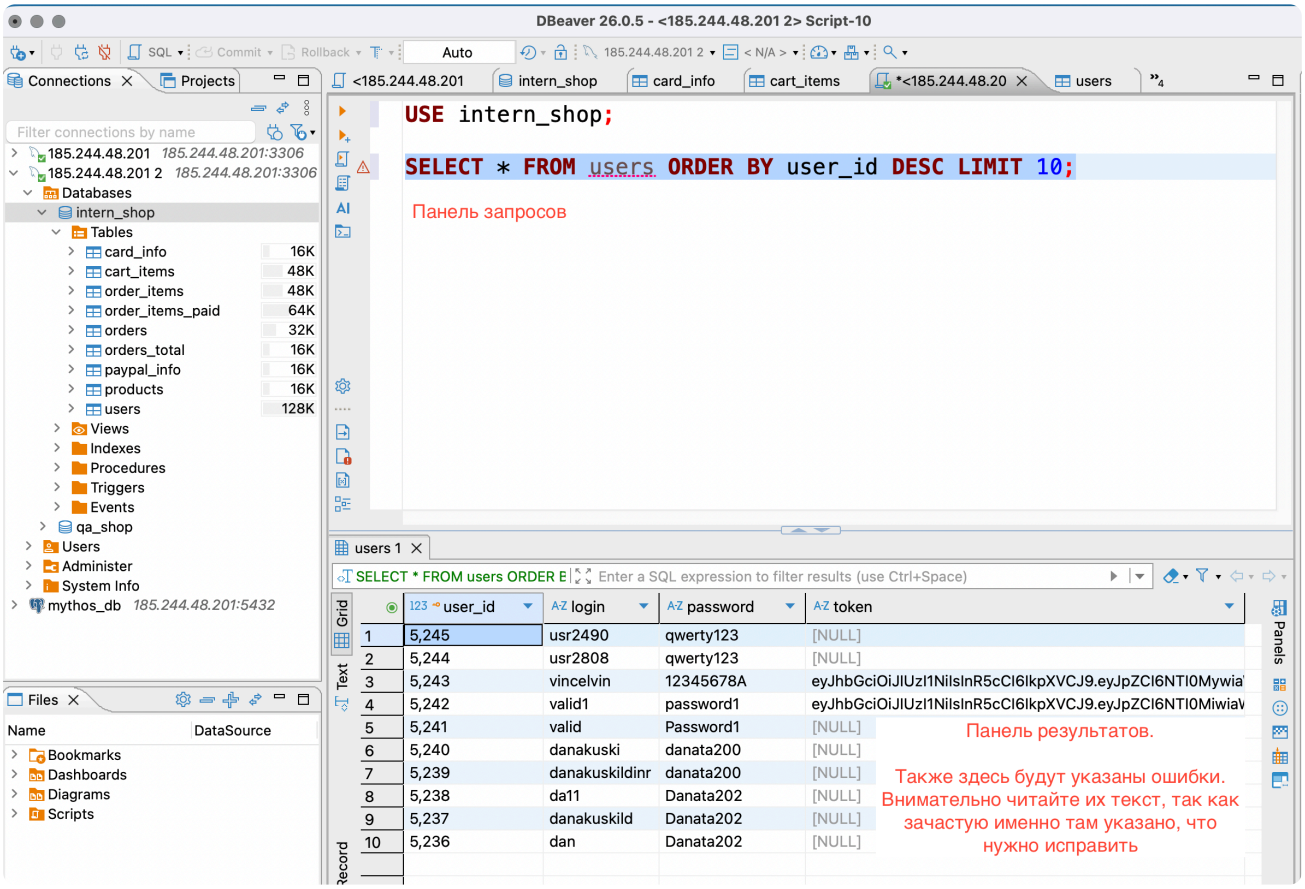
```
SELECT * FROM users ORDER BY user_id DESC LIMIT 10;
```

3. Чтобы выполнить запрос, можно нажать кнопку

в меню. Или поставить курсор мигающим на любую часть строки с запросом и нажмите сочетание клавиш:

- **Windows:** `Ctrl + Enter`
- **macOS:** `Cmd + Enter`
- *(Либо нажмите на оранжевую иконку Play ▶ в левой панели редактора).*

4. В нижней части экрана появится панель **Results (Результаты)** с ответом от базы данных.




5. Отправленные запросы можно сохранять в формате .sql и повторно использовать.

> **Важное замечание по безопасности (Auto-commit):** > По умолчанию в DBeaver включен режим `Auto-commit` (иконка зеленой галочки на верхней панели). Это значит, что если вы напишете и выполните запрос `UPDATE` (изменение данных) или `DELETE` (удаление), база применит их мгновенно и

> безвозвратно. При работе с удаленными базами (особенно если это предпродуктовый стенд) будьте крайне осторожны с этими командами. Обязательно используйте условие `WHERE`, чтобы не удалить все данные в таблице.
>
> В БД курса вы не можете отправлять запросы на создание, удаление и изменение, а только на поиск и чтение (SELECT и JOIN).

### Полезные функции DBeaver для тестировщика

Многие тестировщики боятся баз данных, думая, что для работы с ними нужно знать язык SQL на уровне программиста. DBeaver – это мощный визуальный инструмент, который позволяет выполнять 80% повседневных задач QA вообще без написания кода, работая с базой так же просто, как с таблицей в Excel.

Давайте разберем примеры полезных функций в этом и следующем шаге урока.

### ER-диаграммы (Визуализация структуры базы данных)

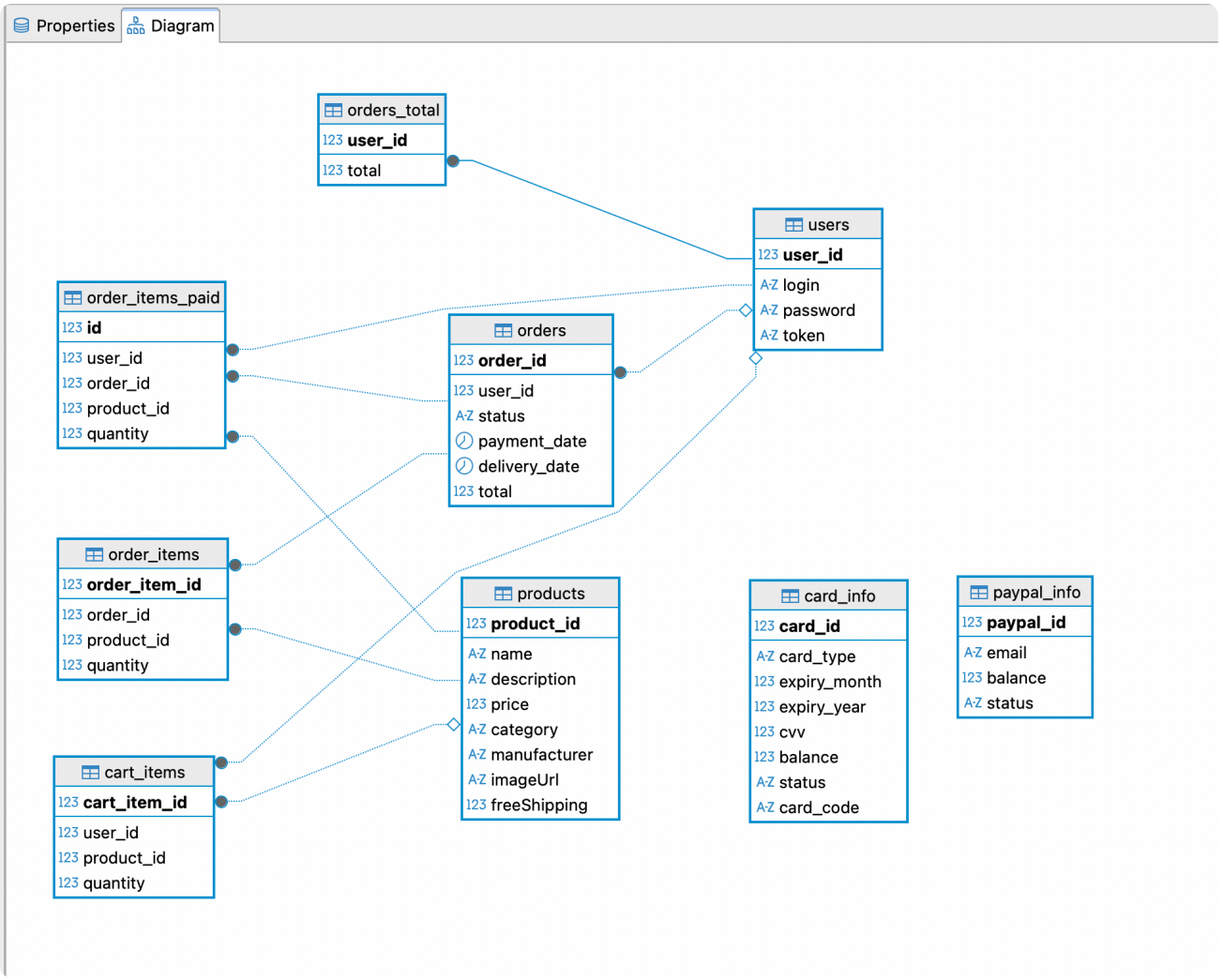
**ER-диаграмма (Entity-Relationship)** – это графическая схема, которая показывает, какие таблицы есть в базе и как они связаны друг с другом через ключи.

**Зачем это тестировщику:** Вам нужно протестировать удаление пользователя. Вы нажимаете кнопку "Удалить" в интерфейсе, но получаете ошибку от сервера. Почему? Открыв ER-диаграмму, вы визуально увидите, что таблица `users` связана линией с таблицей `orders`. База данных не дает удалить пользователя, потому что у него остались активные заказы. Понимание связей позволяет планировать тестовые сценарии глубже и не дергать разработчиков вопросами: "А где лежат данные профиля?".

**Как открыть:**

1. В левой панели (Database Navigator) найдите нужную таблицу (или всю базу данных/схему целиком).
2. Дважды кликните по ней.
3. В правой части окна перейдите на вкладку **Diagram (ER-диаграмма)**.

4. Вы увидите блоки (таблицы) и линии (связи) между ними. Схему можно масштабировать, двигать блоки и даже экспортировать как вложение для прикрепления к баг-репорту.




**Как читать диаграмму?**

### 1. Блоки (Таблицы и Колонки)

Каждый синий прямоугольник на схеме – это отдельная таблица в базе данных (сущность). Внутри прямоугольника перечислены ее колонки (атрибуты).

- **Название таблицы:** Всегда написано на самом верху блока (например, `users`, `orders`, `products`).
- **Иконки типов данных:** Слева от названия каждой колонки стоят маленькие значки. Они подсказывают тестировщику, какие данные лежат внутри, и помогают придумывать проверки:

- **123 (Числа):** Значит, колонка принимает только цифры (Integer). Например, `price` (цена) или `quantity` (количество). Если вы попробуете через API передать в количество букву "A", база выдаст ошибку.
- **A-Z (Строки):** Текстовые данные (Varchar/Text). Например, `login`, `name` или `description`. Здесь тестировщик будет проверять длину строки и спецсимволы.
- **Значок часов (Дата/Время):** Колонка хранит временную метку (Timestamp). Пример: `payment_date` в таблице `orders`.

**2. Жирный шрифт (Первичные ключи)**

Обратите внимание: в каждой таблице одна или несколько колонок выделены **жирным шрифтом**. Это Первичный Ключ (Primary Key или PK).

- **Что это значит:** Первичный ключ – это уникальный идентификатор конкретной строки. Он никогда не может повторяться.
- **Пример со скрина:** В таблице `users` жирным выделен `user_id`. Это значит, что в базе не может быть двух пользователей с `user_id = 1`.
- **Практика для QA:** Если при регистрации пользователя система падает с ошибкой базы данных `Duplicate entry for key PRIMARY`, значит, разработчик попытался записать в базу пользователя с уже существующим ID.

**3. Линии (Связи и Внешние ключи)**

Это самое важное на схеме. Линии показывают, как таблицы "общаются" между собой. Линия всегда соединяет Первичный ключ одной таблицы с такой же колонкой (Внешним ключом / Foreign Key) в другой таблице.

- **Как читать линию (На примере `users` и `orders`):** Найдите линию, идущую от таблицы `users` к таблице `orders`. Она соединяет `user_id` из первой таблицы с `user_id` во второй. Это значит, что каждый заказ жестко привязан к конкретному пользователю.
- **Точки и ромбики на концах линий:** Они показывают тип связи ("Один ко многим", "Один к одному"). DBeaver рисует точку или "вилку" на той стороне, где элементов может быть много.

    - *Читаем скриншот:* От `users` идет простая линия, а к `orders` она подходит с точкой. Перевод на человеческий язык: "У ОДНОГО пользователя может быть МНОГО заказов".

- **Связь через промежуточные таблицы:** Посмотрите на таблицу `order_items` (товары в заказе). Она соединена линиями сразу с двумя таблицами: `orders` (через `order_id`) и `products` (через `product_id`). Это классическая реализация корзины покупок. Чтобы узнать, какие товары купил пользователь, база сначала смотрит в заказ, а из заказа идет в список товаров.

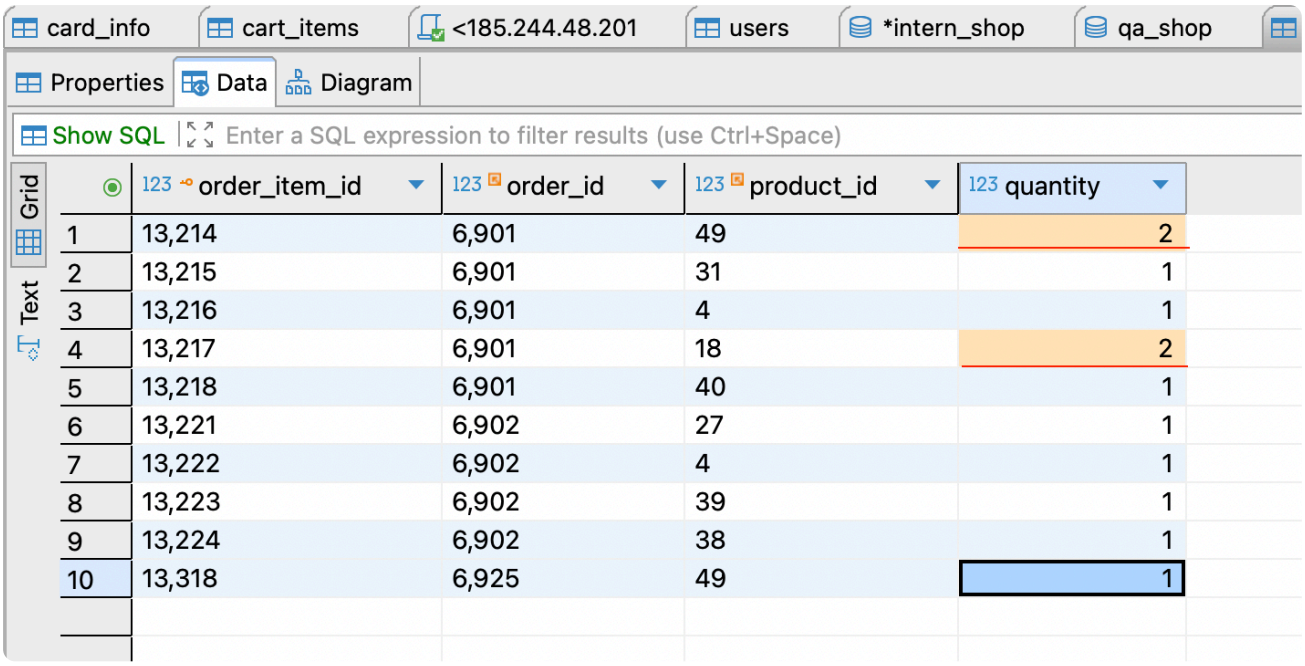
### Редактирование данных "на лету" (GUI Data Editing)

Вам не обязательно писать скрипт `UPDATE users SET status = 'ACTIVE' WHERE id = 5;`, чтобы поменять тестовые данные.

**Зачем это тестировщику:** Это идеальный способ быстро сымитировать различные состояния системы. Например, вам нужно проверить, как отображается интерфейс "Заблокированного" пользователя, но в админке еще не сделали кнопку "Заблокировать".

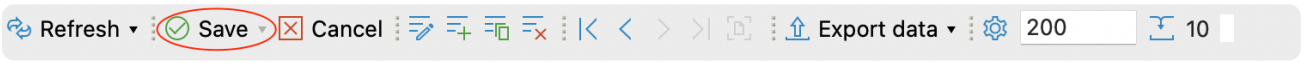
**Как изменить данные (в БД курса вы это сделать не сможете, так как у вас не таких прав):**

1. Откройте таблицу двойным кликом и перейдите на вкладку **Data (Данные)**.
2. Найдите нужную строку и дважды кликните по ячейке, которую хотите изменить (например, колонка quantity).
3. Впишите новое значение (или выберите из выпадающего списка, если это связано с другой таблицей).
4. **Важно:** Ячейка подсветится желтым или оранжевым цветом. Это значит, что изменения пока существуют только в вашем DBeaver и еще не отправлены на сервер.




| | order_item_id | order_id | product_id | quantity |
|---|---|---|---|---|
| 1 | 13,214 | 6,901 | 49 | 2 |
| 2 | 13,215 | 6,901 | 31 | 1 |
| 3 | 13,216 | 6,901 | 4 | 1 |
| 4 | 13,217 | 6,901 | 18 | 2 |
| 5 | 13,218 | 6,901 | 40 | 1 |
| 6 | 13,221 | 6,902 | 27 | 1 |
| 7 | 13,222 | 6,902 | 4 | 1 |
| 8 | 13,223 | 6,902 | 39 | 1 |
| 9 | 13,224 | 6,902 | 38 | 1 |
| 10 | 13,318 | 6,925 | 49 | 1 |

5. Чтобы применить изменения к реальной базе, нажмите кнопку **Save (Сохранить)** в нижней панели окна или сочетание `Ctrl + S` (`Cmd + S` на Mac).




## Экспорт и Импорт данных

**Зачем это тестировщику:**

- **Экспорт:** Вы нашли сложный баг, который воспроизводится только на конкретном наборе данных. Вместо того чтобы делать 10 скриншотов таблицы, вы выгружаете эти строки в CSV-файл и прикладываете к задаче в Jira. Разработчик скачает файл и сразу поймет контекст.
- **Импорт:** Вам нужно провести нагрузочное тестирование или просто наполнить пустую базу. Вместо ручной регистрации 100 пользователей на сайте, вы берете готовый Excel-файл и "заливаете" его в таблицу базы данных за 5 секунд.
- Также экспорт и импорт используется для изучения проблем с данными со стороны клиента. Он может прислать свои данные для исследования.

**Как сделать Экспорт:**

1. Выполните любой SQL-запрос или просто откройте вкладку **Data** нужной таблицы.

2. Нажмите правой кнопкой мыши по любой ячейке в результате.

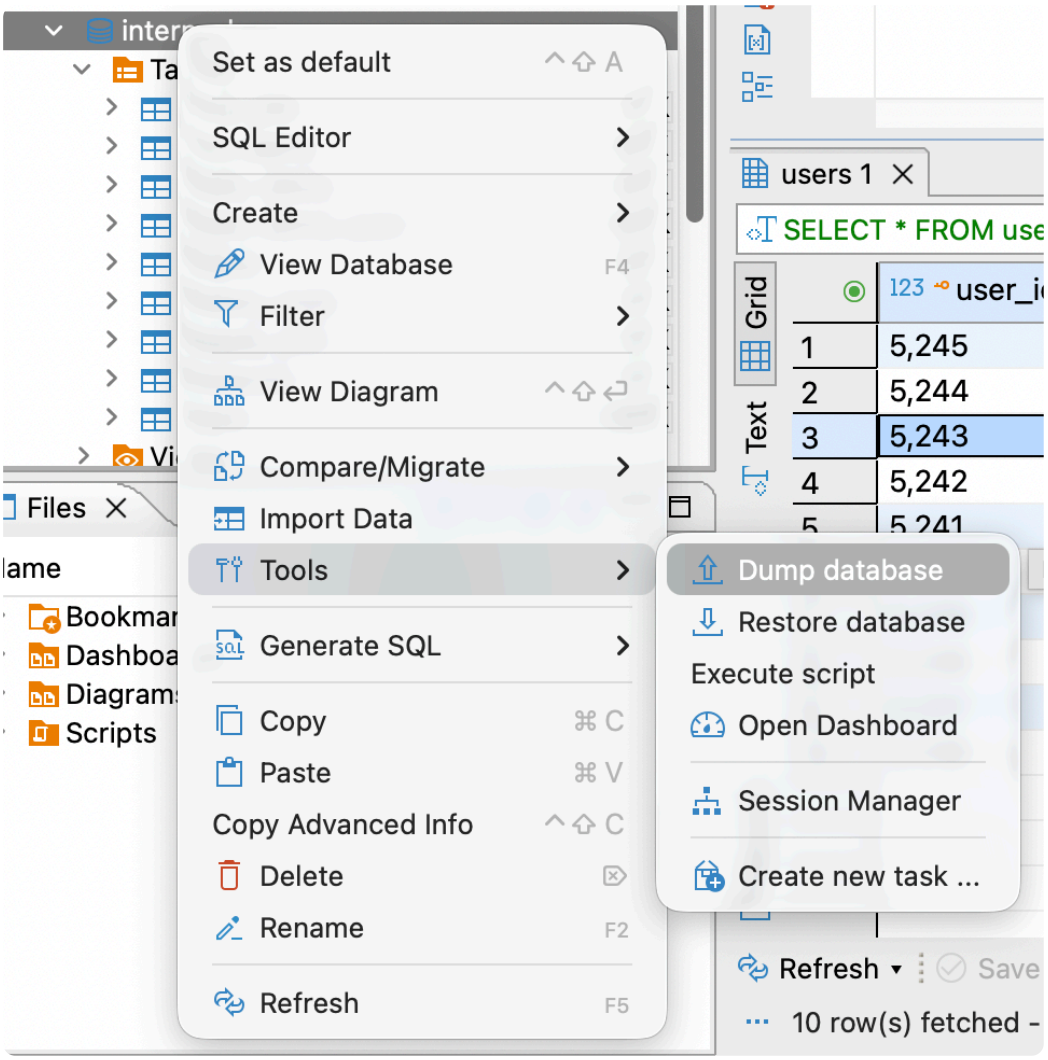
3. Выберите **Export Data (Экспорт данных)**.

4. Выберите формат: **CSV** (самый универсальный), **Excel** или **HTML**. Нажмите Next и укажите папку для сохранения на компьютере.

5. Аналогичным образом вы можете экспортировать все таблицы и данные из них.

6. После это данные можно импортировать обратно, используя опцию **Import Data**.

7. Помимо экспорта и импорта данных из таблиц и результатов, при миграции данных в другие БД создаются специальным дампы через опцию **Dump database**, которые содержат информацию обо всей базе. Это можно сделать на уровне базы.



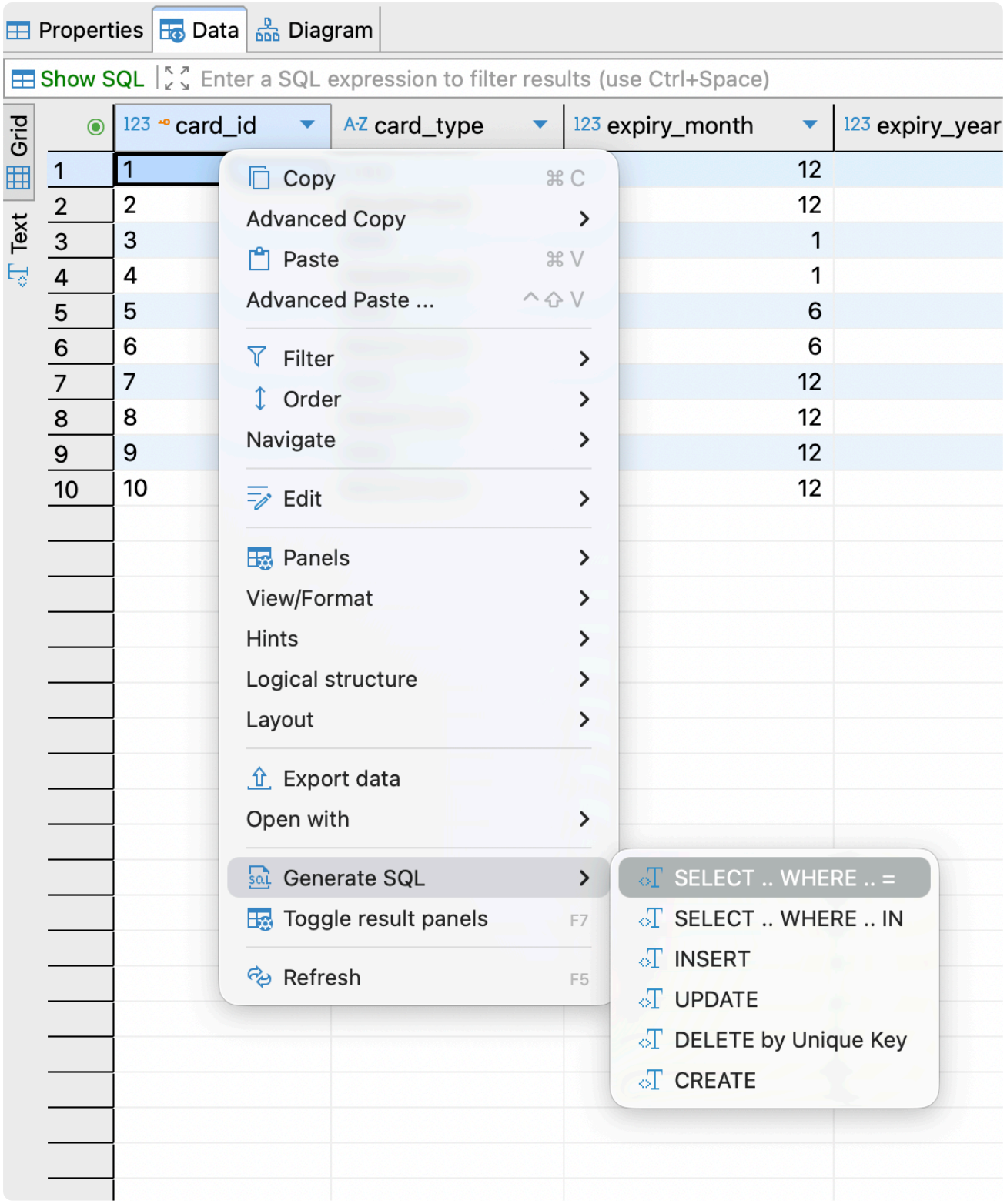
### Автоматическая генерация SQL-кода (Generate SQL)

Эта функция экономит массу времени и избавляет от синтаксических ошибок.

**Зачем это тестировщику:** Вы создали тестового пользователя через интерфейс сайта. Теперь вам нужно быстро создать еще пятерых таких же, поменяв только email. Писать длинный запрос `INSERT INTO...` вручную, перечисляя все 20 колонок таблицы, долго и чревато опечатками.

**Как сгенерировать скрипт:**

1. Во вкладке **Data** нажмите правой кнопкой мыши по нужной строке (например, по вашему тестовому пользователю).
2. Выберите пункт меню **Generate SQL (Сгенерировать SQL)**.



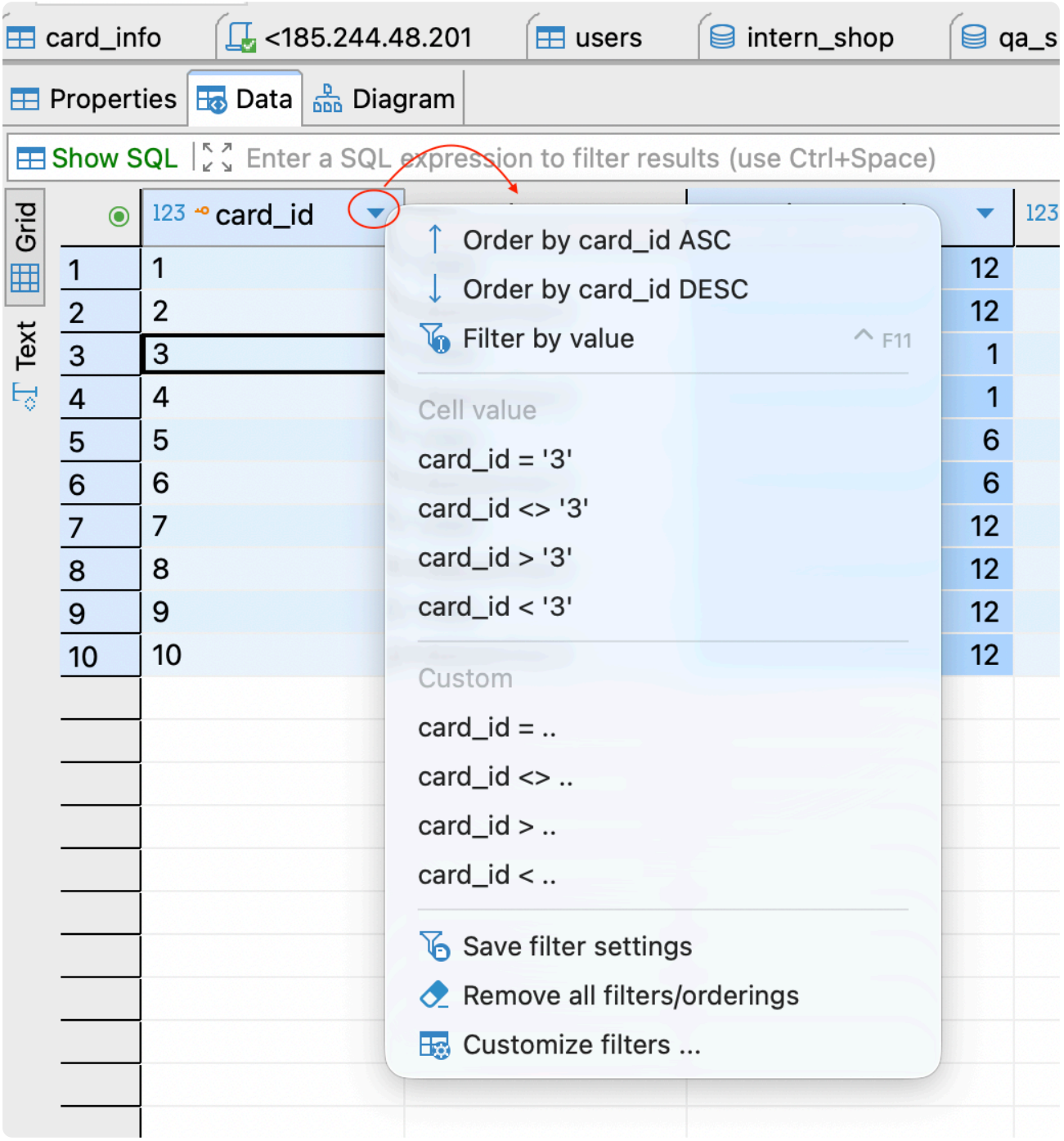

3. Выберите тип запроса:

   - **INSERT:** DBeaver сам напишет готовый скрипт для создания точной копии этой строки.
   - **UPDATE:** Программа напишет шаблон для изменения именно этой строки (с уже подставленным условием `WHERE id = ...`).

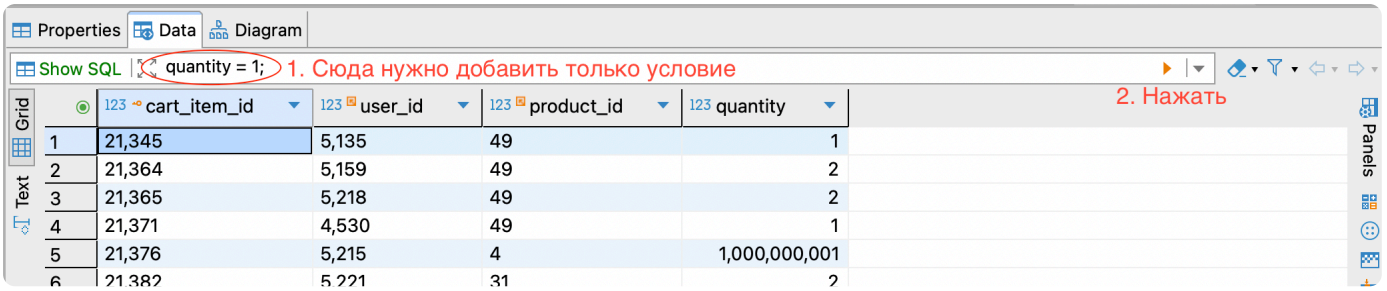
4. Скопируйте готовый код в SQL-редактор, поменяйте нужные значения (например, email) и выполните запрос.

### Использование фильтров и сортировок в таблице

1. Над таблицей во вкладке **Data** есть специальная панель для быстрой фильтрации. Вы можете кликнуть на название любой колонки, чтобы отсортировать данные (по алфавиту, по дате или от большего к меньшему).



2. Также вы можете отправлять добавлять условия для фильтрации через SQL-запрос прямо во вкладке Data. Обратите внимание, что DBeaver автоматически добавляет первую часть запроса до слова WHERE, поэтому вам нужно указать только условие сортировки.




**Типы данных**

**INTEGER (INT)**:

- Пример: age INT
- Этот тип данных используется для хранения целых чисел, например, возраста.

**FLOAT и DOUBLE**:

- Пример: price FLOAT или average_score DOUBLE
- Эти типы данных используются для хранения чисел с плавающей точкой, таких как цены или средние оценки.

**VARCHAR**:

- Пример: first_name VARCHAR(50)
- Этот тип данных предназначен для хранения текстовых строк переменной длины, таких как имена.

**CHAR**:

- Пример: gender CHAR(1)
- CHAR также используется для хранения строк, но фиксированной длины. В этом примере, CHAR(1) будет использован для хранения одного символа (например, "M" или "F" для пола).

**DATE**:

- Пример: birth_date DATE

- Этот тип данных предназначен для хранения даты (год, месяц, день).

**TIME**:

- Пример: appointment_time TIME
- TIME используется для хранения времени (часы, минуты, секунды).

**DATETIME и TIMESTAMP**:

- Пример: created_at DATETIME или updated_at TIMESTAMP
- Эти типы данных используются для хранения даты и времени. DATETIME сохраняет фиксированную дату и время, в то время как TIMESTAMP может использоваться для отслеживания времени создания и обновления записей.

**BOOLEAN**:

- Пример: is_active BOOLEAN
- BOOLEAN предназначен для хранения логических значений, таких как true или false.

**ENUM**:

- Пример: status ENUM('active', 'inactive', 'pending')
- ENUM представляет собой перечисление значений и используется, когда столбец может иметь только одно из ограниченного множества значений.

**BLOB и TEXT**:

- Пример: profile_picture BLOB или comments TEXT
- BLOB (Binary Large Object) и TEXT используются для хранения больших объемов данных, таких как изображения (BLOB) и длинные тексты (TEXT).

**NULL**: Это специальное значение в MySQL, которое указывает на отсутствие данных или неопределенное значение в столбце. Когда столбец имеет тип данных, который допускает NULL (например, VARCHAR, INT, DECIMAL, и т.д.) и в него не внесено конкретное значение, то это значение считается NULL. Это означает, что в ячейке столбца нет данных.

**Пустое значение (Empty Value)**: Это может относиться к строкам VARCHAR или TEXT, где внутри столбца есть строка, но она пуста, т.е., не содержит

символов. Это не то же самое, что NULL. В этом случае, столбец не содержит NULL, но содержит пустую строку.

### Работа с базами данных

**SHOW DATABASES**:

- Получение списка всех доступных баз данных.
- Пример: *SHOW DATABASES;*

**CREATE DATABASE**:

- Создание новой базы данных.
- Пример: *CREATE DATABASE mydb;*

**DROP DATABASE**:

- Удаление существующей базы данных и всех её данных.
- Пример: *DROP DATABASE mydb;*

**USE DATABASE**:

- Выбор базы данных, с которой вы хотите работать (активировать).
- Пример: *USE mydb;*

### Работа с таблицами

**CREATE TABLE**:

- Создание новой таблицы.
- Пример: *CREATE TABLE users ( id INT AUTO_INCREMENT PRIMARY KEY, username VARCHAR(50) NOTNULL, email VARCHAR(100) UNIQUE, birthdate DATE );*

**DROP TABLE**:

- Удаление таблицы и всех её данных.
- Пример: *DROP TABLE users;*

**DESCRIBE (или DESC)**:

- Получение информации о структуре таблицы.
- Пример: *DESCRIBE users;*

**INSERT INTO**:

- Вставка новых записей в таблицу.
- Пример: *INSERT INTO users (username, email, birthdate) VALUES ('john_doe', 'john@example.com', '1990-05-15');*

**DELETE**:

- Удаление записей из таблицы.
- Пример: *DELETE FROM users WHERE id = 1;*

**UPDATE**:

- Обновление данных в таблице.
- Пример: *UPDATE users SET email = 'new_email@example.com' WHERE id = 2;*

**ALTER TABLE**:

- Изменение структуры существующей таблицы.
- Добавление нового столбца: ALTER TABLE users ADD COLUMN phone_number VARCHAR(20);
- Изменение типа столбца: ALTER TABLE users MODIFY COLUMN phone_number INT;
- Удаление столбца: *ALTER TABLE users DROP COLUMN birthdate;*

**Полезные материалы**

1. Презентация
2. Старая версия https://youtu.be/fXxB2pN3_30
3. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1D2YyxYTgi2dqEgAQ5XH_FgMYq-iqYwF_
4. Скачать DBeaver https://dbeaver.io/
5. Скачать MySQL Community Server https://dev.mysql.com/downloads/mysql/
6. Скачать Workbench https://dev.mysql.com/downloads/workbench/
7. MySQL: Типы данных https://metanit.com/sql/mysql/2.3.php
8. Практика по SQL https://www.w3schools.com/sql/
9. https://www.sql-ex.ru/
10. https://stepik.org/course/63054/promo
11. https://sqlbolt.com/

РАБОТА С ДАННЫМИ

# 54. SELECT запросы в SQL. Агрегатные функции

### Работа с SELECT

Вы можете создать таблицу, последовательно отправив два запроса ниже в ваш MySQL и повторив все остальные запросы

### Шаг 1. Создаем таблицы

```
CREATE TABLE departments (
  department_id INT AUTO_INCREMENT PRIMARY KEY,
  department_name VARCHAR(50) NOT NULL
);

CREATE TABLE employees (
  employee_id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  department_id INT,
  salary DECIMAL(10, 2),
  FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

INSERT INTO departments (department_name) VALUES
  ('HR'),
  ('Finance'),
  ('IT');

INSERT INTO employees (first_name, last_name, department_id, salary)
VALUES
  ('John', 'Doe', 1, 50000.00),
  ('Jane', 'Smith', 2, 60000.00),
  ('Michael', 'Johnson', 3, 75000.00),
  ('Emily', 'Davis', 2, 62000.00),
  ('William', 'Brown', 1, 52000.00);
```

### Шаг 2. Теперь давайте рассмотрим SQL-запросы с примерами:

**SELECT *:**

Получение всех данных из таблицы.

```
SELECT * FROM employees;
```

**SELECT DISTINCT:**

Получение уникальных значений столбца.

```
SELECT DISTINCT department_id FROM employees;
```

**SELECT * FROM ... LIMIT:**

Получение первых нескольких записей из таблицы.

```
SELECT * FROM employees LIMIT 3;
```

**SELECT field AS:**

Переименование столбца в результате запроса (псевдонимы).

```
SELECT first_name AS "First Name", last_name AS "Last Name" FROM employees;
```

**SELECT FROM WHERE:**

Получение данных с условием.

```
SELECT * FROM employees WHERE department_id = 1;
```

**SELECT FROM WHERE AND:**

Использование оператора AND для комбинирования условий (оба условия должны быть верны).

```
SELECT * FROM employees WHERE department_id = 2 AND salary > 60000.00;
```

**SELECT FROM WHERE OR:**

Использование оператора OR для комбинирования условий (хотя бы одно условие должно быть верно).

```
SELECT * FROM employees WHERE department_id = 1 OR department_id = 2;
```

**LIKE _:**

Поиск строк, где символ подчеркивания (_) заменяет ровно один любой символ.

Пример (найдет 'Doe'):

```
SELECT * FROM employees WHERE last_name LIKE 'D_e';
```

**LIKE %:**

Поиск строк с любым количеством произвольных символов до или после указанного текста.

Пример (найдет 'John', 'Jane'):

```
SELECT * FROM employees WHERE first_name LIKE 'J%';
```

**LIKE _o%:**

Поиск строк, где второй буквой является "o" (один любой символ в начале, затем "o", а затем любое количество символов).

Пример (найдет 'Doe', 'Johnson', 'Brown'):

```
SELECT * FROM employees WHERE last_name LIKE '_o%';
```

**BETWEEN:**

Поиск значений в заданном диапазоне (включая границы).

```
SELECT * FROM employees WHERE salary BETWEEN 55000.00 AND 75000.00;
```

**IN:**

Поиск значений, соответствующих любому значению из переданного списка.

```
SELECT * FROM employees WHERE department_id IN (1, 2);
```

**WHERE NOT:**

Поиск записей, которые не соответствуют условию.

```
SELECT * FROM employees WHERE NOT department_id = 3;
```

**WHERE (field > value) AND field = value:**

Сложное условие с операторами AND и сравнениями, сгруппированными скобками для читаемости.

```
SELECT * FROM employees WHERE (salary > 60000.00) AND (department_id = 2);
```

**ORDER BY ASC:**

Сортировка результатов по возрастанию (от меньшего к большему). ASC применяется по умолчанию, если ничего не указано.

```
SELECT * FROM employees ORDER BY salary ASC;
```

**ORDER BY DESC:**

Сортировка результатов по убыванию (от большего к меньшему).

```
SELECT * FROM employees ORDER BY salary DESC;
```

**MIN / MAX:**

Получение минимального и максимального значения из столбца.

```
SELECT MIN(salary) AS "Min Salary", MAX(salary) AS "Max Salary" FROM
employees;
```

**COUNT:**

Подсчет количества записей.

```
SELECT COUNT(*) AS "Total Employees" FROM employees;
```

**AVG:**

Получение среднего значения из столбца.

```
SELECT AVG(salary) AS "Average Salary" FROM employees;
```

## UNION, UNION ALL, HAVING

Давайте создадим две таблицы: `employees` и `departments`.

Таблица `employees` содержит данные о сотрудниках, а таблица `departments` – о департаментах, к которым они относятся.

Сначала выполните запросы `CREATE TABLE` по отдельности, а затем запросы `INSERT INTO`.

```
CREATE TABLE departments (
  department_id INT AUTO_INCREMENT PRIMARY KEY,
  department_name VARCHAR(50) NOT NULL
);

CREATE TABLE employees (
  employee_id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  department_id INT,
  salary DECIMAL(10, 2),
  FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

INSERT INTO departments (department_name) VALUES
  ('HR'),
  ('Finance'),
  ('IT');

INSERT INTO employees (first_name, last_name, department_id, salary) VALUES
  ('John', 'Doe', 1, 50000.00),
  ('Jane', 'Smith', 2, 60000.00),
  ('Michael', 'Johnson', 3, 75000.00),
```

```
  ('Emily', 'Davis', 2, 62000.00),
  ('William', 'Brown', 1, 52000.00);
```

Теперь давайте рассмотрим сами запросы:

## UNION:

- `UNION` используется для объединения результатов двух или более запросов в один набор результатов. **Он автоматически удаляет дубликаты строк.**
- Каждый `SELECT` запрос в `UNION` должен иметь одинаковое количество столбцов.
- Столбцы должны иметь совместимые типы данных.
- Столбцы в каждом `SELECT` запросе должны идти в одном и том же порядке.

**Пример** (получение списка всех сотрудников и департаментов в одном столбце):

```
SELECT first_name, last_name, department_id FROM employees
UNION
SELECT department_name AS first_name, NULL AS last_name, department_id FROM
departments;
```

*Этот запрос объединяет список сотрудников из таблицы `employees` с названиями департаментов из таблицы `departments`. Поскольку во втором запросе нет фамилии, мы искусственно подставляем `NULL`, чтобы количество столбцов совпало.*

## UNION ALL:

- `UNION ALL` также используется для объединения результатов двух или более запросов, но в отличие от обычного `UNION`, **он сохраняет абсолютно все строки**, включая дубликаты. Это работает быстрее, так как базе данных не нужно тратить ресурсы на поиск и удаление повторяющихся строк.

**Пример** (получение списка всех сотрудников и департаментов):

```
SELECT first_name, last_name, department_id FROM employees
UNION ALL
SELECT department_name AS first_name, NULL AS last_name, department_id FROM
departments;
```

*В этом запросе используется `UNION ALL`, поэтому если бы в результатах были одинаковые строки, они бы сохранились.*

## HAVING:

- `HAVING` используется для фильтрации результатов агрегированных функций (например, `COUNT`, `SUM`, `AVG`) при использовании группировки `GROUP BY`. Обычный `WHERE` не умеет работать с агрегатными функциями.

**Пример** (получение департаментов со средней зарплатой выше 60000):

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING avg_salary > 60000;
```

*Этот запрос сначала группирует (агрегирует) данные по департаментам, рассчитывает среднюю зарплату, а затем фильтрует результаты, оставляя только те департаменты, у которых средняя зарплата превышает 60000.*

**Порядок выполнения запросов в SQL:** `WHERE` (фильтрация строк) -> `GROUP BY` (группировка) -> `HAVING` (фильтрация групп) -> `ORDER BY` (сортировка результата).

### Полезные материалы

1. Презентация
2. Старая версия https://youtu.be/4uyxTMh0g4E
3. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1YrjX3TEpfNUzmNQsZhgHnX-Jk5lBS9uK
4. Практика по SQL https://www.w3schools.com/sql/
5. https://www.sql-ex.ru/
6. https://stepik.org/course/63054/promo
7. https://sqlbolt.com/

РАБОТА С ДАННЫМИ

# 55. JOIN запросы в SQL

### Работа с JOIN

Давайте создадим две таблицы: `employees` и `departments`.

Таблица `employees` содержит данные о сотрудниках, а таблица `departments` – о департаментах, к которым они относятся.

Сначала выполните запросы `CREATE TABLE` по отдельности, а затем запросы `INSERT INTO`.

Обратите внимание, что создается внешний ключ (`FOREIGN KEY`) для объединения этих таблиц.

```
CREATE TABLE departments (
  department_id INT AUTO_INCREMENT PRIMARY KEY,
  department_name VARCHAR(50) NOT NULL
);

CREATE TABLE employees (
  employee_id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  department_id INT,
  salary DECIMAL(10, 2),
  FOREIGN KEY (department_id) REFERENCES departments(department_id)
);

INSERT INTO departments (department_name) VALUES
  ('HR'),
  ('Finance'),
  ('IT');

INSERT INTO employees (first_name, last_name, department_id, salary) VALUES
  ('John', 'Doe', 1, 50000.00),
  ('Jane', 'Smith', 2, 60000.00),
  ('Michael', 'Johnson', 3, 75000.00),
```

```
  ('Emily', 'Davis', 2, 62000.00),
  ('William', 'Brown', 1, 52000.00);
```

Теперь давайте рассмотрим JOIN-запросы:

## INNER JOIN:

`INNER JOIN` используется для объединения строк из двух таблиц на основе совпадающих значений в указанных столбцах.

**Пример:**

```
SELECT employees.first_name, employees.last_name, departments.department_name
FROM employees
INNER JOIN departments ON employees.department_id =
departments.department_id;
```

Этот запрос объединяет данные из таблиц `employees` и `departments` по значению столбца `department_id`. В результате получается список имен сотрудников и названий департаментов, к которым они относятся.

## LEFT JOIN:

`LEFT JOIN` используется для объединения всех строк из левой таблицы (`employees`) со всеми совпадающими строками из правой таблицы (`departments`).

**Пример:**

```
SELECT employees.first_name, employees.last_name, departments.department_name
FROM employees
LEFT JOIN departments ON employees.department_id = departments.department_id;
```

Этот запрос вернет список всех сотрудников и их департаментов. Если вдруг сотрудник не привязан ни к какому департаменту (в столбце `department_id` у него `NULL`), его данные все равно будут включены в результат.

## RIGHT JOIN:

`RIGHT JOIN` используется для объединения всех строк из правой таблицы (`departments`) со всеми совпадающими строками из левой таблицы (`employees`).

**Пример:**

```
SELECT employees.first_name, employees.last_name, departments.department_name
FROM employees
RIGHT JOIN departments ON employees.department_id =
departments.department_id;
```

Этот запрос вернет список всех департаментов и их сотрудников. Если департамент не имеет ни одного сотрудника, его данные также будут включены в результат.

## Вложенные подзапросы

Вложенные подзапросы позволяют выполнять запросы внутри других запросов.

**Пример** (получение средней зарплаты в департаменте "HR"):

```
SELECT department_name,
  (SELECT AVG(salary) FROM employees WHERE department_id =
departments.department_id) AS avg_salary
FROM departments
WHERE department_name = 'HR';
```

В этом запросе внутренний подзапрос `(SELECT AVG(salary)...)` вычисляет среднюю зарплату для каждого департамента, а внешний запрос выбирает только департамент "HR" и выводит эту рассчитанную среднюю зарплату.

## Псевдонимы таблиц (Aliases)

В MySQL псевдонимы таблиц (aliases) в запросах с использованием `JOIN` позволяют сократить запись и упростить ссылки на таблицы и их поля. Это особенно полезно в запросах с множественными соединениями (`JOIN`) и длинными именами таблиц, или когда одна и та же таблица используется несколько раз в разных соединениях.

### Пример использования псевдонимов таблиц

Предположим, мы хотим соединить наши таблицы по полю `department_id`, чтобы получить информацию о сотрудниках и их отделах. Вот как мы можем использовать псевдонимы для этих таблиц:

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id;
```

В этом примере `employees` получил псевдоним `e`, а `departments` — псевдоним `d`. Это позволяет нам коротко обращаться к полям этих таблиц: `e.first_name` вместо длинного `employees.first_name`.

### Более сложный пример с множественным JOIN

Предположим, у нас есть еще две таблицы: `projects` (проекты) и `project_assignments` (назначения на проекты). Мы хотим получить информацию о сотрудниках, их отделах и проектах, над которыми они работают. Мы можем расширить предыдущий запрос следующим образом:

```
SELECT e.first_name, e.last_name, d.department_name, p.project_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id
JOIN project_assignments pa ON e.employee_id = pa.employee_id
JOIN projects p ON pa.project_id = p.project_id;
```

Здесь `project_assignments` – это промежуточная таблица, которая связывает сотрудников с проектами, и мы использовали для неё псевдоним `pa`. Таблица `projects` получила псевдоним `p`.

Использование псевдонимов делает SQL-запросы более читабельными и позволяет избежать путаницы, особенно когда одни и те же таблицы используются несколько раз в различных соединениях в одном запросе.

### Полезные материалы

1. Презентация
2. Старая версия урока https://youtu.be/e-kPUCAkZyU
3. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1Xe3YpwwdsWwYpmmOqkBlqx-C-4g2PycK
4. Практика по SQL https://www.w3schools.com/sql/
5. https://www.sql-ex.ru/
6. https://stepik.org/course/63054/promo
7. https://sqlbolt.com/

РАБОТА С ДАННЫМИ

# 56. NoSQL база данных MongoDB (версия 2026)

### Установка и настройка NoSQL базы данных на примере MongoDB

В предыдущих уроках мы работали с реляционной базой данных (MySQL), где данные хранятся в строгих таблицах. Однако современные веб-приложения всё чаще используют **NoSQL базы данных**, и самая популярная из них – **MongoDB**.

Здесь нет таблиц, строк и колонок. Данные хранятся в виде **Документов** (очень похожих на JSON-файлы), которые лежат в **Коллекциях**.

**Зачем это тестировщику:**

- **Проверка неструктурированных данных:** Если форма на сайте позволяет пользователю добавлять произвольные поля (например, разные характеристики для разных категорий товаров), разработчики часто сохраняют это в MongoDB. QA должен уметь найти этот JSON-документ и проверить, корректно ли записались вложенные объекты и массивы.
- **Тестирование микросервисов:** В современной архитектуре один микросервис может использовать PostgreSQL, а другой (например, сервис логов или корзин пользователей) – MongoDB. Тестировщику нужно уметь работать с обоими типами хранилищ.

Самый простой и современный способ начать работать с MongoDB: создать бесплатный облачный кластер (сервер) в **MongoDB Atlas**, а для графического управления установить на компьютер программу **MongoDB Compass** (это аналог DBeaver, но специально для Mongo). В DBeaver также можно подключаться к MongoDB, но для этого нужна платная версия этого инструмента.

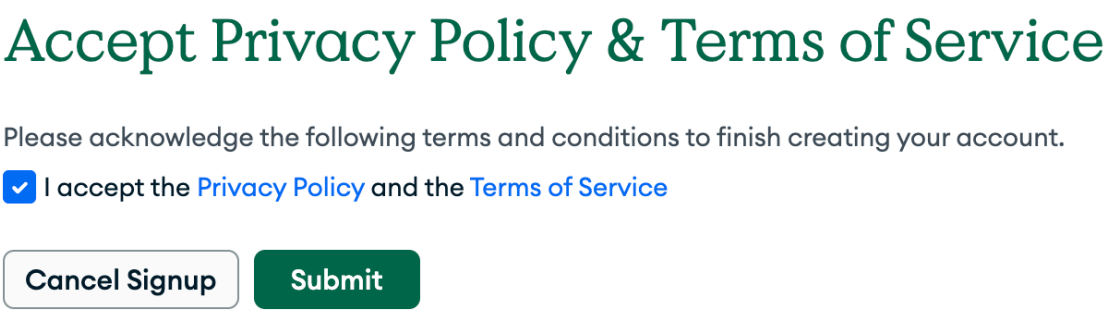
### 1. Создание облачного кластера в MongoDB Atlas

Вам не нужно устанавливать сам сервер базы данных на свой компьютер, мы арендуем бесплатный сервер у самой компании MongoDB.

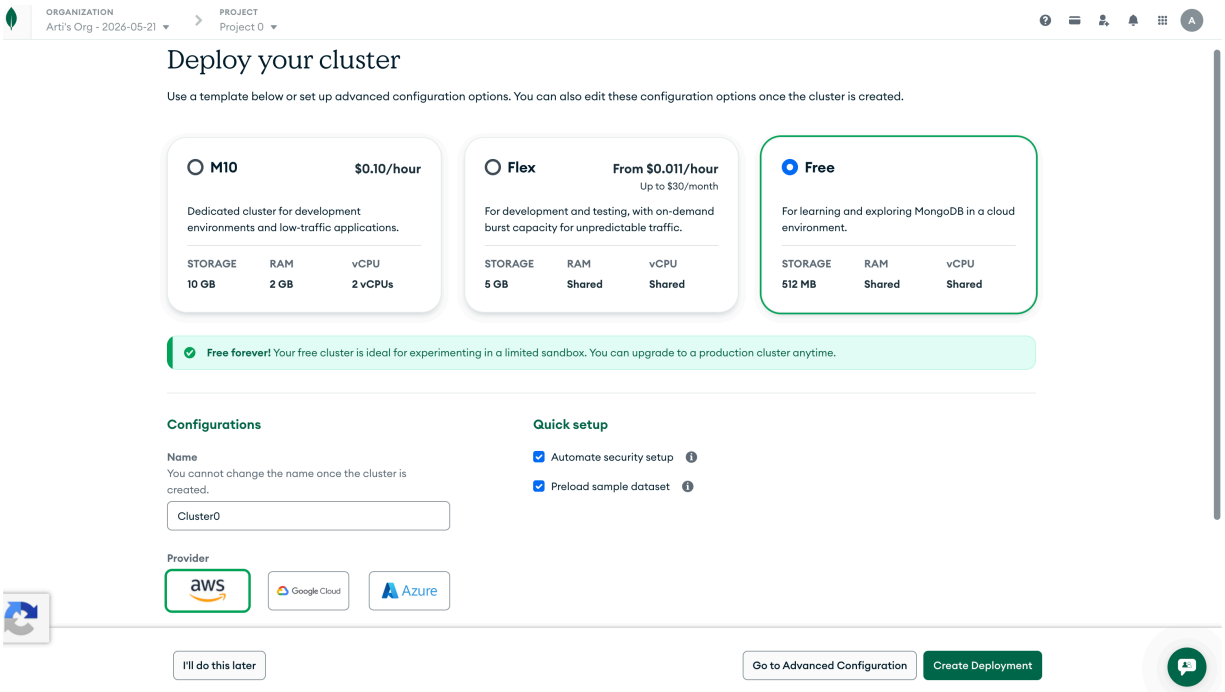
1. Перейдите на сайт **mongodb.com/atlas** и нажмите

.

2. Пройдите регистрацию (удобнее всего авторизоваться через Google-аккаунт).
3. Примите условия использования и нажмите **Submit**.



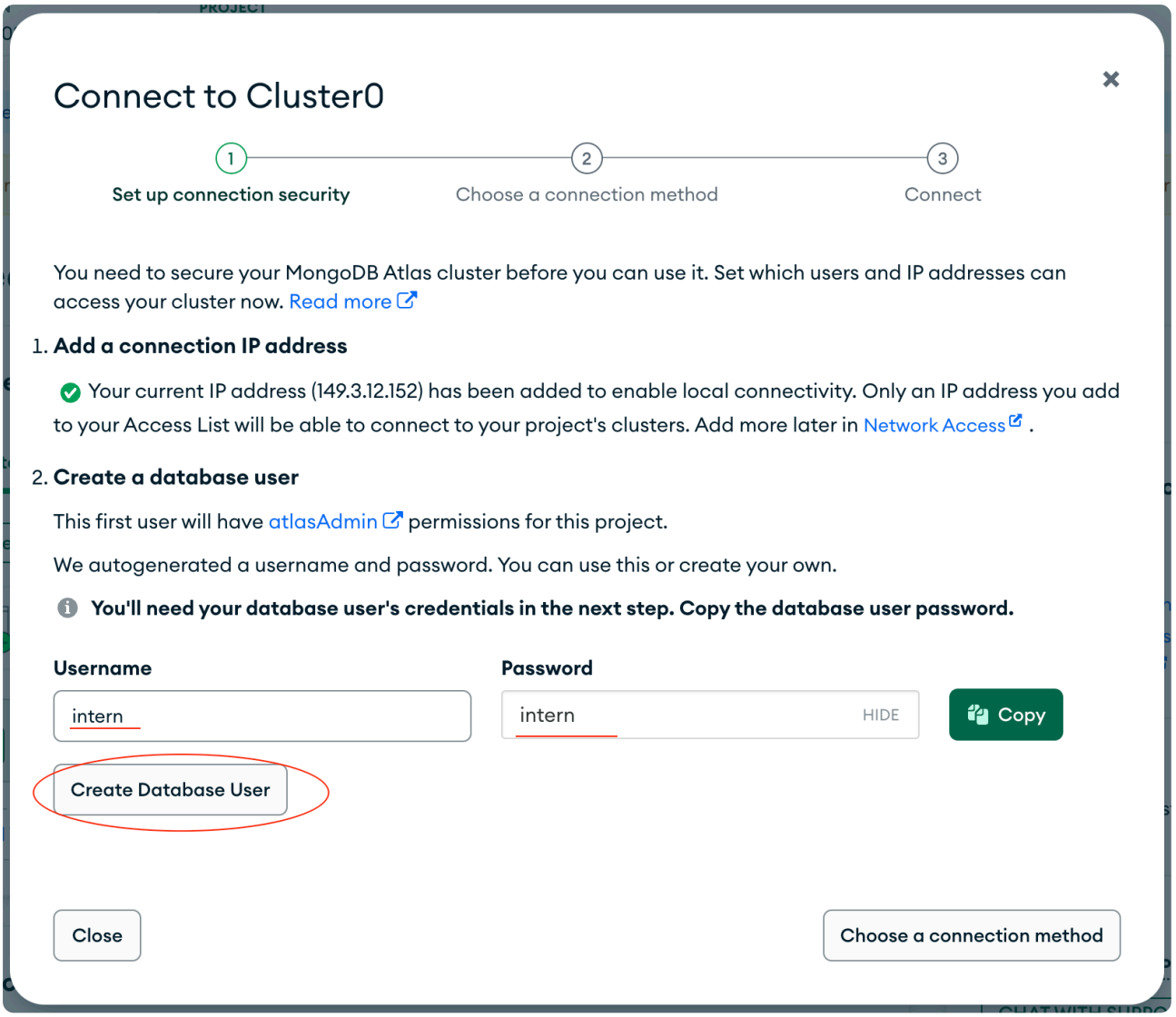

4. После регистрации система предложит вам ответить на пару вопросов о ваших целях (можно выбрать *Skip Personalization*).
5. Вы попадете на экран выбора тарифа (*Deploy your claster*).
    - Обязательно выберите **Free** – это бесплатный кластер навсегда, его мощностей с головой хватит для обучения и тестирования.
    - Выберите провайдера (например, AWS) и ближайший регион (например, Frankfurt). Обычно они предложены автоматом.
    - Нажмите кнопку **Create Deployment (Создать)**.




6. Не закрывайте окно, которое появится после начала создания кластера, оно понадобится нам дальше. Ваш кластер создается на фоне.

## 2. Настройка доступов

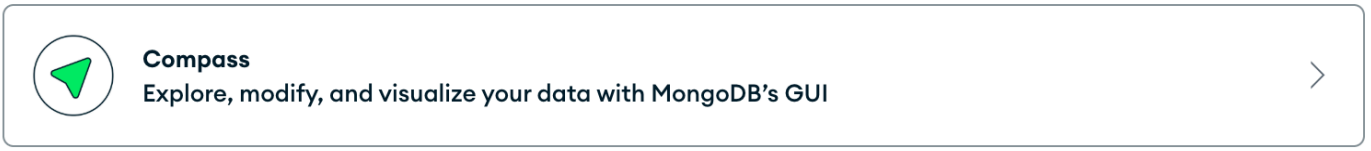
1. База создана, но по умолчанию она закрыта от всего интернета. Нам нужно создать пользователя и разрешить нашему компьютеру к ней подключаться. В окне, которое осталось после манипуляций выше нужно добавить Username и Password вашего пользователя. Затем нажать на **Create Database User**.



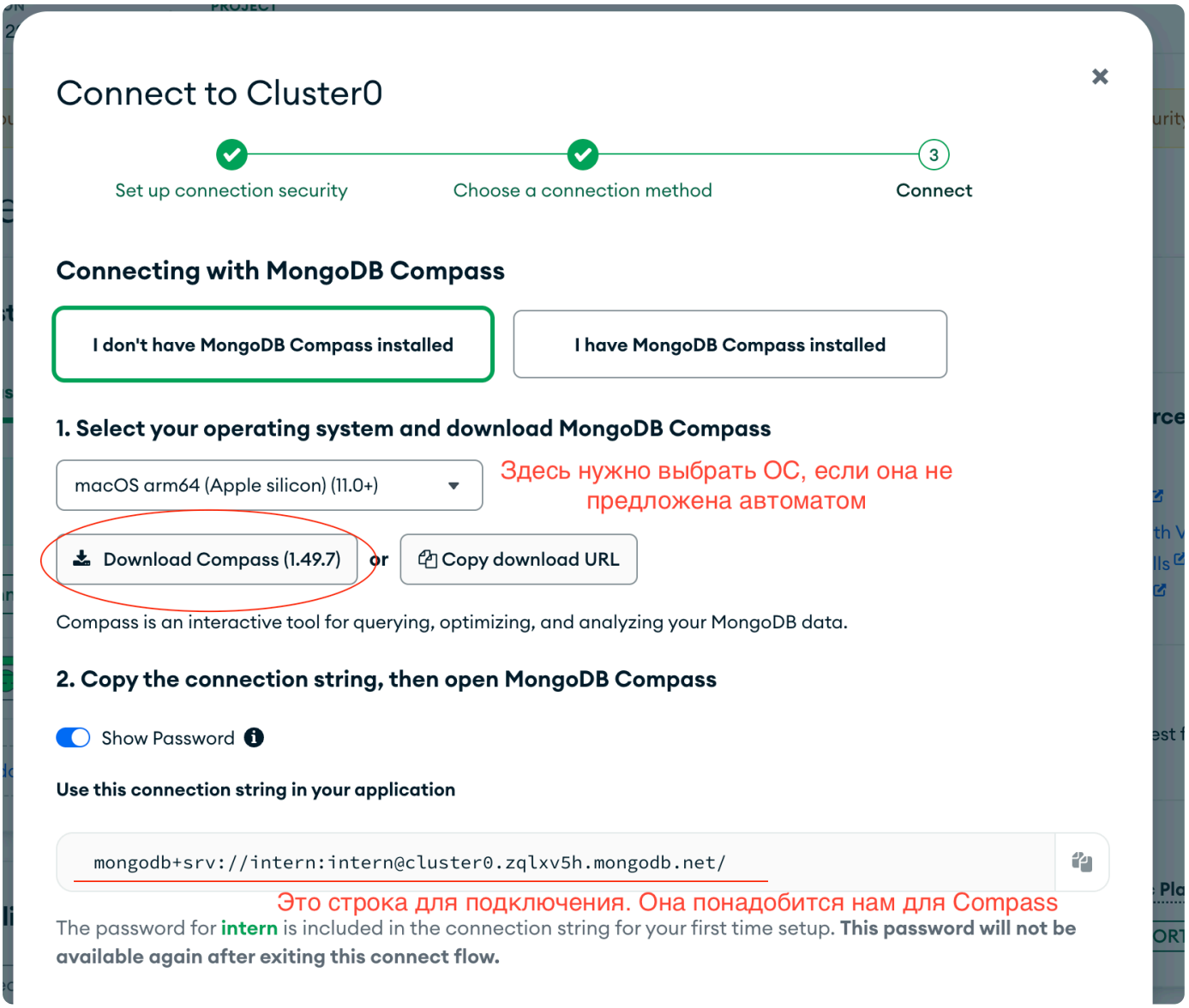

2. Нажмите

.

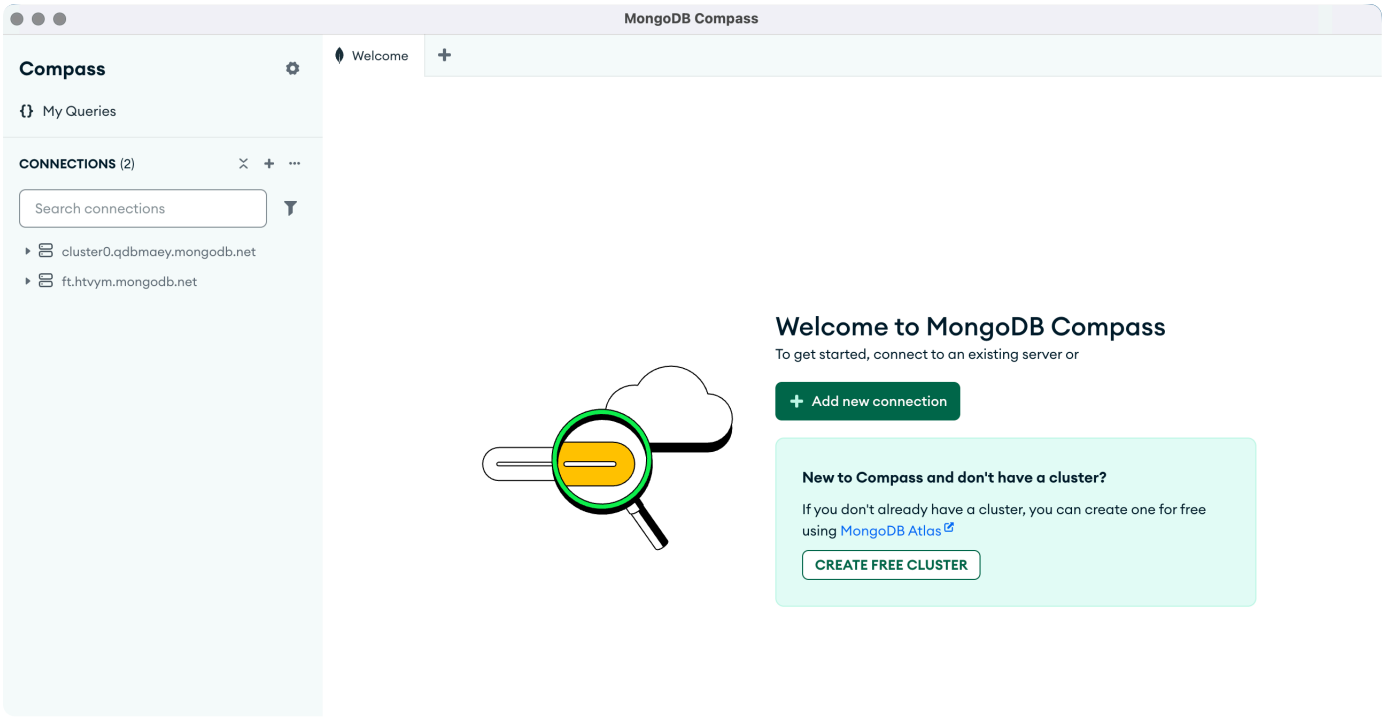
3. Мы будем подключаться к базе через MongoDB Compass, поэтому выберите опцию ниже:

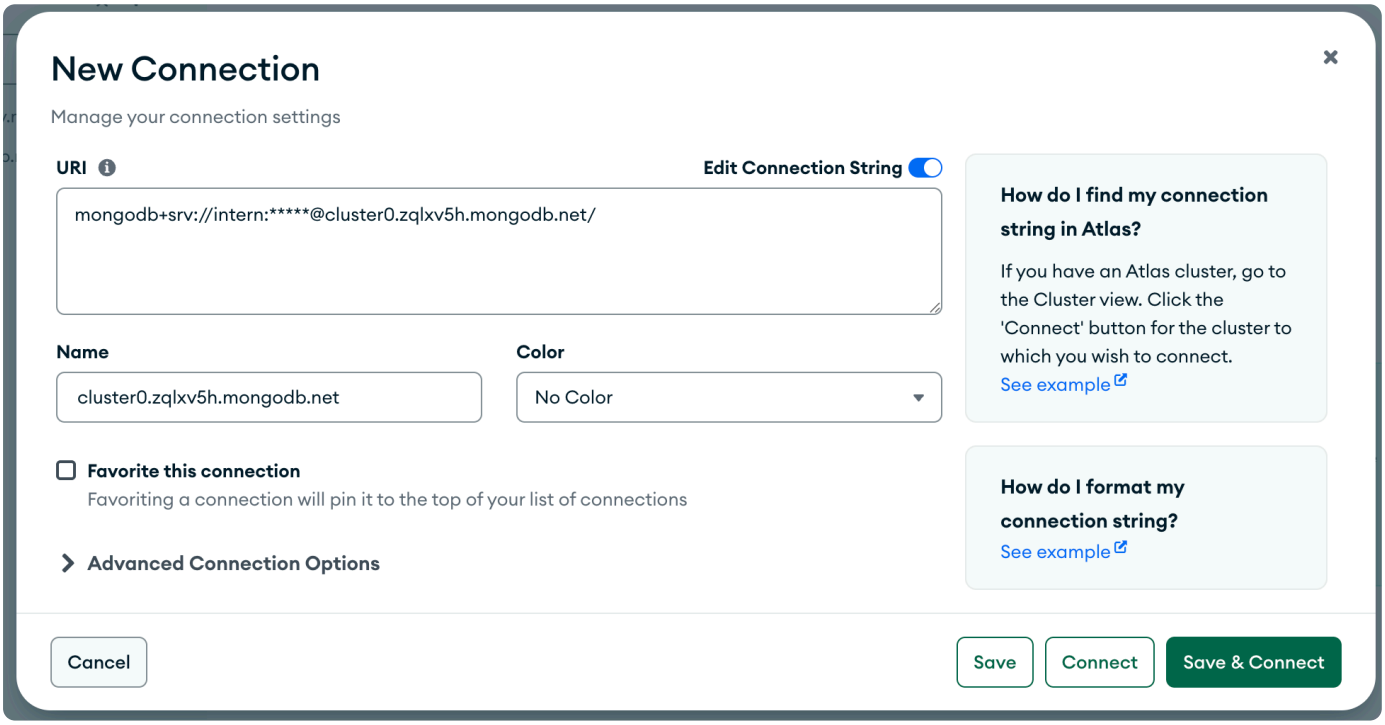



4. Далее вам предложат установить Compass на свой компьютер. Нажмите **Download Compass** и пройдите стандартную установку.

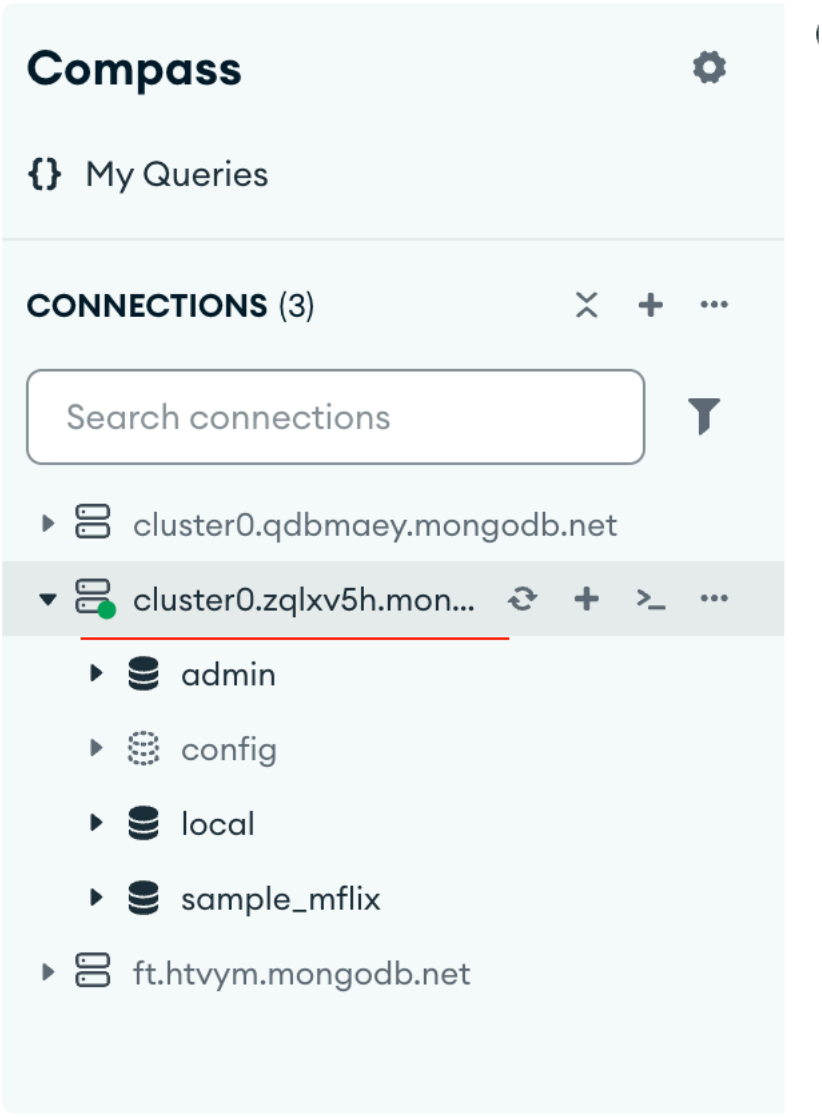



5. Не закрывайте окно из пункта 4. Дождитесь конца установки, скопируйте **Connection string (строка для подключения)** и подключитесь к кластеру, нажав **Add new connection**.




6. Нажмите Save & Connect. Поздравляю вы подключитесь к кластеру и можете создавать свои коллекции!

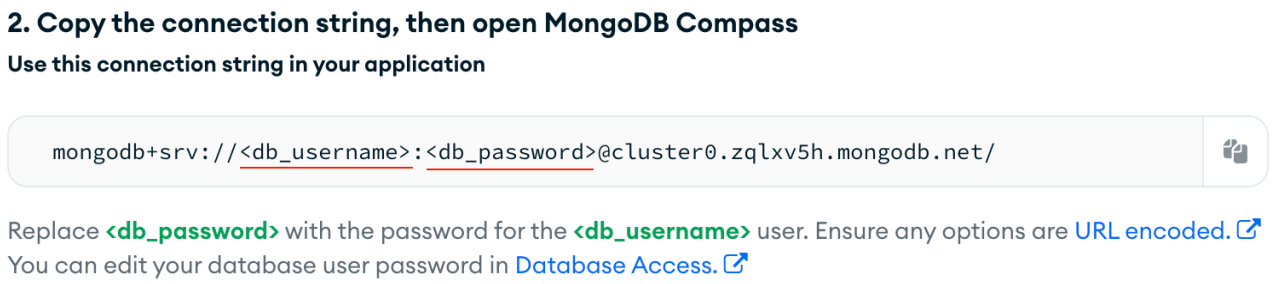



**Если по какой-то причине вы закрыли окно из пункта 6 в шаге "Создание облачного кластера в MongoDB Atlas", то вы можете создать пользователя через интерфейс MongoDB Atlas.**

1. В левом меню (раздел Security) выберите **Database & Network Access**.
2. Нажмите зеленую кнопку **Add New Database User**.
3. Выберите метод *Password*.
4. Придумайте **Username** (например, `qa_tester`) и **Password** (например, `Test1234!`). *Обязательно запишите этот пароль, он вам понадобится.*
5. В разделе *Database User Privileges > Built-in Role* выберите **Read and write to any database** (Чтение и запись).
6. Нажмите **Add User**.

Получить строку подключения в таком случаем можно таким способом:

1. В левом меню выберите **Database > Clusters** (вы вернетесь к вашему кластеру).
2. Нажмите кнопку **Connect** (Подключиться).
3. В появившемся окне выберите вариант **Compass** (GUI).
4. Система покажет вам ссылку подключения (Connection String). В таком варианте вам нужно будет заменить username и пароль в строке на данные вашего пользователя.




5. Скопируйте эту ссылку и используйте ее для подключения в MongiDB Compass (см. выше).

### Настройка сетевого доступа (IP Whitelist)

1. В левом меню (раздел Security) выберите **Database & Network Access**
2. Выберите **Network Access > IP Access List**.
3. Нажмите зеленую кнопку **Add IP Address**.
4. Для обучающих целей выберите **Allow Access From Anywhere** (Разрешить доступ отовсюду). Система подставит IP-адрес `0.0.0.0/0`.
5. Нажмите **Confirm**. Статус изменится на *Active*.
6. Если вы не сделаете это, то подключение к этому кластеру будет возможно только с вашего подключения к сети.

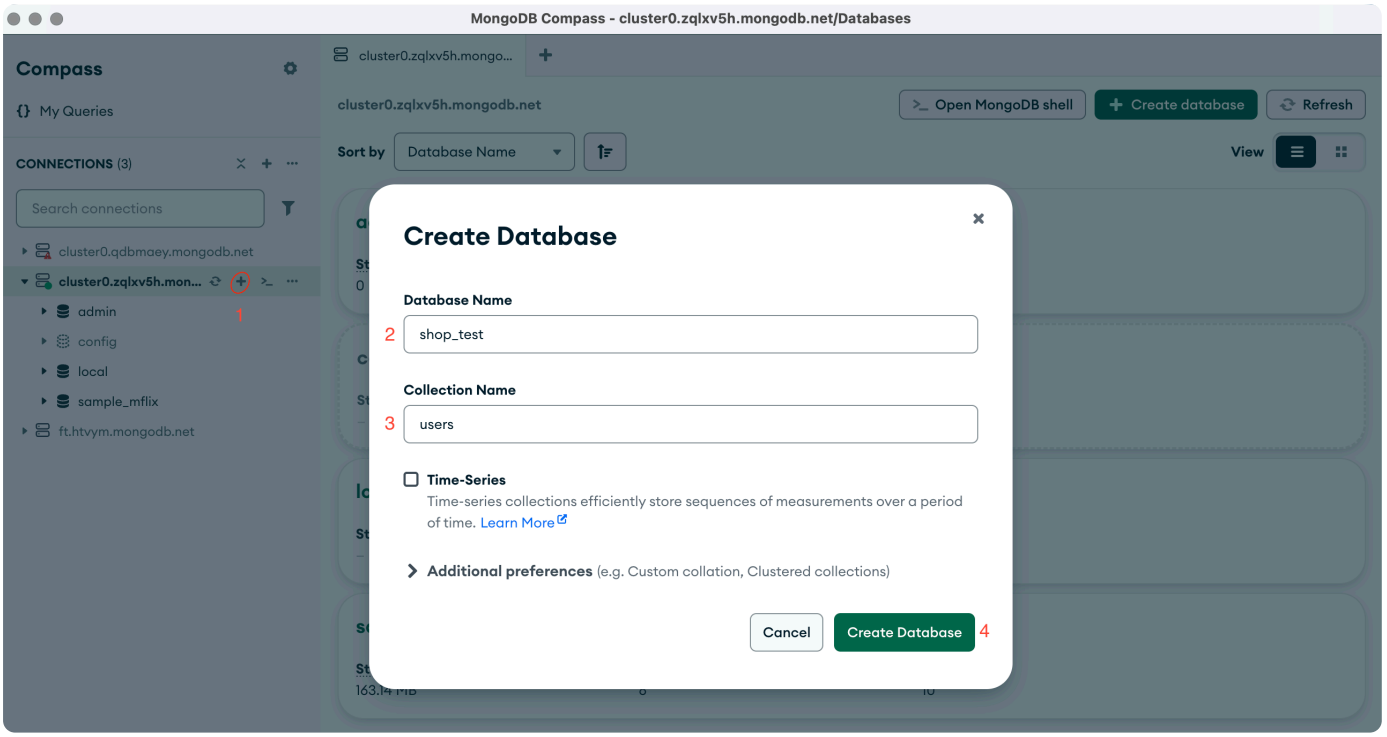
> *Примечание для QA:* На реальных проектах открывать базу всему интернету строго запрещено. Вы будете нажимать *Add Current IP Address*, чтобы разрешить доступ только вашему рабочему компьютеру (или рабочему VPN).

## Работа в Compass: Создание БД и базовые запросы

Давайте создадим базу, добавим в нее документ и попробуем его найти (то, что тестировщик делает чаще всего).

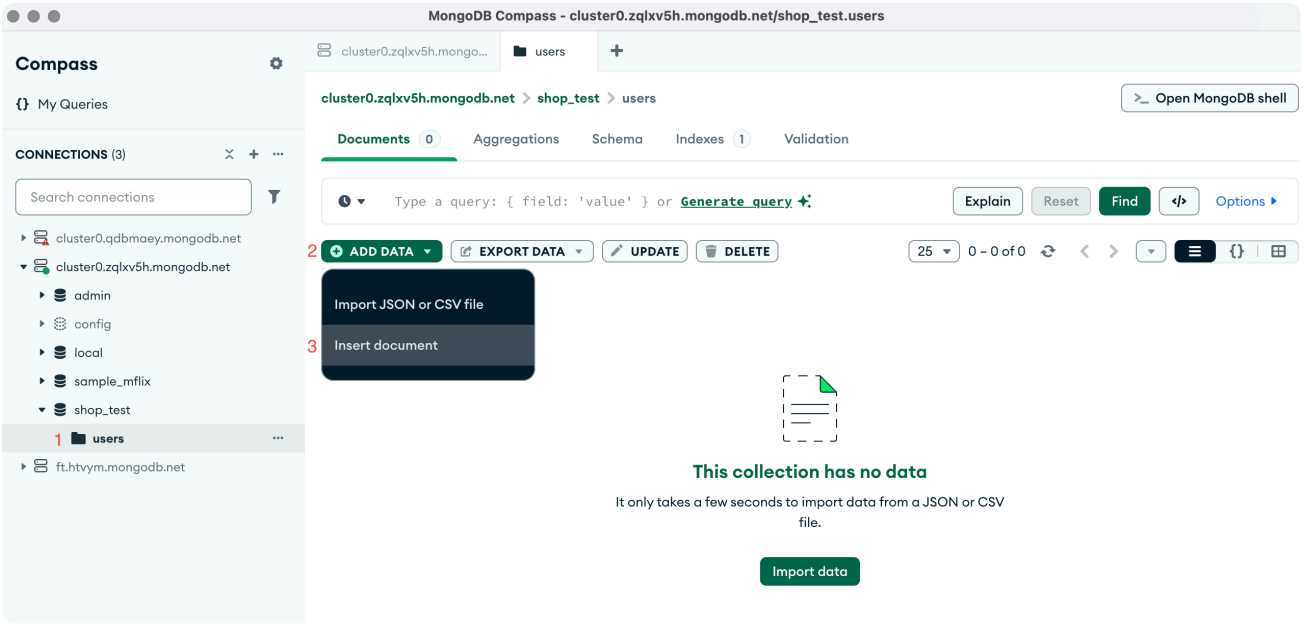
### Шаг 1: Создание базы и коллекции

1. В левом меню нажмите на иконку плюса **+** (Create database).
2. В поле *Database Name* введите `shop_test`.
3. В поле *Collection Name* (аналог названия таблицы) введите `users`.
4. Нажмите **Create Database**.




### Шаг 2: Добавление данных (Insert Document)

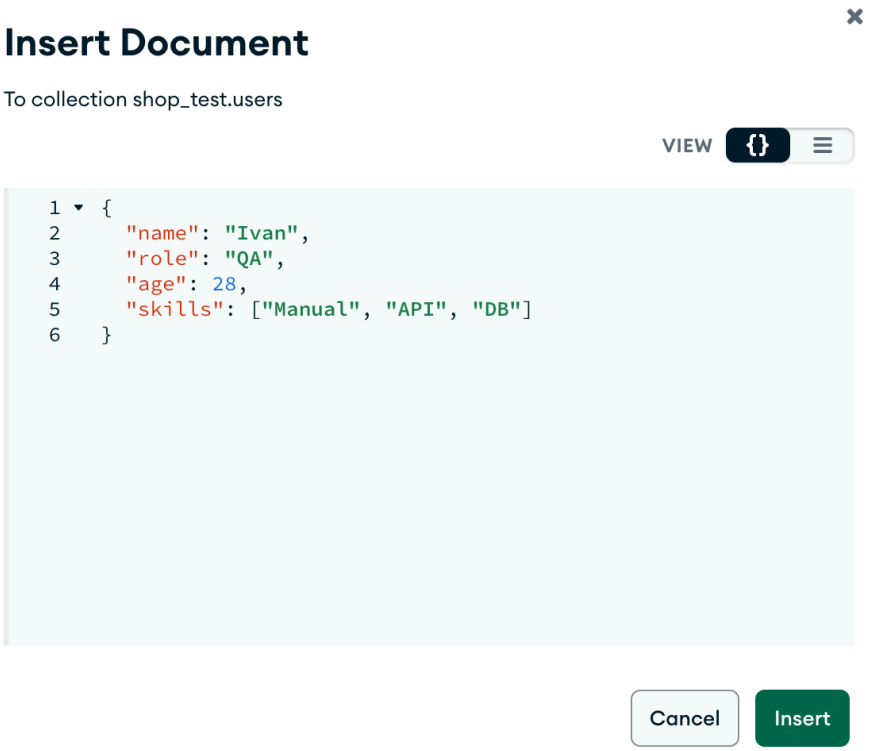
1. Кликните на вашу новую коллекцию `users` в левом меню.
2. В правой части окна нажмите зеленую кнопку **Add Data** -> **Insert document**.

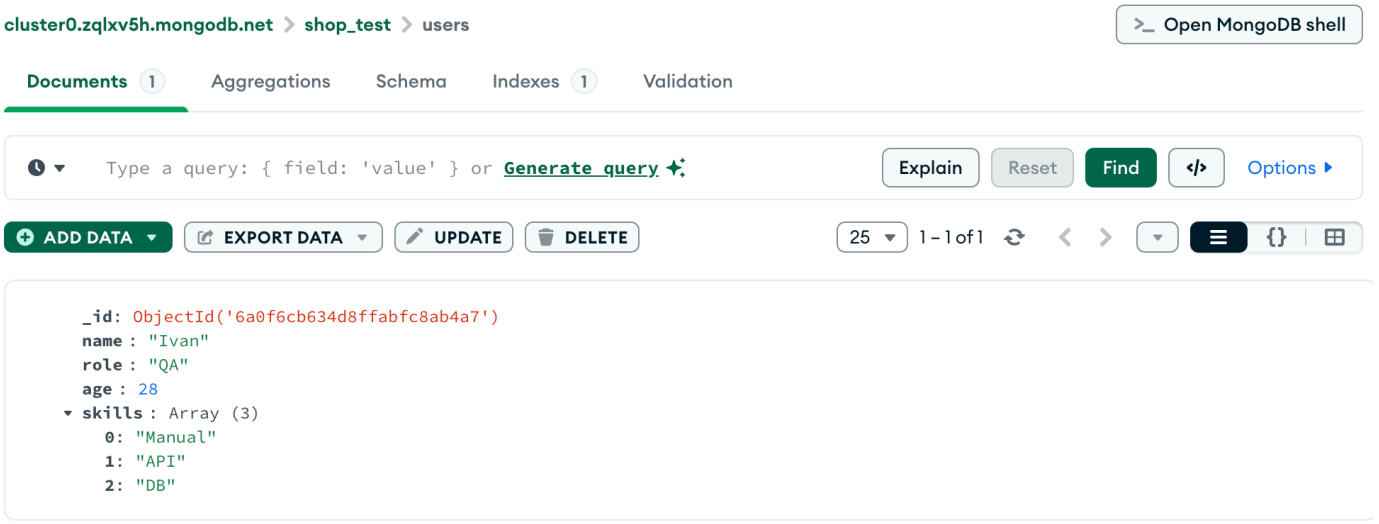



3. Compass покажет вам структуру в формате JSON. Замените ее на тестовые данные:

```
{
  "name": "Ivan",
  "role": "QA",
  "age": 28,
  "skills": ["Manual", "API", "DB"]
}
```

4. Нажмите **Insert**. Вы увидите, что документ появился в списке, а система автоматически добавила ему уникальный идентификатор `_id`.



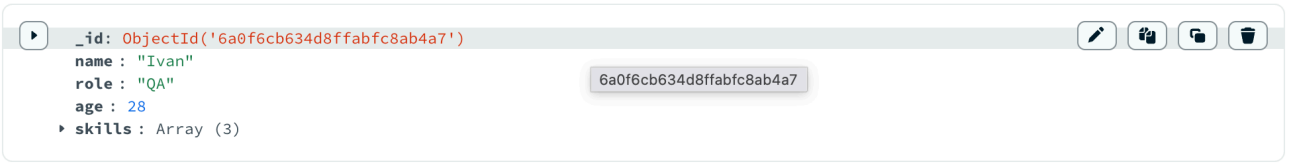

**Шаг 3: Поиск и фильтрация данных (Вместо SQL).** В MongoDB нет языка SQL (команд SELECT, WHERE). Фильтрация здесь происходит путем передачи JSON-объекта с нужными параметрами.

В верхней части экрана есть строка поиска **Type a query...** В нее нужно вводить сами запросы.

- **Сценарий 1 (Точный поиск):** Вам нужно найти пользователя с именем Ivan. Введите в строку Filter: `{ "name": "Ivan" }` Нажмите зеленую кнопку *Find*. Compass оставит на экране только этот документ.
- **Сценарий 2 (Поиск по числу):** Найти всех, кому 28 лет: `{ "age": 28 }`
- **Сценарий 3 (Сложный оператор):** Найти всех пользователей старше 20 лет. Для этого используются операторы с символом `$`. Больше (Greater Than) – это `$gt`. Вводим в Filter: `{ "age": { "$gt": 20 } }`

**Шаг 4: Изменение данных "на лету".** Как и в DBeaver, вам не обязательно писать код для редактирования.

1. Наведите курсор на созданный документ. В правом верхнем углу документа появятся иконки.




2. Нажмите на иконку **карандаша (Edit Document)**.
3. Измените значение (например, возраст с 28 на 30) или добавьте новое поле прямо в интерфейсе.
4. Нажмите **Update (Обновить)**. Данные мгновенно улетят в вашу облачную базу Atlas.

### Работа с командной строкой в Compass

Также в Compass можно вызывать командную строку и отправлять запросы внутри нее. В отличие от поиска по query из прошлого шага, синтаксис будет другой.

Для открытия командной строки нажмите

.

Вот некоторые примеры команд:

**1. Создание базы данных и коллекции:**

- Используйте команду use staff; для создания базы данных с названием "staff".
- Используйте команду db.createCollection("users"); для создания коллекции с названием "users" в базе данных "staff".

**2. Создание документов:**

- Для создания одного документа используйте команду db.users.insertOne({...}); и предоставьте данные для нового документа.
- Для создания нескольких документов используйте команду db.users.insertMany([...]); и предоставьте данные для нескольких документов в формате массива.

**3. Поиск информации:**

- Для поиска всех документов в коллекции используйте команду db.users.find();.
- Для поиска одного документа используйте команду db.users.findOne();.
- Для поиска документов по определенному критерию используйте команду db.users.find({criteria});.

**4. Обновление документов:**

- Для обновления одного документа используйте команду db.users.updateOne({filter}, {update});.
- Для обновления нескольких документов используйте команду db.users.updateMany({filter}, {update});.

**5. Удаление документов:**

- Для удаления одного документа используйте команду db.users.deleteOne({filter});.
- Для удаления нескольких документов используйте команду db.users.deleteMany({filter});.

**6. Работа с операторами:**

- Используйте различные операторы (например, $eq, $gte, $in) для выполнения более сложных запросов к данным в коллекции.

**Дополнительные ресурсы:**

- Для изучения более продвинутых операторов обновления в MongoDB вы можете обратиться к документации.

**Полезные материалы**

1. Презентация
2. Регистрация в MongoDB https://www.mongodb.com/cloud/atlas/register
3. Скачать MongoDB Compass https://www.mongodb.com/products/compass
4. Документация MongoDB https://www.mongodb.com/docs/
5. Практикум и образцы запросов в MongoDB https://www.w3schools.com/mongodb/

# 57. Вступительное слово перед модулем "Тестирование мобильных Apps"

**Вступительное слово перед стартом модуля "Тестирование мобильных приложений"**

Современный мир не может существовать без портативных гаджетов: телефоны, планшеты, умные часы, интернет-вещей - все это необходимо тестировать.

В частности, мы занимаемся тестированием мобильных приложений, которые могут быть установлены на перечисленные выше девайсы.

**В данном модуле мы разберем:**

1. Основные типы мобильных приложений и особенности их тестирования
2. Принципы отбора девайсов для тестирования, а также, что делать если их нет, на примере эмулятора Android Studio
3. Основы работы с симулятором xCode для iOS
4. Работу со снифферами, которые позволяют перехватывать запросы и ответы с мобильных приложений, анализировать и видоизменять их

Дополнительно советую изучить вопросы, связанные с основными отличиями Android и iOS, а также гайдлайны для каждой платформы (Human Interface Guidelines у iOS и Material Design у Android). Все необходимые ссылки вы найдете дальше по модулю.

**Дисклеймер:** моя основная специализация тестирование веб-приложений, об этом знают все мои подписчики и теперь уже вы, студенты. В данном модуле содержится базовая информация по тестированию мобильных приложений, за углубленным изучением советую обратиться к другим авторам.

# 58. Обзор Android и iOS

### Основные отличия Android и iOS (в презентации есть ошибка)

| **Характеристика** | **iOS** | **Android** |
|---|---|---|
| *Компания* | Apple | Google |
| *Ядро* | Облегченная версия MacOS | Linux |
| *Система* | Закрытая | Открытая |
| *Язык разработки* | objective-C / Swiftobjective-C / Swift | Java/Kotlin |
| *Магазин* | App Store | Google Play |
| *Расширение приложений* | ipa | apk |
| *Установка без магазина* | Нет | Да |

Дополнительно можно прочитать здесь

Чтобы уверенно тестировать мобильные приложения, важно хотя бы на базовом уровне понимать **как устроены iOS и Android**. На собеседованиях для новичков такие подробности спрашивают редко.

## 🧱 Общая архитектура Android

Android — это **операционная система от Google**, построенная на базе ядра Linux. Вся система делится на 4 слоя:

1. **Linux Kernel**

   Управляет процессами, памятью, сетью, драйверами и безопасностью.

2. **Hardware Abstraction Layer (HAL)**

   Прослойка между железом и системой. Даёт доступ к камере, GPS, микрофону и т.д.

3. **Android Runtime (ART)**

Виртуальная машина, где исполняется код приложений. Каждый апп запускается в **своём процессе**.

4. **Application Framework**

   То, с чем работают разработчики: активности, фрагменты, контент-провайдеры, сервисы и т.д.

5. **Apps**

   Собственно, пользовательские приложения — они работают поверх всех слоёв.

## ⚙️ Архитектура приложения Android

Android-приложение состоит из **компонентов**:

- **Activity** — один экран UI. Например, LoginActivity, ProfileActivity.
- **Fragment** — часть экрана, может быть внутри активности.
- **Service** — компонент для фоновой работы (например, загрузка).
- **Broadcast Receiver** — ловит системные события (например, зарядка началась).
- **Content Provider** — для обмена данными между приложениями.

Всё это описывается в AndroidManifest.xml.

**Жизненный цикл Activity** (важно для тестирования):

- onCreate() → создаётся
- onStart() → отображается
- onResume() → активна
- onPause() → свернули/вызвали диалог
- onStop() → ушли с экрана
- onDestroy() → уничтожена

Ошибки часто связаны с неправильной обработкой этих состояний: например, краш при повороте экрана — потому что состояние не сохранилось.

## 🍏 Архитектура iOS

iOS — это закрытая мобильная ОС от Apple, построенная на базе **UNIX**. Архитектура тоже состоит из уровней:

1. **Core OS**

   Работа с низкоуровневыми задачами: безопасность, драйверы, память.

2. **Core Services**

   Работа с базами данных, настройками, файлами, сетью.

3. **Media Layer**

   Аудио, видео, графика — всё, что связано с отображением.

4. **Cocoa Touch (UIKit)**

   Верхний уровень, отвечающий за интерфейсы и взаимодействие с пользователем.

## 🧩 Архитектура приложений iOS

iOS-приложение состоит из **контроллеров и экранов**:

- **UIViewController** — аналог Activity в Android. Отвечает за экран или его часть.
- **UIWindow** — корневой контейнер всего UI.
- **AppDelegate** — точка входа в приложение, управляет жизненным циклом.
- **SceneDelegate** — добавлен в новых версиях для работы с несколькими окнами.

**Жизненный цикл UIViewController**:

- viewDidLoad() → загружается
- viewWillAppear() → собирается показаться

- viewDidAppear() → уже на экране
- viewWillDisappear() → собирается скрыться
- viewDidDisappear() → исчез

На этих этапах тоже часто возникают баги — особенно если интерфейс зависит от сети, а она “пропала в момент загрузки”.

**Полезные материалы**

1. Презентация по платформам
2. Архитектура Android (официальная документация) https://developer.android.com/guide/platform
3. Архитектура Android (перевод)https://bimlibik.github.io/posts/how-does-android-work
4. Архитектура Android Application https://vladislaveremeev.gitbook.io/qa_bible/mobilnoe-testirovanie/arkhitektura-android-application
5. Архитектура iOS https://intellipaat.com/blog/tutorial/ios-tutorial/ios-architecture/
6. Архитектура iOS Application https://vladislaveremeev.gitbook.io/qa_bible/mobilnoe-testirovanie/arkhitektura-ios-application
7. https://habr.com/ru/companies/croc/articles/549590/
8. Основные различия Android/iOS https://vladislaveremeev.gitbook.io/qa_bible/mobilnoe-testirovanie/osnovnye-razlichiya-android-ios
9. Информация о возможностях версий Android https://developer.android.com/about/versions
10. Информация о возможностях версий iOS https://developer.apple.com/documentation/ios-ipados-release-notes

# 59. Тестовые фермы

### Устройства для тестирования

1. Реальные устройства гарантируют, что ваше приложение будет работать без проблем на клиентских телефонах. Можно использовать реальную и облачную ферму устройств
2. Эмуляторы воспроизводят все основные компоненты устройства, в том числе процессор, память и устройства ввода/вывода + поведение системы и ее интерфейса (Android Studio)
3. Симуляторы воспроизводят поведение системы и ее интерфейса (xCode)

### Факторы влияющие на выбор

1. Требования и особенности приложения
2. Статистика
3. Изучение целевой аудитории
4. Наличие устройств в тестовой ферме
5. Использование эмуляторов и симуляторов

### Полезные материалы

1. Презентация по тестовой ферме
2. Дэшборд- статистика Android https://developer.android.com/about/dashboards
3. Дэшборд-статистика iOS https://developer.apple.com/support/app-store
4. Android vs iOS https://mixpanel.com/trends/#report/android_vs_ios
5. Статистика по использованию устройств (1) https://gs.statcounter.com/vendor-market-share/mobile/worldwide
6. Статистика по использованию устройств (2) https://www.appbrain.com/stats/top-android-phones-tablets-by-country?country=gl
7. Рекомендации по формированию парка девайсов для тестирования мобильных приложений https://www.browserstack.com/test-on-the-right-mobile-devices
8. Покрытие девайсов https://vladislaveremeev.gitbook.io/qa_bible/mobilnoe-testirovanie/pokrytie-devaisov
9. Спецификации устройств https://www.devicespecifications.com/ru
10. Поддержка версий ОС https://www.gsmarena.com/

11. Эмуляторы, симуляторы или тестовые фермы. Что выбрать для мобильного тестирования? https://habr.com/ru/companies/sbermarket/articles/690906/
12. Выбор мобильных устройств: пошаговая инструкция для начинающих QA
13. https://habr.com/ru/articles/513018/
14. https://habr.com/ru/articles/516160/

# 60. Типы мобильных приложений

### Нативные приложения

1. Написаны на родном для определенной платформы языке программирования. Android - Java/Kotlin, iOS - objective-C или Swift
2. Приложения хранятся на самом устройстве
3. Не обязателен выход в интернет
4. Могут быть установлены из официального магазина (App Store, Google Play)
5. Могут использовать возможности устройства - камера, уведомления, GPS и т.д.

**Примеры:** Pokemon GO, Spotify

**Google Play Instant (прошлое название Android Instant Apps)** - это функция, которая позволяет вам использовать приложение без необходимости полностью загружать его на свой телефон: просто найдите его в Play Store и нажмите «Открыть приложение».

### Веб-приложения

1. Веб-приложения, адаптированные и оптимизированные под мобильное устройство
2. Часто похоже на мобильное приложение
3. Есть возможность добавить закладку на экран “Домой”
4. Кроссплатформенные
5. Могут использовать отдельные аппаратные возможности устройства (камера, микрофон, GPS и др.) через API браузера, но не имеют полного прямого доступа, как нативные приложения.

Специальные **мобильные версии сайтов и приложений** (также известные как m(dot)-сайты)

**Responsive web apps** - адаптируют дизайн к форме и размеру экрана, т. е. к области просмотра

**Adaptive web apps** - выбирают дизайн в соответствии с некоторыми предопределенными размерами. Для этих размеров существуют различные

варианты дизайна, и согласно им регулируются функции, доступные пользователю.

**Progressive Web Apps (PWA)** - позволяют создавать ярлыки определенных веб-страниц на рабочем столе мобильного устройства. Они выглядят как приложения для конкретной платформы и иногда могут работать в офлайн режиме.

**Accelerated Mobile Pages (AMP)** - это технология с открытым исходным кодом, позволяющая создавать веб-страницы, которые быстро загружаются в мобильных браузерах.

## Гибридные и кроссплатформенные приложения

### Гибридные приложения

1. Веб-приложение в обертке нативного приложения, что служит контейнером для отображения веб-приложения через встроенный упрощенный браузер (Chrome webview в Android и WKWebView в iOS)
2. Приложения хранятся на самом устройстве
3. Кроссплатформенность
4. Могут быть установлены из официального магазина (App Store, Google Play)
5. Могут использовать возможности устройства - камера, уведомления, GPS и т.д.

**Примеры:** Uber, Evernote

**Кроссплатформенные приложения** разрабатываются с помощью кроссплатформенных фреймворков: React Native (JavaScript), Flutter (Dart), Ionic (JavaScript), Xamarin (.NET and C#) и т.п. и имеют общий код для iOS и Android.

## Сравнительная таблица

| Характеристика | Native | Web | Hybrid |
|---|---|---|---|
| Установка из магазина | Да | Нет | Да |
| Кроссплатформерность | Нет | Да | И да, и нет |
| Доступ к функциям мобильного устройства | Да | Нет | Да |
| Производительность | Быстрая | Средняя | Средняя |
| Стоимость разработки | Дороже | Дешевле | Средне |
| Выход в интернет | Не всегда обязателен | Обязателен | Обязателен |

## Полезные материалы

1. Презентация
2. Типы мобильных приложений https://vladislaveremeev.gitbook.io/qa_bible/mobilnoe-testirovanie/tipy-mobilnykh-prilozhenii
3. ISTQB. Mobile Application Testing Foundation Level Syllabus https://www.rstqb.org/ru/istqb-downloads.html
4. Старая версия урока https://youtu.be/PIupBjDxeps
5. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1cBg1vbDe-pCce5217XLmw6WHy52I_ECs

# 61. Особенности тестирования мобильных приложений

Тестирование мобильных приложений — это не просто “потыкать кнопки”. В мобильной среде **слишком много переменных**, которые могут повлиять на стабильность, удобство и корректность работы. Ниже я собрал для тебя основные области, которые стоит обязательно учитывать, особенно если ты тестируешь на реальных устройствах.

В этом уроке приведу примеров различных особенностей тестирования мобильных приложений, которые требуют вашего внимания. Помните: что не нужно ТЕСТИРОВАТЬ ВСЕ ИЗ ЭТОГО СПИСКА, каждое приложение уникально и требует свой подход.

## Аппаратное обеспечение и взаимодействие с устройством

Мобильные приложения часто зависят от **железа**. А железо у всех разное:

- **Кнопки** могут быть физическими или виртуальными.
- **Навигация** бывает жестовая, сенсорная, через аппаратные кнопки.
- **Клавиатура** может быть экранной, Bluetooth, или вообще отсутствовать.
- Устройства могут отличаться по наличию: **радио, USB, Bluetooth, камеры, микрофона, наушников, динамиков** — и всё это нужно проверять.

**Что важно:** убедись, что приложение не падает, если устройство *не поддерживает* какую-то функцию, и что поведение адекватное при её наличии.

### Разрешение, плотность пикселей и масштабирование

На Android и iOS разные устройства = разные размеры экранов и ppi (pixels per inch).

- Проверяй, **не перекрываются ли элементы**.
- Масштабируются ли изображения? Не размыты ли?

- Тестируй на **низком и высоком ppi.**

### Поведение при перегреве

Приложение может попасть в неприятности, если устройство нагревается:

- Снижается частота процессора.
- Ограничиваются фоновые процессы.
- Приложение может быть “убито” системой.

**Что важно:** убедись, что данные сохраняются, а состояние не теряется.

### Датчики и сенсоры

Мобильные устройства — это не только экран:

- **Акселерометр** — определяет ускорение, не положение.
- **Гироскоп** — определяет ориентацию устройства.
- **Магнетометр** — определяет стороны света.
- **Датчик освещённости** — регулирует поведение при смене яркости.
- **GPS** — точность и корректность геолокации.

**Что важно:** поведение приложения должно меняться предсказуемо при отключении/включении этих датчиков.

### Ввод с клавиатуры, жесты и камера

- **Клавиатура** должна быть подходящего типа (цифры, текст, email).
- **Жесты:** scroll, swipe, pinch, tap, long tap — все должны работать корректно.
- **Камера:** работает ли при доступе к одной, двум, или ни одной?

**Что важно:** протестируй сценарии, где пользователь взаимодействует с UI через разные типы ввода.

## Жесты

1. **Tap** — касание, нажатие на сенсорный экран. Чтобы открыть любое приложение на смартфоне — мы тапаем на его иконку.

2. **Double Tap** — два коротких касания, двойной тап.
3. **Multi Tap** — три и более тапов подряд по одному элементу.
4. **Long Tap** — нажатие с удержанием на несколько секунд. Позволяет открыть дополнительные опции там, где они есть.
5. **Scroll** — вертикальное пролистывание содержимого скольжением пальца по экрану сверху вниз или снизу вверх.
6. **Swipe** — смахивание вниз, вверх, вправо или влево. Похоже на скролл, только с «легким», коротким касанием.
7. **Pull to refresh (p2r)** — дословный перевод: «потяни для обновления».
8. **Drag&Drop** — изменение положения элементов интерфейса с помощью перетягивания: как говорит нам название — «тащи и бросай»!
9. **Pinch** — жест, используемый для изменения масштаба картинки (увеличения или уменьшения): для уменьшения два пальца касаются экрана и сводятся вместе, для увеличения — разводятся в стороны.

Важно использовать название жестов в тестовой документации, так как они отличаются от типичных действий в веб-приложении. Например, нельзя написать Click вместо Tap.

### Ориентация экрана

- Приложение должно адекватно **переключаться между портретным и альбомным режимом**.
- Данные не должны теряться.
- UI не должен “рассыпаться”.

### Прерывания

Во время использования приложения могут прийти:

- Входящие звонки, SMS
- Уведомления
- **Что важно:** приложение не должно терять сессию, данные, не должно зависать.
- Подключение зарядки
- Смена приложения

- Уход в спящий режим

### Разрешения

- Камера, микрофон, контакты — все требуют разрешений.
- Проверь поведение при **отказе от разрешения**.
- Приложение не должно запрашивать лишнего.
- После отзыва разрешения — тоже не должно крашиться.

### Энергопотребление

Некоторые приложения **жрут батарею как монстр**:

- Проверь, как работает при низком заряде.
- Как ведёт себя в фоновом режиме.
- Как влияет на расход энергии при GPS/сети.

## Тестирование взаимодействия приложения с ПО устройства

### Уведомления и взаимодействие с ОС

- При получении уведомлений приложение должно **реагировать корректно**.
- Уведомление может вести на определённый экран — **deep link** должен работать.
- Ответ на уведомление должен быть **доступен в приложении**.

### Быстрые действия

- Force Touch, 3D Touch, App Shortcuts — проверяй, если такие функции заявлены.
- Приложение не должно ломаться, если устройство не поддерживает эту возможность.

### Настройки ОС

- Язык, тема (светлая/тёмная), энергосбережение, звук.

- Приложение должно **следовать выбранным настройкам** или хотя бы корректно с ними сосуществовать.

### Типы приложений

- **Нативные:** полная интеграция с устройством.
- **Гибридные:** UI через WebView, возможны лаги.
- **Веб:** кроссбраузерность, ограничения в доступе к железу.

**Что важно:** в зависимости от типа приложения, фокус тестирования меняется.

### Кроссплатформенность

- Тестируй на разных версиях iOS и Android.
- Учитывай обратную совместимость.
- Тестируй, как обновления ОС влияют на поведение.

### Взаимодействие с другими приложениями

- Проверь, как приложение работает с календарем, контактами, почтой.
- Бывают **конфликты** — одно приложение отключает GPS, а твоё его ждёт включенным.

### Сеть и типы подключения

- Проверяй на 2G, 3G, 4G, 5G, Wi-Fi, без сети.
- Что произойдёт при потере сети?
- Есть ли сообщения об ошибке, которые помогут пользователю?

### Полезные материалы

1. Презентация
2. Старая версия урока https://youtu.be/dP8IBDRxrsg
3. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1fBIcPZWrmbF0Y9ybC-dR3Kk8gD9R_XD1

4. ISTQB. Mobile Application Testing Foundation Level Syllabus https://www.rstqb.org/ru/istqb-downloads.html
5. Чит-лист для тестирования мобильных приложений (1)
6. Чит-лист для тестирования мобильных приложений (2)
7. UI-элементы и жесты в мобильных приложениях https://habr.com/ru/companies/youla/articles/540768/
8. Гайдлайн по UI для Android https://developer.android.com/guide/topics/ui
9. https://m3.material.io
10. Гайдлайн по UI для Apple https://developer.apple.com/design/human-interface-guidelines/
11. 32 отличия дизайна мобильного приложения под iOS и Android https://habr.com/ru/companies/redmadrobot/articles/491674/
12. Гайдлайны Google Material и Apple Human Interface. Android, iOS и Material You https://youtu.be/28m34MR_g2c
13. Библия QA. Основные проверки при тестировании мобильных приложений https://vladislaveremeev.gitbook.io/qa_bible/mobilnoe-testirovanie/osnovnye-proverki-pri-testirovanii-mobilnogo-prilozheniya

# 62. Типы тестирования мобильных приложений

Тестирование мобильных приложений – это целый набор видов тестирования, заточенных под особенности мобильной среды: установка, сенсоры, нестабильная сеть, малое количество ресурсов. Ниже – ключевые типы тестирования, которые ты должен знать.

### Установка и обновление приложения

Звучит просто — «установить из Google Play или App Store». Но даже здесь могут быть проблемы:

- Установка с **разных источников**: магазин, .apk/.ipa-файл, через iTunes или ADB.
- Установка по **Wi-Fi, мобильной сети, кабелю**.
- Что будет, если **прервать установку** (выключить телефон, пропадёт интернет)?
- При **обновлении** данные должны остаться, при **переустановке без сохранения** — пропасть.
- А если у тебя **внешняя SD-карта**? Приложение может туда не установиться.

Проверяй **все сценарии**, особенно при тестировании новых релизов.

### Стресс и нагрузка

Мобильное приложение — это не сервер, но может легко перегрузить устройство:

- Что будет при **низком заряде**, **малой памяти**, **переполненном хранилище**?
- А при **медленном интернете**, множестве нажатий подряд, высокой активности?
- Приложение не должно тормозить, крашиться или «жрать» всё подряд.

Проверяй **устойчивость к экстремальным условиям**, особенно на слабых устройствах.

### Безопасность

- Шифруются ли **данные при передаче** (например, HTTPS)?
- Хранятся ли **локальные данные** безопасно (без паролей в открытом виде)?
- Очищаются ли **временные данные** при крашах?
- Ввод в полях пароля — **не отображается**, **не сохраняется** в буфере.

Даже если ты не тестируешь банковское приложение — **минимальная безопасность** обязательна.

### Удобство (юзабилити)

- Приложение должно быть **интуитивным**, не требовать инструкций.
- Ошибки пользователя — допустимы, но **должны быть обработаны**.
- Интерфейс — **последовательный**, понятный, видимый на всех экранах.
- Не ломай гайдлайны Android/iOS: **платформенные привычки важны**.

Если приложение неудобное, его удаляют за первые 30 секунд. Буквально.

### Глобализация, локализация, доступность

- **Глобализация (I18N)** — тестируешь под разные страны, даты, форматы.
- **Локализация (L10N)** — конкретная локаль: строки, картинки, бизнес-логика.
- **Доступность** — поддержка VoiceOver, TalkBack, увеличение текста, цвета.

Не забывай тестировать даже там, где кажется «всё и так ясно».

### Базы данных

- Правильно ли **сохраняются и синхронизируются данные**?
- Что происходит при **конфликтах** (несколько устройств)?

- Работает ли **CRUD**: создание, чтение, обновление, удаление?
- **Поиск**, **валидация**, **ограничения** — всё должно быть покрыто.
- Синхронизация с данными ОС (контакты, медиа и др.).

Важно: мобильные приложения часто хранят **часть данных локально** — проверяй.

### Реальные условия эксплуатации

Ты тестируешь в офисе, но пользователь — в метро или на границе Wi-Fi:

- Тестируй с **переключением между сетями**.
- Используй **нестабильные подключения**.
- Имитируй работу **в лифте, на улице, в роуминге**.

**Реальные условия** = реальные баги.

### Публикация и пост-релиз

Перед публикацией:

- Следуй **гайдлайнам App Store и Google Play**.
- Пройдись по **чек-листам**: разрешения, интерфейс, поведение.
- Проверь работу с **новыми и старыми SDK**.

После релиза:

- Установи приложение как обычный пользователь.
- Проверь: **загружается ли**, **работает ли корректно**, **не теряются ли данные**.
- Мониторь **краши и отзывы пользователей**.

### Полезные материалы

1. Презентация
2. Старая версия урока https://youtu.be/dP8IBDRxrsg

3. Схема из старой версии урока https://drive.google.com/drive/u/3/folders/1fBIcPZWrmbF0Y9ybC-dR3Kk8gD9R_XD1
4. ISTQB. Mobile Application Testing Foundation Level Syllabus https://www.rstqb.org/ru/istqb-downloads.html
5. Чит-лист для тестирования мобильных приложений (1) https://bit.ly/2GoTulN
6. Чит-лист для тестирования мобильных приложений (2) https://bit.ly/2Gsjtsy
7. UI-элементы и жесты в мобильных приложениях https://habr.com/ru/companies/youla/articles/540768/
8. Гайдлайн по UI для Android https://developer.android.com/guide/topics/ui
9. https://m3.material.io
10. Гайдлайн по UI для Apple https://developer.apple.com/design/human-interface-guidelines/
11. 32 отличия дизайна мобильного приложения под iOS и Android https://habr.com/ru/companies/redmadrobot/articles/491674/
12. Гайдлайны Google Material и Apple Human Interface. Android, iOS и Material You https://youtu.be/28m34MR_g2c
13. Большой гайд по тестированию Android-приложений https://testengineer.ru/bolshoj-gajd-po-testirovaniyu-android-prilozhenij/
14. Библия QA. Основные проверки при тестировании мобильных приложений https://vladislaveremeev.gitbook.io/qa_bible/mobilnoe-testirovanie/osnovnye-proverki-pri-testirovanii-mobilnogo-prilozheniya

# 63. Android Studio. Функции и логи

**Эмуляторы и симуляторы** - это инструменты, которые используются для тестирования мобильных приложений, позволяя разработчикам и тестировщикам запускать и проверять свои приложения в контролируемой среде без необходимости использования реальных устройств.

**Эмулятор** - это программное обеспечение, которое создает виртуальную машину, имитирующую аппаратное обеспечение и операционную систему реального устройства. Это означает, что он не просто имитирует программное обеспечение, но также обеспечивает эмуляцию аппаратного уровня. Это позволяет тестировщикам в точности воспроизвести поведение приложения, как если бы оно работало на реальном устройстве. *Пример, эмулятор в Android Studio.*

**Симулятор,** в отличие от эмулятора, не пытается эмулировать аппаратное обеспечение устройства. Вместо этого он создает программную среду, которая имитирует внутреннее поведение операционной системы. Симуляторы полезны для тестирования приложений в ситуациях, когда точное воспроизведение аппаратной среды не требуется, например, при разработке пользовательского интерфейса и тестировании функциональности приложения. *Пример, симулятор в xCode.*

***Важно помнить, что ни эмулятор, ни симулятор не могут гарантировать 100% повторение поведения приложения на реальном устройстве и все критические функции приложения нужно тестировать на нем.***

**Полезные материалы**

1. Старая версия урока https://youtu.be/DjsTreMcXcQ
2. Инсталлятор Android Studio + SDK https://developer.android.com/studio
3. Установка SDK Platform Tools https://developer.android.com/tools/releases/platform-tools
4. Статья о включении аппаратной виртуализации (может понадобиться при запуске эмулятора в Android Studio) https://support.bluestacks.com/hc/ru/articles/360043236951#"2"
5. Официальный сайт Genymotion https://www.genymotion.com/
6. Библия QA. Симуляторы и эмуляторы https://vladislaveremeev.gitbook.io/qa_bible/mobilnoe-

testirovanie/simulyatory-i-emulyatory

# 64. ДОП: Xcode. Функции и логи

### Полезные материалы

1. Инсталлятор Xcode (или в App Store) https://developer.apple.com/xcode/resources/
2. Приложение из урока https://github.com/TheDanz/ToDoList
3. Ссылка на iMazing https://imazing.com/
4. Про попарное тестирование с 5 минуты https://www.youtube.com/watch?v=oZOOmYlvfWE

# 65. Charles Proxy. Установка и использование

**Как использовать Charles Proxy в мобильном тестировании**

Charles устанавливается на компьютер тестировщика и подключается к мобильному устройству через Wi-Fi. В настройках прокси телефона указывается IP-адрес компьютера и порт 8888. Оба устройства должны находиться в одной сети. Для работы с HTTPS необходима установка специального сертификата Charles на мобильное устройство — иначе зашифрованный трафик будет недоступен.

После настройки Charles перехватывает и отображает весь сетевой трафик между приложением и сервером. Это позволяет тестировщику видеть «внутреннюю кухню» приложения: какие данные отправляются, что возвращает сервер, как работают заголовки, токены, авторизация и обработка ошибок.

**С помощью Charles можно:**

– Проверить, какие запросы и ответы идут между клиентом и сервером (GET, POST, PUT, DELETE и т.д.)

– Убедиться, что передача данных происходит по HTTPS (а не по HTTP)

– Посмотреть, не передаётся ли пароль или токен в открытом виде

– Проверить статусы ответов сервера (200, 401, 500 и др.)

– Убедиться, что при ошибках сервера приложение показывает корректные сообщения

– Проверить, как работает авторизация и обновление токена (refresh token flow)

– Убедиться, что заголовки запросов и ответов формируются корректно

– Посмотреть содержимое тела запроса и ответа (например, JSON с пользовательскими данными)

– Проверить, что при нестабильном интернете или потере соединения приложение ведёт себя ожидаемо

– Отследить последовательность запросов при запуске приложения, авторизации, выполнении действий

– Проверить, загружаются ли данные из кэша или каждый раз запрашиваются с сервера

**Дополнительные возможности Charles, которые полезны тестировщику:**

– **Throttling** — эмуляция плохой сети: можно замедлить соединение до 2G/3G/EDGE и проверить, будет ли приложение корректно загружаться при слабом интернете. Особенно полезно для проверки прогресс-баров, заглушек, обработки ошибок соединения.

– **Breakpoints** — остановка запроса или ответа на лету. Можно изменить тело запроса, токен или заголовки и отправить их вручную. Это позволяет проверить, как приложение реагирует на неправильные ответы сервера или нестандартные данные.

– **Rewrite** — автоматическая замена частей запросов или ответов по правилам. Используется, например, чтобы подставить другой токен, изменить URL или тело ответа. Это особенно полезно при тестировании A/B-фич или нестабильных окружений.

– **Map Local** — подмена ответа сервера локальным файлом. Позволяет протестировать работу приложения без обращения к реальному серверу. Например, можно отдать кастомный JSON вместо реального API-ответа.

– **Map Remote** — редирект запроса на другой сервер или эндпоинт. Используется при тестировании на разных окружениях (стейдж, дев, прод) без необходимости менять код приложения.

– **SSL Proxying** — расшифровка HTTPS-трафика, если установлен сертификат. Показывает, что реально передаётся в зашифрованных запросах.

– **Save Sessions** — сохранение всех запросов/ответов для последующего анализа. Можно прикладывать к баг-репортам или отправлять разработчику для воспроизведения ошибки.

– **Filter** — фильтрация по домену, методу, статусу, типу контента и другим параметрам. Позволяет сфокусироваться только на нужных запросах и не видеть лишний шум от других приложений.

*Я не буду писать такой же текст для остальных снифферов, так как принцип работы и функции у них аналогичные. Charles Proxy – самый популярный сниффер, так что на проектах вы, скорее всего, будете работать с ним. Также он активно используется и для тестирования веб-приложений, когда возможностей DevTools и Postman недостаточно для специфических случаев.*

**Полезные материалы**

1. Старые версии уроков
2. https://youtu.be/_qKigDAFbP4
3. https://youtu.be/74mvpPOczgI
4. Официальный сайт Charles Proxy https://www.charlesproxy.com/
5. Документация Charles Proxy (информация по установке SSL) https://www.charlesproxy.com/documentation/using-charles/ssl-certificates/
6. Ссылка на приложение из урока https://apps.apple.com/tr/app/plan/id1231862586?l=en
7. Как приручить Charles Proxy https://habr.com/ru/company/youla/blog/527648/
8. Погружение в Charles Proxy https://habr.com/ru/post/663926/

# 66. ДОП: Fiddler Everywhere. Установка и использование

### Полезные материалы

1. Старые версии уроков
2. https://youtu.be/ILPJ653XXrY
3. https://youtu.be/rUXZvVmdb6o
4. Ссылка на скачивание Fiddler Everywhere https://www.telerik.com/download/fiddler-everywhere
5. Ссылка на скачивание Fiddler Classic https://www.telerik.com/download/fiddler
6. Документация Fiddler Everywhere https://docs.telerik.com/fiddler-everywhere/introduction
7. Документация Fiddler Classic https://docs.telerik.com/fiddler/Configure-Fiddler/Tasks/ConfigureFiddler
8. Первые шаги с Fiddler Classic https://habr.com/ru/post/533138/

# 67. ДОП: Proxyman. Установка и использование

### Полезные материалы

1. Ссылка на скачивание Proxyman https://proxyman.io/
2. Документация Proxyman https://docs.proxyman.io/
3. Статья с примерами сниппетов https://docs.proxyman.io/scripting/snippet-code
4. Ссылка на приложение из урока https://apps.apple.com/tr/app/plan/id1231862586?l=en
5. Битва снифферов: Charles vs Proxyman https://habr.com/ru/companies/ozontech/articles/579392/

РАБОТА С GIT И BASH

# 68. Работа с bash

Помимо привычных нам графического интерфейса (GUI) и программного интерфейса (API), существует и третий тип - интерфейс командой строки (CLI).

В данном уроке мы научимся работать с одной из разновидностей командных оболочек для UNIX-подобных систем - bash, на примере базовых команд.

### Для чего тестировщику необходимо это знать?

1. Работа через командную строку позволяет упростить и ускорить взаимодействия пользователя с системой, без использования GUI.
2. У многих серверов, с которыми работают тестировщики, нет графической оболочки и все общение происходит через CLI.
3. В том числе, с помощью команд вы сможете найти необходимые логи и приложить их к отчетам о дефекте.
4. Командную строку можно использовать для создания тестовых данных и базовой автоматизации процессов через скрипты.

### Полезные материалы

1. Старая версия курса
2. На Windows вы можете использовать git bash https://git-scm.com/downloads
3. Или терминал в VSCode https://code.visualstudio.com/
4. Также вы можете установить другие оболочки.
   Например, https://www.cygwin.com/
5. Чит-листы
6. https://github.com/RehanSaeed/Bash-Cheat-Sheet
7. https://www.pcwdld.com/bash-cheat-sheet

### Bash (Bourne Again SHell)

- Bash — это командная оболочка и язык сценариев, который является одним из наиболее широко используемых интерфейсов в Unix и Unix-подобных операционных системах.
- Bash предоставляет интерактивную командную строку и среду для запуска сценариев, которые могут автоматизировать задачи.
- Bash поддерживает шаблоны имен файлов, создание функций, переменные, управление потоками и множество встроенных команд для

управления файлами и процессами.
- Сценарии Bash используются для ряда задач, от простых команд до сложных программ с условными операциями, циклами и вызовами функций.

**Примеры использования Bash:**

- Работа через командную строку позволяет упростить и ускорить взаимодействия пользователя с системой, без использования GUI.
- У многих серверов, с которыми работают тестировщики, нет графической оболочки и все общение происходит через CLI.
- В том числе, с помощью команд вы сможете найти необходимые логи и приложить их к отчетам о дефекте.
- Командную строку можно использовать для создания тестовых данных и базовой автоматизации процессов через скрипты.

**Структура команды в Bash**

Команды Bash обычно состоят из следующих компонентов:

1. **Имя команды**: Определяет, какая программа или встроенная функция Bash должна быть выполнена. Например, ls, mkdir.
2. **Опции/Флаги**: Однобуквенные или многосимвольные опции, обычно предваряемые - (например, -l в ls -l). Опции изменяют поведение команды.
3. **Аргументы**: Данные, над которыми должна выполняться команда. Это могут быть пути к файлам, значения или другие данные, необходимые для выполнения команды. Например, в mkdir newdir, newdir является аргументом.
4. **Перенаправление ввода/вывода**: Используется для изменения стандартных потоков ввода, вывода и ошибок. Например, > используется для перенаправления вывода команды в файл.
5. **Конвейер (Pipe)**: Символ | используется для передачи вывода одной команды в качестве ввода другой команды.

**Команды для работы с файлами и папками**

### 1. cd (Change Directory)

- Команда `cd` используется в командной оболочке Unix и Unix-подобных системах, включая Bash, для изменения текущего рабочего каталога.

**Основное использование:**

- `cd [путь]`: Перемещает в каталог, указанный в `[путь]`.
- Пример: `cd /usr/local/bin` переместит вас в каталог `/usr/local/bin`.

**Вариации команды cd:**

- `cd`: Без аргументов перемещает пользователя в его домашний каталог.
- `cd ~`: `~` является сокращением для домашнего каталога пользователя.
- `cd ..`: Перемещает на один уровень вверх в иерархии каталогов.
- `cd -`: Перемещает в предыдущий каталог, в котором вы были до этого.
- `cd /`: Перемещает в корневой каталог системы.

**2. pwd (Print Working Directory)**

- Команда `pwd` выводит полный путь к текущему рабочему каталогу.
- При вызове `pwd` в командной строке Bash, вы получите полный путь к каталогу, в котором находитесь.
- Эта команда полезна для подтверждения текущего каталога, особенно когда вы перемещаетесь по файловой системе.

**3. ls (List)**

- Команда `ls` используется для просмотра содержимого каталога.

**Вариации команды ls:**

- `ls`: Отображает файлы и каталоги в текущем каталоге.
- `ls -l`: Показывает детальный список файлов и каталогов, включая права доступа, количество ссылок, владельца, группу, размер и время последнего изменения.
- `ls -a`: Отображает все файлы, включая скрытые.
- `ls -h`: Показывает размер файлов в удобочитаемом формате (например, КБ, МБ).
- `ls -R`: Рекурсивно отображает содержимое всех подкаталогов.

**4. touch**

- Команда `touch` используется для создания нового пустого файла или обновления временных меток существующего файла.
- Пример: `touch newfile.txt` создаст пустой файл с именем `newfile.txt`.

### 5. mkdir (Make Directory)

- Команда `mkdir` используется для создания новых каталогов.
- Пример: `mkdir newdir` создаст новый каталог с именем `newdir`.

### 6. rm (Remove)

- Команда `rm` используется для удаления файлов и каталогов.
- Пример: `rm file.txt` удалит файл `file.txt`.
- `rm -r`: Удаляет каталоги и их содержимое рекурсивно.
- `rm -f`: Принудительное удаление, игнорирует несуществующие файлы и аргументы.

### 7. rmdir (Remove Directory)

- Команда `rmdir` используется для удаления пустых каталогов.
- Пример: `rmdir emptydir` удалит каталог `emptydir`, если он пуст.

## Команды для работы с файлами, папками и редакторами

### 1. mv (Move)

- Команда `mv` используется для перемещения или переименования файлов и каталогов.

**Основное использование:**

- `mv [источник] [назначение]`: Перемещает файл или каталог из `[источник]` в `[назначение]`.
- Пример: `mv oldname.txt newname.txt` переименует файл.
- `mv file.txt /path/to/directory/`: Перемещает `file.txt` в указанный каталог.

### 2. cp (Copy)

- Команда `cp` используется для копирования файлов и каталогов.

**Основное использование:**

- `cp [источник] [назначение]`: Копирует файлы или каталоги из `[источник]` в `[назначение]`.
- Пример: `cp file.txt newfile.txt` создает копию файла.
- `cp -r /path/to/source/directory/ /path/to/destination/directory/`: Рекурсивно копирует содержимое каталога.

**Отличия между mv и cp:**

- `mv` перемещает или переименовывает файлы/каталоги, тогда как `cp` создает копию файла или каталога.
- `mv` не сохраняет исходный файл/каталог после перемещения, в то время как `cp` сохраняет и исходный, и копируемый файлы/каталоги.

### 3. cat (Concatenate)

- Команда `cat` используется для вывода содержимого файла, объединения файлов и создания новых файлов.
- Пример: `cat file.txt` выведет содержимое `file.txt` на экран.

### 4. less

- `less` – это программа для просмотра текстовых файлов, позволяющая прокручивать содержимое в обе стороны и искать текст.
- Пример: `less file.txt` откроет `file.txt` для просмотра.

### 5. nano

- `nano` – это простой в использовании текстовый редактор для Unix и Unix-подобных систем.
- Пример: `nano file.txt` откроет `file.txt` для редактирования в `nano`.

### 6. vi/vim

- `vi` – это мощный текстовый редактор с режимами ввода, командным режимом и другими функциями.
- `vim` (Vi IMproved) – это расширенная версия `vi`, содержащая дополнительные функции.
- Пример: `vi file.txt` откроет `file.txt` в `vi` для редактирования.

## Команды для поиска

### 1. grep (Global Regular Expression Print)

- Команда `grep` используется для поиска и вывода строк в файлах, которые соответствуют заданному шаблону (обычно регулярному выражению).

**Основное использование grep:**

- `grep [шаблон] [файл]`: Ищет шаблон в файле и выводит соответствующие строки.
- Пример: `grep "text" file.txt` найдет и выведет все строки, содержащие "text" в файле `file.txt`.

**Вариации команды grep:**

- `grep -i`: Игнорирует регистр символов при поиске.
- `grep -v`: Выводит строки, которые не соответствуют шаблону.
- `grep -r`: Рекурсивный поиск по всем файлам в указанной директории.
- `grep -n`: Выводит номера строк с соответствующими результатами.
- `grep -c`: Подсчитывает количество строк, соответствующих шаблону.
- `grep -E`: Использует расширенные регулярные выражения.
- `grep "text*" file.txt`: Найдет строки, содержащие "text", за которым следует любое количество символов, в файле `file.txt`.
- `grep "^text*" file.txt`: Найдет строки, начинающиеся с "text", в файле `file.txt`.

### 2. find

- Команда `find` используется для поиска файлов и каталогов в файловой системе, которые соответствуют заданным критериям.

**Основное использование find:**

- `find [путь] [опции] [действие]`: Ищет файлы и каталоги, начиная с указанного пути, соответствующие заданным критериям.
- Пример: `find /home -name "file.txt"` найдет все файлы с именем `file.txt` в каталоге `/home`.

**Вариации команды find:**

- `find -name`: Ищет файлы по имени.
- `find -type`: Ищет файлы определенного типа (например, `d` для директорий, `f` для обычных файлов).

- `find -size`: Ищет файлы определенного размера.
- `find -perm`: Ищет файлы с определенными правами доступа.
- `find -mtime`: Ищет файлы, измененные определенное количество дней назад.
- `find -exec`: Выполняет команду для каждого найденного файла.
- `find /path -name "*part*"`: Найдет все файлы, в имени которых содержится "part", в каталоге `/path`.
- `find /path -name "start*"`: Найдет все файлы, имя которых начинается с "start", в каталоге `/path`.
- `find /path -name "*end"`: Найдет все файлы, имя которых заканчивается на "end", в каталоге `/path`.

**Пример с find и exec:**

- `find /path -type f -exec grep "text" {} \;`: Найдет все файлы в `/path` и выполнит команду `grep "text"` для каждого из них.

**Команды с выводом и процессами**

**1. echo**

- Команда `echo` используется для вывода текста или переменных в стандартный вывод (обычно на экран).
- Пример: `echo "Hello World"` напечатает "Hello World".

**2. cat (Concatenate)**

- Команда `cat` используется для чтения, объединения и вывода содержимого файлов.
- Примеры:
  - `cat file.txt` выведет содержимое файла `file.txt`.
  - `cat file1.txt file2.txt > combined.txt` объединит содержимое `file1.txt` и `file2.txt` и сохранит в `combined.txt`.

**3. Вывод в файл**

- В Unix-подобных системах можно перенаправить вывод команды в файл.
- `>` используется для создания файла или перезаписи его, если он существует.
- `>>` используется для добавления вывода в конец файла.

- Пример: `echo "Some text" > file.txt` создаст `file.txt` с текстом "Some text".

### 4. Команды на ввод и вывод

- `|` (pipe): Используется для передачи вывода одной команды в качестве входных данных для другой команды.
- Пример: `cat file.txt | grep "text"` передаст содержимое `file.txt` в команду `grep`.

### 5. Команды с процессами

- **ps (Process Status)**
    - Команда `ps` отображает информацию об активных процессах.
    - Примеры:
        - `ps`: Показывает процессы, запущенные в текущем терминале.
        - `ps aux`: Показывает все запущенные процессы с дополнительной информацией.
- **kill**
    - Команда `kill` используется для отправки сигналов процессам, обычно для их завершения.
    - Пример: `kill [PID]` отправит сигнал SIGTERM процессу с идентификатором `[PID]`.
    - `kill -9 [PID]` принудительно завершит процесс, если обычный `kill` не срабатывает.

## Команды для работы с сетью

### 1. ping

- Команда `ping` используется для проверки доступности узла в сети .

**Основное использование ping:**

- `ping [адрес]`: Отправляет ICMP ECHO_REQUEST пакеты на указанный адрес для проверки связи.
- Пример: `ping google.com` проверит связь с серверами Google.

**Вариации команды ping:**

- `ping -c [количество] [адрес]`: Отправляет определенное количество пакетов и затем останавливается.
- Пример: `ping -c 4 google.com` отправит 4 запроса на сервер Google.
- `ping -i [интервал] [адрес]`: Задает интервал между отправкой пакетов в секундах.
- `ping -t [TTL] [адрес]`: Устанавливает значение Time-To-Live для пакетов.

**2. curl**

- Команда `curl` используется для передачи данных с или на сервер, используя один из протоколов (HTTP, HTTPS, FTP и др.).

**Основное использование curl:**

- `curl [URL]`: Получает содержимое по указанному URL.
- Пример: `curl https://www.example.com` выводит HTML содержимое страницы.

**curl для GET запросов:**

- GET запросы используются для запроса данных с сервера.
- Пример: `curl https://api.example.com/data` выполнит GET запрос к API и выведет полученные данные.

**curl для POST запросов:**

- POST запросы используются для отправки данных на сервер.
- Пример: `curl -X POST -d 'param1=value1&param2=value2' https://api.example.com/submit` отправит POST запрос с данными 'param1=value1&param2=value2' на указанный URL.

**Дополнительные опции curl:**

- `-X [METHOD]`: Указывает метод запроса (например, GET, POST).
- `-d [данные]`: Отправляет данные в теле запроса (используется для POST запросов).
- `-H "Header: Value"`: Добавляет заголовки к запросу.
- `--data-urlencode`: Отправляет данные, автоматически кодируя их для URL.

РАБОТА С GIT И BASH

# 69. Работа с git

**Git и Github используются для хранения кода проекта, работы с ним и отслеживания изменений.**

Это так называемая система контроля версий, которая позволяет в любой момент времени добавить новый код, увидеть изменения или откатиться на старые версии кода, если возникают проблемы.

Не всегда мануальный тестировщик имеет доступ к коду и гитхабу, однако если он есть, то было бы здорово разбираться в основных сущностях и базовых командах, которые используют в гите. На них и будет сделан основной упор в этом модуле.

Хочу обозначить ряд задач, с которыми вы можете столкнуться в реальных боевых задачах, связанных с этим инструментом:

1. Анализ кода на предмет изменений, который можно использовать для создания более точных тест-кейсов или формирования регрессионных наборов
2. Подготовка тестовых окружений для тестирования в рамках процесса CI/CD, где git и подобные системы играют одну из ключевых ролей
3. Понимание общего процесса разработки, а именно на каких ветках работают разработчики, какие из них участвуют в релизе и в процессе тестирования
4. На ряде проектов git и github используется для хранения внутренней документации, поэтому базовые команды также могут вам помочь в быстрой навигации по ней
5. И многое другое

### Что такое git?

**Система контроля версий** позволяет сохранять историю изменений в файлах и возвращаться к предыдущим состояниям. Это необходимо для восстановления предыдущих версий, понимания, кто и когда внес определенные изменения, и управления параллельными изменениями от разных разработчиков.

**Git** — это распределенная система контроля версий, которая позволяет разработчикам отслеживать изменения в файлах и координировать работу над общими проектами. Git популярен в разработке программного обеспечения из-за его мощных возможностей ветвления и слияния, а также из-за его эффективности и надежности.

**Распределенность** означает, что каждый разработчик работает с локальной копией репозитория, которая включает всю историю проекта. Это позволяет работать с кодом и вносить изменения без постоянного доступа к сети и обеспечивает высокий уровень отказоустойчивости.

### Репозиторий и коммит

**Репозиторий** — это хранилище данных, содержащее все файлы проекта и их историю изменений. Репозитории могут существовать локально на компьютере разработчика или удаленно, например, на таких сервисах, как GitHub, GitLab или Bitbucket.

**Коммит** — это запись в истории изменений репозитория, который содержит снимок всех файлов проекта на момент коммита. Каждый коммит имеет уникальный идентификатор (хэш), автора, дату и сообщение, описывающее суть изменений.

### Процесс работы в git

Об этом мы будем говорить дальше более подробно с примерами отправки всех запросов в git bash.

1. **Клонирование репозитория**: Создается локальная копия удаленного репозитория, включая всю историю и ветки проекта. Это делается с помощью команды `git clone`.

2. **Создание новой ветки**: Перед внесением изменений рекомендуется создать новую ветку командой `git branch <branch_name>`. Это позволяет разработчикам работать изолированно от главной линии разработки (обычно ветка `master` или `main`).

3. **Переключение между ветками**: Для начала работы в новой ветке используется команда `git checkout <branch_name>`. Это изменяет рабочую копию файлов на те, что соответствуют состоянию выбранной ветки.

4. **Добавление изменений**: После внесения изменений в файлы они добавляются в индекс с помощью команды `git add`. Это подготавливает измененные файлы к коммиту.

5. **Совершение коммита**: Подготовленные изменения фиксируются в истории репозитория с помощью команды `git commit`. Каждый коммит требует сообщения, описывающего сделанные изменения.

6. **Отправка изменений**: Чтобы поделиться изменениями с другими, коммиты из локальной ветки отправляются в удаленный репозиторий командой `git push`.

7. **Слияние веток**: После завершения работы в ветке её изменения можно объединить с главной веткой (например, `main`). Это обычно делается через запрос на слияние (pull request) в веб-интерфейсе сервиса (например, GitHub) или командой `git merge` в командной строке.

8. **Решение конфликтов**: Если одни и те же части кода были изменены в разных ветках, Git может не суметь автоматически слить их. В этом случае разработчику потребуется вручную разрешить конфликты и выполнить коммит этих изменений.

9. **Обновление локального репозитория**: Регулярно рекомендуется обновлять локальный репозиторий, чтобы он соответствовал удаленному. Для этого используется команда `git pull`.

**Базовая настройка и команды, нужные для старта работы в git**

## 1. git config

- Команда `git config` используется для настройки параметров Git на уровне конкретного репозитория, пользователя или всей системы. Эти настройки контролируют поведение Git, включая такие вещи, как имя пользователя, адрес электронной почты, форматирование вывода, цвет командной строки и многое другое.

**Настройка имени пользователя и электронной почты:**

- Одна из первых задач, которую обычно выполняют с помощью `git config`, — это настройка имени пользователя и адреса электронной почты. Это важно, поскольку каждый Git коммит использует эту информацию, и она включена в коммиты, которые вы отправляете.

**Команды:**

- **Установка имени пользователя:**
  - `git config --global user.name "Your Name"`
  - Пример: `git config --global user.name "John Doe"`
  - Эта команда устанавливает имя, которое будет прикреплено к вашим коммитам и тегам.
- **Установка адреса электронной почты:**
  - `git config --global user.email "your_email@example.com"`
  - Пример: `git config --global user.email "johndoe@example.com"`
  - Этот адрес электронной почты будет использоваться в информации о коммите.

### 2. git init

- Используется для инициализации нового Git репозитория в существующем каталоге. Это первый шаг в создании нового проекта под управлением Git.
- Команда: `git init`
- После выполнения, в текущем каталоге создается подкаталог `.git`, содержащий все необходимые файлы репозитория — скелет нового Git репозитория.

### 3. git clone

- Служит для клонирования репозитория с удаленного репозитория. Эта команда создает полную копию всех данных репозитория, включая всю историю и ветки.
- Команда: `git clone [url]`
- Пример: `git clone https://github.com/example/repo.git`
- Выполнение этой команды создаст каталог `repo` с подкаталогом `.git`, скопирует все данные репозитория и установит текущую рабочую копию в последнюю версию.

### 4. git status

- Показывает состояние изменений как индексированных, так и неиндексированных. Это помогает понять, какие изменения были сделаны и какие из них будут включены в следующий коммит.
- Команда: `git status`

- Вызов этой команды отобразит список измененных файлов, а также файлов, которые готовы к коммиту (после использования `git add`).

### 5. git add

- Добавляет изменения в файлах в индекс (подготовка к коммиту). Можно добавлять отдельные файлы или все изменения сразу.
- Команда: `git add [file]` или `git add .` для добавления всех изменений.
- Пример: `git add index.html` добавит изменения только для файла `index.html`.

### 6. git commit

- Коммитит подготовленные изменения и сохраняет их в истории репозитория. Каждый коммит требует сообщения, которое описывает совершенные изменения.
- Команда: `git commit -m "commit message"`
- Пример: `git commit -m "Add feature X"`
- Это создает новый снимок текущего состояния проекта, который можно потом восстановить или на который можно ссылаться.

### 7. git push

- Отправляет локальные коммиты на удаленный репозиторий. Это позволяет поделиться своей работой с другими.
- Команда: `git push [alias] [branch]`
- Пример: `git push origin main`
- Эта команда отправит изменения из локальной ветки `main` в удаленный репозиторий, связанный с алиасом `origin`.

## Базовые команды

### 1. git fetch

- Команда `git fetch` скачивает коммиты, файлы и ссылки из удаленного репозитория в ваш локальный репозиторий.
- Команда: `git fetch [remote-name]`
- Пример: `git fetch origin`
- `git fetch` обновляет локальную информацию о состоянии удаленного репозитория, но не влияет на вашу рабочую директорию.

### 2. git pull

- Команда `git pull` используется для получения и слияния любых изменений с удаленного репозитория в текущую ветку в локальной рабочей директории.
- Команда: `git pull [remote-name] [branch-name]`
- Пример: `git pull origin master`
- `git pull` является комбинацией команд `git fetch` и `git merge`, обновляя текущую локальную ветку до последней версии на удаленном сервере.

### 3. git show

- Команда `git show` отображает информацию о любом git объекте, например коммите или теге, вместе с соответствующими изменениями.
- Команда: `git show [commit-hash]`
- Пример: `git show 1a410efbd13591db07496601ebc7a059dd55cfe9`
- По умолчанию `git show` отображает информацию о последнем коммите, если не указан конкретный объект.

### 4. git log

- Команда `git log` используется для просмотра истории коммитов в текущей ветке.
- Команда: `git log`
- Дополнительные опции могут быть использованы для настройки вывода, например, `--oneline` для сокращенного представления или `--graph` для визуального представления ветвления.

### 5. git blame

- Команда `git blame` показывает, кто и когда внес изменения в каждую строку файла.
- Команда: `git blame [file-name]`
- Пример: `git blame README.md`
- Это полезно для отслеживания истории изменений и выяснения авторства кода.

### 6. git diff

- Команда `git diff` используется для сравнения изменений между коммитами, ветками, файлами и т.д.
- Команда: `git diff [branch1]..[branch2]`
- Пример: `git diff master..feature`

- Это позволяет увидеть различия в коде между разными состояниями репозитория.

## Базовые команды

### git commit

- Команда `git commit` фиксирует изменения, добавленные в индекс (staging area), в истории репозитория.
- Статусы файлов в Git:
  - **Untracked**: Файлы новые, не отслеживаемые Git'ом. Появляются в репозитории, но не добавлены в индекс с помощью `git add`.
  - **Modified**: Файлы изменены в рабочем каталоге, но еще не добавлены в индекс.
  - **Staged**: Измененные файлы добавлены в индекс и готовы к коммиту.
  - **Committed**: Изменения в файлах зафиксированы в истории репозитория.

### git reset

- Команда `git reset` используется для отмены изменений. Она может изменять индекс (staging area), рабочий каталог и историю коммитов в зависимости от переданных параметров.
- Примеры:
  - `git reset` без параметров сбросит индекс, но оставит рабочий каталог нетронутым.
  - `git reset --soft [commit]` отменит все коммиты после указанного, но оставит изменения в индексе.
  - `git reset --hard [commit]` полностью уберет все изменения после указанного коммита, включая индекс и рабочий каталог.

### git checkout

- Команда `git checkout` используется для переключения между ветками или восстановления рабочих файлов.
- Примеры:
  - `git checkout [branch-name]` переключит HEAD на указанную ветку.
  - `git checkout [commit-hash]` переключит HEAD на указанный коммит.
  - `git checkout -- [file-name]` отменит все изменения в указанном файле, возвращая его к последнему коммиту.

### git stash

- Команда `git stash` используется для временного сохранения изменений, которые вы не готовы коммитить. Это позволяет очистить рабочий каталог, не теряя изменений.
- Примеры:
  - `git stash` сохранит ваши изменения и очистит рабочий каталог.
  - `git stash list` покажет список всех сохраненных изменений.
  - `git stash apply` восстановит последние сохраненные изменения обратно в рабочий каталог.
  - `git stash drop` удалит последние сохраненные изменения из списка stash.

## Работа с ветками

### 1. git branch

- Команда `git branch` используется для управления ветками в Git.

**Создание ветки:**

- Чтобы создать новую ветку:
  - Команда: `git branch [branch-name]`
  - Пример: `git branch feature_x`

**Переключение веток:**

- Чтобы переключиться на другую ветку:
  - Команда: `git checkout [branch-name]`
  - Пример: `git checkout feature_x`

**Удаление ветки:**

- Чтобы удалить ветку:
  - Команда: `git branch -d [branch-name]` (безопасное удаление, удаляет только смерженные ветки)
  - Команда: `git branch -D [branch-name]` (принудительное удаление, используется для удаления несмерженных веток)
  - Пример: `git branch -d feature_x`

**Переименование ветки:**

- Чтобы переименовать ветку:
  - Команда: `git branch -m [old-branch-name] [new-branch-name]`
  - Пример: `git branch -m feature_x feature_y`

**2. git merge**

- Команда `git merge` используется для слияния изменений из одной ветки в другую.
- Перед слиянием необходимо переключиться на ветку, в которую вы хотите слить изменения.
- Команда: `git merge [source-branch]`
- Пример: переключиться на ветку `master` командой `git checkout master`, затем слить в нее ветку `feature_x` командой `git merge feature_x`.
- При слиянии могут возникнуть конфликты, которые нужно разрешать вручную.

**Последовательность действий для создания удаленной ветки:**

1. **Создание новой локальной ветки (если это необходимо):**
   - Команда: `git branch [branch-name]`
   - Пример: `git branch new-feature`
2. **Переключение на новую ветку:**
   - Команда: `git checkout new-feature`
3. **Добавление и коммит изменений (если это необходимо):**
   - Команда: `git add .`
   - Команда: `git commit -m "Initial commit on new-feature branch"`
4. **Первый push и установка upstream (связанной) ветки:**
   - Команда: `git push -u origin new-feature`
   - Эта команда отправит изменения в ветке `new-feature` в удаленный репозиторий (предполагая, что удаленный репозиторий называется `origin`).
   - Опция `-u` устанавливает `new-feature` как upstream ветку для `origin/new-feature`, что позволяет в дальнейшем использовать `git`

`push` и `git pull` без указания имени ветки.

**Что такое Pull Request?**

- **Pull Request** (PR) — это функциональность платформ для совместной разработки, таких как GitHub, GitLab, Bitbucket, которая позволяет разработчикам сообщать о готовности внесенных изменений и запросить их слияние в основную ветку проекта.
- PR используется для обсуждения изменений с другими участниками проекта, проведения код-ревью и запуска автоматизированных тестов перед слиянием изменений.
- Процесс создания PR обычно включает следующие шаги:
    1. Разработчик делает форк (копию) репозитория, вносит изменения и публикует их в своем форке.
    2. Разработчик создает PR на оригинальном репозитории, указывая ветку с изменениями и целевую ветку для слияния.
    3. Другие разработчики проводят ревью изменений, оставляют комментарии и предложения.
    4. После одобрения, изменения могут быть слиты в основную ветку.

**Git Flow** помогает большим командам работать над сложными проектами с множеством параллельных разработок и быстрым циклом выпуска обновлений. В ее основе лежат следующие ветки:

- Ветка `master или main`: Основная ветка, где код в состоянии, готовом к релизу, всегда отражает состояние, развернутое в продакшен.
- Ветка `develop`: Ветка разработки, которая содержит предстоящие изменения для следующего релиза.
- Ветки feature: Создаются от `develop` для новых функций/исправлений багов и мерджатся обратно в `develop`.
- Ветка release: Создаются от `develop` для подготовки релиза; позволяют финализировать релиз, исправлять баги и прочее; мерджатся в `master` и обратно в `develop`.
- Ветки для срочных исправлений hotfix: Создаются от `master` для немедленного решения проблем в продакшене и мерджатся в `master` и `develop`.

**Рабочий процесс Git Flow:**

- От ветки master создается ветка `develop`.

- Создается новая `feature` ветка к `develop` для каждой новой функциональности.
- После завершения работы `feature`а ветка сливается обратно в `develop`.
- Когда функциональность готова к релизу, создается `release` ветка.
- После завершения релиза `release` ветка сливается в `master` и в `develop`.
- Если обнаруживается срочная проблема, создается `hotfix` ветка, которая после исправления сливается обратно в `master` и `develop`.

Стоит отметить, что на разных проектах git flow процесс может отличаться и адаптироваться под их требования.

**Полезные материалы**

1. Старая версия курса
2. Презентация
3. Что такое VCS (система контроля версий) https://habr.com/ru/post/552872/
4. О системе контроля версий Git (на русском языке) https://git-scm.com/book/ru/v2/Введение-О-системе-контроля-версий
5. Инсталлятор GIT https://git-scm.com/downloads
6. Регистрация в GitHub https://github.com/
7. Авторизация в GitHub https://docs.github.com/en/authentication/connecting-to-github-with-ssh
8. Чит-листы с командами
9. https://education.github.com/git-cheat-sheet-education.pdf
10. https://www.atlassian.com/git/tutorials/atlassian-git-cheatsheet
11. https://gist.github.com/avshatalov48/a9c6d5e3b6492a27362dfffe01b07a10
12. Git Kraken клиент https://www.gitkraken.com/

# 70. Тестовые окружения

**Тестовое окружение (Testing Environment)** - это комплекс аппаратного и программного обеспечения, который используется для проведения тестов.

**Deployment (Развертывание)** - это процесс доставки и установки готового программного обеспечения или его обновлений на сервера или рабочие станции пользователей.

**Назначение тестовых окружений**

### Local

- Возможность самостоятельного развертывания приложения
- Разработчики: unit-тестирование отдельных методов и функций.
- Тестировщики: проверка отдельных функций на локальном сервере (своем компьютере).

### Dev

- Песочница. В первую очередь используется разработчиками для тестирования функциональностей приложения до передачи тестировщикам.
- Если на проекте нет выделенного окружения для тестировщиков, то может использоваться для их тестов (плохая практика).

### QA, Test

- Используется только тестировщиками.
- Основные виды тестирования: smoke, critical path, new feature testing, может проводиться регрессионное тестирование.
- Может быть несколько, в зависимости от размера команды.

### Stage, Pre-release (может быть отдельно)

- Стабильная версия приложения
- Регрессионное тестирование (может проводиться на QA)
- Финальное тестирование в среде, максимально приближенной к Prod.
- Проверка готовности приложения к выпуску в продакшн.
- На данном этапе нельзя вносить изменения в код (code freeze)

**UAT**

- Приемочное тестирование группой конечных пользователей

**Prod**

- Мониторинг работы приложения в реальных условиях (smoke, минимальное тестирование новых функциональностей).
- Выявление и анализ проблем, возникающих после релиза.

В зависимости от проекта количество тестовых сред может меняться, а также корректироваться их назначение.

# 71. Что такое CI/CD?

**CI/CD (Continuous Integration – непрерывная интеграция, Continuous Delivery/Deployment – непрерывная доставка/развертывание) —** это технология автоматизации тестирования и доставки новых модулей разрабатываемого проекта заинтересованным сторонам (разработчикам, аналитикам, инженерам качества, конечным пользователям и др.).

**DevOps** — это методология взаимодействия разработчиков, тестировщиков и других ИТ-специалистов в команде. Такая система нужна, чтобы команда работала более эффективно и слаженно, своевременно исправляла ошибки и грамотно взаимодействовала друг с другом.

## Этапы CI/CD процесса

### Continuous Integration (CI) — Непрерывная интеграция

1. **Commit-изменение в коде (GitHub)**:
    - Разработчики коммитят изменения в репозиторий на GitHub.
    - Это инициирует процесс CI, автоматически запуская следующие шаги.
2. **Запуск билда**:
    - Сборка приложения. Код компилируется и собирается в исполняемый файл или пакет.
3. **Уведомление о готовности сборки**:
    - CI-сервер уведомляет команду о завершении процесса сборки.
4. **Запуск юнит и интеграционных тестов**:
    - Автоматическое тестирование для проверки функциональности и интеграции модулей.
5. **Уведомление о результатах тестов**:
    - Команда получает уведомления о результатах тестов.

### Continuous Delivery (CD) — Непрерывная доставка

1. **Приемочные тесты**:
    - Проверка соответствия требованиям и готовности к развертыванию.
2. **Доставка билда на тестовое окружение**:
    - Автоматическое развертывание сборки на тестовом окружении для дополнительного тестирования.
3. **Доставка билда на Prod (в ручном режиме)**:
    - Развертывание в производственной среде после успешного тестирования.
    - Включает ручную проверку и контроль.

**Continuous Deployment (CD) — Непрерывное развертывание**

1. **Приемочные тесты**:
    - Аналогично этапу приемочных тестов в Continuous Delivery.
2. **Доставка билда на тестовое окружение**:
    - Автоматическое развертывание на тестовом окружении.
3. **Доставка билда на Prod (в автоматическом режиме)**:
    - Автоматическое развертывание в производственной среде без ручного вмешательства.

## Инструменты CI/CD

- **Планирование**: Confluence, JIRA.
- **Кодирование**: Git.
- **Сборка**: sbt, Maven.
- **Тестирование**: Selenium, JUnit.
- **Релиз**: Jenkins, Codeship.
- **Развертывание**: Docker, AWS, DC/OS.
- **Эксплуатация**: Chef, Ansible, Kubernetes.
- **Мониторинг**: Nagios, Splunk, Datadog.

# 72. ChatGPT в тестировании

### Полезные материалы

1. Схема из урока

# 73. Основы работы с Jenkins

### Полезные материалы

1. Установщик Jenkins https://www.jenkins.io/download/
2. Установщик Java https://www.oracle.com/java/technologies/downloads/#java21

### Полезные материалы

1. Ссылка на репозиторий https://github.com/artichokeee/jenkins-site

```
Скрипт для MacOS/Ubuntu

# Путь к вашей установке Node.js и npm
NODE_PATH=/Users/rusau/.nvm/versions/node/v16.14.2/bin

# Добавьте путь к npm и node в PATH
export PATH=$NODE_PATH:$PATH

# Проверьте доступность node и npm
echo "Node version:"
node -v
echo "NPM version:"
npm -v

# Перейдите в рабочую директорию Jenkins
cd /Users/rusau/.jenkins/workspace/landing-page

# Установите зависимости
npm install

# Соберите проект
npm run build

# Запустите новый сервер в фоновом режиме на порту 8082
npm start &

# Сохраните PID нового сервера, чтобы можно было остановить его после тестов
```

```
NEW_SERVER_PID=$!

# Дайте серверу время на запуск
sleep 5

# Запустите тесты
npm test

# Остановите новый сервер
kill $NEW_SERVER_PID # Путь к вашей установке Node.js и npm
```

```
Скрипт для Windows (до запуска сервера)

REM Путь к вашей установке Node.js и npm
SET NODE_PATH=C:\Program Files\nodejs
ECHO %NODE_PATH%

REM Добавьте путь к npm и node в PATH
SET PATH=%NODE_PATH%;%PATH%
ECHO %PATH%

REM Установите зависимости
npm install
ECHO "npm install завершен"
```

# 74. Основы работы с GitHub Actions

### Создание личного токена доступа (Personal Access Token)

1. Перейдите в настройки своего аккаунта GitHub: Settings > Developer settings > Personal access tokens > Tokens (classic).
2. Создайте новый токен с правами на запись в репозитории.
3. Скопируйте созданный токен.
4. Перейдите в настройки секретов репозитория: Settings > Secrets and variables > Actions > New repository secret.
5. Создайте новый секрет с именем PERSONAL_ACCESS_TOKEN и вставьте туда токен.

### Полезные материалы

1. Репозиторий из урока https://github.com/artichokeee/jenkins-site/tree/github-actions
2. Документация https://docs.github.com/ru/actions

ДОП: СОЗДАНИЕ РЕЗЮМЕ

# 75. Создание резюме

**Полезные материалы**

1. Презентация
2. Папка с образцами резюме и другими материалами из презентации
3. Гайд по созданию резюме
4. Краш-тесты резюме
5. Мои эфиры с разборами резюме
6. Группы для разбора резюме сообществом
7. https://telegram.me/qaReview - тестировщики
8. https://telegram.me/junohub/13399 - все направления

ДОП: СОЗДАНИЕ РЕЗЮМЕ

# 76. Создание сопроводительного письма

**Полезные материалы**

1. Презентация
2. Большое количество ссылок на полезные материалы, в том числе пример Cover Letter для тестировщика

ДОП: СОЗДАНИЕ РЕЗЮМЕ

# 77. Создание профиля в LinkedIn

### Полезные материалы

1. Страница регистрации в LinkedIn https://www.linkedin.com/
2. Группа, где можно найти людей, которые без проблем добавят вас в сеть
3. Особенности ведения в LinkedIn в разных странах

# Автор курса

Артём Русов — тестировщик, блогер, преподаватель

Сайт

Авторский канал

Бесплатные ресурсы

Спасибо, что учитесь. Больше материалов — по ссылкам выше.